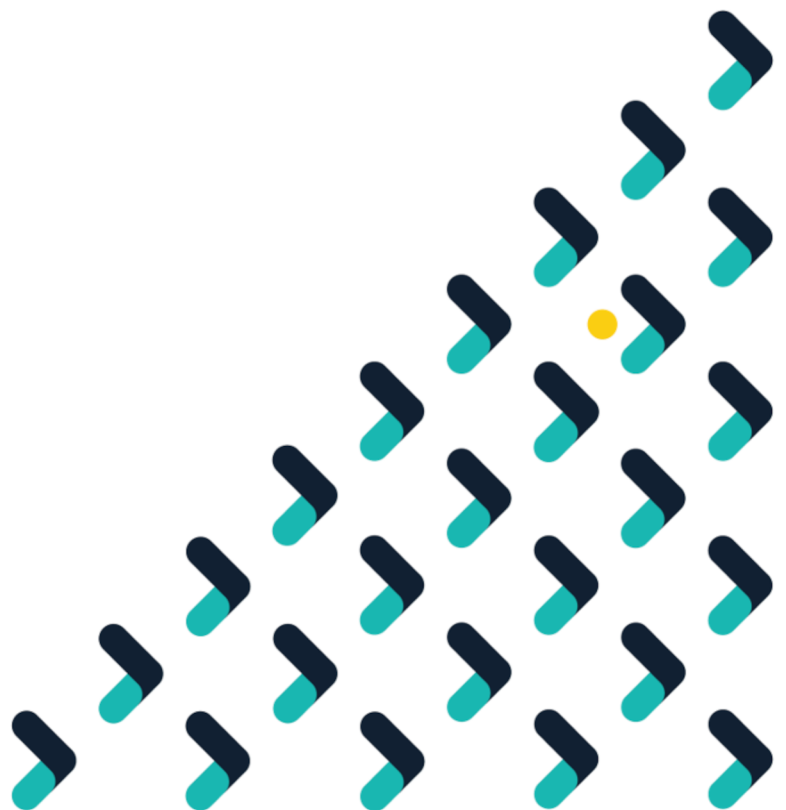




FO-Programmierschnittstelle

Forterro Deutschland Abas GmbH

2026-03-09



Inhaltsverzeichnis

FO-Programmierschnittstelle	1
Erste Schritte mit FO-Programmen	2
Allgemeine Programmierrichtlinien	2
FOP erstellen, ausführen, ändern und löschen	2
Aufruf von FOPs	3
Benennung von FOPs	4
Sprachverzeichnis	4
Direktmodus	5
Ein- und Ausgaben	5
Gliederung der FOPs	5
Individualisierung von FOPs	6
Parameterübergabe an FOPs	7
Array-Parameter	8
Namensparameter	8
Entscheidung zwischen Array-Parametern und Namensparametern	9
Programmbeispiel	10
Der Aufbau der FO-Sprache	12
Interpretation	12
Kommentare	13
Bedingungen	13
Marken und Sprünge	14
Variablen und Typen	15
Unterscheidung von Variablen	15
Verwendung von Variablen	15
Syntax der Variablenverwendung	17
Formatierung von Variableninhalten	19
Verknüpfung von Verweisen	22
Abkürzung von Variablen, Deklarationspflicht	23
Verwendung vordefinierter Variablen	24
Pufferangabe bei Variablen	25
Der A-Puffer	28
Druckeinstellungsvariablen	28
Umgebungsvariablen	28
Punktersetzung und Array-Variablen	29
Vordefinierte Variablen	32
FO-Funktionen	70
Allgemeine Syntax eines Funktionsaufrufs	71
Katalogfunktionen	72
Stringfunktionen	73
Mathematische Funktionen	86
Vergleichsfunktion F equals	92
Kalenderfunktionen	94
Datums- und Zeitfunktionen	128

Erlaubnisfunktionen	129
Dateifunktionen	130
Metafunktionen	133
Finanzbuchhaltungsfunktionen	148
Objektsperrenfunktionen	153
Sonstige Funktionen	155
FO-Kommandos	161
Hinweise zur Notation	161
Interpreter-Einstellungen	162
..!INTERPRETER / ..!INTERPRETER - Arbeitsweise des FO-Interpreters	162
Die Option declaration	163
Die Optionen german und english	164
Die Option englvar	164
Die Option noabbrev	164
Textausgabe und Textformatierung	166
.AUSGABE / .OUTPUT - Ausgabemedium festlegen	166
.SETZE STOP / .SET STOP - Bildschirm seitenweise füllen	170
.PRINTLN / .PRINTLN - Text ausgeben	171
.BLOCKSATZ / .JUSTIFIED - Blocksatz	174
.FLATTERSATZ / .UNJUSTIFIED - Flattersatz	174
.SETZE INLINE / .SET INLINE - Fließtext beibehalten	177
.SETZE LF / .SET LF - Zeilenvorschub setzen	178
.LINKS / .LEFT - Linker Rand	178
.RECHTS / .RIGHT - Rechter Rand	179
.SEITE / .PAGE - Seitenbereich und Seitenumbruch	179
.DRUCKEN / .PRINT - Druckausgabe	180
.ZEIGEN / .VIEW - Fensterbreite festlegen	181
Doppelpunkt bei .drucken und .zeigen	181
.LAENGE / .LENGTH - Seitenlänge	181
.ABSATZ / .PARA - Absatzschutz	182
.OBEN / .UP - Kopfzeilen	183
.UNTEN / .DOWN - Fußzeilen	184
.VTAB / .VTAB - Vertikaltabulatoren	184
.ZEILE / .LINE - Nächste Ausgabezeile	187
.SETZE VERWEIS / .SET REFERENCE - Datenübernahmemodus setzen	187
Verwendung von Variablen bei der Textausgabe	189
HTML- und XML-Ausgabe	189
.FORMAT / .FORMAT - Ausgabeformat umschalten	189
.BROWSER / .BROWSER - URL im Browser anzeigen	193
Schriftattribute setzen	195
.SETZE DOPPELPUNKT / .SET COLON - HTML-Befehle markieren	195
.SETZE VARQUOTE / .SET VARQUOTE - HTML-Code in Variableninhalten	196
.TABELLENSATZ / .TABLERECORD - HTML-Tabellen erzeugen	197
Einschränkungen im HTML-Zeichensatz	200
HTML-Ausgabe und Seitenverteilung	201

HTML-Ausgabe und Seitenwechsel	201
Umleitung der Ausgabe von bestehenden Programmen auf Browser	201
Erfassen von E-Mails	203
Anklickbare Listen	203
Substitutionen - Schützen von Zeichen vor Interpretation	204
Methoden zur Substitution von Sonderzeichen	205
Realisierung der Substitutionsmethoden	205
FO-Schnittstelle für Substitutionen	210
Weitere Verwendungsstellen für die Abas Standardsubstitutionen	210
Substitutionen global einstellen	212
.SETZE SUBSTITUTION / .SET SUBSTITUTION - Substitutionen für Variablen	212
.SETZE FORMATIERT / .SET FORMATTED - Substitutionen für Variablen mit Formatierung	213
.SETZE AUSGABE / .SET OUTPUT - Substitutionen in der Ausgabezeile	213
Variablen und Operationen	214
.ART / .TYPE - Variablenart angeben	215
Spezielle Anwendungen von Variablenarten	217
.FORMEL / .FORMULA - Variablen Werte zuweisen	221
Individuelle Aufzählungen	222
.KOPIEREN / .COPY - Variablenwerte kopieren	222
.ZUWEISEN / .ASSIGN - Zuweisen mit Wertebereichsprüfung	224
Substitutionen in Formelausdrücken	226
.SETZE NACHFRAGE / .SET DEMAND - Variable im Dialog einlesen	226
Operationen	226
Programme aufrufen und beenden	240
.EINGABE / .INPUT - Unterprogramm ausführen	240
.WEITER / .CONTINUE - Rücksprung zum Aufrufer	245
.ENDE / .END - FO-Ende	245
.SETZE FENSTERSCHLIESSEN / .SET CLOSEWINDOW - Ausgabefenster schließen	247
.SETZE DIAGNOSE / .SET ERRORREPORT - Fehlerprotokoll	247
Benutzereingaben anfordern	248
.LESEN / .READ - Tastatureingaben einlesen	248
.BOX / .BOX - Ausgabe mehrzeiliger Meldungen	249
.MENUE / .MENU - Menüauswahlfenster	253
Eingabe von Dateien, Temporärdateien	257
.EDITIERE / .EDITING - Systemaufrufe	257
.HOLE VON / .SELECT FROM - Lesen aus einer Datei	258
.DATEI / .FILE - Lesen aus Dateien	261
Vergleich von .datei und .hole von	267
Temporärdateien in FOPs	268
Datenbankkommandos	276
Allgemeine Datenbankkommandos	276
Anwendungsspezifische Datenbankkommandos	315
Holen des zuvor geholten Datensatzes	359
FO-Zugriffe auf Sachmerkmalleisten- und Selektionsleistenrohlinge	361
Chargenanbindung an die FO	365

Sperren setzen	367
.BELEGEN / .OCCUPY - Programme synchronisieren	367
.SETZE ALLEIN / .SET SINGLEUSER - Allein-Sperre setzen/lösen	370
.SETZE UNTERBRECHUNG / .SET INTERRUPT - Unterbrechung	371
.SETZE SICHERN / .SET SAVE - Sichern-Sperre	371
Kommandos für landesspezifische Einstellungen	372
.SETZE SPRACHEINSTELLUNGEN / .SET LANGUAGESETTINGS - Sprachspezifische Ausgaben	372
.SETZE DEZPUNKT / .SET DECPOINT - Dezimaltrennzeichen setzen	373
.SETZE TSDTRENN / .SET THOUSSEP - Tausendertrennzeichen setzen	374
System- und Abas Kommandos	376
.SYSTEM / .SYSTEM - Aufruf von Betriebssystemkommandos	376
.SETZE HINTERGRUND / .SET BACKGROUND - Bildschirminhalt nicht aktualisieren	378
.KOMMANDO / .COMMAND - Aufruf von Abas Kommandos	379
Dynamische Kommandos	387
Kommandos für die PC-Client-Kommunikation	387
.PC.COPY / .PC.COPY - Datei kopieren	388
.PC.OPEN / .PC.OPEN - Datei öffnen	389
.PC.EXEC / .PC.EXEC - Programm ausführen	391
.PC.CLIP / .PC.CLIP - Inhalt der Zwischenablage holen	392
Kopplung von Abas an Microsoft Word und Excel	393
DDE-Kommandos in FOPs	394
.DDE.INIT / .DDE.INIT - Kanal öffnen	395
.DDE.EXIT / .DDE.EXIT - Kanal schließen	395
.DDE.EXEC / .DDE.EXEC - Befehl senden	396
.DDE.SEND / .DDE.SEND - Daten senden	396
.DDE.REQUEST / .DDE.REQUEST - Daten anfordern	397
.DDE.INFO / .DDE.INFO - Liste der Applikationen	397
.DDE.WAIT / .DDE.WAIT - Warten auf Schließen des Dokuments	398
Unterprogramm DDE.MENUE	398
Unterprogramm DDE.INI	399
Zusammenhang FO-Kommando - DDE-Kommando - Word/Excel-Befehl	399
Kommandos für sprachunabhängige Programmierung	399
.UEBERSETZEN / .TRANSLATE - Text in gewünschter Sprache holen	399
.ATEXT / .ATEXT - Text einer Programmmeldung übersetzen	402
.TEXT / .TEXT - Text einer FOP-Ausgabe übersetzen	403
.SETZE TRANS / .SET TRANS - Von Übersetzung ausklammern	404
.SETZE MENUTRANS / .SET MENUTRANS - Markieren als Menüzeilen	405
Sonstige FO-Kommandos	405
.HILFE / .HELP - Hilfekapitel	406
.SETZE / .SET - Zeichen mit Sonderbedeutung festlegen	406
.SETZE KOMMANDO / .SET COMMAND - Kommandozeichen setzen	408
Kein Schlüsselwort - Angabe von ausgabeabhängigen Kommandos	409
.SETZE KOPIEREN / .SET COPY - Felder über Aufrufparameter initialisieren	409
.SETZE MARKE / .SET LABEL - Zeichen für Marke setzen	410
.SETZE FRAGE / .SET QUESTION - Zeichen für Bedingung setzen	410

.SETZE ERSETZUNG / .SET REPLACEMENT - Variablenersetzungszeichen setzen	410
.SETZE TRENNER / .SET SEPARATOR - Zeilenendezeichen für Variableninhalte setzen	411
.TRENNER / .SEPARATOR - Zeilenendezeichen für Variableninhalte setzen	411
.SETZE LEER / .SET EMPTY - Geschütztes Leerzeichen setzen	411
.SETZE FOEXTERN / .SET FOEXTERN - Verhalten wie in Maske aktivieren	412
.SETZE ABBRUCH / .SET CANCEL - Abbrechen des FOPs verhindern	412
.SETZE BEREICHSENDE / .SET RANGETO - Bearbeitung des Datensatzbereichs beenden	413
.SETZE AUFZAEHLTYP / .SET ENUMERATIONTYPE - Aufzähltyp als aktuellen setzen	413
.SETZE ABLAGEKENNZEICHEN / .SET FILINGSYSTEMCODE - Ablagekennzeichen setzen	414
.SETZE DRUCKEN / .SET PRINT - Zweiten Druckpuffer aktivieren	414
.SETZE WAEHRUNG / .SET CURRENCY - Währung vorbelegen	415
.SETZE AUSWAEHR / .SET OUTCURR - Ausgabewährung vorbelegen	416
.SETZE BUCHUNGSKREIS / .SET SETOFBOOKS - Buchungskreis setzen	416
.SORTIERE / .SORT - Textdateien sortieren	418
Logging und Debugging	419
.SETZE DEBUG / .SET DEBUG - Debuggen von FOPs	419
.SETZE STRACE / .SET STRACE - Testprotokoll für Selektionen	422
.SETZE VTRACE / .SET VTRACE - Protokollierung von Variablenänderungen	425
.SETZE TRACE / .SET TRACE - Protokollierung von FO-Programmen	427
Debug-Modus im Infosystem FO-Programm ausführen	428
Debuggen von EFOPs auf der GUI	428
Zugriff auf Freitexte	429
Lesen eines Freitextes und Ausgabe in eine Liste	429
Lesen von einzelnen Freitextzeilen	429
Anlegen oder löschen von Freitexten	430
Schreiben von Freitexten	430
Kopieren von Freitexten	431
Exportieren von Freitexten	432
Drucken von Freitexten individuell steuern	433
EFOP - Programmierung ereignisgesteuerter FOPs	434
Administration von EFOPs	434
Ereignisse	434
Zusammenwirken mehrerer EFOPs für ein Ereignis	435
Maskenereignisse	437
Feldereignisse	440
Buttonereignisse	444
Zeilenereignisse - EFOPs in Tabellen	454
Selektionsereignis	458
Spezielle, generierte Ereignisse	460
Namenskonventionen für EFOPs	462
Ereigniskürzel	462
Programmnamen für EFOPs in Datenbanken	462
Programmnamen für EFOPs in Infosystemen	463
Konfigurieren von EFOPs	464
Konfigurieren von EFOPs für Selektionsleisten	464

Konfigurieren von EFOPs für Infosysteme	464
Konfigurieren von EFOPs für Datenbankmasken	464
Konfigurieren von EFOPs für maskenlose Kommandos	469
Kommandoänderung während des Editierens	469
Maskenbezogene Kommandos	470
.CURSOR / .CURSOR - Cursorsteuerung	470
.FEHLER / .ERROR - Fehlermeldung	471
.HINWEIS / .NOTE - Meldungen	474
.FARBE / .COLOUR - Setzen von Farben in Masken	475
.SCHUTZ / .PROTECTION - Schreib-/Sichtschutz	481
.CHART / .CHART - Diagramme in Masken	484
Ereignisse bei maskenlosen Kommandos	497
Protokoll der stattfindenden Ereignisse	498
Anzeige und Editieren der konfigurierten EFOPs	499
Datensatzzustand und Zugriffsrechte in EFOPs	499
Zeilenoperationen in EFOPs	499
Steuerung des Dialogs durch EFOPs	501
Steuerung des Aussehens von Masken	501
Cursorsteuerung	501
Betätigung eines Buttons per EFOP	501
Exit-Status	502
Speichern und Fenster schließen per EFOP	504
Dialogzustand und Festplattenzustand eines Datensatzes	504
Mehrsprachige Programmierung	506
Sprachabhängige Listen und Ausdrücke	506
FOPs und Mehrsprachigkeit	506
Wörterbuch für die Bedeutung von Standardvariablen	507
Übersetzungslogik bei FOPs	507
Übersetzung der FO-Befehle eines FOPs	508
FO-Variablen bei Mehrsprachigkeit	509
Entwicklungswerkzeuge	510
Programm zur Auflistung aller FO-Kommandos	510
FO-Batchbetrieb	510
Ausführen von FO-Programmen - runfop.sh	510
Ausführen von FO-Programmen (alt) - batchlg.sh	512
batchlg.spooler.sh	515
batchsperr.sh	517
Variablen in FOPs finden	521
Standard-FOPs	522
KOPF - Gestaltung des Reportkopfes	522
ARB - aufgerufener Arbeitsbereich	522
TABLEEREN - Tabelle leeren	523
DRUCKER - Ausgabemedium wählen	523
DATEI.F - Ausgabeformat für Datei	523
BILD.F - Ausgabeformat für Bildschirm	523

BLANKO.F - Ausgabeformat normal	523
BLANKOB.F - Ausgabeformat breit	523
USTVA.N.BETRAG - Steuer berechnen	523
FIRMA.KURS - Zugriff auf Währungen	523
FIRMA.KURS.GET - Betrag umrechnen	524
ZEITRAUM - Zeitraum lesen	524
URL - Anzeige einer Internetseite konfigurieren	524
ENUMMENU - Standardaufzählungen einschränken	524
EXEC.COMMAND4REFERENCE - Datensatz zu einem Verweisfeld öffnen	525
Quick Reference	526
.ABSATZ / .PARA - Absatzschutz	526
.AKTION / .ACTION - Transaktion	526
.ART / .TYPE - Variablenart angeben	527
.ATEXT / .ATEXT - Text einer Programmmeldung übersetzen	527
.AUSGABE / .OUTPUT - Ausgabemedium festlegen	527
.BELEGEN / .OCCUPY - Programme synchronisieren	529
.HOLE BETRIEBSAUFTRAG / .SELECT WORKORDER - Betriebsauftrag	530
.BLOCKSATZ / .JUSTIFIED - Blocksatz	531
.BOX / .BOX - Ausgabe mehrzeiliger Meldungen	531
.BRINGE / .REWRITE - Datensatz zurückschreiben	531
.BROWSER / .BROWSER - URL im Browser anzeigen	532
.BUDGET / .BUDGET - Summierung von Kontenbeträgen	533
.CHART / .CHART - Diagramme in Masken	533
.CURSOR / .CURSOR - Cursorsteuerung	534
.DATEI / .FILE - Lesen aus Dateien	534
.DAZU / .ADD - Weiteren Datensatz holen	535
.DDE.EXEC / .DDE.EXEC - Befehl senden	535
.DDE.EXIT / .DDE.EXIT - Kanal schließen	536
.DDE.INFO / .DDE.INFO - Liste der Applikationen	536
.DDE.INIT / .DDE.INIT - Kanal öffnen	536
.DDE.REQUEST / .DDE.REQUEST - Daten anfordern	537
.DDE.SEND / .DDE.SEND - Daten senden	537
.DDE.WAIT / .DDE.WAIT - Warten auf Schließen des Dokuments	537
.DRUCKEN / .PRINT - Druckausgabe	537
.EDITIERE / .EDITING - Systemaufrufe	538
.EINGABE / .INPUT - Unterprogramm ausführen	538
.ENDE / .END - FO-Ende	539

.FARBE / .COLOUR - Setzen von Farben in Masken	540
.FEHLER / .ERROR - Fehlermeldung	540
.FLATTERSATZ / .UNJUSTIFIED - Flattersatz	541
.FORMAT / .FORMAT - Ausgabeformat umschalten	541
.FORMEL / .FORMULA - Variablen Werte zuweisen	542
.GEDRUCKT / .PRINTED - Druckwarteschlange	542
Interpreter-Einstellungen	542
HTML- und XML-Ausgabe	542
Eingabe von Dateien, Temporärdateien	543
Allgemeine Datenbankkommandos	543
Anwendungsspezifische Datenbankkommandos	543
Kommandos für Datensätze der Fertigung	543
Kommandos für Datensätze der BDE	543
Kommandos für die Disposition	543
Kommandos für Verbrauchsfolgeverfahren	543
Kommandos für die Materialwirtschaft	543
Kommandos für die Finanzbuchhaltung	543
Holen des zuvor geholten Datensatzes	543
FO-Zugriffe auf Sachmerkmaleisten- und Selektionsleistenrohlinge	543
Zugriff auf einzelne Merkmale einer Sachmerkmaleiste	544
Sperren setzen	544
Sonstige FO-Kommandos	544
Logging und Debugging	544
Maskenbezogene Kommandos	544
.HILFE / .HELP - Hilfefkapitel	544
.HINWEIS / .NOTE - Meldungen	544
.HOLE / .SELECT - Datensätze holen	545
.HOLE AMASKE / .SELECT ASCREEN - Kopf oder Zeile des A-Puffers holen	545
.HOLE BDE / .SELECT PDC - BDE-Datensätze	546
.HOLE BINDUNG / .SELECT LINK - Lagerzugänge und -abgänge	546
.HOLE BUCHUNG / .SELECT ENTRY - Buchungszeile holen	547
.HOLE DATEI / .SELECT FILE - Datensätze aus Zusatztable	547
.HOLE DAZU / .SELECT ADD - Weiteren Datensatz holen	547
.HOLE EINKAUFS / .SELECT PURCHASING - Für Datensätze des Einkaufs	548
.HOLE FERTIGUNGS / .SELECT PRODUCTION - Betriebsauftrag	548
.HOLE FIFO / .SELECT FIFO - Ersten Datensatz holen	549
.HOLE GRUPPE / .SELECT GROUP - Datenbankgruppe holen	549
.HOLE JAHR / .SELECT YEAR - Besetzt die Datumsvariablen	549
.HOLE KONTO / .SELECT ACCOUNT - Konto holen	550

.HOLE KOPF / .SELECT HEADER - Holen des Tabellenkopfes	550
.HOLE LIFO / .SELECT LIFO - Letzten Datensatz holen	550
.HOLE LMENGE / .SELECT SQUANTITY - Lagermengen-Datensatz aus dem Pufferbereich holen	551
.HOLE LOHNFERTIGUNG / .SELECT SUBCONTRACTING - FO-Zugriffe auf Lohnfertigung	551
.HOLE LPLATZ / .SELECT LOCATION - Lagerplatz-Datensatz aus dem Pufferbereich holen	551
.HOLE MASKE / .SELECT SCREEN - Tabellenkopf/-zeilen in EFOPs	551
.HOLE MASKE ZEILE / .SELECT MASK LINE - Zeilenselektion im geladenen Puffer	552
.HOLE MBELEGUNG / .SELECT MACHLOADING - Maschinenbelegung	552
.HOLE MONAT / .SELECT MONTH - Zeitraum für Finanzbuchhaltung wählen	552
.HOLE MZUORDNUNG / .SELECT MALLOCATION - FO-Zugriffe auf Materialzuordnungen	553
.HOLE OBJEKT / .SELECT OBJECT - Allgemeines Datenbankobjekt laden	553
.HOLE PLANKARTE / .SELECT PLANCHART - Beschaffung und Materialreservierung	553
.HOLE SPERREN / .SELECT LOCK - Zugriffssperre beim Holen eines Datensatzes	554
.HOLE TEILE / .SELECT PARTS - Artikeldatensatz an Lagergruppe anpassen	555
.HOLE VERKAUFS / .SELECT SALES - Für Datensätze des Verkaufs	556
.HOLE VERWENDUNG / .SELECT APPLICATION - Verwendungsnachweis holen	557
.HOLE VKZ / .SELECT VKZ - Verkehrszahlen holen	557
.HOLE VON / .SELECT FROM - Lesen aus einer Datei	557
.HOLE WEITER / .SELECT CONTINUE - Weiterschalten	558
.HOLE ZEILE / .SELECT LINE - Holen von Tabellenzeilen	558
..!INTERPRETER / ..!INTERPRETER - Arbeitsweise des FO-Interpreters	558
.KOMMANDO / .COMMAND - Aufruf von Abas Kommandos	559
.KOPIEREN / .COPY - Variablenwerte kopieren	559
.LADE / .LOAD - Datensätze laden	560
.LAENGE / .LENGTH - Seitenlänge	560
.LESEN / .READ - Tastatureingaben einlesen	561
.LINKS / .LEFT - Linker Rand	561
.LOESCHEN / .DELETE - Datensatz löschen	561
.MACHE / .MAKE - Anlegen und Löschen von Datensätzen bzw. Zeilen	562
.MENUE / .MENU - Menüauswahlfenster	562
.MERKE / .RESERVE - Aktuellen Ladezustand von .hole- und .dazu-Puffer holen	563
.MERKE VERWENDUNG / .RESERVE APPLICATION - Beschaffungen zu Materialreservierungen	564
.OBEN / .UP - Kopfzeilen	564
.PC.COPY / .PC.COPY - Datei kopieren	565
.PC.EXEC / .PC.EXEC - Programm ausführen	565

.PC.OPEN / .PC.OPEN - Datei öffnen	565
.PRINTLN / .PRINTLN - Text ausgeben	566
.RECHTS / .RIGHT - Rechter Rand	567
.SCHUTZ / .PROTECTION - Schreib-/Sichtschutz	567
.SEITE / .PAGE - Seitenbereich und Seitenumbruch	569
.SETZE NUMMERNZÄHLER / .SET NUMBERCOUNTER - Nummernzähler setzen	569
.SETZE / .SET - Zeichen mit Sonderbedeutung festlegen	569
.SETZE ABBRUCH / .SET CANCEL - Abbrechen des FOPs verhindern	570
.SETZE ABLAGEKENNZEICHEN / .SET FILINGSYSTEMCODE - Ablagekennzeichen setzen	570
.SETZE ALLEIN / .SET SINGLEUSER - Allein-Sperre setzen/lösen	570
.SETZE AMASKE / .SET ASCREEN - A-Puffer zum M-Puffer machen	571
.SETZE AUFZAEHLTYP / .SET ENUMERATIONTYPE - Aufzähltyp als aktuellen setzen	571
.SETZE AUSGABE / .SET OUTPUT - Substitutionen in der Ausgabezeile	571
.SETZE AUSWAEHR / .SET OUTCURR - Ausgabewährung vorbelegen	572
.SETZE BEREICHSENDE / .SET RANGETO - Bearbeitung des Datensatzbereichs beenden	572
.SETZE BUCHUNGSKREIS / .SET SETOFBOOKS - Buchungskreis setzen	572
.SETZE DEBUG / .SET DEBUG - Debuggen von FOPs	573
.SETZE DEZPUNKT / .SET DECPOINT - Dezimaltrennzeichen setzen	573
.SETZE DIAGNOSE / .SET ERRORREPORT - Fehlerprotokoll	573
.SETZE DOPPELPUNKT / .SET COLON - HTML-Befehle markieren	574
.SETZE DRUCKEN / .SET PRINT - Zweiten Druckpuffer aktivieren	574
.SETZE ERSETZUNG / .SET REPLACEMENT - Variablenersetzungszeichen setzen	574
.SETZE FENSTERSCHLIESSEN / .SET CLOSEWINDOW - Ausgabefenster schließen	575
.SETZE FOEXTERN / .SET FOEXTERN - Verhalten wie in Maske aktivieren	575
.SETZE FORMATIERT / .SET FORMATTED - Substitutionen für Variablen mit Formatierung	575
.SETZE FRAGE / .SET QUESTION - Zeichen für Bedingung setzen	576
.SETZE HINTERGRUND / .SET BACKGROUND - Bildschirminhalt nicht aktualisieren	576
.SETZE INLINE / .SET INLINE - Fließtext beibehalten	576
.SETZE KOMMANDO / .SET COMMAND - Kommandozeichen setzen	576
.SETZE KOPIEREN / .SET COPY - Felder über Aufrufparameter initialisieren	577
.SETZE LEER / .SET EMPTY - Geschütztes Leerzeichen setzen	577
.SETZE LF / .SET LF - Zeilenvorschub setzen	577
.SETZE MARKE / .SET LABEL - Zeichen für Marke setzen	578
.SETZE MENUTRANS / .SET MENUTRANS - Markieren als Menüzeilen	578
.SETZE NACHFRAGE / .SET DEMAND - Variable im Dialog einlesen	578

.SETZE ROHLING / .SET TEMPLATE - Rohling laden	578
.SETZE SELTRENNER / .SET SELSEPARATOR - Trennzeichen setzen	579
.SETZE SICHERN / .SET SAVE - Sichern-Sperre	580
.SETZE SML / .SET CHB - Merkmale der Sachmerkmaliste	580
.SETZE SPRACHEINSTELLUNGEN / .SET LANGUAGESETTINGS - Sprachspezifische Ausgaben ..	580
.SETZE STOP / .SET STOP - Bildschirm seitenweise füllen	581
.SETZE STRACE / .SET STRACE - Testprotokoll für Selektionen	581
.SETZE SUBSTITUTION / .SET SUBSTITUTION - Substitutionen für Variablen	581
.SETZE TRACE / .SET TRACE - Protokollierung von FO-Programmen	582
.SETZE TRANS / .SET TRANS - Von Übersetzung ausklammern	582
.SETZE TRENNER / .SET SEPARATOR - Zeilenendezeichen für Variableninhalte setzen	582
.SETZE TSDTRENN / .SET THOUSSEP - Tausendertrennzeichen setzen	583
.SETZE UNTERBRECHUNG / .SET INTERRUPT - Unterbrechung	583
.SETZE VARQUOTE / .SET VARQUOTE - HTML-Code in Variableninhalten	583
.SETZE VERWEIS / .SET REFERENCE - Datenübernahmemodus setzen	584
.SETZE VTRACE / .SET VTRACE - Protokollierung von Variablenänderungen	584
.SETZE WAHRUNG / .SET CURRENCY - Währung vorbelegen	585
.SORTIERE / .SORT - Textdateien sortieren	585
.SORTIERETABELLE / .SORTTABLE - Tabelle sortieren	586
.SYSTEM / .SYSTEM - Aufruf von Betriebssystemkommandos	586
.TABELLENSATZ / .TABLERECORD - HTML-Tabellen erzeugen	587
.TEXT / .TEXT - Text einer FOP-Ausgabe übersetzen	587
.TRENNER / .SEPARATOR - Zeilenendezeichen für Variableninhalte setzen	587
.UEBERSETZEN / .TRANSLATE - Text in gewünschter Sprache holen	588
.UNTEN / .DOWN - Fußzeilen	589
.VTAB / .VTAB - Vertikaltabulatoren	590
.WEITER / .CONTINUE - Rücksprung zum Aufrufer	590
.ZEIGEN / .VIEW - Fensterbreite festlegen	590
.ZEILE / .LINE - Nächste Ausgabezeile	591
.ZUWEISEN / .ASSIGN - Zuweisen mit Wertebereichsprüfung	591

FO-Programmierschnittstelle

Die FO ("Flexible Oberfläche") ist eine der wichtigsten Programmierschnittstellen von Abas. Sie umfasst die folgenden Hauptfunktionen:

- Grundfunktionen der Textverarbeitung
- Datenbankzugriff
- Reportgenerator
- Ein- und Ausgabe
- übliche Rechen- und Stringoperationen
- Dialogsteuerung in Eingabemasken
- Systemkommandos und Client-Kommunikation

Die Programmierschnittstelle basiert auf einer einfachen, leicht zu erlernenden Programmiersprache. Sie ist mit frühen BASIC-Dialekten vergleichbar und kann auch von Benutzern mit geringer Programmiererfahrung verwendet werden.

Für Fortgeschrittene bzw. mächtigere Programme steht darüber hinaus eine [Java-Schnittstelle](#) mit größerem Funktionsumfang und umfassenden syntaktischen Möglichkeiten zur Verfügung. Sie basiert auf der modernen objektorientierten Programmiersprache Java und ermöglicht auch den Einsatz der Programmiersprache Groovy.

Programme, die für die FO erstellt werden, heißen **FOPs** (FO-Programme). Die Kommandos der FO-Sprache sind mit vordefinierten Funktionen anderer Programmiersprachen vergleichbar. Sie können auf Deutsch oder Englisch angegeben werden.

Sie können sich jederzeit die vollständige Liste aller FO-Kommandos sowie Listen aller Parameter und aller Optionen mit dem Programm `getfocmd` auf Shell-Ebene generieren lassen.

Erste Schritte mit FO-Programmen

FOPs schreibt man meist nicht für sich selbst, sondern für und mit anderen. Damit alle Beteiligten es einfacher haben, gibt es auch in der FO [Richtlinien](#). Ihre Einhaltung machen FOPs robuster und lesbarer.

Programmieren lernt man am besten durch ausprobieren. Wenn Sie noch kein FOP geschrieben haben, sollten Sie gleich mit einfachen Beispielen experimentieren, während Sie diese Einführung lesen.

Als Vorarbeit können Sie auf der grafischen Oberfläche ein [individuelles Benutzermenü](#) anlegen. Für die FO-Programmierung werden vor allem vier Kommandos für die Erstellung, den Aufruf, die Bearbeitung und das Löschen von FOPs benötigt. Diese Kommandos haben folgende Syntax:

Neuerstellung eines FOPs	<Text> Name <Neu>
Aufruf eines FOPs	<Text> Name <Zeigen>
Bearbeiten eines FOPs	<Text> Name <Ändern>
Löschen eines FOPs	<Text> Name <Löschen>

Dabei steht **Name** jeweils für einen [zulässigen Namen eines FO-Programms](#). In den nächsten Kapiteln werden diese Aufrufe näher beschrieben.

Allgemeine Programmierrichtlinien

Unter folgendem Link finden Sie das PDF der aktuellen FO-Programmierrichtlinien:

[FO-Programmierrichtlinien](#)

FOP erstellen, ausführen, ändern und löschen

FOP erstellen

Führen Sie das Kommando `<Text> Name <Neu>` aus und tragen Sie den Namen des FOPs ein.

Für **Name** kann entweder ein einfacher Dateiname oder die Form Verzeichnis/Dateiname verwendet werden (siehe unter [Benennung von FO-Programmen](#)).

Das angegebene Verzeichnis muss direkt unter **MANDANTDIR** liegen und Sie müssen [Recht zum Zugriff](#) haben (siehe unter [Gliederung der FO-Programme](#)). Dieses Recht ist mit Ihrem Passwort verknüpft. FOPs aus anderen Verzeichnissen können weder gelesen noch verändert noch aufgerufen werden. Existiert die genannte Datei noch nicht, wird sie neu angelegt und im [Texteditor](#) zur Bearbeitung angezeigt; andernfalls erfolgt eine Rückfrage.

Beispiel:

Weisen Sie der Variablen **xtext** die Ziffer 1 zu und geben Sie einen Text und den Inhalt der Variablen **xtext** aus.

```
.type text xtext  
.formula U|xtext = "first"
```

```
Hello world - this is the 'U|xtext' F0 program!
```

FOP ausführen

Beispiel:

Führen Sie folgendes Kommando aus, um ein vorhandenes FOP auszuführen.

```
<Text> Name <Zeigen>
```

Darstellung:

```
Hello world - this is the first F0 program!
```

FOPs ändern

Beispiel:

Führen Sie folgendes Kommando aus, um ein vorhandenes FOP zu ändern.

```
<Text> Name <Ändern>
```

Für **Name** kann entweder ein einfacher Dateiname oder die Form Verzeichnis/Dateiname verwendet werden (siehe unter [Benennung von FO-Programmen](#)).

Das angegebene Verzeichnis muss direkt unter MANDANTDIR liegen und Sie müssen [Recht zum Zugriff](#) haben (siehe unter [Gliederung der FO-Programme](#)). Dieses Recht ist mit Ihrem Passwort verknüpft.

FOPs löschen

Beispiel:

Führen Sie folgendes Kommando aus, um ein vorhandenes FOP zu löschen.

```
<Text> Name <Löschen>
```

Hinweis: Die nachfolgenden Kommandos löschen das FOP **Name** in allen Bediensprachen.

```
<Text> Name <Löschen>
```

```
<Text> D/Name <Löschen>
```

Aufruf von FOPs

Die Kommandosyntax für den Aufruf (d.h. die Ausführung) eines FOPs lautet

```
<Text> Name <Zeigen>
```

Wenn das auszuführende FOP in einem Unterverzeichnis liegt, muss dieses (anders als bei der Neuanlage oder Änderung) beim Aufruf nicht angegeben werden. Dadurch können Benutzer mit unterschiedlichen Rechten verschiedene FOPs ausführen, wenn sie ein Kommando geben. Fehlt die Angabe des Unterverzeichnisses, wird nämlich nach dem Namen zunächst im Grundbereich und dann der Reihe nach in allen im Passwort aufgeführten Verzeichnissen gesucht. Beim ersten Erfolg bricht die Suche ab, und das gefundene Programm wird ausgeführt.

Benennung von FOPs

Die allgemeine Struktur eines FOP-Namens ist:

`['Sprachkürzel']/['Arbeitsbereich']/DATEINAME'`

Sprachkürzel muss eins der definierten Kürzel einer konfigurierten Sprache sein.

Die maximale Länge eines FOP-Dateinamens liegt bei 64 Zeichen. Für Dateiname und Verzeichnis zusammen stehen 160 Zeichen zur Verfügung.

Der Name eines FO-Programms muss mit einem Buchstaben beginnen. Er kann Buchstaben, Ziffern, Punkte und Kommas enthalten. Groß- und Kleinschreibung sind nicht relevant.

Abas bringt die Namen in eine Standarddarstellung, wobei das Verzeichnis klein und der Dateiname groß geschrieben wird.

Beispiel:

`ow1/ARTIKELLIST`

Es können also keine Programmdateien erstellt oder ausgeführt werden, deren Name Kleinbuchstaben enthält. Dadurch wird verhindert, dass Dateien mit Kleinbuchstaben für jeden Abas-Benutzer zugänglich sind.

Sprachverzeichnis

Fehlt bei der Angabe des FOP-Namens die Angabe eines Sprachverzeichnisses (z.B. `st/LKU`, `ow1/PROG`), so wird das FOP nur im aktuellen Bediensprachenverzeichnis gesucht.

Durch Voranstellen des Bediensprachenkürzels und eines Slashes (/) kann der Text in jedem beliebigen Bediensprachenverzeichnis, durch Voranstellen von "D/" auch im Installationssprachenverzeichnis geändert werden.

Beispiel:

Die Bediensprache sei Englisch:

`<text> X <edit> ändert E/X`

`<text> E/X <edit> ändert E/X`

`<text> D/X <edit> ändert X`

FOPs, die in mehreren Bediensprachen verwendet werden, werden aus dem Installationssprachenverzeichnis

in alle Bediensprachenverzeichnisse kopiert. Deutschsprachige Texte können nur in der Bediensprache Deutsch geändert werden. Zur Durchführung der Änderung müssen Sie also die Bediensprache wechseln.

Direktmodus

Der FO-Direktmodus wird mit folgendem Kommando gestartet:

<Text> <Zeigen>

Im Direktmodus können einzelne Befehle zeilenweise auf der Tastatur eingegeben werden. Jeder geschriebene Befehl wird sofort nach dem nächsten <Return> ausgeführt. Diese Arbeitsweise eignet sich insbesondere für das entwicklungsbegleitende Testen von neuen FOPs.

Bei Fehlern erhalten Sie entsprechende Hinweise (im Direktmodus erfolgt im Gegensatz zu FOPs kein Abbruch bei Fehlern).

Ein- und Ausgaben

Ein- und Ausgaben des FOPs erfolgen in ein scrollbares Fenster, das nur bei Bedarf geöffnet wird. Ohne Ein- und Ausgaben wird also nicht sichtbar, dass ein FOP aktiv ist. Wenn das FOP nur ein Menü liefert, wird nur dieses dargestellt. Wenn das FOP ausschließlich Leerzeilen ausgibt, erscheint das Fenster ebenfalls nicht.

Gliederung der FOPs

Die mitgelieferten FO-Programme sind in sogenannte Arbeitsbereiche (auch "Textbereiche" genannt) einsortiert. Diese dienen der inhaltlichen Gliederung und der Zugriffskontrolle. Einem Arbeitsbereich entspricht im Dateisystem ein Verzeichnis, dessen Name nur aus Kleinbuchstaben besteht.

Mit der Eingabe Ihres Passwortes erhalten Sie Zugriff auf bestimmte Arbeitsbereiche (siehe [Arbeitsbereich](#)). Einige Bereiche werden bereits bei der Installation eingerichtet. Darüber hinaus können Sie auch eigene Bereiche anlegen und löschen.

Ein spezieller Arbeitsbereich ist der sogenannte Grundbereich. Ein FOP, das im Grundbereich liegt, befindet sich direkt im Mandantenverzeichnis (**\$MANDANTDIR**).

Wenn Sie den Arbeitsbereich beim Aufruf eines FOPs nicht angeben, wird das Programm zuerst im Grundbereich und anschließend in den Arbeitsbereichen gesucht, für die Sie eine Zugangsberechtigung haben. Letzteres geschieht in der im Erlaubnisdatensatz festgelegten Reihenfolge. Das FOP mit dem angegebenen Namen, das zuerst gefunden wird, wird ausgeführt. Gibt es zwei FOPs mit demselben Namen in verschiedenen Verzeichnissen, wird also das FOP aus dem zuerst aufgeführten Arbeitsbereich aufgerufen. Demzufolge können bei zwei verschiedenen Benutzern, die dasselbe FOP aufrufen, durchaus unterschiedliche Programme ausgeführt werden. Wenn Sie dagegen beim Aufruf eines FOPs den Arbeitsbereich angeben (z.B. **1a/LJ**), legen Sie das FOP eindeutig fest. Die beschriebene Suche ist dann nicht erforderlich.

Schließlich beeinflusst auch die Sprache des Benutzers den Suchalgorithmus für FOPs: Die Suche beginnt immer in seinem Sprachverzeichnis. Die Namen von Sprachverzeichnissen bestehen aus einem Buchstaben. Für englischsprachige Benutzer beginnt die Suche z.B. unterhalb des Verzeichnisses **E**, für spanischsprachige unterhalb des Verzeichnisses **S**. Ist die Suche dort nicht erfolgreich, wird ersatzweise ohne Sprachverzeichnis gesucht.

Wenn Sie beim Aufruf eines Programms oder Unterprogramms eine Fehlermeldung der Art "xxxdir: kein gültiger Arbeitsbereich" erhalten, bedeutet das, dass das Verzeichnis "xxxdir" entweder nicht existiert oder nicht zu Ihren erlaubten Arbeitsbereichen gehört.

Individualisierung von FOPs

Firmenspezifische FOPs müssen in einem individuellen Verzeichnis abgelegt werden. Die Namenskonvention für diese Verzeichnisse ist **ow*** (**ow**, dem beliebig viele Kleinbuchstaben - auch keiner - folgen können). Damit sind Namenskonflikte mit Standardverzeichnissen ausgeschlossen und es ist garantiert, dass diese Bereiche bei einem Upgrade nicht verändert werden. Weiter unten finden Sie ein Organisationsbeispiel.

Änderungen an einem Standard-FOP müssen an einer Kopie dieses FOPs in einem individuellen Verzeichnis der Herstellersprache Deutsch vorgenommen werden. Danach können Sie mit dem Übersetzungsprogramm **trans** in alle Sprachverzeichnisse übersetzen. Somit ist eine Änderung nur einmal nötig.

Wenn ein Standardprogramm sicherstellt, dass seine Unterprogramme im selben Arbeitsbereich liegen müssen (z.B. durch Verwendung des Unterprogramms ARB), ist es notwendig, dass auch die Unterprogramme (siehe **.eingabe**) in das individuelle Verzeichnis kopiert werden.

Beispiel:

Sie möchten das Standard-FOP für den Rechnungsdruck (**RE**) aus dem Bereich "Verkauf" (**vk**) an die besonderen Anforderungen Ihres Betriebs anpassen. Beim Aufruf von **RE** soll Ihr individuelles FOP erscheinen. Die Änderungen sollen selbstverständlich auch bei einem Upgrade erhalten bleiben.

1. Rufen Sie das Skript **hilfe.sh** auf.
(Über GUI: In der Kommandoübersicht [Erlaubnis] [Arbeitsbereich] kann im Arbeitsbereich durch Einfügen eines neuen Verzeichnisses ein Verzeichnis angelegt werden.)
2. Wählen Sie h. aus.
3. Legen Sie ein Verzeichnis an. Als Namen können Sie z.B. **ow0** bis **ow9** vergeben.
4. Tragen Sie im Passwortdatensatz (siehe unter [Passwortdefinitionen](#), [Erlaubnisse](#), [Arbeitsbereich](#)) in der Zeile der Verzeichnisse an erster Stelle den neuen Bereich (z.B. **ow**) ein, damit Ihr FOP vor dem entsprechenden Standardprogramm gefunden wird.
5. Kopieren Sie das FOP **RE** mit dem Kommando

<Text> vk/RE <Neu>

(Über GUI: [Listen/Ausdrucke] [Sonstige Listen])

Oder kopieren Sie das FOP, indem Sie im [Administrationswerkzeug](#) **hilfe.sh** erst h., dann b., dann [Kopieren] wählen. Dabei bleibt das originale FOP **vk/RE** im Bereich "Verkauf" erhalten. Änderungen dürfen nur an Kopien vorgenommen werden. Damit haben Sie die Möglichkeit, jederzeit erneut auf das Standard-FOP zurückzugreifen.

6. Die Frage nach dem neuen Namen beantworten Sie mit

*** ow/RE <Return>**

7. Führen Sie die Änderungen in dem ausgegebenen FOP wie geplant durch.
8. Rufen Sie die angepassten FOPs auf mit

<Text> RE <Zeigen>

und kontrollieren Sie die Ausführung. Beruht das angepasste FOP auf der Kopie eines Standard-FOPs, wie hier dargestellt, kann es sogar über denselben Menüpunkt wie vorher aufgerufen werden, vorausgesetzt, der Arbeitsbereich wurde im Menübefehl nicht angegeben.



Die Änderung von FOP-Bezeichnungen (hier "RE") ist nicht empfehlenswert, da sie umfangreichere Anpassungen in weiteren Unterprogrammen und im Menüsystem erforderlich machen.

Parameterübergabe an FOPs

An FOPs können ähnlich wie an JFOPs Parameter übergeben werden. Sie müssen nach dem FOP-Namen aufgeführt und durch Leerzeichen voneinander getrennt werden.

Beispiel: Parameterübergabe in der Kommandosyntax

```
{key-text} PROGNAME param1 param2 param3 {key-zeigen}
```

Ausgeführt wird hier das FOP **PROGNAME**, als Parameter werden **param1**, **param2** und **param3** übergeben.

Parameterübergabe bei EFOPs:

Beispiel: Parameterübergabe in **fop.txt**

```
0 NEU maskein - - - [C] STAMMDATEN KUNDE maskein
```

Ausgeführt wird hier das EFOP **STAMMDATEN** (Spalte 8 in der Datei **fop.txt**), als Parameter werden **KUNDE** und **maskein** übergeben.



Die Parameterübergabe wird bei EventHandlern nicht unterstützt.



Die Parameterübergabe wird bei FO-Unterprogrammen, die mit dem Kommando **.eingabe** aufgerufen werden, nicht unterstützt.

Die Parameterübergabe wird bei JFOPs, die mit dem Kommando **.eingabe** aufgerufen werden unterstützt.

Nach der Abfragemöglichkeit im FOP wird zwischen Array- und Namensparametern unterschieden. Array-Parameter gelangen in die vordefinierten Variablen '**G|arg.i**', Namensparameter hingegen erzeugen frei festlegbare FO-Variablen.

Array-Parameter

Der FOP-Aufruf hat für Array-Parameter die Form

```
{key-text} FOPNAME [ param1 [ param2 [ ... ] ] ] {key-zeigen}
```

Die Parameter können im herkömmlichen FO-Programm mit dem Array '**arg.i**' abgefragt werden. Ihre Anzahl plus eins gibt die Variable '**G|argc**' an. Derart übergebene Parameter werden als **Array-Parameter** bezeichnet. In JFOPs sind diese Variablen nicht gesetzt, die Übergabe erfolgt dort im Parameter **args[]** der **runFop**-Methode.

Dabei ist:

- '**G|argc**' (Argumentcounter) die Anzahl der Parameter + 1
- '**G|arg0**' der Name des FOPs
- '**G|arg1**' der erste Parameter
- '**G|arg2**' der zweite Parameter
- '**G|arg.i**' der i-te Parameter

Für den Gebrauch der Argumente empfehlen wir folgende Konventionen:



- Zuerst stehen Optionen. Diese fangen prinzipiell mit einem Minus an.
- Bestimmte definierte Optionen dürfen ein Folgeargument (ohne Minus) haben, auf das die davor stehende Option angewendet werden soll.
- Nach den Optionen folgen ggf. Argumente (ohne Minus), auf die das Kommando angewandt werden soll.

Beispiel:

```
<Text> EINPROGRAMM = -a 1.1.2024 -e 31.12.2026 -x -y 1 2 3 4 5 <Zeigen>
```

Hier stehen zwei Optionen mit Folgeargument (-a und -e), danach die Optionen -x und -y, danach sonstige Argumente.

Namensparameter

Der FOP-Aufruf hat für Namensparameter die Form

```
{key-text} FOPNAME = varname1=wert1 [ varname2=wert2 ... ] {key-zeigen}
```

- Die Aufrufparameter sind prinzipiell vom Typ **Text**.

- Wenn der erste Aufrufparameter ein `=` ist, erfolgt die Übergabe der Parameter in der Form `variablenname=wert`. Dadurch können beim Aufruf eines FOP, Variablen erzeugt und Werten zugewiesen werden.
- Wenn der erste Parameter kein `=` ist, erfolgt die Übergabe der Parameter in Form eines `Text`-Arrays.
- Das Programm bricht ab, wenn beim FOP-Aufruf Probleme mit den Aufrufparametern auftreten, oder wenn ein unzulässiger Variablenname verwendet wird (z.B. `&=4`).

Beispiel:

Übergeben Sie beim Aufruf des FOPs `EINPROGRAMM` die Parameter `xpstart=1.1.2025` und `xpend=1.2.2025`.

```
<Text> EINPROGRAMM = xpstart=1.1.2025 xpend=1.2.2025 <Zeigen>
```

Das Kommando erzeugt die beiden FO-Variablen `'xpstart'` und `'xpend'` und weist ihnen die angegebenen Werten zu.

Beispiel:

Lesen Sie im aufgerufenen FOP, ein Anfangs- und ein End-Datum ein.

```
.read "start date " U|xpstart ?_defined(U|xpstart)
.read "end date " U|xpend ?_defined(U|xpend)
```

In diesem Beispiel werden die Variablen nur dann eingelesen, wenn sie beim FOP-Aufruf, nicht als Parameter angegeben wurden.

Entscheidung zwischen Array-Parametern und Namensparametern

In einigen Fällen ist es bequemer, Namensparameter zu verwenden. Der Aufruf selbst ist dann zwar etwas länger, aber auch besser lesbar, zumal das aufgerufene Programm deutlich einfacher ist. Ein weiterer wichtiger Vorteil ist, dass die Parameter in beliebiger Reihenfolge angegeben werden können:

```
<Text> X = xpa=1 xpb=2 <Zeigen>
```

ist äquivalent mit

```
<Text> X = xpb=2 xpa=1 <Zeigen>
```

Hingegen ist

```
<Text> X 1 2 <Zeigen>
```

im Normalfall nicht äquivalent mit

```
<Text> X 2 1 <Zeigen>
```

Hierbei handelt es sich um eine bedeutende Fehlerquelle.



Zur besseren Lesbarkeit sollten Parameter mit "xp" beginnen.

Wenn die Anzahl gleichbedeutender Variablen variiert, bietet sich die Verwendung von Array-Parameter an. Sie kommt auch in Betracht, wenn die Gesamtlänge eines Abas Kommandos (einschließlich Parameter) auf 2999 Zeichen beschränkt ist. Durch den Wegfall der Variablennamen können mehr Array-Parameter übergeben werden.

Programmbeispiel

Das folgende Beispiel kann als Muster zum Einlesen der Optionen in FO verwendet werden.

Beispiel: Übergeben von Parametern beim Aufrufen eines FOPs

FOPEXAMPLE [-a start date] [-e end date] [account1 account2 ...]

```
.type int xc xc1
.type date xanf xend
.continue INIT
!OPTIONERROR
.box
..incorrect program call <Text> FOPEXAMPLE
Options:
FOPEXAMPLE [ -a start date ] [ -e end date ] [ account1 account2 ...]
..
.END 1
..
!INIT
.. Initialize option variables with default values
.formula U|xanf = "."
.formula U|xend = "."
..
.. xc always indicates the edited argument
.formula U|xc = 1
..
.. read options
!OPTLOOP
.continue MAIN ? U|xc >= G|argc
.continue OPTIONERROR ? G|arg.xc / "-*" & G|arg.xc <> "-a" & G|arg.xc <> "-e"
.continue MAIN ? G|arg.xc <> "-a" & G|arg.xc <> "-e"
.formula U|xc1 = U|xc + 1
.continue OPTIONERROR ? U|xc1 >= G|argc
.formula U|xanf = G|arg.xc1 ? G|arg.xc = "-a"
.formula U|xend = G|arg.xc1 ? G|arg.xc = "-e"
.formula U|xc = U|xc + 2
.continue OPTLOOP
..
!MAIN
options are read, process arguments ...
start datum = 'U|xanf:8'
end datum = 'U|xend:8'
```

```
..  
!ARGLOOP  
.end ? U|xc >= argc  
.. edit arguments. printing text only  
process G|arg['U|xc'] = 'G|arg.xc ' ...  
.formula U|xc = U|xc + 1  
.continue ARGLOOP  
..  
program end  
.end 0
```

Der Aufbau der FO-Sprache

In den folgenden Kapiteln wird der Aufbau der FO-Sprache beschrieben.

Wenn ein Programm Syntaxfehler oder undefinierte Befehle enthält oder Befehle mit undefinierten Optionen oder falschen Argumenten aufgerufen werden, bricht das Programm normalerweise mit einem Fehlerhinweis ab. Eine Meldung weist auf die betreffende Programmstelle hin.

Die FO-Sprache wird (außer in sehr seltenen, unvermeidbaren Ausnahmefällen) so erweitert, dass lauffähige Programme, die die Dokumentation einhalten, auch noch in späteren Versionen lauffähig sind. Befehle, die "zufällig" ausführbar sind, obwohl sie die Dokumentation nicht einhalten, können dagegen in späteren Versionen genauer geprüft werden und dann einen Laufzeitfehler auslösen.

Die FO ist eine interpretierte Sprache, bei der zur Laufzeit auch syntaxrelevante Zeichen umdefiniert werden können; anders als bei der [Java-Programmierschnittstelle](#) steht daher kein Compiler zur vorherigen Fehlerprüfung zur Verfügung.

Die wichtigsten syntaktischen Zeichen sind in der Standardeinstellung:

' (Apostroph) gefolgt von Variablenname und weiterem Apostroph	Für Ersetzungen von Variablen (siehe Hinweise zur Notation)
.. (Punkt Punkt) am Zeilenanfang	Kommentarzeile (siehe .. - Kommentar)
. (Punkt) am Zeilenanfang	FO-Kommando (siehe Kommandos)
!MARKE	Vergeben von Marken (siehe Marken und Sprünge)
? in Zeilenmitte	Wenn Bedingung (siehe Bedingungen)
? am Zeilenanfang	(bedingtes) Ausgeben der Zeile (Angabe einer Bedingung in Zeilenmitte möglich)

Interpretation

- Jede Zeile, die mit einem Punkt oder Doppelpunkt beginnt, wird normalerweise als FO-Kommando interpretiert (auch Punktbefehl).
- Jede Zeile, die mit einem Ausrufezeichen beginnt, wird normalerweise als Sprungmarke interpretiert.
- Jede andere Zeile wird als reiner Text betrachtet, der ausgegeben werden soll.

Normale Textzeilen dürfen also mit keinem dieser Zeichen in der ersten Spalte beginnen (siehe [Kommandos](#) und [Textausgabe und Textformatierung](#)).



Die Zeilenlänge in FOPs ist auf 2999 Zeichen begrenzt. Das Verhalten bei längeren Zeilen ist undefiniert, d.h. es kann ohne weitere Ankündigung jederzeit geändert werden. Für die Länge der einzelnen Argumente von FO-Kommandos gibt es in aller Regel keine weitere Einschränkung, es sei denn, in der Beschreibung des Kommandos wird eine solche genannt. Wenn Variablen als Argumente angegeben werden, ist die Länge ihres Inhaltes maßgeblich für die Maximallänge 2999.

Kommentare

Längere FOPs sollten Sie mit Kommentaren versehen. Ein Kommentar im FOP beginnt mit dem Kommandozeichen (im Standard der Punkt) und wird mit einem Punkt fortgesetzt. In der Regel wird ein Kommentar also mit 2 aufeinanderfolgenden Punkten begonnen.

```
.. Kommentar.
```

Die Vereinbarung von [Kopfzeilen](#) oder [Fußzeilen](#), die Kommandos [.menue](#) und [.box](#) und um Absätze im [Blocksatz](#) oder [Flattersatz](#) werden mit dem nächsten Kommando beendet. Dies kann z.B. auch ein Kommentar sein.



[..include](#)

Bedingungen

Jedem Kommando kann ein durch Fragezeichen abgetrennter Ausdruck nachgestellt werden. Der Ausdruck muss [G|true](#) oder [G|false](#) als Ergebnis liefern. Der Typ (die Art) des Ausdrucks muss somit "BOOL" sein. Das Kommando wird bedingt ausgeführt, nämlich nur dann, wenn der Ausdruck [G|true](#) liefert. Das Fragezeichen wird als "wenn" gelesen; der Ausdruck nach dem Fragezeichen gibt die Bedingung für das Kommando an.

```
.output new 'U|xfile' ? U|xfile <> ""
```

Die weitere Ausgabe geht nur dann auf die Datei ['U|xfile'](#), wenn ['U|xfile'](#) nicht leer ist. Wenn die Bedingung nicht erfüllt ist, bleibt das Kommando ohne Wirkung. Das Programm wird dann in der nächsten Zeile fortgeführt. Kommandos ohne Bedingung werden immer ausgeführt.

Ein boolescher Ausdruck wird bis zum Ende ausgewertet, selbst wenn das Endergebnis bereits aus einem Zwischenergebnis erschlossen werden kann. Im Zusammenhang mit [defined\(\)](#) ist das bedeutungsvoll. Die folgende Programmzeile ist beispielsweise inkorrekt:



```
.println ja ? defined(U|x) & U|x>10
```

Auch wenn ['U|x'](#) nicht definiert ist, wird die Bedingung [U|x > 10](#) geprüft, und diese Prüfung scheitert wegen der fehlenden Definition.

(Siehe auch unter [Sprünge und Marken](#) und [Variablen und Operationen](#).)

Marken und Sprünge

Im Programm kann man Marken setzen, zu denen man später mit dem Befehl `.weiter` springen kann. Für Marken gelten folgende Konventionen:

- Sprungmarken werden am Ausrufezeichen in der ersten Spalte einer Zeile erkannt: `!MARKENNAME`
- Um Marken leichter zu erkennen, empfiehlt es sich, sie in Großbuchstaben zu schreiben.
- Markennamen können beliebig lang sein; es werden von der FO jedoch nur die ersten sieben Zeichen nach dem Ausrufezeichen ausgewertet.
- Nach dem Sprungmarkenbezeichner kann, durch einen Doppelpunkt getrennt, ein Kommentar angefügt werden.
- Der Markenname gilt nur innerhalb einer Programmdatei. Innerhalb dieser Datei muss er eindeutig sein. In anderen FOPs und Unterprogrammen kann dieser Markenname erneut zur Kennzeichnung verwendet werden.

Beispiel: Sprunganweisung mit Sprungmarke

```
.continue ALRIGHT
..
!ALRIGHT: This is jump label ALRIGHT.
```

- Liegt die Sprungmarke weiter oben im Programm, so spricht man von einem Rücksprung. Rücksprünge werden z.B. für Schleifen verwendet.
- Liegt die Sprungmarke weiter unten im Programm, so spricht man von einem Vorwärtssprung. Vorwärtssprünge werden verwendet, um einen Programmteil zu überspringen.

Das Ausführen einer Sprunganweisung kann an eine Bedingung gekoppelt werden.

Beispiel: Bedingte Sprunganweisung mit Sprungmarke

```
.type bool xbstatus      ? F|defined(U|xbstatus) = false
.formula U|xbstatus = true
..
.continue DONE ? U|xbstatus
task is not done yet
complete the task
.continue END
!DONE : task is done
task is already done
!END
.end
```

Variablen und Typen

FO-Variablen (im Folgenden kurz: Variablen) sind Platzhalter im FO-Programm. Beim Programmieren von Textausgabe wird ihr Name in Apostrophe eingeschlossen, die Apostrophe definieren dann gleichzeitig ein Feld beliebiger Breite innerhalb der Zeile. Bei der Ausgabe des FO-Programms wird der Inhalt der Variablen in dieses Feld eingesetzt. Zum Beispiel können Variablen ersetzt werden:

Text	Ersetzung
'xanschrift'	Günther Zieslowski ...
'G datum'	12.05.10
'xsaldo'	4500.37
'G seite'	12

Ersetzungen werden standardmäßig nur beim Apostroph wirksam. Es können jedoch auch andere Zeichen als Ersetzungszeichen vereinbart werden; siehe [.setze - Zeichen mit Sonderbedeutung festlegen](#). Weitere Details zur Ersetzung siehe im Kapitel [Formatierung von FO-Variablen](#).

Unterscheidung von Variablen

In der FO sind Variablen verschiedener Herkunft und Gültigkeitsdauer zu unterscheiden:

Benutzerdefinierte Variablen	Können vom Benutzer definiert werden und dienen zum Speichern von Zwischenergebnissen und Programmzuständen. Nach dem Beenden eines FOPs oder EFOPs gehen sie verloren.
Systemvariablen	Durch Abas vordefinierte Variablen , meist readonly.
Datenbankvariablen	Variablen der Datensätze der Datenbank. Sie sind Felder eines Datensatzes. Sie können persistent sein, werden also auf dem Speichermedium gespeichert, oder definieren sogenannte Skipfelder, die verloren gehen, sobald der Datensatz nicht mehr geladen ist. Zugriffe in FOPs erfolgen immer über Puffer, die über einen Pufferbuchstaben angesprochen werden.
Umgebungsvariablen	Beim Programmstart in der Shell gesetzte Umgebungsvariable. Sie stehen auch allen anderen Programmen zur Verfügung, die auf derselben Shell oder von Abas aus auf dem Server gestartet werden.
Funktionen	Funktionen sind zwar im eigentlichen Sinn keine Variablen, können aber an allen Verwendungsstellen von Variablen aufgerufen werden.

Die verschiedenen Sorten von Variablen lassen sich durch die Angabe eines [Pufferbuchstabens](#) unterscheiden.

Verwendung von Variablen

Variablen haben drei Arten von Vorkommen im Programm und in den Datensätzen:

Deklarationsstellen	Hier werden Variablen deklariert, d.h. ihre Namen werden mit einem Typ verknüpft. Das Kommando .art deklariert FO-Variablen. Undeklarierte FO-Variablen werden bei ihrer ersten Definition oder Verwendung implizit deklariert. Die Felder von Datenbankobjekten oder Infosystemen werden bei der Objektbeschreibung in der Variablentabelle oder im Infosystem deklariert. Eine Sonderrolle spielen die vordefinierten Variablen, sie werden über den Satz V-13-00 der Variablentabelle deklariert.
Definitionsstellen	Hier bekommt die Variable einen Wert. Hat sie bereits einen Wert, erhält sie eventuell einen neuen. Bereits bei der Deklaration wird jede Variable mit dem Wert leer vorbelegt. Wichtigste Definitionsstelle ist das Kommando .formel. Datenbankvariablen werden auch durch den Abas Kern oder Benutzereingaben definiert.
Verwendungsstellen	Hier wird der aktuelle Wert der Variablen ermittelt und verwendet. Häufigste Art der Verwendung ist der Variablenzugriff mit dem Ersetzungszeichen Apostroph, die Variablenersetzung. Variablenname und die einschließenden Apostrophe werden intern durch den Wert der Variablen ersetzt. In Formeln und in einigen Kommandos können Variablen auch ohne Apostrophe verwendet werden.

Die Deklarationsstellen sind:

.art -Kommando	Hier wird eine FO-Variable mit einer bestimmten Art deklariert.
Variablentabelle und das Infosystem	Hier werden die Datenbankvariablen deklariert.
Definitionsstellen	Hier wird die Art der FO-Variablen implizit durch den Wert deklariert, den sie erhalten soll.
Variablenersetzungen für noch nicht definierte Variablen	Hier wird der Wert einer FO-Variablen im Dialog erfragt (nicht empfohlen).

Die Möglichkeit der impliziten Deklaration von Variablen können Sie mit der **..! INTERPRETER-Anweisung** und der Option **DECLARATION** unterbinden. Lesen Sie hierzu [Die Abkürzung von Variablen, Deklarationspflicht](#).

Die wichtigsten Definitionsstellen sind:

.formel	definiert die auf der linken Seite angegebene Variable.
.lesen	definiert die als Parameter angegebene FO-Variable.
.hole von	definiert die angegebenen FO-Variablen.
.text	und diverse weitere Kommandos definieren ebenfalls die auf der Zeile angegebenen Variablen.

Die Verwendungsstellen sind:

Variablenersetzung in Apostrophen	Die Variablenersetzung (siehe unter Formatierung von FO-Variablen) ist die häufigste Verwendung von Variablen.
Formeln im Kommando <code>.formel</code> und in Bedingungen	Hier können Variablen unmittelbar verwendet werden oder als Variablenersetzung. Bei unmittelbarer Verwendung bestimmt ihre Art die Operatoren und die Ergebnisart der Formel. Näheres siehe unter Rechnen mit Textvariablen . Bei Variablenersetzung ist die Art immer "TEXT".
Kommando <code>.dazu</code> oder <code>.hole dazu</code> (siehe unter Datenbankkommandos)	verwendet die danach angegebene Variable

Syntax der Variablenverwendung

Syntaktisch müssen folgende Arten der Variablenzugriffe unterschieden werden:

<code>nur_varname</code>	Der reine Variablenname, kommt nur in Verknüpfungen vor
<code>lhs_varname</code>	Der Variablennamen an einer Definitions- oder Deklarationsstelle. Hier sind zusätzlich Pufferbuchstaben und Punktersetzung möglich.
<code>rhs_varname</code>	Variablen an Verwendungsstellen, hier sind außer Pufferbuchstaben und Punktersetzung noch Funktionsargumente und Verknüpfungen möglich.
<code>text_zugriff</code>	Beim Textzugriff erweitern mögliche Formatierungsangaben die Syntax des Variablennamens.

Die Syntaxbeschreibung erfolgt hier mit einer einfachen Grammatik, angelehnt an eine modifizierte Backus-Naur-Form: Nichtterminale werden als Bezeichner angegeben, Terminale in Anführungszeichen gesetzt, das Gleichheitszeichen steht für die Definition. Für die Alternative werden mehrere Definitionen für dasselbe Nichtterminal verwendet.

Zunächst werden verbal die grundlegenden Zeichenmengen beschrieben:

- Buchstabe = ein beliebiger Buchstabe
- Funktionsargumente = ein beliebiger Text mit paarigen runden Klammern ohne Apostroph und mit paarigen Anführungszeichen
- Ziffer = Ziffern 0 bis 9
- dblquote = das Anführungszeichen

Für die Zeichenmenge "Funktionsargumente" gelten weitere syntaktische Regeln; sie sind im Kapitel [Funktionen](#) beschrieben.

Nach den formalen Regeln müssten die Zeichen des Unicode-Zeichensatzes aufgezählt werden, die in der jeweiligen Zeichenmenge vorkommen; hier beispielhaft für die Ziffer. Bei den anderen Zeichenmengen soll die verbale Beschreibung genügen.

`Ziffer = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"`

Verwendet werden die folgenden Nichtterminale im FOP an folgenden Stellen:

lhs_varname	links vom Gleichheitszeichen in .formel , .zuweisen , .kopieren und an allen Stellen, wo sonst noch Variablen gesetzt werden
rhs_varname	in Formeln und nach .hole dazu , .dazu und .lade X dazu
text_zugriff	ist fast überall in Punkt-Zeilen und Ausgabezeilen zulässig und bezeichnet einen Variablenzugriff mit Transformation in Text und Formatierungsangaben

Für **lhs_varname** und **rhs_varname** gilt: Die Angabe zu **genau** und **keine_null** sind nur im Kontext für **text_zugriff** zulässig. Dort sind wegen der Formatierung weitere Angaben möglich und sinnvoll; sie werden mit diesen zusätzlichen Regeln für **genau** und **keine_null** ergänzt. Zur genauen Bedeutung dieser Formatierungsangaben siehe [Formatierung von FO-Variablen](#).

```

genau          =
keine_null     =
BoZ            = Buchstabe
BoZ            = Ziffer
rest_varname   =
rest_varname   = BoZ rest_varname
nur_varname    = Buchstabe rest_varname
puf_varname    = Pufferbuchstabe "|" keine_null nur_varname
puf_varname    =               keine_null nur_varname
pkt_varname    =
pkt_varname    = "." puf_varname genau pkt_varname
lhs_varname    =               pkt_varname
lhs_varname    = puf_varname pkt_varname
funcargs       =
funcargs       = "(" Funktionsargumente ")"
verknuepfung   =
verknuepfung   = "^" nur_varname verknuepfung
rhs_varname    = lhs_varname funcargs verknuepfung cast

```

Die Syntax innerhalb von Apostrophen ist ähnlich, aber es können noch folgende syntaktische Erweiterungen genutzt werden:

nur_varname	kann auch mit einer "0" beginnen: das bedeutet Nullunterdrückung.
pkt_varname	kann ein ":" mit einer Genauigkeitsangabe folgen.
rhs_varname	kann eine Typanpassung ähnlich der Form (L="D") und eine Genauigkeitsangabe folgen.

Mehrdeutig sind in dieser Grammatik zwei Punkte:

- Die Genauigkeitsangabe: Hier bindet das Nichtterminal **pkt_varname** stärker!
- Funktionsargument und der **cast**: Die Entscheidung wird durch den Pufferbuchstaben F, oder den Funktionsnamen **defined** getroffen.

Für den Variablenzugriff in Apostrophen, also den **text_zugriff**, gelten obige Regeln und folgende zusätzliche Regeln:

```

keine_null      = "0"
genau           = ":" Genauigkeitsangabe
waehrungskuerzel = Text ohne Apostroph: ein gueltiges Waehrungskuerzel
castangabe      = Text ohne Apostroph: eine gueltige Typanpassungsangabe
rest_zahl       =
rest_zahl       = ziffer rest_zahl
zahl           = ziffer zahl
genauigkeit     = zahl
genauigkeit     = lhs_varname
genauigkeit     = dblquote waehrungskuerzel dblquote
castbuchstabe   =
castarg         = dblquote castangabe dblquote
castarg         = rhs_varname
blanks         =
blanks         = " " blanks
cast           = "(" Buchstabe "=" castarg ")"
text_zugriff    = "'" blanks rhs_varname cast genau blanks "'"

```

Die Bedeutung von Genauigkeitsangaben und Casts sowie weitere Möglichkeiten bei der Formatierung von Variableninhalten werden im Abschnitt [Formatierung von Variableninhalten](#) beschrieben.

Formatierung von Variableninhalten

Zum Setzen von Sonderzeichen wie Dezimaltrennzeichen lesen Sie Kapitel [.SETZE / .SET - Zeichen mit Sonderbedeutung festlegen](#).

Für die Ausgabe von FO-Variablen gibt es folgende Möglichkeiten:

Vollständigen Inhalt darstellen

Ist der Variablenname lückenlos in Apostrophe eingeschlossen, so wird der vollständige Inhalt der Variablen für den Ausdruck '**variablenname**' eingesetzt.

Beispiel:

```
The name is 'U|xname'.
```

wird z.B. angezeigt als

```
The name is Miller.
```

Definierte Feldbreite, links- und rechtsbündig

Steht der Variablenname links- oder rechtsbündig innerhalb der Apostrophe, so wird durch die Apostrophe eine Feldbreite definiert, die von der Ersetzung benutzt werden soll.

Steht der Variablenname linksbündig innerhalb der Apostrophe, so wird auch die Ersetzung linksbündig in

das Feld gesetzt. Entsprechendes gilt für rechtsbündig. Die Feldbreite wird hierbei genau durch die Anzahl der Zeichen zwischen den beiden Apostrophen bestimmt, wobei diese selbst ebenfalls zählen.

Beispiel:

```
The name is ' U|xname'.
```

wird z.B. angezeigt als

```
The name is Miller.
```

Ungenügende Feldbreite

Ist bei links-/rechtsbündiger Ersetzung der Variableninhalt länger als die durch Apostrophe definierte Feldbreite, so wird er abgeschnitten. Im Allgemeinen wird bei linksbündiger Darstellung rechts abgeschnitten und umgekehrt.

Chinesische Zeichen werden mit doppelter Breite gezählt, näheres siehe Abschnitt [Sonderbehandlung chinesischer Zeichen](#).

Ausnahmen beim Abschneiden Bei rechtsbündig dargestellten Zahlen (siehe unter [.art - Variablenart](#)) könnte ein Abschneiden links den Wert erheblich und noch dazu unbemerkt verfälschen. Der Überlauf wird daher mit einer Reihe von Sternen gekennzeichnet. Daneben besteht die Möglichkeit, auch bei rechtsbündiger Darstellung rechts abzuschneiden. Dieses Ausnahmeverhalten wird mit der [Umgebungsvariablen LINKSWEG](#) eingestellt. Es kann z.B. vorübergehend nützlich sein, wenn bei großen Zahlen die Centbeträge nicht ins Gewicht fallen, eine Spaltenerweiterung aber nicht sofort stattfinden kann.



Bei linksbündiger Ausgabe werden Zahlen nicht durch Sterne ersetzt. Daher kann z.B. 123.456 zu 123 werden, aber auch 123456 zu 123.

Nullen unterdrücken

Wenn die Variable eine Zahl enthält, kann deren Darstellung durch zwei zusätzliche Angaben variiert werden: Eine Null vor dem Variablennamen unterdrückt die Ausgabe des Zahlenwerts 0. Zum Beispiel wird bei `a = 0` die Eingabe `'a'` ersetzt durch `"0.00"`, die Eingabe `'0a'` aber durch Leerzeichen.

Anzahl Nachkommastellen festlegen



Veraltet: Zur formatierten Ausgabe von reellen Zahlen (Variablenart real) kann der Doppelpunkt verwendet werden. Die Zahl nach dem Doppelpunkt bestimmt die Anzahl der Nachkommastellen. Dabei wird kaufmännisch gerundet. Diese Formatierung kann für maximal 4 Nachkommastellen verwendet werden.

Alternativ: Verwenden Sie stattdessen die Funktion `F | round`.

Datumsformat

Bei einem Datum kann durch die Formatangabe ":8" die Kurzform des sortierfähigen Datums nach ISO 8601 (z.B. 20100401 für das Datum 01.04.10) erzwungen werden (siehe [Mehrsprachige Programmierung](#)). In dieser Darstellung ist ein Datum sortierfähig, ohne zuvor eine Datumsdifferenz bilden zu müssen. Werden Doppelpunkt und Längenangabe bei Variablen anderer Art eingesetzt, so begrenzen sie die Darstellung auf die angegebene Anzahl von Zeichen.

Mehrzeiliger Text

Bei links- oder rechtsbündiger Ersetzung führt ein mehrzeiliger Variableninhalt (z.B. Anschrift) dazu, dass aus einer Eingabezeile mehrere Ausgabezeilen erzeugt werden. Das Programm kennzeichnet ein Zeilenende intern durch ein Semikolon. Um die Trennung zu verhindern, können Sie das [Zeilentrennzeichen](#) umdefinieren. In Kopf- und Fußzeilen werden mehrzeilige Variablen nicht getrennt, weil sich die Zeilenanzahl in ihnen nicht variablenabhängig verändern darf (siehe unter [Textausgabe und Textformatierung](#)).

Freitextvariablen

Wenn `U|xtftext` eine Freitext-Variable ist, dann ist die Schreibweise '`U|xtftext`' in Ausgabezeilen zulässig.

Es wird dann aber nicht der Freitext, sondern nur sein interner Name ausgegeben. Den Inhalt des Freitextes gibt man normalerweise mit `.eingabe 'U|xtftext'` aus. Näheres siehe [Zugriff auf Freitexte](#).

DVM-Variablen

Wenn `U|xtdvm` eine DVM-Variable ist, dann ist die Schreibweise '`U|xtdvm`' in Ausgabezeilen zulässig.

Die Art der Ausgabe hängt aber von verschiedenen Bedingungen ab; es wird entweder der Name oder der Inhalt der Datei ausgegeben, auf die die DVM-Variable verweist. Bei links- oder rechtsbündiger Angabe wird immer der Dateiname ausgegeben. Andernfalls gilt:

Art DVM2	Es wird der Dateiname ausgegeben.
Art DVM12	Es wird der Dateiinhalt wie ein nicht mit Formatierungsauszeichnungen versehener Text ausgegeben. Bei der Ausgabe wird das letzte Zeilenende in der Datei unterdrückt. Die Ausgabe ist bei <code>.tabellensatz unformatiert</code> mehrzeilig, sonst einzeilig.
Art DVMR2	Es wird der Dateiinhalt wie ein mit Formatierungsauszeichnungen versehener Text ausgegeben. Bei der Ausgabe wird das letzte Zeilenende in der Datei unterdrückt. Die Ausgabe ist bei <code>.tabellensatz unformatiert</code> mehrzeilig, sonst einzeilig.

Explizite Typumwandlungen (Typcast) von Variablen

Die Sprache "0" bei FO-Typanpassungen bewirkt die sprachunabhängige Darstellung von Werten. Bei allen Variablenzugriffen ist zulässig:

`'x(L="0")'`

B wird wie **B1** ausgegeben, alle Real-Arten wie **RU**, alle Aufzählungen wie **AU**, Datum wie `:8`. Sie bewirkt also die sortierfähige Darstellung.

Darstellung von Aufzählungen in der FO

'ev'	liefert in Bediensprache Deutsch: "Einkauf"
'ev(L="0")'	liefert immer: "(86)"
'ev(L=":")'	liefert immer: "(Sales)"
.kopieren M tvktyp = F value(M aktvktyp, "AS163") ? F value(M abteilung, "AS7") = "(Sales)"	
.kopieren M tektyp = F value(M aktektyp, "AS162") ? F value(M abteilung, "AS7") = "(Purchasing)"	

Verknüpfung von Verweisen

Syntax der Verweisverknüpfung:

```
verweisfeld1^[verweisfeld2^verweisfeld3^...]feldname
```

- Verweisverknüpfungen erstellen Sie mit dem Hochzeichen (^).
- Mit Verweisverknüpfungen greifen Sie direkt auf die Felder eines Verweises zu. Mehrstufige Verweisverknüpfungen sind zulässig.
- Mit der Verweisverknüpfung mit Hochzeichen (^) können Sie nur die Inhalte von Kopffeldern lesen.
- Beim Feldzugriff wird das Objekt, auf das verwiesen wird, in den Cache geladen. Wenn ein Zugriff auf ein weiteres Feld der Verweisverknüpfung erfolgt, dann wird der Feldinhalt aus dem Cache gelesen.
- Wenn Sie aus einem Objekt, auf das verwiesen wird, mehrere Feldinhalte lesen wollen, dann **können** Sie das Objekt in einen Puffer laden, z.B. Ladepuffer 0.
Wenn die Feldzugriffe nicht direkt nacheinander erfolgen, ist es **sinnvoll** das Objekt in einen Puffer zu laden. Dadurch vermeiden Sie unnötige Datenbankzugriffe.
Wenn Sie in einem Objekt, auf das verwiesen wird, auf den Tabellenteil zugreifen wollen, **müssen** Sie das Objekt in einen Puffer laden.

Beispiel: Verweisverknüpfung mit 3 Verweisen

```
verweisfeld1^verweisfeld2^verweisfeld3^feldname
```

Beispiel: Postleitzahl aus Verweisfeld Kunde eines Angebots im H-Puffer lesen

```
H|kunde^plz
```



Mit der Funktion **defined** prüfen Sie, ob die Verweisverknüpfung existiert.

Abkürzung von Variablen, Deklarationspflicht

Standardmäßig können Sie Variablennamen in FOPs abkürzen. Die Mindestlänge einer Abkürzung beträgt 3 Zeichen, d.h. nur Variablen mit mehr als 3 Zeichen lassen sich abkürzen.

Ob Variablen innerhalb eines FO-Programms abgekürzt werden dürfen oder nicht, kann man mit der Option **NOABBREV** von **..!INTERPRETER** festlegen. Die Option wirkt auf das Abkürzen im aktuellen Unterprogramm.



Der Verzicht auf Abkürzen ist eine qualitätsverbessernde Maßnahme. Die Verwendung von Abkürzungen verschlechtert die Lesbarkeit, weil nicht klar ist, ob "x123" und "x1234" die gleiche Variable ist. Wenn Sie eine neue Variable in ein Programm einführen, welches Abkürzungen verwendet, müssen Sie prüfen, ob Sie dadurch abgekürzte Verwendungsstellen unbeabsichtigt ändern.

Wenn Sie eigene FOPs schreiben und Variablennamen vergeben, sollten diese zur Unterscheidung von Systemvariablen mit einem **X** beginnen, oder Sie sollten bei eigenen Variablen grundsätzlich mit dem Pufferbuchstaben U arbeiten.

Beispiel:

Unter der Voraussetzung, dass Abkürzen erlaubt ist und keine Deklarationen vorangehen, soll die Wirkung von folgenden Programmzeilen betrachtet werden:

```
.formula U|xpip = 1
.formula U|xpip2 = 2
'U|xpip'
'U|xpip2'
```

Es werden hier zwei Variablen **xpip** und **xpip2** definiert. Vertauscht man die zwei Zeilen wird nur eine Variable definiert:

```
.formula U|xpip2 = 1
.formula U|xpip = 2
'U|xpip'
'U|xpip2'
```

Wegen der Abkürzung setzt die zweite Zeile ebenfalls die Variable **xpip2**. Durch explizite Deklaration lässt sich das Problem vermeiden:

```
.type int U|xpip2
.type int U|xpip
.formula U|xpip2 = 1
.formula U|xpip = 2
'U|xpip'
'U|xpip2'
```

Bei der Deklaration von Variablen ist keine Abkürzung möglich.

Warum Sie auf die Verwendung von Abkürzungen und impliziten Deklarationen verzichten sollten:

- Die Verwendungsstellen einer Variablen lassen sich besser lokalisieren.
- Die `.art` definiert zweifelsfrei die Art der Variablen.
- Das Programm ist robuster gegen spätere Erweiterungen: bei neuen Variablen genügt es zu prüfen ob eine Variable dieses Namens bereits mit `.art` deklariert wird.

Verwendung vordefinierter Variablen

Beim Start der FO sind einige Variablen vordefiniert. Sie enthalten z.B. Konfigurationsdaten aus dem Firmenstamm, Informationen über die aktuelle Sitzung des Benutzers sowie über den aktuellen FO-Zustand (z.B. über das Ausgabegerät). Die in FOPs verfügbaren Systemvariablen sind in **V-13-00** beschrieben. Sie werden auch als Prozeduren bezeichnet, da ihr Inhalt in vielen Fällen vom Programm errechnet wird. Diese Variablen teilen sich in zwei Gruppen auf:

- pufferbezogene Variablen
- nicht pufferbezogene Variablen

Pufferbezogene Variablen sind in der Beschreibung mit "Pufferbezogen:" gekennzeichnet. Für sie wird die Angabe des **Pufferbuchstaben** empfohlen.

```
'H|lockMode'
```

Nicht pufferbezogene, vordefinierte Variablen können mit **G** oder **T** (nicht empfohlen) qualifiziert werden. Der Buchstabe **G** ist exklusiv für diese Variablen reserviert. Mit dem Buchstaben **T** werden auch vom Benutzer definierte FO-Variablen erreicht.

```
'G|date'
```

Einige der nicht pufferbezogenen Variablen sind Arrays; im Kommentar ist dann die untere und obere Grenze für den Index angegeben.

"Array(0.. maxkdatnam) :" Die vollständige Aufzählung der vordefinierten Variablen finden Sie im gleichnamigen **Kapitel**.

Konstantendefinition für die Namen von Datenbanken und Kommandos

Für den Zugriff auf die Felder **kaktnam**, **kkomname**, **kdatname**, **hdatname** stehen Integer-Konstanten zur Verfügung.

Sie sind in den FOPs **KAKTNAME.KONST**, **KDATNAME.KONST** und **KKOMNAME.KONST** definiert, die bei

Bedarf mit **.eingabe** eingezogen werden müssen:

FO-Programm	Variable beginnt mit	Beispiel
KAKTNAME.KONST enthält die Variablen für Aktionsnamen	koakt	koaktaendern
KKOMNAME.KONST enthält die Variablen für die Tippkommandos	kokom	kokominfosystem
KDATNAME.KONST enthält die Variablen für Dateinamen	kodat	kodatinfosystem

Man kann damit einfach und lesbar sprachunabhängige Vergleiche und Zuweisungen definieren.

Beispiel:

```
.input "KAKTNAME.KONST"
.println Edit ? G|evtkommd = 'kaktname.koaktaendern'
```

Die Variablen sind jeweils von der Art **int**. Die Variablennamen werden aus den jeweiligen Bezeichnern nach folgender Regel gebildet:

- Minus werden entfernt
- das Präfix **koakt**, **kokom**, **kodat** wird davor gesetzt
- Normtexten
- Abschneiden auf 19 Zeichen

Pufferangabe bei Variablen

FO-Variablen können und sollten nur mit einem Bezugsobjekt verwendet werden. Die Angabe des Bezugsobjekts stellt sicher, dass die Variable nur im damit definierten Objekt gesucht wird. Ist sie dort nicht deklariert, bricht das FO-Programm mit einer Fehlermeldung ab. Um Namenskollisionen zu vermeiden, wird dringend geraten, das Bezugsobjekt immer mit anzugeben.

Wenn die Pufferangabe weggelassen wird, bezieht das Programm die Angabe entsprechend der normalen Suchreihenfolge auf den ersten Puffer, der einen "passenden" Datensatz enthält. Die Suche nach diesem erhöht auch die Laufzeit.

Das Bezugsobjekt, auch als Pufferbuchstabe bezeichnet (da aktuell nur Bezugsobjekte mit einem Buchstaben existieren), besteht aus einem Buchstaben oder einer Ziffer und dem Sonderzeichen Pipe (|). Danach folgt der Variablenname. Beispiel: **U|artnr**

Datenbankobjekte sind nicht direkt zugänglich, sondern müssen immer zuerst in einen Puffer geladen werden. Das kann durch ein FO-Kommando geschehen, oder der Puffer steht bereits zu Beginn des FO-Programms geladen zur Verfügung.

F 	Buchstabe für FO-Funktionen. Näheres hierzu siehe Kapitel Funktionen .
S 	Sachmerkmalleisten-Variable
D 	Mit .dazu geladener Datensatz
H 	Mit .hole geladener oder mit .mache angelegter Datensatz
M 	Nur für EFOPs: Der Maskenpuffer oder M-Puffer ist nur bei ereignisgesteuerten FOPs definiert
U 	Benutzervariablen, die mit .art oder durch implizite Deklaration definiert wurden
G 	Verwenden Sie den G-Puffer für globale Variablen, die vom System (in V-13-00) definiert und nicht pufferbezogen sind. Für die möglichen Werte von G evtkommd (englisch G evtCmd) und G evttaste (englisch G evtFunctKey) stehen für die internationale Programmierung englische Konstanten in der Datei EVTCMD.CONST zur Verfügung. Die Konstanten haben die Form evtCmd+Aktion, z.B. evtCmdView, evtCmdEdit, usw. Beispiel: <pre> ..!interpreter english declaration noabbrev englvar .input EVTCMD.CONST .. .continue EDIT ? G evtCmd = U evtCmdEdit .continue TRANSF ? G evtFunctKey = U evtCmdTransfer .continue END !EDIT : comment .println edit .continue END !TRANSF : comment .println transfer .continue END !END : comment .end </pre>
T 	Die Vereinigung der Pufferbuchstaben G und U. Sind gleichnamige Variable in G und U definiert, hat G Vorrang vor U, bei abgekürzten Zugriffen die kleinere Variable, bei gleichnamigen ebenfalls die aus G.  Der T-Puffer darf laut Programmierrichtlinie nicht mehr verwendet werden. Es wird empfohlen, den Namen von benutzerdefinierten Variablen mit x beginnen zu lassen.
P 	Druckeinstellungen im P-Puffer
A 	Nur in bestimmten Submasken verfügbar, z.B. im Druckdialog: der A-Puffer

E	Umgebungsvariable
0	Die Puffer des Kommandos <code>.lade</code> im <code>.lade</code> -Puffer
1	
...	
9	
L	Selektionsleiste: Mit diesem Bezugsobjekt können Variablen in Selektionsleisten gezielt angesprochen werden. Zugriff auf das Feld <code>qint: 'L qint'</code> Suchschlüssel: <code>'L schluessel'</code>

Warum sollte der Pufferbuchstabe angegeben werden?

Durch die Angabe von Pufferbuchstaben gewinnen die Programme an Lesbarkeit und Robustheit:

- Tippfehler werden sofort aufgedeckt.
- Ein FO-Programm, das mit `.hole` und `.dazu` arbeitet, läuft fehlerfrei. Nach einem Upgrade werden in den `.dazu`-Datensatz neue Felder aufgenommen, die nach den Sichtbarkeitsregeln Felder des `.hole`-Datensatz verdecken. Das FOP wird unbemerkt Fehler erzeugen. Ein mit einem Pufferbuchstabe versehenes FOP verhindert diese Fehler.
- Der Benutzer muss nicht die ca. 10.000 Abas-Variablenamen kennen.

Beispiel:

In einem ereignisgesteuerten FOP sind drei Datensätze gleichzeitig geladen: Der Maskendatensatz sowie ein mit `.hole` und ein mit `.dazu` beschaffter Datensatz. In allen drei Datensätzen kommt das Feld `name` vor. Nach der Suchreihenfolge für Variablen wäre `name` über den zuletzt geladenen Datensatz (im Beispiel `.dazu`) verfügbar.

Mit den Pufferbuchstaben sind folgende Konstruktionen möglich:

```
.formula M|name = D|name
.read "new name" H|name
```

Suchreihenfolge von Variablen ohne Pufferbuchstaben

Fehlt die Angabe von Pufferbuchstaben, so werden einige der Pufferbuchstaben in einer vordefinierten Standardreihenfolge durchsucht. Diese Reihenfolge definiert eine Vorrangregel. Bei der Suche werden nur die Pufferbuchstaben berücksichtigt, die gefüllt sind. Es gilt folgende Reihenfolge:

S	Sachmerkmalleisten-Variablen
D	Mit <code>.dazu</code> geladener Datensatz (nur bei Verwendungsstellen!).
H	Mit <code>.hole</code> geladener oder mit <code>.mache</code> angelegter Datensatz.
M	Der Maskendatensatz (nur bei ereignisgesteuerten FOPs)

Alle anderen Pufferbuchstaben, also die Druckeinstellungsvariablen, Umgebungsvariablen, **.lade**-Puffer und der A-Puffer müssen immer mit Bezugsobjekt angegeben werden.



Zu obiger Tabelle gibt es zwei Ausnahmen: Der Puffer **S** wird bei pufferbezogenen vordefinierten Variablen nicht mit herangezogen, der Puffer **D** wird bei Definitionsstellen insbesondere auch auf der linken Seite von **.formel** nicht durchsucht.

Der A-Puffer

Der A-Puffer oder Vaterpuffer steht nur zur Verfügung, wenn eine Maske als Submaske z.B. über einen BU8-Button (siehe Beschreibung der Art **Button**) erreicht wurde oder in **Selektionsergebnissen**, wenn die Selektion aus einer Maske gestartet wurde.

Der A-Puffer ist im Wesentlichen im **Dialogzustand** und kann wie der Maskenpuffer in EFOPs angesprochen und verändert werden. Für den A-Puffer gibt es eigene Ereignisvariablen z.B. **G|avtkommd**.

Die Zeilenoperationen im A-Puffer entsprechen den **Zeilenoperationen im Maskenpuffer**, wenn dort **maske** durch **amaske** ersetzt wird.

Druckeinstellungsvariablen

In FOPs kann über den P-Puffer jederzeit auf die Variablen der momentan gültigen Druckeinstellungen zugegriffen werden. Näheres siehe unter **Druckeinstellungen**.

Umgebungsvariablen

Umgebungsvariablen und Shell-Variablen können in FOPs bei Voranstellung des Bezugsbereichs "E" ("Environment") durch **FO-Variablen** ausgedrückt werden.

Beispiel:

1. **'E|ARCHNAME'**: Architektur des Rechners. In **\$HOMEDIR/\$ARCHNAME** liegen die architekturabhängigen Dateien.
2. **.system "xclock &" ? E|DISPLAY <> ""**: bringt eine Uhr auf den Bildschirm (aber nur an einem UNIX-Arbeitsplatz oder einem X-Terminal!)
3. **.formel xtmp = "rmtmp/" + E|HOST_HOMEDIR + pid**: vergibt einen auch in Rechnernetzen eindeutigen Namen für eine Temporärdatei. Die Variable **HOST_HOMEDIR** wird in **homedir.env** gesetzt und enthält den Namen des Rechners, auf dem Abas installiert ist.

```
.type text xtmp
'E|ARCHNAME'
'E|DISPLAY'
.system "xclock &" ? E|DISPLAY <> ""
'E|HOST_HOMEDIR'
.formula U|xtmp = "rmtmp/" + 'E|HOST_HOMEDIR' + G|processId
```

'U|xtmp'

Umgebungsvariablen werden in **homedir.env profile** gesetzt. Mit dem Bezugsbereich "E" steht ein bequeme Möglichkeit des Transports von Informationen von der Shell-Ebene zum Programm zur Verfügung. Die Menge aller gesetzten Umgebungsvariablen ist mit dem Shell-Kommando **env** erhältlich. (**.system "env"**)



Nicht gesetzte Variablen erzeugen bei Abfrage keinen Programmfehler, sondern liefern einen Leertext. Beachten Sie die Groß- und Kleinschreibung, da sie von der Shell unterschieden wird.

Punktersetzung und Array-Variablen

Die Punktersetzung ersetzt die nach den Punkten stehenden Variablennamen durch die Textdarstellung ihres Inhalts. Der zusammengesetzte Text wird dann als Variablenname interpretiert. Er darf auch selbst Punkte enthalten. Der Teil vor dem ersten Punkt kann auch komplett fehlen. Die Punktersetzung arbeitet bis zur nächsten Verknüpfung, Typanpassung oder Genauigkeitsangabe. Danach wird der Ausdruck weiter von links nach rechts abgearbeitet.

Ausnahme: Die Genauigkeitsangabe gehört zu dem Variablennamen, der unmittelbar auf sie folgt.



Wenn der resultierende Variablenname einen Punkt enthält, kann auf diesen Variablennamen nicht direkt zugegriffen werden, sondern nur über den Umweg der Punktersetzung.

Durch die Punktersetzung können Variablennamen auch dynamisch bestimmt werden. An allen Stellen, an denen ein Variablenname eingesetzt wird, kann die Punktersetzung zur Berechnung von Variablennamen verwendet werden. Damit lassen sich auch Arrays simulieren.

Für die Syntax von Variablen gelten folgende Regeln:

- optional vor der Variablen: Pufferbuchstabe "|"
- optional: der Anfang des Variablennamens mit führendem Buchstaben
- danach mehrfach: ein Punkt gefolgt von einem weiteren Variablennamen

xtname0xx1, **a**, **b**, **c** seien die Namen von FO-Variablen. Dabei gilt:

a hat den Wert 0.

b hat den Wert xx.

c hat den Wert 1.

anstatt	xtname0xx1 U xtname0xx1
auch möglich	xtname.a.b.c U xtname.a.b.c

Beispiel:

Bei der Punktersetzung wird der Ausdruck `xname.a.b.c` intern durch `"name" + 'a' + 'b' + 'c'` ersetzt und der entstehende Text als Variablenname betrachtet.

```
.type text xname0xx1 a b c
.type text xname
.formula U|xname = "information string"
.formula U|a = "0"
.formula U|b = "xx"
.formula U|c = "1"
'U|xname'
.formula U|xname.a.b.c = U|xname
'U|xname.a.b.c'
```

Beispiel:

```
.type int xi
.type text xt
.type R3.2 xr
.formula xi = 2
.formula xr = 2.3
.formula xt = "suf"
'U|xi'
'U|xr'
'U|xt'
```

ursprünglicher Ausdruck	Resultierender Variablenname	
<code>U xa.U xi</code>	<code>U xa2</code>	Da die Art der Variable xr zwei Nachkommastellen vorsieht, wird der resultierende Variablenname mit einer Null aufgefüllt.
<code>U xa.U xt</code>	<code>U xasuf</code>	
<code>U xa.U xi.U x</code>	<code>U xa2suf</code>	
<code>t</code>	<code>U xa2suf</code>	
<code>U xa.U xi.U x</code>	<code>U xa2.30</code>	
<code>t</code>		
<code>U xa.U x</code>		

Beispiel: Punktersetzung, die unmittelbar mit dem Punkt beginnt

```
.type text xname
.type int xint
.formula U|xname="xint"
```

```
? equal: ? U|xint = .U|xname
```

Beispiel: Ein einfaches Array

```
.type text xtfield
.type int xi
.formula U|xixi=0
!NEXT
.formula U|xtfield.xi= "array element" + 'U|xi'
'U|xi' -> 'U|xtfield.xi'
.formula U|xixi=U|xixi+1
.continue NEXT ? U|xixi<10
```

Die Schleife definiert die Variablen **xtfield1 .. xtfield10** mit ihrem Namen als Wert.

Beispiel: Ein zweidimensionales Array

```
.type int xi1 xi2
.type text xtsep
.type text xfield
.formula U|xtsep="."
.formula U|xixi=1
.formula U|xixi=1
.formula U|xfield = "field"
..
!NEXT
.formula U|xfield.U|xixi.U|xtsep.U|xixi = U|xfield + 'U|xixi' + 'U|xtsep' + 'U|xixi'
display: 'U|xfield.U|xixi.U|xtsep.U|xixi'
.formula U|xixi=U|xixi+1
.continue NEXT ? U|xixi<=10
.formula U|xixi=0
.formula U|xixi=U|xixi+1
.continue NEXT ? U|xixi<=10
..read field 4.row/7.column and display content
.formula U|xfield.U|xixi.U|xtsep.U|xixi = U|xfield + "4" + 'U|xtsep' + "7"
'U|xfield.U|xixi.U|xtsep.U|xixi'
```

Die Schleife definiert ein zweidimensionales Feld von Variablen:

field1.1 ... field 10.1 ... field10.10

Die Variable **'U|xtsep'** zwischen den Dimensionen ist notwendig, damit die Felder identifiziert werden können: z.B. **field1.9** oder **field4.7**.

anstatt	<code>'U field.U xi1.U xtsep.U xi2'</code>
auch möglich	<pre> .type int xi1 xi2 .type text xtindex xtfeld .formula U xi1=1 .formula U xi2=2 .formula U xtindex='xi1'+"."+xi2' .formula U xtfeld.U xtindex = "Feldinhalt von: " + 'U xi1' + "." + 'U xi2' 'U xtfeld.U xtindex' </pre>

Aufsplitten von FO-Textvariablen in ein Array

Das FOP **SPLITSTR** splittet den Inhalt der FO-Textvariable **xtsplit** auf speichert das Ergebnis in einem Array **xasplit** ab.

Das FOP **SPLITSTR** verwendet immer die Raute (#) als Trennzeichen.

Sie können mithilfe der Textvariablen **xt delim** ein benutzerdefiniertes Trennzeichen festlegen, dann darf der zu trennende Text keine Raute (#) enthalten.

Beispiel: Aufsplitten einer FO-Textvariable in ein Array

Splitten Sie, mithilfe des FOPs **SPLITSTR** den Inhalt der Variablen **xtsplit**.

Geben Sie den Originalstring, das Trennzeichen, die Anzahl der Arrayelemente und die Arrayelemente selbst aus.

```

.type text xtsplit      ? F|defined(U|xtsplit) = false
.formula U|xtsplit = "1#23#456"
.input SPLITSTR
original:              'U|xtsplit'
separator:             'U|xt delim'
number of array elements: 'U|xispmx'
array element 1:       'U|xasplit1'
array element 2:       'U|xasplit2'
array element 3:       'U|xasplit3'

```

Vordefinierte Variablen

Im folgenden sind alle vordefinierten Variablen beschrieben. Bei der [Verwendung vordefinierter Variablen](#) muss der Pufferbezug beachtet werden. Variablen, die sich auf einen geladenen Datensatz beziehen, sind als "pufferbezogene FO-Variablen" beschrieben. Sonstige Systemvariablen sind erkennbar am Pufferbuchstaben G.

Allgemeine FO-Variablen

G abteilung	Diese FO-Variable nennt die Abteilung des Bearbeiters aus dem Passwortdatensatz. Mit der FO-Variablen ' G abteilung ' werden nur die ersten 40 Zeichen ausgegeben.
aeob	Diese pufferbezogene FO-Variable ist für Ein- und Verkaufsköpfe sowie für Betriebsauftrag/Rückmeldung (P9:2) definiert. Sie hat den selben Wert wie die Variable uebertr dieser Datensätze. Wenn die Variable true liefert, dann handelt sich also um einen aktiven Vorgang, d.h. auch dass Inhalte im Vorgang änderbar sind. Wurde der Vorgang aber gebucht (übertragen) und ist nicht mehr aktiv, dann liefert die Variable false .
aktzeile	Diese pufferbezogene FO-Variable enthält die Nummer der aktuellen Tabellenzeile.  Die Variable 'aktzeile' kann sowohl in FOPs als auch in EFOPs verwendet werden, allerdings nicht bei FOPs im Ein- und Verkauf.
G apostroph	Diese Konstante enthält den Apostroph und kann z.B. in Formeln verwendet werden, wenn aus syntaktischen Gründen kein Apostroph geschrieben werden kann.
G arber	Diese FO-Variable enthält die für den jeweiligen Benutzer im Arbeitsbereich definierten gültigen Textverzeichnisse.
G arg	Array mit den an das aufgerufene FOP übergebenen Argumenten. Siehe hierzu auch: Array-Parameter . In JFOPs nicht gesetzt; die Übergabe erfolgt dort im Parameter args[] der Startmethode runFOP .
G argc	Anzahl der an das aufgerufene FOP übergebenen Argumente. Siehe hierzu auch: Array-Parameter . In JFOPs nicht gesetzt; die Übergabe erfolgt dort im Parameter args[] der Startmethode runFOP .
G aufrufern	Die Maskenvariable enthält die Aufrufernummer des Datenbankprozesses. Sie ist für die Dauer der Sitzung innerhalb eines Mandanten eindeutig und fest. Kommt der Prozess zum Ende, wird sie vom nächsten wieder verwendet. Sie ermöglicht es, den zu einem Prozess gehörenden Bildschirm zu erkennen und identifiziert die Prozesse z.B. beim Warten auf eine Sperre. Innerhalb einer Aufrufernummer kann es beim Aufruf von Submasken mehrere G aktionnr geben, z.B. beim Aufruf des Druckdialogs. Die FO-Variable G aufrufern ist schreibgeschützt. Näheres siehe unter Eindeutiger Name im Mandanten.
G aktionnr	Mit G aktionnr können Masken unterschieden werden, die im gleichen Prozesskontext laufen. Dies ist in erster Linie bei FOPs von Interesse, die innerhalb der EDP-Schnittstelle ablaufen, denn dort können mehrere Infosysteme innerhalb eines Prozesses ausgeführt werden. Die EFOPs jedes Infosystems haben dabei eine eigene, private G aktionnr je ausgeführter Infosystem-Instanz.

G|astd

Diese FO-Variable enthält die Anzahl der verfügbaren Arbeitsstunden des Betriebskalenders oder der Abteilung/Maschinengruppe, die im Hauptpuffer geladen ist, im Zeitraum G|adatum bis G|edatum. Die verfügbaren Arbeitsstunden einer Abteilung ergeben sich als Summe der Arbeitsstunden aller Maschinengruppen der Abteilung.

Beispiel: Anzahl Arbeitsstunden im aktuellen Zeitraum

```
.. load work center or department
.select capacity "12345"
.. display working hours for the next 14 days (by the week and
complete)
.formula G|startDate = "+1"
.formula G|dueDate = G|startDate + 6
working hours for the next 7 days : 'G|countWorkHrs'
.formula G|startDate = G|dueDate + 1
.formula G|dueDate = G|startDate + 6
working hours for the ensuing 7 days: 'G|countWorkHrs'
.formula G|startDate = "+1"
the next 14 days
```



In einigen FO-Programmen findet sich vor der Ausgabe noch der Befehl **.hole plankarte** (Verweis auf **.hole plankarte**), mit dem früher die Arbeitsstundentabelle geladen wurde. Dieser ist unschädlich, wird aber nicht mehr benötigt.

G|batchmode

Diese Variable erlaubt die Unterscheidung zwischen interaktiver Bedienung am Terminal und automatisierten Lader- oder Batchläufen. Die Variable G|batchmode ist vom Typ "Bool".



Verbindung mit DIALOGMODE

Wenn die EDP-Sitzungsoption **DIALOGMODE** gesetzt ist, wird für G|batchmode der Wert **false** ausgegeben, um den FO-Programmen anzuzeigen, dass Dialoge ausführbar sind. Ist die Sitzungsoption nicht gesetzt, hat die FO-Variable den Wert **true**.

G bemail	Die der FO bekannte globale Variable G bemail liefert die passwortbezogene E-Mail-Adresse aus dem Feld E-Mail (pemail). Die Variable wird beim Start und bei jeder weiteren Anmeldung im System mit der im Feld E-Mail der [Passwortdefinitionen] hinterlegten Adresse belegt. Um eine passwortbezogene E-Mail-Adresse zu hinterlegen, gehen Sie so vor: Laden Sie die [Passwortdefinitionen] in die Maske mit <Firma> PASs <Ändern> und geben Sie ins Feld E-Mail die Adresse ein, z.B. name@firma.de. Beim Start eines FOPs oder im Direktmodus liefert 'G bemail' dann die gewünschte Adresse, hier name@firma.de.
G bvmge	Diese FO-Variable wird gesetzt, nachdem die Kommandopaare .merke verwendung/.hole verwendung und .merke bindung/.hole bindung aufgerufen wurden. Nähere Informationen finden Sie im Kapitel Kommandos für Verbrauchsfolgeverfahren oder im Kapitel Kommandos für die Disposition.
G bvken	Diese FO-Variable wird gesetzt, nachdem die Kommandopaare .merke verwendung/.hole verwendung und .merke bindung/.hole bindung aufgerufen wurden. Nähere Informationen finden Sie im Kapitel Kommandos für Verbrauchsfolgeverfahren oder im Kapitel Kommandos für die Disposition.
G bzeichen	Diese Variable enthält normalerweise das Bearbeiterzeichen, das für den aktuellen Benutzer im Passwortdatensatz des Firmenstamms eingetragen ist. Wenn der aktuelle Prozess aber nur der Ausführende für einen anderen Prozess, den Auftraggeber, ist, kann man dafür sorgen, dass die Variable die Form "Eigenes Zeichen/Auftraggeber-Zeichen" erhält. Dies ist unter Batchspooler näher beschrieben. Ferner kann die Variable ggf. auch durch die Identnummer des Webbenutzer-Datensatzes ergänzt werden, siehe Variable G neueswebpassw.
G clhostname	Die FO-Variable G clhostname enthält den Rechnernamen des PCs, auf dem die GUI läuft. Wenn die GUI in einer Remotesitzung läuft (Windows-Remotedesktop), ist das der Rechnernamen des Remotedesktop-Servers. Die Variable ist auch im EDP-Server gesetzt, wenn der EDP-Client diese Information beim Anmelden angegeben hat. Das ist unter anderem dann der Fall, wenn das EDP-ActiveX-Control oder die Shellskripte edpimport.sh, edpexport.sh oder edpinfosys.sh verwendet werden.
G clhostadr	Die FO-Variable G clhostadr enthält die IP-Adresse des PCs, auf dem die GUI läuft. Wenn die GUI in einer Remotesitzung läuft (Windows-Remotedesktop), ist das die IP-Adresse des Remotedesktop-Servers. Die Variable ist auch im EDP-Server gesetzt, wenn der EDP-Client diese Information beim Anmelden angegeben hat. Das ist unter anderem dann der Fall, wenn das EDP-ActiveX-Control verwendet wird.
G cllocalname	Die FO-Variable cllocaladdr enthält den Rechnernamen des PCs, auf dem die GUI angezeigt wird. Wenn die GUI in einer Remotesitzung läuft (Windows-Remotedesktop), ist das der Rechnernamen des PCs, auf dem die Remotesitzung läuft. Wenn das nicht der Fall ist, enthält die Variable den Rechnernamen des PCs, auf dem die GUI läuft (also dasselbe wie die Variable G clhostname).

G cllocaladr	Die FO-Variable G cllocaladr enthält die IP-Adresse des PCs, auf dem die GUI angezeigt wird. Wenn die GUI in einer Remotesitzung läuft (Windows-Remotedesktop), ist das die IP-Adresse des PCs, auf dem die Remotesitzung läuft. Wenn das nicht der Fall ist, enthält die Variable die IP-Adresse des PCs, auf dem die GUI läuft (also dasselbe wie die Variable G clhostadr).
G clstddrucker	Die FO-Variable G clstddrucker enthält den Namen des Standard-Druckers auf dem PC auf dem die GUI läuft. Die Variable ist nur in der GUI gesetzt.
G clwindir	Die FO-Variable G clwindir enthält den Pfad des Unterverzeichnisses win des Mandanten aus Sicht der GUI. Dieser Pfad kann auf unterschiedlichen PCs verschieden sein. Diese Variable ist nur bei der Arbeit mit GUI belegt.
G cltempdir	Die FO-Variable G cltempdir enthält den Namen des GUI-Temporärverzeichnisses auf dem GUI-Client. Das Verzeichnis wird beim Start der GUI angelegt und beim Beenden der GUI gelöscht. In dieses Verzeichnis können Dateien mit .pc.copy kopiert und dann über .pc.open oder .pc.exec mit einem anderen Programm geöffnet werden. Diese Datei muss dann nicht im win-Verzeichnis liegen, wo sie auch von anderen Benutzer geöffnet werden kann. Der Verzeichnisname enthält als Pfadtrennzeichen einen Backslash (\), damit der Verzeichnisname leichter an andere Windows-Programme übergeben werden kann. Beispiel: Kopieren und öffnen einer vorhandenen Datei In diesem Beispiel wird die vorhandene Datei TEST.TXT vom Mandantenverzeichnis in das GUI-Temporärverzeichnis kopiert. Die Kopie wird dann mit einem passenden Programm (z.B. Notepad) geöffnet. Die Anführungszeichen G dblquote müssen angegeben werden, da G cltempdir Leerzeichen enthalten kann. <pre style="background-color: #f9f9f9; padding: 10px;">.type text xtfiler TempDir: 'G clientTempDir' .formula U xtfile = G marksQuot + G clientTempDir + "\" + "TEST.TXT" + G marksQuot .pc.copy -pc -conv TEST.TXT 'U xtfile' .pc.open 'U xtfile'</pre>
G datum	Diese Variable enthält das Datum beim Start des FO-Programms.  Dies ist nicht notwendigerweise das aktuelle Tagesdatum. Falls Sie dieses brauchen, können Sie es durch Zuweisung von "." an G datum erhalten (die Variable ist nicht schreibgeschützt).

G dblquote	Diese Konstante enthält die Anführungszeichen und kann z.B. in Formeln verwendet werden, wenn aus syntaktischen Gründen keine Anführungszeichen geschrieben werden können.
G dde	 Die FO-Variable G dde wird ausschließlich im Abas Windows Client unterstützt. Diese FO-Variable ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. Diese FO-Variable gibt an, ob es sich um einen DDE-fähigen Arbeitsplatz handelt und ob die DDE-Kommunikation freigeschaltet ist. Siehe hierzu auch Kommandos für die PC-Client-Kommunikation.
G ddesrvname	 Die FO-Variable G ddesrvname wird ausschließlich im Abas Windows Client unterstützt. Diese FO-Variable ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. Diese FO-Variable enthält den Namen des DDE-Servers. Siehe Konfiguration des DDE-Servers.
G ddekanal	 Die FO-Variable G ddekanal wird ausschließlich im Abas Windows Client unterstützt. Diese FO-Variable ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. Diese FO-Variable enthält die aktuelle DDE-Kanalnummer. Siehe hierzu auch Koppelung von Abas an Microsoft Word und Excel.
G ddename	 Die FO-Variable G ddename wird ausschließlich im Abas Windows Client unterstützt. Diese FO-Variable ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. Diese FO-Variable enthält den Namen des geöffneten Excel/Word- Dokuments. Siehe hierzu auch Koppelung von Abas an Microsoft Word und Excel.

G dderow	 Die FO-Variable G dderow wird ausschließlich im Abas Windows Client unterstützt. Diese FO-Variable ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. Diese FO-Variable enthält das Kürzel der Zeile einer geöffneten Excel-Tabelle. Die Ausgabe auf Excel 2002 oder Excel 2003 wird von FO-Programmen unterstützt, die eigene DDE-Befehle zur Formatierung der Excel-Tabelle verwenden. Dazu existiert die Variable G dderow, in der das Kürzel der Zeile gespeichert ist. Siehe hierzu auch Koppelung von Abas an Microsoft Excel.
G ddecol	 Die FO-Variable G ddecol wird ausschließlich im Abas Windows Client unterstützt. Diese FO-Variable ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. Diese FO-Variable enthält das Kürzel der Spalte einer geöffneten Excel-Tabelle. Die Ausgabe auf Excel 2002 oder Excel 2003 wird von FO-Programmen unterstützt, die eigene DDE-Befehle zur Formatierung der Excel-Tabelle verwenden. Dazu existiert die Variable G ddecol, in der das Kürzel der Spalte gespeichert ist. Siehe hierzu auch Koppelung von Abas an Microsoft Excel.
G dnsn	Die FO-Variable G dnsn enthält die Datei und Datensatznummer der in H und N geladenen Sätze. Die Werte korrespondieren mit den von id gelieferten Werten. (Nur für Testzwecke.)
G dnsn3	Es gilt dasselbe wie für die FO-Variable G dnsn . Zusätzlich wird noch die Datei und Datensatznummer des Maskenpuffers geliefert. (Nur für Testzwecke.)
G itname	Diese FO-Variable enthält den Namen der geladenen Druckerbeschreibung.
drob	Diese pufferbezogene FO-Variable enthält Ein- und Verkaufsobjekte aus der Druckwarteschlange.
G dezpunkt	Diese FO-Variable enthält den Wert des Feldes "Dezimaltrennzeichen" der entspr. Sprache aus dem Konfigurationsdatensatz. Sind die Felder leer, ist G dezpunkt mit einem Punkt belegt. Sie kann durch .setze dezpunkt verändert werden. Das gewünschte Zeichen kann auch per Variable angegeben werden. Neben .setze dezpunkt ist also auch .setze dezpunkt 'xtrenner' möglich.
G dezpunktchar	Diese FO-Variable enthält den aktuellen Wert für Dezimaltrennzeichen.  Der Wert 0 steht für den Standarddezimalpunkt aus dem Konfigurationsdatensatz. Siehe auch: .setze dezpunkt.

disponiert	Wenn die Disposition parallel zum laufenden Betrieb arbeitet, kann man mit dieser pufferbezogenen Variablen ermitteln, ob ein bestimmter Vorgang schon von der Disposition eingeplant wurde oder nicht. Diese Frage beantwortet disponiert für folgende Vorgänge: • Auftrags- und Bestellpositionen (V-03-02, V-04-02) • Bestell- und Fertigungsvorschläge (V-04-02, V-04-05) • Reservierungen (V-04-04) • Artikel (V-02-01) in Kombination mit der Lagergruppe • Dienstleistungen (V-02-05) in Kombination mit der Lagergruppe • Maschinengruppen (V-08-02) in Kombination mit der Lagergruppe Ist disponiert gleich G true, so ist der Vorgang bereits eingeplant und es liegen für die Disposition keine Aufträge zu Mengen-, Termin- und anderen Änderungen vor. Ist disponiert gleich G false, so ist der Vorgang entweder noch nicht eingeplant oder er muss von der Disposition nochmals geändert werden.
G edp	Mit der FO-Variablen G edp kann abgefragt werden, ob das FOP auf der EDP-Schnittstelle läuft. Siehe auch unter EDP-Schnittstelle und weitere Schnittstellen .
G eigensperr	Diese FO-Variable gibt an, ob die Sperre an der eigenen Sperre scheitert.

G|einmalpw

Das Einmalpasswort kann explizit erfragt werden, um es an anderer Stelle statt des herkömmlichen Passworts einzugeben. Es ist genau eine Minute lang gültig. Zur expliziten Weitergabe des Einmalpassworts dient die FO-Variable **G|einmalpw** (englisch: **G|oneTimePwd**), die bei jeder Abfrage ein neues Passwort des aktuell angemeldeten Benutzers oder eines **fremd nutzbaren Passworts** enthält. Mit dem Einmalpasswort können Prozesse im Namen und mit den Rechten des Benutzers, für den es ausgestellt wurde, gestartet werden.

Beispiel:

Aufruf eines EDP-Shellskripts mit expliziter Weitergabe der Benutzeridentität:

```
..!interpreter english noabbrev englvar
.type text xsys
..
.formula U|xsys = "edpimport.sh -b ... -I data.csv -p " + G|oneTimePwd
.system 'U|xsys'
```



Ein Einmalpasswort kann beim **Systemaufruf** auch direkt über Parameter mitgegeben werden.



Ein Einmalpasswort an einen Spooler **batchlg.spooler.sh** zu übergeben, ist nicht sinnvoll und nicht möglich wegen der auf eine Minute begrenzten Lebensdauer. Richten Sie Spooler immer so ein, dass sie mit einem festen Passwort arbeiten. Die Umgebungsvariable AUZEICHEN sorgt beim Spooler für eine Zuordenbarkeit des Auftrages anhand des Aufruferzeichens in der Form Spooler-Zeichen/Auftraggeberzeichen.

G|extuser

Bei der Anbindung von Fremdsoftware (etwa Faxlösungen unter Novell und NT) entsteht das vor allem für die GUI relevante Problem, dass der Benutzer nicht eindeutig zuzuordnen ist, weil der GUI-Anwendungsserver immer als derselbe UNIX-Benutzer (erp, demo, ...) gestartet wird. Zur Behebung dieser Schwierigkeit dient das Feld **G|extuser** im Passwortdatensatz, in das sich für den externen User eine Kennung in Form eines beliebigen Textes hinterlegen lässt. Der Wert dieses Feldes steht als globale FO-Variable **G|extuser** zur Verfügung. Diese Variable kann bei der Ausgabe auf Fax oder E-Mail angegeben werden. Sie wird dann beim Systemaufruf ausgewertet.

Zur Standardlieferung gehören bereits angepasste Programme zur Übergabe des Namens des externen Users:

```
$HOMEDIR/ind/ow/bin/printemail.sh
$HOMEDIR/ind/ow/bin/printlangner.sh
$HOMEDIR/ind/ow/bin/printnetfax.sh
$HOMEDIR/ind/ow/bin/printsmo.sh
$HOMEDIR/refI/MAND.INI Druckprogramme:
```

```
ek/BE LSE REE, vk/AN AU LS RE
```

Vorgehen bei der Anpassung

1. MAND.INI

Die Variable '**xextuser**' ist entsprechend den Vorgaben aus **\$HOMEDIR/refI/MAND.INI** zu definieren.



Soll der Name des externen Users nicht ausgewertet werden, reichen zur Anpassung folgende Zeilen in **MAND.INI** aus:

```
.art text xextuser
.formel xextuser="#"
```

2. FO-Programm (hier am Beispiel von **vk/AN**)

Der Name des externen Users wird als letzter Parameter übergeben.

Beispiel:

bisher:

```
.output 'G|currOutMed' 'U|xtfax' : 'U|xtprio' 'U|xttime'
'U|xtcomment'
```

neu:

G false	Diese FO-Variable enthält den Wert von G false in der jeweiligen Bediensprache. Sie ist vom Typ "Bool".
G frei	Diese FO-Variable enthält die noch freie Zeile auf der Ausgabeseite.
fterm	Diese pufferbezogene FO-Variable enthält den frühesten Endtermin.
G gesperrt	Die Sperre wurde erfolgreich gesetzt.
gbest	Diese pufferbezogene FO-Variable enthält die Bestandssumme aller Dispositionslager. Wird der Wert für einen Artikel aufgerufen, ist die Bestandssumme der internen Lagergruppe enthalten. In den Lagergruppeneigenschaften ist die Bestandssumme der betreffenden Lagergruppe enthalten.
G geraet	Diese Variable nennt das aktuelle Ausgabegerät. Es kann mit .ausgabe auf umgeschaltet werden. Näheres siehe unter Textausgabe und Textformatierung .
G grafik	Der Wert dieser Variable ist G true , wenn die GUI läuft. In diesem Fall darf ein FOP keine interaktiven UNIX-Programme starten, weil für diese die Eingabe scheitert. Zum Beispiel führt .system vi zur Meldung "input read error". Bestehende Programme sollten deshalb mit einer Abfrage ausgerüstet werden, z.B. .weiter Fehler ? grafik .
grschl	(Intrastat) Diese pufferbezogene FO-Variable beinhaltet die Schlüsselnummer der Rechnung des deutschen Ein- und Ausfuhrhafens.
G hdatname	Dieses schreibgeschützte Array enthält die Namen für die Dateiangabe für .hole Kunde (Lieferant, ...) in der aktuellen FO-Sprache. Es ist von Index 0 bis 'G maxkdatnam' gefüllt. Das FOP s3sys/KOMLIST gibt die Namen aus. Das Array ist nützlich für sprachunabhängige Programmierung, siehe auch unter Schlüsselwörter ermitteln. Beispiel: Wenn die Bediensprache Deutsch ist, enthält 'hdatname0' "Kunde". database name: 'G selectDBNames0'
ident	Diese pufferbezogene FO-Variable enthält die Nummer bzw. das Suchwort bei geladenem Artikel oder Arbeitsgang.
G incst	Diese FO-Variable enthält das Datum ('G adatum') und die Anzahl der Arbeitsstunden ('G std').
G icook	 für den Status "OK".
G icowarn	 für den Status "Warnung".

G icoerror	 Diese FO-Variable enthält im Standard das Icon für den Status "Fehler".
icoukrit	 Diese pufferbezogene FO-Variable enthält im Standard das Icon für den Status "Unkritisch".
G icovkrit	 Diese FO-Variable enthält im Standard das Icon für den Status "Vorgänger kritisch".
G icokrit	 Diese FO-Variable enthält im Standard das Icon für den Status "Kritisch".
G instnr	Diese FO-Variable enthält die Abas-Installationsnummer. Sie gibt Aufschluss über Umfang und Dauer der Abas Installation.
G interpreter	'G interpreter' liefert je nach aktueller FOP-Sprache eine ..!INTERPRETER-Zeile und kann somit beim Generieren von FOPs aus FOPs heraus verwendet werden. <pre>..!interpreter english declaration</pre>
G kanalnummer	Diese FO-Variable enthält die Nummer desjenigen Prozesses, der für den aktuellen Prozess verantwortlich ist. Verantwortlich in diesem Sinne ist: • bei einem per Systemaufruf ausgeführten EDP-Skript die Prozessnummer des Aufrufers Zu beachten ist, dass die Variable nicht in jedem Fall die Parent-Process-ID des Betriebssystems enthält; so zählt z.B. bei ".kommando" der auslösende Prozess nicht als verantwortlich für den ausgelösten Prozess. Ist kein anderer Prozess im hier beschriebenen Sinne verantwortlich, dann enthält 'G kanalnummer' die eigene Prozessnummer. Man findet den verantwortlichen Prozess auch im Infosystem "Sperrsituationsanalyse", dort unter der Bezeichnung "Vaterprozess".
G kettenvers	Diese schreibgeschützte FO-Variable bestimmt den aktuellen Wert des Versionszählers, ohne ihn zu ändern (Versionszähler ändern siehe G versionneu). Die Art der Prozedur ist ZAE1.

G kdatname	Dieses schreibgeschützte Array enthält alle Dateinamen (Kunde, Lieferant, ...) in der aktuellen Bediensprache. Für konfigurierte Zusatztabellen mit noch nicht festgelegtem Namen wird "DateiZusatztabellennummer", also z.B. "Datei19" geliefert. Nicht konfigurierte Zusatztabellen erkennt man daran, dass bei ihnen in G kdatname Leertext geliefert wird. Das Array ist von Index 0 bis 'G maxkdatnam' belegt. Die Namen werden vom FOP s3sys/KOMLIST ausgegeben. Das Array ist nützlich für die sprachunabhängige Programmierung, z.B. für Datenbankkommandos, siehe auch unter Schlüsselwörter ermitteln. <pre>Gruppenname: 'G kdatname0' group nname: 'G loaderDBName0'</pre>
G kkomname	Dieses schreibgeschützte Array enthält die Namen für Tippkommandos (Passwort, Datum,...) in der aktuellen Bediensprache. Es ist von Index 1 bis 'G maxkkomnam' belegt. Die Namen werden vom FOP s3sys/KOMLIST ausgegeben. Das Array ist nützlich für die sprachunabhängige Programmierung, siehe auch unter Schlüsselwörter ermitteln.
G kaktnam	Dieses schreibgeschützte Array enthält die Namen für "Aktionsnamen" der Abas Kommandos, z.B. fertig, hole usw. in der aktuellen Bediensprache. Es ist von Index 1 bis 'G maxkaktnam' belegt. Die Namen werden vom FOP s3sys/KOMLIST ausgegeben. Das Array ist nützlich für die sprachunabhängige Programmierung, z.B. für Datenbankkommandos, siehe auch unter Schlüsselwörter ermitteln.
G laenge	Diese Variable enthält die mit .laenge eingestellte Seitenlänge.
laschl	(Intrastat) Diese pufferbezogene FO-Variable entnimmt der Tabelle des Länderverzeichnisses den Länderschlüssel des Lieferlandes, für Dänemark z.B. das Kürzel DK.
G links	Diese FO-Variable enthält den aktuell gesetzten linken Rand. Siehe hierzu auch Kapitel .links - Linker Rand.
lndart	(Intrastat) Diese pufferbezogene FO-Variable nennt den Wirtschaftsraum ("Landart"), in dem der Kunde, Lieferant und Mitarbeiter ansässig ist.
G loginname	Diese FO-Variable enthält den Systemloginnamen, der für den aktuellen Benutzer im Passwortdatensatz des Firmenstamms eingetragen ist.
G maxkdatnam	Maximal zulässiger Index für das Array 'G kdatname'. Diese FO-Variable ist schreibgeschützt.
G maxkaktnam	Maximal zulässiger Index für das Array 'G kaktnam'. Diese FO-Variable ist schreibgeschützt.
G maxkkomnam	Maximal zulässiger Index für das Array 'G kkomname'. Diese FO-Variable ist schreibgeschützt.
G mandant	Diese Variable enthält den Namen des Mandanten laut mandantdir.env.

G mbe1	Diese Variable enthält die Anzahl der belegten Stunden einer Maschinengruppe oder Abteilung (Summe aller Maschinengruppen der Abteilung) im Zeitraum "G adatum" bis "G edatum" nach .hole mbelegung. Die Belegung einer Abteilung ergibt sich als Summe der Belegungen aller Maschinengruppen der Abteilung. Beispiel: .hole MBELEGUNG und 'G mbe1' <pre> .. load work center or department .select capacity "102" .continue END ? G success = false .. read data table of loaden capacity for the desired period .formula G startDate = "." .formula G dueDate = "+30" .select machloading display allocation for the next 14 days and complete .formula G startDate = "+1" .formula G dueDate = G startDate + 6 allocation for the next 7 days: 'G machineLoadHrs' .formula G startDate = G dueDate + 1 .formula G dueDate = G startDate + 6 allocation for the following 7 days: 'G machineLoadHrs' .formula G startDate = "+1" complete for the next 14 days: 'G machineLoadHrs' !END .end </pre>
G mehr	Diese Variable vom Typ BOOL gibt den Erfolgsstatus bei bestimmten FO-Kommandos aus.
G status	Diese Variable vom Typ INT kann bei bestimmten Kommandos genauere Angaben zum Erfolgsstatus liefern als G mehr. Die spezifische Bedeutung wird in der Beschreibung des jeweiligen Kommandos festgelegt. Ist keine festgelegt, setzt das Kommando G status auf 0.
G menue	Diese Variable enthält die Nummer des gewählten Menüpunktes bei .menue.
G mentxt	Diese Variable enthält den zum gewählten Menüpunkt definierten Text bei .menue.
G mitlinks	Diese FO-Variable gibt an, ob der integrierte Browser verwendet wird. Die Variable ist nur in der GUI gesetzt.

G|neuespassw

Über die Zuweisung eines Passwort-Verweises an die FO-Variable **G|neuespassw** können Sie festlegen, dass das nächste Einmalpasswort, das mit **.system ... password** verwendet oder mit **'G|einmalpw'** ausgegeben wird, das verwiesene Passwort sein soll.

Damit nachvollziehbar bleibt, wer Kommandos mit den Rechten eines fremden Passworts ausgeführt hat, setzt sich das Bearbeiterzeichen des Kommandos aus dem Bearbeiterzeichen des fremd nutzbaren Passworts und dem Bearbeiterzeichen des FOP-Ausführenden zusammen. Getrennt werden die Bearbeiterzeichen durch den Slash (/), z.B. JP/sy.



Die Zuweisung wirkt nur, wenn in den Passwortdefinitionen die Fremdvergabe zugelassen ist, d.h., das Feld **fremdok** auf **G|true** und das Feld **inaktiv** auf **G|false** gesetzt ist. Andernfalls wird eine Fehlermeldung ausgegeben.

Beispiel:

Ein EDP-Export soll mit den Rechten eines fremden Benutzers ausgeführt werden.

```
..

```

Besser noch mit verdeckter Passwortübergabe:

```
..

```

G neueswebpassw	Einmalpasswörter können außer für andere Abas Benutzer auch für fremde Webbenutzer vergeben werden. Dazu ist der Betreffende vor der Erzeugung des Einmalpassworts in die FO-Variable 'G neueswebpassw' einzutragen.  Die Zuweisung wirkt nur, wenn in den Passwortdefinitionen die Fremdvergabe zugelassen ist, d.h., das Feld fremdok auf G true und das Feld inaktiv auf G false gesetzt ist. Andernfalls wird eine Fehlermeldung ausgegeben. Bei der Anmeldung mit dem Einmalpasswort gelten dieselben Regeln wie bei der normalen Anmeldung des Webbenutzers (derzeit nur mit EDP möglich): • Die FO-Variable webuser enthält einen Verweis auf den Webbenutzer-Datensatz. • Die FO-Variable passw enthält einen Verweis auf den Passwortdatensatz, der dem Webbenutzer direkt oder indirekt über eine Rolle zugeordnet ist. Das Bearbeiterzeichen 'G bzeichen' setzt sich dann zusammen aus dem Zeichen aus dem Passwortdatensatz und der Identnummer des Webbenutzer-Datensatzes, z.B. "Mei/123".
ofob	Diese pufferbezogene FO-Variable enthält offene Ein- und Verkaufsobjekte.
ort	Die pufferbezogene FO-Variable ort enthält die Postleitzahl, das Landeskennzeichen und den Sitz von Kunden und Lieferanten aus der Rechnungsanschrift sowie die entsprechenden Adressdaten aus Lager, Lagergruppe und Serviceprodukt. Wenn die Felder "Rechnungsanschrift" und "Postleitzahl" bei Kunden oder Lieferanten leer bleiben, werden die Daten der Variablen 'ort2' entnommen.
ort2	Die pufferbezogene FO-Variable ort2 enthält die Postleitzahl, Landeskennzeichen und den Sitz von Kunden und Lieferanten der Versandanschrift. Wenn die Felder "Versandanschrift" und "Postleitzahl" leer bleiben, werden die Daten der Variablen 'ort' entnommen.
G passw	Die Variable stellt einen Verweis auf den Passwortdatensatz des angemeldeten Benutzers her. Durch Verknüpfung sind somit alle Felder aus dem Passwortdatensatz und aus dem Mitarbeiterstamm erreichbar. Beispiel: Bezeichnung des Passwortdatensatzes 'G currUserPwd^descr' Name des zugeordneten Mitarbeiters 'G currUserPwd^employeePwdRef^descr'

G panummer	Die Variable enthält eine für ein Benutzerpasswort eindeutige Nummer. Sie eignet sich, um für einen Benutzer eindeutige Namen zu definieren.
pfterm	Diese pufferbezogene FO-Variable enthält den Zeitpunkt des frühesten Endtermins.
ppreis	Diese pufferbezogene FO-Variable enthält den aktuellen Planpreis.
G pi	Diese FO-Variable enthält die Kreiszahl PI (3.141592654) zur Verwendung mit mathematische Funktionen .
G pid	Diese FO-Variable enthält die Nummer des Datenbankprozesses. Sie ist für die Dauer der Sitzung für jeden Anwendungsserver eindeutig und fest und gilt hinsichtlich aller Betriebssystemprozesse dieses Servers. Kommt der Prozess zum Ende, wird sie irgendwann von einem anderen Prozess erneut verwendet. Die Variable ist geeignet, um für den Anwendungsserver eindeutige Namen zu definieren. Siehe hierzu auch: Eindeutiger Name auf einem Rechner .
plsatz	Diese pufferbezogene FO-Variable enthält den aktuellen Planstundensatz.
G progvers	Mit der FO-Variablen 'G progvers' lässt sich (wie auch mit dem Kommando *service) die Version von Programmen abfragen.
pugruppe	Die pufferbezogene FO-Variable enthält die Gruppe des Puffers. Sie ist in der Regel identisch mit 'gruppe'. Nur bei selektierbaren Hülsengruppen kann die Gruppennummer abweichen. Beispiel: Für Gruppe 21, Angebote im Verkauf, liefert "pugruppe" den Wert 21 und "gruppe" den Wert 1.
G rechts	Diese FO-Variable enthält den aktuell gesetzten rechten Rand. Siehe hierzu auch Kapitel .rechts - Rechter Rand .
G seite	Diese Variable gibt die Seitennummer der aktuellen Seite aus.
G sekundenz	Die Variable enthält die aktuelle Uhrzeit in Sekunden im UTC-14-Format, das Sortierung erlaubt. Für eindeutige Texte sollte die Prozedur versionneu verwendet werden; sie enthält diesen Zeitstempel plus eine eindeutige Nummer. Siehe die Variablen G kettenvers und G versionneu . Die Art der Prozedur ist "Text". Diese Variable ist schreibgeschützt. Zur Ermittlung des aktuellen Systemzeitpunkts verwenden Sie die Variable G zeitpunkt .
selzeile	Die pufferbezogene FO-Variable enthält die Trefferzeile der letzten Selektion des angegebenen Puffer. Sie eignet sich bei Zeilenselektionen für den Zugriff auf die passende Zeile mit .hole zeile 'selzeile' .
G std	Diese FO-Variable enthält die Stundenzahl für 'G incst'.
G stddez	Diese FO-Variable enthält den Wert des Standard-Dezimaltrennzeichens im Konfigurationsdatensatz, das für die Arten R und M genutzt wird.
sperrmodus	Sperre des Puffers: Die pufferbezogene FO-Variable beantwortet die Frage, welche Sperre im Puffer liegt. Ihre Werte entsprechen den Sperroptionen. Die Variable zeigt den Sperrmodus bei jedem belegten Puffer an. Näheres siehe unter Sperroptionen bei .hole, .lade und .belegen .

G sperraufrufe rnr	Diese Variable wird gesetzt, wenn bei einem .hole oder .belegen eine Sperre angefordert wurde. Konnte sie erlangt werden, enthält die Variable den Wert 0. Andernfalls enthält sie die Aufrufernummer des "Sperrgegners", also desjenigen, der die Sperre hält. Beispiel: Ein Aufrufer ändert gerade den Datensatz für den Kunden 1. Ein anderer Aufrufer setzt folgenden Befehl ab: <pre>.select nowait writelock Customer "1"</pre> Dann haben die für ihn relevanten Variablen folgende Werte: <pre>G success: 'G success' G locked: 'G locked' G callerNoOfLocker: 'G callerNoOfLocker' G lockDescription: 'G lockDescription'</pre>
G sperrtext	Diese Variable wird gesetzt, wenn bei einem .hole oder .belegen eine Sperre angefordert wurde. Konnte sie erlangt werden, enthält die Variable den leeren Text. Andernfalls enthält sie die Angaben, wie sie aus der Datensatz-belegt-Meldung bekannt sind. Siehe auch G sperraufruferrnr Beispiel: Aufrufer 47 ändert den Kunden 1. <pre>.select nowait writelock Customer "1"</pre> Der Datensatz ist in einem Editor gesperrt, dann hat die Variablen folgende Werte: <pre>G success: 'G success' G locked: 'G locked' G lockDescription: 'G lockDescription'</pre>
G standort	Diese FO-Variable enthält den Standorteintrag des Terminals aus devices.txt.
G strich1	Einfache Unterstreichung für das Ergebnis von .text.
G strich2	Doppelte Unterstreichung für das Ergebnis von .text.

G true	Diese FO-Variable enthält den Wert von G true in der jeweiligen Bediensprache. Sie ist vom Typ "Bool".
G txname	Diese Variable enthält das ausgeführte Hauptprogramm (ohne Sprachverzeichnis). Zum Ermitteln der aktuell aktiven Unterprogramme steht das Array G stackframe zur Verfügung.
G tsdtrenn	Diese FO-Variable enthält den Wert des Feldes "Tausendertrennzeichen" der entspr. Sprache aus dem Konfigurationsdatensatz. Sind die Felder leer, ist G tsdtrenn mit einem Punkt belegt. Initial enthält die Variable die Einstellung des Konfigurationsdatensatz. Sie kann durch .setze tsdtrenn verändert werden. Das gewünschte Zeichen kann auch per Variable angegeben werden. Neben .setze tsdtrenn ist also auch .setze tsdtrenn 'xtrenner' möglich.

G|tsdtrennchar

Wichtig bei der Ausgabe auf Excel. Diese FO-Variable enthält den jeweils aktuell gesetzten Wert für das Tausendertrennzeichen. Ist die Variable ohne Wert, wirkt sie sich auf die Arten "RDTx" und "MDTx" aus.



Der Wert 0 steht für das Standard-Tausendertrennzeichen aus dem Konfigurationsdatensatz. Siehe auch:

link:[https://extranet.abas.de/sub_de/documentation/de/online-help-page/online-help.html?target=/sub_de/help/hd/html/2.6.17.11.html&lang=de#german-oh\[{foc-SETZE} dezpunkt`\]](https://extranet.abas.de/sub_de/documentation/de/online-help-page/online-help.html?target=/sub_de/help/hd/html/2.6.17.11.html&lang=de#german-oh[{foc-SETZE} dezpunkt`]) und
link:[https://extranet.abas.de/sub_de/documentation/de/online-help-page/online-help.html?target=/sub_de/help/hd/html/2.6.17.11.html&lang=de#german-oh\[{foc-SETZE} tsdtrenn`\]](https://extranet.abas.de/sub_de/documentation/de/online-help-page/online-help.html?target=/sub_de/help/hd/html/2.6.17.11.html&lang=de#german-oh[{foc-SETZE} tsdtrenn`]).

Wird das Tausendertrennzeichen mit dem Befehl **.setze tsdtrenn** (ohne weiteres Argument) zurückgesetzt, so ist damit das Standardverhalten eingestellt. Dies ist in drei Fällen sinnvoll:

- beim Start der FO oder nach:

```
.set thoussep
.set decpoint
```

Alle Zahlen der Arten "Real" und "Rx" oder "Mx" werden mit dem Dezimaltrennzeichen Punkt und ohne Tausendertrennzeichen ausgegeben. Alle Zahlen der Arten "RDTx" und "MDTx" werden mit **'G|dezpunkt'** und **'G|tsdtrenn'** ausgegeben.

- bei der Ausgabe mit den Standardtrennzeichen:

```
.set thoussep 'G|thsdSep'
.set decpoint 'G|decPt'
```

Alle Real-Zahlen werden unabhängig von ihrer Art mit dem Dezimaltrennzeichen **'G|dezpunkt'** und dem Tausendertrennzeichen **'G|tsdtrenn'** ausgegeben. Auch feste Werte können eingesetzt werden. Dabei ist zu beachten, dass die Variable **'Tausendertrennzeichen'** leer sein darf, die Variable **'Dezimaltrennzeichen'** nicht. Ist sie leer, wird der Punkt verwendet.

- bei der Ausgabe in eine Datei und bei der Datenübergabe an Fremdsysteme:

```
.set thoussep ""
.set decpoint
```

Die Variable für das Tausendertrennzeichen ist hier immer leer. Das

tzeilen	Diese pufferbezogene FO-Variable enthält die Anzahl der Tabellenzeilen. Benutzen Sie diese Variable ausschließlich mit Bezugsobjekt und nie für Puffer D. Wenn kein Bezugsobjekt angegeben wurde, wird bei der Suche das Objekt in D ausgelassen, da Datensätze in den D-Puffer aus Effizienzgründen ohne Tabelle eingelesen werden.  Der Wert von 'D tzeilen' ist deshalb immer 0. Die tatsächliche Zeilenzahl des Objektes in D kann man aber über das it-Kopffeld 'D ozeilen' ermitteln. Wir behalten uns eine Änderung dieses "traditionellen" Verhaltens vor. Wenn 'tzeilen' immer mit Pufferangabe und nie für Puffer D verwendet wird, ist diese Änderung kompatibel.
G ustid	Diese Variable enthält die Umsatzsteueridentnummer aus [Betriebsdaten zeigen]. Somit können in FOPs einige .hole-Aufrufe für den Betriebsdatensatz ersatzlos gestrichen werden!
G ursache	Diese FO-Variable wird gesetzt, nachdem .merke bindung/.hole bindung aufgerufen wurden. Nähere Informationen finden Sie im Kapitel Kommandos für Verbrauchsfolgeverfahren, oder im Kapitel Kommandos für die Disposition.
G versionneu	Die Variable enthält bei jedem Aufruf einen neuen Wert des Versionszählers und verändert ihn. Sie lässt sich zur Bildung eindeutiger Dateinamen verwenden. Der gelieferte Wert ist, sofern Text, sortierfähig (vgl. die Variable 'G kettenvers'). Die Art der Prozedur ist ZAE1.
G varmaske	Dieses schreibgeschützte Array enthält die Referenz des Variablentabellensatzes zu einer gegebenen Maskennummer. 'varmaske.evtmaske^vdntxt' liefert "Kunde", falls G evtmaske 0 enthält.  Wenn die Nummer einer nicht existierenden Maske angegeben wird, wird die Nullreferenz (also "(0,0,0)") geliefert.
G vt	Array(1..4). Dieses Array enthält den jeweiligen Vertikaltabulator. Nähere Informationen finden Sie im Kapitel .vtab - Vertikaltabulatoren.
G webuser	Die Variable stellt einen Verweis auf den Webbenutzer-Datensatz des angemeldeten Benutzers her, sofern sich dieser über ein Weblogin angemeldet hat (derzeit ist das nur an der EDP-Schnittstelle möglich). Durch Verknüpfung sind damit alle Felder aus dem Webbenutzer-Datensatz sowie alle mit ihnen verbundenen Felder im zugehörigen Passwortdatensatz und im Mitarbeiterstamm erreichbar (sofern die Verweiskette vollständig ist). webuser^name: Name des Webbenutzers webuser^passw^name: Bezeichnung des Passwortdatensatzes webuser^passw^passmitarb^name: Name des zugeordneten Mitarbeiters
G wartung	Diese FO-Variable gibt an, ob der Wartungsmodus eingeschaltet ist.

G zeile	Diese FO-Variable enthält die Zeilennummer der aktuellen Ausgabezeile.
selzeilenweise	Die pufferbezogene FO-Variable enthält den Inhalt des Objektauswahlfeldes "Zeilenweise" der letzten Selektion des angegebenen Puffers.
G zeit	Diese FO-Variable enthält die aktuelle Uhrzeit.
G zeitpunkt	Diese FO-Variable enthält das aktuelle Datum und Uhrzeit.

FO-Konstante für Typenbezeichnungen

Folgende FO-Variablen haben immer einen festen Wert, der nicht veränderlich ist. Durch die Verwendung der Konstanten werden die konkreten Werte verborgen und die FO-Programme lesbarer.

G abko	Typenbezeichnung des Kopfes von Auftrag bzw. Bestellung
G abspos	Typenbezeichnung der Absatzposition
G anko	Typenbezeichnung des Angebotskopfes
G arpos	Typenbezeichnung der Artikelposition oder einer Zusatzposition mit dem Typ AU/BE, BV
G chko	Typenbezeichnung des Chancenkopfes
G delpos	Typenbezeichnung der DEL-Zuschlagsposition
G espos	Typenbezeichnung einer Zusatzposition des Typs Endsumme oder Nettosumme
G fevor	Typenbezeichnung des Fertigungsvorschlags
G kvko	Typenbezeichnung des Kostenvoranschlagkopfes
G liko	Typenbezeichnung des Lieferscheinkopfes
G mipos	Typenbezeichnung des Mindestbestellwerts
G prpos	Typenbezeichnung der Prozentposition
G rako	Typenbezeichnung des Rahmenauftragskopfes
G reko	Typenbezeichnung des Rechnungskopfes
G reserv	Typenbezeichnung der Reservierung
G rollwr	Typenbezeichnung der rollierenden Wertereihe
G sbko	Typenbezeichnung des Serviceauftragskopfes
G seitpos	Typenbezeichnung der Seitenposition
G sipos	Typenbezeichnung der Steuerpositionen (inklusive)
G snko	Typenbezeichnung des Serviceangebotkopfes
G spko	Typenbezeichnung des Speditionsauftrages
G srko	Typenbezeichnung des Reparaturauftragskopfes

G stpos	Typenbezeichnung der Steuerposition (zuzüglich)
G supos	Typenbezeichnung der Zusatzposition mit dem Typ Gesamtsumme
G trpos	Typenbezeichnung der Trennposition
G wako	Typenbezeichnung des Webauftragskopfes
G zako	Typenbezeichnung der Zahlungserinnerung-/träger
G zspos	Typenbezeichnung der Zwischensummenposition
G zupos	Typenbezeichnung der Zusatzposition mit dem Typ Neutrale Positionen und Leer

FO-Variablen für Stack-Informationen

G stack	Die Variable liefert den gesamten Stack (alle Unterprogramme, jeweils mit Zeilennummer). Näheres siehe unter Stack-Informationen für FOPs .
G linenumber	Array(1..framecount) : Die Variable liefert die Zeilennummer des auf dem Stack liegenden FOPs (0 wenn unbekannt). Näheres siehe unter Stack-Informationen für FOPs .
G stackframe	Array(1..framecount) : Die Variable liefert den Namen des auf dem Stack liegenden FOPs bzw. JFOPs. Näheres siehe unter Stack-Informationen für FOPs .
G framecount	Die Variable liefert die Anzahl der auf dem Stack liegenden FOPs einschließlich JFOPs. Näheres siehe unter Stack-Informationen für FOPs .

FO-Variablen für Aufzähltypen

G enumdescr	Die Variable liefert den Titel des gesetzten Aufzähltyps. Näheres Sie unter .setze aufzaehltyp - Aufzähltyp als aktuellen setzen .
G enumnumpos	Array(1..enumnumpos) : Die Variable enthält den Namen der Position des gesetzten Aufzählungstyps. Nähere Informationen finden Sie unter .setze aufzaehltyp - Aufzähltyp als aktuellen setzen .
G enumpostext	Die Variable enthält die Anzahl der Positionen des gesetzten Aufzählungstyps. Näheres siehe unter .setze aufzaehltyp - Aufzähltyp als aktuellen setzen .
G enumposcode	Array(1..enumnumpos) : Die Variable enthält den Code der Position des gesetzten Aufzählungstyps. Näheres siehe unter .setze aufzaehltyp - Aufzähltyp als aktuellen setzen .
G enummax	Die Variable enthält die Anzahl der definierten Aufzählungstypen. Näheres siehe unter .setze aufzaehltyp - Aufzähltyp als aktuellen setzen .

FO-Variablen für Mehrsprachigkeit

G espri	Diese FO-Variable enthält das ISO-Sprachkürzel der Excel-Sprache. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
---------	---

G bspri	Diese FO-Variable enthält das ISO-Sprachkürzel der Bediensprache. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G kspri	Diese FO-Variable enthält das ISO-Sprachkürzel der Korrespondenzsprache und kann nur mit G gekennzeichnet werden, obwohl sie sich immer auf einen Datensatz in einem der Puffer A, D, H, M bezieht. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G lspri	Diese FO-Variable enthält das ISO-Sprachkürzel der FO-Befehlssprache der aktuellen Programmdatei (im JFOP leer). Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G ispri	Diese FO-Variable enthält das ISO-Sprachkürzel der Installationssprache. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G kspr	Kürzel der Korrespondenzsprache. Diese FO-Variable kann nur mit G gekennzeichnet werden, obwohl sie sich immer auf einen Datensatz in einem der Puffer A, D, H, M bezieht. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G ksi	Korrespondenzsprachenindex Diese FO-Variable kann nur mit G gekennzeichnet werden, obwohl sie sich immer auf einen Datensatz in einem der Puffer A, D, H, M bezieht. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G esprache	Diese FO-Variable enthält das Kürzel der Excel-Sprache. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G bsprache	Diese FO-Variable enthält das Kürzel der Bediensprache. Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G lsprache	Diese FO-Variable enthält das Kürzel der FO-Befehlssprache der aktuellen Programmdatei (im JFOP leer). Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G isprache	Diese FO-Variable enthält das Kürzel der Installationssprache (derzeit immer "D"). Näheres siehe FO-Variablen bei Mehrsprachigkeit .
G mehrsprachig	Diese FO-Variable zeigt an, ob in der Konfiguration mindestens eine weitere Bediensprache außer Deutsch aktiviert ist. Sie ist vom Typ "Bool". Steht ' G mehrsprachig ' auf true , ist außer Deutsch eine weitere Bediensprache aktiviert. false steht somit für eine rein deutschsprachige Bedienung.

FO-Variablen für Selektions- und Sachmerkmalelisten

G selrohl	Diese FO-Variable enthält den Rohling der letzten Selektion. In ' G selrohl ' befindet sich der einleitende Teil, auch mit der Rohlings-ID.
G selrohlref	Diese FO-Variable enthält die Referenz des Rohlings der letzten Selektion. Gemeint ist die Referenz des Rohlings aus der Variablen ' G selrohl '.
G selkrit	Diese FO-Variable enthält die Kriterien der letzten Selektion. In ' G selkrit ' befinden sich die kompletten Selektionskriterien mit allen gefüllten Variablen und Optionen. Wenn eine Selektion über mehrere Dateien erfolgt, enthält G selkrit die komplette Angabe von Datei und Gruppe mit derjenigen Option, die zu Beginn angegeben wurde, also entweder @datei oder @gruppe . Beispiel: " @datei=2:1 " bzw. " @gruppe=2:1.2 "

G selfelder	Diese FO-Variable enthält die Kriterien der letzten Selektion. In ' G selfelder ' befinden sich die kompletten Selektionskriterien mit allen Variablen ohne Werte sowie die Optionen. Wenn eine Selektion über mehrere Dateien erfolgt, enthält G selfelder die komplette Angabe von Datei und Gruppe mit derjenigen Option, die zu Beginn angegeben wurde, also entweder @datei oder @gruppe .
G selkritfelder	Diese FO-Variable enthält die Kriterien der letzten Selektion. In ' G selkritfelder ' befinden sich die kompletten Selektionskriterien mit allen Variablen und Werten sowie die Optionen. Wenn eine Selektion über mehrere Dateien erfolgt, enthält G selkritfelder die komplette Angabe von Datei und Gruppe mit derjenigen Option, die zu Beginn angegeben wurde, also entweder @datei oder @gruppe .
G seltreffercirca	Diese FO-Variable enthält nach dem Absetzen einer Selektion die geschätzte Größe der Treffermenge im verwendeten Suchschlüssel.
G selschluesse1	Diese FO-Variable (Typ: Text) enthält den Schlüssel der vorangegangenen Selektion. Das ist der, auf den sich G seltreffercirca bezieht.
sml	Diese pufferbezogene FO-Variable enthält den gesamten Inhalt der Sachmerkmaliste, Felder durch Pipe () getrennt. Voraussetzung ist, dass ein Puffer angegeben oder gefunden wurde, der einen Artikel enthält. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .
select	Diese pufferbezogene FO-Variable enthält den gesamten Inhalt einer Selektionszeile, Felder durch Pipe () getrennt, wobei die zuvor mit .setze SEL festgelegte SEL zugrunde gelegt wird. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .
lgzeile	Diese pufferbezogene FO-Variable enthält genau wie die Variablen ' sml ', und ' select ' sinngemäß eine Objektauswahlzeile. ' lgzeile ' funktioniert im Unterschied zu den anderen rohlingsbezogenen Prozeduren mit jeder Rohlingsart. Er bezieht sich auf den Rohling, der mit dem Kommando .setze rohling ausgewählt wurde. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .
G rohueb	Diese FO-Variable enthält die Merkmalsüberschriften der Überschriftszeile durch Pipe () getrennt. Kann bis zu acht Zeilen lang sein. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .
G rohein	Diese FO-Variable enthält die Einheitenzeile, ebenfalls durch Striche getrennt. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .
G rohstro	Diese FO-Variable enthält die obere Strichzeile. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .
G rohstrm	Diese FO-Variable enthält die mittlere Strichzeile. Näheres siehe unter FO-Zugriffe auf Sachmerkmalisten- und Selektionsleistenrohlinge .

G rohstru	Diese FO-Variable enthält die untere Strichzeile. Näheres siehe unter FO-Zugriffe auf Sachmerkmalleisten- und Selektionsleistenrohlinge.
G evtorigid, G avtorigid	Die FO-Variablen G evtorigid und G avtorigid mit der Art VID99 enthalten im EFOP die ID des Vorlagedatensatzes des Maskenpuffers bzw. des Parentpuffers und sind ansonsten (kein EFOP) leer. Damit ist es einfach möglich, beim Kommando Kopieren auf die ID des kopierten Datensatzes zuzugreifen. Dazu gibt es den EDP-Editorstatus ORIGREF mit der ID des Vorlagedatensatzes. Bei nicht aktiven Editoren oder bei Editoren ohne Puffer ist der Wert "(0,0,0)". Das Speichern und Wiederladen verändert den Wert von ORIGREF nicht. In Subeditoren (PARENT != 0) ist ORIGREF stets "(0,0,0)". Bei Infosystemen ist ORIGREF stets "(0,0,0)". Die Klasse de.abas.jfop.base.event.Event enthält jeweils einen Getter für die FO-Variablen: • getOriginalReference() für G evtorigid • getOriginalReferenceOfParent() für G avtorigid.

FO-Variablen für ereignisgesteuerte FOPs

Die folgenden Variablen charakterisieren den Zustand eines Ereignisses. Sie können nur in ereignisgesteuerten FOPs verwendet werden. Siehe unter [Umgebungsinformation eines FOPs](#).

Bei ereignisgesteuerten FOPs sind von den folgenden Variablen nur '**G|evtmaske**', '**G|evtmaskprio**', '**G|evtkommd**', '**G|evtobjekt**' bei jedem Ereignis sinnvoll zu verwenden. Die Variablen, die mit '**G|evtvar**' beginnen, sind nur bei Feld-, Button- oder Funktionstastenergebnissen und bei [Buttonarten BU10 und BU12](#), auch in den anderen Maskenergebnissen gesetzt.

Deutsch	Englisch	Beschreibung																																															
G evtart	G evtType	Diese Variable gibt die Art des Ereignisses an; ihr Inhalt ist nicht sprachabhängig. Mögliche Werte sind:																																															
		Deutsch	Englisch	noevent	NO_EVENT	maskein	SCREEN_ENTER	maskpruef	SCREEN_VALIDATION	maskaus	SCREEN_EXIT	workflow	SCREEN_WORKFLOW	maskende	SCREEN_END	maskabbr	SCREEN_CANCEL	fkttaste	FUNCTION_KEY_AFTER	buttonvor	BUTTON_BEFORE	buttonnach	BUTTON_AFTER	zeileeeinvor	ROW_INSERTION_BEFORE	zeileeeinnach	ROW_INSERTION_AFTER	zeileloevor	ROW_DELETION_BEFORE	zeileloenach	ROW_DELETION_AFTER	zeilemark	ROW_MARKED	zeilebewvor	ROW_MOVED_BEFORE	zeilebewnach	ROW_MOVED_AFTER	zeilewechsel	ROW_CHANGED	feldfuell	FIELD_FILL	feldpruef	FIELD_VALIDATION	feldaus	FIELD_EXIT	trefferpruef	HIT_VALIDATION	druck	PRINT_STARTED
		Deutsch	Englisch																																														
		noevent	NO_EVENT																																														
		maskein	SCREEN_ENTER																																														
		maskpruef	SCREEN_VALIDATION																																														
		maskaus	SCREEN_EXIT																																														
		workflow	SCREEN_WORKFLOW																																														
		maskende	SCREEN_END																																														
		maskabbr	SCREEN_CANCEL																																														
		fkttaste	FUNCTION_KEY_AFTER																																														
		buttonvor	BUTTON_BEFORE																																														
		buttonnach	BUTTON_AFTER																																														
		zeileeeinvor	ROW_INSERTION_BEFORE																																														
		zeileeeinnach	ROW_INSERTION_AFTER																																														
		zeileloevor	ROW_DELETION_BEFORE																																														
		zeileloenach	ROW_DELETION_AFTER																																														
		zeilemark	ROW_MARKED																																														
		zeilebewvor	ROW_MOVED_BEFORE																																														
		zeilebewnach	ROW_MOVED_AFTER																																														
		zeilewechsel	ROW_CHANGED																																														
		feldfuell	FIELD_FILL																																														
		feldpruef	FIELD_VALIDATION																																														
		feldaus	FIELD_EXIT																																														
		trefferpruef	HIT_VALIDATION																																														
		druck	PRINT_STARTED																																														
		Beispiel:																																															

Deutsch	Englisch	Beschreibung
G evtartengl	G evtTypeEnglish	Art des Events, z.B. BUTTON_BEFORE , SCREEN_ENTER . Die Bezeichner sind identisch zu denen in AJO, Klasse de.abas.erp.axi.event.EventType . Der englische Ereignisname wird auch beim Ereignis-Trace gemeldet.
G evtmaske	G evtScrNo	Diese Variable gibt die Maskennummer an. Diese kann auch bei maskenlosen Kommandos gesetzt sein, und zwar auf die Maskennummer derjenigen Maske, in der die betreffenden Objekte editiert werden.
G evtmaskprio	G evtScrPrio	Enthält die Priorität der Maske. Der Wert ist ein Kleinbuchstabe, z.B. 'a' . Wenn ein EFOP durch ein maskenloses Kommando aufgerufen wird, dann ist der Wert gleich "-" . Beispiel: Die Maske 200 existiere in Versionen für die Prioritäten A, C und E. Ein Mitarbeiter mit Priorität D löst in dieser Maske ein Maskenaustritts-FOP aus. Dann gilt: <pre>G evtScrNo = 200 G evtScrPrio = "c"</pre>
G evtkommd	G evtCmd	Diese Variable gibt die Kommando-Aktion (z.B. "zeigen") in Bediensprache an. Verwenden Sie zur sprachunabhängigen Abfrage die Konstanten in EVT_CMD.CONST .
G evtobjekt	G evtObjType	Diese Variable gibt den Namen der Datenbankgruppe des Maskenobjekts in Bediensprache an, bzw. bei Infosystemen das Suchwort.
G evtzeile	G evtRow	Diese Variable gibt die aktuelle Zeile an.
G evtvar	G evtVar	Die Variable enthält bei Umbenennung eines Feldes in der Vartab den neuen Namen, sonst den original Feldnamen.
G evtvarori	G evtVarOrig	Die Variable enthält den original Feldnamen aus der Vartab .  Ein FOP, das 'G evtvarori' verwendet, kann unabhängig von Feldumbenennungen zwischen verschiedenen Abas Installationen ausgetauscht werden.
evtwarengl	evtwarengl	englisch 'evtVarEnglish' . Die Variable enthält den englischen Feldnamen aus der Vartab .
G evttaste	G evtFuncKey	Die Variable enthält den Namen der gedrückten Funktionstaste bzw. bei einigen Ereignissen den Namen der ausgeführten Funktion in Bediensprache.

Deutsch	Englisch	Beschreibung
G evtfeldneu	G evtFieldNew	Diese Variable enthält den gerade vorgenommenen Feldeintrag des Nutzers. Sie ist nur beim Ereignis feldfuell gesetzt. Dieses Ereignis tritt vor der Syntaxprüfung und einer eventuell notwendigen Objektauswahl ein. Die Variable ist änderbar. Der geänderte Wert wird vom Programm als Benutzereingabe interpretiert. Auf diese Weise ist es z.B. möglich, per FOP Abkürzungen durch den ausgeschriebenen Wert zu ersetzen. Die Abkürzungen müssen noch nicht einmal syntaktisch zum Feld passen, da das Ereignis feldfuell vor den Syntaxprüfungen kommt.
G evtvarart	G evtVarType	Art des Feldes.
G evtvarprio	G evtVarPrio	Priorität des Feldes.
G evtvaraend	G evtVarEdit	Änderungspriorität des Feldes.
G evtsubeditor id	G evtSubeditor Id	Nach Klick auf einen Subeditor-Button enthält diese FO-Variable in der Button-Nachbehandlung die ID des im Subeditor verwendeten Datensatzes. Der Wert kann leer sein, z.B. wenn kein Datensatz verwendet wurde. Bei allen anderen Ereignissen ist der Wert ebenfalls leer.
G fensterid	G scrWindowId	Die FO-Variable G fensterid enthält die ID des Maskenfensers. Diese ID wird normalerweise automatisch vergeben; siehe Verändern einer bereits geöffneten Datenmaske. Sie kann jedoch beim Aufruf des Kommandos, das das Maskenfenster geöffnet hat, festgelegt werden. Die Variable ist nur in der GUI gesetzt. Beispiel: Wenn Sie eine Maske aus einem FOP mit einem der folgenden Befehle öffnen, dann enthält die Variable G fensterid den Wert von U scrid; siehe Kommando ausführen und auf dessen Ende warten. <pre>.kommando -parallel -id U scridkommando -warten -id U scridkommando -fenster 'U scrid' ... -neu ... -id ymaskneu ...</pre>

Die folgenden Variablen charakterisieren den Zustand eines Ereignisses der [Vatermaske](#). Sie können nur in ereignisgesteuerten FOPs verwendet werden, wenn die aktuelle Maske eine Submaske ist und z.B. durch einen Button (BU8) geöffnet wurde. Sie entsprechen im übrigen den **evt**-Variablen.

FO-Variable **G|avtart** entsprechend **G|evtart**

FO-Variable **G|avtartengl** entsprechend **G|evtartengl**

FO-Variable **G|avtmaske** entsprechend **G|evtmaske**

FO-Variable **G|avtmaskprio** entsprechend **G|evtmaskprio**

FO-Variable **G|avtkommd** entsprechend **G|evtkommd**

FO-Variable **G|avtobjekt** entsprechend **G|evtobjekt**

FO-Variable **G|avtzeile** entsprechend **G|evtzeile**

FO-Variable **G|avtvar** entsprechend **G|evtvar**

FO-Variable **G|avtvarori** entsprechend **G|evtvarori**

FO-Variable **G|avtvarengl** entsprechend **G|evtwarengl**

FO-Variable **G|avttaste** entsprechend **G|evttaste**

FO-Variable **G|avtfeldneu** entsprechend **G|evtfeldneu**

FO-Variable **G|avtvarart** entsprechend **G|evtvarart**

FO-Variable **G|avtvarprio** entsprechend **G|evtvarprio**

FO-Variable **G|avtvaraend** entsprechend **G|evtvaraend**

FO-Variablen für Kontenbereiche

Die im Folgenden aufgeführten globalen FO-Variablen enthalten die Verkehrszahlen der Kontenbereiche und Personenkontenbereiche.

jasaldo	Diese pufferbezogene FO-Variable enthält den Jahresanfangssaldo.
asaldo	Diese pufferbezogene FO-Variable enthält den Anfangssaldo.
nasaldo	Diese pufferbezogene FO-Variable enthält den Anfangssaldo (vorzeichenverkehrt).
asoll	Diese pufferbezogene FO-Variable enthält das Anfangs-Soll.
ahaben	Diese pufferbezogene FO-Variable enthält das Anfangs-Haben.
ssum	Diese pufferbezogene FO-Variable enthält die Soll-Bewegung im Zeitraum.
hsum	Diese pufferbezogene FO-Variable enthält die Haben-Bewegung im Zeitraum.
esaldo	Diese pufferbezogene FO-Variable enthält den Endsaldo.
nesaldo	Diese pufferbezogene FO-Variable enthält den Endsaldo (vorzeichenverkehrt).
esoll	Diese pufferbezogene FO-Variable enthält das End-Soll.

ehaben

Diese pufferbezogene FO-Variable enthält das End-Haben.

Die FO-Variablen '**asaldo**', '**esaldo**', '**ssum**', '**hsum**' für Personenkonten (Kunden, Lieferanten, Mitarbeiter) werden auch dann gesetzt, wenn der definierte Zeitraum sich mit den Nachbuchungsmonaten 14 und 15 überschneidet. Es müssen in der FO-Programmierung also keine Ausnahmen bei der Bestimmung des Zeitraums für die Personenkonten gemacht werden.

Die Verkehrszahlen-Kennzahlen ('**asaldo**', '**ssum**', '**hsum**', '**esaldo**', '**kbesaldo**', ...) der Konten und Kontenbereiche werden zuerst in der Buchungswährung ermittelt und, falls die Ausgabewährung von der Buchungswährung abweicht, in die Ausgabewährung umgerechnet. Die Umrechnung wird mit dem am Ende des Zeitraums ('**G|edatum**') gültigen Kurs durchgeführt.

- Zur Ermittlung der Beträge eines Kontenbereichs werden die Verkehrszahlen der Konten dieses Bereichs herangezogen. Umfasst ein Kontenbereich nicht nur Konten, sondern auch Kontenbereiche, so wird die Verdichtungsstruktur aufgelöst und auf die Verkehrszahlen der Konten zugegriffen. Dabei werden Summen gebildet. Zum Beispiel ist '**jasaldo**' für einen Kontenbereich gleich der Summe aller Jahresanfangssalden der betreffenden Konten.
- Die Ermittlung der Beträge für einen Kontenbereich erfolgt analog zur Ermittlung mit dem Kommando **.budget**. Siehe dazu auch [Datenbankkommandos](#). Hier muss eine Gruppe von Konten (Nummernkreis z.B. "18100!18300") angegeben werden.
- Das Kommando **.budget** bleibt weiterhin bestehen.
- Die Vorteile gegenüber dem Kommando **.budget** sind:
 - Die Konten können beliebig zu einem Kontenbereich zusammengefasst werden und sind als Datenbankobjekte immer verfügbar.
 - Diese FO-Variablen werden nicht nur für Gruppen von Sachkonten ermittelt, sondern auch für Gruppen von Personenkonten (Kunden, Lieferanten, Mitarbeiter), für gemischte Gruppen von Sach- und Personenkonten sowie für Kostenstellen und Kostenträger.
 - Bevor die Variablen in einem FO-Programm abgefragt werden, müssen der Zeitraum und ein Kontenbereich bestimmt werden.
 - Bei der Ermittlung dieser Variablen werden keine Berechnungsarten und keine Anteile der Kontenbereiche berücksichtigt.



Der Kontenbereich 1000, "Banken", besteht aus den Konten 18100, 18200 und 18300. Die Verkehrszahlen dieser Konten sind im Jahr 2010:

asaldo	ssum	hsum	esaldo	
18100	1000.00	2000.00	3000.00	0.00
18200	0.00	0.00	0.00	0.00
18300	500.00	2000.00	0.00	2500.00
1500.00	4000.00	3000.00	2500.00	

Die Verkehrszahlen aller Konten, die zu einem Kontenbereich gehören, können auf folgende Art und Weise ermittelt werden:

```
.select year 10 1 12
.select accrange "1000"
start balance:'H|openBal' debit:'H|debTotal' credit:'H|credTotal' end balance:'H|closingBal'
```

```
.select year 10 1 12
.budget "18100!18300"
start balance:'H|openBal' debit:'H|debTotal' credit:'H|credTotal' end balance:'H|closingBal'
```

pesoll	Diese pufferbezogene FO-Variable enthält die Plan-End-Soll-Summe der Konten, Kunden, Lieferanten und Mitarbeiter. Diese FO-Variable ist analog der bereits vorhandenen FO-Variable 'esoll' für die Ermittlung der Ist-Daten, d.h.: • Für die Ermittlung der Plandaten werden die Planverkehrszahlen der betroffenen Konten genommen. • Bevor die Variablen abgefragt werden, müssen der Zeitraum (z.B. .hole JAHR 10 1 12) und das Konto bestimmt werden.
pehaben	Diese pufferbezogene FO-Variable enthält die Plan-End-Haben-Summe der Konten, Kunden, Lieferanten und Mitarbeiter. Diese FO-Variable ist analog der bereits vorhandenen FO-Variable FO-Variable 'asoll' für die Ermittlung der Ist-Daten, d.h.: • Für die Ermittlung der Plandaten werden die Planverkehrszahlen der betroffenen Konten genommen. • Bevor die Variablen abgefragt werden, müssen der Zeitraum (z.B. .hole JAHR 10 1 12) und das Konto bestimmt werden.
pesaldo	Diese pufferbezogene FO-Variable enthält den Plan-Endsaldo.
pjasaldo	Diese pufferbezogene FO-Variable enthält Plan-Jahresanfangssaldo.
pssum	Diese pufferbezogene FO-Variable enthält die Plan-Soll-Summe der Buchung in Auszug.
pasaldo	Diese pufferbezogene FO-Variable enthält den Plan-Anfangssaldo.
phsum	Diese pufferbezogene FO-Variable enthält die Plan-Haben-Summe der Buchung in Auszug.
kbjasaldo	Diese pufferbezogene FO-Variable enthält den Jahresanfangssaldo.
kbasaldo	Diese pufferbezogene FO-Variable enthält den Anfangssaldo der Ist-Daten.

kbesaldo

Diese pufferbezogene FO-Variable enthält den Endsaldo der Ist-Daten.

- Zur Ermittlung der Beträge werden wiederum die Verkehrszahlen des Kontenbereichs oder die Verkehrszahlen der an der Verdichtungsstruktur beteiligten Konten herangezogen.
- Bei der Ermittlung dieser Variablen werden aber die Berechnungsarten der beteiligten Kontenbereiche und die im Kopfteil und in der Tabelle genannten Anteile berücksichtigt. Das hat den Vorteil, dass auswertungsrelevante Werte direkt in der FO-Programmierung abgefragt werden können.



Stellt die GuV die Bewegung z.B. in einem Zeitraum dar (Soll-Haben ('**s sumhsum**')), so sind die Verkaufspositionen negativ, weil im Haben gebucht. In der Auswertung sollen diese Verkaufspositionen aber positiv dargestellt werden. Für diesen Zweck können Kontenbereiche mit der Berechnungsart "Soll-Haben" gebildet werden. Falls der Anteil im Kopfteil der Kontenbereichsmasken der Verkaufskonten auf -1.00 festgelegt ist, werden die Werte intern mit -1.00 multipliziert und dadurch positiv.

Die Anteile in der Kontenbereichstabelle sind vor allem bei Kontenbereichen mit der Berechnungsart "Summe" hilfreich. Soll in der GuV die Position "Rohertrag" ausgegeben werden, so wird sie als Summe hinterlegt, z.B. von Umsatz und Materialaufwand (Rohertrag = Umsatz - Materialaufwand). In der Tabelle wird der Umsatz mit einem Anteil von 1.00, der Materialaufwand mit einem Anteil von -1.00 definiert.

- Bevor die Variablen in einem FO-Programm abgefragt werden, müssen der Zeitraum und ein Kontenbereich bestimmt werden.

Beispiel:

Der Kontenbereich "Turnover" (idno/swd) besteht aus mehreren Verkaufskonten. Die Berechnungsart ist "Soll-Haben", der Anteil im Kopfteil -1.00.

```
.select year 10 1 12
.select accrange "Turnover"
debit:'H|debTotal' credit:'H|credTotal' end balance:'H|closingBal'
AR end balance: 'kbesaldo'
```

Für die Ermittlung von **kbesaldo** kann man für Konten, Kunden, Lieferanten, Mitarbeiter, Kontenbereiche, Kostenstellen und Kostenträger durch die Vorbelegung von **reregeln2** Einfluss nehmen. Es kann eine abweichende Berechnungsart im Feld **reregeln2** vorbelegt werden. Ist die abweichende Berechnungsart für Kostenstelle bzw. Kostenträger nicht vorbelegt, dann gilt wie bisher "Endsaldo" als Berechnungsart.

kbpasaldo	Diese pufferbezogene FO-Variable enthält den Plan-Anfangssaldo der Kontenbereiche, Personenkontenbereiche sowie der Personen- und Sachkonten.
kbpesaldo	Diese pufferbezogene FO-Variable enthält den Plan-Endsaldo der Kontenbereiche, Personenkontenbereiche sowie der Personen- und Sachkonten. Die Vorbelegung dieser FO-Variablen ist analog zu den bereits vorhandenen FO-Variablen für die Ermittlung der Ist-Daten: kbasaldo - Anfangssaldo kbesaldo - Endsaldo Das heißt:  • Zur Ermittlung der Plandaten werden die Planverkehrszahlen der im Kontenbereich bzw. in der Verdichtungsstruktur beteiligten Konten als Grundlage genommen • bei der Ermittlung dieser Variablen werden die Berechnungsarten der beteiligten Kontenbereiche sowie die Anteile aus dem Kopfteil und der Tabelle der Kontenbereiche berücksichtigt • bevor die Variablen abgefragt werden, müssen der Zeitraum, z.B. .hole jahr 10 1 12 und das Konto bzw. Kontenbereich bestimmt werden.
feldnr	Pufferbezogene FO-Variable. In Abhängigkeit davon, ob ein BAB-Feld (.hole KOSTEN FELD) oder ein Kostenarten-BAB-Feld (.hole KOART FELD) im Zugriff ist, enthält diese Variable die Identnummer eines Kontos (wenn ein BAB-Feld im Zugriff ist) bzw. einer Kostenart oder Rechenregel (wenn ein Kostenarten-BAB-Feld im Zugriff ist) gefolgt von der Identnummer einer Kostenstelle oder eines Kostenträgers (beim BAB- und beim Kostenarten-BAB-Feld). Bei der ersten Identnummer werden führende Leerzeichen nicht ausgegeben, bei der zweiten werden sie ausgegeben. Beispiel: <pre>.select costs field 'H EDSFieldNo'</pre>
G ajahr	Die FO-Variable G ajahr enthält das Anfangsgeschäftsjahr des aktuellen Systemdatums. Die Variable kann bei .hole jahr , .hole monat oder .hole zeitraum gesetzt werden. Der Wert kann nicht beeinflusst werden.
G abgjahr	Die FO-Variable G abgjahr enthält das letzte abgeschlossene Geschäftsjahr.
G amonat	Die FO-Variable G amonat enthält den Anfangsgeschäftsmonat des aktuellen Systemdatums. Die Variable kann bei .hole jahr , .hole monat oder .hole zeitraum gesetzt werden. Der Wert kann nicht beeinflusst werden.

G adatum	Die FO-Variable G adatum enthält das Anfangsdatum bei .hole jahr , .hole monat oder .hole zeitraum .
-----------------	--

G|bukreis

Diese Variable enthält den aktuell ausgewählten Buchungskreis. Standardmäßig ist die Variable mit dem ersten Buchungskreis vorbelegt, d.h. dem, in dem die Buchungskreiszurordnung "Buchungskreis 1" enthalten ist. Der Buchungskreis und damit die Variable kann verändert werden über das Kommando **.setze buchungskreis**.

Nach Ausführung des Kommandos **.setze buchungskreis 'bukreis^nummer'** kann ein falscher Buchungskreis ausgewertet werden oder das FO-Kommando bricht mit einer Fehlermeldung ab.

Dies geschieht, wenn die Identnummer der Buchungskreisconfiguration als Eingabeparameter für **.setze buchungskreis** genutzt wird. Die Mehrdeutigkeit der Eingangsparameter für dieses FO-Kommando kann zum Fehler führen, wenn die Identnummer der Buchungskreisconfigurationen nicht mit den zugeordneten Buchungskreisen übereinstimmt. Verwenden Sie zum Setzen des Buchungskreises immer die Buchungskreiszurordnungsnummer (**rs**)**bknr**, wenn Sie über die Buchungskreisconfiguration gehen, z.B.:

```
.setze buchungskreis 'M|bukreis^bknr'
.setze buchungskreis 'G|bukreis^bknr'
```

Beispiel:



Buchungskreisconfiguration mit der Identnummer 2, Suchwort FIBU, aber mit der Buchungskreiszurordnung: Buchungskreis 1. Eine Buchungskreisconfiguration mit der Buchungskreiszurordnung Buchungskreis 2 ist nicht vorhanden.

Verhalten bei den einzelnen Kommandos, wenn **G|bukreis** mit der Buchungskreisconfiguration 2 vorbelegt ist:

1. **.setze buchungskreis 'G|bukreis' □ .setze buchungskreis "FIBU"**
2. **.setze buchungskreis 'G|bukreis^such' □ .setze buchungskreis "FIBU"**
3. **.setze buchungskreis 'G|bukreis^bknr' □ .setze buchungskreis 1**
4. **.setze buchungskreis 'G|bukreis^nummer' □ .setze buchungskreis 2 □ FEHLER.**

Im 4. Fall tritt ein Fehler auf, weil hier der Buchungskreis 2 angesprochen werden soll und nicht die Buchungskreisconfiguration 2. Im Gegensatz dazu ist der 3. Fall ohne Fehler, weil hier direkt die Zuordnungsnummer des

G bueuro	Die FO-Variable G bueuro enthält das Umstellungsdatum der Bewegungsdaten in Euro ab Buchungsdatum aus dem Termindatensatz.
G emonat	Die FO-Variable G emonat enthält den Endgeschäftsmont des aktuellen Systemdatums. Die Variable kann bei .hole jahr , .hole monat oder .hole zeitraum gesetzt werden. Der Wert kann nicht beeinflusst werden.
G edatum	Die FO-Variable G edatum enthält das Enddatum bei .hole jahr , .hole monat oder .hole zeitraum .
G ejahr	Die FO-Variable G ejahr enthält das Endgeschäftsjahr des aktuellen Systemdatums. Die Variable kann bei .hole jahr , .hole monat oder .hole zeitraum gesetzt werden. Der Wert kann nicht beeinflusst werden.
G gjahr	Die FO-Variable G gjahr enthält das aktuelle Geschäftsjahr.
G offen	Die FO-Variable G offen gibt an, ob ein Buchungszeitraum noch offen ist.
G gmonat	Die FO-Variable G gmonat enthält den Geschäftsmonat des aktuellen Systemdatums. Der Wert kann nicht beeinflusst werden.
G gquartal	Die FO-Variable G gquartal enthält die Geschäftsperiode des aktuellen Systemdatums. Der Wert kann nicht beeinflusst werden.
G jokenn	Journalkennzeichen. Es kann mit dem FO-Kommando .hole jokennzeichen gesetzt werden und wirkt sich über den Befehl .hole buchung jokennzeichen auf die Selektion von Finanzbuchungen aus.
G tagkorr	Die Addition einer Zeitdauer zu einer Uhrzeit ergibt in FO wieder eine Uhrzeit. Die FO-Variable G tagkorr enthält dann die Anzahl von Tagen, die zwischen der ursprünglichen Uhrzeit und der Uhrzeit des Ergebnisses liegen. Ein Beispiel hierzu finden Sie im Kapitel Datums- und Zeitoperationen .
bush	Die pufferbezogene FO-Variable bush zeigt an, ob die aktuell im Zugriff befindliche Buchungszeile im Soll oder Haben gebucht wurde.
G gjmenue	Diese FO-Variable enthält die Geschäftsjahre für die Generierung einer Menüauswahl über Geschäftsjahre.
G eurokurs	Diese FO-Variable enthält bei aktivierter Euro-Umstellung den Euro-Kurs aus der Kurstabelle.
G auebereuro	Diese FO-Variable enthält bei aktivierter Euro-Umstellung den Beginn der Euro-Übergangsphase.
G euebereuro	Diese FO-Variable enthält bei aktivierter Euro-Umstellung das Ende der Euro-Übergangsphase.
G aktinland	Diese FO-Variable enthält bei aktivierter Euro-Umstellung die aktuelle Inlandswährung. Vor der Umstellung der Bewegungsdaten auf Euro ist dies die alte Inlandswährung. Sie ist vom Typ "Währungsverweis" (PS94:1) mit Suchwortecho. Für Zuweisungen wird das Suchwort getrimmt, also ohne Leerzeichen geliefert.
G ainland	Diese FO-Variable enthält die alte Inlandswährung. Sie ist vom Typ "Währungsverweis" (PS94:1) mit Suchwortecho. Für Zuweisungen wird das Suchwort getrimmt, also ohne Leerzeichen geliefert.

G ninland	Diese FO-Variable enthält die neue Inlandswährung. Sie ist vom Typ "Währungsverweis" (PS94:1) mit Suchwortechno. Für Zuweisungen wird das Suchwort getrimmt, also ohne Leerzeichen geliefert.
G inland	Diese FO-Variable enthält die Inlandswährung. Sie ist vom Typ "Währungsverweis" (PS94:1) mit Suchwortechno. Für Zuweisungen wird das Suchwort getrimmt, also ohne Leerzeichen geliefert.
G waehrung	Die FO-Variable G waehrung enthält die aktuelle Währung. Sie ist vom Typ "Währungsverweis" (PS94:1) mit Suchwortechno. Für Zuweisungen wird das Suchwort getrimmt, also ohne Leerzeichen geliefert. Siehe hierzu .setze waehrung .
G auswaehr	Die FO-Variable gibt die Währung aus. Sie ist vom Typ "Währungsverweis" (PS94:1) mit Suchwortechno. Für Zuweisungen wird das Suchwort getrimmt, also ohne Leerzeichen geliefert. Siehe hierzu .setze auswaehr .
G steuro	Diese FO-Variable enthält bei aktivierter Euro-Umstellung das Datum, an dem die Euro-Umstellung für die Stammdaten vorgenommen wurde.
G iwaehrwahl	Diese FO-Variable enthält die gültigen Inlandswährungen.
G waehrwahl	Diese FO-Variable enthält die gültigen Währungen.

FO-Variablen für den aktuellen Wert der FO-Sonderzeichen

Alle FO-Sonderzeichen sind als FO-Variablen bekannt. Ihr Name setzt sich aus der Bedeutung und dem Suffix **char** zusammen. Näheres zu den FO-Sonderzeichen finden Sie auch im Kapitel **.setze - Zeichen mit Sonderbedeutung festlegen**.

G kommandochar	Diese Variable enthält das aktuelle Kommandozeichen.
G kommando2char	Diese Variable enthält das aktuelle ausgabeabhängige Kommandozeichen.
G ersetzungchar	Diese Variable enthält das aktuelle Ersetzungszeichen.
G fragechar	Diese Variable enthält das aktuelle Zeichen für den Beginn einer Bedingung.
G markechar	Diese Variable enthält das aktuelle Zeichen zur Kennzeichnung von Sprungmarken.
G trennerchar	Diese Variable enthält das aktuelle Trennzeichen.

FO-Funktionen

Die FO-Schnittstelle kennt vordefinierte FO-Funktionen, wie sie in anderen Programmiersprachen üblich sind, z.B. zur Stringmanipulation oder für mathematische Operationen. Ferner gibt es Funktionen, die für Abas spezifisch sind, z.B. Katalogfunktionen (Informationen über Variablen aus der Variablen-tabelle).

Für alle Funktionen gilt, dass sie zur eindeutigen Namensunterscheidung gegenüber Variablen über den Pufferbuchstaben **F|** angesprochen werden müssen. So wird also z.B. die Funktion zum Errechnen einer Quadratwurzel mit **F|sqrt(wert)** angesprochen. Einzige Ausnahme: Aus Gründen der Abwärtskompatibilität kann die Funktion **defined(variable)** auch ohne Angabe des Pufferbuchstabens verwendet werden.

Allgemeine Syntax eines Funktionsaufrufs

`'F|Funktionsname(Argument1[, Argument2][, Argument3][, ...]) [Cast-Operator]'`

- Angaben in `[]` sind optional.
- Zwischen dem Funktionsnamen und der öffnenden Klammer darf kein Leerzeichen stehen.
- Groß-/Kleinschreibung wird bei Funktionsnamen nicht unterschieden.

Bei der Ausgabe des Wertes einer Variablen gibt es die Möglichkeit, eine Umwandlungsangabe (Cast-Operator) oder Genauigkeitsangaben anzuhängen, um eine vom Standardverhalten abweichende Ausgabe zu erzielen.

```
'F|expr(G|date + 7):8'
'F|expr(H|balance * U|xfactor):4
'F|defined(U|xvar)(L="0")
```

Siehe hierzu auch [Explizite Typumwandlungen \(Typcast\) von Variablen](#).

Argumente

Funktionen werden mit Argumenten aufgerufen, aus denen sich das [Funktionsergebnis](#) ermittelt. Die Argumente müssen bestimmten Voraussetzungen genügen.

- Es gibt Argumente, die einen Ausdruck darstellen dürfen
- und Argumente, die eine Variable bezeichnen müssen.

Bei Variablen sind die üblichen Angaben zulässig, d.h., es kann ein Pufferbuchstabe angegeben werden und Teile des Namens können über die [Punktersetzung](#) ermittelt werden. Außerdem sind Funktionsaufrufe Variablen gleichgestellt, d.h., überall wo eine Variable erwartet wird, darf auch ein Funktionsaufruf stehen.

Ein Ausdruck entspricht dem, was z.B. bei `.formel` auf der rechten Seite des Gleichheitszeichens stehen kann. Ein Ausdruck kann auch aus einer einfachen Konstante oder einem Funktionsaufruf bestehen. In unten stehender Beschreibung der Funktionen werden folgende Namen für die einzelnen Typen von Ausdrücken verwendet:

expr	Beliebiger Ausdruck, d.h., der Ergebnistyp des Ausdrucks darf beliebig sein.
stringexpr	Stringausdruck, d.h., der Ausdruck muss einen String als Ergebnis liefern.
regexpr	Regulärer Ausdruck (regular expression), d.h., der Ausdruck muss einen String als Ergebnis liefern, der einen zulässigen regulären Ausdruck darstellt.
numexpr	Numerischer Ausdruck, d.h., der Ausdruck muss einen numerischen Wert (Integer oder Real) liefern.
intexpr	Integer-Ausdruck, d.h., der Ausdruck muss einen Integer-Wert (Ganzzahl) liefern.
dateexpr	Datumsausdruck, d.h., der Ausdruck muss einen Datumswert liefern.

verweise xpr	Verweisausdruck, d.h. der Ausdruck muss einen Verweis liefern. Dies kann sowohl ein Wert mit der passenden Verweisart sein als auch ein String, der bei der Eingabe in ein entsprechendes Verweisfeld zulässig wäre. (Siehe hierzu auch Abschnitt Laufzeitverhalten)
boolexpr	Boolescher Ausdruck, d.h. ein Wert der Art B (z.B. G true oder G false) oder ein String, der bei der Eingabe in ein boolesches Feld zulässig wäre.
artexpr	Artausdruck, d.h., der Ausdruck muss einen String als Ergebnis liefern, der eine gültige Artangabe wie in der Variablentabelle darstellt. Für die möglichen Artausdrücke siehe Übersicht der Variablenarten .
unitexpr	Ausdruck, der eine Einheit ergibt. Dies kann sowohl ein Wert mit der passenden Art (TAZ"...") sein als auch ein String, der bei der Eingabe in ein entsprechendes Einheitenfeld zulässig wäre.
variable	Das Argument muss eine Variable bezeichnen. Diese muss einschließlich Pufferbuchstaben (H M ...) angegeben werden. Punktersetzung ist zulässig, z.B. H name . P spr .
lang	Das Argument muss eine Sprachangabe bezeichnen. Dies kann wahlweise ein Sprachkürzel, ein Sprachname in Bediensprache oder ein Ausdruck mit der Art einer Sprachaufzählung (A158 oder ähnlich) sein.

Ein Funktionsaufruf verändert niemals den Wert eines der Argumente. Das einzige Produkt eines Funktionsaufrufs ist sein Ergebniswert.

Funktionsergebnis

Eine Funktion liefert genau ein Ergebnis als Rückgabewert. Dieser hat ebenso wie Funktionsargumente einen bestimmten Datentyp. Da Funktionen überall dort eingesetzt werden können, wo auch die Angabe einer Variablen zulässig ist, kann der Rückgabewert z.B. einer Variable zugewiesen, in einem Ausdruck mitverwendet oder auch einfach ausgegeben werden (innerhalb von Apostrophen).

Katalogfunktionen

Katalogfunktionen liefern Informationen über Variablen aus der Variablentabelle. Um eine Katalogfunktion aufzurufen, muss stets eine Variable mit Pufferbuchstaben angegeben werden. Dies bedeutet, dass diese Funktionen nur auf Variablen von geladenen Puffern angewendet werden können, z.B. nach einem erfolgreichen **.hole**-Kommando.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
varname(variable[, lang])	String	Liefert den Namen einer Variablen in der gewünschten Sprache. Dies kann "D" (deutsch) oder "E" (englisch) sein, Standardwert ist Deutsch.
vardescr(variable[, lang])	String	Liefert die Beschreibung der Variablen aus der Variablentabelle in der gewünschten Sprache (Sprachkürzel). Alle Sprachen sind zulässig, unabhängig davon, ob sie konfiguriert sind oder nicht, Standardwert ist "D" (deutsch).
skip(variable)	Boolean	Liefert G true , wenn die angegebene Variable ein Skipfeld ist, ansonsten G false .
scrprotection(variable)	Ein Wert der Art A126	Liefert den Maskenschreibschutz für eine Variable gemäß Variablentabelle. Die Variable muss für diese Abfrage im FOP zumindest sichtbar sein.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
foprotection(variable)	Ein Wert der Art A126	Liefert den FO-Schreibschutz für eine Variable gemäß der Variablen-tabelle. Die Variable muss für diese Abfrage im FOP zumindest sichtbar sein.
varpriordisp(variable)	Ein Wert der Art A104 (Priorität)	Liefert die Priorität für eine Variable gemäß der Variablen-tabelle, die für das Anzeigen des Wertes mindestens benötigt wird. Die Variable muss für diese Abfrage im FOP zumindest sichtbar sein.
varprioedit(variable)	Ein Wert der Art A104 (Priorität)	Liefert die Priorität für eine Variable gemäß der Variablen-tabelle, die für das Ändern des Wertes mindestens benötigt wird. Die Variable muss für diese Abfrage im FOP zumindest sichtbar sein.
iscustom(variable)	Boolean	Liefert die Information, ob ein Feld ein individuelles Reservefeld oder Aliasfeld ist. Die Variable muss für diese Abfrage im FOP zumindest sichtbar sein.
visible(variable)	Boolean	Liefert die Information, ob ein Feld in der Maske sichtbar ist.
isalias(variable)	Boolean	Liefert die Information, ob ein Feld ein Aliasname (Standard oder individuell) für ein anderes Feld ist. Die Variable muss für diese Abfrage im FOP zumindest sichtbar sein.

Stringfunktionen

Stringfunktionen arbeiten mit Zeichenketten.

Übersicht

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
substr(text stringexpr[, int start][, int length])	String	Die Funktion substr liefert einen Teilstring des Ausdrucks stringexpr. Es wird inklusive der Position start, eine definierte Anzahl von Zeichen length gelesen und zurück gegeben. Der Teilstring wird definiert durch die Position start und die Anzahl der Zeichen (length). Die Parameter start und length sind optional. Ist start nicht angegeben, dann wird der Teilstring beginnend vom Anfang ermittelt. Ist length nicht angegeben, dann wird der längstmögliche Teilstring ermittelt. Der Betrag von start muss zwischen 1 und F length(U stringexpr) liegen. • start = positiv → Startposition wird von links (Stringanfang) ermittelt. • start = negativ → Startposition wird von rechts (Stringende) ermittelt. Der Parameter length definiert die Leserichtung. • length = positiv → Teilstring wird nach rechts ermittelt. • length = negativ → Teilstring wird nach links ermittelt. Beispiele: Lesen Sie im String 1234567890 ab der 4. Pos. von links, 3 Zeichen nach rechts. <pre>'F substr("1234567890", 4, 3)' -> 456</pre> Lesen Sie im String 1234567890 ab der 4. Pos. von rechts, 3 Zeichen nach rechts. <pre>'F substr("1234567890", -4, 3)' -> 789</pre> Lesen Sie im String 1234567890 ab der 4. Pos. von links, 3 Zeichen nach links.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
toupper(stringexpr) tolower(stringexpr)	String	Wandeln einen String komplett in Groß- bzw. Kleinbuchstaben um. Es erfolgt jedoch keine Normierung, z.B. von Umlauten, mit Ausnahme des "ß", da es dieses nicht als Großbuchstaben gibt. Beispiel: <pre>'F toupper("New York")' -> NEW YORK 'F tolower("New York")' -> new york</pre>
trim(stringexpr) ltrim(stringexpr) rtrim(stringexpr)	String	Entfernen Leerzeichen am Anfang bzw. am Ende eines Stringausdrucks. trim am Anfang und am Ende ltrim nur am Anfang (links) rtrim nur am Ende (rechts) Beispiel: <pre>'F expr "[" + F trim(" At the beginning and at the end ") + "]")' [At the beginning and at the end] 'F expr "[" + F ltrim(" At the beginning ") + "]")' [At the beginning] 'F expr "[" + F rtrim(" At the end ") + "]")' [At the end]</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
strindex(stringexpr1, stringexpr2[, start])	Integer	Ermittelt die Position des ersten Vorkommens von stringexpr2 innerhalb von stringexpr1 ab einer bestimmten Startposition. Ist der Parameter start nicht angegeben, wird bei 1 beginnend nach rechts gezählt. Das Ergebnis enthält die Position des ersten Zeichens des Suchstrings stringexpr2 im durchsuchten String stringexpr1. Kommt stringexpr2 nicht in stringexpr1 vor, ist das Ergebnis 0. Ist start negativ, wird die Startposition vom Ende der Zeichenfolge aus gezählt und ab dort nach links gesucht; die Position von stringexpr2 wird vom Anfang der Zeichenfolge gezählt. Beispiel: Prüfen ob Suchstring abc bzw. xyz im Originalstring This String contains abc vorhanden ist <pre>'F strindex("This String contains abc", "abc")' = 21 'F strindex("This String contains abc", "xyz")' = 0</pre> Der Suchstring abc ist 2-mal (Position 3 und 6) im Originalstring ababcabcd enthalten. <pre>'F strindex("ababcabcd", "abc")' = 3 'F strindex("ababcabcd", "abc", 1)' = 3 'F strindex("ababcabcd", "abc", 5)' = 6 'F strindex("ababcabcd", "abc", 7)' = 0</pre> Die Suche nach dem String abc beginnt an der 4. Position von rechts im Originalstring ababcabcd mit der Suchrichtung von rechts nach links. <pre>'F strindex("ababcabcd", "abc", -4)' = 6</pre> Die Suche nach dem String abc beginnt an der 5. Position von rechts im Originalstring ababcabcd mit der Suchrichtung von rechts nach links. <pre>'F strindex("ababcabcd", "abc", -5)' = 3</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
string(stringexpr, intexpr)	String	Erzeugt einen String, der den durch stringexpr angegebenen String so oft enthält, wie in intexpr angegeben. Beispiel: Zeile mit 80 Sternen ausgeben: <pre>'F string(" ", 80)' ***** *****</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
formatstr(formatstring, expr1, expr2, ...)	String	Erzeugt einen String aus einem Format-String und Werten. Der Format-String ist ein String mit Platzhaltern für variable Teile. Ein Platzhalter hat die Form "{<index>}", also den Argumentindex des einzufügenden Arguments, eingeschlossen in geschweifte Klammern. Das erste Argument hinter dem Format-String hat den Index 1, das zweite den Index 2 usw. Jedes Argument kann beliebig oft im Format-String verwendet werden. Es sind maximal 20 Formatargumente möglich. Alle Argumente werden automatisch in Strings umgewandelt, auch wenn sie eine andere Art besitzen, z.B. Zahlen oder Datumsangaben. Erzeugen einer Fehlermeldung mit variablem Inhalt <pre> ..!interpreter english noabbrev .type text xsearch xerrmsg .formula U xsearch = "xyz" .formula U xerrmsg = F formatstr("string {1}{2}{1} is no valid search string", G dblquote, U xsearch) .. 'U xerrmsg' .. .end </pre> Diese Funktion dient dazu, übersetzbare Fehler- und Hinweismeldungen zu erzeugen. Die Verwendung dieser Funktion erleichtert die Übersetzung der Meldungen, da stets ein vollständiger Datensatz zu übersetzen ist und nicht Fragmente davon.  Die Funktion sollte nicht direkt in Zeilen mit einem der FO-Kommandos .fehler, .hinweis, .menue, .box sowie in Ausgabezeilen benutzt werden, da sonst der ganze Funktionsaufruf ins Wörterbuch übernommen wird und nicht nur die Texte. In Zeilen mit den oben genannten Kommandos und in Ausgabezeilen benötigen Sie diese Funktion nicht, da dort Variablen durch normale Ersetzung verwendet werden können.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
<code>regindex(stringexpr, regexpr[, ignorecase][, startindex])</code>	Integer	Sucht ein Muster regexpr im String stringexpr und beginnt dabei ab Position startindex. Fehlt startindex, beginnt die Suche am Anfang. Groß-/Kleinschreibung wird normalerweise beachtet. Soll die Groß-/Kleinschreibung ignoriert werden, ist für das Argument ignorecase der boolesche Wert G true zu übergeben. Die Funktion liefert den Positionsindex des ersten Vorkommens des Musters im durchsuchten String zurück oder 0 (null), wenn das Muster nicht gefunden wurde. Alle Indexangaben werden bei 1 beginnend gezählt. Beispiel: An welcher Position steht eine mindestens dreistellige Zahl? <code>'F regindex("1 12 123 1234", "\d{3,}")' -> 6</code> An welcher Position steht ein kleines x? <code>'F regindex("There is no X in the string", "x")' -> 0</code>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
regmatch(stringexpr, regexpr[, ignorecase])	Boolean	Prüft, ob der String stringexpr das Muster regexpr enthält. Dabei wird stets der vollständige String für den Vergleich herangezogen, d.h., der String abcdef passt nicht auf das Muster cd, obwohl es darin vorkommt. Der String passt aber auf das Muster .cd., wobei .* für beliebig viele Zeichen steht. Groß-/Kleinschreibung wird normalerweise beachtet. Soll die Groß-/Kleinschreibung ignoriert werden, ist für das Argument ignorecase der boolesche Wert G true zu übergeben. Beispiel: Beginnt der Originalstring mit M und endet mit ller? <pre>'F regmatch("Müller", "M.*ller")' = yes 'F regmatch("Müller-Lüdenscheid", "M.*ller")' = no</pre> Endet der Originalstring mit müller oder Müller? <pre>'F regmatch("Lüdenscheid-Müller", ".*[Mm]üller")' = yes</pre> Endet der Originalstring mit müller? <pre>'F regmatch("Dammüller", ".*müller")' = yes 'F regmatch("Müller-Lüdenscheid", ".*müller")' = no</pre> Kommt mindestens eine Ziffer im Originalstring vor? <pre>'F regmatch("This is the number 1", ".*[0-9].*")' = yes .type text xtext .formula U xtext = "This is a text." 'F regmatch(U xtext, ".*\d.*")' = no</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
<pre>regreplace(stringexpr, regexpr, substitute[, ignorecase]) regreplaceall(stringexpr, regexpr, substitute[, ignorecase])</pre>	String	Ersetzen eines Musters in einem String durch einen Ersetzungstext. Dabei ist es zulässig, im Muster sogenannte "capture groups" anzugeben. Das sind Teilausdrücke, die in runde Klammern eingeschlossen werden. Diese Teilausdrücke können im Ersetzungstext mit "\$1", "\$2" usw. verwendet werden, um so Teile der gefundenen Muster in der Ersetzung beizubehalten. Diese Teilausdrücke werden von 1 beginnend durchnummeriert; 0 (null) ist der gesamte Text, der auf das Muster passt. Groß-/Kleinschreibung wird normalerweise beachtet. Soll die Groß-/Kleinschreibung ignoriert werden, ist für das Argument ignorecase der boolesche Wert G true zu übergeben. regreplace ersetzt nur das erste Vorkommen des Musters, regreplaceall ersetzt alle Vorkommen des Musters im String. Beispiel: Ersetzt das erste Leerzeichen durch einen Unterstrich <pre>'F regreplace("a b c d", " ", "_")' = a_b c d</pre> Ersetzt alle Leerzeichen durch je einen Unterstrich <pre>'F regreplaceall("a b c d", " ", "_")' = a_b_c_d</pre> Ersetzt alle zusammenhängenden Leerzeichen durch genau einen Unterstrich <pre>'F regreplaceall("a b c d", " +", "_")' =; a_b_c_d</pre> Entfernt alle Leerzeichen, die unmittelbar vor einem Satzzeichen stehen <pre>'F regreplaceall("a , b : c . d ? e", " +([.,;:!?])", "\$1")' =; a, b: c. d? e</pre> Ersetzt alle Punkte durch einen Unterstrich <pre>.type text xtime .formula U xtime = "12.30"</pre>
		Forterro Deutschland Abas GmbH, Gartenstraße 67, 76135 Karlsruhe, Germany © Forterro Deutschland Abas GmbH Alle Rechte vorbehalten. https://abas-erp.com

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
urlencode(url)	String in URL-Codierung	Ersetzt alle Zeichen, für die das nötig ist, durch eine oder mehrere Sequenzen der Form %XX, wobei XX der Hexadezimalcode des Zeichens im Zeichensatz UTF-8 ist. Das Leerzeichen wird nach + konvertiert. Damit konvertiert F urlencode in das MIME-Format application/x-www-form-urlencoded. Beim Öffnen eines Links in dem Webbrowser, der in die GUI integriert ist, wird das Zeichen + wieder in ein Leerzeichen umgewandelt. Beispiel: FOP mit urlencode(url) <pre> .type text xparam xurl .formula U xparam = F urlencode("<Customer> 1 <edit>") .formula U xurl ="http://portal.abas.de/Z11Intranet/erpstart?kommando=" + U xparam .output new "XX.HTML" <html> <BODY> <P> xyurl=<A> HREF="'U xurl'"> Kunde 1 </P> </BODY> </html> .output new DEVNULL .system "mv XX.HTML win/tmp/" background .browser -file "win/tmp/XX.HTML" </pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung						
<code>F strcmp(string1, string2[, IgnoreCase][, Vergleichslänge])</code>	Einer von 3 möglichen Werten, der sich an der Implementierung ähnlicher Funktionen in anderen Sprachen orientiert: <table><tr><td>-1</td><td>erster String kleiner als zweiter</td></tr><tr><td>1</td><td>erster String größer als zweiter</td></tr><tr><td>0</td><td>beide Strings gleich.</td></tr></table>	-1	erster String kleiner als zweiter	1	erster String größer als zweiter	0	beide Strings gleich.	Die Funktion vergleicht zwei Strings in Unicode-Logik, d.h., dass spezielle Zeichen verschiedener Sprachen, z.B. Umlaute, beim "Basisbuchstaben" einsortiert werden. So ist z.B. Händler größer als Handel, aber kleiner als Hendel. Die Reihenfolge ist also: Handel, Händler, Hendel.  Die in verschiedenen Verzeichnissen, z.B. Telefonbüchern, bekannte Einordnung von "ä" bei "ae" gilt hier nicht. Weitere Parameter: - IgnoreCase (Groß-/Kleinschreibung ignorieren): Bool, Default: false Wird der Wert <code>true</code> übergeben, werden Groß- und Kleinbuchstaben als gleichwertig behandelt und haben keinen Einfluss auf das Vergleichsergebnis. - Maximale Vergleichslänge: Integer, Default: 0 Soll nur der Anfang der beiden Strings verglichen werden, kann hiermit die Anzahl der Zeichen angegeben werden. Wird der Parameter weggelassen oder 0 übergeben, wird auf der vollen Länge verglichen. Beispiel: <pre>F strcmp("Anton", "anton")' = -1 F strcmp("anton", "Anton")' = 1 F strcmp("anton", "Anton", true)' = 0 F strcmp("Händler", "Handel")' = 1 F strcmp("Händler", "Hendel")' = -1</pre>
-1	erster String kleiner als zweiter							
1	erster String größer als zweiter							
0	beide Strings gleich.							

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung																																																
checkdigitvda4902(stringexpr)	Prüfziffer	Der String errechnet aus dem übergebenen String eine Prüfziffer nach dem Modulo-43-Verfahren nach VDA 4902. Dabei werden die numerischen Werte aller übertragenen Zeichen addiert und das Ergebnis durch 43 dividiert. Der verbleibende Rest wird wieder in ein Zeichen umgewandelt und als Prüfziffer zurückgegeben. Die Zuordnung von Zeichen zu numerischen Werten ist in nachfolgender Tabelle dargestellt. <table border="1"> <thead> <tr> <th>Zeichen = Wert</th><th>Zeichen = Wert</th><th>Zeichen = Wert</th></tr> </thead> <tbody> <tr><td>0 = 0</td><td>F = 15</td><td>U = 30</td></tr> <tr><td>1 = 1</td><td>G = 16</td><td>V = 31</td></tr> <tr><td>2 = 2</td><td>H = 17</td><td>W = 32</td></tr> <tr><td>3 = 3</td><td>I = 18</td><td>X = 33</td></tr> <tr><td>4 = 4</td><td>J = 19</td><td>Y = 34</td></tr> <tr><td>5 = 5</td><td>K = 20</td><td>Z = 35</td></tr> <tr><td>6 = 6</td><td>L = 21</td><td>- = 36</td></tr> <tr><td>7 = 7</td><td>M = 22</td><td>. = 37</td></tr> <tr><td>8 = 8</td><td>N = 23</td><td>Sp = 38</td></tr> <tr><td>9 = 9</td><td>O = 24</td><td>\$ = 39</td></tr> <tr><td>A = 10</td><td>P = 25</td><td>/ = 40</td></tr> <tr><td>B = 11</td><td>Q = 26</td><td>+ = 41</td></tr> <tr><td>C = 12</td><td>R = 27</td><td>% = 42</td></tr> <tr><td>D = 13</td><td>S = 28</td><td></td></tr> <tr><td>E = 14</td><td>T = 29</td><td></td></tr> </tbody> </table> Beispiel: Die Zeichenkette 12345ABCDE ergibt die Prüfziffer W. $1+2+3+4+5+10+11+12+13+14 = 75 \rightarrow 75 : 43 = 1 \text{ Rest } 32 \rightarrow 32$ entspricht dem Zeichen W. Enthält der String ein ungültiges Zeichen, z.B. "#", wird ein Leerzeichen als Prüfziffer zurückgegeben.	Zeichen = Wert	Zeichen = Wert	Zeichen = Wert	0 = 0	F = 15	U = 30	1 = 1	G = 16	V = 31	2 = 2	H = 17	W = 32	3 = 3	I = 18	X = 33	4 = 4	J = 19	Y = 34	5 = 5	K = 20	Z = 35	6 = 6	L = 21	- = 36	7 = 7	M = 22	. = 37	8 = 8	N = 23	Sp = 38	9 = 9	O = 24	\$ = 39	A = 10	P = 25	/ = 40	B = 11	Q = 26	+ = 41	C = 12	R = 27	% = 42	D = 13	S = 28		E = 14	T = 29	
Zeichen = Wert	Zeichen = Wert	Zeichen = Wert																																																
0 = 0	F = 15	U = 30																																																
1 = 1	G = 16	V = 31																																																
2 = 2	H = 17	W = 32																																																
3 = 3	I = 18	X = 33																																																
4 = 4	J = 19	Y = 34																																																
5 = 5	K = 20	Z = 35																																																
6 = 6	L = 21	- = 36																																																
7 = 7	M = 22	. = 37																																																
8 = 8	N = 23	Sp = 38																																																
9 = 9	O = 24	\$ = 39																																																
A = 10	P = 25	/ = 40																																																
B = 11	Q = 26	+ = 41																																																
C = 12	R = 27	% = 42																																																
D = 13	S = 28																																																	
E = 14	T = 29																																																	

Reguläre Ausdrücke (regular expressions)

Ein regulärer Ausdruck ist im Wesentlichen ein Suchmuster zum Auffinden bestimmter Textmuster innerhalb eines Strings. Sie werden z.B. bei den UNIX-Kommandos awk und grep oder in Perl sehr häufig verwendet. Seit der Version 1.4 besitzt auch der Java-Standard Klassen und Methoden für die Verwendung von regulären Ausdrücken. Die FO-Funktionen, die mit regulären Ausdrücken arbeiten, verwenden dieselbe Implementierung wie Java, d.h., es stehen Ihnen grundsätzlich alle Möglichkeiten zur Verfügung wie in Java, zudem sind die Funktionen vollständig Unicode-kompatibel.

Durch die Verwendung einer allgemeinen Bibliothek für reguläre Ausdrücke erhält der Backslash (\) eine Sonderbedeutung und ist als Literal doppelt anzugeben: "\\ ". Beispiel:

Beispiel:

Ersetzen aller / in einem Dateinamen durch \

```
.type text xfilename xfilename2
.formula U|xfilename = "C:/de/abas/test/file.txt"
'U|xfilename'
.formula U|xfilename2 = F|regreplacell(U|xfilename, "/", "\\")
'U|xfilename2'
```

Weitergehende Informationen zu regulären Ausdrücken finden Sie z.B. unter

[Stichwort "Regulärer Ausdruck" in Wikipedia](#)

[Dokumentation zur Java-Klasse Pattern](#)

Mathematische Funktionen

Alle mathematischen Funktionen besitzen als erstes Argument einen numerischen Ausdruck, der wahlweise vom Typ Integer oder Real sein darf. Das Ergebnis ist in den meisten Fällen ein Real-Wert.

Winkelangaben im Bogenmaß

Beim Aufruf von Winkelfunktionen sind Winkel stets in Radianten anzugeben (Bogenmaß), nicht in Grad!

360° entsprechen einem Bogenmaß von $2 * \pi$, also ca. 6.28. Die Umrechnung eines Winkels ins Bogenmaß funktioniert wie folgt:

$$\text{Bogenmaß} = \text{Winkelmaß} / 180 * \pi$$

Beispiel:



Die Berechnung des Sinus für 60° muss also so aussehen:

```
.formula U|xsinus = F|sin(60 / 180 * G|PI)
'U|xsinus'
```

Umgekehrt liefern die Arcusfunktionen (Umkehrfunktionen der Winkelfunktionen) natürlich ein Bogenmaß.

Für diese Umrechnungen steht die vordefinierte Textvariable **G|pi** zur Verfügung.

Genauigkeit und Weiterverwendung von Funktionsergebnissen



Solange Real-Ausdrücke nicht ausgegeben werden, haben sie eine maximale interne Genauigkeit (ca. 17 Stellen). Wird ein Real-Ausdruck an eine Variable zugewiesen, wird der Wert auf die Genauigkeit der Variablen gerundet. Vermeiden Sie also unnötige Zuweisungen an Variablen. Folgendes Beispiel soll dies verdeutlichen:

Beispiel:

```
.type R1.9 xr
.formula U|xr = F|sqrt(10)
'U|xr'
3.162277660
square this number
'F|power(U|xr, 2)'
10
'F|power(F|sqrt(10),2)'
10
```

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
abs(numexpr)	Real mit gleicher Vor- und Nachkommastellenzahl wie das Argument	Absolutwert Berechnet den Absolutwert für einen numerischen Ausdruck. Dies ist der Wert ohne Vorzeichen, negative Zahlen werden also zu positiven.
sign(numexpr)	Real mit gleicher Vor- und Nachkommastellenzahl wie das Argument	Vorzeichenwert Ermittelt das Vorzeichen für eine Zahl. Für negative Werte ist dies -1, ansonsten +1. Beispiel: <pre>.type int xnumber .formula U xnumber = -12 value: 'F abs(U xnumber)' sign: 'F sign(U xnumber)'</pre>
exp(numexpr)	Real	Exponentialfunktion exponiert e = 2.718281828 mit dem Exponenten numexpr . Beispiel: <pre>.formula U xnumber = 3 'F exp(U xnumber)'</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
log(numexpr)	Real	Natürlicher Logarithmus Logarithmus zur Basis e = 2.718281828. Beispiel: <pre>.formula U xinumber = 3 'F log(F exp(U xinumber))'</pre>
log10(numexpr)	Real	Zehnerlogarithmus Logarithmus zur Basis 10. Der aufgerundete Zehnerlogarithmus einer Zahl entspricht der Anzahl Vorkommastellen im Dezimalsystem. Beispiel: Ermittlung der Stellenzahl eines Werts: <pre>'F log10(1000)' = 3</pre>
sqrt(numexpr)	Real	Liefert die Quadratwurzel von numexpr. numexpr muss größer oder gleich 0 sein.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
power(base, exponent)	Real	Liefert die Potenz zur Basis base mit dem Exponent exponent, also "base hoch exponent". Basis und Exponent dürfen beliebige Integer- oder Real-Ausdrücke sein (numexpr). Dadurch ist es auch möglich, Wurzeln beliebigen Grades zu ermitteln (dritte Wurzel, vierte Wurzel usw.). Beispiel: Berechnung von zwei hoch zehn: <pre>'F power(2, 10)'</pre> Berechnung der dritten Wurzel von zehn entspricht 10 hoch 1/3: <pre>'F power(10, 1/3)'</pre> base = 0, exponent < 0 Der Aufruf von F power mit base = 0 und exponent < 0 ist nicht zulässig, weil diese Kombination zu einer Division durch 0 führt.
sin(numexpr) cos(numexpr)	Real	Liefert den Sinuswert bzw. Cosinuswert von numexpr (Bogenmaß). numexpr darf einen beliebigen Wert annehmen, das Ergebnis liegt stets zwischen -1 und +1.
asin(numexpr) acos(numexpr)	Real	Liefert den Arcussinus bzw. Arcuscosinus von numexpr (Bogenmaß), also den Wert, dessen Sinus bzw. Cosinus numexpr ergibt. numexpr muss im Bereich zwischen -1 und + 1 liegen, das Ergebnis liegt stets zwischen 0 und 2 * PI.
tan(numexpr)	Real	Liefert den Tangens von numexpr (Bogenmaß). numexpr darf nicht gleich 0 sein.
atan(numexpr)	Real	Liefert den Arcustangens von numexpr , also den Wert, dessen Tangens numexpr ergibt.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
round(numexpr[, precision])	Real	Rundet den Wert von numexpr auf precision Stellen nach dem Komma. Ist die Stelle precision + 1 kleiner als 5, wird abgerundet, ansonsten aufgerundet ("kaufmännische Rundung"). precision muss eine Ganzzahl sein (intexpr) und darf einen Wert von -15 bis 9 annehmen. Negative Werte bewirken Rundung auf der entsprechenden Vorkommastelle. Wird precision weggelassen, gilt der Defaultwert 0, d.h., es wird auf ganze Zahlen gerundet. Beispiel: Rundung auf Standardstellenzahl 0, auf zwei Nachkommastellen, auf zwei Vorkommastellen: <pre>'F round(12345.678)' = 12346 'F round(12345.678, 2)' = 12345.68 'F round(12345.678, -2)' = 12300</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung						
mround(numexpr[, multiple][, mode])	Real	Rundet den Wert von numexpr auf ein Vielfaches ("multiple") von multiple. Das Vielfache darf ein beliebiger positiver Real- oder Integerausdruck sein und kann auch weggelassen werden, dann gilt der Defaultwert 1, d.h., es wird auf ganze Zahlen gerundet. Mit mode steuern Sie den Rundungsmodus. Derzeit gibt es folgende Modi: <table><tr><td>-1</td><td>Betragsmäßiges Abrunden, d.h. hin zur 0. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -1, aus 1.5 wird 1.</td></tr><tr><td>0</td><td>Kaufmännisch runden, d.h. ist die Ziffer hinter der zu rundenden Stelle kleiner als 5, wird abgerundet, sonst auf. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -2, aus 1.5 wird 2, aus -1.4 wird -1, aus 1.4 wird 1.</td></tr><tr><td>1</td><td>Betragsmäßiges Aufrunden, d.h. weg von der 0. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -2, aus 1.5 wird 2.</td></tr></table> Beispiel: Kaufmännische Rundung, Abrundung, Aufrundung auf ein Vielfaches von 0.05: <pre>'F mround(12345.678, 0.05)' = 12345.70 'F mround(12345.678, 0.05, -1)' = 12345.65 'F mround(12345.678, 0.05, 1)' = 12345.70</pre> Kaufmännische Rundung, Abrundung, Aufrundung auf ganze Zwanziger: <pre>'F mround(12345.678, 20)' = 12340 'F mround(12345.678, 20, -1)' = 12340 'F mround(12345.678, 20, 1)' = 12360</pre>	-1	Betragsmäßiges Abrunden, d.h. hin zur 0. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -1, aus 1.5 wird 1.	0	Kaufmännisch runden, d.h. ist die Ziffer hinter der zu rundenden Stelle kleiner als 5, wird abgerundet, sonst auf. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -2, aus 1.5 wird 2, aus -1.4 wird -1, aus 1.4 wird 1.	1	Betragsmäßiges Aufrunden, d.h. weg von der 0. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -2, aus 1.5 wird 2.
-1	Betragsmäßiges Abrunden, d.h. hin zur 0. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -1, aus 1.5 wird 1.							
0	Kaufmännisch runden, d.h. ist die Ziffer hinter der zu rundenden Stelle kleiner als 5, wird abgerundet, sonst auf. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -2, aus 1.5 wird 2, aus -1.4 wird -1, aus 1.4 wird 1.							
1	Betragsmäßiges Aufrunden, d.h. weg von der 0. Bei einem Vielfachen von 1 gilt: aus -1.5 wird -2, aus 1.5 wird 2.							
floor(numexpr)	Real mit 0 Nachkommastellen	Rundet den Wert von numexpr auf den nächstniedrigeren ganzzahligen Wert. Bei positiven Zahlen wird zur Null hin gerundet: aus 1.5 wird 1. Bei negativen Zahlen wird von der Null weg gerundet: aus -1.5 wird -2.						

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
ceiling(numexpr)	Real mit 0 Nachkommastellen	Rundet den Wert von numexpr auf den nächstgrößeren ganzzahligen Wert. Bei positiven Zahlen wird von der Null weg gerundet: aus 1.5 wird 2. Bei negativen Zahlen wird zur Null hin gerundet: aus -1.5 wird -1.
trunc(numexpr)	Real mit 0 Nachkommastellen	Schneidet die Nachkommastellen von numexpr ab. Es wird immer zur Null hin gerundet: aus 1.5 wird 1, aus -1.5 wird -1.

Vergleichsfunktion F|equals

Funktionsname und -argument	Typ des Rückgabewerts	Beschreibung
F equals(arg1, arg2)	Boolean	Die Funktion prüft für beliebige Arten die Gleichheit zweier Argumente (Werte). Das 1. Argument bestimmt die Art, auf deren Basis der Vergleich erfolgt. Das 2. Argument darf ein String sein. Ist das 2. Argument ein String, wird dieser in einen Wert mit der Art des ersten Arguments konvertiert. Der Wert des 2. Arguments muss einer zulässigen Eingabe für die Art entsprechen. Wenn die Ausdrücke gleich sind ist der Rückgabewert der Funktion true, ansonsten false. Integerwerte vergleichen <pre> zahl1 = 102 zahl2 = 101 F equals(U zahl1, U zahl2) -> false </pre> Dezimalwerte vergleichen <pre> zahl1 = 102.123 zahl2 = 102.123 F equals(U zahl1, U zahl2) -> true </pre> Stringvergleich <pre> .var text name1 = "Meier" .var text name2 = "meier" F equals(U name1, U name2) -> false </pre> Aufzählungselemente vergleichen <pre> beschaffungsart1 = "Fremdbeschaffung" F equals(U beschaffungsart1, "Fremdbeschaffung") -> true F equals(U beschaffungsart1, "(ExternalProcurement)") -> true F equals(U beschaffungsart1, "(InhouseProduction)") -> false </pre> Datum vergleichen <pre> datum = "25.07.26" F equals(U datum, ".") F equals(U datum, "20260727") -> false </pre>

Kalenderfunktionen

Parameterbeschreibung

Parameter	Typ	Beschreibung
kalBezugsObj	VP26	Kalenderbezugsobjekt: Ein Kalenderbezugsobjekt ist ein Objekt, für das ein Kalender definiert ist, z.B. Kunde, Lieferant, Maschinengruppe, Abteilung, Mitarbeiter usw.
lgruppe	P39:1	Lagergruppe
datum anfdat enddat	GD, GD19, Text	Datum mit und ohne Uhrzeit, Anfangsdatum, Enddatum
anztage	Integer	Anzahl der Tage
anzstd	Real	Anzahl der Stunden (≥ 0)
rund	Bool	optional (default: false)
kapaz	P8:1.2	Kapazität

Grundsätzliche Parameterkombination für Kalenderbezugsobjekt und Lagergruppe

Mit der Parameterkombination Kalenderbezugsobjekt **kalBezugsObj** - Lagergruppe **lgruppe** wird der verwendete Kalender definiert.

Kalenderbezugsobjekt	Lagergruppe	Kalender
""	""	Standard
kalBezugsObj	""	benutzerdefiniert
""	lgruppe	benutzerdefiniert
kalBezugsObj	lgruppe	benutzerdefiniert



- Wenn für die Parameterkombination **kalBezugsObj**, **lgruppe** ein Kalender definiert wurde, dann wird dieser verwendet.
- Wenn für die Parameterkombination **kalBezugsObj**, **lgruppe** kein Kalender definiert wurde, dann wird der in der Kalenderhierarchie nächstmögliche Kalender verwendet.
- Wenn in der Kalenderhierarchie kein passender Kalender gefunden wird, dann wird der Standardkalender verwendet.
- Wenn die Parameterkombination **kalBezugsObj=""**, **lgruppe=""** ist, dann wird immer der Standardkalender verwendet.

Grundsätzliches zur internen Prüfung der Eingabegrößen

Bei den Kalenderfunktionen gibt es eine interne Prüfung der Eingabegrößen. Diese geht davon aus, dass die Auswertung der Kalender von mehr als 50 Jahren selten sinnvoll ist. In der Regel sind für diesen Zeitraum Feiertage und sonstigen Ausnahmen nicht gepflegt. Außerdem weisen so lange Durchlaufzeiten beim Einplanen von Arbeitsgänge auf eine fehlerhafte Parametrierung hin. Deshalb liefern die Kalenderfunktionen

den Fehler **Zeitbereich ist unplausibel**, wenn sie für einen Zeitraum für mehr als 20000 Tagen aufgerufen werden.

Falls die unplausiblen Daten, die an die Kalenderfunktionen übergeben werden, selbst aus Terminberechnungen stammen, kann dies verschiedene Ursachen haben:

- Fehlerhafte Terminvorgaben
- Ein zu geringes Kapazitätsangebot im geltenden Kalender durch
 - zu fehlerhafte Kapazitätsgrößen (Einzelkapazität) oder
 - zu lange Ausfallzeiten (1 Jahr und 2 Wochen statt 2 Wochen).

Ermittlung des gültigen Kalenders

Die folgenden Funktionen ermitteln den gültigen Kalender zu einem Datum.

Funktionsname und -argumente	Typ des Rückgabewerts
<code>F getwcal(kalBezugsObj, lgruppe, datum)</code>	P137
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	

Ermittlung des Kalenders für alle Kalenderbezugsobjekte und alle Lagergruppen (Standardkalender)



- Der verwendete Kalender wird durch die [Parameterkombination](#) `kalBezugsObj`, `lgruppe` definiert.
- Die Parameterkombination `kalBezugsObj=""`, `lgruppe=""` verwendet den Standardkalender.

Beispiel: Ermitteln des Kalenders für alle Kalenderbezugsobjekte und alle Lagergruppen

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` der Kalender ermittelt, der am 12.05.2026 gültig ist.

```
.type P137 xpcalendar
.formula U|xpcalendar = F|getwcal("", "", "12.05.2026")
valid calendar: 'U|xpcalendar' - 'U|xpcalendar^swd' - 'U|xpcalendar^descr0perLang'
```

```
valid calendar: 1 - KALSTD - Standardkalender
```

Ermittlung des Kalenders für ein Kalenderbezugsobjekt und alle Lagergruppen



- Der verwendete Kalender wird durch die [Parameterkombination](#) `kalBezugsObj`, `lgruppe` definiert.

- Wenn für die Parameterkombination **kalBezugsObj**, **lgruppe** kein passender Kalender vorhanden ist, dann wird der in der Kalenderhierarchie nächstmögliche Kalender verwendet.
- Wenn in der Kalenderhierarchie kein passender Kalender gefunden wird, dann wird der Standardkalender verwendet.

Beispiel: Ermitteln des Kalenders für ein definiertes Kalenderbezugsobjekt und alle Lagergruppen

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj**="K 5", **lgruppe**="" der Kalender ermittelt, der für Kunde 5 - **ohne** Angabe einer Lagergruppe, am 12.05.2026 gültig ist. Für Kunde 5 - ohne Angabe einer Lagergruppe, wurde ein Kalender (Dispokalender) definiert.

```
.type P137 xpcalendar
.formula U|xpcalendar = F|getwcal("K 5", "", "12.05.2026")
valid calendar: 'U|xpcalendar' - 'U|xpcalendar^swd' - 'U|xpcalendar^descr0perLang'
```

valid calendar: 23 - KALK5 - calendar customer 5

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj**="K 1", **lgruppe**="" der Kalender ermittelt, der für Kunde 1 - **ohne** Angabe einer Lagergruppe, am 12.05.2026 gültig ist. Für Kunde 1 - ohne Angabe einer Lagergruppe, wurde kein Kalender (Dispokalender) definiert.

```
.type P137 xpcalendar
.formula U|xpcalendar = F|getwcal("K 1", "", "12.05.2026")
valid calendar: 'U|xpcalendar' - 'U|xpcalendar^swd' - 'U|xpcalendar^descr0perLang'
```

valid calendar: 1 - KALSTD - Standardkalender

Ermittlung des Kalenders für ein Kalenderbezugsobjekt und eine Lagergruppe



- Der verwendete Kalender wird durch die **Parameterkombination** **kalBezugsObj**, **lgruppe** definiert.
- Wenn für die Parameterkombination **kalBezugsObj**, **lgruppe** kein passender Kalender vorhanden ist, dann wird der in der Kalenderhierarchie nächstmögliche Kalender verwendet.
- Wenn in der Kalenderhierarchie kein passender Kalender gefunden wird, dann wird der Standardkalender verwendet.

Beispiel: Ermitteln des Kalenders für ein Kalenderbezugsobjekt und eine Lagergruppe

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj**="K 5", **lgruppe**="BERLIN" der Kalender ermittelt, der für Kunde 5 - Lagergruppe **BERLIN**, am 12.05.2026 gültig ist. Für Kunde 5 - Lagergruppe **BERLIN**, wurde ein Kalender (Dispokalender) definiert.

```
.type P137 xpcalendar
.formula U|xpcalendar = F|getwcal("K 5", "BERLIN", "12.05.2026")
valid calendar: 'U|xpcalendar' - 'U|xpcalendar^swd' - 'U|xpcalendar^descr0perLang'
```

valid calendar: 22 - K5BERLIN - Customer 5, BERLIN

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj**="114", **lgruppe**="KARLSRUHE" der Kalender ermittelt, der für Abteilung/Maschinengruppe 114 - Lagergruppe **KARLSRUHE**, am 16.05.2026 gültig ist. Für Abteilung/Maschinengruppe 114 - Lagergruppe **KARLSRUHE**, wurde ein Kalender (Kapazitätsangebot) definiert.

```
.type P137 xpcalendar
.formula U|xpcalendar = F|getwcal("114", "KARLSRUHE", "12.05.2026")
valid calendar: 'U|xpcalendar' - 'U|xpcalendar^swd' - 'U|xpcalendar^descr0perLang'
```

valid calendar: 24 - MA114_KA - work center 114, KARLSRUHE

Prüfung eines Datums auf Arbeits-, Liefer- oder Anliefertag

Die folgenden Funktionen prüfen, ob ein Datum ein Arbeits-, Liefer- oder Anliefertag ist. Der Rückgabewert ist im Erfolgsfall **true**, ansonsten **false**.

Funktionsname und -argumente	Typ des Rückgabewerts
F isworkday (kalBezugsObj, lgruppe, datum)	bool
F isdelivday (kalBezugsObj, lgruppe, datum)	bool
F isreceiptday (kalBezugsObj, lgruppe, datum)	bool

Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle [Parameterbeschreibung](#).



- Arbeits-, Liefer- bzw. Anliefertage werden in den Kalendern definiert.
- Der verwendete Kalender wird durch die **Parameterkombination** **kalBezugsObj**, **lgruppe** definiert.

- Wenn für die Parameterkombination **kalBezugsObj**, **lgruppe** kein passender Kalender vorhanden ist, dann wird der in der Kalenderhierarchie nächstmögliche Kalender verwendet.
- Wenn in der Kalenderhierarchie kein passender Kalender gefunden wird, dann wird der Standardkalender verwendet.

Beispiel: Prüfen, ob ein Datum im Standardkalender, einem Arbeitstag entspricht

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** geprüft, ob das Datum Sonntag 01.01.2012, im verwendeten Kalender (Standardkalender), ein Arbeitstag ist. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: So 01.01.2012 (Feiertag/kein Arbeitstag) – Ergebnis: nein

```
[source,shell]
```

```
is 1.1.2012 workday? 'F|isworkday("", "", "1.1.2012")'
```

```
is 1.1.2012 workday? no
```

Beispiel: Prüfen, ob ein Datum im Standardkalender, einem Arbeitstag entspricht

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** geprüft, ob das Datum Montag 02.01.2012, im verwendeten Kalender (Standardkalender), ein Arbeitstag ist. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Mo 02.01.2012 (Arbeitstag) – Ergebnis: ja

```
is 2.1.2012 workday? 'F|isworkday("", "", "2.1.2012")'
```

```
is 2.1.2012 workday? yes
```

Beispiel: Prüfen, ob ein Datum im Standardkalender, einem Liefertag entspricht

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` geprüft, ob das Datum Mittwoch 11.05.2012, im verwendeten Kalender (Standardkalender), ein Liefertag ist. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Mi 11.05.2012 (Liefertag) – Ergebnis: ja

```
is 11.05.2012 delivery day? 'F|isdelivday("", "", "11.05.2012")'
```

```
is 11.05.2012 delivery day? yes
```

Beispiel: Prüfen, ob ein Datum im Standardkalender, einem Anliefertag entspricht

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` geprüft, ob das Datum Montag 09.01.2012, im verwendeten Kalender (Standardkalender), ein Anliefertag ist. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Di 09.01.2012 (Anliefertag) – Ergebnis: ja

```
is 09.01.2012 receipt day? 'F|isreceiptday("", "", "09.01.2012")'
```

```
is 09.01.2012 receipt day? yes
```

Beispiel: Prüfen, ob ein Datum in einem benutzerdefiniertem Kalender einem Arbeitstag entspricht

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="M 69002"`, `lgruppe=""` geprüft, ob das Datum Freitag 18.05.2012, im Kalender für Mitarbeiter 69002 - ohne Angabe einer Lagergruppe, ein Arbeitstag ist. Für Mitarbeiter 690025 - ohne Angabe einer Lagergruppe, wurde ein Kalender (Dispokalender) definiert.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo
Zusätzlich ist der 18.05.2012 als arbeitsfreier Tag eingetragen.

```
is 18.05.2012 work day? 'F|isworkday("M 69002", "", "18.05.2012")'
```

is 18.05.2012 work day? no

Addition von Arbeits-, Liefer- oder Anliefertagen zu einem Datum

Die folgenden Funktionen addieren eine Anzahl von Arbeits-, Liefer- oder Anliefertagen zu einem Datum.

Funktionsname und -argumente	Typ des Rückgabewerts
F addworkdays(kalBezugsObj, lgruppe, datum, anztage)	wie Parameter datum
F adddelivdays(kalBezugsObj, lgruppe, datum, anztage)	wie Parameter datum
F addreceiptdays(kalBezugsObj, lgruppe, datum, anztage)	wie Parameter datum
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	



- Der verwendete Kalender wird durch die [Parameterkombination](#) **kalBezugsObj**, **lgruppe** definiert.
- Ist **anztage=0** und der Inhalt des Parameters **datum** kein Arbeits-, Liefer- oder Anliefertag, dann wird das Datum des nächstmöglichen Arbeits-, Liefer- bzw. Anliefertages berechnet.
- Bei Addition und Subtraktion von Tagen darf die Summe der Tage nicht mehr als 20000 betragen.

Berechnen des nächstmöglichen Arbeits-, Liefer- bzw. Anliefertages

Beispiel: Berechnen des nächstmöglichen Arbeitstages nach einem Datum

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** der nächste Arbeitstag nach einem Datum, Samstag 26.05.2012, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – Ergebnis: Di 29.05.2012

```
.type GD xgddate
.formula U|xgddate = F|addworkdays("", "", "26.05.2012", 0)
next work day: 'U|xgddate'
```

next work day: 29.05.2012

Beispiel: Berechnen des nächstmöglichen Liefertages nach einem Datum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` der nächste Liefertag nach einem Datum, Samstag 26.05.2012, berechnet.
Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.
Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Liefertag) – Ergebnis: Mi 30.05.2012

```
.type GD xgddate  
.formula U|xgddate = F|adddelivdays("", "", "26.05.2012", 0)  
next delivery day: 'U|xgddate'
```

next delivery day: 30.05.2012

Beispiel: Berechnen des nächstmöglichen Anliefertages nach einem Datum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` der nächste Anliefertag nach einem Datum, Samstag 26.05.2012, berechnet.
Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.
Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Anliefertag) – Mo 28.05.2012 (Feiertag) – Ergebnis: Mo 04.06.2012

```
.type GD xgddate  
.formula U|xgddate = F|addreceiptdays("", "", "26.05.2012", 0)  
next receipt day: 'U|xgddate'
```

next receipt day: 04.06.2012

Addieren von Arbeits-, Liefer- oder Anliefertagen zu einem Datum

- Der verwendete Kalender wird durch die **Parameterkombination** `kalBezugsObj`, `lgruppe` definiert.

Beispiel: Zu einem Datum des Standardkalenders werden Arbeitstage addiert

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` zu einem Datum, Samstag 26.05.2012, 4 Arbeitstage addiert.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – Berechnungsbeginn: Di 29.05.2012 (nächster Arbeitstag)
Di 29.05.2012 + 4d = Mo 04.06.2012 – Ergebnis: Mo 04.06.2012

```
.type GD xgddate
.formula U|xgddate = F|addworkdays("", "", "26.05.2012", 4)
calculated work day: 'U|xgddate'
```

calculated work day: 04.06.2012

Beispiel: Zu einem Datum des Standardkalenders werden Liefertage addiert

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` zu einem Datum, Samstag 26.05.2012, 4 Liefertage addiert.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Liefertag) – Berechnungsbeginn: Mi 30.05.2012 (nächster Liefertag)
Mi 30.05.2012 + 4d = Mi 27.06.2012 – Ergebnis: Mi 27.06.2012

```
.type GD xgddate
.formula U|xgddate = F|adddelivdays("", "", "26.05.2012", 4)
calculated delivery day: 'U|xgddate'
```

calculated delivery day: 27.06.2012

Beispiel: Zu einem Datum des Standardkalenders werden Anliefertage addiert

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` zu einem Datum, Samstag 26.05.2012, 4 Anliefertage addiert.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Anliefertag) – Mo 28.05.2012 (Feiertag) -- Berechnungsbeginn: Mo 04.06.2012 (nächster Anliefertag)
 Mo 04.06.2012 + 4d = Mo 02.07.2012 – Ergebnis: Mo 02.07.2012

```
.type GD xgddate
.formula U|xgddate = F|addreceiptdays("", "", "26.05.2012", 4)
calculated receipt day: 'U|xgddate'
```

calculated receipt day: 02.07.2012

Beispiel: Zu einem Datum eines benutzerdefinierten Kalenders werden Arbeitstage addiert

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="K 5"**, **lgruppe="BERLIN"** zu einem Datum, Donnerstag 03.05.2012, 4 Arbeitstage addiert. Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde 5 - Lagergruppe **BERLIN**, definiert wurde. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo. Zusätzlich ist die Zeit vom 02.05.12 - 04.05.12 als arbeitsfrei eingetragen.

Berechnungsstart: Do 03.05.2012 (arbeitsfrei) – Berechnungsbeginn: Mo 07.05.2012 (nächster Arbeitstag)
 Mo 07.05.2012 + 4d = Fr 11.05.2012 – Ergebnis: Fr 11.05.2012

```
.type GD xgddate
.formula U|xgddate = F|addworkdays("K 5", "BERLIN", "03.05.2012", 4)
calculated work day: 'U|xgddate'
```

calculated work day: 11.05.2012

Beispiel: Zu einem Datum eines benutzerdefinierten Kalenders werden Liefertage addiert

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="K 5"**, **lgruppe="BERLIN"** zu einem Datum, Donnerstag 03.05.2012, 2 Liefertage addiert. Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde 5 - Lagergruppe **BERLIN**, definiert wurde. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo. Zusätzlich ist die Zeit vom 02.05.12 - 04.05.12 als arbeitsfrei eingetragen.

Berechnungsstart: Do 03.05.2012 (arbeitsfrei) – Berechnungsbeginn: Mi 09.05.2012 (nächster Liefertag)
 Mi 09.05.2012 + 2d = Mi 23.05.2012 – Ergebnis: Mi 23.05.2012

```
.type GD xgddate
.formula U|xgddate = F|adddelivdays("K 5", "BERLIN", "03.05.2012", 2)
calculated delivery day: 'U|xgddate'
```

calculated delivery day: 23.05.2012

Beispiel: Zu einem Datum eines benutzerdefinierten Kalenders werden Anliefertage addiert

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj="K 5"**, **lgruppe="BERLIN"** zu einem Datum, Donnerstag 03.05.2012, 1 Anliefertag addiert. Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde 5 - Lagergruppe **BERLIN**, definiert wurde. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo. Zusätzlich ist die Zeit vom 02.05.12 - 04.05.12 als arbeitsfrei eingetragen.

Berechnungsstart: Do 03.05.2012 (arbeitsfrei) – Berechnungsbeginn: Mo 07.05.2012 (nächster Anliefertag)
 Mo 07.05.2012 + 1d = Mo 14.05.2012 – Ergebnis: Mo 14.05.2012

```
.type GD xgddate
.formula U|xgddate = F|addreceiptdays("", "", "03.05.2012", 1)
calculated receipt day: 'U|xgddate'
```

calculated receipt day: 14.05.2012

Subtraktion von Arbeits-, Liefer- oder Anliefertagen von einem Datum

Die folgenden Funktionen subtrahieren eine Anzahl von Arbeits-, Liefer- oder Anliefertagen von einem Datum.

Funktionsname und -argumente	Typ des Rückgabewerts
F subworkdays(kalBezugsObj, lgruppe, datum, anztage)	wie Parameter datum
F subdelivdays(kalBezugsObj, lgruppe, datum, anztage)	wie Parameter datum

Funktionsname und -argumente	Typ des Rückgabewerts
F subreceiptdays(kalBezugsObj, lgruppe, datum, anztage)	wie Parameter datum
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	



- Der verwendete Kalender wird durch die [Parameterkombination](#) kalBezugsObj, lgruppe definiert.
- Ist anztage=0 und der Inhalt des Parameters datum kein Arbeits-, Liefer- oder Anliefertag, dann wird das Datum des nächstmöglichen Arbeits-, Liefer- bzw. Anliefertages berechnet.
- Bei Addition und Subtraktion von Tagen darf die Summe der Tage nicht mehr als 20000 betragen.

Berechnen des letzten Arbeits-, Liefer- bzw. Anliefertages

Beispiel: Berechnen des letzten Arbeitstages vor einem Datum

In diesem Beispiel wird mithilfe der Parameterkombination kalBezugsObj="", lgruppe="" der letzte Arbeitstag vor einem Datum, Samstag 26.05.2012, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Ergebnis: Fr 25.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subworkdays("", "", "26.05.2012", 0)
last work day: 'U|xgddate'
```

```
last work day: 25.05.2012
```

Beispiel: Berechnen des letzten Liefertages vor einem Datum

In diesem Beispiel wird mithilfe der Parameterkombination kalBezugsObj="", lgruppe="" der letzte Liefertag vor einem Datum, Samstag 26.05.2012, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Liefertag) – Ergebnis: Mi 23.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subdelivdays("", "", "26.05.2012", 0)
last delivery day: 'U|xgddate'
```

```
last delivery day: 23.05.2012
```

Beispiel: Berechnen des letzten Anliefertages vor einem Datum

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** der letzte Anliefertag vor einem Datum, Samstag 26.05.2012, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Anliefertag) – Ergebnis: Mo 21.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subreceiptdays("", "", "26.05.2012", 0)
last receipt day: 'U|xgddate'
```

```
last receipt day: 21.05.2012
```

Subtrahieren von Arbeits-, Liefer- bzw. Anliefertagen



- Der verwendete Kalender wird durch die **Parameterkombination** **kalBezugsObj**, **lgruppe** definiert.

Beispiel: Subtrahieren von Arbeitstagen von einem Datum im Standardkalender

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** von einem Datum, Samstag 26.05.2012, 4 Arbeitstage subtrahiert. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Berechnungsbeginn: Fr 25.05.2012 (letzter Arbeitstag)
Fr 25.05.2012 - 4d = Mo 21.05.2012 – Ergebnis: Mo 21.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subworkdays("", "", "26.05.2012", 4)
calculated work day: 'U|xgddate'
```

calculated work day: 21.05.2012

Beispiel: Subtrahieren von Liefertagen

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** von einem Datum, Samstag 26.05.2012, 2 Liefertage subtrahiert. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Berechnungsbeginn: Mi 23.05.2012 (letzter Liefertag)
 Mi 23.05.2012 - 2d = Mi 09.05.2012 – Ergebnis: Mi 09.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subdelivdays("", "", "26.05.2012", 2)
calculated delivery day: 'U|xgddate'
```

calculated delivery day: 09.05.2012

Beispiel: Subtrahieren von Anliefertagen

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** von einem Datum, Samstag 26.05.2012, 1 Anliefertag subtrahiert. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Berechnungsbeginn: Mo 21.05.2012 (letzter Anliefertag)
 Mo 21.05.2012 - 1d = Mo 14.05.2012 – Ergebnis: Mo 14.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subreceiptdays("", "", "26.05.2012", 1)
calculated receipt day: 'U|xgddate'
```

calculated receipt day: 14.05.2012

Addition von Arbeits-, Liefer- oder Anlieferstunden zu einem Datum mit Uhrzeit

Die folgenden Funktionen addieren eine Anzahl von Stunden zu einem Datum mit Uhrzeit.

Beginn und Ende der Tagesarbeitszeit

z.B. Arbeitszeit: 8 Std

Arbeitstag	Uhrzeit
Beginn	00:00
Ende	08:00

z.B. Arbeitszeit: 7,5 Std

Arbeitstag	Uhrzeit
Beginn	00:00
Ende	07:30

Funktionsname und -argumente	Typ des Rückgabewerts
F addworkhours(kalBezugsObj, lgruppe, datum, anzstd, rund)	GD/GD19
F adddelivhours(kalBezugsObj, lgruppe, datum, anzstd, rund)	GD/GD19
F addreceiphours(kalBezugsObj, lgruppe, datum, anzstd, rund)	GD/GD19
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	



- Bei Addition und Subtraktion von Stunden darf die Summe der Stunden nicht mehr als 480000 betragen.

Berechnen des nächstmöglichen Arbeits-, Liefer- bzw. Anliefertages nach einem Datum mit Uhrzeit



- Im Kalender, der zur Berechnung verwendet wird, muss die Einheit **Arbeitsstunden** definiert sein.
- Der verwendete Kalender wird durch die **Parameterkombination** **kalBezugsObj**, **lgruppe** definiert.

- Ist **anzstd** = 0, wird das Datum mit Uhrzeit zum Beginn des nächsten Arbeits-, Liefer- bzw. Anliefertages berechnet.
- Ist **rund** = **true**, ist die Art des gelieferten Datums GD (Datum ohne Uhrzeit).
- Ist **rund** = **false**, ist die Art des gelieferten Datums GD19 (Datum mit Uhrzeit).

Beispiel: Berechnen des nächstmöglichen Arbeitstages nach einem Datum mit Uhrzeit

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** der nächstmögliche Arbeitstag nach einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – nächster Arbeitstag: Di 29.05.2012

```
.type GD xgddate
.formula U|xgddate = F|addworkhours("", "", "26.05.2012 00:00", 0, true)
next work day: 'U|xgddate'
```

next work day: 29.05.2012

Beispiel: Berechnen des nächstmöglichen Liefertages nach einem Datum mit Uhrzeit

In diesem Beispiel wird, mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** der nächstmögliche Liefertag nach einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Liefertag) – nächster Liefertag: Mi 30.05.2012

```
.type GD xgddate
.formula U|xgddate = F|adddelivhours("", "", "26.05.2012 00:00", 0, true)
next delivery day: 'U|xgddate'
```

next delivery day: 30.05.2012

Beispiel: Berechnen des nächstmöglichen Anliefertages nach einem Datum mit Uhrzeit

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` der nächstmögliche Anliefertag nach einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, berechnet. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Anliefertag) – Mo 28.05.2012 (Feiertag) – nächster Anliefertag: Mo 04.06.2012

```
.type GD xgddate
.formula U|xgddate = F|adreceipthours("", "", "26.05.2012 00:00", 0, true)
next receipt day: 'U|xgddate'
```

next receipt day: 04.06.2012

Addieren von Arbeits-, Liefer- bzw. Anlieferstunden zu einem Datum mit Uhrzeit, mit Runden des Ergebnisses



- Im Kalender, der zur Berechnung verwendet wird, muss die Einheit **Arbeitsstunden** definiert sein.
- Der verwendete Kalender wird durch die **Parameterkombination** `kalBezugsObj`, `lgruppe` definiert.
- Ist `rund = true`, ist die Art des gelieferten Datums GD (Datum ohne Uhrzeit).
- Ist `rund = false`, ist die Art des gelieferten Datums GD19 (Datum mit Uhrzeit).

Beispiel: Addieren von Arbeitsstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` zu einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 15 Arbeitsstunden addiert. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – Berechnungsbeginn: Di 29.05.2012 00:00 (nächster Arbeitstag)
 Di 29.05.2012 00:00 + 7,5h = Di 29.05.2012 07:30 – Berechnung fortsetzen:
 Mi 30.05.2012 00:00
 Mi 30.05.2012 00:00 + 7,5h = Mi 30.05.2012 07:30 – Ergebnis gerundet: Do 31.05.2012

```
.type GD xgddate
.formula U|xgddate = F|addworkhours("", "", "26.05.2012 00:00", 15, true)
calculated work day: 'U|xgddate'
```

```
calculated work day: 31.05.2012
```

Beispiel: Addieren von Arbeitsstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** zu einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 16 Arbeitsstunden addiert. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – Berechnungsbeginn: Di 29.05.2012 00:00 (nächster Arbeitstag)
 Di 29.05.2012 00:00 + 7,5h = Di 29.05.2012 07:30 – Berechnung fortsetzen:
 Mi 30.05.2012 00:00
 Mi 30.05.2012 00:00 + 7,5h = Mi 30.05.2012 07:30 – Berechnung fortsetzen:
 Do 31.05.2012 00:00
 Do 31.05.2012 00:00 + 1,0h = Do 31.05.2012 01:00 – Ergebnis gerundet: Fr 01.06.2012

```
.type GD xgddate
.formula U|xgddate = F|addworkhours("", "", "26.05.2012 00:00", 16, true)
calculated work day: 'U|xgddate'
```

```
calculated work day: 01.06.2012
```

Beispiel: Addieren von Lieferstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="K 4"**, **lgruppe=""** zu einem Datum mit Uhrzeit, Samstag 07.05.2012 00:00, 15 Lieferstunden addiert. Das Ergebnis wird gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde 4 - ohne Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo, Anliefertag: Mo-Fr

Berechnungsstart: Mo 07.05.2012 00:00 (Liefertag) – Berechnungsbeginn: Mo 07.05.2012 00:00

Mo 07.05.2012 00:00 + 7,5h = Mo 07.05.2012 07:30 – Berechnung fortsetzen: Mo 14.05.2012 00:00

Mo 14.05.2012 00:00 + 7,5h = Mo 14.05.2012 07:30 – Ergebnis gerundet: Mo 21.05.2012

```
.type GD xgddate
.formula U|xgddate = F|adddelivhours("K 4", "", "07.05.2012 00:00", 15, true)
calculated delivery day Liefertag: 'U|xgddate'
```

calculated delivery day Liefertag: 21.05.2012

Beispiel: Addieren von Anlieferstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="L 2008"**, **lgruppe=""** zu einem Datum mit Uhrzeit, Samstag 07.05.2012 00:00, 15 Anlieferstunden addiert. Das Ergebnis wird gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Lieferant **2008 - ohne** Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Mo

Berechnungsstart: Mo 07.05.2012 00:00 (Anliefertag) – Berechnungsbeginn Mo 07.05.2012 00:00

Mo 07.05.2012 00:00 + 7,5h = Mo 07.05.2012 07:30 – Berechnung fortsetzen: Mo 14.05.2012 00:00

Mo 14.05.2012 00:00 + 7,5h = Mo 14.05.2012 07:30 – Ergebnis gerundet: 21.05.2012

```
.type GD xgddate
.formula U|xgddate = F|addrecepthours("L 2008", "", "07.05.2012 00:00", 15, true)
calculated receipt day: 'U|xgddate'
```

calculated receipt day: 21.05.2012

Addieren von Arbeits-, Liefer- bzw. Anlieferstunden zu einem Datum mit Uhrzeit, ohne Runden des Ergebnisses



- Im Kalender, der zur Berechnung verwendet wird, muss die Einheit **Arbeitsstunden**

definiert sein.

- Der verwendete Kalender wird durch die **Parameterkombination** `kalBezugsObj`, `lgruppe` definiert.
- Ist `rund = true`, ist die Art des gelieferten Datums GD (Datum ohne Uhrzeit).
- Ist `rund = false`, ist die Art des gelieferten Datums GD19 (Datum mit Uhrzeit).

Beispiel: Addieren von Arbeitsstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` zu einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 15 Arbeitsstunden addiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo.

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – Berechnungsbeginn: Di 29.05.2012 00:00 (nächster Arbeitstag)
 Di 29.05.2012 00:00 + 7,5h = Di 29.05.2012 07:30 – Berechnung fortsetzen:
 Mi 30.05.2012 00:00
 Mi 30.05.2012 00:00 + 7,5h = Mi 30.05.2012 07:30 – Ergebnis: 30.05.2012 07:30

```
.type GD19 xgddate
.formula U|xgddate = F|addworkhours("", "", "26.05.2012 00:00", 15, false)
calculated work day: 'U|xgddate'
```

calculated work day: 30.05.2012 07:30:00

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` zu einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 16 Arbeitsstunden addiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo.

Berechnungsstart: Sa 26.05.2012 (kein Arbeitstag) – Mo 28.05.2012 (Feiertag) – Berechnungsbeginn: Di 29.05.2012 00:00 (nächster Arbeitstag)
 Di 29.05.2012 00:00 + 7,5h = Di 29.05.2012 07:30 – Berechnung fortsetzen:
 Mi 30.05.2012 00:00
 Mi 30.05.2012 00:00 + 7,5h = Mi 30.05.2012 07:30 – Berechnung fortsetzen:
 Do 31.05.2012 00:00

Do 31.05.2012 00:00 + 1,0h = Do 31.05.2012 01:00 – Ergebnis: Do 31.05.2012 01:00

```
.type GD19 xgddate
.formula U|xgddate = F|addworkhours("", "", "26.05.2012 00:00", 16, false)
calculated work day: 'U|xgddate'
```

calculated work day: 31.05.2012 01:00:00

Beispiel: Addieren von Lieferstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="K 4"**, **lgruppe=""** zu einem Datum mit Uhrzeit, Samstag 07.05.2012 00:00, 15 Lieferstunden addiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde **4** - ohne Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo, Anliefertag: Mo.

Berechnungsstart: Mo 07.05.2012 00:00 (Liefertag) – Berechnungsbeginn: Mo 07.05.2012 00:00

Mo 07.05.2012 00:00 + 7,5h = Mo 07.05.2012 07:30 – Berechnung fortsetzen:
Mo 14.05.2012 00:00

Mo 14.05.2012 00:00 + 7,5h = Mo 14.05.2012 07:30 – Ergebnis: Mo 14.05.2012 07:30

```
.type GD19 xgddate
.formula U|xgddate = F|adddelivhours("K 4", "", "07.05.2012 00:00", 15, false)
calculated delivery day: 'U|xgddate'
```

calculated delivery day: 14.05.2012 07:30:00

Beispiel: Addieren von Anlieferstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="L 2008"**, **lgruppe=""** zu einem Datum mit Uhrzeit, Samstag 22.05.2012 00:00, 16 Anlieferstunden addiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Lieferant **2008** - ohne Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Mo.

Berechnungsstart: Di 22.05.2012 00:00 (kein Anliefertag) – Mo 28.05.2012 00:00 (Feiertag) – Berechnungsbeginn: Mo 04.06.2012 00:00 (nächster Anliefertag)

Mo 04.06.2012 00:00 + 7,5h = Mo 04.06.2012 07:30 – Berechnung fortsetzen: Mo 11.06.2012 00:00

Mo 11.06.2012 00:00 + 7,5h = Mo 11.06.2012 07:30 – Berechnung fortsetzen: Mo 18.06.2012 00:00

Mo 18.06.2012 00:00 + 1.0h = Mo 18.06.2012 01:00 – Ergebnis: Mo 18.06.2012 01:00

```
.type GD19 xgddate
.formula U|xgddate = F|addrecepthours("L 2008", "", "22.05.2012 00:00", 16, false)
calculated receipt day: 'U|xgddate'
```

calculated receipt day: 18.06.2012 01:00:00

Subtraktion von Arbeits-, Liefer oder Anlieferstunden von einem Datum mit Uhrzeit

Die folgenden Funktionen subtrahieren eine Anzahl von Stunden von einem Datum mit Uhrzeit.

Funktionsname und -argumente	Typ des Rückgabewerts
F subworkhours(kalBezugsObj, lgruppe, datum, anzstd, rund)	GD/GD19
F subdelivhours(kalBezugsObj, lgruppe, datum, anzstd, rund)	GD/GD19
F subrecepthours(kalBezugsObj, lgruppe, datum, anzstd, rund)	GD/GD19

Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle [Parameterbeschreibung](#).



- Im Kalender, der zur Berechnung verwendet wird, muss die Einheit **Arbeitsstunden** definiert sein.
- Der verwendete Kalender wird durch die **Parameterkombination** **kalBezugsObj**, **lgruppe** definiert.
- Ist **anzstd** = 0, wird das Datum mit Uhrzeit zum Ende des letzten Arbeits-, Liefer- bzw. Anliefertages berechnet.
- Ist **rund** = **true**, ist die Art des gelieferten Datums GD (Datum ohne Uhrzeit).

- Ist **rund = false**, ist die Art des gelieferten Datums GD19 (Datum mit Uhrzeit).
- Bei Addition und Subtraktion von Stunden darf die Summe der Stunden nicht mehr als 480000 betragen.

Berechnen des letzten Arbeits-, Liefer- bzw. Anliefertages vor einem Datum mit Uhrzeit

Beispiel: Berechnen des letzten Arbeitstages vor einem Datum mit Uhrzeit

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** der letzte Arbeitstag vor einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, berechnet. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 – letzter Arbeitstag: Fr 25.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subworkhours("", "", "26.05.2012 00:00", 0, true)
former work day: 'U|xgddate'
```

```
former work day: 25.05.2012
```

Beispiel: Berechnen des letzten Liefertages vor einem Datum mit Uhrzeit

In diesem Beispiel wird mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** der letzte Liefertag vor einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, berechnet. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 – letzter Liefertag: Mi 23.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subdelivhours("", "", "26.05.2012 00:00", 0, true)
former delivery day: 'U|xgddate'
```

```
former delivery day: 23.05.2012
```

Beispiel: Berechnen des letzten Anliefertages vor einem Datum mit Uhrzeit

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` der letzte Anliefertag vor einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, berechnet. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 – letzter Anliefertag: Mo 21.05.2012

```
.type GD xgddate
.formula U|xgddate = F|subrecepthours("", "", "26.05.2012 00:00", 0, true)
former receipt day: 'U|xgddate'
```

```
former receipt day: 21.05.2012
```

Subtrahieren von Arbeits-, Liefer- bzw. Anlieferstunden von einem Datum mit Uhrzeit, mit Runden des Ergebnisses



- Im Kalender, der zur Berechnung verwendet wird, muss die Einheit **Arbeitsstunden** definiert sein.
- Der verwendete Kalender wird durch die **Parameterkombination** `kalBezugsObj`, `lgruppe` definiert.
- Ist `anzstd = 0`, wird das Datum mit Uhrzeit zum Ende des letzten Arbeits-, Liefer- bzw. Anliefertages berechnet.
- Ist `rund = true`, ist die Art des gelieferten Datums GD (Datum ohne Uhrzeit).
- Ist `rund = false`, ist die Art des gelieferten Datums GD19 (Datum mit Uhrzeit).

Beispiel: Subtrahieren von Arbeitsstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` von einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 15 Arbeitsstunden subtrahiert. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 00:00 (kein Arbeitstag) – Berechnungsbeginn: Fr 25.05.2012 07:30 (letzter Arbeitstag)
Fr 25.05.2012 07:30 - 7,5h = Fr 25.05.2012 00:00 – Berechnung fortsetzen:
Do 24.05.2012 07:30
Do 24.05.2012 07:30 - 7,5h = Do 24.05.2012 00:00 – Ergebnis gerundet: Do

24.05.2012

```
.type GD19 xgddate
.formula U|xgddate = F|subworkhours("", "", "26.05.2012 00:00", 15, true)
calculated work day: 'U|xgddate'
```

calculated work day: 24.05.2012

Beispiel: Subtrahieren von Arbeitsstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj=""**, **lgruppe=""** von einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 16 Arbeitsstunden subtrahiert. Das Ergebnis wird gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 00:00 (kein Arbeitstag) – Berechnungsbeginn: Fr 25.05.2012 07:30 (letzter Arbeitstag)
 Fr 25.05.2012 07:30 - 7,5h = Fr 25.05.2012 00:00 – Berechnung fortsetzen:
 Do 24.05.2012 07:30
 Do 24.05.2012 07:30 - 7,5h = Do 24.05.2012 00:00 – Berechnung fortsetzen:
 Mi 23.05.2012 07:30
 Mi 23.05.2012 07:30 - 1,0h = Mi 23.05.2012 06:30 – Ergebnis gerundet: Mi 23.05.2012

```
.type GD19 xgddate
.formula U|xgddate = F|subworkhours("", "", "26.05.2012 00:00", 16, true)
calculated work day: 'U|xgddate'
```

calculated work day: 23.05.2012

Beispiel: Subtrahieren von Lieferstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="L 2008"**, **lgruppe=""** von einem Datum mit Uhrzeit, Montag 07.05.2012 00:00, 15 Lieferstunden subtrahiert. Das Ergebnis wird gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Lieferant **2008 - ohne** Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo, Anliefertag: Mo-Fr

Berechnungsstart: Mo 07.05.2012 00:00 (Liefertag) – Berechnungsbeginn: Mo 30.04.2012 07:30

Mo 30.04.2012 07:30 - 7,5h = Mo 30.04.2012 00:00 – Berechnung fortsetzen: Mo 23.04.2012 07:30

Mo 23.04.2012 07:30 - 7,5h = Mo 23.04.2012 00:00 – Ergebnis gerundet: Mo 23.04.2012

```
.type GD19 xgddate
.formula U|xgddate = F|subdelivhours("L 2008", "", "07.05.2012 00:00", 15, true)
calculated delivery day: 'U|xgddate'
```

calculated delivery day: 23.04.2012

Beispiel: Subtrahieren von Anlieferstunden, mit Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="K 4"**, **lgruppe=""** von einem Datum mit Uhrzeit, Montag 07.05.2012 00:00, 15 Anlieferstunden subtrahiert. Das Ergebnis wird gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde 4 - ohne Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Fr

Berechnungsstart: Mo 07.05.2012 00:00 (kein Anliefertag) – Berechnungsbeginn: Fr 04.05.2012 07:30 (letzter Anliefertag)

Fr 04.05.2012 07:30 - 7,5h = Fr 04.05.2012 00:00 – Berechnung fortsetzen: Fr 27.04.2012 07:30

Fr 27.04.2012 07:30 - 7,5h = Fr 27.04.2012 00:00 – Ergebnis gerundet: Mo 27.04.2012

```
.type GD19 xgddate
.formula U|xgddate = F|subrecepthours("K 4", "", "07.05.2012 00:00", 15, true)
calculated receipt day: 'U|xgddate'
```

calculated receipt day: 27.04.2012

Subtrahieren von Arbeits-, Liefer- bzw. Anlieferstunden von einem Datum mit Uhrzeit, ohne Runden des Ergebnisses



- Im Kalender, der zur Berechnung verwendet wird, muss die Einheit **Arbeitsstunden** definiert sein.
- Der verwendete Kalender wird durch die **Parameterkombination** `kalBezugsObj`, `lgruppe` definiert.
- Ist `anzstd = 0`, wird das Datum mit Uhrzeit zum Ende des letzten Arbeits-, Liefer- bzw. Anliefertages berechnet.
- Ist `rund = true`, ist die Art des gelieferten Datums GD (Datum ohne Uhrzeit).
- Ist `rund = false`, ist die Art des gelieferten Datums GD19 (Datum mit Uhrzeit).

Beispiel: Subtrahieren von Arbeitsstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` von einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 15 Arbeitsstunden subtrahiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 00:00 (kein Arbeitstag) – **Berechnungsbeginn:** Fr 25.05.2012 07:30 (letzter Arbeitstag)
 Fr 25.05.2012 07:30 - 7,5h = Fr 25.05.2012 00:00 – Berechnung fortsetzen:
 Do 24.05.2012 07:30
 Do 24.05.2012 07:30 - 7,5h = Do 24.05.2012 00:00 – **Ergebnis:** Do 24.05.2012 00:00:00

```
.type GD19 xgddate
.formula U|xgddate = F|subworkhours("", "", "26.05.2012 00:00", 15, false)
calculated work day: 'U|xgddate'
```

```
calculated work day: 24.05.2012 00:00:00
```

Beispiel: Subtrahieren von Arbeitsstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` von einem Datum mit Uhrzeit, Samstag 26.05.2012 00:00, 16 Arbeitsstunden subtrahiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung wird der Standardkalender (Dispokalender) verwendet.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mi, Anliefertag: Mo

Berechnungsstart: Sa 26.05.2012 00:00 (kein Arbeitstag) – Berechnungsbeginn: Fr 25.05.2012 07:30 (letzter Arbeitstag)
 Fr 25.05.2012 07:30 - 7,5h = Fr 25.05.2012 00:00 – Berechnung fortsetzen:
 Do 24.05.2012 07:30
 Do 24.05.2012 07:30 - 7,5h = Do 24.05.2012 00:00 – Berechnung fortsetzen:
 Mi 23.05.2012 07:30
 Mi 23.05.2012 07:30 - 1,0h = Mi 23.05.2012 06:30 – Ergebnis: Mi 23.05.2012 06:30:00

```
.type GD19 xgddate
.formula U|xgddate = F|subworkhours("", "", "26.05.2012 00:00", 16, false)
calculated work day: 'U|xgddate'
```

calculated work day: 23.05.2012 06:30:00

Beispiel: Subtrahieren von Lieferstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination **kalBezugsObj="L 2008"**, **lgruppe=""** von einem Datum mit Uhrzeit, Montag 07.05.2012 00:00, 15 Lieferstunden subtrahiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Lieferant **2008 - ohne** Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo, Anliefertag: Mo

Berechnungsstart: Mo 07.05.2012 00:00 (Liefertag) – Berechnungsbeginn: Mo 30.04.2012 07:30
 Mo 30.04.2012 07:30 - 7,5h = Mo 30.04.2012 00:00 – Berechnung fortsetzen:
 Mo 23.04.2012 07:30
 Mo 23.04.2012 07:30 - 7,5h = Mo 23.04.2012 00:00 – Ergebnis: Mo 23.04.2012 00:00:00

```
.type GD19 xgddate
.formula U|xgddate = F|subdelivhours("L 2008", "", "07.05.2012 00:00", 15, false)
calculated delivery day: 'U|xgddate'
```

calculated delivery day: 23.04.2012 00:00:00

Beispiel: Subtrahieren von Anlieferstunden, ohne Runden des Ergebnisses

In diesem Beispiel werden mithilfe der Parameterkombination `kalBezugsObj="K 4"`, `lgruppe=""` von einem Datum mit Uhrzeit, Montag 22.05.2012 00:00, 16 Anlieferstunden subtrahiert. Das Ergebnis wird nicht gerundet.

Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde 4 - ohne Angabe einer Lagergruppe, definiert wurde.

Die Arbeitszeit pro Tag beträgt 7,5 Stunden.

Im Standardkalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Fr

Berechnungsstart: Di 22.05.2012 00:00 (kein

Anliefertag) – Berechnungsbeginn: Fr 18.05.2012 07:30 (letzter Anliefertag)

Fr 18.05.2012 07:30 - 7,5h = Fr 18.05.2012 00:00 – Berechnung fortsetzen:

Fr 11.05.2012 07:30

Fr 11.05.2012 07:30 - 7,5h = Fr 11.05.2012 00:00 – Berechnung fortsetzen:

Fr 04.05.2012 07:30

Fr 04.05.2012 07:30 - 1,0h = Fr 04.05.2012 06:30 – Ergebnis: Fr 04.05.2012 06:30:00

```
.type GD19 xgddate
.formula U|xgddate = F|subrecepthours("K 4", "", "22.05.2012 00:00", 16, false)
calculated receipt day: 'U|xgddate'
```

```
calculated receipt day: 04.05.2012 06:30:00
```

Explizites Runden bei Kalenderfunktionen

Die Funktionen zum Addieren und Subtrahieren von Kalendertagen erlauben das Runden auf den Tagesbeginn des darauffolgenden bzw. des davorliegenden Arbeits-, Liefer- oder Anliefertags.

Funktionsname und -argumente	Typ des Rückgabewerts
<code>F addworkdays(kalBezugsObj, lgruppe, datum, anztage, rund)</code>	GD/GD19
<code>F addreceiptdays(kalBezugsObj, lgruppe, datum, anztage, rund)</code>	GD/GD19
<code>F adddelivdays(kalBezugsObj, lgruppe, datum, anztage, rund)</code>	GD/GD19
<code>F subworkdays(kalBezugsObj, lgruppe, datum, anztage, rund)</code>	GD/GD19

Funktionsname und -argumente	Typ des Rückgabewerts
F subreceiptdays(kalBezugsObj, lgruppe, datum, anztage, rund)	GD/GD19
F subdelivdays(kalBezugsObj, lgruppe, datum, anztage, rund)	GD/GD19
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	

Wenn der Parameter nicht angegeben ist, wird nicht gerundet. Beim Subtrahieren von 0 Tagen ab einem freien Tag wird das Tagesende des davorliegenden Arbeits-, Liefer- oder Anliefertages geliefert. Wenn gerundet wird, wird das Datum auf den Tagesbeginn abgerundet. Der Typ des Ergebniswertes dieser Funktionen ist GD, falls gerundet wird und sonst GD19.

Bestimmen der Anzahl der Arbeits-, Liefer- bzw. Anliefertage in einem Zeitraum

Die folgenden Funktionen bestimmen die Anzahl der Arbeits-, Liefer- bzw. Anliefertage zwischen Anfangsdatum (einschließlich) und Enddatum (ausschließlich).

Funktionsname und -argumente	Typ des Rückgabewerts
F numworkdays(kalBezugsObj, lgruppe, anfdat, enddat)	integer
F numdelivdays(kalBezugsObj, lgruppe, anfdat, enddat)	integer
F numreceiptdays(kalBezugsObj, lgruppe, anfdat, enddat)	integer
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	



- Der verwendete Kalender wird durch die [Parameterkombination](#) `kalBezugsObj`, `lgruppe` definiert.
- Anfangsdatum `<code>anfdat</code>` <= Enddatum `<code>enddat</code>`
- Das Ergebnis ist immer `>= 0`.
- Beim Bestimmen der Anzahl der Arbeits-, Liefer- bzw. Anliefertage in einem Zeitraum darf die Summe der Tage nicht mehr als 20000 betragen.

Beispiel: Ermitteln der Arbeitstage in einem Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` die Anzahl der Arbeitstage, im Zeitraum 01.05.2012 - 31.05.2012 ermittelt. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Mo-Fr

```
.type int xianzahl  
.formula U|xianzahl = F|numworkdays("", "", "01.05.2012 ", "01.06.2012")  
number of work days: 'U|xianzahl'
```

number of work days: 20

Beispiel: Ermitteln der Liefertage in einem Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="L 2008"`, `lgruppe=""` die Anzahl der Liefertage, im Zeitraum 01.05.2012 - 31.05.2012, ermittelt. Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Lieferant **2008** - ohne Angabe einer Lagergruppe, definiert wurde. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo, Anliefertag: Mo-Fr>

```
.type int xianzahl  
.formula U|xianzahl = F|numdelivdays("L 2008", "", "01.05.2012 ", "01.06.2012")  
number of delivery days: 'U|xianzahl'
```

number of delivery days: 3

Beispiel: Ermitteln der Anliefertage in einem Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="K 4"`, `lgruppe=""` die Anzahl der Anliefertage, im Zeitraum 01.05.2012 - 31.05.2012, ermittelt. Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde **4** - ohne Angabe einer Lagergruppe, definiert wurde. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Fr.

```
.type int xianzahl  
.formula U|xianzahl = F|numreceiptdays("K 4", "", "01.05.2012 ", "01.06.2012")  
number of receipt days: 'U|xianzahl'
```

number of receipt days: 4

Bestimmen der Anzahl der Arbeits-, Liefer- bzw. Anlieferstunden in einem Zeitraum

Die folgenden Funktionen bestimmen die Arbeits-, Liefer- bzw. Anlieferstunden zwischen Anfangsdatum (einschließlich) und Enddatum (ausschließlich).

Funktionsname und -argumente	Typ des Rückgabewerts
<code>F numworkhours(kalBezugsObj, lgruppe, anfdat, enddat)</code>	integer
<code>F numdelivhours(kalBezugsObj, lgruppe, anfdat, enddat)</code>	integer
<code>F numreceipthours(kalBezugsObj, lgruppe, anfdat, enddat)</code>	integer

Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle [Parameterbeschreibung](#).



- Der verwendete Kalender wird durch die [Parameterkombination](#) `kalBezugsObj`, `lgruppe`= definiert.
- Anfangsdatum `<code>anfdat</code> <= Enddatum <code>enddat</code>`
- Das Ergebnis ist immer `>= 0`.
- Beim Bestimmen der Anzahl der Arbeits-, Liefer- bzw. Anlieferstunden in einem Zeitraum darf die Summe der Stunden nicht mehr als 480000 betragen.

Beispiel: Ermitteln der Arbeitsstunden in einem Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj=""`, `lgruppe=""` die Anzahl der Arbeitsstunden, im Zeitraum 01.05.2012 - 31.05.2012 ermittelt. Zur Berechnung wird der Standardkalender (Dispokalender) verwendet. Die Arbeitszeit pro Tag beträgt 7,5 Stunden. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Mo-Fr.

```
.type real xranzahl
.type int xianzahl
.formula U|xranzahl = F|numworkhours("", "", "01.05.2012", "01.06.2012")
.formula U|xianzahl = F|numworkdays("", "", "01.05.2012", "01.06.2012")
'U|xianzahl' work days * 7,5h = 'U|xranzahl' work hours
```

```
20 work days * 7,5h = 150.00 work hours
```

Beispiel: Ermitteln der Lieferstunden in einem Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="L 2008"`, `lgruppe=""` die Anzahl der Lieferstunden, im Zeitraum 01.05.2012 - 31.05.2012 ermittelt.
 Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Lieferant **2008** - ohne Angabe einer Lagergruppe, definiert wurde.
 Die Arbeitszeit pro Tag beträgt 7,5 Stunden.
 Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo, Anliefertag: Mo-Fr>

```
.type real xranzahl
.type int xianzahl
.formula U|xranzahl = F|numdelivhours("L 2008", "", "01.05.2012", "01.06.2012")
.formula U|xianzahl = F|numdelivdays("L 2008", "", "01.05.2012", "01.06.2012")
'U|xianzahl' delivery days * 7,5h = 'U|xranzahl' delivery hours
```

3 delivery days * 7,5h = 22.50 delivery hours

Beispiel: Ermitteln der Anlieferstunden in einem Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="K 4"`, `lgruppe=""` die Anzahl der Anlieferstunden, im Zeitraum 01.05.2012 - 31.05.2012 ermittelt.
 Zur Berechnung soll der Kalender (Dispokalender) verwendet werden, der für Kunde **4** - ohne Angabe einer Lagergruppe, definiert wurde.
 Die Arbeitszeit pro Tag beträgt 7,5 Stunden.
 Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Fr

```
.type real xranzahl
.type int xianzahl
.formula U|xranzahl = F|numrecepthours("K 4", "", "01.05.2012", "01.06.2012")
.formula U|xianzahl = F|numreceiptdays("K 4", "", "01.05.2012", "01.06.2012")
'U|xianzahl' receipt days * 7,5h = 'U|xranzahl' receipt hours
```

4 receipt days * 7,5h = 30.00 receipt hours

Bestimmen der Gesamtkapazität einer Abteilung oder Maschinengruppe in einen Zeitraum

Die folgenden Funktionen bestimmen die Gesamtkapazität für eine Abteilung/Maschinengruppe zwischen Anfangsdatum (einschließlich) und Enddatum (einschließlich).

Funktionsname und -argumente	Typ des Rückgabewerts
<code>F totalcapacityhours(kapaz, lgruppe, anfdat, enddat)</code>	real
Für weitere Informationen zu den einzelnen Funktionsargumenten siehe Tabelle Parameterbeschreibung .	



- Der verwendete Kalender wird durch die [Parameterkombination](#) `kalBezugsObj`, `lgruppe` definiert.
- Anfangsdatum `<code>anfdat</code>` <= Enddatum `<code>enddat</code>`
- Das Ergebnis ist immer ≥ 0.0 .
- Beim Bestimmen der Gesamtkapazität darf der Zeitraum nicht mehr als 20000 Tage umfassen.

Beispiel: Ermitteln der Gesamtkapazität einer Maschinengruppe in einen Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="122"`, `lgruppe=""` die Gesamtkapazität einer Maschinengruppe, im Zeitraum 01.05.2012 - 31.05.2012, ermittelt. Zur Berechnung soll der Kalender (Kapazitätsangebot) verwendet werden, der für Maschinengruppe **122** - ohne Angabe einer Lagergruppe, definiert wurde. Die Arbeitszeit pro Tag beträgt 6,0 Stunden. Die Maschinenanzahl der Maschinengruppe beträgt 2,0. Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr, Anliefertag: Fr.

```
.type real xrcapacity
.type int xianzahl
.formula U|xrcapacity = F|totalcapacityhours("122", "", "01.05.2012", "31.05.2012")
.formula U|xianzahl = F|numworkdays("122", "", "01.05.2012", "01.06.2012")
'U|xianzahl' work days * 6h * 2 = 'U|xrcapacity'h total capacity
```

```
20 work days * 6h * 2 = 240.00h total capacity
```

Beispiel: Ermitteln der Gesamtkapazität einer Abteilung für einen Zeitraum

In diesem Beispiel wird mithilfe der Parameterkombination `kalBezugsObj="12"`, `lgruppe=""` die Gesamtkapazität einer Abteilung, im Zeitraum 01.05.2012 - 31.05.2012, ermittelt. Zur Berechnung soll der Kalender (Kapazitätsangebot) verwendet werden, der für Abteilung **12** - ohne Angabe einer Lagergruppe, definiert wurde. Die Abteilung 12 besteht aus 2 Maschinengruppen.

- Maschinengruppe 104: Maschinenanzahl 1 - Einzelkapazität 8 Std/Tag.
- Maschinengruppe 105: Maschinenanzahl 3 - Einzelkapazität 8 Std/Tag.
- Gesamtkapazität: Maschinenanzahl 4 * 8 Std/Tag * 20 Arbeitstage = 640h

Im Kalender für dieses Beispiel sind Feiertage eingetragen. Arbeitstage: Mo-Fr, Liefertag: Mo-Fr,

Anliefertag: Fr.

```
.type real xrcapacity
.formula U|xrcapacity = F|totalcapacityhours("12", "", "01.05.2012", "31.05.2012")
total capacity: 'U|xrcapacity'h
```

total capacity: 640.00h

Datums- und Zeitfunktionen

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
date(Datum)	GDATUM	Wandelt ein SDATUM oder einen String mit einer Datumsangabe in ein GDATUM um.
sdate(Datum)	SDATUM	Wandelt ein GDATUM oder einen String mit einer Datumsangabe in ein SDATUM um.
addyears(Datum, Anzahl von Jahren)	GDATUM/SDATUM	Addiert zu einem GDATUM oder SDATUM eine bestimmte Anzahl von Jahren (INT). Der Rückgabebetyp entspricht dem des ersten Arguments.
addmonths(Datum, Anzahl von Monaten)	GDATUM/SDATUM	Addiert zu einem GDATUM oder SDATUM eine bestimmte Anzahl von Monaten (INT). Der Rückgabebetyp entspricht dem des ersten Arguments.
addweeks(Datum, Anzahl von Wochen)	GDATUM/SDATUM	Addiert zu einem GDATUM oder SDATUM eine bestimmte Anzahl von Wochen (INT). Der Rückgabebetyp entspricht dem des ersten Arguments.
adddays(Datum, Anzahl von Tagen)	GDATUM/SDATUM	Addiert zu einem GDATUM oder SDATUM eine bestimmte Anzahl von Tagen (INT). Der Rückgabebetyp entspricht dem des ersten Arguments.
addhours(Datum, Anzahl von Stunden)	GDATUM	Addiert zu einem GDATUM eine bestimmte Anzahl von Stunden.
addminutes(Datum, Anzahl von Minuten)	GDATUM	Addiert zu einem GDATUM eine bestimmte Anzahl von Minuten.
addseconds(DATUM, Anzahl von Sekunden)	GDATUM	Addiert zu einem GDATUM eine bestimmte Anzahl von Sekunden.
firstdayofmonth(Datum)	GDATUM/SDATUM	Liefert den Monatsersten zu einem GDATUM oder SDATUM. Der Rückgabebetyp entspricht dem des ersten Arguments.
lastdayofmonth(Datum)	GDATUM/SDATUM	Liefert den Monatsletzten zu einem GDATUM oder SDATUM. Der Rückgabebetyp entspricht dem des ersten Arguments.
dayofweek(Datum)	A204	Liefert den Wochentag eines Datums (SDATUM, GDATUM) als Aufzählungselement.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
day(dateexpr) month(dateexpr) year(dateexpr)	Integer	Liefern den Tagesanteil, Monatsanteil oder Jahresanteil eines Datums als Integer-Zahl.
hour(dateexpr) minute(dateexpr) second(dateexpr)	Integer	Liefern die Stunden, Minuten oder Sekunden des Zeitanteils eines Datums als Integer-Zahl. Diese Funktionen liefern 0, falls dateexpr keinen Zeitanteil hat.
hasime(dateexpr)	Bool	Liefert true , falls dateexpr einen Zeitanteil hat, ansonsten false .

Erlaubnisfunktionen

Erlaubnisfunktionen liefern Informationen darüber, welche Kommandos ein Benutzer ausführen darf und auf welche Arbeitsbereiche er zugreifen darf.

Alle Erlaubnisfunktionen haben ein Argument "Passwort", in dem ein Passwortdatensatz-Verweis angegeben werden kann. Ist das Argument angegeben, wird die Erlaubnis für diesen Passwortdatensatz geprüft, ansonsten für den aktuellen Benutzer. Um eine Erlaubnisfunktion mit einem Passwortargument aufrufen zu können, muss der aktuelle Benutzer die Erlaubnis besitzen, Erlaubnisdatensätze anzuzeigen.

Für die Datenbanknummern und Kommandonummern stehen **Konstanten** zur Verfügung.
ist das Kommando **Kunde ändern** erlaubt?



```
.input "KDATNAME.KONST"
.input "KAKTNAME.KONST"
'F|cmdpermitted(U|kodatkunde, U|koaktaendern)'
```

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
F cmdpermitted(Datenbanknummer[, Kommandonummer][, Passwort][, Datenbankgruppe])	Boolean	Liefert die Information, ob der angegebene bzw. aktuelle Benutzer das angegebene Kommando für die angegebene Datenbank ausführen darf. Fehlt der Parameter Kommandonummer, prüft die Funktion nicht die Erlaubnis für ein bestimmtes Kommando, sondern, ob der angegebene bzw. aktuelle Benutzer die Erlaubnis für irgendeine Aktion mit der angegebenen Datenbank besitzt. Ist eine Datenbankgruppe angegeben, wird nur diese Gruppe geprüft. Andernfalls werden alle Gruppen geprüft und es genügt, dass der Benutzer für eine Gruppe die Erlaubnis besitzt.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
F objectcmdpermitted(Objekt, Kommando)	Boolean	Liefert die Information, ob der aktuelle Benutzer das angegebene Kommando für das angegebene Objekt ausführen darf. Im Unterschied zu F cmdpermitted prüft diese Funktion die Erlaubnis für ein konkretes Objekt statt allgemein für eine Datenbank. Die Parameter Objekt und Kommando sind Pflichtparameter; Kommando kann als Kommandonummer oder als benannte Konstante angegeben werden.
F grouppermissions(Datenbanknummer[, Datenbankgruppe][, Passwort])	Boolean	Liefert die Information, ob der angegebene bzw. aktuelle Benutzer gruppenabhängige Erlaubnisse für die angegebene Datenbank besitzt, bzw. ob er für eine Datenbankgruppe eigene Erlaubnisse besitzt.
F tcmdpermitted(Tippkommandonummer[, Passwort])	Boolean	Liefert die Information, ob der angegebene bzw. aktuelle Benutzer das angegebene Tippkommando ausführen darf.
F infosyspermitted(Infosystemangabe[, Passwort])	Boolean	Liefert die Information, ob der angegebene bzw. aktuelle Benutzer das angegebene Infosystem ausführen darf. Ein grundlegendes Kriterium dafür ist die Erlaubnis , Infosysteme ausführen zu dürfen.
F diraccessible(stringexpr[, Passwort])	Boolean	Liefert die Information, ob der angegebene bzw. aktuelle Benutzer auf den angegebenen Arbeitsbereich zugreifen darf.
F cmdprio(Datenbanknummer[, Kommando][, Passwort][, Datenbankgruppe])	Art A 103	Liefert die Priorität, die bei der Ausführung eines Kommandos durch den aktuellen oder den angegebenen Benutzer verwendet wird. Darf der Benutzer das Kommando nicht ausführen, ist das Ergebnis "-".
F tcmdprio(Tippkommandonummer[, Passwort])	Art A 103	Liefert die Priorität, die bei der Ausführung eines Tippkommandos durch den aktuellen oder den angegebenen Benutzer verwendet wird. Darf der Benutzer das Kommando nicht ausführen, ist das Ergebnis "-".

Dateifunktionen

Die Dateifunktionen der FO dienen der Unterstützung der Arbeit mit Dateien und stellen im wesentlichen die wichtigsten Funktionen des Shell-Kommandos "test" zur Verfügung. Sie besitzen auch dessen Funktionsweise und funktionieren unabhängig von den Arbeitsbereich-Definitionen und -Erlaubnissen in Abas.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
<code>fileexists(Dateiname)</code>	Boolean	Liefert G true , wenn die angegebene Datei existiert. Dabei wird keine Annahme über den Typ der Datei getroffen, d.h., es kann sich auch um ein Verzeichnis handeln. Entspricht dem Shell-Kommando "test -e"
<code>filereadable(Dateiname)</code>	Boolean	Liefert G true , wenn die angegebene Datei für den aktuellen Benutzer auf Betriebssystemebene lesbar ist. Entspricht dem Shell-Kommando "test -r"
<code>filewriteable(Dateiname)</code>	Boolean	Liefert G true , wenn die angegebene Datei für den aktuellen Benutzer auf Betriebssystemebene beschreibbar ist. Entspricht dem Shell-Kommando "test -w"
<code>fileexecuteable(Dateiname)</code>	Boolean	Liefert G true , wenn die angegebene Datei für den aktuellen Benutzer auf Betriebssystemebene ausführbar ist. Entspricht dem Shell-Kommando "test -x"
<code>filenotempty(Dateiname)</code>	Boolean	Liefert G true , wenn die angegebene Datei nicht leer ist. Entspricht dem Shell-Kommando "test -s" Diese Funktion darf nur für normale Dateien aufgerufen werden, nicht für Verzeichnisse.
<code>isdir(Dateiname)</code>	Boolean	Liefert G true , wenn die angegebene Datei ein Verzeichnis ist. Entspricht dem Shell-Kommando "test -d"
<code>filenewer(Dateiname1, Dateiname2)</code> <code>fileolder(Dateiname1, Dateiname2)</code>	Boolean	<code>filenewer</code> liefert G true , wenn <code>dateiname1</code> jünger ist als <code>dateiname2</code> . <code>fileolder</code> liefert G true , wenn <code>dateiname1</code> älter ist als <code>dateiname2</code> . <code>filenewer</code> entspricht dem Shell-Kommando "test -nt" , <code>fileolder</code> entspricht dem Shell-Kommando "test -ot" . Existiert eine Datei nicht, so gilt sie als älter als jede existierende Datei, so wie dies auch beim Shell-Kommando "test" der Fall ist.
<code>filemtime(Dateiname)</code>	Ein Wert der Art GD19 (Zeitpunkt)	Liefert das Änderungsdatum einer Datei, d.h., wann die Datei zuletzt geändert wurde ("mtime"). Entspricht dem Shell-Kommando "stat -c '%y' " . Das Änderungsdatum wird in Lokalzeit ausgegeben, d.h., in Abhängigkeit von der eingestellten Zeitzone.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
<code>filesize(Dateiname)</code>	Integer	Liefert die Dateigröße in Bytes. Entspricht dem Shell-Kommando <code>"stat -c '%s' "</code> Diese Funktion darf nur für normale Dateien aufgerufen werden, nicht für Verzeichnisse.
<code>isfopname(Dateiname, Boolean)</code>	Boolean	Es wird geprüft, ob der angegebene Name ein gültiger FO-Dateiname ist. Ist ein Arbeitsbereich angegeben, muss dieser für den aktuellen Benutzer erlaubt sein. Die Prüfung bezieht sich aber nur auf den Namen, es muss keine Datei mit dem angegebenen Namen existieren. Mit dem boolschen Parameter können Sie angeben, dass der Dateiname als Java-FOP-Name interpretiert werden soll. <pre> F isfopname("ow1/abc") -> TRUE F isfopname("ow1/abc", FALSE) -> TRUE F isfopname("ow1/abc", TRUE) -> FALSE F isfopname("de.firma.abas.MeinFOP.java") -> FALSE F isfopname("de.firma.abas.MeinFOP.java", FALSE) -> FALSE F isfopname("de.firma.abas.MeinFOP.java", TRUE) -> TRUE </pre> Manche Namen sind nicht eindeutig als Java-FOP-Name erkennbar: <pre> F isfopname("ow1/test.java", FALSE) -> TRUE F isfopname("ow1/test.java", TRUE) -> TRUE </pre>  Die Funktion F isfopname lieferte immer TRUE. Vermeiden Sie FOPs mit solchen Namen.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
<code>fopath(Dateiname)</code>	String mit Dateinamen	Liefert den vollständigen Namen inklusive Pfad des FO-Programms, das bei .eingabe 'dateiname' tatsächlich ausgeführt würde. Die Funktion erwartet einen gültigen FO-Dateinamen als Argument. Zurückgeliefert wird der vollständige Pfad relativ zum Mandantenverzeichnis, ggf. mit Sprachverzeichnis, benutzerspezifischem Arbeitsbereich und Dateiname. Wird kein Programm gefunden, liefert die Funktion einen Leer-String.



Relative Datei- und Pfadnamen sind relativ zum aktuellen Mandantenverzeichnis anzugeben. Außer bei `isfopname` und `fopath` dürfen auch absolute Pfade verwendet werden.

Metafunktionen

Metafunktionen dienen der Information über Variablen und Werte.

Übersicht

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
<code>defined(variable)</code>	Boolean	Liefert G true , wenn die angegebene Variable bereits bekannt ist, also definiert wurde, ansonsten G false . Bei dieser Funktion darf das Präfix F zur Bewahrung der Kompatibilität zu älteren Programmversionen entfallen.
<code>typeof(variable)</code>	String, der eine Art bezeichnet	Liefert die Art einer Variablen oder eines Funktionsergebnisses. Dies ist die Art, wie sie normalerweise in der Vartab im Feld 'vitf' ("Art") dokumentiert ist. Ist eines der Felder 'vitnf' oder 'vitlgf' gefüllt, gilt dieser Eintrag. <pre>.type real xreal .select Customer "1" 'F typeof(U xreal)' = R12.2 'F typeof(H idno)' = NK0 'F typeof(H salutation)' = P12:3</pre> Der Parameter Art auflösen der Funktion typeof veranlasst die Auflösung direkt bei der Typbestimmung für ein Feld oder einen Ausdruck.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
acttype	effektive Art	ermittelt die effektive Art zu einer Artangabe der Variablen. Bei benannten Arten wird die dort definierte Art übernommen, ansonsten die Originalart. Für die individuellen Aufzählungen AZL/AZK* wird durch acttype nichts aufgelöst; hier können Sie die unterliegende Verweisart ausgeben.
typelength(artexpr[, Breite])	Integer	Länge von Werten einer Art Liefert die maximale Länge von Werten dieser Art, wenn diese mit 'variable' textuell ausgegeben werden. Dabei gelten alle gewohnten Regeln zur Ausgabesteuerung, z.B. mit .setze spracheinstellungen. Über einen zusätzlichen booleschen Parameter kann gesteuert werden, dass nicht die Breite eines Feldes, sondern dessen maximale Inhaltslänge ausgegeben wird. Breite: • bei allen Textarten die Zahl hinter den Artbuchstaben • bei DVMI2 und DVMR2 die maximale FO-Zeilenlänge: 2999 • bei Freitexten, historisch bedingt, 1 Inhaltslänge: • bei mehrzeiligen Textarten 2999 • bei den dateiwertigen Arten DVMI2, DVMR2 und TNn die größte in Abas darstellbare Integerzahl: 999.999.999 (1 Milliarde minus 1) Länge von Werten der Art "GD" ausgeben Die Länge von Werten der Art "GD" ist: <pre>'F typelength("GD")'</pre> Breite und Länge von Werten der Art "T40" ausgeben Die Breite von Werten der Art "T40" ist: <pre>'F typelength("T34")' = 34 'F typelength("T34", true)' = 34 'F typelength("T34", false)' = 2999</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
ismultiline(artexpr)	Boolean	Liefert die Information für eine Art, ob Felder dieser Art mehrzeiligen Inhalt haben können. Mehrzeilige Arten sind: * Textarten Tn, BTn, TUn, STn * Dateiverweisarten DVMI2, DVMR2 * Freitexte TNn
empty(variable)	Boolean	Liefert G true, wenn die angegebene Variable leer ist. Numerische Variablen sind leer, wenn ihr Wert == 0 ist. Boolesche Variablen sind leer, wenn ihr Wert == G false ist. Terminfelder im Ein- und Verkauf Leer angezeigte Termine im Einkauf und Verkauf sind u.U. nicht tatsächlich leer, sondern intern mit dem sog. Abrufdatum gekennzeichnet - z.B. eine ohne Termin erfasste Auftragsposition. Das Abrufdatum sorgt dafür, dass Vorgänge mit solchen Terminen am Ende, nicht am Anfang von terminsortierten Listen erscheinen. Für diese Felder liefert F empty(variable) somit G false. Um zu ermitteln, ob z.B. in dem Feld H term ein sichtbares Datum steht, prüfen Sie das Feld statt mit F empty(H term) mit 'H term'="".
modifiable(variable)	Boolean	Liefert G true , wenn die Variable für FO änderbar ist, also eine Zuweisung an die Variable zulässig ist. Die Funktion ermittelt stets die aktuelle Änderbarkeit im Moment des Aufrufs. Die Variable muss stets mit vorangestelltem Pufferbuchstaben angegeben werden. Als Pufferbuchstaben sind Datenbankvariablen eines geladenen Puffers und Textvariablen zulässig. Beachten Sie hier die Wirkung von .setze foextern
visible(variable)	Boolean	Liefert G true , wenn die Variable in der Maske sichtbar sein kann, also nicht durch den Kern, nicht durch eine zu geringe Priorität oder einen per .schutz -unsichtbar gesetzten Schutz ausgesternt wird. Die Funktion ermittelt stets die aktuelle Sichtbarkeit im Moment des Aufrufs. Die Variable muss stets mit vorangestelltem Pufferbuchstaben angegeben werden. Als Pufferbuchstaben sind Datenbankvariablen eines geladenen Puffers und Textvariablen zulässig.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
protected([variable][, Zeile][, Ausnahme])	Boolean	Liefert G true, wenn die Variable, die Zeile, die Spalte, die Tabelle oder der ganze Maskenpuffer mit .schutz schreibgeschützt wurde. variable muss eine Variable im Maskenpuffer sein und auch mit dem Pufferbuchstaben M angegeben werden. Zeile ist die gewünschte Zeilennummer oder 0 für den Kopfteil. Werden alle Argumente weggelassen, wird die Information für den Maskenpuffer geliefert, der durch das Kommando .schutz -alles gesetzt wurde. Alternativ können Sie auch den Wert -1 für die Zeile verwenden. Wird nur die Variable weggelassen, wird die Information für eine ganze Zeile, bei fehlender Zeile oder Zeile 0 die Information für den Kopfteil geliefert. Das entspricht dem Wert der Option .schutz -zeile. Wird beides angegeben, wird der Schutz für das Feld geliefert, entsprechend .schutz -feld. Wird nur die Variable weggelassen und die Zeile -2 verwendet, entspricht das dem .schutz -tabelle. Wird die Variable mit der Zeile -2 verwendet, entspricht das dem .schutz -spalte. Ist die Option Ausnahme G true, entspricht das der Option .schutz -ausnahme.
hidden([variable][, Zeile][, Ausnahme])	Boolean	Diese Funktion liefert den mit dem Kommando .schutz -unsichtbar eingestellten Schutz. Für die übrigen Parameter gilt das bei der Funktion protected beschriebene Verhalten.
modified(variable)	Boolean	Liefert G true, wenn die Variable nach dem Lesen oder dem letzten Speichern im selben Prozess geändert wurde, also eine Zuweisung an die Variable stattgefunden hat. Die Variable muss stets mit vorangestelltem Pufferbuchstaben angegeben werden. Als Pufferbuchstaben sind Bezugsobjekte für geladene Datenbankpuffer sowie T, U und G zulässig.
oldvalue(variable)	verschieden	Liefert den ursprünglichen Wert der Variablen. Dieser ist bei FO-Variablen und bei Feldern neuer Datensätze immer leer. Bei Feldern von Datensätzen, die von der Festplatte gelesen wurden, enthält oldvalue den Wert, den die Variable unmittelbar nach dem Lesen bzw. nach dem letzten Speichern des Datensatzes hatte. D.h., nach dem Speichern eines geänderten Datensatzes herrscht derselbe Zustand wie nach einem erneuten Lesen: alle Felder gelten nicht mehr als geändert und der Wert von F oldvalue(variable) ist identisch mit dem Wert von F value(variable).

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
compatible(artexpr1 , artexpr2)	Boolean	Liefert genau dann G true, wenn ein Wert der Art artexpr1 mit der Funktion value(variable, artexpr2) in einen Wert der Art artexpr2 wandelbar ist. Voraussetzung für die Umwandlung ist, dass die Arten zur selben Basisart gehören und kein Genauigkeitsverlust eintritt. Diese Beziehung zwischen 2 Arten ist asymmetrisch. artexpr2 ist zu artexpr1 kompatibel, dies bedeutet aber nicht, dass artexpr1 auch immer zu artexpr2 kompatibel ist. Bei Kompatibilität können Werte von Variablen der Art artexpr1 mit .kopieren an Variablen der Art artexpr2 zugewiesen werden. Allgemeine Regeln zur Kompatibilität: • Exakt identische Arten sind natürlich immer gegenseitig kompatibel zueinander. • Im Allgemeinen sind alle Unterarten einer Hauptart untereinander kompatibel, sofern der Wertebereich der neuen Art mindestens den Wertebereich der Originalart umfasst. • Die Verweisarten P*, PS* und ID* sind bei gleicher Unterart (Datenbank/Gruppe) gegenseitig kompatibel. • Mehrzeilige Texte sind zu einzeiligen kompatibel, aber nicht umgekehrt Der Wertebereich wird größer, U status = ja <pre style="background-color: #e0e0e0; padding: 5px;">.type bool xbstatus .formula U xbstatus = 'F compatible("R7.2", "R8.3")' ?true ? 'U xbstatus' = "ja" ?false ? 'U xbstatus' = "nein"</pre> Der Wertebereich wird kleiner, U status = nein Beispiel: 12345678.12 passt nicht in eine Variable der Art R7.2 <pre style="background-color: #e0e0e0; padding: 5px;">.type bool xbstatus .formula U xbstatus = 'F compatible("R8.3", "R7.2")' ?true ? 'U xbstatus' = "ja" ?false ? 'U xbstatus' = "nein"</pre> Es ändert sich nur die Darstellung, U status = ja

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
value(variable[, artexpr])	verschieden	Liefert den Inhalt einer Variable mit einer bestimmten Art. Dabei ist es möglich, den Wert mit einer beliebigen, kompatiblen Art (siehe <code>compatible(artexpr1, artexpr2)</code>) auszugeben. Die Variable xrd hat die Art R7.2. Mit dem folgenden Funktionsaufruf kann sie mit vier statt mit zwei Nachkommastellen ausgegeben werden: <pre>.type R7.2 xrd .formula U xrd = 123,77 'F value(U xrd, "R7.4")'</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
tostring(variable[, artexpr[, formatstring]])	String	Stellt den Inhalt der Variable variable dar und verwendet dabei die optionale Art artexpr zur Darstellung. Dies ist z.B. bei Verweisen nützlich, um stets die Identnummer auszugeben, unabhängig von der konkreten Art des Feldes. Im Gegensatz zu value(variable, artexpr) ist das Ergebnis dieser Funktion immer ein String. Darstellung von Real-Zahlen mit abweichender Art: <pre>.type R7.2 xr .formula U x = 1.25 'F tostring(U x, "R9.4")' 1.2500</pre> Mit dem Argument formatstring können Sie optional eine Formatangabe übergeben, wie in Explizite Typumwandlungen (Typcast) von Variablen beschrieben. <pre>.hole teil "12345" .art text xfmt .. Zeichenfolge L="0" erzeugen .formel xfmt = "L=" + G dblquote + "0" + G dblquote 'F tostring(H bsart, , xfmt)'</pre> Diese Zeilen würden für einen Artikel mit Beschaffungsart "Eigenfertigung" den Text "(69)" ausgeben, also die sprachunabhängige Darstellung des Wertes.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
tovalue(stringexpr, artexpr[, Fehler])	verschieden	Wandelt den String stringexpr in einen Wert einer bestimmten Art um. Dieser Wert kann an allen Stellen verwendet werden, wo Variablen dieser Art zulässig sind. Wird für den Parameter Fehler false übergeben, bricht die Funktion bei einem ungültigen Wert nicht ab, sondern liefert als Ergebnis den Leerwert zur Art. Bei bestimmten Arten kann man sich hiermit die vorherige Prüfung mit F isvalue sparen und am Ergebnis erkennen, ob die Umwandlung erfolgreich war. Wandelt den String 12.12.14 in einen Wert der Art GD um <pre>'F tovalue("12.12.14", "GD")' = 12.12.2014</pre> Der Versuch, den String 12.13.14 (ungültiges Datum) in einen Wert der Art GD zu wandeln, bricht mit Fehler ab. <pre>'F tovalue("12.13.14", "GD")'</pre> <pre>'F tovalue("12.13.14", "GD", true)'</pre> Der Versuch, den String 12.13.14 (ungültiges Datum) in einen Wert der Art GD zu wandeln, liefert ein leeres Datum. <pre>'F tovalue("12.13.14", "GD", false)'</pre> Der Versuch, den String 12.13.14 (ungültiges Datum) einer Variablen der Art GD zuzuweisen, bricht mit Fehler ab. <pre>.type GD2 xgd</pre> <pre>.formula xgd = "12.13.14"</pre> Der Versuch, den String abc in die Art I9 zu wandeln, bricht mit Fehler ab. <pre>'F tovalue("abc", "I9")'</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
isvalue(stringexpr, artexpr)	Boolean	Liefert die Information, ob ein Text eine gültige Eingabe für ein Feld einer bestimmten Art ist bzw. ob der Text an ein Feld dieser Art zugewiesen werden könnte. Liefert <code>F isvalue</code> den Wert <code>true</code>, so wird <code>F tovalue</code> erfolgreich sein und keinen Fehler erzeugen. Die Prüfung, ob der String <code>10.11.12</code> zur Art <code>GD</code> passt, liefert true. <pre>'F isvalue("10.11.12", "GD")'</pre> Der Versuch, den String <code>10.11.12</code> einer Variablen der Art <code>GD</code> zuzuweisen, funktioniert. <pre>.type GD2 xgd .formula U xgd = "10.11.12"</pre> Die Prüfung, ob der String <code>12.13.14</code> zur Art <code>GD</code> passt, liefert false. <pre>'F isvalue("12.13.14", "GD")'</pre> Der Versuch, den String <code>12.13.14</code> einer Variablen der Art <code>GD</code> zuzuweisen, bricht mit Fehler ab. <pre>.type GD2 xgd .formula U xgd = "12.13.14"</pre> Die Prüfung, ob der String <code>abc</code> zur Art <code>I9</code> passt, liefert false. <pre>F isvalue("abc", "I9")</pre> Der Versuch, den String <code>abc</code> einer Variablen der Art <code>int</code> zuzuweisen, bricht mit Fehler ab. <pre>.type int xint .formula U xint = "abc"</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
expr(expr)	verschieden	Ermittelt den Wert des Ausdrucks und bestimmt dessen Art. Das Ergebnis des Funktionsaufrufs kann beliebig weiterverwendet werden: Zuweisung an eine Variable, Verwendung in einem anderen Ausdruck, Verwendung in einem FO-Kommando oder einfach Ausgabe auf dem Bildschirm. Beispiel: Vereinfachter Systemaufruf Mit Hilfe der expr-Funktion kann ein Systemaufruf in einer Zeile hingeschrieben werden, hier z.B. das Umbenennen einer Datei, deren alter und neuer Name in den FO-Variablen xfile1 und xfile2 steht: <pre>.type text U xfile1 U xfile2 .formula U xfile1 = "owfohelp/HALLO.FOP" .formula U xfile2 = "owfohelp/HELLO.FOP" .system 'F expr("mv " + U xfile1 + " " + U xfile2)'</pre>
condexpr(boolexpr, expr1, expr2)	verschieden	Ähnlich expr, aber mit bedingter Auswertung eines von zwei Ausdrücken. Die Funktion ermittelt zuerst den Wert des booleschen Ausdrucks boolexpr. Ist das Ergebnis G true, wird das Ergebnis des Ausdrucks expr1 und dessen Art bestimmt, ansonsten das Ergebnis und die Art von expr2. Beispiel: Bedingter Text Die Anzahl gefundener Datensätze soll sprachlich korrekt ausgegeben werden. <pre>.type int xicount .formula U xicount = 3 result: 'U xicount' 'F condexpr(U xicount = 1, "record", "records")' found.</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
max(expr1, expr2) min(expr1, expr2)	verschieden	Liefert Maximum bzw. Minimum von zwei Ausdrücken. Die beiden Argumente müssen vergleichbar sein, d.h. der Operator "<" muss für den Vergleich der beiden Argumente zulässig sein. Das Maximum könnte man so ausdrücken: <pre>.type int xia xib .formula U xia = 4 .formula U xib = 5 'F condexpr(U xia > U xib, U xia, U xib)'</pre> So ist es deutlicher und einfacher: <pre>.type int xia xib .formula U xia = 4 .formula U xib = 3 'F max(U xia, U xib)'</pre>
eval(stringexpr)	verschieden	Die Funktion eval ermittelt zuerst den Wert des übergebenen Stringausdrucks und wertet diesen String dann als normalen Ausdruck aus. Dadurch ist es möglich, dynamische Ausdrücke auszuwerten, die z.B. erst aufgrund einer Benutzereingabe zustandekommen. Diese Funktion ist im Prinzip identisch mit dem UNIX-Shell-Kommando eval. Beispiel: Dynamische Auswertung von Variablen <pre>.type text xtexpr .read "insert expression: " U xtexpr expression: 'U xtexpr' - value: 'F eval(U xtexpr)'</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
length(expr)	Integer	Länge Liefert die Länge der textuellen Darstellung des Ausdrucks expr. Dazu muss expr kein Stringausdruck sein, sondern die Stringdarstellung wird automatisch ermittelt. Der Wert dieser Funktion ist für Stringausdrücke identisch dem Wert '+(expr)', kann aber im Gegensatz dazu in einer Ersetzung innerhalb von Apostrophen mit ausgegeben werden. Zudem kann mit der Funktion die Länge beliebiger Ausdrücke ermittelt werden. Länge des Inhalts einer Real Variablen: <pre>.type r7.3 xnumber .formula U xnumber = 123,45 length of 'U xnumber' is 'F length(U xnumber)'</pre> Länge des Inhalts einer Textvariablen: <pre>.type text xttext .formula U xttext = 123,45 length of 'U xttext' is 'F length(U xttext)'</pre> Länge des Ergebnisses einer Berechnung: <pre>.type real xrresult .formula U xrresult = 7 * 8 length of 'U xrresult' is 'F length(U xrresult)'</pre> Länge eines Textausdrucks: <pre>.type text xt1 xt2 xt3 .formula U xt1 = "composed" .formula U xt2 = " text" .formula U xt3 = U xt1 + U xt2 length of 'G singleQuotMark' 'U xt3' 'G singleQuotMark' is 'F length(U xt3)'</pre>

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
nextelem(variable, Indexteil[, vorwaerts])	String	Ein initialisiertes Array kann vorwärts (ascending) bzw. rückwärts (descending) durchlaufen werden. Um die Richtung zu bestimmen, wird die Meta-Funktion F nextelem verwendet. • Mit der Meta-Funktion F nextelem(variable, index, vorwaerts) kann auf die Array-Elemente auf- bzw. absteigend zugegriffen werden. • Wenn ein Array-Element vorhanden ist, dann wird der Index des Array-Elements als String zurückgegeben. • Wenn kein Array-Element vorhanden ist, dann wird ein Leerstring zurückgegeben. • Mit Variable und Index kann auf den Inhalt zugegriffen werden. Das Beispiel gibt den Inhalt der Array-Variablen in absteigender Reihenfolge aus. <pre> .type text xtindex .. .type text xarray1 xarray2 xarray3 xarray4 xarray5 .. .formula U xarray1 = "These" .formula U xarray2 = "are" .formula U xarray3 = "some" .formula U xarray4 = "array" .formula U xarray5 = "elements" .. !NEXT .. forward/backward .formula U xtindex = F nextelem(U xarray, U xtindex, G false) .continue END ? U xtindex = "" 'U xtindex' 'U xarray.xtindex' .continue NEXT .. !END end </pre>



Die Metafunktionen sind beeinflusst durch den Zustand, der mit **.setze foextern** gesetzt wurde. Das gilt vor allem, wenn nach Eigenschaften gefragt wird, die für die FO und den Benutzer unterschiedlich sein können, z.B. für den Schreibschutz. Z.B. wirkt der per FO mit **.schutz** gesetzte Schutz nur gegen den Benutzer, nicht gegen die FO. Dies

muss beachtet werden, will man den Schutz mit **F|modifiable** erfragen.

defined(variable) - Ist eine Variable definiert?

Außerdem besteht die Möglichkeit der Abfrage, ob eine FO-Variable definiert ist. Sie ist hilfreich in Unterprogrammen, die nicht in jedem Fall von einem Hauptprogramm aufgerufen werden, das die Variable definiert. Eine weitere wichtige Anwendung ist die Abfrage von Datensatzvariablen. Diese können jederzeit mit Hilfe der Variablentabelle definiert und auch wieder entfernt werden, sind also sowohl kunden- als auch versionsabhängig. Die Abfrage erleichtert in diesem Fall das Schreiben von portablen austauschbaren und rückwärtskompatiblen FOPs, die auf die nicht definierte Variable mit einer geeigneten Fehlerbehandlung reagieren können. Schließlich kann die Abfrage auch verwendet werden um festzustellen, ob eine Variable nur in einigen Gruppen einer zu durchlaufenden Datenbank definiert ist.

Die Abfrage erfolgt über die boolesche Funktion **defined">defined(Variablenname)**, die in Formeln oder Bedingungen verwendet werden kann. Die Abfrage liefert **G|true**, wenn die Variable **'variablenname'** definiert ist. Dabei kann entweder ein einfacher Variablenname angegeben werden oder die Schreibweise "Objekt|Variablenname" verwendet werden. Letzteres ist nahezu immer zu empfehlen. Auch die Angabe von Arrays oder Verknüpfungen mit dem Hochzeichen (^) ist zulässig.

Um Irrtümer oder Verwechslungen zu verhindern, darf die Variable nicht abgekürzt werden. Sie gilt sonst als nicht definiert.

Beispiel: Prüfen, ob eine Variable definiert ist

Das nachfolgende FOP prüft, ob die benutzerdefinierte Variable **U|xsum** definiert ist und gibt **U|xsum defined** aus.

Wenn die Definitionszeile der Variable **U|xsum** auskommentiert wird, gibt das Programm **U|xsum not defined** aus.

Eine Variable kann in einem Unterprogramm definiert und im Hauptprogramm verwendet werden.

```
.type real xsum      ? F|defined(U|xsum) = false
?U|xsum not defined  ? F|defined(U|xsum) = false
?U|xsum defined      ? F|defined(U|xsum) = true
```

Beispiel: Prüfen, ob eine Array-Variable definiert ist

Das nachfolgende FOP prüft, ob die benutzerdefinierten Array-Variablen **U|xt1** und **U|xt2** definiert sind.

Wenn beide definiert sind, wird **U|xt.1 definiert>** und **U|xt.2 definiert** ausgegeben.

Wenn Sie auf eine Array-Variable zugreifen, die noch nicht definiert wurde, z.B. **U|xt.3**, wird **U|xt.3 NICHT definiert** ausgegeben.

Eine Variable kann in einem Unterprogramm definiert und im Hauptprogramm verwendet werden.

```
.type int xi          ? F|defined(U|xi) = false
.. definition of array variable xt1 and xt2
.type text xt1        ? F|defined(U|xt1) = false
.type text xt2        ? F|defined(U|xt2) = false
..
.. check the definition of array variable xt.1
.formula U|xi = 1
.continue NOTDEF ? F|defined(U|xt.xi) = false
U|xt.'U|xi' defined
..
.. check the definition of array variable xt.2
.formula U|xi = 2
.continue NOTDEF ? F|defined(U|xt.xi) = false
U|xt.'U|xi' defined
..
.. check the definition of array variable xt.3
.formula U|xi = 3
.continue NOTDEF ? F|defined(U|xt.xi) = false
U|xt.'U|xi' defined
..
.continue END
!NOTDEF
U|xt.'U|xi' NOT defined
!END
.end
```

Ein boolescher Ausdruck wird im FOP bis zum Ende ausgewertet, selbst wenn das Endergebnis bereits aus einem Zwischenergebnis erschlossen werden könnte. Im Zusammenhang mit **defined()** ist das bedeutungsvoll. Die folgende Programmzeile z.B. ist inkorrekt:



```
.println ja ? defined(x) & x>10
```

Wenn x nicht definiert ist, wird die Bedingung **x > 10** dennoch geprüft, und diese Prüfung scheitert aufgrund eines Fehlers, da **x** nicht definiert ist.

Verknüpfungen und defined()

Die Verwendung von Verknüpfungen bei **defined()** ist zulässig.

Beispiel: Prüfen, ob ein Feld ein Verweisfeld ist

Das nachfolgende FOP prüft, ob es sich bei dem verwendeten Verweis um einen echten Verweis handelt.

```
.select Customer "4"
```

```
?H|stateOfTaxOffice is reference ? F|defined(H|stateOfTaxOffice^id) = true
?H|town is reference ? F|defined(H|town^id) = true
```



defined(verweis^feldname)

Wenn ein Verweisfeld **verweis** leer ist, gilt folgende Regelung:

Der Ausdruck **defined(verweis^feldname)** liefert genau dann den Wert **G|true**, wenn in mindestens einer Gruppe, auf die der Verweis **verweis** zeigen kann, ein Feld **feldname** existiert.

Es kann also durchaus passieren, dass bei einem leeren Feld **verweis** der Ausdruck **defined(verweis^feldname)** **G|true** liefert, aber nach Zuweisung eines Wertes an **verweis** der selbe Ausdruck **G|false** liefert.

Finanzbuchhaltungsfunktionen

Finanzbuchhaltungsfunktionen dienen der Information über Währungskurse und Geschäftsjahre.

Währungsfunktionen

Für die Kursermittlung, basierend auf der **Kurstabelle**, gibt es die zwei Funktionen **fxrate** und **fxamount**.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
fxrate(verweisexpr, dateexpr, verweisexpr)	Real	Liefert bis zu 6 Nachkommastellen.
fxamount(verweisexpr, numexpr, dateexpr, verweisexpr, boolexpr)	Real	Liefert unterschiedlich viele Nachkommastellen. Dies ist abhängig von den Rundungsangaben.

Die Parameter der Funktionen **fxrate** und **fxamount** finden Sie in der folgenden Tabelle. Alle Parameter sind optional.

Parameterreihenfolge	Argument	Standardparameter	Funktionsname
Ausgabewährung (=Ergebniswährung)	verweisexpr	Buchungswährung am Umrechnungsdatum	fxrate und fxamount
Eingabebetrag in Eingabewährung	numexpr	1.0	fxamount
Umrechnungsdatum	dateexpr	Systemdatum	fxrate und fxamount
Eingabewährung	verweisexpr	Buchungswährung am Umrechnungsdatum	fxrate und fxamount

Parameterreihenfolge	Argument	Standardparameter	Funktionsname
Ergebnisbetrag gemäß Ausgabewährung runden?	<code>boolexpr</code>	ja	<code>fxamount</code>

Datenkonsistenz

Durch einen Mehrfachaufruf von `fxrate` oder `fxamount` innerhalb einer Schleife mit denselben Parametern besteht die Möglichkeit, dass sich der Währungskurs während der Auswertung verändert, wenn dieser durch einen anderen Benutzer zwischenzeitlich geändert wird.

Durch die Ermittlung des Kurses vor der Schleife ist dessen Konsistenz während des Schleifendurchlaufs gewährleistet.

Aufrufbeispiele mit Ergebnissen

Systemdatum = 07.12.2009

Kurse:

27.11.2009 1 EUR = 1.051423 USD (Systemdatum-10 Tage)
 28.08.2009 1 EUR = 1.063341 USD (Systemdatum-100 Tage)
 02.01.2008 1 EUR = 1.048999 USD
 02.01.2008 1 EUR = 1.655328 CAD
 02.01.2008 1 EUR = 1.451500 CHF / 100 CHF = 68.894250 EUR

```
.type text xtcurrency
.formula U|xtcurrency = "EUR"
'F|fxrate()'
'F|fxrate(,,)'
'F|fxrate("",,,)'
'F|fxrate("USD",,,)'
'F|fxrate(,,,"usd")'
'F|fxrate("USD","26.11.2011",)'
'F|fxrate("USD",-100,)'
'F|fxrate("USD",-101,)'
USD -> EUR: 'F|fxrate("USD",-100,"EUR")'
EUR -> USD: 'F|fxrate("EUR",-100,"USD")'
'F|fxrate("USD",-100,"cad")'

-----
'F|fxamount()'
'F|fxamount("USD",100,-100,"cad")'
'F|fxamount("USD",100,-100,"cad",false)'
'F|fxamount("USD",100,-100,"cad",)'
'F|fxamount(,100,"27.08.2011","cad")'
'F|fxamount("cad",100,"27.08.2011", U|xtcurrency)'
'U|xtcurrency'
'F|fxrate("cad",,"chf")'
'F|fxamount("cad",100,"27.8.",,"chf")'
```

Laufzeitverhalten

Die Funktion **fxamount** verwendet intern mehrere Unterfunktionen mit Datenbankzugriffen. Im Einzelnen wird darin Folgendes implementiert:

Hier erfolgen immer zwei Datenbank-Lesezugriffe:

```
.formula U|xrcurrencyrate = F|fxrate(U|xtinputcurrency, U|xddate, U|xtoutputcurrency)
```

Hier erfolgt immer ein Datenbank-Lesezugriff:

```
.type PS94:1 xrefcurrency
.type R7.2 xinputvalue xoutputvalue
.type bool xbround
.type int xdecimalplaces
.formula U|xbround = false
.formula U|xrefcurrency = "EUR"
.formula U|xinputvalue = 10.00
..Find out currency decimal places
.formula U|xdecimalplaces = U|xrefcurrency^decPlaces
.formula U|xdecimalplaces = 1 ? U|xdecimalplaces=0
..currency translation with optional rounding
.formula U|xoutputvalue = U|xinputvalue * U|xrcurrencyrate
.formula U|xoutputvalue = F|round(U|U|xinputvalue * U|xrcurrencyrate, U|xdecimalplaces)?
U|xbround = G|true
output value: 'U|xoutputvalue'
```

Insbesondere die zwei Umrechnungskurse für Eingabewährung und Ausgabewährung werden dabei immer aus der Datenbank ermittelt. Deshalb sollte **fxamount** nur dann eingesetzt werden, wenn diese Kursermittlung explizit immer benötigt wird, ansonsten werden möglicherweise sehr viele überflüssige Datenbankzugriffe gemacht.

Beispiel:

Ruft man **fxamount** z.B. für jede Rechnungszeile auf, erfolgen für jede Zeile wenigstens 3 überflüssige Datenbankzugriffe, da eine Rechnung nur einen einzigen relevanten Kurs und eine einzige Währung hat, die beide im Rechnungskopf stehen.

Gibt es einen Kurs oder einen anderen Wert, der für eine ganze Schleife gilt, sollte dieser unbedingt schon vor der Schleife ermittelt und in einer **U|**-Variablen gemerkt werden! Hier das Beispiel dazu:

Vor der Schleife:

Hier erfolgen immer zwei Datenbank-Lesezugriffe:

```
.formula U|xrcurrencyrate = F|fxrate(U|xtinputcurrency, U|xddate, U|xtoutputcurrency)
```

Hier erfolgt immer ein Datenbank-Lesezugriff:

```
.type PS94:1 xrefcurrency
.type int xdecimalplaces
.formula U|xrefcurrency = "EUR"
.formula U|xdecimalplaces = U|xrefcurrency^decPlaces
.formula U|xdecimalplaces = 1 ? U|xdecimalplaces=0
```

In der Schleife:

```
.type real xroutputvalue xinputvalue xrxcurrencyrate
.type int xdecimalplaces
.type bool xbround
.formula U|xrouputvalue = U|xinputvalue * U|xrxcurrencyrate
.formula U|xrouputvalue = F|round(U|U|xinputvalue * U|xrxcurrencyrate, U|xdecimalplaces)?
U|xbround = G|true
```

Man sollte davon ausgehen, dass die Funktion **fxrate** immer zwei Datenbankzugriffe ausführt. Deshalb sollten Sie abwägen, ob der Aufruf unbedingt innerhalb einer Schleife benötigt wird (vgl. [Beispiel fxamount](#)).

Funktionen für Geschäftsjahre

Im Folgenden werden die Funktionen für Geschäftsjahre beschrieben. Wenn ein Geschäftsjahr in einer FO-Geschäftsjahresfunktion verwendet wird, das nicht im Gültigkeitsbereich der Geschäftsjahrestabelle in [Termine zeigen] liegt, erhalten Sie folgende Fehlermeldung: **Geschäftsjahr im Termindatensatz nicht definiert..**

Folgende Argumente stehen für diese Funktionen zur Verfügung:

Argumente	Bedeutung	FO-Art
gj	Geschäftsjahr	GJ oder String
gm	Geschäftsmonat	Integer
gd	gregorianisches Datum	GD
mit_nbm	mit Nachbuchungsmonaten	Bool

Funktionsname und -argumente	Art des Rückgabewerts	Beschreibung
F fystartdate(gj, [gm]) F fyenddate(gj, [gm])	GD2	Funktionen zur Ermittlung des Start- und Enddatums eines Geschäftsmonats
F currentfy()	GJ	Funktion zur Ermittlung des aktuellen Geschäftsjahres
F nextfy(gj) F prevfy(gj)	GJ	Funktionen zur Ermittlung des nachfolgenden und vorhergehenden Geschäftsjahres

Funktionsname und -argumente	Art des Rückgabewerts	Beschreibung
F lastfy() F firstfy()	GJ	Funktionen zur Ermittlung des letzten und ersten Geschäftsjahres
F getfy(gd)	GJ	Funktion zur Ermittlung des Geschäftsjahres zu einem Datum  Das aktuelle Kalenderjahr entspricht nicht zwangsläufig dem aktuellen Geschäftsjahr. getfy(<today>) <> currentfy()
F getfm(gd[, mit_nbm])	Integer	Funktion zur Ermittlung eines Geschäftsmonats zu einem Datum. Wenn mit_nbm=true, werden Nachbuchungsmonate berücksichtigt. Außerdem ist dann <datum> ein Datum aus einem vollständig abgeschlossen Geschäftsjahr.
F fycmp(gj1, gj2)	-1 wenn gj1 < gj2 0 wenn gj1 = gj2 1 wenn gj1 > gj2	Vergleich von Geschäftsjahren

Abkürzungen:

fy	fiscal year
fm	fiscal month
cmp	compare
prev	previous

Geschäftsjahresfunktion xgj=20	Ergebnis
F fystartdate(xgj,1)	01.01.20
F fyenddate(xgj,1)	31.01.20
F fystartdate(xgj,3)	01.03.20
F fyenddate(xgj,3)	31.03.20
F fystartdate(xgj,12)	01.12.20
F fyenddate(xgj,12)	31.12.20
F fysortdat(xgj,1)	20200101

Geschäftsjahresfunktion xgj=20	Ergebnis
F currentfy()	10
F nextfy(xgj)	11
F prevfy(xgj)	09
F lastfy()	13
F firstfy()	09
F fystartdate(xgj)	01.01.20
F fyenddate(xgj)	31.12.20
F getfy("20190331")	10
F getfm("20190331")	3
F getfm("20190331", G true)	13
F getfy("20200331")	10
F getfm("20200331")	3
F getfm("20200331", G true)	3
F getfy("20210331")	11
F getfm("20210331")	3
F getfm("20210331", G true)	3
F getfy("20220331")	12
F getfm("20220331")	3
F getfm("20220331", G true)	3
F getfy("20230331")	13
F getfm("20230331")	3
F getfm("20230331", G true)	3

Objektsperrenfunktionen

Die Objektsperrenfunktionen ermitteln die Sperrwirkung bei Eintrag eines sperrbaren Objektes in ein Verweisfeld oder für das Verhalten des Systems an bestimmten Prozessstellen.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung																										
<code>F reflockingeffect(dbno, groupno, tablefield, fieldname, object)</code>	Wert der Art A334 (Sperrwirkung)	Liefert für das angegebene Objekt die konfigurierte Sperrwirkung (keine Sperre, Gesperrt, Hinweis) des angegebenen Verweisfeldes. Das Verweisfeld wird über Datenbanknummer, Gruppennummer, Tabellenkennzeichen und Feldnamen identifiziert. Die Funktion hat folgende Argumente: <table><tr><th>Argument</th><th>Typ</th><th>Beschreibung</th></tr><tr><td>dbno</td><td>integer</td><td>Datenbanknummer</td></tr><tr><td>groupno</td><td>integer</td><td>Gruppennummer innerhalb der Datenbank</td></tr><tr><td>tablefield</td><td>bool</td><td>Kennzeichen, ob sich das Verweisfeld im Tabellenteil der Gruppe befindet</td></tr><tr><td>fieldname</td><td>string</td><td>Feldname des Verweisfelds</td></tr><tr><td>object</td><td>VP88</td><td>Verweis auf das zu prüfende, sperrbare Objekt</td></tr></table> Mögliche Rückgabewerte der Art A334 (Sperrwirkung) sind: <table><tr><th>Wert</th><th>Bedeutung</th></tr><tr><td>Leer</td><td>Keine Sperre vorhanden</td></tr><tr><td>Gesperrt</td><td>Harte Sperre</td></tr><tr><td>Hinweis</td><td>Weiche Sperre</td></tr></table> Beispiel: Artikel <pre>F reflockingeffect(3, 24, G true, "artikel", "2 12345")</pre> Dieser Aufruf prüft, ob der Artikel 12345 für den Eintrag in das Feld (artikel) in der Tabelle einer Verkaufsrechnung gesperrt ist.	Argument	Typ	Beschreibung	dbno	integer	Datenbanknummer	groupno	integer	Gruppennummer innerhalb der Datenbank	tablefield	bool	Kennzeichen, ob sich das Verweisfeld im Tabellenteil der Gruppe befindet	fieldname	string	Feldname des Verweisfelds	object	VP88	Verweis auf das zu prüfende, sperrbare Objekt	Wert	Bedeutung	Leer	Keine Sperre vorhanden	Gesperrt	Harte Sperre	Hinweis	Weiche Sperre
Argument	Typ	Beschreibung																										
dbno	integer	Datenbanknummer																										
groupno	integer	Gruppennummer innerhalb der Datenbank																										
tablefield	bool	Kennzeichen, ob sich das Verweisfeld im Tabellenteil der Gruppe befindet																										
fieldname	string	Feldname des Verweisfelds																										
object	VP88	Verweis auf das zu prüfende, sperrbare Objekt																										
Wert	Bedeutung																											
Leer	Keine Sperre vorhanden																											
Gesperrt	Harte Sperre																											
Hinweis	Weiche Sperre																											

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung															
<code>F processlockingeffect(Prozessstelle, verwexpr)</code>	Wert der Art A334 (Sperrwirkung)	Ermittelt die Sperrwirkung des verwiesenen Objektes an der Prozessstelle. Die Funktion hat folgende Argumente: <table><tr><th>Argument</th><th>Typ</th><th>Beschreibung</th></tr><tr><td>Prozessstelle</td><td>A336</td><td>Prozessstelle</td></tr><tr><td>verwexpr</td><td>VP88</td><td>Verweisausdruck auf sperrbares Objekt</td></tr></table> Mögliche Rückgabewerte der Art A334 (Sperrwirkung) sind: <table><tr><th>Wert</th><th>Bedeutung</th></tr><tr><td>Leer</td><td>Keine Sperre vorhanden</td></tr><tr><td>Gesperrt</td><td>Harte Sperre</td></tr></table>  Derzeit kann die Sperrwirkung Hinweis hier nicht vorkommen. <pre>+.type P2:4 U zupos +.assign U zupos = "7" +'G mehr' ja +'U zupos^sperrkonfiguration' Harte ZP-Sperre +'F processlockingeffect("Prozessstelle", U zupos)' Gesperrt</pre> Dieses Beispiel demonstriert das Funktionsergebnis Gesperrt für die Prozessstelle "Prozessstelle" für eine Zusatzposition.	Argument	Typ	Beschreibung	Prozessstelle	A336	Prozessstelle	verwexpr	VP88	Verweisausdruck auf sperrbares Objekt	Wert	Bedeutung	Leer	Keine Sperre vorhanden	Gesperrt	Harte Sperre
Argument	Typ	Beschreibung															
Prozessstelle	A336	Prozessstelle															
verwexpr	VP88	Verweisausdruck auf sperrbares Objekt															
Wert	Bedeutung																
Leer	Keine Sperre vorhanden																
Gesperrt	Harte Sperre																

Sonstige Funktionen

Verweise auf Einmalobjekte im Firmenstamm

Für Zugriffe auf Einmalobjekte im Firmenstamm steht folgende Funktion zur Verfügung:

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung																										
sysinfo(Gruppe[, Abbruch])	Verweis der Art P12:gruppe	Diese Funktion liefert einen Verweis, auf das (Einmal-)Objekt mit der Gruppe gruppe im Firmenstamm. Wenn das bezeichnete Objekt nicht existiert, • erfolgt ein Fehlerabbruch des FO-Programms, sofern der Parameter abbruch auf true steht, • liefert der Aufruf ein leeres Ergebnis ohne Fehlermeldung, sofern der Parameter abbruch auf false steht. Der abbruch-Parameter kann weggelassen werden. Er wird dann als true angenommen. Die Gruppe muss die Nummer einer Firmenstammgruppe sein, zu der es nur ein Objekt geben kann. Folgende Gruppen sind zulässig: <table><tr><td>0</td><td>Steuerungsumstellung</td></tr><tr><td>7</td><td>Kontenrahmen</td></tr><tr><td>9</td><td>Nummernzähler</td></tr><tr><td>10</td><td>Betrieb</td></tr><tr><td>12</td><td>Termine</td></tr><tr><td>13</td><td>Lohngruppen</td></tr><tr><td>14</td><td>Lohnarten</td></tr><tr><td>15</td><td>Bewertungskonfiguration</td></tr><tr><td>16</td><td>Kurstabelle</td></tr><tr><td>17</td><td>Materialzuschläge</td></tr><tr><td>18</td><td>Konfiguration</td></tr><tr><td>22</td><td>Zolltabelle</td></tr><tr><td>27</td><td>Verweisdarstellung</td></tr></table> Das Ergebnis des Aufrufs ist ein Verweis auf das angegebene Objekt. Es hat die Art P12:gruppe Liefert die Identnummer des Betriebsdatensatzes bzw. den Leertext, wenn kein Betriebsdatensatz existiert <pre>'F sysinfo(10, false)'</pre> Liefert die interne Lagergruppe aus dem Betriebsdatensatz. Das FO-Programm bricht mit einer Fehlermeldung ab, wenn der Betriebsdatensatz nicht	0	Steuerungsumstellung	7	Kontenrahmen	9	Nummernzähler	10	Betrieb	12	Termine	13	Lohngruppen	14	Lohnarten	15	Bewertungskonfiguration	16	Kurstabelle	17	Materialzuschläge	18	Konfiguration	22	Zolltabelle	27	Verweisdarstellung
0	Steuerungsumstellung																											
7	Kontenrahmen																											
9	Nummernzähler																											
10	Betrieb																											
12	Termine																											
13	Lohngruppen																											
14	Lohnarten																											
15	Bewertungskonfiguration																											
16	Kurstabelle																											
17	Materialzuschläge																											
18	Konfiguration																											
22	Zolltabelle																											
27	Verweisdarstellung																											

Forterro Deutschland Abas GmbH, Gartenstraße 67, 76135 Karlsruhe, Germany

15

Steuersequenzen zum Drucken

Die Funktion **F|printersequence(string)** liefert dem aktuellen Drucker die Steuersequenz mit dem angegebenen Namen. Damit sind Inline-Attribute möglich.

Beispiel: FOP

```
.input DRUCKER
Following text is 'F|printersequence("in.fett")'bold'F|printersequence("in.norm")'
```

Die Funktion liefert dann einen Wert, wenn zuvor als Gerät ein Drucker eingestellt wurde, z.B. mit **.eingabe drucker** und der Name der Sequenz, z.B. **in.fett**, existiert. Andernfalls liefert die Funktion einen Leerstring. Die Funktion selbst hat im Unterschied zum Kommando **.drucken** keine Effekte wie das Setzen der Schriftgröße. Man kann auch nur eine Steuersequenz angeben.



Die von **F|printersequence()** gelieferten Escape-Sequenzen laufen ganz normal über die Textausgabe und somit auch über die Zeichensatz-Konvertierung in termcap.zs. In termcap.zs sollten deshalb keine Zeichen konvertiert werden, die als einzelne Bytes in einer Escapesequenz vorkommen können.
In der von uns standardmäßig ausgelieferten termcap.zs befindet sich aber für fast jeden Drucker eine Konvertierung des Zeichens "\005" nach "\040". Dabei handelt es sich um das Zeichen "^E" für Seitenumbruch.
Entfernen Sie ggf. diese Konvertierung für Ihren Drucker, wenn Escape-Sequenzen das Zeichen "\005" enthalten.



Verwenden der alten Drucklösung

In termcap.txt ist das Ersatzzeichen "\177" statt der binären 0 in Escape-Sequenzen zu verwenden.

DMS: Funktion zur Ermittlung der Belegart

Die Funktion **F|getdoctype(verweisexpr, boolexpr)** ermittelt die Belegart, die zu dem in Parameter **verweisexpr** eingetragenen Datensatz passt. Der Parameter **boolexpr** ist optional und bestimmt, ob die Belegart zum Drucken (TRUE) oder für Barcode und Drag-and-drop (leer oder FALSE) gesucht werden soll.

Die Funktion berücksichtigt alle aktiven Belegarten, deren Kontext, Verwendung und Bedingung zum Datensatz passen und gibt die Belegart mit der höchsten ID-Nummer zurück.

```
.type VP99 xrechnung          ? _F|defined(U|xrechnung)
.type VP88:11 xbelegart       ? _F|defined(U|xbelegart)
.formula U|xrechnung = "3 999073"
..Document type for barcode/drag and drop
.copy U|xbelegart = F|getdoctype(U|xrechnung)
```

```
..Document type for printing
.copy U|xbelegart = F|getdoctype(U|xrechnung, TRUE)
```

Einheitenumrechnung

Folgende Umrechnungsfunktion zur Einheitenumrechnung steht zur Verfügung:

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
unitconvfactor(unitexprvon, unitexprnach)	Real	Diese Funktion liefert den Umrechnungsfaktor zwischen zwei Einheiten. Das Ergebnis ist der Faktor, mit dem ein Wert der Einheit unitexprvon multipliziert werden muss, um den Wert in Einheit unitexprnach umzurechnen. Zulässig sind nur Umrechnungen zwischen Einheiten derselben Dimension, ansonsten kommt es zu einer Fehlermeldung.

```
conversion factor (km - m): 'F|unitconvfactor("km", "m")'
```

Stornospezifische Detailursache übergeben

Der Funktion **F|isreversalcause** wird eine Detailursache als Parameter übergeben. Als Ergebnis liefert die Funktion **ISREVERSALCAUSE**:

- Ja, wenn es sich um eine stornospezifische Detailursache handelt,
- ansonsten Nein.

Die Detailursache kann entweder als sprachabhängiger Text oder als Variable der Art A29 oder als AJO-Aufzählungselement an die Funktion übergeben werden.

Status von Paketen prüfen

Mit der FO-Funktion **F|ispackageactive()** kann man prüfen, ob ein **Paket** aktiv ist. Diese FO-Funktion kann beim Customizing eingesetzt werden, um z.B. FO-Code-Bereiche ein- und ausschaltbar zu machen.

Die Funktion akzeptiert als Argument den Inhalt des Feldes **classname**, z.B. "MeinPaket". Dieser muss innerhalb der Datenbankgruppe 87:10 (Paket) eindeutig sein. **F|ispackageactive()** liefert **true**, wenn in der Datenbankgruppe 87:10 (Paket) ein Datensatz existiert, der im Feld **classname** einen Eintrag, im Beispiel "MeinPaket", enthält und wenn das Feld **(de)paketaktiv** auf **true** steht.

Ebenso können Sie den **Standardschlüssel** "Classname" über das Feld **(it)classname** in der Datenbankgruppe 87:10 verwenden.

Abfrage der Allein-Sperre (Single-User-Modus)

Mit der FO-Funktion `F|issingleusermode()` kann geprüft werden, ob die Allein-Sperre von dem Benutzer selbst gehalten wird. Dies ist insbesondere für Administratoren relevant, um während Wartungsarbeiten oder beim Customizing den aktuellen Sperrstatus zu prüfen.

Funktionsname und -argumente	Typ des Rückgabewerts	Beschreibung
issingleusermode()	Boolesch	Die Funktion liefert true , wenn die Allein-Sperre vom aktuellen Benutzer gehalten wird. Sie liefert false , wenn keine Allein-Sperre aktiv ist. Die Funktion akzeptiert keine Argumente.

```
.continue BINALLEIN ? F|issingleusermode() = G|true
? Diese Funktion ist nur im Allein Modus ausführbar
.ende 2
!BINALLEIN
? Admin Funktionen können ausgeführt werden
...
```


FO-Kommandos

FO-Kommandos beginnen immer mit einem Punkt am Zeilenanfang. Man spricht deshalb auch von Punktbefehlen. Nach dem Punkt muss eines der definierten und in diesem Kapitel beschriebenen Kommandowörter folgen.

Das Kommando für den Seitenvorschub lautet z.B. `.page` (deutsch: `.seite`). Die Befehlssprache kann im Programmkopf festgelegt werden (siehe dazu [..!interpreter](#)).

Zu der Übersicht aller Kommandos gelangen Sie auf verschiedenen Wegen:

- Über den Menüeintrag **Schnellreferenzen** in der Kopfleiste des Dokumentationsportals gelangen Sie direkt zur [Quick Reference](#) der FO-Programmierschnittstelle.
- Sie können sich jederzeit die vollständige Liste aller FO-Kommandos sowie Listen aller Parameter und aller Optionen mit dem Programm `getfocmd` auf Shellebene generieren lassen.

Kommandowörter dürfen nicht abgekürzt werden. Dadurch sind die FO-Programme besser lesbar, außerdem lassen sie sich von Editoren mit Syntax-Highlighting besser unterstützen.

Für alle Kommandos gilt: Bei fehlerhafter Syntax des Kommandos bricht das FOP ab.

In den folgenden Kapiteln werden die FO-Kommandos und ihre Optionen beschrieben.

Hinweise zur Notation

In den Abschnittsüberschriften ist jedes Kommando in den FO-Sprachen Deutsch und Englisch notiert:

.absatz / .para - Absatzschutz Im Index finden Sie unter der Rubrik "FO-Kommandos" sowohl die deutschsprachigen als auch die englischsprachigen FO-Kommandos aufgelistet.

```
...  
.add  
.absatz  
.aktion  
...  
.output  
.page  
.para  
.pc.clip  
...
```

Zu Beginn jeden Kapitels ist die Syntax des FO-Kommandos in Kurzform aufgeführt. Wenn ein entsprechender Gebrauch zulässig ist, steht an erster Stelle das Kommando ohne Attribute, darunter mit Argumenten und Optionen. Im Anschluss daran werden die weiteren Verwendungsmöglichkeiten aufgeführt.

```
.hole  
.hole einkaufs druckbaren  
.hole einkaufs offene
```

In Kommandonamen und Optionen spielen Groß/Kleinschreibung keine Rolle; ferner sind dort statt der deutschen **Sonderzeichen** Ä, Ö, Ü und ß auch ihre Umschreibung **AE**, **OE**, **UE** bzw. **SS** möglich.

Bei der **syntaktischen** Beschreibung der Kommandos werden folgende Notationen verwendet:

Notation		Bedeutung	Beispiele
	Großbuchstaben	nicht variable Angabe, z.B. Kommando, Schlüsselwort oder Option	<code>.hole einkaufs druckbaren</code>
<code>" "</code>	Anführungszeichen	Kennzeichnen von Zeichenketten	<code>"Hallo"</code>
<code>...</code>	drei Punkte	Die Angabe kann aus mehreren Teilen bestehen; meist ein Text, der mehrere Wörtern und Satzzeichen enthalten darf.	<code>Text...</code>
<code>' '</code>	Apostrophe	Platzhalter für eine der folgenden Möglichkeiten: * beliebige Variable eingeschlossen in Apostrophe, wobei deren Inhalt an das Kommando übergeben wird * Text in Anführungszeichen, falls bei dem Kommando inhaltlich zulässig * Zahl, falls bei dem Kommando inhaltlich zulässig	<code>'xnummer'</code> <code>"Hallo"</code> <code>256</code>
<code>< ></code>	spitze Klammern	Platzhalter für eine Angabe ohne Apostrophe oder Anführungszeichen	<code>xnummer</code>
<code>[]</code>	eckige Klammern	optionale Angabe, kann weggelassen werden	<code>[MASKE]</code> <code>['Überschrift']</code>
<code> </code>	Pipe	Oder (genau eine der angegebenen Möglichkeiten)	<code>KOPF ZEILE</code> <code>+ - ++ --</code>

Interpreter-Einstellungen

..!INTERPRETER / ..!INTERPRETER - Arbeitsweise des FO-Interpreters

.. !INTERPRETER	.. !INTERPRETER
deutsch	englisch
..! interpreter ..! interpreter [declaration] [german english] [englvar] [translate] [noabbrev]	..! interpreter ..! interpreter [declaration] [german english] [englvar] [translate] [noabbrev]
declaration verlangt, dass jede vom Benutzer definierte Variable vor ihrer Verwendung mit .art deklariert werden muss. german oder english legt die Befehlssprache des FOPs fest. englvar legt die Variablensprache des FOPs auf Englisch fest. Ist die Option translate gesetzt, wird bei einer Übersetzungsanforderung die Befehlssprache dieses Programms umgestellt. noabbrev verlangt, dass Variablen nicht abgekürzt werden.	

Die Einstellungen werden nicht durch einen FO-Befehl im herkömmlichen Sinne, sondern durch die Angabe von **..**!interpreter** in der ersten Zeile** des FOPs definiert. (Dies sieht auf den ersten Blick wie ein Kommentar aus. Jedoch beginnen tatsächliche Kommentare nicht zwangsläufig mit einem Punkt, sondern mit dem Zeichen, das aktuell **als Kommandozeichen definiert** ist.)

Nach der Einleitung **..**!interpreter**** können Optionen in beliebiger Reihenfolge folgen. Diese Optionen sind wie das Kommando unabhängig von der Bedien- und der FO-Befehlssprache. Sie werden nicht übersetzt und dürfen nicht in Variablen stehen.

Alle Optionen gelten ausschließlich für die aktuelle Programmdatei. Unterprogramme sind von ihnen also nicht betroffen. Daher können auch FOPs mit unterschiedlichen Interpretoreinstellungen problemlos zusammenarbeiten. z.B. kann ein FOP mit der Befehlssprache Englisch problemlos in Unterprogrammen mit Deutsch als Befehlssprache aufgerufen werden.

Die Option **declaration**

Jede vom Benutzer (also im Puffer '**U** ') definierte Variable muss vor ihrer Verwendung mit **.art** deklariert werden. Auf diese Weise lassen sich Tippfehler eher erkennen. Außerdem wird dadurch vermieden, dass der Typ einer Variablen implizit und möglicherweise anders als beabsichtigt festgelegt wird.



Mehrfache Deklarationen sind trotzdem zulässig. Daher können Unterprogramme, die zufällig den selben Variablennamen verwenden wie das Hauptprogramm, auch bei **DECLARATION** die gleichnamige Variable eines Aufrufers überschreiben. Bei mehrfachen Deklarationen gilt immer der zuerst angegebene Typ.

Wird die Option nicht genutzt, gilt keine Deklarationspflicht. Es empfiehlt sich jedoch, in jedes neu entwickelte FOP möglichst strenge Prüfungen einzubauen. Dies erleichtert die Entwicklung und den Test von Programmen und erhöht die Lesbarkeit.

Die Optionen german und english

Es darf nur eine der beiden Optionen, **german** oder **english** angegeben werden. Fehlt die Option oder wird in der FOP-Datei nicht mit **..!interpreter** gearbeitet, wird als Befehlssprache die im Konfigurationsdatensatz definierte FO-Sprache verwendet.



Wenn Sie FOPs zwischen verschiedenen Installationen austauschen wollen, ist es empfehlenswert, die Befehlssprache immer explizit anzugeben. Auch in generierten FOPs sollte in diesem Fall eine entsprechende erste Zeile aufgenommen werden. Das Programm wird damit robuster gegen Änderungen seiner Umgebung.

Die Option englvar

Bei der Variablensprache Englisch werden alle verwendeten Variablennamen außer denen in Selektionsausdrücken als englische Variablennamen interpretiert. In Selektionsausdrücken wird die Variablensprache unabhängig davon durch eine **Selektionsoption** ausgedrückt. Fehlt die Option oder wird in der FOP-Datei nicht mit **..!interpreter** gearbeitet, wird als Variablensprache Deutsch verwendet.



Obwohl englische Variablen in der Variablen-tabelle mit Groß- und Kleinschreibung angezeigt werden, ist in der FO die Schreibweise nicht relevant.

Beispiel: **'H|aVar'** und **'H|avAr'** bezeichnen dieselbe Variable und bei Fehlermeldungen zu dieser Variablen wird auf **'H|avar'** verwiesen.

Die Option noabbrev

Mit dieser Option wird festgelegt, dass Variablen nicht abgekürzt werden dürfen. Die Option **noabbrev** impliziert die Option **declaration**, d.h. Variablen müssen vor dem Verwenden deklariert werden.

```
..!interpreter declaration
.type int xcount1
.type int xnumber
.formula U|xcount1 = 1
.formula U|xnumber = 1
```

Das Programm bricht in Zeile 3 mit einem Fehlerhinweis ab, da die Variable 'xcoun1' (Tippfehler, es fehlt das t) nicht deklariert wurde.

In einem englischen FO-Programm wird ein deutsches FO-Programm SUBPROGRAM aufgerufen. Keines der Programme muss etwas über die Sprache des anderen wissen:

```
..!interpreter english declaration
.type int xi
.type text xm
```

```
.formula U|xi = 1
.input SUBPROGRAM
'U|xm'
.end
```

Das deutsche FO-Programm **SUBPROGRAM** hat folgenden Inhalt:

```
..!interpreter german
.formel U|xrahmen = "|||"
.formel = U|xrahmen + G|mandant + U|xrahmen
.weiter
```

- Im Hauptprogramm müssen alle Variablen deklariert werden
- Im Unterprogramm kann die Variable **U|xrahmen** ohne Deklaration verwendet werden.

Ein Programm **PROGRAM1** rufe ein Unterprogramm **SUB1** auf

Die Programmdatei **PROGRAM1** hat folgenden Inhalt:

```
.. locale variable 'xi'
.type int xi
.select PARTS "402"
.formula U|xi = H|stock
stock: 'U|xi'
.input SUB1
.formula U|xi = U|xi + 1
.end
```

Die Programmdatei **SUB1** (allgemein verwendbares Unterprogramm) hat folgenden Inhalt:

```
.. locale variable 'xi'
.type int xi
!WHILE
.formula U|xi = U|xi + 1
counter: 'U|xi'
.continue WHILE          ? U|xi < 5
.continue
```

Ablauf:

Das Programm **SUB1** überschreibt die Zählvariablen '**U|xi**' des Programms **PROGRAM1**.

.art initialisiert U|x_i.

Textausgabe und Textformatierung

Mit FO können Texte einfach ausgegeben und formatiert werden, wobei auch das Ausgabemedium (z.B. Datei oder Drucker) flexibel festgelegt werden kann.

Das Ausgeben eines Textes geschieht wahlweise dadurch, dass man den gewünschten Text in die Programmdatei schreibt oder dadurch, dass man ihm das Kommando `.println` voranstellt, bei dem man zusätzlich die Wahl zwischen verschiedenen Optionen hat.

FO geht im Standard davon aus, dass der Text formatiert werden soll, d.h. auf Seiten verteilt wird und entsprechend der gesetzten Ränder formatiert wird. Mit Angabe einer Formatdatei können Sie dies ändern.

Die Formatdateien liegen im `$MANDANTDIR` und haben die Endung `.F`.

Andernfalls bleibt die beim FOP-Start eingestellte Standardeinstellung gültig. Diese geht von einer Seite mit 72 Zeilen und 80 Spalten aus. Die Werte sind im Abas Kern initialisiert.

Einlesen der Formatdatei DATEI.F

```
.input DATEI.F
```

.AUSGABE / .OUTPUT - Ausgabemedium festlegen

.AUSGABE	.OUTPUT
Deutsch	Englisch
<code>.AUSGABE</code>	<code>.OUTPUT</code>
<code>.AUSGABE BILDSCHIRM</code>	<code>.OUTPUT TERMINAL</code>
<code>.AUSGABE PAUSE</code>	<code>.OUTPUT PAUSE</code>
<code>.AUSGABE S1 [KOPIEN='Anzahl']</code>	<code>.OUTPUT S1 [COPIES='number']</code>
<code>.AUSGABE FAX 'Faxnummer'</code>	<code>.OUTPUT FAX 'facsimile number'</code>
<code>.AUSGABE F1 'Faxnummer'</code>	<code>.OUTPUT F1 'facsimile number'</code>
<code>.AUSGABE MAIL [lokale Domain]</code>	<code>.OUTPUT MAIL [local domain] parameter1</code>
Parameter1 ... Parameter8	... parameter8
<code>.AUSGABE AN 'Dateiname'</code>	<code>.OUTPUT ADDEDTO 'file name'</code>
<code>.AUSGABE AUF 'Dateiname'</code>	<code>.OUTPUT NEW 'file name'</code>
<code>.AUSGABE AUF AN 'Dateiname'</code>	<code>.OUTPUT NEW ADDEDTO 'file name'</code>
<code>-ENCODIERUNG 'Encodierung'</code>	<code>-ENCODING 'encoding'</code>
<code>.AUSGABE W [: Vorlage] - Keine REST API-Unterstützung</code>	<code>.OUTPUT W [: template] - No REST API support</code>
<code>.AUSGABE E [: Parameter] - Keine REST API-Unterstützung</code>	<code>.OUTPUT E [: parameter] - No REST API support</code>
<code>.AUSGABE D [: CLIP CLIP+] - Keine REST API-Unterstützung</code>	<code>.OUTPUT D [: CLIP CLIP+] - No REST API support</code>
<code>.AUSGABE A [KOPIEN='Anzahl'] - veraltet, wird abgeschafft</code>	<code>.OUTPUT A [COPIES='number'] - deprecated, will be deleted</code>

.AUSGABE	.OUTPUT
<code>.ausgabe</code> und <code>.ausgabe bildschirm</code> steuert die Ausgabe auf den Bildschirm. Der vorhergehende Bildschirminhalt wird gelöscht. Bei <code>.ausgabe bildschirm</code> erfolgt die Ausgabe nach jeder Zeile. <code>.ausgabe ...</code> bzw. <code>.ausgabe pause</code> bzw. <code>.ausgabe bildschirm ...</code> bzw. <code>.ausgabe bildschirm pause</code> bewirkt einen Stopp. Die FO wartet mit der weiteren Erzeugung von Ausgabertext, damit der angezeigte Text gelesen werden kann. Auf dem Bildschirm wird die Stoppstelle im Ausgabertext durch drei Punkte angezeigt. Erst nach <Return> wird der Bildschirminhalt gelöscht und das FOP arbeitet weiter. <code>.ausgabe S1 [kopien='Anzahl']</code>, <code>.ausgabe S2 [kopien='Anzahl']</code> usw. steuert die Ausgabe auf den Systemdrucker 1, Systemdrucker 2 usw. Als Anzahl kann angegeben werden, wie viele Exemplare gedruckt werden sollen. <code>.ausgabe fax 'Faxnummer'</code> <code>.ausgabe F1 'Faxnummer'</code>, <code>.ausgabe F2 'Faxnummer'</code> usw. steuert den Faxkanal 1, Faxkanal 2 usw. an. Dahinter folgt die Faxnummer des Empfängers. Die Verwendung von Variablen ist erlaubt, z.B. <code>.ausgabe F1 'faxnr'</code>. <code>.ausgabe mail [lokale Domain] Parameter1 ... Parameter8</code> steuert die Ausgabe auf E-Mail. <code>.ausgabe an 'Dateiname'</code> fügt den Ausgabertext an das Ende der angegebenen Datei an. Der bisherige Text in der Datei bleibt erhalten, er wird um den angefügten Text verlängert. <code>.ausgabe auf 'Dateiname'</code> steuert die Ausgabe auf eine Datei des angegebenen Namens. Der bisherige Text in der Datei geht verloren! <code>.ausgabe auf an 'Dateiname' -encodierung 'Encodierung'</code> legt die Datei in der angegebenen Encodierung an. Zur Verfügung stehen: ABAS-S3, UTF-8, ISO-8859-1 und ISO-8859-5. Nur die Encodierungen ABAS-S3 und UTF-8 unterstützen alle Unicode-Zeichen, bei den anderen gehen die Zeichen verloren, die nicht im angegebenen Zeichensatz enthalten sind.	
Die Kommandos <code>.ausgabe W</code>, <code>.ausgabe E</code> und <code>.ausgabe D</code> werden ausschließlich im Abas Windows Client unterstützt. Diese Kommandos sind über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. <code>.ausgabe W</code> und <code>.ausgabe W : Vorlage</code> steuern die Ausgabe auf Microsoft Word. <code>.ausgabe E</code> und <code>.ausgabe E : Parameter</code> steuern die Ausgabe auf Microsoft Excel. <code>.ausgabe D : CLIP</code> steuert die Ausgabe bis zum nächsten <code>.ausgabe</code>-Kommando zeilenweise in die Zwischenablage. Dort steht immer die zuletzt ausgegebene Zeile. <code>.ausgabe D : CLIP+</code> merkt alle folgenden Ausgaben und kopiert sie beim nächsten <code>.ausgabe</code>-Kommando in die Zwischenablage. <code>.ausgabe D</code> bewirkt dasselbe wie <code>.ausgabe D : CLIP</code>.	

.AUSGABE	.OUTPUT
 Verwenden Sie das FO-Kommando .ausgabe A nicht mehr. Dieses FO-Kommando ist veraltet und wird mit einem der nächsten Upgrades abgeschafft. .ausgabe A [kopien='Anzahl'] steuert die Ausgabe auf den Arbeitsplatzdrucker. Als Anzahl kann angegeben werden, wie viele Exemplare gedruckt werden sollen.	

Beim Schreiben von Informationen in oder an eine Datei können die Sie folgende Trennzeichen verwenden:

Kommando	(FO-/FO2) White List
.ausgabe auf	! # \$ % & () * + , - / : ; < = > @ [\] _ ` { } ~ " ' . ? ^ Space
.ausgabe an	! # \$ % & () * + , - / : ; < = > @ [\] _ ` { } ~ " ' . ? ^ Space



Wenn Sie Daten aus einer Datei lesen wollen, dann können Sie aber nur folgende Trennzeichen aus der Whitelist verwenden:

Kommando	(FO-/FO2) Whitelist	(FO-/FO2) Blacklist
.hole von	! # \$ % & () * + , - / : ; < = > @ [\] _ ` { } ~ Space	" ' . ? ^
.datei -lesen	! # \$ % & () * + , - / : ; < = > @ [\] _ ` { } ~ Space	" ' . ? ^

.ausgabe legt fest, welches Ausgabegerät verwendet wird. Die Ausgabe selbst erfolgt erst beim nächsten **.ausgabe** bzw. beim Ende des FOPs. Der Befehl hat folgende Syntax:

```
.ausgabe
.ausgabe 'Gerät'
.ausgabe 'Gerät' 'Geräteoption'
.ausgabe 'Gerät' : Option1 Option2 ...
```

Weitere Angaben werden durch Doppelpunkt abgetrennt. Nach dem Doppelpunkt können z.B. die Priorität, Sendezeit und ein Kommentar angegeben werden. Die Angaben sind fakultativ und müssen im entsprechenden Druckertreiber ausgewertet werden.

```
.output F1 27126 : 3 12:37 Comment
```


Ausgabe auf E-Mail

`.ausgabe mail [lokale Domain] Parameter1 ... Parameter8`

Die Ausgabe kann mit verschiedenen Parametern spezifiziert werden.

Parameter	Inhalt	Erklärung
1	Laufende Nummer	Angabe der laufenden Nummer. Jedes Dokument, das per E-Mail versendet wird, wird in dem angegebenen Verzeichnis (Parameter2) als MAIL.lfdNR gespeichert.
2	Verzeichnis	Angabe des Verzeichnisses, in dem die versendeten Daten gespeichert werden.
3	Mail-SMTP-Server	
4	Absender der E-Mail-Adresse	Wird im Journal (<Zeigen> EM <Zeigen>, Journal zeigen) gespeichert.
5	Erster Empfänger aus Empfängerliste	Wird im Journal (<Zeigen> EM <Zeigen>, Journal zeigen) gespeichert.
6	Kopiekennzeichen (ja/nein)	Muss G false sein, wenn ein Dokument gespeichert werden soll. Wird das Dokument erneut versendet, wird der Parameter auf G true gesetzt, d.h. es findet keine weitere Speicherung statt.
7	Kommentar	Wird im Journal (<Zeigen> EM <Zeigen>, Journal zeigen) gespeichert.
8	Name des externen Users	Kann für den Systemaufruf in \$HOMEDIR/OW/BIN/PRINT*.SH verwendet werden.

Ausgabe in Microsoft Word und Excel

`.ausgabe W`

startet Microsoft Word und öffnet ein neues Dokument mit dem Word-Kommando NEU().

`.ausgabe W : Vorlage`

startet Microsoft Word und öffnet ein neues Dokument unter Verwendung einer Word-Vorlage. Die Vorlagendatei muss im Mandantenverzeichnis **win/winword** liegen.

Alle folgenden Ausgaben werden zeilenweise direkt in das Word-Dokument geschrieben. Die Nummer des

DDE-Kanals, die beim Kommando **.ausgabe W** vergeben wurde, steht in der FO-Variablen **G|ddkana1** zur Verfügung.

.ausgabe E

startet Microsoft Excel und öffnet eine neue Mappe mit dem Excel-Kommando NEU().

.ausgabe E : Parameter

Parameter	
1	siehe Excel-Handbuch, Kommando NEU()
2	
3	
4	
5	
Name einer Microsoft-Excel-Mustervorlage	führt zum Excel-Kommando NEU("Parameter")
nicht angegeben	führt zu NEU(1)

Alle folgenden Ausgaben werden direkt in diese Tabelle geschrieben. Die Startposition ist oben links, also Zelle A1. Jede neue Zeile wird in eine neue Excel-Zeile geschrieben. Spaltentrennzeichen ist das Semikolon.

Jedes weitere **.ausgabe E**-Kommando öffnet eine neue Mappe. Der Name der zuletzt geöffneten Mappe steht in der FO-Variablen **G|ddename**. "Mappe1" steht für die erste geöffnete Mappe. Die Nummer des DDE-Kanals, die beim **.ausgabe E**-Kommando vergeben wurde, steht in der FO-Variablen **G|ddkana1** zur Verfügung.

Beachten Sie bei der Verwendung dieses Kommandos die [DDE-Systemvoraussetzungen](#).

Lesen Sie hierzu auch Kapitel [Kopplung von Abas an Microsoft Word und Excel](#).

Ausgabe in OpenOffice

FO-Programme können wie in Microsoft Excel auch in OpenOffice Calc (OO Calc) bzw. StarOffice Calc ausgegeben werden. Die FO-Programme **OC.INIT** bzw. **OC.ISINIT** sind anstelle des FO-Kommandos **.ausgabe E** einzulesen, um die Ausgabe nach OpenOffice Calc zu beginnen. Zusätzlich muss am Ende der Ausgabe das FOP **OC.ENDE** eingelesen werden.

.SETZE STOP / .SET STOP - Bildschirm seitenweise füllen

.SETZE STOP	.SET STOP
deutsch	englisch
.SETZE STOP +	.SET STOP +
.SETZE STOP -	.SET STOP -

.SETZE STOP	.SET STOP
gilt nur auf der ASCII-Oberfläche	
.setze stop + wartet nach jeder Bildschirmseite auf Tastendruck, um die nächste Seite anzuzeigen. Wenn jedoch <Fertig> gedrückt wird, nicht mehr warten. .setze stop - scrollt den ausgegebenen Text weiter, ohne zu warten.	

.PRINTLN / .PRINTLN - Text ausgeben

.PRINTLN	.PRINTLN
deutsch	englisch
.PRINTLN [-TEXT] [-XML] [-LFSUPPRESS] [-NOMOREOPTIONS] [-SCREEN] [-DEBUG] [-LOGLEVEL x] 'Text'	.PRINTLN [-TEXT] [-XML] [-LFSUPPRESS] [-NOMOREOPTIONS] [-SCREEN] [-DEBUG] [-LOGLEVEL x] 'text'
.println -text gibt den reinen Text ohne Interpretation, z.B. von Variableninhalten, aus. .println -xml maskiert Metazeichen nicht, wenn das Ausgabeformat XML oder HTML ist. .println -lfsuppress gibt den angegebenen Text aus, ohne dass ein Zeilenendezeichen angefügt wird. .println -nomoreoptions zeigt an, dass die folgende Angabe nicht als Option, sondern als Text interpretiert werden soll. .println -screen sorgt für die Ausgabe von Texten in das Ausgabefenster statt in eine evtl. geöffnete Ausgabedatei. .println -debug und .println -loglevel x wirken wie -screen, nur dass die Ausgabe mit .println 'Quelldatei': 'Quellezeile': eingeleitet wird. Bei -loglevel x erfolgt die Ausgabe nur, wenn P loglevel > x ist.	

Das Kommando gibt Text inklusive Variablen aus. Obwohl Text in FOPs auch einfach dadurch ausgegeben werden kann, dass man ihn hinschreibt (ohne ein Befehlswort davor zu setzen), hat der Befehl eine eigenständige Bedeutung, da er die Angabe von Optionen ermöglicht. In JFOPs kann *ausschließlich* über **println()** ausgegeben werden.

Anführungszeichen in Textausgaben



Direkt Text ausgebende Kommandos (**.hinweis**, **.fehler**, **.menue**, **.box** und **.println**) stellen den Text genau so dar wie angegeben, analog dem **Direktmodus**. So ist es auf einfache Weise möglich, Anführungszeichen auszugeben, z.B.:

.fehler Im Unterprogramm trat der Fehler "keine Netzverbindung" auf.

.hinweis Rechnungsabschlusspositionen wurden ergänzt.

Option -text

Die Option **-text** hat die gleiche Bedeutung wie bei **-text**: Man kann damit alle darauffolgenden Angaben uninterpretiert als Text ausgeben, ohne dass z.B. ein Apostroph eine Sonderbedeutung hat und ohne dass ein Minus als Kennzeichnung einer Option interpretiert wird (andere gewünschte Optionen müssen also vor **-text** kommen).

Beispiel: Ausgabe von Variableninhalt bzw. Variablennamen

In diesem Beispiel gibt das Kommando **.println** Text und den Inhalt der Variable **U|xicount** aus. Das Kommando **.println -text** gibt nur uninterpretierten Text aus.

```
.type int xicount          ? F|defined(U|xicount) = false
.formula U|xicount = 1
.println No. 'U|xicount'
.println -text No. 'U|xicount'
```

Die Ausgabe ist unabhängig der Einstellung von **'ersetzungchar'**:

```
No. 'U|xicount'
```

Die Option **-text** ist vor allem in JFOPs interessant, da man sich bei **F0.println(String string)** nicht mehr um eventuell in **string** vorhandene Sonderzeichen kümmern muss.

Option -xml

Wenn das Ausgabeformat XML oder HTML ist (siehe **.format**), werden Metazeichen wie **'<'** nicht maskiert (gequotet), sondern eins zu eins ausgegeben. Diese Option ist nützlich für Zeilen, die HTML- bzw. XML-Code enthalten.

-xml entspricht dem Doppelpunkt bei einfachen Textzeilen. Beim Doppelpunkt ist es aber nicht möglich, per Fragezeichenoperator eine Bedingung anzugeben. Durch die Option **-xml** kann der Doppelpunkt ersetzt werden, also kann gleichzeitig in einer solchen Zeile eine Bedingung angegeben werden.

Beispiel: PRINTLN mit/ohne XML-Ausgabeformatierung

```
.format xml
xml output format
.println -xml Text contains xml-Code: <B>bold</B>
without xml output format
.println Text contains xml-Code: <B>bold</B>
```

display:

```
xml output format
Text contains xml-Code: <B>bold</B>
without xml output format
Text contains xml-Code: &lt;B&gt;bold&lt;/B&gt;
```

Option -lfsuppress

Die Option gibt Texte und Variablen aus, ohne dass ein Zeilenendezeichen erzeugt wird. Die Ausgabe des folgenden Befehls schließt also direkt an die Ausgabe dieses `.println`-Befehls an. Nützlich ist dies z.B. für ein flexibleres Generieren von Dateien sowie für kleine Fortschrittsmeldungen während eines lange laufenden Programms.

Beispiel: Fortschrittsmeldung

In diesem Beispiel wird ein FOP mit längerer Laufzeit simuliert. Solange das FOP läuft, werden Punkte, danach wird "Programmende" ausgegeben.

```
.type int xiindex1      ? F|defined(U|xiindex1) = false
.type int xiindex2      ? F|defined(U|xiindex2) = false
.formula U|xiindex1 = 0
..
!WHILE
.println -lfsuppress .
.formula U|xiindex2 = 0
..
.. simulate longer process
!WAIT
.formula U|xiindex2 = U|xiindex2 + 1
.continue WAIT ? U|xiindex2 < 10000
..
.formula U|xiindex1 = U|xiindex1 + 1
.continue WHILE ? U|xiindex1 < 10
..
Program end
```

Ergebnis:

Bei jedem Schleifendurchlauf wird ein Punkt ausgegeben, wobei die Punkte nebeneinander in der selben Zeile stehen.

Option -nomoreoptions

Die Option zeigt an, dass alle folgenden Argumente nicht als Option interpretiert werden sollen, auch wenn sie mit Minus beginnen.

Beispiel: Optionen nicht interpretieren

```
.println -nomoreoptions -xml is XML-Option
```

```
-xml is XML-Option
```



Upgradefähigkeit

Verwenden Sie **.println** *immer*, wenn der auszugebende Text mit Minus beginnt oder beginnen könnte (bei Variableninhalten). Zwar verhält sich **.println** großzügig und gibt auch alles, was mit Minus beginnt als Text aus, so lange es keine definierte Option ist. In zukünftigen Versionen können aber neue Optionen hinzukommen.

Option -debug oder -loglevel x

Die Optionen **-debug** oder **-loglevel x** sind für Testausgaben gedacht. Sie wirken wie **-screen**, d.h., die Ausgabe von Texten erfolgt in das Ausgabefenster statt in eine eventuell geöffnete **Ausgabedatei**, wird aber zusätzlich mit **.println ' Quelldatei ': ' Quellzeile ':** eingeleitet. Der Parameter **x** bei der Option **-loglevel x** sorgt für eine bedingte Ausgabe: Die Ausgabe erfolgt nur, wenn **P|loglevel > x** ist.

```
.println -debug 1. control point reached ? P|loglevel > 1
.println -loglevel 1 1. control point reached
```

daten:467: control point X reached

Beide Zeilen erzeugen dieselbe Testausgabe, falls **P|loglevel** größer oder gleich **1** ist.

.BLOCKSATZ / .JUSTIFIED - Blocksatz

.BLOCKSATZ	.JUSTIFIED
deutsch	englisch
.BLOCKSATZ	.JUSTIFIED
Verteilt den Text durch Auffüllen mit Leerzeichen vom linken bis zum rechten Rand.	

.FLATTERSATZ / .UNJUSTIFIED - Flattersatz

.FLATTERSATZ	.UNJUSTIFIED
deutsch	englisch
.FLATTERSATZ	.UNJUSTIFIED

.FLATTERSATZ	.UNJUSTIFIED
Füllt die Zeilen wortweise vom linken bis zum rechten Rand. Das Schriftbild ist links bündig und "flattert" rechts.	

Block- und Flattersatz bewirken Fließtext, d.h.

- Zeilenumbruch wortweise
- Silbentrennung
- Entfernen von Leerzeichen, die auf ein Leerzeichen folgen
- Einfügen von Leerzeichen nach Punkt, Frage- und Ausrufezeichen
- [Absatzschutz](#)

.tabellensatz beendet einen Flutter- oder Blocksatz, also auch den Fließtext. Der Text wird zwar am Ende der Zeile auf eine neue Zeile umgebrochen, aber ohne Beachtung der Wörter.

Beispiel: Fließtext

```
.unjustified
```

```
.tablerecord
```

Mit **.tabellensatz** [\[Optionen\]](#) wird auch die HTML-Ausgabe einer Tabelle gesteuert. Näheres finden Sie in den Kapiteln [HTML-Tabellen erzeugen](#), [Alle Formatierungen abschalten](#) und [Durchgehende Spaltentrenner erzeugen](#).

Besonderheiten gibt es bei der [Darstellung im Browser](#).

Um Sperrdruck zu ermöglichen, gilt im FO-Programm der Unterstrich als nicht zu entfernendes und nicht trennendes Leerzeichen (siehe **.setze leer**).

Leerzeilen und Kommandos unterbrechen den Fließtext und erzeugen dadurch einen neuen Absatz. Um *nur* einen Zeilenumbruch, d.h. einen neuen Absatz, zu erzeugen, kann das [Kommentarkommando](#) verwendet werden, das aus zwei Punkten (..) besteht.

Beispiel: Zeilenumbruch im Fließtext

(Fließtext gibt es nur im Block- oder Flattersatz.)

Ein Zeilenumbruch im FOP unterbricht den Fließtext in der Ausgabe nicht:

```
.justified
```

Einen Zeilenumbruch in der Ausgabe erreicht man mit dem Kommentarkommando:

```
.justified
```

Leerzeilen und Kommandos unterbrechen den Fließtext und erzeugen dadurch einen neuen Absatz.

Um nur einen Zeilenumbruch, d.h. einen neuen Absatz, zu erzeugen, kann das Kommentarkommando verwendet werden, das aus zwei Punkten besteht.

Formatierte Ausgabe des FO-Programms

Manchmal benötigt das System für die Silbentrennung einen Hinweis des Benutzers, da es diese bei zusammengesetzten Wörtern nicht erkennen kann. Dieser seltene Fall tritt vor allem dann ein, wenn das angehängte Wort mit einem Selbstlaut beginnt. In solchen Fällen können Sie die Trennstelle mit dem Hochzeichen (^) kennzeichnen. Der Rechner bevorzugt im Bedarfsfall diese Stelle für eine Trennung, ansonsten entfernt er die Kennzeichnung.

Das Vorrücken (Text beginnt weiter links) und Einrücken (Text beginnt weiter rechts) am Absatzanfang (siehe [.links](#) - Linker Rand) ist nur im Block- und Flattersatz möglich.

Zeilen, die mit einem Leerzeichen in der ersten Spalte beginnen, werden nicht in den Fließtext einbezogen, sondern als Einzelzeilen belassen, wobei das kennzeichnende Leerzeichen für die Ausgabe entfernt wird.

Bei Einzelzeilen und einzeliligen Absätzen wird die erste Zeile generell nicht eingerückt.

Beispiel: Einrücken und Vorrücken

```
1-----2-----3-----4-----5-----6-----7-----8-----
.unjustified
.left +3 +5
A line with a leading space character is a single line.
This is a multiline paragraph, whose first line will be feed 5 characters to the right. This
is a line to increase the paragraph.
.left -1
As of now all lines will be feed 1 character to the left.
Also the single lines with a leading space character.
```

Result

```
1-----2-----3-----4-----5-----6-----7-----8-----
A line with a leading space character is a single line.
This is a multiline paragraph, whose first line will be feed 5 charac-
ters to the right. This is a line to increase the paragraph.
As of now all lines will be feed 1 character to the left.
Also the single lines with a leading space character.
```

[.tabellensatz](#) beendet den Flatter- oder Blocksatz. Im Ausgabebetext bleibt die Zeilenstruktur wie im Programm angegeben, erhalten. In jeder Zeile können definierte Felder rechts- oder linksbündig durch [FO-Variablen](#) ausgefüllt werden, so dass mehrere Spalten eine Tabellenstruktur bilden.

Beispiel: Tabellenstruktur

Das nachfolgende FO-Programm holt den Kunden mit der Identnummer 1 und gibt Feldinhalte aus.

Zugriff mithilfe englischer Feldnamen

```
.select Customer "1"
.continue END    ? G|success = false
'H|addr'  'H|phoneNo'
'H|street'
'H|town'  'H|comments'
```

Der Tabellensatz ist besonders geeignet, um Einträge der Datenbank in FOPs oder Schriftstücken wiederzugeben. Zum Datenbankzugriff über FO-Variablen siehe auch [Datenbankkommandos](#).)

.SETZE INLINE / .SET INLINE - Fließtext beibehalten

.SETZE INLINE	.SET INLINE
deutsch	englisch
.SETZE INLINE + .SETZE INLINE -	.SET INLINE + .SET INLINE -
.setze inline + verhindert, dass durch Kommandozeilen ein Absatzende (Zeilenumbruch) erzeugt wird. .setze inline - hebt die Wirkung wieder auf.	

Leerzeilen und Kommandozeilen (Zeilen, die mit Punkt oder Doppelpunkt beginnen, sofern nicht die [Punktbedeutung umdefiniert](#) wurde) beenden im Block- und Flattersatz den aktuellen Absatz. Mit **.setze inline +** kann jedoch veranlasst werden, Kommandozeilen nicht als Absatzende zu werten.

Beispiel: Absatzende unterdrücken

```
.justified
.set inline +
The warehouse stock currently contains
.formula x=15
'x' products which are past their sell-by date.
```

display

The warehouse stock currently contains 15 products which are past their sell-by date.

Im Inline-Modus ist der bisher abgearbeitete Text beim Ausführen eines FO-Kommandos nicht notwendigerweise bereits ausgegeben, da die Ausgabe im Block- und Flattersatz gepuffert erfolgt. Dies muss mit **.absatz** explizit erzwungen werden, sofern die Ausgabe bereits für das nächste Kommando erforderlich ist. Normalerweise ist das nicht notwendig, da im Inline-Modus typischerweise in eine Datei ausgegeben wird.

Bei der HTML-Ausgabe ist es möglich, innerhalb eines Absatzes unterschiedliche Schriftattribute zu setzen. Lesen Sie dazu Kapitel [Schriftattribute setzen](#).

.SETZE LF / .SET LF - Zeilenvorschub setzen

.SETZE LF	.SET LF
deutsch	englisch
.SETZE LF .SETZE LF + .SETZE LF -	.SET LF .SET LF + .SET LF -
gilt nur für die Ausgabe nach Microsoft Windows .setze lf + sendet [Einfügen Absatz] nach jeder Zeile und erzeugt damit einen Zeilenvorschub (line feed). Diese Einstellung ist der Standard. .setze lf - verhindert einen Zeilenwechsel. Dies ist wichtig für Formulare unter Microsoft Word.	

.LINKS / .LEFT - Linker Rand

.LINKS	.LEFT
deutsch	englisch
.LINKS 'n' [MM] .LINKS + - 'n' [MM] .LINKS 'n' + - 'm' [MM]	.LEFT 'n' [MM] .LEFT + - 'n' [MM] .LEFT 'n' + - 'm' [MM]
.links 'n' setzt den linken Rand bis zur Spalte n , d.h., der folgende Text beginnt in Spalte n . .links +'n' vergrößert, .links -'n' verkleinert einen bestehenden linken Rand um n Spalten. .links 'n' +'m' vergrößert, .links 'n' -'m' verkleinert in jedem mehrzeiligen Absatz den Abstand der ersten Zeile zum linken Rand um m Spalten. .links 'n' [MM] setzt den linken Rand auf n Millimeter.	

Einrücken = Rand vergrößern = Text nach rechts setzen
Vorrücken = Rand verkleinern = Text nach links setzen

Der linke Rand kann nicht kleiner als Null sein. Wird ein negatives Maß verlangt, erscheint eine Fehlermeldung.

Bei **.links 'n' +|- 'm'** darf das Plus vor **m** fehlen. Soll **n** Null sein, muss **+0** geschrieben werden.

Siehe auch [Beispiel](#) zum Ein- und Vorrücken.

Bei Millimeterangaben hängt die Zeichenanzahl von der Zeichenbreite des Bildschirms oder Druckers ab. Die Zeichenbreite können Sie der Bedienungsanleitung des Bildschirms oder Druckers entnehmen. Sie wird meistens in Zeichen pro inch ("char per inch", cpi) angegeben. Typische Werte bei Druckern sind 10 oder 12 cpi für Normalschrift und 16 bis 20 cpi für Engschrift. Für die Umrechnung gilt: 1 inch = 25.4 mm. Wird die Zeichenbreite geändert (siehe **.zeigen - Druckausgabe** und **.zeigen - Fensterbreite festlegen**), bleibt jede Randeinstellung in der zuletzt genannten Einheit erhalten.

.RECHTS / .RIGHT - Rechter Rand

.RECHTS	.RIGHT
deutsch	englisch
.RECHTS 'n' [MM] .RECHTS + - 'n' [MM]	.RIGHT 'n' [MM] .RIGHT + - 'n' [MM]
.rechts 'n' setzt den rechten Rand bis zur Spalte n, d.h., der Text endet spätestens in Spalte n - 1. Überhängender Text wird auf die Folgezeilen verteilt, wobei im Blocksatz und Flattersatz die Silbentrennung wirksam wird. .rechts +'n' vergrößert, .links -'n' verkleinert den rechten Rand um n Spalten. .rechts 'n' [MM] setzt den rechten Rand auf n Millimeter.	

Der rechte Rand wird bei Beginn der Formatierung automatisch auf die Fensterbreite gesetzt. Der rechte Rand kann bis zur Spalte 250 reichen. Wenn mehr als 80 Spalten auf dem Bildschirm ausgegeben werden sollen, empfiehlt sich die Umschaltung auf eine [komprimierte Schrift](#), sofern der Bildschirm zu dieser Darstellung in der Lage ist.

Zur manuellen Silbentrennung lesen Sie Kapitel [Formatierte Ausgabe des FO-Programms](#).

Bei Millimeterangaben hängt die Zeichenanzahl von der Zeichenbreite des Bildschirms oder Druckers ab. Die Zeichenbreite können Sie der Bedienungsanleitung des Bildschirms oder Druckers entnehmen. Sie wird meistens in Zeichen pro inch ("char per inch", cpi) angegeben. Typische Werte bei Druckern sind 10 oder 12 cpi für Normalschrift und 16 bis 20 cpi für Engschrift. Für die Umrechnung gilt: 1 inch = 25.4 mm. Wird die Zeichenbreite geändert (siehe **.zeigen - Druckausgabe** und **.zeigen - Fensterbreite festlegen**), bleibt jede Randeinstellung in der zuletzt genannten Einheit erhalten.

.SEITE / .PAGE - Seitenbereich und Seitenumbruch

.SEITE	.PAGE
deutsch	englisch
.SEITE .SEITE 'Seitennummer' .SEITE 'erste Seite'-'letzte Seite'	.PAGE .PAGE 'page number' .PAGE 'first page'-'last page'

.SEITE	.PAGE
<code>.seite</code> löst einen Seitenvorschub aus, falls die bisherige Seite schon angebrochen war. Auf dem Bildschirm wird der Seitenwechsel durch eine Raute (#) gekennzeichnet. <code>.seite 'Seitennummer'</code> löst einen Seitenvorschub aus und setzt einen Seitenzähler. Die Nummer der neuen Seite wird in der FO-Variablen <code>G seite</code> gespeichert. <code>.seite 'erste Seite'-'letzte Seite'</code> gibt die Seiten des angegebenen Bereichs aus. Mit der ersten Zeile der letzten Seite beendet sich der FO-Interpreter.	

Mit diesem Kommando kann man einen Seitenumbruch erzwingen oder den auszugebenden Seitenbereich festlegen.

.DRUCKEN / .PRINT - Druckausgabe

.DRUCKEN	.PRINT
deutsch	englisch
<pre>.DRUCKEN 'Anzahl Exemplare' .DRUCKEN 'Schrifteigenschaft 1' 'Schrifteigenschaft 2' ... 'Schrifteigenschaft n'</pre>	<pre>.PRINT 'number of copies' .PRINT 'font attribute 1' 'font attribute 2' ... 'font attribute n'</pre>
<code>.drucken 'Anzahl Exemplare'</code> löst den Ausdruck aus. Bei <code>.ausgabe</code> auf Bildschirm oder Datei bleiben die <code>.drucken</code>-Kommandos unberücksichtigt. <code>.drucken 'Schrifteigenschaft 1' 'Schrifteigenschaft 2' ... 'Schrifteigenschaft n'</code> steuert die Schriftgestaltung, die der Drucker ermöglicht.	



Bei `.drucken 'Anzahl Exemplare'` wirkt `.drucken 0` wie `.drucken 1`; es wird ein Exemplar gedruckt.

Schriftgestaltung

Sie können Schrifteigenschaften wie Schriftdicke, Größe, Intensität usw. festlegen. Diese Eigenschaften müssen in der Gerätebeschreibung des Drucker definiert sein. Lesen Sie dazu das Kapitel [Druckerbeschreibung: \\$HOMEDIR/termcap.txt](#).

Auch am Bildschirm kann die Schriftart verändert werden, z.B. um Zeilen mit mehr als 80 Zeichen in komprimierter Schrift wiederzugeben. Dazu dient das Kommando `.zeigen`.

Jede Schrifteigenschaft besteht aus einer Eigenschaft und einem Wert. Zum Beispiel kann die Eigenschaft "Schriftart" die Werte "pica", "normal" oder "elite" annehmen; die Eigenschaft "Intensität" die Werte "fett" oder "normal". Eigenschaft und Wert werden durch einen Punkt verbunden, z.B. `sa.norm` für "Schriftart normal". Wenn keine Eigenschaften angegeben werden, wird der Drucker in den Grundzustand versetzt.

```
.print sg.groß in.fett str.an
```

stellt den Drucker auf Großschrift, Fettdruck und Unterstreichen ein.

Um geräteunabhängig zu bleiben, sollten ähnliche Eigenschaften verschiedener Geräte gleichartig benannt werden.



Meist können nicht alle Eigenschaften eines Druckers beliebig gemischt werden. Im Zweifel gibt ein Versuch oft schneller Auskunft als die Bedienungsanleitung des Druckers.

Zum Drucken mit Steuersequenzen siehe auch Kapitel [Steuersequenzen zum Drucken](#).

.ZEIGEN / .VIEW - Fensterbreite festlegen

.ZEIGEN	.VIEW
deutsch	englisch
.ZEIGEN SG.NORM SG.KLEIN	.VIEW SG.NORM SG.KLEIN
legt die Zeichenanzahl pro Zeile und damit die Breite des FO-Ausgabefensters fest	
SG.NORM normale Zeichenbreite, 80 Zeichen	
SG.KLEIN kleine Zeichenbreite, 132 Zeichen	

Die tatsächliche Breite des Fensters hängt von der Hardware, der Bedienoberfläche (GUI oder ASCII) und von Bildschirmanpassungen und anderen Konfigurationen ab. Das Einschalten der großen Fensterbreite kann z.B. bedeuten, dass das Fenster unverändert bleibt, aber die Schriftgröße verkleinert wird.

Doppelpunkt bei .drucken und .zeigen

Bei **.zeigen** auf den Bildschirm oder in eine Datei bleiben **.zeigen**-Kommandos unberücksichtigt, bei **.drucken** auf einen Drucker oder in eine Datei bleiben **.drucken**-Kommandos unberücksichtigt. Manchmal sollen solche Kommandos jedoch erhalten bleiben, ohne dass sie bei der Zeilenzählung berücksichtigt werden, z.B. wenn der Dateiinhalte dazu bestimmt ist, später wieder als FO-Programme zu dienen. In diesem Fall ist der Kommandopunkt durch einen Doppelpunkt zu ersetzen:

```
:print ...
```

wirkt bei Druckerausgabe wie **.drucken**, wird aber bei Dateiausgabe mit in die Datei geschrieben. Diese sehr spezielle Schreibweise ist normalerweise nicht nötig; siehe dazu [Ausgabeabhängige Kommandos](#).

.LAENGE / .LENGTH - Seitenlänge

.LAENGE	.LENGTH
deutsch	englisch
<code>.LAENGE</code> <code>.LAENGE 'Anzahl Zeilen'</code>	<code>.LENGTH</code> <code>.LENGTH 'number lines'</code>
legt die Anzahl der Zeilen pro Seite fest	

Die Standardeinstellung geht von einer Seite mit 72 Zeilen aus. Das Kommando `.laenge` ermöglicht, einen anderen Wert festzulegen und unterstützt somit unterschiedliche Papierformate.

Wenn die Angabe `'Anzahl Zeilen'` fehlt, wird die Seitenlänge auf 72 Zeilen gesetzt.

.ABSATZ / .PARA - Absatzschutz

.ABSATZ	.PARA
deutsch	englisch
<code>.ABSATZ</code> <code>.ABSATZ 'Zeilenanzahl 1'</code> <code>.ABSATZ 'Zeilenanzahl 1' 'Zeilenanzahl 2'</code>	<code>.PARA</code> <code>.PARA 'number lines 1'</code> <code>.PARA 'number lines 1' 'number lines 2'</code>
<code>.absatz</code> setzt für den unmittelbar folgenden Absatz den Absatzschutz auf <code>'Zeilenanzahl 2'</code> und für alle weiteren Absätze wieder auf <code>'Zeilenanzahl 1'</code>. <code>.absatz 'Zeilenanzahl 1'</code> setzt für alle Absätze bis auf weiteres den Absatzschutz auf <code>'Zeilenanzahl 1'</code>, d.h. mindestens die angegebene Anzahl Zeilen eines Absatzes bleibt zusammen auf einer Seite. <code>.absatz 'Zeilenanzahl 1' 'Zeilenanzahl 2'</code> setzt für alle Absätze bis auf weiteres den Absatzschutz auf <code>'Zeilenanzahl 1'</code> und merkt die <code>'Zeilenanzahl 2'</code> vor.	

Beim Seitenumbruch kann es geschehen, dass ein Textabsatz durch den Seitenwechsel zerrissen wird. Dies kann mit einem Absatzschutz verhindert werden. Ein neuer Absatz wird komplett auf die nächste Seite gestellt, wenn auf der Seite weniger als die angegebene Anzahl Zeilen verbleiben, wobei ein Absatz mit einer Leerzeile oder eine Punktbefehlzeile beginnt.

Ist auf einer Seite weniger als die angegebene Mindestzeilenanzahl frei, ruft `.absatz` einen Seitenumbruch hervor, auch wenn der Absatz tatsächlich weniger Zeilen beansprucht.

Der Absatzschutz ist nur dann wirksam, wenn der `Blocksatz` oder `Flattersatz` eingeschaltet ist.

Beim Start der FO steht der Absatzschutz entsprechend `.absatz 5 10`.

```
.para 3
```

Der Absatz muss mindestens 6 Zeilen haben, um durch einen Seitenumbruch zerrissen zu werden. Hat der Absatz 5 oder Zeilen oder weniger, wird er entweder komplett auf die alte oder komplett auf die neue Seite gesetzt, weil einer der beiden Teile aus weniger als 3 Zeilen bestehen würde.

Mit einem einzigen Aufruf können auch zwei verschiedene Mindestzeilenanzahlen für Absätze vorgemerkt werden, zwischen denen mit geringem Schreibaufwand hin- und hergeschaltet werden kann:

```
.para 3 4
```

setzt den Absatzschutz zunächst auf 3 Zeilen.

```
.para
```

Schaltet für den unmittelbar folgenden Absatz den Absatzschutz auf 4 Zeilen. Nach diesem Absatz ist automatisch wieder der Wert 3 wirksam.

```
Text one consists of 7 lines
.para 8 25
Text two consists of 8 lines
.para
Text three consists of 20 line
Text four consists of 18 line
.para 50
Text five consists of 9 line
```

Der 7-zeilige Text eins kann zerrissen werden, da die Standardeinstellung 5 gilt.

Der 8-zeilige Text zwei kann nicht zerrissen werden, denn vorher wurde die Mindestzeilenanzahl auf 8 Zeilen eingestellt.

Der 20-zeilige Text drei ist ebenfalls geschützt, denn es wurde durch **.ABSATZ** die früher gemerkte 25 aktiviert. Sie gilt aber nur für einen Absatz, so dass der 18-zeilige Text vier nicht mehr geschützt ist.

Der 9-zeilige Text fünf wird in keinem Fall zerrissen, aber mit hoher Wahrscheinlichkeit auf die nächste Seite gesetzt, so dass die Vorgängerseite bis zu 50 Leerzeilen aufweisen kann.

.OBEN / .UP - Kopfzeilen

.OBEN	.UP
deutsch	englisch
.OBEN	.UP

.OBEN	.UP
definiert den Obenpuffer. Der folgende Text bis zum nächsten Kommando erscheint als Kopf auf jeder folgenden Seite.	

.UNTEN / .DOWN - Fußzeilen

.UNTEN	.DOWN
deutsch	englisch
.UNTEN	.DOWN
definiert den Untenpuffer. Der folgende Text bis zum nächsten Kommando erscheint als Fuß auf jeder folgenden Seite.	

Kopf- und Fußzeilen dürfen **Variablen** enthalten. Die Namen der Variablen müssen eindeutig sein. Variablen von Include-Dateien müssen sowohl im FOP selbst als auch im Oben- bzw. Untenpuffer eindeutig sein. Die Auswertung der Variablen erfolgt bei jedem Seitenumbruch.

Der Oben- und Untenpuffer endet jeweils mit dem nächsten Kommando bzw. mit der nächsten Kommentarzeile. Um das FOP übersichtlich zu gestalten, definieren Sie das Ende des Oben- oder Untenpuffers am besten mit einem **Kommentar-Kommando**.

Beispiel: Kopfzeilen

```
.up
trade accounts - 'G|FY'           page 'G|page'
=====
..
```

.VTAB / .VTAB - Vertikaltabulatoren

.VTAB	.VTAB
deutsch	englisch
.VTAB 'Zeilennummer 1' 'Zeilennummer 2' ...	.VTAB 'line number 1' 'line number 2' ...
Definiert Vertikaltabulatoren (Zeilen) für ein Ausgabegerät oder das Kommando .zeile . Die Zeilennummern werden Inhalte der Variablen vt1 , vt2 usw.	
Standardeinstellungen sind vt1=1 , vt2=70 , vt3=10 und vt4=20 .	

Um bestimmte Positionen auf einer Seite unabhängig vom Drucker und Papierformat zu erreichen, müssen Vertikaltabulatoren statt absoluter Zeilennummern angegeben werden.


```
.vtab 4 68 10 20
vt1=4, vt2=68, vt3=10, vt4=20
```

Die Standardeinteilung gilt für eine Seite mit 72 Zeilen und 80 Spalten entsprechend einer Schreibdichte von 10 cpi und 6 lpi auf 12"-Papier.

cpi	characters per inch = Zeichen pro Zoll
lpi	lines per inch = Zeilen pro Zoll
"	Zoll, 1 Zoll = 2,54 cm
12"-Papier	8 Zoll x 12 Zoll = 21,0 cm x 30,5 cm

Mit Hilfe der Vertikaltabulatoren können Zeilen angesteuert werden:

.zeile 'vt1'

hat nach obigem Beispiel dieselbe Wirkung wie **.ZEILE 4**. Um diesen Mechanismus einheitlich nutzen zu können, gelten für alle FO-Programme, die Texte erzeugen, folgende Vereinbarungen:

Kennzeichen	Bedeutung	Bemerkung
vt1 vt2	erste beschreibbare Zeile der Seite letzte beschreibbare Zeile der Seite	Zeile 1 einer Seite ist zwar vom Drucker erreichbar, sollte aber z.B. nicht beschrieben werden, weil sie im vorgedruckten Briefkopf liegt oder weil das Layout schlecht aussieht, wenn die Überschrift direkt am oberen Rand steht. Deshalb beginnt jeder Text erst bei vt1 . Analog verhält es sich mit dem unteren Rand der Seite: Der Text endet bei vt2 .
vt3	erste Zeile des Anschriftsfelds	Das Anschriftsfeld beginnt je nach Briefpapier und Drucker üblicherweise bei Zeile 9, 10 oder 5. Es soll mit dem Adressfenster des Umschlags korrespondieren.
vt4	erste Zeile nach einer Anschrift	Die erste Zeile nach der Anschrift enthält im Schriftverkehr häufig die Angaben Ihr Zeichen, Unser Zeichen usw. Da Briefpapier oft vorbedruckt ist, muss diese Position zeilengenau angegeben werden können.

Beispiel: Eine Seite mit Anschrift

oberer Rand des Papiers

(Zeile 1)

(vorgedruckter Bereich)

- Zeile vt1 | Kopfzeilen (z.B. Absenderangaben)
- Zeile vt3 | Anschrift
- Zeile vt4 | Ihr Zeichen, Unser Zeichen usw.
- | evtl. Fußzeilen
- Zeile vt2 | Letzte Textzeile

(vorgedruckter Bereich)

unterer Rand des Papiers

Zu Beginn der Formatierung gilt folgende Standardeinstellung:

.vtab 1 70 10 20

Möglicherweise passt diese Einstellung auf Ihren Drucker und Ihr Papier nicht genau. Um die Zeilennummern für Ihren Drucker und Ihren Briefkopf zu bestimmen, gehen Sie für jede Papiersorte und jeden Drucker Ihrer Installation wie folgt vor:

1. Spannen Sie das Papier in definierter, wiederholbarer Weise ein, z.B. mit der Perforation an der Abrisskante oder einer anderen geeigneten Markierung des Druckers. Schalten Sie den Drucker ein und drucken Sie das FOP **LINEAL** mit dem Kommando **<Text> LINEAL <Zeigen>**

Das Papier wird mit absoluten Zeilennummern "1, 2, 3, ..." bedruckt, so dass für jede Position auf dem Papier die Zeilennummer abgelesen werden kann.

2. Die so bestimmten Zeilennummern werden in folgende FOPs eingetragen:

BLANKO.F	Normales Papier DIN A4-hoch
BRIEF.F	Briefpapier mit Kopfvordruck
VE.F	Vorgänge des Ein- und Verkaufs

In diesen Unterprogrammen finden Sie geräteabhängige Alternativen für **.vtab**-Kommandos. Alle Unterprogramme für FOPs und Briefe nehmen Bezug auf diese formatdefinierenden Bausteine.

3. Sie beginnen also mit einem der Kommandos

.eingabe BLANKO.F

.eingabe BRIEF.F

Wenn Sie eigene FOPs erfassen, sollten Sie den gleichen Weg wählen. Standardmäßig ist für Stammdaten-FOPs Blankopapier und für Briefe Papier mit Briefkopf vorgesehen.

.ZEILE / .LINE - Nächste Ausgabezeile

.ZEILE	.LINE
deutsch	englisch
<code>.ZEILE 'vt1'</code> <code>.ZEILE 'Zeilennummer'</code>	<code>.LINE 'vt1'</code> <code>.LINE 'line number'</code>
<code>.zeile 'vt1'</code> springt zu der durch <code>.VTAB</code> vereinbarten Zeilenposition <code>vt1</code> .	
<code>.zeile 'Zeilennummer'</code> schreibt den nachfolgenden Text in die jeweilige Zeile.	

Die '`Zeilennummer`' muss eine positive ganze Zahl sein. Eine Rückwärtspositionierung auf der aktuellen Seite ist nicht möglich. Ist die '`Zeilennummer`' kleiner als die Nummer der nächsten Ausgabezeile, findet ein Seitenwechsel statt, d.h. die Ausgabe wird auf der nächsten Seite ausgeführt.

Liegt die '`Zeilennummer`' außerhalb des möglichen Ausgabebereichs, (vgl. `.vtab` und `.laenge`), wird der Befehl ohne eine Fehlermeldung ignoriert.

Beispiel: Festlegen der nächsten Ausgabezeile

Als erstes wird eine Ausgabe in Zeile 1 ausgeführt. Die nächste Ausgabe wird in Zeile 5 ausgeführt.

```
.type int xline ? F|defined(U|xline) = false
Display in line 1
.formula U|xline = 5
.line 'U|xline'
Display in line 'U|xline'
```

.SETZE VERWEIS / .SET REFERENCE - Datenübernahmemodus setzen

.SETZE VERWEIS	.SET REFERENCE
deutsch	englisch
<code>.SETZE VERWEIS +</code> <code>.SETZE VERWEIS -</code> <code>.SETZE VERWEIS 'Format'</code>	<code>.SET REFERENCE +</code> <code>.SET REFERENCE -</code> <code>.SET REFERENCE 'format'</code>

.SETZE VERWEIS	.SET REFERENCE
.setze verweis + schaltet den Übernahmemodus ein, d.h. Felder verschiedener Arten werden in speziellen Darstellungen ausgegeben. .setze verweis - schaltet den Übernahmemodus wieder aus; alle Felder werden wie gewohnt ausgegeben. .setze verweis 'Format' bestimmt die Darstellung von Verweisen:	
Format	Darstellung
EXT	Verweise werden dargestellt, wie durch die Art des Feldes definiert. Das ist die Standardeinstellung.
REF	Verweise werden in der ID-Schreibweise, also wie bei der Art ID bzw. VID dargestellt.
NUM	Verweise werden mit Identnummernecho dargestellt, also wie bei der Art P oder VP.
SW	Verweise werden mit Suchwortecho dargestellt, also wie bei der Art PS oder VPS.

Verweisfelder sind Felder der Arten P, PS, VP, VPS, AZ, und AZK. Mit **'Format'** kann man zentral die Darstellung aller Verweisfelder beeinflussen.

Wirkung des Übernahmemodus auf die einzelnen Feldarten

B ZAEO	Für boolesche Felder und für Felder der Art ZAEO wird statt ja/nein (true/false) 1/0 ausgegeben.
alle Verweisarten	Für Verweisfelder wird eine interne Nummer in einem speziellen Format ausgegeben. Je nach Verweisart wird '(Zahl)' oder '(Zahl1, Zahl2)' ausgegeben.
alle Real-Arten	Felder einer Real-Art werden immer im Format [-]Vorkommastellen.Nachkommastellen ausgegeben. Das Vorzeichen wird nur bei negativen Zahlen ausgegeben, Dezimaltrennzeichen ist immer der Punkt.
A* TAZ*	Für Felder mit Aufzählungen wird der interne Code in Klammern ausgegeben: '(Code)' . Der Code ist immer eine ganze positive Zahl.
TN*	Für Freitextfelder wird eine eindeutige Zahl ausgegeben.
GD	Für Datumsangaben wird ohne Uhrzeit im 8-Ziffern-Format, bzw. mit Uhrzeit im 14-Ziffern-Format ausgegeben. Für z.B. 16.06.2016 12:01:00 also ohne Uhrzeit: 20160616 bzw. mit Uhrzeit: 20160616120100.

Verwendung von Variablen bei der Textausgabe

Zur Verwendung von Variablen bei der Textausgabe siehe [Formatierung von Variableninhalten](#) und [Variablensyntax](#).

HTML- und XML-Ausgabe

Die Ausgabe der FO erfolgt normalerweise im multinationalen Abas Zeichensatz. Für das Generieren von HTML- oder XML-Seiten ist das ungeeignet, da Browser diese Codierung normalerweise nicht kennen. Ferner ist das korrekte Behandeln von HTML/XML-Sonderzeichen, z.B. <, schwierig. Daher werden auch HTML-Ausgabe und XML-Ausgabe unterstützt. HTML ist ein standardisiertes Format, das von Fremdprogrammen leicht weiterzuverarbeiten ist. Dateien in diesem Format können auch als E-Mail verschickt werden. Die entsprechenden Dokumente werden über einen Browser angezeigt. Wenn Sie den in Abas integrierten Browser verwenden, können Sie per Klick auf einen entsprechend gestalteten Link auch Abas Kommandos ausführen.

.FORMAT / .FORMAT - Ausgabeformat umschalten

.FORMAT	.FORMAT
deutsch	englisch
.FORMAT HTML XML [-ENCODIERUNG 'Encodierung'] .FORMAT ABAS	.FORMAT HTML XML [-ENCODING 'encoding'] .FORMAT ABAS
.format schaltet das Ausgabeformat auf HTML, XML oder den Standardwert (Abas Zeichensatz).	
Zulässige Werte für Encodierung sind UTF-8 und ISO-8859-1. Ohne Angabe gilt ISO-8859-1.	

Die Angabe eines Ausgabeformats hat keinen Einfluss auf das Ausgabegerät, so dass HTML-Texte z.B. auch auf Bildschirm oder Drucker ausgegeben werden können.

Wahl der Codierung

Aspekt Editierbarkeit	ISO-8859-1 ist in Editoren eventuell besser lesbar, es gibt allerdings auch Editoren, die UTF-8 beherrschen.
Aspekt Dateigröße	Wenn viele westeuropäische Zeichen (Umlaute, Buchstaben mit Akzent usw.) vorkommen, ergibt ISO-8859-1 kleinere Dateien (1 Byte statt 2 Byte pro westeuropäischem Zeichen). Kommen viele nicht-westeuropäische Zeichen vor, ergibt UTF-8 deutlich kleinere Dateien (2-4 Byte statt 8 Byte pro Zeichen).
Aspekt Webserver	U.u. diktiert die Konfiguration des Webserver die Codierung (zumindest, wenn man diese nicht ändern kann oder will), da die Angabe im Dateikopf vom Browser meist ignoriert wird, wenn der Webserver eine andere Angabe macht.



Der Befehl `.formel` kann ausnahmsweise nicht weiter abgekürzt werden, da sonst ein Namenskonflikt mit dem Kommando `.formel` entsteht.

Beenden der HTML- oder XML-Ausgabe

`.format ABAS`

macht die Wirkung des Befehls `.format HTML` oder `.format XML` wieder rückgängig.

`.format XML` - XML-Dokumente generieren

Die Angabe `.format XML [-encoding 'Encodierung']` bewirkt prinzipiell folgendes:

- Ausgabezeichensatz UTF-8 mit `-encoding UTF-8` oder ISO-8859-1 mit `-encoding ISO-8859-1`. Es werden sämtliche Zeichen korrekt abgebildet, nötigenfalls durch eine Unicode-Entity (`&#xnnnn;`). Diese ist Standard und sollte von allen verarbeitenden Programmen (Browser, Suchmaschinen, XML-Parser usw.) korrekt behandelt werden.



Beachten Sie, dass die `charset`-Angabe im Kopf der generierten Datei zur gewählten Encoding-Option passen muss, z.B.

```
<META http-equiv="content-type" content="text/html; charset=ISO-8859-1">.
```

- Unterstützung für die Behandlung von Sonderzeichen, z.B. `<`

Im Detail gilt: Die Ausgabe wird vom Abas Zeichensatz auf UTF-8 oder ISO 8859-1 umgeschaltet und Zeichen mit Sonderbedeutung in XML werden je nach Wunsch gequotet oder nicht. Die notwendigen Befehle entsprechen denen der HTML-Generierung. Jedoch wird nur solcher XML-Code erzeugt, der explizit angegeben wurde. Komfortfunktionen wie die automatische Tabellenerzeugung und die Umsetzung von FO-Befehlen in HTML-Code entfallen. Damit haben Sie als FO-Programmierer vollständige Kontrolle über die Ausgabe, müssen aber auch die notwendige Syntax genau kennen.

Das XML-Feature kann auch für die HTML-Generierung genutzt werden. Das ist sinnvoll, wenn die Komfortfunktionen unerwünscht sind und Sie selbst die Kontrolle über den generierten HTML-Code ausüben möchten.

Die Syntax entspricht jener der HTML-Generierung. Das XML-Format wird gesetzt mit `.format XML`. Zusätzlich kann angegeben werden

`.setze doppelpunkt` + Zeilen, die mit Doppelpunkt beginnen, werden als bereits fertiger XML-Code betrachtet. In den nicht variablen Teilen findet keine Zeichensatzkonvertierung und kein Quoting statt. (Siehe auch `.setze doppelpunkt` - [HTML-Befehle markieren](#).)

In Zeilen, die mit Doppelpunkt beginnen, werden Variableninhalte (und nur diese) gequotet und nach ISO-Zeichensatz umgesetzt. Auch in Formeln werden auf der rechten Seite Variableninhalte gequotet, sofern die Variable in Apostrophe eingeschlossen ist. Quoting bedeutet, dass z.B. das Zeichen `<` durch `<` ersetzt wird (siehe unten).

.setze inline + In der Ausgabe wird keine neue Zeile erzeugt, wenn ein FO-Kommando durchlaufen wird.

Die XML-Generierung wird typischerweise eingeleitet mit dem Befehl **.format XML**

Dieser setzt alle normalerweise verwendeten Modi. Dies sind:



```
.format XML
.setze doppelpunkt +
.setze varquote +
.eingabe DATEI.F
```

Zeichensatzumsetzung

Zeichen des multinationalen Abas Zeichensatzes werden in die entsprechenden Zeichen des Zeichensatzes UTF-8 oder ISO 8859-1 umgewandelt. Im Gegensatz zu **.format HTML** wird Liniengrafik nicht in Ersatzzeichen wie Plus und Pipe (|) umgewandelt. Alle nicht im Zeichensatz UTF-8 bzw. ISO 8859-1 enthaltenen Zeichen werden durch Leerzeichen umgesetzt.

Zeichen mit Sonderbedeutung in XML und ihre Umschreibungen

Die folgenden Sonderzeichen werden, falls gewünscht, automatisch in das entsprechende Quoting umgewandelt:

<	<
>	>
&	&
'	'
"	"

Soll der XML-Parser die Bedeutung von Zeichen ignorieren, gibt es in XML auch die Möglichkeit der Auszeichnung als CDATA-Abschnitte.

```
<abasbaseFeld>
<![CDATA[Liniengrafik |- und <>&-Sonderzeichen]]>
</abasbaseFeld>
```

.format HTML - HTML-Dokumente generieren

Der Befehl **.format HTML** [``-encodierung 'Encodierung'``] bewirkt prinzipiell folgendes:

- Ausgabezeichensatz UTF-8 mit **-encodierung UTF-8** oder ISO-8859-1 mit **-encodierung ISO-8859-1**. Es werden sämtliche Zeichen korrekt abgebildet, nötigenfalls durch eine Unicode-Entity (&#xnnnn;). Diese ist Standard und sollte von allen verarbeitenden Programmen (Browser, Suchmaschinen, XML-Parser usw.) korrekt behandelt werden.



Beachten Sie, dass die **charset**-Angabe im Kopf der generierten Datei zur gewählten Encoding-Option passen muss, z.B.

```
<META http-equiv="content-type" content="text/html;  
charset=ISO-8859-1">.
```

- Automatisches Generieren bestimmter HTML-Tags, um z.B. ohne die Kenntnis der HTML-Syntax eine Tabelle generieren zu können
- Unterstützung für die Behandlung von Sonderzeichen, z.B. <

Folgende Befehle stehen im Zusammenhang mit **.format HTML**:

- **.browser** - URL im Browser anzeigen
- **.blocksatz**, **.flattersatz** und **.tabellensatz** - Textausrichtung
- **.setze inline** - Fließtext beibehalten
- **.setze doppelpunkt +** - HTML-Befehle markieren
- **.setze varquote +|** - HTML-Code in Variableninhalten
- **.tabellensatz** - HTML-Tabellen erzeugen
- Einschränkungen im HTML-Zeichensatz
- HTML-Ausgabe und Seitenverteilung
- HTML-Ausgabe und Seitenwechsel
- Umleitung der Ausgabe von bestehenden Programmen auf Browser
- Erfassen von E-Mails
- Anlickbare Listen

Im Detail gilt folgendes:

- Der Ausgabe wird ein notwendiger Vorspann vorangestellt.
- 8-Bit-Zeichen (z.B. Umlaute) werden in den entsprechenden HTML-Befehl umgewandelt.
- Geschützte Leerzeichen werden in das entsprechende HTML-Zeichen umgewandelt ("nobreaking space").
- Zeichen mit HTML-Sonderbedeutung (&, <, >) werden geschützt.
- Liniengrafik und diverse Akzentzeichen werden nicht mehr auf ASCII-Ersatzzeichen abgebildet, sondern korrekt dargestellt.
- FO-Befehle der Art **.drucken 'Schrifteigenschaft'** werden in entsprechende HTML-Anweisungen umgesetzt.

Beispiel: Schriftfarbe ändern

Dieser FO-Befehl schaltet die Schriftfarbe auf Rot um. Bei Programmende wird an das Dokument der syntaktisch zum HTML-Vorspann gehörende Abschluss angefügt.


```
.print sf.rot
```

Beispiel: Schriftgröße ändern

Dieser FO-Befehl schaltet die Schriftgröße um. Für 'U|xin' muss eine natürliche Zahl eingesetzt werden.

```
.print sg 'U|xin'
```

Dieser Befehl erzeugt den HTML-Code

```
.print sg.5
```

Sie können auch einen eigenen HTML-Code erzeugen, indem Sie ihn, wie im Kapitel [HTML-Befehle markieren](#) beschrieben, in eine Doppelpunkt-Zeile schreiben.

.BROWSER / .BROWSER - URL im Browser anzeigen

.BROWSER	.BROWSER
Deutsch	Englisch
<pre>.BROWSER [-NEU] [-KOMMANDOFENSTER] 'URL' .BROWSER [-KOMMANDOFENSTER] 'URL' - Keine REST API-Unterstützung .BROWSER [-NEU] -DATEI 'filename' - Keine REST API-Unterstützung</pre>	<pre>.BROWSER [-NEW] 'URL' .BROWSER [-COMMANDWINDOW] 'URL' - No REST API support .BROWSER [-NEW] -FILE 'filename' - No REST API support</pre>
<pre>.browser 'URL' zeigt die angegebene URL in einem Browserfenster an.</pre>	

.BROWSER	.BROWSER
 Die Kommandos <code>.browser [-FILE]</code> und <code>.browser [-COMMANDWINDOW]</code> werden ausschließlich im Abas Windows Client unterstützt. Diese Kommandos sind über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. <code>.browser -kommandofenster</code> zeigt die URL nicht in einem Browserfenster an, sondern in der Kommandoübersicht. <code>.browser -datei 'filename'</code> zeigt eine Datei an. 'filename' ist der Name einer Datei im Unterverzeichnis <code>win</code> des Mandantenverzeichnisses. Sie müssen den Pfad relativ zum Mandantenverzeichnis und beginnend mit <code>win/</code> angeben. Mit der Option <code>-neu</code> wird immer ein neues Browserfenster geöffnet.	

In dem Browserfenster wird der in die Abas GUI integrierte `Browser` verwendet. Sie können auch einen externen `Browser` einrichten. Damit die Anzeige in beiden Fällen funktioniert, sollte außer bei `.browser -datei` immer eine vollständige URL angegeben werden, d.h. inklusive des Protokolls (z.B. `http:` oder `file:`).

Wird das FO-Kommando `-neu` mehrfach aufgerufen, dann wird bei dem in die Abas GUI integrierten Browser jeweils dasselbe Browserfenster zur Anzeige verwendet. Um gezielt ein neues Fenster zu öffnen, verwenden Sie die Option `-neu`. Wenn Sie einen externen Browser verwenden, dann ist das Verhalten abhängig vom Browser.

```
.browser http://www.abas.de
.browser -datei "win/vkhtml/Liste.html"
.browser -neu "http://www.abas.de"
.browser -neu -datei "win/vkhtml/Liste.html"
```

Beispiel: `.browser -kommandofenster`

`.browser -kommandofenster` öffnet die URL in der Kommandoübersicht.

```
.browser -kommandofenster "http://www.google.de"
```



Sie können das Kommando `.browser` auch in EDP verwenden. In diesem Fall wird von EDP eine `Statusmeldung (S)` mit der Meldungsart `BROWSER` ausgegeben. Die Optionen `-datei`, `-neu` und `-kommandofenster` sind bei der Verwendung mit EDP nicht

zulässig.

Einfluss von .flattersatz, .blocksatz und .tabellensatz auf die Darstellung im Browser

.flattersatz	Der Text erscheint in Proportionalsschrift, der linke Rand ist bündig, der rechte "flattert". Der Browser formatiert den Absatz so, dass er die gegenwärtig eingestellte Fensterbreite ausfüllt. Die Angabe von Rändern (.links , .links) wird daher im Browser nicht umgesetzt. Es wird nur bei Leerzeichen umbrochen. Durch Bindestriche getrennte Wörter werden nicht wieder zusammengefügt. Geschützte Leerzeichen werden respektiert.
.blocksatz	Im derzeitigen HTML-Standard ist kein Blocksatz vorgesehen. Der Text erscheint genauso wie bei Flattersatz.
.tabellensatz	Der Text erscheint in Schreibmaschinenschrift (fixed font). Der Browser bricht Zeilen nicht um.

Schriftattribute setzen

Mit **.setze inline +** ist es möglich, HTML-relevante Kommandos inmitten eines Absatzes zu platzieren. Lediglich die Kommandos **.absatz**, **.browser** und **.ausgabe** geben dann noch den alten Absatz vollständig aus und leiten einen neuen ein.

Beispiel: Schriftgröße innerhalb der Zeile verändern

```
.justified
.set inline +
The paragraph contains a
.print sg.gross
large
.print sg.norm
word and will be joined by browser.
```

Die beiden **.drucken**-Befehle wirken wegen **.setze inline +** nicht als Absatzende. Im Browser wird der gesamte Text zu einem Abschnitt zusammengefügt und entsprechend der Fenstergröße gesetzt.

.SETZE DOPPELPUNKT / .SET COLON - HTML-Befehle markieren

.SETZE DOPPELPUNKT	.SET COLON
deutsch	englisch
.SETZE DOPPELPUNKT +	.SET COLON +

.SETZE DOPPELPUNKT	.SET COLON
setzt den Doppelpunkt als Interpretationszeichen für HTML-Befehle	

Abas setzt ausgaberelevante Befehle in entsprechende HTML-Befehle um, so dass keine HTML-Kenntnisse erforderlich sind und auch keine eigenen HTML-Befehle eingefügt werden müssen. Darüber hinaus kann aber auch in HTML programmiert werden, um weitergehende Funktionen wie Hyperlinks und die Einbindung von Grafiken oder HTML-Formularen zu realisieren.

HTML-Befehle dürfen nicht einfach ins FOP geschrieben werden, da sie sonst nur als Klartext erscheinen, statt ausgeführt zu werden. Zeilen mit HTML-Befehlen müssen mit einem Doppelpunkt eingeleitet werden. Der Doppelpunkt hat normalerweise eine andere Funktion (siehe [Doppelpunkt bei .drucken und .zeigen](#)). Damit er die Aufgabe der HTML-Markierung übernimmt, muss zuvor das Kommando **.setze doppelpunkt +** ausgeführt werden.

Abas ersetzt in den mit Doppelpunkt begonnenen Zeilen auch Variablen und setzt Acht-Bit-Zeichen in HTML-Ausdrücke um. Die Metazeichen <, > und & werden unmarkiert in der Ausgabe des FO-Programms belassen.

```
.setze doppelpunkt +
:Informationen zu Abas finden Sie auf der <A
HREF="http://www.abas.de">ABAS-Homepage</A>.
```

Informationen zu Abas finden Sie auf der [ABAS-Homepage](http://www.abas.de).

Abas Variablen, die innerhalb von Anführungszeichen (doublequotes) stehen, werden nicht ersetzt. Um dies zu vermeiden, können Sie mit der Konstanten **G|dblquote** arbeiten.

```
<A HREF="mailto:info@abas.de">info@abas.de</A>
```

funktioniert nicht. Richtig ist

```
<A HREF='G|dblquote 'mailto:info@abas.de'G|dblquote '>info@abas.de</A>
```

Die Korrektheit der selbst eingefügten HTML-Befehle wird von Abas nicht überwacht.

.SETZE VARQUOTE / .SET VARQUOTE - HTML-Code in Variableninhalten

.SETZE VARQUOTE	.SET VARQUOTE
deutsch	englisch
.SETZE VARQUOTE + .SETZE VARQUOTE -	.SET VARQUOTE + .SET VARQUOTE -

.SETZE VARQUOTE	.SET VARQUOTE
.setze varquote + bewirkt, dass in Zeilen, die mit Doppelpunkt beginnen, HTML-relevante Zeichen in Variableninhalten gequotet, also nicht interpretiert werden. .setze varquote - bewirkt, dass in Doppelpunktzeilen HTML-Code in Variableninhalten interpretiert wird. (Standard)	

Zeilen mit selbst eingefügtem HTML-Code müssen mit einem **Doppelpunkt** beginnen, damit der HTML-Code interpretiert wird. Wenn diese Zeilen allerdings auch Abas Variablen enthalten, in denen zufällig HTML-relevante Zeichen vorkommen, können Fehler in der HTML-Syntax entstehen. Ein vorher durchlaufenes **.setze varquote +** bewirkt, dass HTML-relevante Zeichen gequotet und nicht interpretiert werden.

Das Zeichen < hat in HTML eine Sonderbedeutung. Wenn der Variableninhalt **3<4** ausgegeben werden soll, muss < durch **<** ersetzt (gequotet) werden.

Auch in **generierten HTML-Tabellen** kann **eigener HTML-Code** benutzt werden (z.B. ein Hyperlink oder ein anderer Font in einer Tabellenzelle, siehe **Beispiel**). Da Doppelpunktzeilen während einer Tabellengenerierung aber als Tabellenzellen betrachtet und nicht wie sonst als HTML ausgegeben werden, sorgt **.setze varquote -** dafür, dass der Variableninhalt nicht gequotet, sondern als HTML interpretiert wird.

.setze varquote +| - wirkt auch in Formeln, wenn die Variable in Apostrophe gesetzt wird. Die Option hat nur bei **.format HTML** eine Bedeutung.

.TABELLENSATZ / .TABLERECORD - HTML-Tabellen erzeugen

.TABELLENSATZ	.TABLERECORD
deutsch	englisch
.TABELLENSATZ .TABELLENSATZ TITEL .TABELLENSATZ UEBERSCHRIFT .TABELLENSATZ ZEILE .TABELLENSATZ ENDE .TABELLENSATZ UNFORMATIERT .TABELLENSATZ SPALTE	.TABLERECORD .TABLERECORD TITLE .TABLERECORD HEADLINE .TABLERECORD LINE .TABLERECORD END .TABLERECORD UNFORMATTED .TABLERECORD COLUMN
.tabellensatz beendet einen Flatter- oder Blocksatz. Mit .tabellensatz titel, .tabellensatz ueberschrift, .tabellensatz zeile, .tabellensatz ende, .tabellensatz unformatiert und .tabellensatz spalte wird die HTML-Ausgabe einer Tabelle gesteuert.	

Alle Programme, die Tabellen ausgeben, können weiterhin verwendet werden. Höheren ästhetischen Ansprüchen genügt die Browserausgabe nicht, da sie in Schreibmaschinenschrift (fixed font) erfolgen

muss, damit untereinander gehörende Texte tatsächlich untereinander gesetzt werden. Außerdem sind die Tabellenüberschriften nicht hervorgehoben und es werden keine Tabellenlinien gezeichnet, es sei denn, das FOP sorgt selbst dafür.

Mit kleinen Anpassungen können aber HTML-Tabellen generiert werden, die vom Browser unterstützt werden. Dieser passt z.B. die Formatierung der einzelnen Tabellenzellen so an, dass die Tabelle möglichst in das gerade eingestellte Fenster passt. Im FOP muss nur markiert werden, wo die Tabelle beginnt und endet und was Tabellenüberschrift und Tabellenzeile ist.

Bestandteile der Tabelle markieren

Zum Markieren der Bestandteile einer Tabelle werden folgende Optionen des Kommandos

`.tabellensatz` benutzt. Alle Optionen werden bei der Ausgabe im Abas Format ignoriert, sind aber unschädlich.

<code>.tabellensatz titel</code>	leitet die Tabelle ein. Was in der folgenden Zeile steht, wird der Titel der Tabelle. Eine Tabelle muss keinen Titel haben, sie kann auch durch die folgenden Befehle eingeleitet werden.
<code>.tabellensatz ueberschrift</code>	Was in der folgenden Zeile ausgegeben wird, ist die Überschrift der Tabelle. Sie wird vom Browser fett dargestellt. Eine Tabelle muss keine Überschrift haben.
<code>.tabellensatz zeile</code>	Was in den folgenden Zeilen ausgegeben wird, wird als normale Tabellenzeilen interpretiert. Es dürfen aber weiterhin mit <code>ueberschrift</code> gekennzeichnete Zeilen folgen, z.B. um in großen Tabellen in regelmäßigen Abständen eine Zwischenüberschrift einzuschalten.
<code>.tabellensatz ende</code>	zeigt an, dass die Tabelle beendet ist. Diese Angabe sollte nicht weggelassen werden.

Spalteneinteilung der Tabelle

Abas erkennt Tabellenspalten daran, dass sie durch Leerzeichen voneinander getrennt sind. Sollen Leerzeichen ausnahmsweise nicht als Spaltentrenner wirken, müssen sie durch geschützte Leerzeichen ersetzt werden. Dazu wird normalerweise der Unterstrich verwendet. Eine FO-Variable erzeugt immer nur eine Spalte, auch wenn sie Leerzeichen zum Inhalt hat.

Links- und Rechtsbündigkeit und Mehrzeiligkeit von Variablen werden auch bei der Browserausgabe unterstützt und können wie gewohnt angegeben werden. Deutlich einfacher ist es, HTML-Tabellen als Tabellen für die Bildschirmausgabe zu programmieren:

- Der Browser rechnet selbst die optimale Spaltenbreite aus.
- Die Einschränkung auf eine maximale Gesamtbreite von 132 Zeichen besteht nicht.
- Links- oder rechtsbündige Felder nehmen nicht drei Zeichen mehr Platz ein als der Variablenname.
- Spaltenüberschriften und Spalteninhalte müssen nicht übereinander stehen.
- Wird eine zu geringe Feldbreite angegeben, so ist dies unschädlich. Von jeder Variablen wird der gesamte Inhalt angezeigt; der Browser bricht bei zu großem Inhalt die Spalte um.

In Tabellen eigenen HTML-Code integrieren

Tabellen werden von Abas automatisch erkannt und in HTML umgesetzt. Spalten werden anhand von Leerzeichen erkannt. Soll eigener HTML-Code in eine Tabelle aufgenommen werden, so sollte dieser zusammen mit dem auszugebenden Variableninhalt in eine neue Variable geschrieben werden. Zu diesem Zeitpunkt muss `.setze varquote +` durchlaufen sein. Die Tabellenzeilen müssen mit einem Doppelpunkt beginnen, damit der HTML-Code durch Abas nicht wieder entfernt wird. [Doppelpunktzeilen](#) werden während einer Tabellengenerierung nicht wie sonst als HTML ausgegeben, sondern als Tabellenzeile betrachtet.

In einer Tabelle soll der Wert in Spalte 2 rot eingetragen werden, wenn dieser kleiner als null ist.

```
.output new TESTFILE.HTML
.input DATEI.F
..
.format html
.set varquote -
.set colon +
..
.formula x1 = 2
.formula x2 = -10
.type text xwert1      ? _F|defined(U|xwert1)
.type text xwert2      ? _F|defined(U|xwert2)
..
.tablerecord title
my test table
.tablerecord headline
name                value
.tablerecord line
.formula U|xwert1 = 'U|x1'
.formula U|xwert1 = "<FONT COLOR=RED>" + 'U|x1' + "</FONT>" ? x1 < 0
:one                ' U|xwert1'
.tablerecord line
.formula U|xwert2 = 'U|x2'
.formula U|xwert2 = "<FONT COLOR=RED>" + 'U|x2' + "</FONT>" ? x2 < 0
:two                ' U|xwert2'
.tablerecord end
```

Öffnen Sie zur Kontrolle die Datei **TESTFILE.HTML** in einem Browser.

Alle Formatierungen abschalten

.tabellensatz unformatiert

schaltet auch die Formatierungen der FO ab, die im Tabellensatz noch wirksam sind.

Damit haben Sie vollständige Kontrolle über die Ausgabe, was beim Generieren bestimmter Dateien (z.B. für Fremdprogramme) sehr wichtig sein kann:

- Bei Variablen der Arten DVM12 und DVMR2 wird der Dateiinhalt mehrzeilig ausgegeben.

- Die Angaben zum linken und rechten Rand sind wirkungslos.
- Es werden keine Zeilenumbrüche erzwungen.
- Es können beliebig breite Zeilen ausgegeben werden.
- Es wird keine Seitenformatierung vorgenommen (Kopf- und Fußzeilen).
- Die bündigen Schreibweisen, z.B. 'x', haben keine Sonderbedeutung.
- Mehrzeilige Variablen werden nicht wie üblich gesetzt, sondern so, wie sie ausgegeben sind, wobei anstelle 'G|trennerchar' ein Line Feed (\n) erscheint.

```
.tablerecord unformatted
.type text xtttext      ? F|defined(U|xtttext) = false
.formula U|xtttext = "line 1;line 2"
?BEGIN 'U|xtttext' CENTER 'U|xtttext' ' END
```

display:

```
BEGIN line 1
line 2 CENTER line 1
line 2 END
```

Durchgehende Spaltentrenner erzeugen

.tabellensatz spalte

erzeugt durchgehende Spaltentrenner. Werden durch mehrzeilige Textvariablen Folgezeilen erstellt, werden diese statt mit Leerzeichen mit den konstanten Inhalten der FO-Zeile aufgefüllt.

```
.tablerecord column
.type text xtx xty
.formula U|xtx = "line 1;line 2"
.formula U|xty = "value2"
?| 'U|xtx' ' | 'U|xty' ' |
```

display:

```
| line 1 | value2 |
| line 2 |         |
```

Einschränkungen im HTML-Zeichensatz

Im HTML-Zeichensatz gibt es keine Liniengrafik. Liniengrafik wird deshalb durch möglichst ähnliche Zeichen

umgesetzt. Zum Beispiel wird der senkrechte Strich durch Pipe (|) wiedergegeben.

HTML-Ausgabe und Seitenverteilung

Im derzeitigen HTML-Standard gibt es keinen Seitenbegriff. Alle FO-Befehle bezüglich der Seitenverteilung werden deshalb ignoriert. Der Befehl **.absatz** wird dagegen respektiert. Oben- und Unterpuffer werden nicht ausgegeben. Wenn Sie vom Browser aus drucken, gibt es keine Möglichkeit, Seitenumbrüche anzugeben. Diese hängen vom Drucker, dem Browser und der konfigurierten Schriftgröße ab.

HTML-Ausgabe und Seitenwechsel

Im Gegensatz zur gedruckten Ausgabe, bei der alle Informationen auf ein Papier bestimmter Größe aufgeteilt werden müssen, brauchen bei der Ausgabe der Informationen auf dem Bildschirm keine Seitengrenzen eingehalten zu werden. Sie können mit den Funktionstasten oder dem Scrollbalken Ihres Browsers fortlaufend und beliebig weit nach oben und unten gehen. Daher gibt es in HTML keinen Befehl für einen Seitenwechsel (page break). Neue Druckseiten oder neue Fenster (neue Webseiten) erreichen Sie durch den Wechsel auf eine neue HTML-Datei.

Innerhalb von HTML-Dateien (Webseiten) können Sie für Trennlinien (die dem Seitenumbruch beim Ausdruck entsprechen) den Befehl `<hr>` oder die Befehlsfolge `</p> <hr> <p>` verwenden. In Layoutvorlagen besteht darüber hinaus die Möglichkeit, Stellen zu kennzeichnen, die beim Druck auf einer neuen Seite beginnen sollen, oder - umgekehrt - solche Bereiche zu markieren, die möglichst nicht durch Seitenwechsel getrennt werden sollen. Diese Möglichkeiten werden aber von den meisten Webbrowsern noch nicht unterstützt.

Umleitung der Ausgabe von bestehenden Programmen auf Browser

Bei bestehenden Programmen kann die Ausgabe nicht ohne weiteres auf Browser umgeleitet werden. Es ist möglicherweise erforderlich, kleinere Anpassungen vorzunehmen. Dabei muss Folgendes festgelegt werden:

- Der Zeitpunkt, zu dem die bisherige Ausgabe im Browser tatsächlich angezeigt werden soll
- Welche Ausgaben nicht per Browser, sondern weiterhin auf dem Bildschirm erfolgen sollen
- Ob bei einem Wechsel des Ausgabemediums (z.B. Datei) die Ausgabe weiterhin im HTML-Format erfolgen soll

Da nicht garantiert werden kann, dass in jedem Fall das vom Benutzer gewünschte Ergebnis erscheint, wurde die Ausgabe auf Browser nicht in das Standard-Medienwahlmenü (FOP "DRUCKER") aufgenommen. Mitgeliefert wird aber ein erweitertes Medienwahlmenü mit dem Namen DRUCKER2, das die Ausgabe auf Browser als weiteren Unterpunkt enthält. Dieses Unterprogramm kann in allen FOPs aufgerufen werden, die entsprechend angepasst wurden. Damit stehen, anders als bei der Ausgabe auf einen ASCII-Bildschirm, sämtliche Features zur Verfügung, die der Browser anbietet und zukünftig anbieten wird:

- Einbau von Grafiken
- Einbau nützlicher Hyperlinks (Verweise auf andere Seiten)
- Start weiterer Auswertungen per Mausklick
- Erfassen von E-Mails
- Übersichtlichere und attraktivere Gestaltung der Ausgabe durch Verwendung verschiedener Schriftarten, Schriftgrößen und Farben
- Erzeugung "echter" HTML-Tabellen ohne eigene Formatierung
- Darstellung von Verweiskfeldern als Links, über die der Benutzer in ein Fenster mit diesem Objekt

navigieren kann

- Definition von FOP-Verweisen, die ebenfalls als Link dargestellt werden und bei deren Verfolgung ein FOP gestartet wird, dessen Ausgabe im Browser betrachtet werden kann
- Start von Suchmaschinen mit situationsabhängigen Suchbegriffen
- Anfordern von Benutzereingaben über HTML-Formulare

Der Endbenutzer hat nun folgende Möglichkeiten:

- Ausdruck von Grafiken und Inline-Attributen
- Suchen und scrollen in der Ausgabe
- Setzen von Lesezeichen auf häufig benötigte Auswertungen
- Verfolgen von Hyperlinks
- Sichern der Ausgabe als Datei
- Nutzung der Tasten <Pfeil links> und <Pfeil rechts>, um zu früheren Auswertungen zurückzukehren

Ausgabe von FOPs auf Browser

Alle FOPs, die auf Browserausgabe umgestellt werden, beinhalten ein **.eingabe BROWSER.INI**. Die Initialisierungsdatei **BROWSER.INI** dient der Übergabe von Parametern an die Druckerdatei und befindet sich im **\$MANDANTDIR**.



Für die Browserausgabe müssen alle individuellen Druckerdateien (DRUCKER*) angepasst werden. Als Vorlage kann z.B. **\$HOMEDIR/refI/DRUCKER*** dienen.

Unterschied zur Bildschirmausgabe

Die Ausgabe auf Browser weist einen wichtigen Unterschied zur Ausgabe auf den Bildschirm auf: Sie erfolgt nicht zeilenweise, da Browser derzeit keine entsprechende Schnittstelle anbieten. Stattdessen wird die Ausgabe zunächst in einer temporären Datei gesammelt. Diese Datei wird im Browser angezeigt, wenn das FOP eine entsprechende Anweisung durchläuft (vgl. **.browser**). Das sollte normalerweise erst dann geschehen, wenn das Programm die Ausgabe zu einem gewissen Abschluss gebracht hat.

Nachteilig wäre es, wenn im FOP öfter **.browser**-Anweisungen vorkommen würden. Zum einen würde es beim Lesen eines Dokumentes stören, wenn das Dokument plötzlich gelöscht und wieder neu aufgebaut wird. Zum anderen ist die Datei während ihres Entstehens nicht unbedingt zu jedem Zeitpunkt syntaktisch vollständig. Bis der Browser das Ende eines bestimmten Strukturelementes (z.B. einer Tabelle) durchlaufen hat, kann er es nicht wie gewünscht darstellen.

Programme, die ausschließlich auf Browser ausgegeben werden

Sollen Programme ausschließlich auf Browser ausgegeben werden, so braucht das Unterprogramm **drucker** nicht aufgerufen zu werden. Stattdessen kann das Programm wie folgt aufgebaut sein:

```
.input html  
..
```

```
.. Program ...  
..  
.input browser
```

Die Unterprogramme **HTML** und **BROWSER** werden mitgeliefert. **HTML** sorgt für die Ausgabe im HTML-Format, die auf eine Temporärdatei erfolgt. **BROWSER** zeigt diese Temporärdatei an.

Druckerauswahl bei Listen mit Browserausgabe

Bei der Browserausgabe kann mit Hilfe eines HTML-Formulars der Drucker ausgewählt werden.

```
.set colon +  
:<form action="exp:<text> PLAN %PARAM <zeigen>" method=post>  
..  
.. HTML text formular  
..  
: <select name="xpargeraet" size=1>  
.input htmldrucker  
: </select>  
..  
.. remaining HTML-Text formular  
..  
:</form>  
.set colon -
```

Erfassen von E-Mails

Das Erfassen von E-Mails wird unterstützt, indem in die Ausgabe des FOP-Programms ein E-Mail-Hyperlink in der bekannten HTML-Syntax eingefügt wird.

```
<A HREF="mailto:'email'"> Nachricht erfassen </A>
```

Wenn Sie auf den Link klicken, wird das E-Mail-Formular des Browsers gestartet. Der Adressat ist bereits angegeben. Das Textfeld der E-Mail ist nicht vorbelegt, da die Browser dafür keine Schnittstelle anbieten. Der Text kann aber z.B. vom FOP ausgegeben werden, so dass Sie ihn nur noch mit Cut&Paste übertragen müssen. Daneben können Sie auch einfach eine Abas Ausgabe im HTML-Format als Anlage (Attachment) verschicken.

Anklickbare Listen

Bei der Ausgabe auf einen Browser können HTML-Links generiert werden. Wenn Sie den in die Abas GUI integrierten Browser verwenden, führen Sie ein Abas Kommando aus.

```
.type text xstartid
```

```
.type text xstart
.type text xtdat
.type text xtkom
.type text xttip
.type text xtlink
..
.select parts "401"
.continue END ? G|success = false
.formula U|xstartid = H|id
.formula U|xstart = H|swd
.formula U|xtdat = "(Part)"
.formula U|xtkom = "(View)"
.formula U|xttip = "View article"
.formula U|xtlink = "<a href='exp:<" + U|xtdat + ">" + U|xstartid + "<" + U|xtkom + ">"
title='" + U|xttip + "'>" + U|xstart + "</a>" ? G|usingIntegratedBrowser
'U|xlink'
!END
.end
```



Die Angaben

`.formel U|xtlink ..... ? G|usingIntegratedBrowser`
 müssen in einer Zeile erscheinen.

Im Beispiel wird eine Maske geöffnet, in der der Artikel angezeigt wird, der gerade im `.hole`-Puffer geladen ist. Wenn sich die Maus über dem Link befindet, wird der Text "Artikel zeigen" als Tooltip angezeigt. Als Text des Links wird das Suchwort des Artikels angezeigt.

Syntax der HTML-Links

`exp: 'Abas Kommando'`

Zum Beispiel `exp:<(Part)> (123,2,0) <(View)>` oder `exp:<(Sales)> <(View)>`.



Diese HTML-Links dürfen nur ausgegeben werden, wenn der in die Abas GUI integrierte Browser verwendet und die FO-Ausgabe im Browser angezeigt wird (d.h. die Variablen `'browseraus'` aus dem Lieferprogramm `drucker` und `'G|usingIntegratedBrowser'` stehen beide auf `G|true`). Andere Browser verstehen HTML-Links nicht, die mit `exp:` beginnen.

Siehe auch [Konfiguration eines externen Browsers](#).

Substitutionen - Schützen von Zeichen vor Interpretation

Gelegentlich ist es notwendig, Zeichen innerhalb eines Textes so zu markieren, dass sie z.B. nicht als Text- sondern als Steuerzeichen gewertet werden. Diese Zeichen müssen also substituiert werden. Andererseits müssen diese Substitutionen dann wieder aufgehoben werden, wenn z.B. nach ihnen gesucht werden soll, der Suchalgorithmus jedoch eben diese Zeichen ebenfalls als Steuerzeichen interpretiert.

Ein Beispiel für Substitutionen ist die Funktion "Normtext". Abas wandelt mit dieser Funktion ß in ss, ä in ae und Klein- in Großbuchstaben um. Jede Substitution ist prinzipiell umkehrbar. Dabei entsteht aber nicht unbedingt der Ausgangstext, wie das Beispiel der Rückübersetzung von ae in ä zeigt. Hierbei wird aus dem Namen "Michael" das Wort "Michäl".

Weitere Substitutionen sind notwendig z.B. bei Dateien der Formate amk, html, csv.

In den folgenden Artikeln geht es um das Formulieren eindeutiger umkehrbarer Substitutionen.

Methoden zur Substitution von Sonderzeichen

Es gibt zwei Standardmethoden, um Zeichen zu schützen: das Voranstellen eines Schutzzeichens und die Ersetzung durch einen numerischen Wert.

Schutz durch Voranstellen eines Schutzzeichens

Ein Beispiel für den Schutz durch Voranstellen ist der Schutz für Sonderzeichen der Matchcodeerkennung (+?*(\)/^_`+). Diese Sonderzeichen lassen sich durch Voranstellen eines Backslashes (\) ihrer Sonderbedeutung entheben. Das Schutzzeichen selbst wird ebenfalls durch Voranstellen eines Backslashes (\), also durch Verdoppelung des Zeichens, geschützt. Nach der Interpretation der Sonderzeichen kann der vorangestellte Backslash (\) wieder entfernt werden.

Schutz durch numerische Ersetzung

Beispiele für die Ersetzung durch einen numerischen Wert gibt es in vielen Programmiersprachen:

- Der Backslash (\) hat die Nummer 92 im ASCII-Code.
- Er kann in einem String in C oder Java durch \134 oder \u005C codiert werden.
- In C oder Java ist aber auch ein Schutz durch Verdoppelung \\ möglich.

Die Substitution durch ein Schutzzeichen ist leicht lesbar und einfach zu schreiben. Sie muss an jeder Stelle mit interpretierten Sonderzeichen beachtet werden. Die Substitution durch eine Nummer ist dagegen universell (alle Zeichen lassen sich schützen). Sie ist mühsamer zu schreiben (man braucht eine Codetabelle [s3code.gif](#)) und schwerer zu lesen. Die zu schützenden Zeichen werden komplett aus dem Text entfernt; Sie können den Text beliebig einbetten.

Lesen Sie hierzu auch Kapitel [Abas Zeichensatz und Dateienkonvertierung](#).

Realisierung der Substitutionsmethoden

Für den Schutz durch **Voranstellen** eignet sich eine Tabelle mit Paaren von Zeichenketten, die aus dem Original und der zugehörigen geschützten Zeichenkette bestehen.

Bei der **numerischen Methode** wird einfach das Verfahren benannt.

Zusätzlich muss beim Schützen angegeben werden, welche Zeichen ersetzt werden sollen.

Im Programm wird eine Schreibweise benötigt, um das Substitutionsverfahren auf einen Text anzuwenden zu können. Das geschieht im Formelkontext durch den Operator *, bei der Ausgabe durch **.setze** -Anweisungen nur für die Variablenzugriffe oder für die gesamte Ausgabezeile.

Aufzählen der Ersetzungen

Mit diesem Verfahren können Sie die Ersetzung von ß durch ss, ä durch ae usw. oder den Schutz und Freilegung von Sonderzeichen gegen die Matchcodeerkennung implementieren. Eine Substitution durch Aufzählung wird durch die Angabe einer Substitutionsvorschrift definiert, z.B. durch **+LX/ß/ss/ /ä/ae/**

In dieser Substitutionsvorschrift bedeuten

- **+LX**: die Substitution durch Aufzählung von links nach rechts.
- **/**: Das erste Zeichen nach **+LX** ist das Trennzeichen.
- **/muster/wert/**: Das erste und zweite Trennzeichen begrenzen das zu ersetzende Muster, das zweite und dritte den einzusetzenden Wert.

Im angeführten Beispiel einer Substitutionsvorschrift soll also ß durch ss und ä durch ae ersetzt werden.

Wenden Sie diese Ersetzung auf den Text "häßlich" an, so entsteht "haesslich". Ersetzen Sie **+LX** durch **-LX**, so geschieht dies von rechts nach links. Aus "haesslich" wird dann wieder "häßlich", aber aus "Tasse" wird dann "Taße" und aus "Michael" wird "Michäl". Die Muster werden immer der Reihenfolge nach von links nach rechts auf **vollständige Übereinstimmung** durchprobiert. Das erste übereinstimmende Muster wird ausgeführt.

Eine andere Substitutionsvorschrift könnte lauten **+LX/A/X/ /Ab/Auf/**. Das zweite Muster wird hier nie wirksam, weil vorher schon das erste Muster auf "A" matcht und A durch X ersetzt. So wird zum Beispiel bei der Anwendung auf den Text Ablage aus "Ablage" "XblXge".

Dagegen hat die Substitution **+LX/Ab/Auf/ /A/X/** das gewünschte Resultat: Aus "Ablage" wird "Auflage", aus "Aerxes" wird "Xerxes".

Zusammenfassung

Die Reihenfolge der Muster ist von Bedeutung: Haben zwei Muster denselben Anfang, muss das längere immer zuerst angegeben werden, weil es sonst nie auf einen Match trifft.

In diesem Beispiel wird das zweite Muster durch das erste Muster verdeckt, so dass es niemals verwendet wird.

```
.type text xtoriginal xtchanged
.assign U|xtoriginal = "Ablage"
.assign U|xtchanged = 'U|xtoriginal' * "+LX/A/X/ /Ab/Auf/"
original: 'U|xtoriginal'
changed: 'U|xtchanged'
```

Die Substitution erfolgt nach folgendem Verfahren: Alle Muster werden mit dem Text verglichen. Wird ein passendes Muster erkannt, wird dieser Text entfernt und durch den Inhalt "Wert" ersetzt. Es geht weiter bei 1 mit dem ersten Zeichen hinter der Ersetzung. Wird kein passendes Muster erkannt, wird bei 1 mit dem nächsten Zeichen fortgesetzt.

Die Ersetzung von Schutz durch Aufzählung lässt sich beim Schutz von Sonderzeichen gegen Matchcodeersetzung anwenden. Die Sonderzeichen **? * () / ^ _** haben Bedeutung bei der Matchcodeerkennung. Sie müssen durch Voranstellen des Backslashes (****) geschützt werden. Auch ein einzelner Backslash (****) kann so geschützt werden:

```
.type text xtoriginal xtchanged
.assign U|xtoriginal = "Text?without\protection*(additional characters)"
.assign U|xtchanged = 'U|xtoriginal' * "+LX,?,\?, ,*,\*, ,(,\(, ),\), ,/,\/,
,^\,\^, ,_,\_ ,\,\,\"
original: 'U|xtoriginal'
changed: 'U|xtchanged'
```

Beachten Sie, dass hier das Komma als Trennzeichen verwendet wird.

Das erste Sonderzeichen nach LX definiert das Trennzeichen.

Beispiel: Aufzählen der Ersetzungen

Sammlung von Beispielen zum Substitutionsverfahren "Aufzählen der Ersetzungen"

+LX/ß/ss/ /ä/ae/	aus häßlich wird haesslich
-LX/ß/ss/ /ä/ae/	aus haesslich wird häßlich aber:
aus Tasse wird Taße und aus Michael wird Michäl	+LX/A/X/ /Ab/Auf/
Zweites Muster wertlos: aus Ablage wird XblXge	+LX/Ab/Auf/ /A/X/
Aus Ablage wird Auflage, aus Aerxes wird Xerxes	+LX,*,*, ,(,\(,),\), ,_,_ ,^\,\^,

Syntax der Angabe von Substitutionen

Bei der Substitution durch Aufzählung gilt folgende Syntax: **+ oder - LX Ersetzungsliste**

Dabei meinen + und - das ASCII-Plus bzw. -Minus. "Ersetzungsliste" ist eine Liste der Form: **Trennzeichen - Muster - Trennzeichen - Wert - Trennzeichen** mit beliebiger Wiederholung der Elemente.

Bei -LX ist die Bedeutung von Muster und Wert vertauscht.

Als Trennzeichen kann ein beliebiges ASCII-Zeichen dienen. **Das erste Zeichen nach LX wird als Trennzeichen verwendet.** In den Elementen Muster und Wert darf kein Trennzeichen vorkommen. Ist dies dennoch der Fall, wird auf diese Trennzeichen die Abas Standardsubstitution **-#X** angewandt.

Es wird eine Substitutionsvorschrift **+LX///slash:/** erstellt, die das Zeichen / durch den Text **slash:** ersetzen soll. Als Trennzeichen wurde der Slash (/) gewählt, der aber auch als Muster verwendet wird. Da das verboten ist, muss der Slash (/) als Muster durch seine Abas Standardsubstitution ersetzt werden. Diese heißt **#2FX**. Im Muster wurde zusätzlich auch der Doppelpunkt durch seine Abas Standardsubstitution **#3AX** ersetzt. Die korrekte Formulierung der Substitutionsvorschrift erfordert deshalb die Schreibweise **+LX/#2FX/slash#3AX/**

```
.type text xtoriginal xtchanged
.assign U|xtoriginal = "C:/tmp/files"
.assign U|xtchanged = 'U|xtoriginal' * "+LX/#2FX/slash#3AX/"
original: 'U|xtoriginal'
changed: 'U|xtchanged'
```

Abas Standardsubstitution durch numerische Ersetzung

Mit diesem Verfahren lassen sich alle Zeichen ersetzen und alle bis auf insgesamt 18 Zeichen aus dem Text endgültig entfernen. Die Bedeutung als Abas Standardschutz erhält das Verfahren dadurch, dass es an Stellen, an denen Sonderzeichen in Abas interpretiert werden, zum Schutz genau dieser Sonderzeichen eingesetzt werden kann.

Der Abas Standardschutz ersetzt jedes Zeichen durch folgende Sequenz: - **hexadezimal codierte Nummer - X. Die hexadezimal codierte Nummer setzt sich aus Ziffern und einem der Großbuchstaben A bis F zusammen. Sie ergibt sich aus dem Unicode-Zeichensatz. Versehen mit den Zeichen "" und "X" wird so aus dem Text ,A folgende Abas Standardsubstitution: 2CX#41X.**

Dabei hat das Komma im Unicode-Zeichensatz den Code 2C und der Buchstabe A den Code 41. Von Interesse ist diese Ersetzung aber in der Regel nur bei ASCII-Sonderzeichen, da diese in vielfältiger Weise interpretiert werden. **Die Substitutionsregel +#X wendet dieses Ersetzungsverfahren auf alle ASCII-Sonderzeichen an.** Aus [.trans]das:ist's# wird das#3AXist#27Xs. Auch hier kann die Ersetzungsrichtung durch -#X umgekehrt werden. Alle korrekt gebauten Sequenzen der passenden Form, also z.B. -#3AX, werden durch das entsprechende Sonderzeichen ersetzt. Genau diese Funktion wird nun in Abas an allen Stellen verwendet, an denen bestimmte Sonderzeichen in Werten eine besondere Bedeutung tragen. Aus dem im ersten Beispiel erzeugten Text #2CX#41X wird durch die Anwendung von -#X also wieder ,A.

Um nicht alle Sonderzeichen zu entfernen oder auch andere Zeichen wie z.B. nur den Buchstaben A zu ersetzen, kann die Zeichenmenge, auf die die Ersetzung wirken soll, auch explizit angegeben werden: #X='a=. Die zu ersetzende Zeichenmenge steht dabei in Gleichheitszeichen. Im Beispiel #X='a= sollen nur die Sonderzeichen Plus und Apostroph sowie der Buchstabe A ersetzt werden.

```
.type text xtoriginal xtchanged
.assign U|xtoriginal = "das:ist's"
.assign U|xtchanged = 'U|xtoriginal' * "+#X=+'a="
original: 'U|xtoriginal'
changed: 'U|xtchanged'
```

Hier wird aus das:ist's durch die Anwendung von #X=+'a= ` das Literal [.trans]#`d#61Xs:ist#27Xs`#. + Nur der Apostroph `` und der Buchstabe `a` werden also jetzt noch ersetzt. + Eine Zeichenmenge darf auch bei ` -#X` angegeben werden, z.B. ` -#X='a=. Die Zeichenmenge wird dort aber immer ignoriert, das heißt, dass alle korrekten Schutztexte ersetzt werden.

Die numerische Abas Standardsubstitution hat die Syntax: **+|- #X Zeichenmenge**
 Die Zeichenmenge kann fehlen. Sie sieht wie folgt aus:

```
\= [CPaA08mp] [ - { einzelzeichen } ] [ + { einzelzeichen } ] [ # einzelzeichen einzelzeichen ] =
```

Einzelzeichen sind alle ASCII-Zeichen außer Plus, Minus und Raute (#). Die beiden Einzelzeichen nach der Raute (#) erlauben es, die Zeichen **#X** durch andere zu ersetzen. Die Buchstaben stehen für größere Gruppen von zu schützenden Zeichen:

C	control character: 0-037, 0177, die ASCII-Steuerzeichen.
P	ASCII-Interpunktionszeichen: !"#\$%&'()*+,-./:;<=>?@[\\]^_`{ }~
a	ASCII-Kleinbuchstaben a-z
A	ASCII-Großbuchstaben A-Z
0	Ziffern 0 bis 9
8	Alle Nicht-ASCII-Zeichen, z.B. auch Umlaute und sonstige Unicode-Zeichen
p	Das Zeichen + weglassen
M	Das Zeichen - ergänzen
e	Das Zeichen = weglassen
E	Das Zeichen = ergänzen
N	Das Zeichen # ergänzen

Die Einzelzeichen nach dem Plus oder Minus können nicht eigens geschützt werden. Plus, Minus und Gleichheitszeichen sowie die Raute (#) müssen vielmehr mit ihrem Kennbuchstaben als Menge angegeben werden. Zum Beispiel schützt **+#X=CPaA08=** alle Zeichen. Es kommen nur noch 18 Zeichen in der Ausgabe vor, nämlich: **#XABCDEFG0123456789**.

+#X=P=	schützt alle ASCII-Interpunktionszeichen
+#X=P+a-- !=	schützt alle ASCII-Interpunktionszeichen und das A, aber nicht das Minus und das Ausrufezeichen Lassen Sie die Angabe ganz weg, wirkt sie wie +#X=S=

Die Wirkung von **#X`** lässt sich wie die Ersetzung durch Aufzählung definieren. Sollen genau die beiden Zeichen Punkt und Komma ersetzt werden, so geschieht das durch **`#X=+,.=** oder durch **+LX/,/#23X2CX/ /./#23X2EX/**



Beim Interpretieren dieser Substitutionsvorschrift wird einmal die Ersetzung **-#X** angewandt. Aus **#23X2CX** wird also zunächst wie gewünscht **#2CX**. Effektiv wird also das Komma durch **#2CX**, der Punkt durch **#2EX** ersetzt.

Die Wirkung von **-#X** lässt sich analog durch die obige Liste durch eine **-LX**-Ersetzung ausdrücken. Allerdings müssten hierbei rund 150.000 Ersetzungsregeln, eine für jedes Unicode-Zeichen, angegeben werden.

FO-Schnittstelle für Substitutionen

Für Formeln in FOPs dient der Stern als Substitutionsoperator.

<code>.zuweisen U xtest = "das:ist's" * "#X='='"</code>	'U xtest' erhält den Wert "das:ist#27Xs"
<code>.zuweisen U xtest = "häßlich" * "+LX/ß/ss/ /ä/ae/"</code>	'U xtest' erhält den Wert "haesslich".

Für die Ausgabe kann man Substitutionen auf drei Arten einstellen:

- für alle Variablenzugriffe mit **.setze substitution**
- für Variablenzugriffe mit Formatierung **.setze formatiert**
- für die komplette Ausgabezeile mit **.setze ausgabe**

Weitere Verwendungsstellen für die Abas Standardsubstitutionen

Die Abas Standardsubstitution **-#X** wird an folgenden Stellen auf Variablenwerte angewandt:

- bei Selektionsvorbelegungen
- bei Vorbelegungsangaben in Kommandos
- bei der Definition von Mustern und Werten bei der **+LX**-Substitution.
- beim Text einer **.text**-Zeile im FOP
- für das Kommandologbuch, hier allerdings mit **|X** als klammerndem Zeichen.

Für die Suche nach dem Inhalt eines mehrzeiligen Textfelds in einer Selektion gilt die folgende Vorgehensweise:

```
.type text xtsselstr xsub
.type T34 xt34text
.formula U|xt34text = "das ist;ein mehrzeiliger;Text"
.assign U|xsub = 'U|xt34text' * "+#X"
.assign U|xtsselstr = "%,name=" + ( 'U|xt34text' * "+#X" )
'U|xsub'
```

Durch die Angabe der Abas-Standardsubstitution werden die Semikolons im Wert von

'U|xt34text' nun durch #BX ersetzt und in der Selektion dann wieder zurückübersetzt.

Für die Vorbelegungsangaben in Kommandos geht man wie folgt vor:

```
.type text xtcommand xname
.formula U|xtvorgang = "XDSIK-1"
.assign U|xtcommand = "<(Infosystem)> EVV <(Call)> ? [englvar]=1 |vorgang=" + ( 'U|xtvorgang'
* "+#X" )
U|xtcommand: 'U|xtcommand'
```

Auf diese Weise können beliebige Variableninhalte an ein Infosystem weitergereicht werden. Auch die Sonderzeichen Fragezeichen, Pipe (|) und Gleichheitszeichen werden nicht mehr zum Problem.

Bei Variablen der Art BU8 kann ebenfalls ein Wert als Vorbelegungsangabe mitgegeben werden. Er definiert die Vorbelegungsangaben für die Felder der Submaske, die über diesen Button geöffnet wird.

Der Button 'budruck' öffnet den Druckdialog.

Sie wollen nun ein Infosystem öffnen, initialisieren und dann ausdrucken. D.h. Beim Klick auf den Button 'budruck' sollen die Felder 'zudrucken' und 'drucker' vorbelegt und dann sofort gedruckt werden. Sie schreiben den Wert für 'budruck' in die Variable U|xtprintcommand:

```
.formula U|xps88layout = "XML.MASKEN.P"
.formula U|xps91printer = "BILDSCHIRM"
.assign U|xtprintcommand = "[englvar]=1 |toBePrinted=" + 'U|xps88layout^id' + "|printer=" +
('U|xps91printer^id' * "+#X")
.assign U|xtprintcommand = U|xtprintcommand+ "|[invisible]=1"
```

Sie haben hier exemplarisch den Wert von 'U|xps91printer^id' mit der Abas Standardsubstitution geschützt. Das ist bei einem Verweis zwar überflüssig, aber unschädlich. Nun öffnen Sie das Infosystem, füllen es, klicken auf den Startbutton und drucken dann:

```
.formula U|xps2product = "V2"
.assign U|xtinfocommand = "<(Infosystem)> BESTAND <(Call)> ? [englvar]=1 |artikel=" +
'U|xps2product^id' + "|start="
.assign U|xtinfocommand = U|xtinfocommand + "[englvar]=1|toBeprinted=" + (U|xtprintcommand *
"+#X")
```

Sie müssen also den gesamten Wert für 'budruck' durch die Abas Standardsubstitution schützen. Die doppelte Substitution für das Feld 'drucker' macht hier keine Problem, da der Schutz auch zweimal wieder weggenommen wird: Einmal beim Klick auf den Button 'budruck' in der Vatermaske, ein zweites Mal beim Einfüllen in 'drucker' in der Submaske.

Die Abas Standardsubstitution kann auch zum Schutz bei der Ein- und Ausgabe von Dateien verwendet werden. Um eine Datei spaltentreu mit dem Trennzeichen Pipe (|) zu erzeugen, gehen Sie wie folgt vor:

- 'x1' | 'x2' | 'x3'
- Eingelassen wird die Datei wieder mit: `.hole von` Datei x1 | x2 | x3 ^
- Damit keine Pipes (|) in 'x1', 'x2' oder 'x3' die Spalten zerstören, können diese vor der Ausgabe mit `.zuweisen U|x1 = x1 * "+#X"` geschützt und damit entfernt werden.
- Nach dem Wiedereinlesen werden sie wieder sichtbar gemacht mit `.zuweisen U|x1 = x2 * "-#X"`
- Ebenso wirksam wäre bei der Ausgabe `.setze substitution "+#X"`

Für die externe Bearbeitung mit z.B. `awk` oder `perl` enthält die Datei immer spaltentreu genau drei Pipes (|) pro Zeile. Die Abas Standardersetzung ersetzt alle Pipes (|) durch `#7CX`.

Substitutionen global einstellen

Substitutionen bei der Ausgabe können global eingestellt werden. Dies geschieht mit den folgenden `.setze`-Kommandos:

- `.setze substitution` - Substitutionen für Variablen
- `.setze formatiert` - Substitutionen für Variablen mit Formatierung
- `.setze ausgabe` - Substitutionen in der Ausgabezeile

.SETZE SUBSTITUTION / .SET SUBSTITUTION - Substitutionen für Variablen

.SETZE SUBSTITUTION	.SET SUBSTITUTION
deutsch	englisch
<code>.SETZE SUBSTITUTION "Ersetzung"</code>	<code>.SET SUBSTITUTION "replacement"</code>
erlaubt, Substitutionen bei Variablenzugriffen auf Ausgabezeilen global einzustellen.	

Die Ersetzung wirkt genau auf die Variablenzugriffe in Ausgabezeilen im FO-Programm

Das Kommando `.setze substitution ""` hebt die Substitutionsregel wieder auf.

Beispiel:

In diesem Beispiel hat die Variable 'xttext' den Wert `0 < 1 & 1 < 2`.

Das Kommando `.SETZE SUBSTITUTION "+LX/</</ /&/&/"` bewirkt, dass **alle in Variablen enthaltenen Zeichen** `<` und `&` durch die in HTML und XML üblichen Zeichenketten `<` und `&` ersetzt werden.

```
.type text xttext
.formula U|xttext = "0 < 1 & 1 < 2"
before: 'U|xttext'
.set substitution "+LX/</&lt;/ /&/&";/"
after: 'U|xttext'
```

```
before: 0 < 1 & 1 < 2
after: 0 &lt; 1 &amp; 1 &lt; 2
```

.SETZE FORMATIERT / .SET FORMATTED - Substitutionen für Variablen mit Formatierung

.SETZE FORMATIERT	.SET FORMATTED
deutsch	englisch
<code>.SETZE FORMATIERT "Ersetzung"</code>	<code>.SET FORMATTED "replacement"</code>
erlaubt, Substitutionen bei Variablenzugriffen auf formatierte Variablen für die Ausgabe global einzustellen	

Das Kommando definiert für alle Variablen der Arten "BT" oder "DVMR2" eine abweichende Ersetzung von der bei `.setze substitution` angegebenen Ersetzung.

Das folgende Beispiel hebt diese Regel wieder auf

```
.set formatted ""
```

Auch für Variablen dieser Arten gilt dann wieder die Standardregel von `.setze substitution`.

.SETZE AUSGABE / .SET OUTPUT - Substitutionen in der Ausgabezeile

.SETZE AUSGABE	.SET OUTPUT
deutsch	englisch
<code>.SETZE AUSGABE "Ersetzung"</code>	<code>.SET OUTPUT "replacement"</code>
erlaubt, Substitutionen für die Ausgabezeile global einzustellen	

Das Kommando wirkt auf die komplette Ausgabezeile nach der Variablenersetzung und nach einer Ersetzung durch `.setze substitution`.

Beispiel:

In diesem Beispiel hat die Variable 'xttext' den Wert 0 < 1 & 1 < 2.

Das Kommando `.setze ausgabe "+LX/</</ /&/&"/` bewirkt, **dass alle Vorkommen der Zeichen < und &** durch die in HTML und XML üblichen Zeichenketten `<` und `&` ersetzt werden.

```
.type text xttext
.formula U|xttext = "0 < 1 & 1 < 2"
before: 'U|xttext'
.set output "+LX/</&lt;/ /&/&"/
after: 'U|xttext'
```

```
before: 0 < 1 & 1 < 2
after: 0 &lt; 1 & 1 &lt; 2
```

Variablen und Operationen

Alle FO-Variablen sind typisiert. Statt "Typ" wird im Folgenden der oft gleichbedeutende Begriff "Art" verwendet.

```
.type text xttext1
.type text xtfile
```

Explizite Typdeklarationen ermöglichen Syntaxprüfungen. Tragen Sie in der Interpreterzeile die Option zur Deklaration sämtlicher Variablen für ihr FO-Programm ein (siehe unter [...!INTERPRETER-Befehl](#)).

Ist die Art für eine Variable explizit gesetzt, wird durch erneutes Setzen mit `.art` nicht die Art verändert, sondern nur die Variable geleert.



```
.type GD xdate
.formula U|xdate = "10.11.11"
date: 'U|xdate'
..
.type GD xdate
date: 'U|xdate'
```

Zur Überprüfung, ob Variablen definiert sind, stehen Ihnen verschiedene [Möglichkeiten](#) zur Verfügung.

.ART / .TYPE - Variablenart angeben

.ART	.TYPE
deutsch	englisch
<code>.ART 'Art' 'Variablenname'</code> <code>'Variablenname' ...</code>	<code>.TYPE 'specify type' 'variable name'</code> <code>'variable name' ...</code>
deklariert die aufgeführten Variablen mit der angegebenen Art. Zugriffe auf diese Variablen erfolgen mit Pufferangabe U .	

Die angegebene Art gilt für alle aufgeführten [Variablen](#). Sie werden nach "einfachen Variablenarten" und "Variablentabellenarten" unterschieden:

Einfache Variablenarten

INTEGER	für ganze Zahlen
REAL	für reelle Zahlen mit zwei Nachkommastellen
REAL0	für reelle Zahlen mit maximal vier Nachkommastellen Sind die letzten Nachkommastellen 0, werden sie durch Leerzeichen ersetzt. Die Anzahl der Vorkommastellen bleibt aus Gründen der Tabellentreue gleich. 123,4567 123,456 123,45 123,4 123 123,0007
REAL2	für reelle Zahlen (Kommazahlen) mit zwei optionalen Nachkommastellen; siehe hierzu Tabelle zu Zahlentypen
REAL4	für reelle Zahlen mit vier Nachkommastellen
REALX	für reelle Zahlen mit höchstmöglicher Rechengenauigkeit. Die Ausgabe umfasst jedoch nur maximal vier Nachkommastellen.  Beim Rechnen mit Variablen der Art REALX wird nicht gerundet. Daher ist es überflüssig, solche Variablen mit konstanten Werten (z.B. $x = 0$) zu vergleichen.
BOOL	für Wahrheitswerte (<code>G true</code> , <code>G false</code>)

TEXT	für beliebige Texte  Datenbankvariablen der Art TEXT, die über die Variablentabelle geändert wurden, müssen die Textart BTEXT (für "Blank-Text") erhalten, wenn sie führende Leerstellen beibehalten sollen (siehe Textarten).
DATUM	für tagesgenaue Termine (z.B. 12.05.10)
WOCHE	für wochengenaue Termine (z.B. 36/10)
TERMIN	für tages- bzw. wochengenaue Termine, je nach der benannten Art für das Datum
ZEIT	für Uhrzeitangaben (z.B. 12:32)

Deklarieren einer Variablen 'xivar1' vom Typ **integer**, die 3-stellige, positive Zahlen speichern kann

```
.type IP3 xivar1
```

Deklarieren einer Variablen 'xrvar1' vom Typ **real** mit sieben Vorkommastellen und sieben Nachkommastellen

```
.type R7.7 xrvar1
```

Zahlentypen

Im folgenden eine Übersicht über die zur Verfügung stehenden Zahlentypen in FOPs.

Art	Vorkommastellen im Ausgabeformat	Nachkommastellen im Ausgabeformat	interne Genauigkeit
INTEGER	9	-	-
REAL	12	2	zwei Stellen
REAL0	11	0-4*	vier Stellen
REAL2	12	0-2*	zwei Stellen
REAL4	11	4	vier Stellen
REALX	12	4	rechnergenau**

Bemerkungen: **Abschließende Nullen werden nicht dargestellt. Daher schwankt die Anzahl der Nachkommastellen zwischen den angegebenen Werten.**

* Rechnergenau heißt, dass die Zahlendarstellung von den Möglichkeiten des jeweiligen Rechners abhängt. Es gilt die Faustformel: Hat eine Zahl x Vorkommastellen, so sind 14 - x Nachkommastellen genau.

Bei der Variablenart REAL2 werden Nachkommastellen nur dann dargestellt, wenn sie von 0 verschieden sind. Somit gilt z.B. für $x = 13$:



x liefert "13.00" nach **.art REAL x**

x liefert "13 " nach **.art REAL2 x**

'x' liefert "13.0000" nach **.art REAL4 x**

Neben den einfachen Arten können alle Arten angegeben werden, die in der Variablentabelle für **benutzerdefinierte** Felder zugelassen sind (siehe Übersichten der [Variablenarten](#)).

Siehe auch Variablentabelle: Artentabelle im [Feld "Neue Art"](#).

Spezielle Anwendungen von Variablenarten

Texte für Variablenart BOOL

Die natürlichsprachlichen Texte "ja" und "nein" bzw. ihre Übersetzungen sollten vermieden werden und dürfen nur in den Programmen der jeweiligen Sprachverzeichnisse vorkommen. Zur sprachunabhängigen Programmierung dienen die vordefinierten Werte **G|true** und **G|false**.

```
.type bool xbstatus  
.formula U|xbstatus = G|true
```

Platzbedarf bei Texten

Beim Rechnen mit Texten (siehe unter [Variablen und Operationen](#)) entsteht ein variabler, nicht voraussehbarer Platzbedarf im Speicher. Speicherverbrauch und Rechenaufwand steigen beim Verlängern von Texten. Deshalb kann es sinnvoll sein, gleich bei der Artvereinbarung eine größere Anzahl von Zeichen für Textvariablen (d.h. Variablen vom Typ TEXT) zu reservieren, weil dadurch eine mehrfache Verlängerung vermieden wird. Dies geschieht im Kommando **.art** per [Doppelpunkt](#) etwa so:

```
.type text xta:200 xtb:150
```

Für die Textvariablen **'U|xta'** und **'U|xtb'** werden 200 bzw. 150 Zeichen reserviert.

Datumsrechnung bei DATUM/WOCHE/TERMIN

Die Datumsrechnung erfolgt unabhängig vom gewählten Datumsformat tagesgenau.

Datumsdarstellung in FOPs

Die Datumsarten sind - wie auch DATUM - der FO bekannt. Es besteht die Möglichkeit, ein Datum zu konvertieren. Dabei muss der [Wertebereich](#) berücksichtigt werden.

```
.type date xddate
.type gd xgddate
.formula U|xddate = "1.1.2010"
'U|xddate' --> 01.01.10
.formula U|xgddate = U|xddate
'U|xgddate' --> 01.01.2010
```

Wird ein Datum konvertiert, das nicht im Wertebereich der Zielart liegt, erscheint die Fehlermeldung "unzulässiges Datum".

Für das Datum gelten alle Rechenregeln. Zusätzlich kann mit Positiv-Unendlich und Negativ-Unendlich gerechnet werden. Dabei verändert sich der Wert jedoch nicht.

```
.type gd xgddate
.formula U|xgddate = "++"
'U|xgddate' --> ++.++.++++
.formula U|xgddate = xg + 1
'U|xgddate' --> ++.++.++++
```

Bei den Variablen der Art GD2 und DATUM ändert sich das Verhalten der FO.

Während einer Rechnung darf der Wertebereich kurzzeitig überschritten werden (31.12.2080). Der Wert darf jedoch nicht als Zwischenergebnis ausgegeben werden.

```
.type date xddate
.. last valid date
.formula U|xddate = "31.12.2080"
.formula U|xddate = (U|xddate + 10) - 10
'U|xddate' --> 31.12.2080
```

Das Gleiche gilt auch an der unteren Grenze des Datums (1.1.1981). Für Variablen der Arten GD, GD8, GW4 gilt das Assoziativgesetz an den Grenzen des Wertebereichs nicht.

```
.type gd xgddate
.formula U|xgddate = "1.1.1981"
.formula U|xgddate = (U|xgddate - 10) + 10
'U|xgddate' --> 01.01.1981
```

Die direkte Zuweisung einer Variablen vom Typ "GD" zu einer Textvariablen unterliegt der Forderung nach Kompatibilität mit einem Datum in Abas. Die Textvariable enthält nach Zuweisung eine Datumsangabe mit

zweistelliger Jahreszahl und liegt im Wertebereich des Datums. Statt der direkten Zuweisung ist die Zuweisung mit einer Konvertierung in TEXT zu empfehlen.

```
.type gd xgddate
.type text xttext
.formula U|xgddate = "8.3.2010"
.formula U|xttext = U|xgddate
'U|xttext' --> 08.03.10
..
.. much better
.formula U|xttext = 'U|xgddate'
'U|xttext' --> 08.03.2010
```

Zur Datumsdarstellung in FO-Programmen werden die [Datumsformatfelder](#) der Bediensprachen herangezogen, sofern die Datumsdarstellung nicht von den [Datumsformatfeldern](#) der Korrespondenzsprachen abhängig sein soll. Dazu müssen Sie an das auszugebende Datum eine Umwandlungsangabe (Cast-Operator) anhängen. Diese hat folgendes Format:

'datum(L="E") '	für die Korrespondenzsprache E
'datum(L="F") '	für die Korrespondenzsprache F
'datum(L=U xkspr) '	für die Korrespondenzsprache, die in 'U xkspr' hinterlegt ist. Dabei ist 'U xkspr' eine beliebige FO-Textvariable, in der das Kürzel der Korrespondenzsprache steht.

Es dürfen nur definierte Korrespondenzsprachen verwendet werden.

'datum(F="Y/D/M") '	Formatangabe wird direkt angegeben, Datumsformatfelder bleiben unberücksichtigt
'datum(F=U xdatfor) '	Formatangabe steht in der FO-Variablen 'U xdatfor'

Die Notation ist nur gültig, wenn die Variable in Apostrophe eingeschlossen ist.

Richtig:

```
.type text xttext
.formula U|xttext = 'G|date(F="Y@M@D")'
'U|xttext'
```

Falsch:

```
.type text xttext
.formula U|xttext = G|date(F="Y@M@D")
```

'U|xtext'

Der Befehl **.lesen** berücksichtigt die Datumsformatfelder der Bediensprachen. Es muss also ein Datum eingegeben werden wie in Masken.

Datumsvariablen in unterschiedlichen Kalendern ausgeben

Es ist möglich, per FO Datumsvariablen in unterschiedlichen Kalendern auszugeben.

Für den Hijri (islamischen) Kalender geben Sie statt **Y** im Format-String ein **H** an.

Für den persischen Sonnenkalender geben Sie statt **Y** im Format-String ein **P** an.

```
.type date xdate
.formula U|xdate = "20100320"
G|date(F="D.M.Y"): 'U|xdate(F="D.M.Y")' | 'U|xdate(F="D.M.Y"):4' | 'U|xdate(F="D.M.Y"):8'
G|date(F="D/M/H"): 'U|xdate(F="D/M/H")' | 'U|xdate(F="D/M/H"):4' | 'U|xdate(F="D/M/H"):8'
G|date(F="P/M/D"): 'U|xdate(F="P/M/D")' | 'U|xdate(F="P/M/D"):4' | 'U|xdate(F="P/M/D"):8'
```



1. Bei sortierfähiger Ausgabe des Datums wird immer das gregorianische Datum ausgegeben.
2. Da weder im Hijri noch im persischen Kalender Kalenderwochen verwendet werden, werden Kalenderwochen immer nach abendländischem Kalender ausgegeben.

Die unterschiedlichen Kalender funktionieren nur bei der Ausgabe per FO.

Variablen der Art "Zähler"

Variablen der Art "Zähler" sind unabhängig von der Unterart (**ZAE** und **ZAE1**) kompatibel:

- Den Variablen können Wahrheitswerte oder Zählerwerte zugewiesen werden.
- Die Variablen können einer anderen Zählervariablen zugewiesen werden.
- Die Variablen können verglichen (`<code>=</code>`, `<code><></code>`, `<code><</code>`, `<code>></code>`, `<code>=</code>`, `<code>>=</code>`) werden, wobei immer die Zählerwerte verglichen werden.

Lesen Sie hierzu auch [Versionszähler und inkrementelle Datenübergabe](#).

Variablen der Art "Zähler" und der Art BOOL sind in FOPs kompatibel. Zählervariablen werden als **G|false** behandelt, wenn sie leer sind, sonst als **G|true**:

- Zählerfelder und Checkboxes können wechselseitig zugewiesen werden.
- Die booleschen Operatoren NICHT, UND und ODER können auch bei Zählervariablen (**.formel xzaehler = xbool**) benutzt werden.
- Ist bei einer Operation ein Operand BOOL und der andere "Zähler", so wird der "Zähler" in BOOL

umgewandelt und dann der Operator angewandt.

- Zählervariablen können als Bedingungen benutzt werden.

.FORMEL / .FORMULA - Variablen Werte zuweisen

.FORMEL	.FORMULA
deutsch	englisch
<code>.FORMEL 'Variablenname' = 'Ausdruck'</code>	<code>.FORMULA 'variable name' = 'expression'</code>
Der ' Variablen ' wird ein neuer Inhalt zugewiesen, und zwar das Rechenergebnis des ' Ausdrucks '.	

Vergleich der Zuweisungsoperationen .formel, .zuweisen, .kopieren

.formel ist eine der drei vorhandenen Möglichkeiten der Wertzuweisungen in der FO. Die folgende Übersicht gibt einen Überblick über ihre jeweiligen Einsatzgebiete.

	.formel	.zuweisen	.kopieren
Funktion	zuweisen über Umwandlung in einen Basistyp, ohne Wertebereichsprüfung	zuweisen mit Wertebereichsprüfung (außer Stringlänge)	kopieren eines Feldes ohne Wertebereichsprüfung
G mehr / G success wird gesetzt	nein	ja, zeigt Erfolg an	nein
Prüfung auf Artverträglichkeit	großzügig	großzügig	streng
Für Verweise und Aufzählungen geeignet	nein	ja	ja
Für REAL-Zahlen geeignet	bedingt, rundet bei Umwandlung nach TEXT auf 2 Nachkommastellen	ja	ja
Für Buttons und Freitexte zulässig	ja	nein	nein

Bei **.formel** handelt es sich (genau wie bei den anderen beiden Zuweisungen) nicht um eine Gleichung im mathematischen Sinne, sondern um eine Operation. Mithilfe des Gleichheitszeichens wird ein Ausdruck bzw. das Ergebnis eines Ausdrucks einer Variablen zugewiesen.

Der Ausdruck kann Variablen, Zahlen, Texte, Klammern, Operationen oder weitere Kommandos (s.u.) enthalten. Texte müssen durch Anführungszeichen gekennzeichnet sein.

Wird im EFOP einer Variablen des Datensatzes im Bezugsbereich **M** ein Wert zugewiesen, so werden auch die **Anwendungsmethoden** dieses Datensatzes aufgerufen, so dass die Anwendungslogik gewahrt bleibt. Auf diese Weise werden die Editierbarkeit des Feldes und die Zulässigkeit der Wertzuweisung geprüft. Falls diese Prüfungen scheitern, bricht das Programm ab. Außer beim **M**-Datensatz gilt diese Logik auch für Datensätze, die mit **.mache** neu angelegt wurden.

In allen anderen Fällen entscheidet der Eintrag in der Variablentabelle über die Editierbarkeit von Feldern. Dort werden standardmäßig nur solche Felder als editierbar gekennzeichnet, für die kein Aufruf einer Anwendungsmethode nötig ist. Das ist dann der Fall, wenn keine Folgeaktionen, kein dynamischer Schreibschutz, keine nichtsyntaktischen Prüfungen o.ä. gebraucht werden.



Produkte aus Ganzzahlen liefern ein Ergebnis vom Typ real. Das führt, beim Addieren oder Subtrahieren dieses Ergebnisses zu einem Datum, zu einer Fehlermeldung. Weisen Sie in diesem Fall Ihr Ergebnis einer Variable vom Typ integer zu und verwenden Sie diese.

Individuelle Aufzählungen

Um mit einer Variablen vom Typ **benutzerdefinierte Aufzählung** für **Individuelle Aufzählungen** auf Variablen des betreffenden Datenbankbereiches z.B. Artikel zuzugreifen, verfahren Sie wie folgt:

Hinter der Variablen für eine individuelle Aufzählung verbirgt sich der Verweis auf das entsprechende Objekt.

Beispiel: Individuelle Aufzählung

Sie haben eine individuelle Aufzählung (1000001) auf den Teilestamm (P2:1) angelegt, die z.B. die Elemente E1, E2 und E3 enthält.

Auf der FO definieren Sie zunächst eine Variable vom Typ der Aufzählung:

`.art azk1000001 xekt` Nun weisen Sie der Variablen einen Wert zu:

`.formel U|xekt = "E2"` Jetzt können Sie z.B. den Einkaufspreis einfach erfragen, bzw. ausgeben über

`'U|xekt^epr'`

.KOPIEREN / .COPY - Variablenwerte kopieren

.KOPIEREN	.COPY
deutsch	englisch
<code>.KOPIEREN 'X Variablenname' = 'Ausdruck'</code>	<code>.COPY 'X variable name' = 'expression'</code>
Der ' Variablen ' wird das Rechenergebnis des ' Ausdrucks ' zugewiesen. X steht für ein Bezugsobjekt.	

Das Kommando ist vor allem beim Kopieren von Feldern aus einem Datenbankpuffer in einen anderen nützlich, insbesondere bei Infosystemen. Auch Verweise können kopiert werden.

Das Kommando `.formel` ist eine Modifikation des Befehls `.formel`, jedoch muss zur Variablen immer das Bezugsobjekt (**H** | **D** | ...) angegeben werden und der Variablenname darf nicht abgekürzt werden.

Bedingung ist, dass die Art der **Variablen** und die Art des **Ausdrucks** kopierverträglich sind. Das heißt fast immer, dass sie übereinstimmen müssen oder dass zumindest ein Wert der Art der rechten Seite in *jedem* Fall dem Wertebereich der Art der linken Seite angehört. Es sind lediglich Artvariationen wie bei Feldern mit **Variablentabellen-Schutzkennzeichen** Änderbares Standardfeld zulässig. So darf z.B.

'**Variablenname**' zur Art "I4", '**Ausdruck**' zur Art "I3" gehören. Sind die Arten nicht verträglich, bricht das FO-Programm ab. Andernfalls wird der Wert unbedingt und ohne Prüfung (abgesehen von Prüfungen durch die **Anwendungsmethoden**, z.B. für einen Datensatz im M-Puffer) kopiert.



Die Arten "BU" und "TN" lassen sich mit **.kopieren** grundsätzlich nicht bearbeiten.

Siehe auch **.zuweisen**.

M|yart sei ein Feld der Art "P2:1.2.4" im Infosystem. Im H-Puffer liege eine Auftragsposition mit dem Feld **art** der Art "PS2:1.2.4".

```
.copy M|yart=H|art
..
.type int xint
.type I3 xi3
.type I4 xi4
right:
.copy U|xi4 = U|xi3
.copy U|xint = U|xi3
wrong:
.copy U|xi3 = U|xi4
.copy U|xi3 = U|xint
.copy U|xi4 = U|xint
```

Konstante Texte und .kopieren

Für das Kommando **.kopieren** gilt für die Verträglichkeit der Textarten folgende Regel: Wenn die Art der Variablen auf der linken Seite eine der vier folgenden Eigenschaften hat, muss auch die Art des Wertes auf der rechten Seite diese Eigenschaft haben.

Diese Eigenschaften sind:

- Nicht-Unicode-fähige Textart: GLU, GLA, TU, NT
- Normtext: NT
- Einzeilige Textart: NT, GL, BGL, GLU, GLA, NT
- Getrimmte Textart: alle außer BGL, BT

Dazu muss außerdem die Länge passen, d.h. die Länge der linken Seite muss größer oder gleich der Länge der rechten Seite sein. Folgende Zuweisung ist daher nicht erlaubt:

```
.type text xtext
```

```
.type t10 xt10text
.copy U|xt10text = U|xttext
```

Artumwandlungen anlegen

Kompatibilität der Arten GRN, GRB

- GRN ist, unabhängig von der Unterart, immer kompatibel
- GRB ist nur bei gleicher Unterart kompatibel

GRN ist immer kompatibel zu "int" bzw. zu den "I"-Arten ab "IP3". Hier wird deshalb nicht zwischen Gruppe 1 im Kunden oder Lieferanten unterschieden.

Bei GRB, also der Gruppenbezeichnung, ist das anders: Gruppe 1 in der Datenbank Kunden heißt **kunde** und in der Datenbank Lieferanten **lieferant**.

Sichtbar wird das beim **.kopieren** von **gruppe** und beim Verwenden von **grbez** in Verknüpfungen hinter einem Verweisfeld einer "V"-Verweisart:

Beispiel: Kopieren an eine I3-Variable

```
.type I3 xi
.copy U|xi = H|gruppe
```

Beispiel: Verknüpfung

Im H-Puffer liegt eine Artikelzeile. Dort gibt es das Feld **elem**, das ist ein VP2.

```
.copy U|xi = H|elem^dnr
```

.ZUWEISEN / .ASSIGN - Zuweisen mit Wertebereichsprüfung

.ZUWEISEN	.ASSIGN
deutsch	englisch
.ZUWEISEN 'X Variablenname' = 'Ausdruck'	.ASSIGN 'X variable name' = 'expression'
Der ' Variablen ' wird das Rechenergebnis des ' Ausdrucks ' zugewiesen, wenn es im Wertebereich der Variablen liegt. X steht für ein Bezugsobjekt.	

Das Kommando **.formel** entspricht (außer bei den Arten "BU" und "TN") dem Kommando **.formel** mit abgeschaltetem Fehlerabbruch. Jedoch muss zur Variablen immer das Bezugsobjekt (**H|**, **D|**, ...) angegeben

werden und der Variablenname darf nicht abgekürzt werden.

Die Art der Variablen auf der linken Seite und die Art des Formelausdrucks auf der rechten Seite müssen zum selben Basistyp gehören, es sei denn, links oder rechts steht ein Text. Der im 'Ausdruck' enthaltene Wert wird nur dann zugewiesen, wenn er im Wertebereich der Variablen liegt, an die zugewiesen werden soll. Liegt er außerhalb, bricht das Programm nicht ab, sondern leert die Variable und setzt G|mehr auf G|false. Wenn die Zuweisung erfolgreich war, wird G|mehr auf G|true gesetzt.

Auch Verweise lassen sich problemlos zuweisen.

Siehe auch .kopieren.

Basistypen

Basistypen sind diejenigen Typen (Arten), über die Ausdrücke berechnet werden, also Integer, Real, Text, Datum, Zeit, Bool, Woche, Termin. Dazu kommen die Typen Verweis, Zähler und Geschäftsjahr, für die es aber keine Rechenoperationen außer der Zuweisung gibt. Die Arten BU und TN lassen sich mit .zuweisen grundsätzlich nicht bearbeiten.

In diesem Beispiel wird der Variablen U|xi4 der Wert 999 zugewiesen.

Um das Kommando .assign erfolgreich auszuführen, darf der Wert in U|xi4 zwischen -99 und 999 liegen. Wenn das Kommando .assign erfolgreich ausgeführt wurde, dann befindet sich in der Variablen U|xi3 der zugewiesene Wert und in der Variablen G|success steht G|true.

Wenn das Kommando .assign nicht erfolgreich ausgeführt wurde, dann befindet sich in der Variablen U|xi3 der initiale Wert 0 und in der Variablen G|success steht G|false

```
.type I3 xi3
.type I4 xi4
.formula U|xi4 = 999
.assign U|xi3 = U|xi4
'U|xi3' - 'G|success'
```

In diesem Beispiel wird der Variablen U|xi3 ein Ausdruck zugewiesen, der außerhalb des Wertebereiches liegt. Das Zuweisen U|xi3 = 1000 funktioniert nicht. Der Inhalt von G|success ist G|false und der Inhalt von U|xi3 ist 0.

Das Zuweisen U|xi4 = U|xi3 funktioniert. Der Inhalt von G|success ist G|true und der Inhalt von U|xi4 ist 0.

```
.type I3 xi3
.type I4 xi4
.assign U|xi3 = 1000
'U|xi3' - 'G|success'
.assign U|xi4 = U|xi3
```

```
'U|x14' - 'G|success'
```

Substitutionen in Formelausdrücken

In Formeln dient der Stern als Substitutionsoperator.

In diesem Beispiel wird das **Apostroph** durch dessen **HEX-Code** ersetzt.

```
.type text xtest
.assign U|xtest = "St Peter's Square" * "+#X=+'="
St Peter#27Xs Square
'U|xtest'
```

In diesem Beispiel wird im Originalstring **häßlich** das **ß** durch **ss** und **ä** durch **ae** ersetzt.

```
.type text xtest
.assign U|xtest = "häßlich" * "+LX/ß/ss/ /ä/ae/"
'U|xtest'
haesslich
```

Siehe auch unter [Substitutionen - Schützen von Zeichen vor Interpretation](#).

.SETZE NACHFRAGE / .SET DEMAND - Variable im Dialog einlesen

.SETZE NACHFRAGE	.SET DEMAND
deutsch	englisch
.SETZE NACHFRAGE + .SETZE NACHFRAGE -	.SET DEMAND + .SET DEMAND -
.setze nachfrage + liest eine im FOP verwendete Variable, die noch nicht belegt ist, im Dialog ein. .setze nachfrage - beendet das Einlesen. Wenn das FOP es auf eine noch nicht belegte Variable trifft, bricht es mit einer Fehlermeldung ab.	

Operationen

In Zuweisungen (**.zuweisen**, **.zuweisen**, **.zuweisen**) und in Bedingungen stehen die folgenden Operationen zur Verfügung:

Stringoperationen

Beachten Sie bei den folgenden Beschreibungen von Stringoperationen, dass es **Sonderbehandlungen für doppelt breite chinesischen Zeichen** gibt, die zusammengefasst weiter unten beschrieben werden.

Vergleichsoperationen

Für Zeichenketten gelten die allgemeinen Vergleichsoperationen. Werden Texte verglichen, so gilt der alphabetisch spätere Text als "größer". Bei Datums- und Zeitangaben gilt das zeitlich spätere Datum als größer.

<	kleiner
>	größer
<=	kleiner gleich
>=	größer gleich
=	gleich
<>	ungleich

Abschneiden und Zusammenfügen

Operator	Syntax	Funktion	Beschreibung	Beispiel
-	-t	Normieren	wandelt den Text t in Großbuchstaben, Umlaute in ae, oe, ue und ß in ss um	- "Maßbänder" = MASSBAENDER
""	t	Länge	liefert die Länge des Textes t als ganze Zahl	+ "ABC" = 3
+	t1 + t2	Zusammenfügen	liefert die Zusammensetzung t1 gefolgt von t2	"A" + "B" = "AB"
<\$	t1 <\$ t2	Abschneiden	liefert die Zeichen aus t1 links vom letzten Vorkommen von t2 zurück ist t2 nicht enthalten, so wird t1 <\$ t2 = t1	"ABCDECFG"<\$"C" = "ABCDE"
>\$	t1 >\$ t2	Abschneiden	liefert die Zeichen aus t1 rechts vom letzten Vorkommen von t2 zurück ist t2 nicht enthalten, so wird t1 >\$ t2 = ""	"ABCDECFG">\$"C" = "FG"

Operator	Syntax	Funktion	Beschreibung	Beispiel
<<	t1 << t2	Abschneiden	liefert die Zeichen aus t1 links vom ersten Vorkommen von t2 zurück ist t2 nicht enthalten, so wird t1 << t2 = t1	"ABCDECFG"<<"C" = "AB"
>>	t1 >> t2	Abschneiden	liefert die Zeichen aus t1 rechts vom ersten Vorkommen von t2 zurück ist t2 nicht enthalten, so wird t1 >> t2 = ""	"ABCDECFG">>"C" = "DECFG"
<<	t << ""	Abschneiden	schneidet Leerzeichen am Ende des Textes t ab	"ABCDECFG" <<" " = "ABCDECFG"
>>	t >> ""	Abschneiden	schneidet führende Leerzeichen vor t ab	" ABCDECFG" >>" " = "ABCDECFG"
<<	t << i	Abschneiden	liefert die linken i Zeichen von t zurück i ist eine positive Zahl	"ABCDE" << 2 = "AB"
>>	t >> i	Abschneiden	Liefert die rechten i Zeichen von t zurück i ist eine positive Zahl	"ABCDE" >> 2 = "DE"
<<	t << i	Abschneiden	schneidet die linken i Zeichen von t ab i ist eine negative Zahl	"ABCDE" << -2 = "CDE"
>>	t >> i	Abschneiden	schneidet die rechten i Zeichen von t ab i ist eine negative Zahl	"ABCDE" >> -2 = "ABC"

Für Formeln im FOP dient der Stern als Substitutionsoperator. Durch

`.zuweisen U|xtest = "das:ist's" * "#X='"` erhält 'xtest' den Wert: "das:ist#27Xs"
Durch:

`.zuweisen U|xtest = "häßlich" * "+LX/ß/ss/ /ä/ae/"` erhält 'xtest' den Wert
"haesslich".

Näheres siehe unter [Substitutionen und Schutz von Sonderzeichen](#).

Sonderbehandlung chinesischer Zeichen

Doppelt breite fernöstliche Zeichensätze in Unicode werden wie folgt unterstützt: An allen Stellen, wo Zeichen gezählt werden, werden diese Zeichen wie zwei Zeichen bewertet. Damit wird bei Verwendung von nichtproportionalen Schriften eine saubere Formatierung erreicht. Auch bei Verwendung von proportionalen Schriften lässt sich so die Breite eines Textes besser schätzen. Aufgrund dieser Änderung halbiert sich die Anzahl der chinesischen Zeichen, die ein Textfeld aufnehmen kann. Diese Halbierung gilt auch für Wörterbuchtexte.

Auch die FO-Operationen für Zeichenketten (+, <<, >>) verändern sich. Bei der Prüfung der Längengrenze wird stets abgerundet. Das heißt, dass in ein Textfeld der Länge 1 kein chinesisches Zeichen und in ein Textfeld der Länge 3 nur ein chinesisches Zeichen passt. Bei gemischten Texten wird addiert. Leerzeichen sind ebenfalls immer nur ein Zeichen breit. In ein Textfeld der Länge 10 passen vier chinesische Zeichen und ein oder zwei Leerzeichen.

Ausgabeoperationen

In FOPs wirkt sich die Änderung auf allen formatierenden Ausgabeoperationen aus. Auch hier wird beim Abschneiden immer abgerundet und dann wie üblich mit Leerzeichen aufgefüllt. Daher ist der Text nach dem Abschneiden mit Sicherheit nicht mehr zu breit.

Rechnen mit Texten

Beim Rechnen mit Texten gibt es Auswirkungen auf folgende Operatoren:

<code>+t</code>	Längenbestimmungsoperator: unäres Plus auf Zeichenketten
<code>t << n</code> <code>t >> n</code>	Abschneideoperator für Texte mit numerischem Argument

Bei der Längenbestimmung mit `+t` wirkt sich die doppelte Zählung aus: Ein chinesisches Schriftzeichen hat die Länge 2. Beim Abschneiden wird bei positivem Argument, also dem Abschneiden der ersten oder letzten n Zeichen immer aufgerundet:

`.formel xerg = xtext << 1`

schneidet also immer ein Zeichen ab.

`.formel xint = + (xtext << 1)`

kann aber in 'xint' den Wert 2 liefern. Es gilt:

`.formel xerg = xtext << xn`

Die Länge des Textes 'xerg' für positive x_n ist x_n oder $x_n + 1$.
Bei negativem Argument (Rest ohne die ersten oder letzten n Zeichen) wird abgerundet:

```
.formel xerg = xtext << -xn
```

x_n selbst ist hier positiv! Die Länge des Textes 'xerg' für negative Argumente (also positive x_n) ist x_n oder $x_n - 1$. Da diese Operation in der Regel in Schleifen mit der Konstanten 1 vorkommt, garantiert dieses Rundungsverhalten den Fortschritt der Schleife. Die jeweils andere Rundung ist durch folgende Ausdrücke zu erreichen. Um aufzurunden, schreiben Sie:

```
.formel xerg1 = xtext << 1
```

Um abzurunden, schreiben Sie:

```
.formel xerg2 = xtext >> - (+xtext - 1)
```

Ist das erste Zeichen ein doppelt breites chinesisches Zeichen, so enthält 'xerg1' dieses Zeichen, 'xerg2' einen Leertext.

Matchoperation für Strings

t_1 / t_2 hat den Wert $G|true$, wenn t_1 auf das Muster t_2 passt. Anders gesagt, t_1 muss sich aus dem Matchcode t_2 erzeugen lassen, wenn man die folgenden Regeln anwendet:

- Das Fragezeichen steht für ein beliebiges Zeichen.
- Ein Stern steht für einen beliebigen Text, auch für den Leertext.
- Ein Bereich (a-e) steht für ein Zeichen von Anfang a bis e. (A-Z) steht für einen Großbuchstaben, (0-9) für eine Ziffer usw. Der Strich zwischen den Grenzen kann fehlen.
- Eine Menge (/.../) steht für eines der Zeichen zwischen den Slashes.
- Eine Negation (Unterstrich) verneint die folgende Bedingung.

Die bedeutungstragenden Zeichen $()/*_?$ werden im Matchtext interpretiert. Wenn auf eines dieser Zeichen selbst geprüft werden soll, muss es seiner Bedeutung enthoben werden. Dazu stellen Sie ein Backslash (\) voran. Dieser gehört somit auch zu den bedeutungstragenden Zeichen.

t / "?"*	t enthält mindestens ein Zeichen
t / "(AZ)*"	t beginnt mit einem Großbuchstaben
t / "*_./,/"	t endet nicht mit Punkt oder Komma
t / "?5*"	t enthält eine Fünf an zweiter Stelle
t / "\(*"	t beginnt mit einer runden öffnenden Klammer
t / "*"	t beginnt mit einem Backslash (\)

Mathematische Operationen

+	Addition
-	Subtraktion
*	Multiplikation
/	Division

Vergleichsoperationen

<	kleiner
>	größer
<=	kleiner gleich
>=	größer gleich
=	gleich
<>	ungleich

Rundung

Ist r eine reelle und i eine ganze Zahl, so bildet:

r:i	r gerundet auf i Stellen hinter dem Komma (siehe Formatierung von FO-Variablen). Grundsätzlich wird in Abas kaufmännisch gerundet : bei Ergebnissen mit 1, 2, 3 oder 4 wird auf die Währungseinheit ab-, bei Ergebnissen mit 5, 6, 7, 8 oder 9 aufgerundet.
-----	---

Beispiel: Kaufmännisch runden

```
.type r5.7 xnumber1          ? F|defined(U|xnumber1) = false
.type real xrresult          ? F|defined(U|xresult) = false
.formula U|xnumber1 = 12.2355678
.formula U|xresult = U|xnumber1:2
..
Number: 'U|xnumber1'
Result: 'U|xresult'
```

Display:

```
Number: 12.2355678
Result: 12.24
```

Wenn r eine reelle Zahl und x ein Text ist, z.B. "EUR", wird kaufmännisch **gerundet**:

Beispiel: Kaufmännisch runden mit Währungscode

Das nachfolgende FOP runden ein reelle Zahl kaufmännisch, wenn nach dem Doppelpunkt ein Text mit einem gültigen Währungscode angegeben wird.
 z.B. USD, CHF, EUR, ... Die Anzahl der Nachkommastellen entspricht der definierten Anzahl der Nachkommastellen der angegebenen Währung.
 Bei Angabe eines falschen Währungscode wird eine Fehlermeldung ausgegeben.

```
.type real xrvalue1      ? F|defined(xrvalue1) = false
.type real xrvalue2      ? F|defined(xrvalue2) = false
.type text xtext         ? F|defined(xtext) = false
..
.formula U|xvalue1 = 4.56666
..
.formula U|xvalue2 = U|xvalue1:"CHF"
U|xvalue2 = 'U|xvalue2'
.. oder
.formula U|xtext="CHF"
.formula U|xvalue2 = U|xvalue1:xtext
U|xvalue2 = 'U|xvalue2'
..
.. invalid currency code -> Error
...formula U|xvalue2 = U|xvalue1:"XYZ"
..U|xvalue2 = 'U|xvalue2'
```

(Siehe auch **.setze ersetzung**).

Restbildung

Mit dem Operator **//** kann der Rest einer Ganzzahldivision ermittelt werden. Die Restbildung wird in der Mathematik auch "modulo" genannt.

```
.type int xirest          ? F|defined(U|xirest) = false
.formula U|xirest = 14//3
14//3 int rest: 'U|xirest'
.formula U|xirest = 14//2
14//2 int rest: 'U|xirest'
```

Display:

```
14//3 int rest: 2
14//2 int rest: 0
```

14//3 : 14 geteilt durch 3 ist gleich 4 Rest 2" oder "14 modulo 3 ist 2

14//1 : Durch 1 kann immer ohne Rest dividiert werden.



Die Ganzzahldivision selbst gibt es nicht. Zum Beispiel wird 14/3 auf zwei Stellen gerundet, nämlich auf 4.67. Bei Zuweisung dieses Werts an eine Ganzzahl wird **kaufmännisch** gerundet.

```
.type int xresult          ? F|defined(U|xresult) = false  
.formula U|xresult = 14/3  
.formula U|xresult = U|xresult * 3  
result: 'U|xresult'
```

Display:

```
result: 15
```

Rechnen mit Real-Werten

Real-Werte sollten in Masken nicht in Feldern der Art BT dargestellt werden, da sonst eventuell weit rechts stehende Ziffern nicht mehr im Feld sichtbar sind.



Bei Zuweisung einer Real-Variablen an eine Textvariable wird auf zwei Nachkommastellen gerundet.

Verwenden Sie auf der rechten Seite die Umwandlung einer Variablen nach Text per Apostroph, dann bleibt die Genauigkeit erhalten; z.B.

```
.formula U|xtext = 'U|xreal'
```

Kalenderoperationen



Produkte aus Ganzzahlen liefern ein Ergebnis vom Typ real. Das führt, beim Addieren oder Subtrahieren dieses Ergebnisses zu einem Datum, zu einer Fehlermeldung. Weisen Sie in diesem Fall Ihr Ergebnis einer Variable vom Typ integer zu und verwenden Sie diese.

Rechnen mit Arbeitstagen

Per & kann auch die Anzahl der Arbeitstage eines Zeitraums errechnet werden. Sind d1 und d2 von der Art "Datum", so liefert d1 & d2 die Anzahl der Arbeitstage im Zeitraum. Die Reihenfolge von Untergrenze und Obergrenze ist beliebig, somit ist d1 & d2 = d2 & d1. Die Obergrenze selbst wird nicht mitgezählt, damit in einem nur aus Arbeitstagen bestehenden Zeitraum gilt: Kalendertage = Arbeitstage, also d2 - d1 = d2&d1. Aus diesem Grund ist auch für d1 = d2 das Ergebnis 0.

Außerhalb des Arbeitstagekalenders wird die Anzahl der Arbeitstage pro Woche berücksichtigt, aber nicht ihre Lage im Kalender. Beginnt also bei fünf Arbeitstagen pro Woche ein zwölf-tägiger Zeitraum mit Montag, so dass nur ein Wochenende enthalten ist, werden trotzdem pauschal 12 * 5/7 Arbeitstage gewertet.

Abgerundet ergibt das acht Tage.

Beim Addieren von Arbeitstagen mit "datum & Ganzzahl" ist das Ergebnis immer ein Arbeitstag. Liegen z.B. fünf Arbeitstage im Zeitraum von d1 = 26. April bis d2 = 1. Mai (Feiertag), dann liefert "d1 & 5" trotzdem nicht d2, sondern den ersten Arbeitstag danach. Also liefert "d1 & (d2 & d1)" nicht immer d2! Die Kalenderrechnung ist dagegen frei von diesem sprunghaften Verhalten. Hier gilt immer: $d1 + (d2 - d1) = d2$.

Bei der Rechnung mit Arbeitstagen gilt:



Ist nach der Addition von null Arbeitstagen das Ausgangsdatum ein Arbeitstag, so ist das Resultat gleich dem Ausgangsdatum. Ist dagegen das Ausgangsdatum ein arbeitsfreier Tag, so ist das Resultat der erste Arbeitstag, der auf das Ausgangsdatum folgt.

```
.type GD2 xgd2date           ? F|defined(U|xgd2date) = false
.type GD2 xgd2result        ? F|defined(U|xgd2result) = false
.formula U|xgd2date = "10.11.2012"
.formula U|xgd2result = U|xgd2date & 0
'U|xgd2date' & 0 workingday(s) = 'U|xgd2result'
```

display:

```
10.11.12 & 0 workingday(s) = 12.11.12
```

Ausgabe: 12.11.2012 da der 10.11.2012 ein Samstag und der 11.11.2012 ein Sonntag ist.

Bei der Rechnung mit Kalendertagen hingegen gilt:

```
.type GD2 xgd2date           ? F|defined(U|xgd2date) = false
.type GD2 xgd2result        ? F|defined(U|xgd2result) = false
.formula U|xgd2date = "10.11.2012"
.formula U|xgd2result = U|xgd2date + 0
'U|xgd2date' + 0 calendarday(s) = 'U|xgd2result'
```

display:

```
10.11.12 + 0 calendarday(s) = 10.11.12
```

Datums- und Zeitoperationen

Vergleichsoperationen

Sie können GD-Werte und SD-Werte untereinander und miteinander vergleichen

<	kleiner
>	größer
<=	kleiner gleich
>=	größer gleich
=	gleich
<>	ungleich

Uhrzeitangaben

Ist z eine Uhrzeit und i eine **ganze Zahl**, so bildet:

$z : i$	z gerundet auf i Minuten. Zulässige Werte für i sind 5, 10, 15, 20, 30, 60
$z + i$	die Uhrzeit i Minuten nach z
$z - i$	die Uhrzeit i Minuten vor z

Ist z eine Uhrzeit und r eine **reelle Zahl**, dann bildet

$z + r$	die Uhrzeit r Stunden nach z
$z - r$	die Uhrzeit r Stunden vor z

Die Zahlen i und r können auch negativ sein. Der Tageswechsel wird bei jeder Rechenoperation berücksichtigt. Er wird in Kalendertagen relativ zum Ausgangsdatum in der Variablen **G | tagkorr** angezeigt, kann also ebenfalls positiv oder negativ sein.

Operation	G tagkorr	Bemerkung
12:00 - 17,5	-1	ein Tagesdatum rückwärts
23:00 + 26,5	2	zwei Tagesdaten vorwärts
12:00 - 5,0	0	kein Tageswechsel

Sind z_1 und z_2 Uhrzeitangaben und liegt z_1 vor z_2 , so bildet

$z_2 - z_1$	die Spanne in Minuten zwischen z_1 und z_2
$z_1 - z_2$	die negative Spanne in Minuten zwischen z_2 und z_1

Restbildung

Die Restbildung wird für zyklische Vorgänge wie Uhrzeiten und Wochentage benötigt.

Datum modulo 7 liefert die Nummer des Wochentags. (Mo = 1 - So = 7)

Datum modulo 1 liefert den Namen des Wochentags.

Es können keine andere Werte als 7 und 1 auf ein Datum angewandt werden!

Die bedingte Ausgabe beginnt mit einem Fragezeichen.

In diesem Beispiel wird geprüft, ob es sich beim aktuellen Datum um einen Wochentag oder um das Wochenende handelt.

G|date liefert das aktuelle Tagesdatum.

```
? weekend. ? G|date//7 > 5
? weekday ? G|date//7 <= 5
```

Ausgabe:

```
weekday
```

In diesem Beispiel erfolgt eine bedingte Ausgabe eines Hinweises, wenn der Termin auf ein Wochenende fällt.

```
.type gd2 xgd2date      ? F|defined(U|xgd2date) = false
.read "Fill in date:" U|xgd2date
?Attention: 'U|xgd2date' is weekend date      ? U|xgd2date // 7 > 5
```

```
Attention: 11.11.12 is weekend date
```

Rechnen mit Zeitdauern

Die **Variablenart** GP ist eine **Zeitdauer**. Die unterschiedlichen Ausprägungen GP4, GP3, GP2 oder GP1 bestimmen das Ausgabeformat, haben jedoch keinen Einfluss auf das Rechnen (**.formel**).

Zeitdauern können mit Zeitpunkten (Arten: GD19, GD20 und GD14) verrechnet werden. Eine Zeitdauer entsteht z.B. durch Differenz zweier Zeitpunkte.



Zeitdauern und reine Datumsfelder

Reine Datumsfelder (Arten: GD, GD2, GD8, GW, GW2) können nicht mit Zeitdauern verrechnet werden. Hier werden, historisch bedingt, **Kalendertage** als ganzzahlige Variable ("INT") addiert und subtrahiert.

- **Addition**
- **Subtraktion**
- **Vergleiche**

Addition

1. GD19 + GP = GD19
2. GP + GP = GP

Beispiel: GD19 + GP = GD19

```
.type GD19 xgd19
.type GD19 xgd19b
.type GP4 xgp4
..
.formula U|xgd19="21.03.2010 00:00:00"
U|xgd19 = 'U|xgd19'
..
.formula U|xgp4 = "4D12h"
U|xgp4 = 'U|xgp4'
..
.formula U|xgd19b = U|xgd19 + U|xgp4
U|xgd19 + U|xgp4 = 'U|xgd19b' (25.03.2010 12:00:00)
```

Beispiel: GP + GP = GP

```
.type GP4 xgp4
.type GP3 xgpa xgpb
..
.formula U|xgpa = "25D04h03m123s"
U|xgpa = 'U|xgpa' (25D04h05m)
..
.formula U|xgpb = "1D1h01m1s"
U|xgpb = 'U|xgpb' (1D01h01m)
..
.formula U|xgpb = U|xgpa + U|xgpb
U|xgpb = 'U|xgpb' (26D05h06m)
..
.formula U|xgp4 = U|xgpb
U|xgp4 = U|xgpb 'U|xgp4' (26D05h06m04s)
```

Subtraktion

1. GD19 - GD19 = GP
2. GD19 - GP = GD19
3. GP - GP = GP
4. SD - SD = Integer
5. SD - GD = Integer
6. GD - SD = Integer



Der Integerwert beschreibt die Anzahl der Tage zwischen den beiden Daten.
 SD und GD dürfen nicht leer sein.
 GD darf kein Sonderwert wie unendlich oder Abruf sein.

GD darf keinen Zeitanteil enthalten.
 Unterarten von GD, die einen Zeitanteil enthalten können, sind nicht zugelassen.

Beispiel: GD19 - GD19 = GP

```
.type GD19 xgd19 xgd19b
.type GP4 xgp4
..
.formula U|xgd19 = "21.03.2010 00:00:00"
U|xgd19 = 'U|xgd19'
..
.formula U|xgd19b = "20.04.2010 12:00:01"
U|xgd19b = 'U|xgd19b'
..
.formula U|xgp4 = U|xgd19 - U|xgd19b
U|xgd19 - U|xgd19b = 'U|xgp4' (-30D12h00m01s)
```

Beispiel: GD19 - GP = GD19

```
.type GD19 xgd19 xgd19b
.type GP4 xgp4
..
.formula U|xgd19="21.03.2010 00:00:00"
U|xgd19 = 'U|xgd19'
..
.formula U|xgp4="4D12h"
U|xgp4 = 'U|xgp4'
..
.formula U|xgd19b = U|xgd19 - U|xgp4
U|xgd19 - U|xgp4 = 'U|xgd19b' (16.03.2010 12:00:00)
```

Beispiel: GP - GP = GP

```
.type GP4 xgp4
.type GP3 xgp xgpb
..
.formula U|xgp = "25D4h03m123s"
U|xgp = 'U|xgp' (25D04h05m)
..
.formula U|xgpb = "1D1h01m1s"
U|xgpb = 'U|xgpb' (1D01h01m)
..
.formula U|xgpb = U|xgp - U|xgpb
U|xgp - U|xgpb = 'U|xgpb' (24D03h04m)
```

```
..
.formula U|xgp4 = U|xgpb
U|xgp4 = 'U|xgp4' (24D03h04m02s)
24D03h04m02s
```

Vergleiche

Man kann vergleichen nach dem Muster "Ist eine Zeitdauer größer als eine andere Zeitdauer?".

Datum und Woche

T kann eine Terminvariable der Art Datum oder Woche sein. Die Art ist nur für das Ausgabeformat eines Termins maßgebend, hat aber auf das Rechnen (**.formel**) keinen Einfluss. Ist **T** ein Termin und **i** eine ganze Zahl, dann gilt:

- **T + i = i** Kalendertage nach **T** (gilt für GD und SD)
- **T - i = i** Kalendertage vor **T** (gilt für GD und SD)
- **T & i = i** Arbeitstage nach **T** (gilt nur für GD)



i kann auch negativ sein, so dass mit der **&**-Operation auch arbeitstagesweise rückwärts gerechnet werden kann.

Sind **T1** und **T2** Terminangaben und liegt **T1** zeitlich vor **T2**, so bildet **T2 - T1** die Spanne von Kalendertagen zwischen **T1** und **T2**, **T1 - T2** die entsprechend negative Spanne. Variablen der Art "Datum" und "Woche" reagieren in Verbindung mit dem Kommando **.formel** auf die Angabe **:8**, indem ein Termin in die Kurzform des sortierfähigen Datums nach ISO 8601 (**jjjjmmtt**, z.B. **20100418**) umgewandelt wird. In diesem Format sind Termine sortierfähig. (Siehe unter [Sortieren von Terminen](#)).

.formel x = datum:8

Das Ergebnis ist ein Text.



Anders als Variablen der Art "Datum" reagieren Variablen der Art "Woche" bei der Ausgabe nicht auf **:8** (siehe **.art - Variablenart**), wohl aber in Formeln.

Die Reihenfolge, in der die verschiedenen Operationen abgearbeitet werden, wird durch Klammern und durch die folgenden Prioritäten (in fallender Reihenfolge) geregelt.

+ -	mit einem Operanden
< < > >	Teilttexte
:	Rundung
* /	"Punktrechnung"
+ -	"Strichrechnung"
< > <>	Vergleiche

-	nicht
&	und
;	oder

Logische Operationen

&	und
;	oder
-	nicht

Programme aufrufen und beenden

Beim Start der FO werden Befehle oder Text aus einer Programmdatei oder von der Tastatur gelesen. Text wird auf den Bildschirm oder das aktuelle Ausgabegerät ausgegeben. Ein- und Ausgabekanal können während der Abarbeitung eines FO-Programms durch die Angabe entsprechender Befehle gewechselt werden.

.EINGABE / .INPUT - Unterprogramm ausführen

.EINGABE	.INPUT
deutsch	englisch
.EINGABE .EINGABE 'Dateiname' .EINGABE -TEXT 'Dateiname' .EINGABE <Klassenname>.java [Argumente]	.INPUT .INPUT 'file name' .INPUT -TEXT 'file name' .INPUT <class name>.java [arguments]
.eingabe startet den Direktmodus. .eingabe 'Dateiname' interpretiert den Dateinamen bzw. den Inhalt der Variablen als FO-Programm und führt es aus. .eingabe -text 'Dateiname' interpretiert die Datei als reinen Text und gibt ihn auf das aktuelle Ausgabegerät aus. .eingabe <Klassenname>.java [Argumente] interpretiert <Klassenname>.java als JFOP und führt es aus.	

Für wiederkehrende Aufgaben sind Unterprogramme nützlich. Mit dem Kommando

.eingabe 'Dateiname'

wird das genannte Programm als Unterprogramm abgearbeitet. Existiert die Datei nicht, erfolgt ein Fehlerabbruch. Die Existenz der Datei kann vorher mit **.datei -test 'Dateiname'** geprüft werden.

Fehlt der Dateiname oder ist "", wird von der Tastatur gelesen. Dies wird als **Direktmodus** bezeichnet.

Wurde **'Dateiname'** per Variable angegeben und ist der Inhalt der Variablen leer, dann wird das Kommando ignoriert. Auf diese Weise können Sie z.B. Freitextverweise einfach behandeln: es muss nicht abgefragt werden, ob der Freitext vorhanden ist.

Das aufgerufene Unterprogramm kann seinerseits wieder beliebige FO-Kommandos enthalten, so dass Textbausteine benutzt werden können, um Seitenformate festzulegen. FOPs können geschachtelt sein, d.h. sie dürfen selbst wieder **.eingabe**-Kommandos enthalten. So können bis zu zehn Unterprogramme geschachtelt werden.

Bei **.eingabe** gilt dieselbe **Suchlogik** wie beim Aufruf von FOPs. Den Arbeitsbereich des Aufrufers können Sie bei Bedarf mit dem **Standard-FOP ARB** ermitteln.

Näheres zu individuellen Programmverzeichnissen siehe Kapitel **Individualisierung von FOPs**, über Unterprogramme das Kapitel **Allgemein nützliche Unterprogramme**.

Reine Textausgabe ohne Interpretation

Mit dem Kommando

```
.eingabe -text 'Dateiname'
```

können Sie bei herkömmlichen FOPs die Interpretation von Metazeichen abschalten und eine Datei als reinen Text statt als FO-Programm interpretieren lassen. Es werden keine Kommandos ausgeführt und keine Variablen durch Inhalte ersetzt.

Auf die Datei werden vor der Ausgabe alle angeschalteten Formatierungen (z.B. Blocksatz, Seitenränder, Kopf- und Fußzeilen) angewendet. Das geschützte Leerzeichen (leerchar), ist ein formatierendes Zeichen und wird daher ebenfalls interpretiert, d.h. durch ein Leerzeichen ersetzt. (Die Standardeinstellung für die Ersetzung dieses Metazeichens ist allerdings ohnehin das Leerzeichen). Alle anderen Metazeichen, z.B. **. :** **' ?** stehen für sich selbst.

Bei **.eingabe -text** können, obwohl die Variablenersetzung für den eigentlichen Text abgeschaltet ist, im Oben- und Unterpuffer Variablen in Apostrophen wie z.B. **'Seite'** verwendet werden.

JFOPs als Unterprogramme von FOPs aufrufen

Sie können JFOPs aus FOPs als Unterprogramme aufrufen. Typische Anwendungsbeispiele sind:

- Sie nutzen in bestehenden FOPs Funktionen, die bereits per Java implementiert wurden.
- Sie lagern einzelne Programmteile nach Java aus, z.B. Mustererkennung eines Stringliterals.
- Sie nutzen Java-Features, die in FO nicht zur Verfügung stehen, z.B. Netzwerkprogrammierung, Webservices, XML-Verarbeitung.

Das **aufgerufene JFOP** läuft im integrierten Modus analog zu eingebetteten FOPs. Im aufgerufenen JFOP haben Sie Zugriff auf die bekannten Puffer, z.B. U-Puffer. Änderungen im U-Puffer sind anschließend im aufrufenden FOP verfügbar.

Die FO-Variable **G|status** (Typ INT) liefert den Returnwert des Java-FOPs.

Ist das aufgerufene JFOP nicht ausführbar oder endet es mit einer Exception, wird das Programm abgebrochen. Dies ist analog zum Verhalten von **-text** bei Nichtausführbarkeit bzw. Fehler. Die Option

-text ist für JFOP unzulässig und führt ebenfalls zum Abbruch. Verwenden Sie die Methode `F0.println(Object obj, boolean noEvaluation)`, wenn Sie bei Textausgabe keine Variablenersetzung wünschen.

Aufruf

`.eingabe "paket.klassenname.class" [ Argumente ]`

Das aufgerufene Programm muss das Interface `FOPRunnable` oder `ContextRunnable` implementieren und von einem Klassenlader gefunden werden. Siehe auch: [Aufruf eines Java-FOPs \(JFOP\)](#).

Wenn ein Argument in Apostrophen übergeben wird, dann wird der Inhalt des Arguments nach String konvertiert. Die Argumente stehen im JFOP im String-Array `String[] args` zur Verfügung.

```
..!interpreter english noabbrev
.. owbfhilfe/GET.TIME.MILLIS
..
.type realx xrmillis      ? F|defined(U|xrmillis) = G|false
..
.input "de.abas.training.basic.anfragen.GetTimeMillis.class"
..
Time/msec: 'U|xrmillis'
..
.end
```

```
public class GetTimeMillis implements ContextRunnable {
@Override
public int runFop(FOPSessionContext ctx, String[] arg1) throws FOPEException {
DbContext dbContext = ctx.getDbContext();
run(dbContext, arg1);
return 0;
}
private void run(DbContext dbContext, String[] arg1) {
UserTextBuffer userTextBuffer = BufferFactory.newInstance(true).getUserTextBuffer();
long currentTimeMillis = System.currentTimeMillis();
if(userTextBuffer.isVarDefined("xrmillis")){
userTextBuffer.setValue("xrmillis", currentTimeMillis);
}
}
}
```

Das FO-Programm ruft das JFOP auf. Dieses ermittelt die aktuelle Systemzeit und speichert das Ergebnis in der benutzerdefinierten FO-Variablen `xrmillis` ab. Das FO-Programm gibt anschließend die ermittelte Systemzeit aus.

Stack-Informationen für FOPs



- Es kann immer nur ein JFOP aktiv sein.
- Rufen Sie aus einem vom FOP aufgerufenen JFOP kein FOP auf.

Sie können den Aufruf-Stack der FO (d.h. die geschachtelt aufgerufenen Unterprogramme einschließlich der Zeilennummern und einschließlich JFOPs) abfragen, um eine Reihe von Informationen zu ermitteln:

- Aufruf-Stack einschließlich Zeilennummern und Sprachverzeichnis für Fehlerprotokolle (Mechanismus Fehlerprotokoll, siehe hierzu `.setze diagnose`)
- der Name und die aktuelle Zeile des gerade ausgeführten Unterprogramms, z.B. für Testausgaben, Fehlermeldungen
- die aktuelle Schachtelungstiefe (auch für lokale Bezeichner verwendbar)
- der direkte Aufrufer des gerade ausgeführten Unterprogramms
- ob ein bestimmtes FOP auf dem Stack liegt, d.h. zumindest indirekter Aufrufer ist
- aus welchen Arbeitsbereichen die Aufrufer stammen
- aus welchem Sprachverzeichnis ein FOP stammt

Dafür stehen folgende Variablen zur Verfügung:

Deutsch	Englisch	Beschreibung
<code>G framecount</code>	<code>G frameCount</code>	Anzahl der auf dem Stack liegenden FOPs bzw. JFOPs
<code>G stackframe</code>	<code>G stackFrame</code>	<code>Array(1..frameCount)</code> : Name des auf dem Stack liegenden FOPs bzw. JFOPs (jeweils in normierter Schreibweise, also bei FOPs mit der Groß-Klein-Schreibung wie im Dateisystem, bei JFOPs in Paketschreibweise, Endung ".class")
<code>G linenumber</code>	<code>G lineNo</code>	<code>Array(1..frameCount)</code> : Zeilennummer des auf dem Stack liegenden FOPs; 0 wenn unbekannt, z.B. nach Programmende oder in JFOPs
<code>G stack</code>	<code>G totalStack</code>	gesamter Stack mit allen Unterprogrammen einschließlich JFOPs, jeweils gefolgt von der Zeilennummer, jeweils eingeleitet durch <code>□</code>

```
stack = 'G|totalStack'
```

Ausgabe

```
stack = -> owprenglish/STACK.1:3
```

Jeder Programmname wird also links von `[` und rechts von `:` begrenzt, wodurch man mit dem Matchcode "`[Programmname:]`" leicht nach einem bestimmten Programm suchen kann. Anders als bei '`G|txname`' ist in '`G|stackFrame`' auch der Name des Sprachverzeichnisses ersichtlich, sofern das FOP in einem solchen gefunden wurde.

<Text> vk/LKU <Zeigen>	txname=vk/LKU	stackframe1=vk/LKU
<Text> E/vk/LKU <Zeigen>	txname=vk/LKU	stackframe1=E/vk/LKU

Aktuelle Zeile des aktuellen FOPs ausgeben:

```
'G|lineNo.G|frameCount' -> 'G|stackFrame.G|frameCount'
```

Direkten Aufrufer ausgeben:

```
.type int before
.formula U|before = G|frameCount - 1
caller: 'G|stackFrame.U|before': 'G|lineNo.U|before'
```

Stack als Text ausgeben:

```
'G|totalStack'
```

Stack per Schleife ausgeben:

```
.type int xiindex
frameCount: 'G|frameCount'
.formula U|xiindex = 0
!WHILE
.formula U|xiindex = U|xiindex + 1
.continue ENDWHILE ? U|xiindex > G|frameCount
G|stackFrame['U|xiindex']: 'G|stackFrame.U|xiindex': 'G|lineNo.U|xiindex'
.continue WHILE
!ENDWHILE
```

Damit lässt sich ein Unterprogramm schreiben, das in Fehlersituationen aufgerufen werden kann und den Stack (außer dem eigenen Stackframe) ausgibt.

Zur Prüfung, ob ein bestimmtes FOP (im Beispiel `vk/ABC`) auf dem Stack liegt, gehen Sie wie folgt vor:

```
.println ja ? G|totalStack / "vk/ABC:"
```

In der Variablen '`totalStack`' und im Array `stackFrame.i` sind alle aktiven Unterprogramme inklusive JFOP zu finden.

`stack = XXX.FOP:12utils.Abc.java:0YYY.FOP:20tools/STACKOUT:15` Im Beispiel wurde in Zeile 12 von `XXX.FOP` das JFOP `utils.Abc` aufgerufen, welches wiederum (in unbekannter Zeilennummer) mit `FO.eingabe` das FOP `YYY.FOP` aufrief.

Wenn das JFOP ein bestehendes FOP aufruft, das das Standardprogramm ARB verwendet, so muss ggf. die Variable '`U|baseframe`' auf die Tiefe nach dem JFOP gesetzt werden (siehe [Standard-FOP ARB](#)).

.WEITER / .CONTINUE - Rücksprung zum Aufrufer

.WEITER	.CONTINUE
deutsch	englisch
.WEITER .WEITER "Markenname"	.CONTINUE .CONTINUE "label name"
<code>.weiter</code> bewirkt einen Rücksprung zum aufrufenden FOP bzw. auf oberster Ebene das Ende des FO-Programms. <code>.weiter "Markenname"</code> bewirkt einen Sprung zur Marke.	

Die Rückkehr in das aufrufende FOP erfolgt

- am Ende der aufgerufenen Programmdatei oder
- durch das Kommando `.weiter`

Beispiel: Schleife

```
!BEGINNING
.. program code
.continue BEGINNING
```

.ENDE / .END - FO-Ende

.ENDE	.END
Deutsch	Englisch
<pre>.ENDE .ENDE ['Exit-Status'] [.] .ENDE 'Exit-Status' 'Buttonname' .ENDE 'Abas Kommando' - Keine REST API-Unterstützung .ENDE 0 'Abas Kommando'</pre>	<pre>.END .END ['exit status'] [.] .END 'exit status' 'button name' .END 'Abas command' - No REST API support .END 0 'Abas command'</pre>
.ende beendet das aktuell laufende FO-Programm, aber nicht das aufrufende Java-Programm. Mit 'Exit-Status' kann man den Erfolg des Programms definieren. Als Werte sind natürliche Zahlen zulässig, wobei 0 oder das Fehlen einer Angabe für den Erfolg und jeder andere Wert für einen Misserfolg steht. Ausnahme: Bei den Ereignissen maskaus und maskende werden Werte ungleich 0 zur Dialogsteuerung verwendet. Siehe auch Grafik Maskenereignisse. Ausnahme: .ende 0 'Abas Kommando' legt bei individuellen BU8-, BU10- und BU12-Buttons das Kommando fest, mit dem die Submaske geöffnet wird. Siehe auch Buttons. Diese drei Buttonarten öffnen beim Klick durch den Benutzer eine Submaske. Für Standardbuttons ist die Submaske vorgegeben, für individuelle Buttons legt der Benutzer im buttonvor-Event fest, welche Submaske geöffnet werden soll, und zwar mit dem .ende Kommando. Mit einem abschließenden Punkt kann das FO-Ausgabefenster ohne Quittierung geschlossen werden. Mit Buttonname kann im buttonnach-FOP und im felldaus-FOP das Klicken eines Ende-Buttons (Art "BU10", "BU12" oder "BU18") simuliert werden. Siehe auch: Speichern und Fenster schließen per EFOP.  Das Kommando .ende 'Abas Kommando' wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. .ende 'Abas Kommando' hat im EFOP keine Wirkung. Der Exitstatus ist 0. Im FOP hat der Exitstatus keine Wirkung. Nach dem FOP wird das angegebene 'Abas Kommando' ausgeführt.	

Wenn **.weiter** von einem eingebetteten FOP aufgerufen wird, geht die Kontrolle zurück ans aufrufende Java-Programm (näheres siehe Javadoc zu **F0.ende()**). Anders als bei **.weiter**, wird also nicht nur das aktuelle Unterprogramm beendet.

In EFOPs wird bei bestimmten Ereignissen durch den Exit-Status der weitere Programmablauf gesteuert; die Wirkung wird in der jeweiligen Ereignisbeschreibung spezifiziert. Es sollten nur die dort definierten Werte verwendet werden, da andere Werte in zukünftigen Versionen eine eigenständige Bedeutung erlangen können. Im JFOP kann der Exit-Status als Return-Wert von **F0.eingabe()** abgefragt werden.



Entscheidung zwischen 'FOP beenden' und 'zurück zum aufrufenden FOP'

Im Allgemeinen werden FOPs flexibler, wenn man statt **.ende** das Kommando **.ende** verwendet. Der Aufrufer des FOPs kann dann selbst entscheiden, ob das Gesamtprogramm weiter laufen soll oder nicht. Sinnvolle Einsatzbeispiele für **.ende** sind nicht behebbare Fehler oder die Auswahl einer Option wie "Programm beenden" durch den Benutzer.

Die Option **Buttonname** sollte wegen der ungewöhnlichen Benutzerführung nur in einzelnen Spezialfällen benutzt werden.

```
..!interpreter english noabbrev
.. your code
.end 0 "FileCancelClose"
```



Wenn ein Ende-Button in der Tabelle gedrückt wurde, steht die Variable **G|currTabRow** bei Wiederladen immer auf dem Wert der Ereigniszeile des auslösenden Ereignisses, selbst wenn das EFOP vor dem Drücken in eine andere Zeile positioniert hat.

Lesen Sie auch Kapitel **.kommando** - Aufruf von Abas Kommandos.

.SETZE FENSTERSCHLIESSEN / .SET CLOSEWINDOW - Ausgabefenster schließen

.SETZE FENSTERSCHLIESSEN	.SET CLOSEWINDOW
deutsch	englisch
.SETZE FENSTERSCHLIESSEN + -	.SET CLOSEWINDOW + -
.setze fensterschliessen + legt fest, dass am Ende das FO-Ausgabefenster geschlossen werden soll.	
.setze fensterschliessen - schaltet diesen Programmzustand wieder aus.	

Das Kommando **.ende** entspricht dem Kommando **.setze fensterschliessen**.

.setze fensterschliessen ist insbesondere in JFOPs nützlich, da dort das **.ende**-Kommando der FO nicht vorhanden ist.

.SETZE DIAGNOSE / .SET ERRORREPORT - Fehlerprotokoll

.SETZE DIAGNOSE	.SET ERRORREPORT
deutsch	englisch
.SETZE DIAGNOSE +	.SET ERRORREPORT +

.SETZE DIAGNOSE	.SET ERRORREPORT
erstellt bei Abbruch der FO wegen eines Fehlers die Administrationsmeldung 2399: FOP: Programm-Abbruch. Durch das Feld Vorbelegung für FO-Kommando .setze diagnose in [Konfiguration zeigen] können Sie die Vorbelegung für das Fehlerprotokoll für alle FO-Programme steuern. Ist die Option gesetzt, wirkt die Funktion des Feldes wie das Kommando .setze diagnose + zu Beginn jedes FO-Programms. Sie können das Fehlerprotokoll jederzeit mit .setze diagnose - deaktivieren.	

Benutzereingaben anfordern

.LESEN / .READ - Tastatureingaben einlesen

.LESEN	.READ
deutsch	englisch
.LESEN "Text" Variablenname	.READ "text" variablename
Liest Tastatureingaben ein. Die Eingabe muss zur Art der Variablen passen. "Text" wird als Anforderungstext zur Eingabe benutzt.	



Veraltet: Beginnt der **Variablenname** mit einem Großbuchstaben, dann wird der eingelesene Inhalt in Großbuchstaben umgewandelt.

Alternativ: Wenn Sie eingegebenen Text zu Großbuchstaben konvertieren wollen, dann verwenden Sie die Funktion **F|toupper(U|variable)**.

Das FO-Kommando **.lesen** liest die Eingabe so lange, bis eine zur Art der Variablen passende Eingabe erfolgt. Der Wert wird in die angegebene Variable eingetragen. Der Dialog bleibt so lange stehen, bis der Benutzer entweder eine korrekte Eingabe macht, oder durch <Abbruch> das FO-Programm abbricht. Ist <Abbruch> gesperrt durch **.setze abbruch -**, wird **G|mehr** auf **G|false** gesetzt. Die eingelesene Variable ist bei Abbruch und bei Leereingabe leer.

Im FO-Programm steht folgendes Kommando:

```
.type text xtform
.read "short (s) or long (l) type?" U|xtform
type: 'U|xtform'
```

Es erscheint die Rückfrage: "Kurze (s) oder lange (l) Form?". Ihre Antwort wird von der Tastatur gelesen. Der getippte Text wird Inhalt der Variablen **'U|xtform'**, so dass die weitere Ausgabe davon abhängig gemacht werden kann.

Mehrere Texte einlesen

Durch die Angabe mehrerer Variablen können Sie mehrere Texte nacheinander einlesen.

Beispiel: Mehrere Texte einlesen

```
.type text xtdb
.type text xtgroup
.type text xtido
.read "Please insert database, group, ident number:" U|xtdb U|xtgroup U|xtido
'U|xtdb': 'U|xtgroup' -> 'U|xtido'
```

Please insert database, group, ident number:

0

Please insert database, group, ident number:(2)

1

Please insert database, group, ident number:(3)

1001

Mehrzeiliger Anforderungstext

In der GUI werden beim Kommando **.lesen "Text" Variablenname** vorhandene Semikolons im **text** durch einen Zeilenumbruch ersetzt. Damit kann ein mehrzeiliger Anforderungstext angegeben werden.

Beispiel: Mehrzeiliger Anforderungstext

```
.type text xtdb
.type text xtgroup
.type text xtido
.READ "Please insert database,; group,; ident number:" U|xtdb U|xtgroup U|xtido
'U|xtdb': 'U|xtgroup' -> 'U|xtido'
```

Please insert database,
group,
ident number:

.BOX / .BOX - Ausgabe mehrzeiliger Meldungen

.BOX	.BOX
deutsch	englisch
<code>.BOX [-BUTTON 'Buttonset'] [-TITEL Text...]</code>	<code>.BOX [-BUTTON 'buttonset'] [-TITLE text...]</code>
gibt eine Dialogbox mit einer mehrzeiligen Meldung per FOP oder EFOP aus. <code>-button 'Buttonset'</code> legt fest, welche der Buttons "OK", "Ja", "Nein" und/oder "Abbruch" angezeigt werden. <code>-titel text</code> schreibt den angegebenen Text in den oberen Rahmen der Dialogbox.	

Ein `.box`-Kommando sollte immer mit zwei Punkten (Kommentar) abgeschlossen werden, andernfalls endet die Box am ersten Punktbefehl nach `.box`. Ein `.box`-Kommando endet außerdem am Dateiende des gerade aktiven Unterprogramms. Die Dialogbox wird nach einem Tastendruck geschlossen.

`-button 'Buttonset'` legt fest, welche Buttons angezeigt werden sollen. Fehlt die Option, wird ein OK-Button angezeigt. Zulässige Werte für `'Buttonset'` sind

```
ok
yes:no
no:yes
ok:cancel
cancel:ok
yes:no:cancel
no:yes:cancel
```

Andere Kombinationen, z.B. `cancel:yes`, sind *nicht zulässig*.

Der erste genannte Button ist jeweils der Default-Button (dieser Button ist aktiviert und wird bei <Return> ausgelöst), also z.B. bei `yes:no:cancel` "Ja", bei `no:yes:cancel` "Nein".

Alternativ können die im FOP `BOX.CONST` definierten Konstanten verwendet werden. Vorausgesetzt, `BOX.CONST` ist vorhanden.

```
buOk
buYesNo
buNoYes
buOkCancel
buCancelOk
buYesNoCancel
buNoYesCancel
buOkCancel
buCancelOk
```

Erzeugen Sie eine Box mit dem Hinweis "file saved".

```
.box
file saved
```

Die Option **-titel text** muss als letzte Option stehen.

Nach **-titel** können Variablen oder Texte stehen; die Ausgabe erfolgt so wie bei FO-Zeilen.

Anführungszeichen in Textausgaben



Direkt Text ausgebende Kommandos (**.hinweis**, **.fehler**, **.menue**, **.box** und **.println**) stellen den Text genau so dar wie angegeben, analog dem **Direktmodus**. So ist es auf einfache Weise möglich, Anführungszeichen auszugeben, z.B.:

```
.fehler Im Unterprogramm trat der Fehler "keine Netzverbindung"
auf.
```

```
.hinweis Rechnungsabschlusspositionen wurden ergänzt.
```

Abfrage der Benutzereingabe

Nach der Benutzereingabe enthält die Variable **'G|box'** vom Typ AK242 eine der folgenden Konstanten aus dem Satz **V-13-00** der Variablentabelle:

boxyes	Ok- oder Ja-Button gedrückt
boxno	Nein-Button gedrückt
boxcancel	Cancel-Button gedrückt

Beispiel: Ja/Nein-Abfrage

```
.box -button yes:no
accept values ?
..
.continue ACCEPT ? G|box = G|boxyes
```

Beispiel: Ja/Nein/Abbruch

```
.type text xtfile
.formula U|xtfile = "text.txt"
..
.input BOX.CONST
.box -button 'buYesNoCancel' -title file 'U|xtfile' changed
```

Drop changes ?

```
..
.continue DROP ? G|box = G|boxyes
.continue SAVE ? G|box = G|boxno
.continue DIALOG ? G|box = G|boxcancel
```

Zur sprachlichen Ausgabe des geklickten Buttons muss der Inhalt der Variablen **G|box** von AK242 nach A242 konvertiert werden.

```
.type text xtfile
.formula U|xtfile = "text.txt"
..
.input BOX.CONST
.box -button 'buYesNoCancel' -title file 'U|xtfile' changed
Drop changes ?
.println selected button "'F|tostring(G|box, "A242")'" (short "'G|box'")
```

Darstellung

Wie bei Dialogboxen aus dem Abas Kern: selbstständiges Fenster, das über das aktuelle Fenster gelegt wird und dessen Größe sich nach dem Umfang des Textes richtet.

Steuerung durch Lader

Die Steuerung der FO-Boxen durch den Lader ist möglich. Eine Box (außer der OK-Box) entspricht einer Frage, daher wird empfohlen, die Angabe wie bei sonstigen Fragen mit **<?>** einzuleiten. Die Zeile wird überlesen, wenn gar keine Frage gestellt wurde.

Boxen mit **-button 'buOk'** müssen im Lader nicht behandelt werden, sie werden vom Lader automatisch bestätigt. Für andere Boxen sind folgende Werte angebracht:

```
<?>ja
<?>nein
<?><Abbruch>
```

Die Angabe **<?><Return>** ist zwar auch möglich (klickt den Default-Button), aber weniger stabil, da sich der Default-Button durch Programmänderungen ändern kann.



Das Kommando **.box** wird manchmal dazu gebraucht, mehrzeilige Meldungen auszugeben, was vor allem auf der grafischen Oberfläche verwirrend ist. An diesen Stellen müssen lediglich die Befehle **.box** und **.box** ausgetauscht werden.

Im Unterschied zum Ergebnis von **.menue** ist das Fenster nicht scrollend und der Text wird abgeschnitten. Außerdem werden **'G|mentxt'** und **'menu'** nicht gesetzt.

Verwandte Themen

.menue - Menüauswahlfenster

.fehler - Fehlermeldung

.hinweis - Meldungen

.MENUE / .MENU - Menüauswahlfenster

.MENUE	.MENU
deutsch	englisch
<pre>.MENUE [(0,0,'Zeilennummer')]["Überschrift"] ['Spaltenformatzeile'] Menüzeile 1 [>> 'Punktbefehl' Text] Menüzeile 2 [>> 'Punktbefehl' Text] Menüzeile n [>> 'Punktbefehl' Text] ..</pre>	<pre>.MENU [(0,0,'line number')] ["header"] ['tab line'] menu line 1 [>> 'dot command' text] menu line 2 [>> 'dot command' text] menu line n [>> 'dot command' text] ..</pre>
.menue erstellt in der Ausgabe ein Menü, auf dessen Zeilen mit den Tasten <Pfeil auf> und <Pfeil ab> oder dem Anfangsbuchstaben der gewünschten Zeile positioniert werden kann. Mit <Return> wird die Menüzeile ausgewählt. Die Überschrift wird über die Fensterbreite zentriert. Die getroffene Auswahl kann mit der Variablen 'G menue' abgefragt werden. Bei <Abbruch> ist der Wert der Variablen 'G menue' = 0, ansonsten gleich der Nummer der gewählten Menüzeile. Die Zeilennummer legt die Position des Menüauswahlbalkens fest. Ist die Zeilennummer größer als die Anzahl der Menüzeilen, wird auf die erste Zeile positioniert. Die Menüzeilen dürfen Texte und Variablen enthalten (z.B. 'G frei'), die wie üblich durch ihren Wert ersetzt werden. Die Spaltenformatzeile ermöglicht, die Auswahltexte in Spalten darzustellen. Ein Auswahltext wird nur dann entsprechend dem Spaltenformat angezeigt, wenn die Spaltentexte durch ein Leerzeichen getrennt sind. Auswahltexte, die nicht zu diesem Spaltenformat passen, werden in der GUI unformatiert, d.h. durchgängig angezeigt, z.B. Trennlinien. Durch das Anbinden von Punktbefehlen und text kann das Menü interaktiv bedient werden.	

Durch **.menue** wird jede einzelne Zeile wie bei der Textausgabe formatiert. Mehrzeilige Variablen werden mehrzeilig ausgegeben; auch der linke Rand wird berücksichtigt.

Außerdem eignet sich **.lesen** wie das Kommando **.lesen** zur Anforderung von Antworten.

Ein **.menue**-Kommando sollte immer mit zwei Punkten (Kommentar) abgeschlossen werden, andernfalls endet das Menü am ersten Punktbefehl nach **.menue**. Ein **.menue**-Kommando endet außerdem am

Dateiende des gerade aktiven Unterprogramms. Alle Zeilen zwischen `.menue` und diesem Abschluss bilden die Menüzeilen, also die auf dem Bildschirm sichtbaren Anforderungstexte des Menüs.

Ein Menü kann maximal 1000 Zeilen enthalten, einschließlich Überschrift. In JFOPs kann die maximal mögliche Zeilenzahl über `C.MULTILINE_MAXLINES` abgefragt werden.

Die Breite und Länge eines Menüs kann sich grundsätzlich über den ganzen Bildschirm erstrecken. Beim Schreiben des Menütextes ist jedoch zu berücksichtigen, dass der Rahmen an allen Rändern Platz beansprucht. Da die Menüs scrollbar sind, ist ihre Größe nicht auf das aktuelle Fenster beschränkt. Bei EFOPs können bis zu 99 Alternativen aufgeführt werden. Es wird garantiert, dass durch das Menü nicht der gesamte ausgegebene Text weggescrollt wird. Es bleiben wenigstens zwei Zeilen erhalten.

Anführungszeichen in Textausgaben



Direkt Text ausgebende Kommandos (`.hinweis`, `.fehler`, `.menue`, `.box` und `.println`) stellen den Text genau so dar wie angegeben, analog dem `Direktmodus`. So ist es auf einfache Weise möglich, Anführungszeichen auszugeben, z.B.:

```
.fehler Im Unterprogramm trat der Fehler "keine Netzverbindung"
auf.
```

```
.hinweis Rechnungsabschlusspositionen wurden ergänzt.
```

Anbinden von Punktbefehlen und Text

Nach jeder Menüzeile kann, durch zwei Größer-Zeichen (`>>`) abgetrennt, ein Punktbefehl angegeben werden, der nicht auf dem Bildschirm hinter dem Menüfenster sichtbar ist:

Beispiel: Schreibaufwändige Standardeingaben im EFOP per Menü in die Maske übernehmen

```
..!interpreter english englvar noabbrev
.type text xtnotice xta xtb xtc
.formula U|xta = "Please notice the changes since last quotation."
.formula U|xtb = "subject to modifications an amendments."
.formula U|xtc = "We issue a warranty of 12 month."
..
.menu Select text
'U|xta' >> .formula U|xtnotice = U|xta
'U|xtb' >> .formula U|xtnotice = U|xtb
'U|xtc' >> .formula U|xtnotice = U|xtc
..
'G|noSelectedMenuOption': 'U|xtnotice'
```

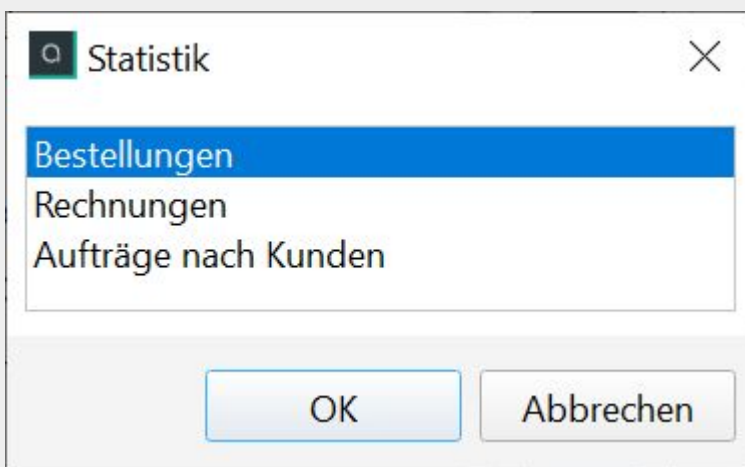
Ein nach `>>` stehendes Kommando wird sofort nach der Auswahl ausgeführt.

```

..!interpreter english englvar noabbrev
.type bool xbstatus
.formula U|xbstatus = G|false
.menu Statistik
Bestellungen          >>
Rechnungen            >>   .continue RECHNUNG
Aufträge nach Kunden  >>   Belegen
?Monatsbericht        ?    U|xbstatus = G|true
..
Wahl: 'G|noSelectedMenuOption' -> 'G|menuText'

```

Am Bildschirm erscheint folgende Ausgabe:



Auswahl	Wert von G menue	Wert von G mentxt	Aktion
Bestellungen	1		
Rechnungen	2		Sprung zur Marke RECHNUNG
Aufträge nach Kunden	3	Belegen	
Monatsbericht	4		Diese Zeile erscheint nur, wenn U xbstatus = true
<Abbruch>	0		

Nach >> kann auch ein Text angegeben werden. Dieser darf *nicht* mit einem Punkt beginnen.

Er wird Inhalt der Variablen 'G|menuText' und erscheint ebenfalls nicht in der Menüausgabe auf dem Bildschirm:

Beispiel: Anbinden von Text

```
.type text xtfilename
```

```
.formula U|xtfilename = "owfohelp/INPUT9.txt"
.menu Reporting
file new          >> new
append result    >> append
..
.output new 'U|xtfilename' ? G|menuText="new"
.output addedto 'U|xtfilename' ? G|menuText="append"
..
File 'U|xtfilename' -> 'G|menuText'
```



Die beiden Menüvariablen '**G|noSelectedMenuOption**' und '**G|menuText**' können beim nächsten **.menu**-Kommando überschrieben werden.

Optische Menügestaltung

Das Menüfenster wird immer in die Mitte des Bildschirms positioniert. Die Länge des Balkens, der die gewählte Menüzeile hervorhebt, entspricht immer der gesamten Fensterbreite.

Festlegen der Position des Menüauswahlbalkens Damit lassen sich mehrstufige Menüsysteme realisieren, bei denen Sie nach der Rückkehr in ein bereits besuchtes Menü den Auswahlbalken an der früheren Position wiederfinden.

.menu (0,0,'Zeilennummer') "Überschrift"

Spaltendarstellung Vor der ersten Menüzeile kann eine Zeile mit Angaben zur Ausrichtung der Spalten stehen. Erlaubt sind Plus und Minus mit Leerzeichen dazwischen.

- **+** kennzeichnet den Beginn einer rechtsbündigen Spalte
- **-** kennzeichnet den Beginn einer linksbündigen Spalte.

```
..!interpreter english noabbrev englvar
.output terminal
.input BILD.F
..
.menu
+      -      +
Number Name      Points
10  Meier      1253  >>Meier
111 Schulze      999  >>Schulze
2000 Schmidt      17  >>Schmidt
..
menu selection: 'G|noSelectedMenuOption' -> 'G|menuText'
```

Verwandte Themen

.box
.fehler
.hinweis

Eingabe von Dateien, Temporärdateien

.EDITIERE / .EDITING - Systemaufrufe

.EDITIERE	.EDITING
deutsch	englisch
.EDITIERE [-READONLY -WRITE] [-RIGHT 'n'] [-HTML [-CONTENTSWIDTH 'w']] 'Datei'	.EDITING [-READONLY -WRITE] [-RIGHT 'n'] [-HTML [-CONTENTSWIDTH 'w']] 'file'

Das Kommando ruft den Texteditor für die angegebene Datei auf.

Mit **-READONLY** wird die Datei im Nur-Lesen-Modus angezeigt, mit **-WRITE** kann die Datei geschrieben werden.

-RIGHT 'n' setzt in der GUI den rechten Rand in Spalte 'n'.

-HTML öffnet in der GUI den [HTML-Freitexteditor](#).

-CONTENTSWIDTH 'w' setzt in der GUI die Größe des [HTML-Freitexteditors](#) so, dass die für den Inhalt verfügbare Breite 'w' Millimeter beträgt. Der Standardwert ist 180; dies entspricht der Breite einer DIN A 4 Seite im Hochformat abzüglich links und recht jeweils 3 Zentimeter Rand.

Die Variable **G|success** wird gesetzt und zeigt an, ob der Editoraufruf erfolgreich war.

```
.type text xtfiler
.formula U|xtfiler = "XYZ"
.editing 'U|xtfiler'
```

```
.type text xtfiler
.formula U|xtfiler = "XYZ"
.editing -right 100 'U|xtfiler'
```

```
.type text xtfiler
.formula U|xtfiler = "XYZ"
.editing -html -contentwidth 120 'U|xtfiler'
```

```
.type text xtmod
```

```
.type text xtfiler
.type text xtmargin
.type bool xbwriteprotect
.formula U|xtfiler = "XYZ"
.formula U|xbwriteprotect = false
.formula U|xtmod = "-readonly" ? U|xbwriteprotect = G|true
.formula U|xtmod = "-write" ? U|xbwriteprotect = G|false
.formula U|xtmargin = 30
.editing 'U|xtmod' -right 'U|xtmargin' 'U|xtfiler'
```

.HOLE VON / .SELECT FROM - Lesen aus einer Datei

.HOLE VON	.SELECT FROM
deutsch	englisch
.HOLE VON .HOLE VON 'Dateiname' 'Freitext' 'Variablenliste' [^]	.SELECT FROM .SELECT FROM 'file name' 'free text' 'list of variables' [^]
.hole von 'Dateiname' 'Variablenliste' [^] liest eine Zeile aus einer FO-Datei (einschließlich Freitext) in die Variablen der Variablenliste ein. Wie bei .zuweisen und anders als bei .formel findet eine Wertebereichsprüfung statt. Die Variablenliste besteht aus Variablenname Trennzeichen [Variablenname Trennzeichen ...]. Trennzeichen sind ASCII-Sonderzeichen, z.B. ; (Semikolon). Die Trennzeichen dürfen innerhalb des Befehls variieren. Erlaubte Trennzeichen Die Angabe des Hochzeichens (^) erzwingt das Lesen bis zum Zeilenende. .hole von (ohne weitere Argumente) schließt Dateien, die mit .hole von 'Dateiname' 'Variablenliste' geöffnet wurden.	

.hole von 'Dateiname' 'Variablenliste' liest eine Zeile aus einer Textdatei und weist deren Bestandteile den angegebenen Variablen zu. Jedes nachfolgende **.hole von** liest eine weitere Zeile.

- Überschüssige Bestandteile einer Zeile werden ignoriert.
- Überschüssige Variablen im Kommando bewirken, dass die eingelesene Zeile zu kurz ist, um alle Variablen zu bedienen; es erfolgt eine Fehlermeldung.



Zeilen ohne Einträge werden übersprungen, wenn nicht mit Hochzeichen (^) das Lesen bis zum Zeilenende erzwungen wird.

In der Datei **NUMERALS** stehen folgende Einträge:

1 2 3 4 5

```
6 7 8
9 1
```

Zunächst werden die Leerzeichen als Trennzeichen zwischen den Einträgen betrachtet. Mit den folgenden Kommandos wird gelesen:

```
.type text xta xtb xtc xtd xte xtf xtg xth xti
.select from NUMERALS U|xta U|xtb
.select from NUMERALS U|xtc U|xtd U|xte
.select from NUMERALS U|xtf U|xtg U|xth
.select from NUMERALS U|xti
```

Fehlermeldung. Es gilt

U|xta=1, U|xtb=2

U|xtc=6, U|xtd=7, U|xte=8

U|xtf=9, U|xtg=1, U|xth findet keinen Wert vor und bewirkt die Fehlermeldung.

Trennzeichen

Meistens ist das Leerzeichen zur Trennung von Einträgen ungeeignet, weil die Einträge selbst Leerzeichen enthalten.

Die Datei **EMPLOYEEES** hat folgenden Inhalt:

```
Otto Strasser ; 123.45 ; 12-08-82
Karl Minze ; 12.56 ; 4-12-81
```

Das Kommando **.hole von EMPLOYEEES** soll Name, Betrag und Termin lesen. Als unerwünschte Ausgabe erhalten Sie:

Name = **Otto**

Betrag = **Strasser**

Termin = **;** usw.

Sinnvoller wäre es, das Semikolon als Trennzeichen zu interpretieren. Dazu können Sie in der Variablenliste dieselben Trennzeichen angeben, die in der Datei erwartet werden:

.hole von EMPLOYEEES Name;Betrag;Termin Damit erzielen Sie das gewünschte Ergebnis. Das Verfahren funktioniert aber nur, wenn keiner der Einträge das gewählte Trennzeichen enthält. Deshalb können Trennzeichen frei gewählt werden und auch innerhalb der Zeile wechseln. Wichtig ist nur, dass die Einträge und Trennzeichen in der gelesenen Datei mit der Variablenliste korrespondieren. Denkbar ist

.hole von EMPLOYEEES Name - Betrag ; Termin

Wenn die FO das geforderte Trennzeichen in der gelesenen Zeile nicht findet, erhält die Variable den

gesamten Zeilenrest als Inhalt. Dies kann nützlich sein, um beliebige Textzeilen zu lesen.

Ist unklar, ob eine Zeile nicht doch das gewählte Trennzeichen enthält, lesen Sie am besten keine einzelnen Einträge, sondern ganze Zeilen. Mit Hilfe des Hochzeichens (^) kann das Lesen bis zum Zeilenende erzwungen werden.

Beispiel: Lesen bis zum Zeilenende

Die gesamte nächste Zeile der Datei **EMPLOYEES** wird gelesen und der Variablen **U|xtline** zugewiesen.

```
.type text xtline  
.select from EMPLOYEES U|xtline ^  
'U|xtline'
```

Da hier nicht einzelne Einträge, sondern eine gesamte Zeile gelesen wird, erhält **xtline** auf jeden Fall den Inhalt der nächsten Zeile, auch wenn diese nur aus Leerzeichen besteht oder gar keine Zeichen enthält.

Die FO kann somit nicht nur Textdateien ausgeben sondern auch Textdateien einlesen. Näheres hierzu: [Textausgabe und Textformatierung](#)



Holen von Datumsangaben

Die Datumsdarstellung in den Masken ist abhängig von der [Bediensprache](#). So bedeutet z.B. **01.02.10** in deutscher Bediensprache 1. Februar 2010, in ungarischer jedoch 3. Februar 2010. Damit würde das Holen der Datumsangaben aus einer Datei die FO behindern. Deshalb ist für die Angabe von Datum und Woche die Kurzform des sortierfähigen Datums nach ISO 8601 **jjjjmmtt** zulässig. Termine in diesem Format sind sortierfähig. Siehe auch unter [.art - Variablenarten](#) und [Variablen und Operationen](#).

Offene Eingabedateien schließen

Offene Eingabedateien werden geschlossen durch:

- **.hole von** mit einem neuen Dateinamen
- **.hole von**
- Erreichen des EOF in der Datei
- FOP-Ende
- Fehler

Das Schließen ist immer ratsam und in folgendem Fall sogar entscheidend: Freitexte unterschiedlicher Datenobjekte können verlinkt sein, d.h. den Freitext gibt es nur einmal, er ist aber von verschiedenen Datensätzen aus zugänglich. Daher kann es vorkommen, dass an zwei Datensätzen eines Datenbereichs derselbe Freitext hängt. Ohne Schließen mit **.hole von** würde dann das zweite **.hole von** auf die schon geöffnete Datei wirken und statt der gewünschten ersten die zweite Zeile liefern.

Sie möchten jeweils die erste Zeile des Freitextes **f_{text1}** vom Auftrag **BKUNDE** und vom Lieferscheins **LKUNDE** lesen. Die Freitexte sind gelinkt.

```
.type text xtread1
.select SALES "BKUNDE"
.select from 'H|ftext1' U|xtread1^
'U|xtread1'
```

liest Zeile 1 aus dem Freitext.

```
.type text xtread2
.select SALES "LKUNDE"
.select from 'H|ftext1' U|xtread2^
'U|xtread2'
```

liest Zeile 2 aus dem Freitext, was nicht beabsichtigt ist.
Richtig ist:

```
.type text xtread1
.type text xtread2
.select SALES "BKUNDE"
.select from 'H|ftext1' U|xtread1^
.select from
.select SALES "LKUNDE"
.select from 'H|ftext1' U|xtread2^
```

Lesen Sie hierzu auch Kapitel [Vergleich von .datei und .hole von](#).

.DATEI / .FILE - Lesen aus Dateien

.DATEI	.FILE
deutsch	englisch
.DATEI	.FILE
.DATEI -OEFFNEN <Dateivariablen> 'Dateiname'	.FILE -OPEN 'descriptor name' 'file name'
.DATEI -TEST -OEFFNEN <Dateivariablen> 'Dateiname'	.FILE -TEST -OPEN 'descriptor name' 'file name'
.DATEI -LESEN <Dateivariablen> 'Variablenliste'	.FILE -READ 'descriptor name' 'list of variables'
.DATEI -SCHLIESSEN <Dateivariablen>	.FILE -CLOSE 'descriptor name'
.DATEI -TEST 'Dateiname'	.FILE -TEST 'file name'
.DATEI -TEMPNAME Variable	.FILE -TEMPNAME variable
.DATEI [-FOP]	.FILE [-FOP]
.DATEI [-MASKE]	.FILE [-SCREEN]
.DATEI [-FREIGEBEN 'Variable']	.FILE [-RELEASE 'variable']
.DATEI [-UEBERNEHMEN "Text"]	.FILE [-ACCEPT "text"]
.DATEI [-SUFFIX 'Text']	.FILE [-SUFFIX 'text']

.DATEI	.FILE
<code>.datei -oeffnen <Dateivariablen> 'Dateiname'</code> setzt die Dateivariablen, mit der im Folgenden die Datei referenziert wird. <code>.datei -lesen <Dateivariablen> 'Variablenliste'</code> liest in der zuvor geöffneten Datei mit dem angegebenen Deskriptor. Erlaubte Trennzeichen <code>.datei -schliessen <Dateivariablen></code> schließt die Datei mit dem angegebenen Deskriptor. <code>.datei -test 'Dateiname'</code> prüft, ob die angegebene Datei mit <code>.datei -oeffnen ...</code> geöffnet werden könnte und setzt <code>G mehr</code> entsprechend. <code>.datei -tempname Variable</code> ist eine Grundfunktion. Sie definiert einen für die Dauer eines FO-Programmlaufs eindeutigen Temporärdateinamen und sorgt dafür, dass die Datei bei Programmende automatisch gelöscht wird. Die Variable ist vom Typ <code>text</code>. Bei <code>-fop</code> und im Standardfall gilt der Dateiname nur für einen FO-Programmlauf. Danach wird die Datei gelöscht. Bei <code>-maske</code> gilt der Dateiname für mehrere EFOPs, vom Maskeneintritts-FOP bis zum Ende- oder Abbruch-FOP. Bei <code>-freigeben</code> wird die Datei samt Verzeichnis so umbenannt, dass ihr Name an asynchron gestartete Programme übergeben werden kann. Die Datei wird dann nicht mehr gelöscht. Bei <code>-freigeben</code> kann eine zuvor durch <code>-freigeben</code> umbenannte Datei in den Temporärbereich übernommen werden. Mit <code>-suffix</code> führt mehrfaches Aufrufen im selben FOP mit derselben Variable bzw. demselben Text zum selben Dateinamen.	

Das Kommando ermöglicht das Lesen aus einer FO-Datei. Während bei `.hole von` immer nur von einer Datei gelesen werden kann, ist es mit dem Befehl `.datei` möglich, mehrere Dateien sowie aus einer Datei mehrere Stellen gleichzeitig zu lesen. Auch eine reine Existenzprüfung ist möglich.

Lesen Sie hierzu auch Kapitel [Vergleich von .datei und .hole von](#).

Zu `.datei -tempname 'Textvariable'` und `.datei [-uebernehmen "Text"]` lesen Sie auch das Kapitel [Temporärdateien in FOPs](#).

Der typische Ablauf enthält die Schritte Öffnen, Lesen und Schließen.

Die Befehle lauten wie folgt:

Öffnen

```
.datei -oeffnen <Dateivariablen> 'Dateiname'
.datei -test -oeffnen <Dateivariablen> 'Dateiname'
```

Beide Befehle öffnen die Datei mit dem angegebenen Namen zusätzlich zu den bereits offenen Dateien.

Dateivariable ist der Name einer FO-Variablen vom Typ INT. Wenn sie noch nicht deklariert ist, wird sie automatisch angelegt (außer bei der **Interpreter-Option DECLARATION**). Bei den späteren Befehlen zum Lesen und Schließen wird zur Kennzeichnung der Datei immer diese Variable angegeben und nicht mehr der Dateiname.



Es können mit diesem Befehl bis zu 9 Dateien gleichzeitig geöffnet sein. Wenn Sie mehr Dateien öffnen, erfolgt ein Fehlerabbruch.

Option -test Bei **-test** wird nicht gelesen. Existiert die angegebene Datei nicht oder fehlen die Leserechte, so liegt ein Fehler vor und **G|mehr** wird auf **G|false** gesetzt. Die Fehlerbehandlung ist davon abhängig, ob die Option **-test** angegeben wurde oder nicht:

ohne -test	Es ist keine eigene Fehlerbehandlung notwendig, aber das Programm bricht (außer im Direktmodus) ab. Dies entspricht dem Verhalten von .hole von ; siehe auch unter Eingabe von Dateien , Temporärdateien . Das Programm gibt eine spezifische Meldung zur Ursache aus (z.B. "Fehlende Leserechte" oder "Datei nicht vorhanden"). Diese Variante ist geeignet, wenn die Datei unbedingt existieren muss und keine andere Fehlerbehandlung außer Abbruch möglich ist.
mit -test	Das Programm gibt keine Meldung aus und läuft weiter. Die spezifische Fehlerursache muss, falls von Interesse, mit .system ermittelt werden. Die Variable G mehr ist gesetzt und muss unbedingt abgefragt werden. Andernfalls liegt ein Programmierfehler vor. Diese Variante ist geeignet, wenn die Datei nicht zwangsläufig vorhanden sein muss oder wenn eine Fehlerbehebung möglich ist. Liegt kein Fehler vor, so wird nach dem Öffnen auf die Datei nicht mehr über ihren Dateinamen, sondern nur noch über die FO-Variable <Dateivariable> zugegriffen. Sie enthält nach dem Befehl eine kleine Zahl, den sog. Deskriptor der Datei. Eine weitere Datei (ohne Deskriptor) steht weiterhin mit dem Befehl .hole von 'Dateiname' ... zur Verfügung.

Lesen

.datei -lesen <Dateivariable> 'Variablenliste' Erstes oder weiteres Lesen in der zuvor geöffneten Datei mit dem angegebenen Deskriptor, wie bei **.hole von 'Dateiname' 'Variablenliste'**. **G|mehr** wird am Dateiende gleich **G|false**. Der Versuch, in einer noch nicht geöffneten Datei zu lesen oder nach dem Erreichen des Dateiendes weiterzulesen, führt zum Abbruch.

Schließen

.datei -schliessen <Dateivariablen> Schließen der Datei mit dem angegebenen Deskriptor. Die Option kann immer, muss aber in den meisten Fällen nicht angegeben werden. Wenn das Dateiende erreicht ist, wird die Datei automatisch geschlossen. Zudem werden offene Dateien am FOP-Ende ohne Fehlermeldung für den Benutzer geschlossen. Der Versuch, in einer bereits geschlossenen Datei zu lesen, führt zum Abbruch. Die Variable <Dateivariablen> wird auf den ungültigen Wert 0 gesetzt, so dass mit ihr zu einem späteren Zeitpunkt nicht mehr versehentlich gelesen werden kann. Das mehrmalige Schließen einer Datei ist unzulässig und führt zum Abbruch.

Sie sollten die Programmstruktur so festlegen, dass zu erkennen ist, welche Dateien offen sind und welche nicht. Darüber hinaus sollte eine Datei im selben Unterprogramm geschlossen werden, in dem sie geöffnet wurde. Sollten Sie ausnahmsweise nicht wissen, ob eine Datei durch ein Unterprogramm geschlossen wurde oder nicht, hilft die Anweisung

.datei -schliessen <Dateivariablen> ? <Dateivariablen> > 0

Aufgabe: Je eine Zahl von den beiden gleich langen Dateien **NUMBERS1** und **NUMBERS2** lesen und zusammen ausgeben.

```
.type int xi1 xi2 xidescriptor1 xidescriptor2
.file -open U|xidescriptor1 "NUMBERS1"
.file -open U|xidescriptor2 "NUMBERS2"
!WHILE
.file -read U|xidescriptor1 U|xi1
.file -read U|xidescriptor2 U|xi2
.end ? G|success = false
'U|xi1' 'U|xi2'
.continue WHILE
```

Ist die Datei **NUMBERS1** im Gegensatz zur Vorgabe kleiner als **NUMBERS2**, beendet sich das Programm in der ersten Zeile mit **-lesen**. Es erscheint die Fehlermeldung "Dateiende bereits erreicht".

Existiert die Datei **NUMBERS1** oder **NUMBERS2** nicht, so bricht das Programm ebenfalls ab. Dies kann bei Bedarf mit der Option **-test** vermieden werden.



An die Variable <Dateivariablen> sollten Sie niemals Werte zuweisen. Der Befehl **.datei** setzt die Variable immer auf einen passenden Wert: Beim erfolgreichen Öffnen ist der Wert ungleich 0, beim Schließen, sei es explizit oder durch Erreichen des Dateiendes beim Lesen, ist er 0.

Existenz einer Datei ermitteln

.datei -test 'Dateiname' ermittelt, ob eine Datei vorhanden ist. Dabei wird die Variable **G|mehr** gesetzt.

```
.file -test "AN"
?File AN exists ? G|success = G|true
?File AN does not exist ? G|success = G|false
.file -test "vk/AN"
?File vk/AN exists ? G|success = G|true
?File vk/AN does not exist ? G|success = G|false
```

Die Datei wird in dem angegebenen Verzeichnis gesucht. Fehlt diese Angabe, wird sie in allen zulässigen Arbeitsverzeichnissen gesucht.

Temporäre Datei erzeugen

Beispiel: FO

1. Temporäre Datei erzeugen.
2. Name der temporären Datei ausgeben.
3. Temporäre Datei zum Anfügen von Daten öffnen.
4. Kundendatensätze selektieren und in temporäre Datei schreiben.
5. Temporäre Datei schließen.
6. Temporäre Datei im Editor öffnen.
7. Bei **.end** wird die temporäre Datei automatisch gelöscht.

```
..!interpreter english declaration noabbrev
..
.type text tmpfilename ? F|defined(U|tmpfilename) = G|false
..
.file -tempname U|tmpfilename
.println TempFile: 'U|tmpfilename'
.output addedto 'U|tmpfilename'
.input DATEI.F
..
.select customer "$,nummer=70000!70100"
.continue CHECK
!CHECK
.continue END ? G|mehr = G|false
.println 'H|nummer' - 'H|such' - 'H|plz'
.select continue
.continue CHECK
..
!END
.output
.editing 'U|tmpfilename'
.println continue program
.end
```

Beispiel: FO2

1. Temporäre Datei erzeugen.
2. Name der temporären Datei ausgeben.
3. Temporäre Datei zum Anfügen von Daten öffnen.
4. Kundendatensätze selektieren und in temporäre Datei schreiben.
5. Temporäre Datei schließen.
6. Temporäre Datei im Editor öffnen.
7. Bei **.end** wird die temporäre Datei automatisch gelöscht.

```
..!interpreter english declaration noabbrev englvar
..
..<VAR folded>
.type text tmpfilename ? F|defined(U|tmpfilename) = G|false
..</VAR>
def main() {
.file -tempname U|tmpfilename
.println TempFile: 'U|tmpfilename'
.call initTempFileToWrite()
.select customer "$,idno=70000!70100;@language=en"
.call writeRecordsIntoTempFile()
..
.output
.editing 'U|tmpfilename'
.println continue program
.end
}
def writeRecordsIntoTempFile( ) {
while(G|success = G|true){
.println 'H|idno' - 'H|swd' - 'H|zipCode'
.select continue
}
}
def initTempFileToWrite( ) {
.output addedto 'U|tmpfilename'
.input DATEI.F
}
```

Export von Freitexten

Dazu steht das Kommando **.datei -eingabe** zur Verfügung.

Vergleich von .datei und .hole von

Im Gegensatz zu **.hole von** (und übrigens auch zu **.lade 0**) kann **.datei** problemlos in Unterprogrammen eingesetzt werden. Es beeinflusst das Hauptprogramm nicht, selbst wenn beide

Programme mit `.datei` arbeiten. Sind im Hauptprogramm bereits Dateien geöffnet, so erhält das Unterprogramm die nächsten freien Deskriptoren zugewiesen. Dieser Vorgang ist für den Programmierer transparent.

Auf diese Weise kann das Unterprogramm eine Datei einlesen, die auch das Hauptprogramm gerade geöffnet hat, da mit Deskriptorvariable und nicht mit Dateiname gearbeitet wird. In allen Fällen sollte darauf geachtet werden, dass nicht mehr benötigte Dateien geschlossen werden, wenn das Dateieinde erreicht ist. Dadurch werden zum einen Systemressourcen geschont, zum anderen könnte die Höchstgrenze von neun offenen Dateien überschritten werden, z.B. wenn ein Unterprogramm immer wieder in einer Schleife durchlaufen wird.

Das Öffnen von PC-Dateien ist mit `.datei` nicht möglich. Ein versehentliches Lesen über das Dateieinde hinaus wird mit `.datei` unterbunden, selbst wenn vergessen wird, `G|mehr` abzufragen. Bei `.hole von` wird erneut gestartet. Dies kann nicht mehr geändert werden, da es wahrscheinlich schon als Feature genutzt wird.

Temporärdateien in FOPs

Temporäre Dateien werden in FO-Programmen für die unterschiedlichsten Zwecke benötigt. Meistens dienen sie dazu, Zwischenergebnisse abzuspeichern und sie später wieder aufzugreifen. Auch die Übergabe an andere Prozesse wird über Temporärdateien abgewickelt.

Speziell für Temporärdateien steht der Arbeitsbereich `rmtmp` zur Verfügung. Alle dort abgelegten Dateien werden nach einer gewissen Zeit vor der `Datensicherung` gelöscht, sofern Sie die Datensicherung des Lieferumfangs nutzen oder zumindest das Skript `raeum_tmp.sh` verwenden. Trotzdem ist es gut, Temporärdateien möglichst bald nach Gebrauch wieder zu löschen.

Das Verzeichnis `rmtmp` kann auch dann verwendet werden, wenn es nicht in den eigenen Arbeitsbereichen eingetragen ist. Auch kann es Unterverzeichnisse besitzen, auf die jeder Benutzer Zugriff hat.

```
.output new "rmtmp/abc/XXX"
```

Das Kommando funktioniert, wenn es die Verzeichnisse `rmtmp` und `rmtmp/abc` gibt.

Wenn eine Datei `rmtmp/INPUT` existiert, kann sie mit nachfolgendem Kommando geöffnet werden.

```
.select from "rmtmp/INPUT"
```

```
.input "dir/FILENAME"
```

Die Datei `dir/FILENAME` wird aber nur gefunden, wenn der Arbeitsbereich `dir` für den Benutzer eingetragen ist.

Wenn Dateien in FOPs nur für einen kurzen Zeitraum benötigt werden, empfiehlt sich das Verzeichnis `rmtmp`. Möglich, aber etwas unstrukturierter ist die Namensgebung `X*.TMP`. Bei jeder Abas Datensicherung

werden mehrere Tage alte Dateien dieses Namens gelöscht.

Das Kommando `.datei -tempname` hat den Zweck, Dateinamen für Temporärdateien zu bilden und dafür zu sorgen, dass sie auch automatisch zum richtigen Zeitpunkt wieder gelöscht werden. Da die gewünschte Lebensdauer von Temporärdateien aber nicht in allen Situationen gleich ist, gibt es weitere Optionen um diesen Unterschieden gerecht zu werden.

- Der einfachste Fall sind Temporärdateien, die für die Dauer eines FO-Aufrufes gebraucht werden. Sie werden im selben Programm erstellt, verwendet und zum Programmende auch wieder gelöscht.
- Der zweite Fall sind maskenbezogene Temporärdateien. Sie werden so lange gebraucht, wie das Objekt in der Maske geladen ist.
- Der dritte Fall sind Temporärdateien, die in einem Prozess erzeugt, aber in einem anderen, unabhängig laufenden, verarbeitet werden. Beispiele dafür sind die Parameterübergabe an ein Spoolerskript, einen Lader oder ein mit `.kommando -parallel` geöffnetes zweites Fenster, oder an ein FO-Programm, das per `batchlg.sh` im Hintergrund gestartet wird. In dieser Situation darf der erzeugende Prozess die Dateien gar nicht löschen - das ist Aufgabe des verarbeitenden Prozesses. Die Prozesse laufen ja unabhängig voneinander, und es ist offen, welcher der beiden früher zu Ende kommt.

Für die drei Situationen hat das Kommando `.datei -tempname` spezielle Optionen.

Grundfunktion

`.datei -tempname <Variablenname>`

In der einfachsten Form hat das Kommando `.datei -tempname` folgenden Aufruf:

In diesem Beispiel wird ein Temporärdateiname erzeugt. Die Variable `U|xtvarname` kann alternativ auch mit der Art `DVM2` deklariert werden.

```
.type text xtvarname
.file -tempname U|xtvarname
file name: 'U|xtvarname'
```

Die Wirkung des Kommandos ist:

- Die angegebene Variable (im Beispiel `U|xtvarname`) enthält einen für FO gültigen Temporärdateinamen in einem Verzeichnis unterhalb von `rmtmp` (das Kommando legt selbst keine Dateien an).
- Nach dem ersten Aufruf des Kommandos existiert dieses Verzeichnis und ist leer und beschreibbar.
- Ein zweiter Aufruf desselben Kommandos im selben Programmablauf liefert stets denselben Namen.
- Der Name endet wie der Variablenname.
- Nach dem Ende des FO-Programmes werden alle in diesem Verzeichnis liegenden Dateien gelöscht. (Beachten Sie dazu den Abschnitt [Entwickeln und Testen](#))

Anschließend kann eine Temporärdatei mit dem in `"U|xtvarname"` stehenden Namen angelegt werden.

Es können aber auch weitere Dateien im selben Verzeichnis angelegt werden, so können z.B. durch Anhängen eines weiteren Textes an 'U|xtvarname' weitere Temporärdateinamen gebildet werden. Die Dateien müssen nicht per FO angelegt werden, auch ein Anlegen per Shellskript, das aus der FO gestartet wird, ist zulässig.

Weitere Optionen

- **-suffix**: Steuert die Bildung der Temporärdateinamen.
- **-fop**: Optional, die Temporärdateinamen gelten für den Lauf eines FO-Programmes.
- **-maske**: Die Temporärdateinamen gelten bis zum Ende des Maskenende-FOPs.
- **-freigeben**: Die Temporärdateien werden freigegeben, das heißt umbenannt und nicht mehr gelöscht.
- **-freigeben**: Übernimmt vorher freigegebene Temporärdateien in den Bereich der eigenen Temporärdateien. Analog zum **-freigeben** werden auch hier die Dateien nur umbenannt.

Gezieltes Benennen von Temporärdateien

-suffix

Ohne Angabe von **-suffix`text`** wird der Temporärdateiname aus dem Variablennamen abgeleitet. Der nach der Option **-suffix** angegebene Text erlaubt es gezielt Dateinamen zu bilden. Der Parameter darf auch Unterverzeichnisse enthalten, sie werden dann automatisch angelegt. Der Wert von 'U|xtvarname' endet mit "ABC" als Dateinamen.

```
.type text xtvarname  
.file -tempname U|xtvarname -SUFFIX ABC  
file name: 'U|xtvarname'
```

Der Wert von 'U|xvarname' endet wieder mit "ABC" als Dateinamen, die Datei liegt im Verzeichnis xyz unterhalb des Temporärverzeichnisses **imtmp**.

```
.type text xtvarname  
.file -tempname U|xtvarname -SUFFIX xyz/ABC  
file name: 'U|xtvarname'
```

Steuerung der Lebensdauer

-maske, -maske

Zwei Optionen steuern die Lebensdauer von Temporärdateien:

- **-fop**: Standardwert, Temporärdateien existieren für die Laufzeit eines FO-Programmes. Das Verzeichnis für diesen Dateinamen wird beim ersten Aufruf in einem FOP angelegt und ist zu diesem Zeitpunkt garantiert leer. Das Verzeichnis wird nach Ende des FOPs mitsamt allen darin liegenden Dateien gelöscht.

- **-maske**: Die Temporärdateien existieren bis zum Ende des Maskenende-FOPs. Die Temporärdateien liegen bei **-maske** in einem anderen Verzeichnis als bei **-fop**. Diese Verzeichnis wird beim ersten Aufruf in einem EFOP angelegt. Erst nach Ende des Maskenende-FOPs wird das Verzeichnis gelöscht. Im Kommando-FOP ist die Lebensdauer identisch zur Option **-fop**, die Namen werden aber ebenfalls anders gebildet.

Im folgenden Beispiel sind die beiden Variablen im gleichen Programmlauf immer mit demselben Wert gefüllt:

```
.type text xtvar1 xtvar2
.file -tempname U|xtvar1 -suffix "TMP"
.file -tempname U|xtvar2 -suffix "TMP"
U|xtvar1: 'U|xtvar1'
U|xtvar2: 'U|xtvar2'
```

Display:

```
U|xtvar1: rmtmp/temp.fop.32/x.0001/TMP
U|xtvar2: rmtmp/temp.fop.32/x.0001/TMP
```

In dem folgenden Beispiel enthalten die beiden Variablen nie denselben Wert.

```
.type text xtvar1 xtvar2
.file -fop -tempname U|xtvar1 -suffix "TMP"
.file -screen -tempname U|xtvar2 -suffix "TMP"
U|xtvar1: 'U|xtvar1'
U|xtvar2: 'U|xtvar2'
```

Display:

```
U|xvar1: rmtmp/temp.fop.32/x.0001/TMP
U|xvar2: rmtmp/temp.maske.32/x.0002/TMP
```

Es sollen interpretierte Kopien von Freitexten in der Tabelle eines Infosystems geladen und ausgedruckt werden. Dazu wird ein Feld **M|pftext** der Art **DVMR2** in der Tabelle eines Infosystems verwendet, gemeinsam mit der Option **-maske**:

```
.input DATEI.F
.file -screen -tempname M|pftext
.output new 'M|pftext'
.input 'H|pftext'
.output terminal
```

Übergabe an asynchron laufende Prozesse

-uebernehmen, -uebernehmen

Wenn ein Programm Temporärdateien erzeugt, die ein anderes asynchron laufendes Programm verarbeiten soll, dann muss besondere Sorgfalt bei der Übergabe und beim Löschen der Dateien walten, um schwierig zu analysierende Fehler bei der Synchronisation zu vermeiden.

Der korrekte Ablauf ist folgender:

- Der Erzeuger erstellt die Temporärdateien.
- Der Erzeuger signalisiert dem Empfänger den Auftrag, samt dem Namen der Temporärdateien.
- Der Empfänger erhält den Auftrag und verarbeitet die Dateien.
- Der Empfänger löscht die Dateien.

Typische Fehler in einem solchen Ablauf sind:

- Der Erzeuger schreibt nach dem Absenden des Auftrages noch in die Temporärdateien, weil er sie zum Beispiel noch geöffnet hat.
- Der Erzeuger löscht die Temporärdateien, statt das dem Empfänger zu überlassen.

Der korrekte Ablauf wird folgendermaßen unterstützt:

- Der Erzeuger erstellt die Temporärdateien mit Hilfe von Dateinamen, die mittels `.datei -fop -tempname U|xtmp` gebildet wurden.
- Der Erzeuger gibt die Temporärdateien frei mit Hilfe des Kommandos `.datei -freigeben -tempname U|xtmp`.
- Der Erzeuger signalisiert dem Empfänger den Auftrag und den freigegebenen Temporärdateinamen.
- Der Empfänger - vorausgesetzt er ist ebenfalls als FOP implementiert - übernimmt die Temporärdateien mit `.datei -uebernehmen 'temp' -tempname U|xtmp` in seinen Temporärbereich.
- Der Empfänger verarbeitet die Temporärdateien.
- Der Empfänger - vorausgesetzt er ist ebenfalls als FOP implementiert - löscht die Daten zum Programmende automatisch, weil sie in seinem Temporärbereich liegen.

Die Funktion der `-freigeben`-Option ist:

- Es wird der normale Temporärdateinamen gebildet.
- Es wird ein neuer Temporärdateinamen gebildet, der nicht im Bereich der von diesem Prozess gelöschten Verzeichnisse liegt.
- Alle Dateien die in dem durch den normalen Temporärdateinamen gebildeten Verzeichnis liegen, werden in das Verzeichnis den neuen Temporärdateinamens verschoben.
- Achtung: Bei `-freigeben` wird das Verzeichnis umbenannt, es darf also nur einmal aufgerufen werden, die freigegebene Datei ist danach bereits umbenannt.

Die Funktion der UEBERNEHMEN-Option ist:

- Der beim Übernehmen angegebene Temporärpfad wird reserviert und in der Variablen bei `-tempname`

gespeichert.

- Achtung: Bei der Option **-suffix** wird das Verzeichnis umbenannt, sie darf also nur einmal aufgerufen werden. Der Name des Zielverzeichnisses wird aus dem übergebenen Verzeichnis gebildet, ein **-suffix** ist hier nicht zulässig.

Dadurch, dass ein komplettes Verzeichnis umbenannt wird, können mehrere Temporärdateien "auf einen Schlag" übergeben werden. Sinnvollerweise legt man die Übergabedateien in ein separates Unterverzeichnis, um sie von anderen nicht zu übergebenden Dateien zu trennen.

auf Erzeugerseite:

```
.type DVM2 xdvm2file      ? F|defined(U|xdvm2file) = false
.file -tempname U|xdvm2file -suffix "verz/ABC"
1.Temporary file: 'U|xdvm2file'
.output new 'U|xdvm2file'
.file -release -tempname U|xdvm2file -suffix "verz/ABC"
.output terminal
2.Temporary file: 'U|xdvm2file'
```

auf Empfängerseite:

```
.type DVM2 xdvm2          ? F|defined(U|xdvm2) = false
.type text xtparam        ? F|defined(U|xtparam) = false
.file -tempname U|xdvm2 -suffix "verz/ABC"
.output new 'U|xdvm2'
.file -release -tempname U|xdvm2 -suffix "verz/ABC"
.assign U|xtparam = U|xdvm2
.file -accept 'U|xtparam' -tempname U|xdvm2 -suffix "verz/ABC"
.output terminal
accept: 'U|xtparam'
```

Verwenden Sie für den Temporärdateinamen bei der Option **-suffix** und **-suffix** immer die Option **-suffix** mit einem Unterverzeichnis. Dann können Sie das Programm später leichter zur Übergabe mehrere Dateien erweitern.



```
.type DVM2 xdvm2file1 xdvm2file2 xdvm2filelocal
.file -tempname U|xdvm2file1 -suffix "by/FILE"
.file -tempname U|xdvm2file2 -suffix "by/FILE.2"
.output new 'U|xdvm2file1'
.output new 'U|xdvm2file2'
.. Next file will be created local, not in subdir 'by'
.file -tempname U|xdvm2filelocal -suffix "FILE"
.output new 'U|xdvm2filelocal'
.. 3 file will be created
```

```
.. only 2 files in subdir 'by' will be released
.file -release -tempname U|xdvm2file1 -suffix "by/FILE"
.output terminal
file release: 'U|xdvm2file1'
```

Die dritte Datei, deren Namen in `U|xdvm2filelocal` steht, wird nicht mit übergeben. Der Empfänger bekommt nach wie vor nur einen Namen zugeschickt. Der Name der zweiten Datei muss aus dem Namen, der beim `-uebernehmen` gebildet wird, abgeleitet werden:

```
.. continue previous example
.type text xtnew
.assign U|xtnew = U|xdvm2file1
.file -accept 'U|xtnew' -tempname U|xdvm2file1 -suffix "ueber/FILE"
.output terminal
U|xdvm2file1 = 'U|xdvm2file1'
```

Entwickeln und Testen von FO-Programmen mit Temporärdateien

Treten im Zusammenhang mit FO-Programmen und Temporärdateien Probleme auf, so ist es häufig interessant, die Datei auch nach Programmende noch studieren zu können. Dazu ist interessant zu wissen wie genau die Namen gebildet werden, und wann die Dateien tatsächlich gelöscht werden:

- Das Verzeichnis unterhalb `rmtmp` wird nicht sofort bei Ende des FO-Programmes, sondern erst bei Ende des aufrufenden Prozesses tatsächlich gelöscht.
- Durch Vergabe neuer Namen beim nächsten FO-Programmlauf, bzw. bei der nächsten Maske ist sichergestellt, dass dieses verzögerte Löschen keine Probleme macht.
- Unterbleibt das Löschen, weil der Anwendungsprozess wegen äußerer Ursachen nicht normal terminiert, wird das Verzeichnis durch den nächsten Aufrufer mit derselben Nummer gelöscht. Da die Aufrufernnummern wieder verwendet werden, bleibt so nichts länger liegen.
- Die Verzeichnisse werden so benannt, dass ihre Namen die Reihenfolge der Erzeugung widerspiegelt. Dadurch ist eine gezielte Suche nach den Zwischenergebnissen vergangener FO-Programmläufe möglich.
- Nach 100 FO-Aufrufen im selben Prozess werden die Temporärverzeichnisse sofort bei FO-Programmende bzw. Maskenaustritt gelöscht, um eine Überlaufen des Verzeichnisses unterhalb von `rmtmp` zu vermeiden.

Eindeutige Namen

Einmaliger eindeutiger Name im Mandanten

Die Variable `'G|versionneu'` liefert eine Nummer, die innerhalb eines Mandanten nie wieder verwendet wird. Diese Nummer kann als Bestandteil des Temporärdateinamens benutzt werden, um diesen eindeutig zu machen. Mit `versionneu` können Sie in einem Programm verschiedene eindeutige Namen generieren.

Das nachfolgende FOP baute einen solchen Namen zusammen.

```
.. create temporary file
.type text xtmpname xtprefix
.formula U|xtprefix = "MP"
.formula U|xtmpname = "rmtmp/" + -U|xtprefix + 'G|newVersionCountVal' + ".TMP"
'U|xtmpname '
```

Variablen, die auf diese Art und Weise gebaut werden, haben folgende Eigenschaften:

- Sie haben ein gemeinsames Präfix für jedes gleichartige Vorkommen von Temporärnamen, falls **'xprefix'** geeignet definiert wurde.
- Sie sind nach dem Entstehungsalter sortiert, so dass **versionneu** einen Zeitstempel enthält.
- Ihr Name ist innerhalb des Mandanten eindeutig und wird nie wieder verwendet.

Eindeutiger Name im Mandanten

Die Variable **'aufrufern'** liefert eine Nummer, die innerhalb eines Mandanten und auch bei Netzinstallationen eindeutig ist. Jeder Datenbankprozess eines Mandanten erhält eine eindeutige Nummer. Diese Nummer kann als Bestandteil des Namens der Temporärdatei benutzt werden, um diesen eindeutig zu machen. Der Name bleibt während der Lebensdauer des Aufrufers konstant.

Die Werte für "aufrufern" sind in verschiedenen Mandanten mit hoher Wahrscheinlichkeit identisch. Daher darf die Variable **'G|callerNo'** nur für solche Dateien verwendet werden, die im Arbeitsbereich eines Mandanten liegen. Für Dateien, die außerhalb des Mandanten liegen (z.B. \$TMP), muss die folgende Variante verwendet werden.

```
.. create temporary filed
.type text xtmp
.formula U|xtmp = "XABC" + G|callerNo + ".TMP"
'U|xtmp'
```

Beispiel: Eindeutiger Name auf einem Rechner

Die Variable **'pid'** enthält die rechnerweit eindeutige Prozessnummer. Mit ihr können rechnerweit eindeutige Namen erzeugt werden.

'G|processId' ist in einem Mandanten nicht mehr eindeutig, wenn auf mehreren Rechnern Datenbank-Clients laufen (Netzinstallationen). Um Kollisionen zu vermeiden, falls die Prozessnummer (ausnahmsweise) auf zwei Rechnern und die Zwischendatei identisch sind, ist bei Netzinstallationen darauf zu achten, dass das Verzeichnis (z.B. Temporärbereich **\$TMP**), in das Sie schreiben wollen, auf jedem Rechner lokal eingerichtet ist.

```
.. create temporary file
.type text xtmp
.formula U|xtmp = "XYZ" + G|processId + ".TMP"
```

```
'U|xttmp'
```

Eindeutiger Dateiname für einen Benutzer

Soll für jeden Abas Benutzer eine eindeutige Datei erstellt werden, muss jeder Benutzer ein eigenes Passwort benutzen. Die Variable '[G|uniqueNoUser](#)' liefert die eindeutige Nummer des Datensatzes der [Passwortdefinitionen]. Der Dateiname kann wie folgt vereinbart werden:

Da die Nummer des Datensatzes der [Passwortdefinitionen] sich nicht ändert, kann auf eine **DAT.uniqueNoUser**-Datei immer wieder zugegriffen werden, zum Beispiel zur Ausführung der Funktion "Druck wiederholen".

```
.type text xtben
.. define user file name
.formula U|xtben = "DAT." + G|uniqueNoUser
'U|xtben'
```

Datenbankkommandos

Die FO-Schnittstelle ermöglicht auch den Zugriff auf gespeicherte Datensätze. Diese werden z.B. mit den Kommandos **.hole** oder **.lade** aus der Datenbank gelesen. Anschließend bleiben sie so lange im Hauptspeicher, bis sie wieder durch ein FO-Kommando freigegeben werden. Man spricht davon, dass sie "geladen" sind. Im Hauptspeicher befinden sich die Datensätze in sogenannten Puffern; das sind Datenstrukturen, die beliebige Datensätze aufnehmen können. Diese Puffer können in der FO mit ihren aus einem Zeichen bestehenden Namen (z.B. H, D, O, 1, ...) angesprochen werden. Siehe hierzu auch: [Pufferbuchstaben von Variablen](#)

Wenn ein Datensatz in einer Maske bearbeitet wird, befindet er sich bereits in im Puffer M, ohne dass er vom FO-Programmierer geladen werden muss. Auch auf diesen Puffer kann per FO zugegriffen werden (siehe [EFOP](#)). Zu den unterschiedlichen Zugriffsrechten im Dialogfall bzw. beim expliziten Laden siehe Abschnitt [Dialogzustand](#).

Allgemeine Datenbankkommandos

.HOLE / .SELECT - Datensätze holen

.HOLE	.SELECT
deutsch	englisch
.HOLE Datenbankname	.SELECT database name [selection criteria]
[Selektionskriterien]	.SELECT database name ""
.HOLE Datenbankname ""	.SELECT -
.HOLE -	

.HOLE	.SELECT
.hole Datenbankname holt den nächsten Datensatz aus diesem Bereich. .hole Datenbankname "Identnummer/Suchwort" holt den Datensatz mit der Identnummer bzw. dem Suchwort aus der angegebenen Datenbank. .hole Datenbankname 'Textvariable' holt den Datensatz bzw. die Datensätze aus der angegebenen Datenbank. Die Textvariable beinhaltet Identnummer, Suchwort, Id oder einen Selektionsstring. .hole Datenbankname "" fordert die Eingabe eines Suchbegriffs über Tastatur an. .hole - gibt den geholten Datensatz wieder auf. Wenn ein Datensatz mit dem Kommando .hole gemerkt wurde, dann können Sie diesen mit dem Kommando .hole laden. Der Sperrzustand bleibt dabei erhalten.	

Beispiel: Selektion aller Datensätze aus parts

```

..!interpreter english declaration noabbrev
..
.select parts
!CHECK
.continue END ? G|mehr = G|false
..<META H|= 'P2'>
.println 'H|nummer' - 'H|such' - 'H|namebspr'
.select continue
.continue CHECK
..
!END
.end

```

Beispiel: Selektion mit Identnummer

```

..!interpreter english declaration noabbrev
..
.select parts "10002"
!CHECK
.continue END ? G|mehr = G|false
..<META H|= 'P2:1'>
.println 'H|nummer' - 'H|such' - 'H|namebspr'
.select continue
.continue CHECK
..
!END

```

```
.end
```

Beispiel: Selektion mit Identnummer aus Textvariable

```
..!interpreter english declaration noabbrev
..
.type text criteria ? F|defined(U|criteria) = G|false
..
.formula U|criteria = "10002"
.select parts 'U|criteria'
!CHECK
.continue END ? G|mehr = G|false
..<META H|= 'P2:1'>
.println 'H|nummer      ' - 'H|such      ' - 'H|namebspr      '
.select continue
.continue CHECK
..
!END
.end
```

Beispiel: Selektion mit Selektionsstring

```
..!interpreter english declaration noabbrev
..
.type text criteria ? F|defined(U|criteria) = G|false
..
.formula U|criteria = "$,nummer=10002!10099"
.select parts 'U|criteria'
!CHECK
.continue END ? G|mehr = G|false
..<META H|= 'P2:1'>
.println 'H|nummer      ' - 'H|such      ' - 'H|namebspr      '
.select continue
.continue CHECK
..
!END
.end
```

Beispiel: Selektion mit Eingabeaufforderung

Sie können in die Eingabeaufforderung folgendes eintragen:

- Suchwort
- Identnummer

- Id
- Selektionsstring

```

..!interpreter english declaration noabbrev
..
.select parts ""
!CHECK
.continue END ? G|mehr = G|false
..<META H|= 'P2'>
.println 'H|nummer' - 'H|such' - 'H|namespr'
.select continue
.continue CHECK
..
!END
.select
.end

```

.HOLE GRUPPE / .SELECT GROUP - Datenbankgruppe holen

.HOLE GRUPPE	.SELECT GROUP
deutsch	englisch
.HOLE GRUPPE 'Gruppenangabe' 'Selektionsangabe'	.SELECT GROUP 'group specification' 'selection specification'
Holt Datensätze aus der 'Gruppenangabe'. Als 'Gruppenangabe' können Datenbanken, Gruppenlisten oder Verweisarten verwendet werden.	

Um auf einen Datensatz zugreifen zu können, muss er zunächst aus der Datenbank eingelesen werden. Dazu dient das Kommando

.hole 'Datenbank/Gruppenangabe' ['Selektionsangabe']

```

.type text xtssel
.formula U|xtssel = "$,idno=100900;@language=en"
.select sales 'U|xtssel'
.select group "3" 'U|xtssel'
.select group "P3" 'U|xtssel'
.select group "ID3" 'U|xtssel'

```

haben jeweils genau dieselbe Wirkung.

Zweck der Option **gruppe** ist es, eine gezielte Gruppenangabe zu machen.

Das folgende Kommando wird nur geladene Verkaufsauftragshülsen als Ergebnis liefern; unabhängig von

der Selektionsangabe in **U|xtsel**.

```
.select group "3:22" 'U|xtsel'
```

Die Selektionsangabe darf diesem nicht widersprechen und z.B. ein "**@file=3:24**" enthalten.
Eine Angabe von "**@group=22**" ist nicht notwendig. Die Angabe der Gruppe ist auch im Zusammenhang mit **dazu** zulässig.



Wenn als Gruppenangabe ein polymorpher Verweis verwendet wird, dann wird die Ergebnismenge in jedem Fall nur Objekte einer einzigen Datenbank enthalten. Die gewünschte Datenbank sollte in den Selektionskriterien angegeben werden, ansonsten ist das Verhalten unspezifiziert.

Das Kommando

.hole gruppe "3:22" dazu U|xpos lädt ebenfalls eine Auftragshülse, falls der Verweis auf einen Auftragskopf oder eine Auftragsposition zeigt.



Dieser Befehl leert den D-Puffer.

Ein Datensatz kann per Selektionsmaske gefunden und geholt werden. Dabei ist es möglich, die Selektionsmaske auszugeben und im Dialog zu belegen (%-Selektion) oder **dialoglos zu suchen** und die **Selektion** ohne Eingabe in die geöffnete Selektionsmaske zu nutzen.

.HOLE DATEI / .SELECT FILE - Datensätze aus Zusatztable

.HOLE DATEI	.SELECT FILE
deutsch	englisch
.HOLE DATEI	.SELECT FILE
.HOLE DATEI2	.SELECT FILE2
...	...
.HOLE DATEI60	.SELECT FILE60
.HOLE DATEI VORLAGE	.SELECT FILE TEMPLATE
.hole datei greift auf die Zusatztabellen 1 bis 60 zu.	
.hole datei vorlage holt Objekte aus der Zusatztable zum Wiedervorlagetermin.	

```
.select file
'G|success'
.select file10
'G|success'
.select file60
```


'G|success'



Die Zusatztabellen (File51) bis (File60) sind für Lokalisierungen der Abas Partner reserviert und dürfen nur dafür verwendet werden.

.DAZU / .ADD - Weiteren Datensatz holen

.DAZU	.ADD
deutsch	englisch
.DAZU	.ADD
.DAZU 'Verweisfeld'	.ADD 'reference field'
.DAZU VKZ 'Daten'	.ADD VKZ 'data'
.dazu gibt den D-Puffer frei. .dazu 'Verweisfeld' lädt ein Verweisobjekt in den D-Puffer. .dazu vkz 'Parameter' lädt die monatlichen Verkehrszahlen und Umsatzzähler in den D-Puffer. Funktion und Parameter sind wie bei .hole vkz.	

Der Befehl `.dazu 'Verweisfeld'` setzt voraus, dass bereits ein Datensatz geladen ist, entweder explizit durch `.hole` / `.lade` oder implizit in den Maskenpuffer bei [ereignisgesteuerten FOPs](#). Für `'Verweisfeld'` ist der Name eines Verweisfeldes aus diesem Datensatz einzusetzen. Das verwiesene Objekt wird in den D-Puffer geladen.

```
.select sales "55"
.continue END          ? G|success = G|false
'H|idno'
.add customer
.continue END          ? G|success = G|false
'D|idno'
.add
..
!END
.end
```

`'D|idno'` liefert hier die Kundennummer.

`'H|idno'` liefert hier die Auftragsnummer.

`.add` beendet den Zugriff auf den Kundendatensatz.



Das Kommando **.hole dazu** leistet dasselbe wie **.dazu**, jedoch wird der H-Puffer durch den geladenen Datensatz überlagert.

.HOLE DAZU / .SELECT ADD - Weiteren Datensatz holen

.HOLE DAZU	.SELECT ADD
deutsch	englisch
.HOLE DAZU .HOLE DAZU 'Datensatzverweis'	.SELECT ADD .SELECT ADD 'record reference'
lädt, wie .dazu , ein weiteres Verweisobjekt. Im Unterschied zu .dazu wird das Verweisobjekt in den H-Puffer geladen. Um den Ausgangsdatsatz zu laden, verwenden Sie erneut .hole`dazu`	

Unterschiede zwischen .dazu und .hole

Anders als bei **.hole** kann bei **.hole** der Datensatz nicht mit Sperre gelesen, verändert und zurückgeschrieben werden. Außerdem besteht keine Möglichkeit, Felder aus einer Tabelle abzufragen. Dafür ist **.hole** bei Sätzen mit einer großen Tabelle wesentlich schneller als **.hole**.

```
..!interpreter english noabbrev
.select customer "70010"
.continue END ? G|mehr = G|false
.select add H|zbed
.continue END ? G|mehr = G|false
'H|nummer' - 'H|namebspr'
.select add
-> customer idno: 'H|nummer'
..
!END
end
.end
```

.HOLE OBJEKT / .SELECT OBJECT - Allgemeines Datenbankobjekt laden

.HOLE OBJEKT	.SELECT OBJECT
deutsch	englisch
.HOLE OBJEKT "sel"	.SELECT OBJECT "sel"
holt den über die Objektauswahl oder Bereichsdefinition sel gefundenen Datensatz.	

Die datenbankübergreifende Option **objekt** überlässt der Objektauswahl bzw. der Selektionsangabe **@Datenbank=** die eigentliche Auswahl der Datenbank. In FOPs muss in diesem Fall nicht umständlich das richtige Schlüsselwort ermittelt werden.

```
.select object "(157,7,0)"
?'H|swd'      ? G|success = G|true
.select object "%,@database=7;idno="
?'H|swd'      ? G|success = G|true
```



Nach einem **.hole** objekt 'sel' muss mit **.hole weiter** auf den nächsten Datensatz weitergeschaltet werden, nicht (analog zu konkreten Datenbankangaben) mit **.hole** objekt.

.HOLE WEITER / .SELECT CONTINUE - Weiterschalten

.HOLE ['Sperroptionen'] WEITER	.SELECT ['Sperroptionen'] CONTINUE
deutsch	englisch
.HOLE WEITER	.SELECT CONTINUE
schaltet im Bereich weiter, ohne dass der Datenbankname wiederholt werden muss	

```
.select customer
'H|swd'
.select continue
'H|swd'
```

Schlüsselwörter ermitteln

Mit der Nummer einer Datenbank können Sie das zugehörige Schlüsselwort für **.hole** ermitteln. Dazu steht das Array 'G|hdatname.i' zur Verfügung.

```
'G|selectDBNames0'
'G|selectDBNames3'
'G|selectDBNames15'
```

display

```
customer
sales
file
```

Können Objekte der Datenbank mit der Nummer **n** nicht mit **.hole** geladen werden, so ist die Variable **'hdatname.n'** leer. Der maximale Wert für **n** steht in **'G|maxkdatnam'**.

Mit dem folgenden Programm können Sie alle Schlüsselwörter in der aktuellen FO-Sprache ermitteln:

```
.type int xiindex
.output new "KEYWORDS.TMP"
.input DATEI.F
..
!WHILE
'G|hdatname.U|xiindex'
.formula U|xiindex = U|xiindex + 1
.continue WHILE ? U|xiindex < 160
```

Selektionsangaben sind optional. Durch sie wird die Auswahl eingengt. Selektionsangaben können Nummern, Suchwörter, Ergänzungen oder Bereiche beinhalten. Es gelten die Regeln, die auch im Dialogbetrieb für **Nummern** und **Suchwörter** gelten.

```
.select customer "Maier"
.select customer
.select product "1000!"
```

Datensätze der angegebenen Datenbank werden also vom Datenträger gelesen ("geholt"). Mit Hilfe von Variablen können dann Einträge der Datensätze dargestellt und verändert werden. Zum Beispiel ist die Variable **nort** undefiniert, solange kein Datensatz geholt wurde. Sie enthält aber die Ortsangabe eines Kunden oder Lieferanten, sobald ein solcher geholt wird:

```
.select customer "Maier"
'H|town'
```

display

Mannheim

Nach den Befehlen **.hole** und **.lade** liegt der Datensatz (anders als im EFOP) im **Festplattenzustand** vor. Die Kommandos **.hole 'Datenbank/Gruppenangabe'** und **.lade 'Datenbank/Gruppenangabe'** können von Benutzern ausgeführt werden, die für **'Datenbank/Gruppenangabe'** die Erlaubnis **<Zeigen>** besitzen.

.HOLE SPERREN / .SELECT LOCK - Zugriffssperre beim Holen eines Datensatzes

.HOLE SPERREN		.SELECT LOCK	
deutsch		englisch	
.HOLE ['Sperroptionen'] ['Sperrmodifikatoren'] 'Datenbankname' 'Suchbegriff' .HOLE 'Sperroptionen' ZEILE		.SELECT 'lock options' ['lock modifier'] 'file name' 'search term' .SELECT 'lock options' LINE	
Sperroptionen	Sperrmodifikatoren	lock options	lock modifier
SPERREN	KEINABBRUCH	LOCK	NOTERMINATION
KEINESPERRE	ABBRUCH	NOLOCK	CANCEL
LESESPERRE	LANGWARTEN	READLOCK	WAITLONG
AENDERNSPERRE	KURZWARTEN	MODIFICATIONLOCK	WAITSHORT
SCHREIBSPERRE	NICHTWARTEN	WRITELOCK	NOWAIT
FREIGEBEN	WARTEN	RELEASE	WAIT
.hole 'Sperroptionen' ['Sperrmodifikatoren'] 'Datenbankname' 'Suchbegriff' liest den angegebenen Datensatz und sperrt ihn für andere Benutzer. .hole 'Sperroptionen' zeile beschafft eine Einkaufs-/Verkaufsposition.			

Wenn zwei Benutzer gleichzeitig einen Datensatz lesen und verändern, würde nur die zuletzt geschriebene Version wirksam. Deshalb müssen Sätze, die später geändert werden sollen, schon beim Lesen für andere mit einer Sperre belegt werden.

```
.select lock customer "1"
.continue END          ? G|success = G|false
.continue NOLOCK       ? G|locked = G|false
.formula H|zipCode = "12345"
.rewrite
.continue GO
!GO
do something
.continue END
!NOLOCK
record not locked
!END
end
.end
```

Das Kommando **.hole sperren** 'Datenbankname' 'Suchbegriff' kann vom Benutzer ausgeführt werden, wenn er für 'Datenbankname' die Erlaubnis <Zeigen> besitzt. Die Erlaubnis für <Ändern> ist nicht notwendig.

Bei Ein- und Verkaufsobjekten ist folgendes zu beachten: Auch wenn alle Positionszeilen in derselben Maske dargestellt werden, so handelt es sich doch um eigenständige Datensätze, die mit **.hole sperren zeile** beschafft werden.

Um das Sperren gezielt zu beeinflussen, können **Sperroptionen** hinzugefügt werden.

Lesesperre

bei **.hole einkaufs**, **.hole verkaufs** und **.hole fertigungs**



Im FOP wird ein Einkaufs-/Verkaufs-/Fertigungsvorgang immer mit Lesesperre geholt. Soll z.B. durch **.formel M|beleg = H|id** eine Position an diesen Vorgang angefügt werden, ist dafür eine exklusive Sperre nötig. Würden zwei Benutzer gleichzeitig diesen Aktion ausführen wollen, käme es zu einem Deadlock, denn jeder bekäme die Lesesperre aber keiner die exklusive Sperre. Das Programm reagiert deshalb mit einer Diagnosemeldung.

Damit beim Holen des Datensatzes keine Lesesperre angefordert wird, verwenden Sie die **Option keinesperre**, z.B. **.hole verkaufs keinesperre**

Beachten Sie aber, dass **keinesperre** bewirkt, dass diese Vorgänge inkonsistent gelesen werden!

Siehe auch **.belegen - Dateiausgabe synchronisieren**.

Aufgeben eines Datensatzes

Mit dem Befehl **.hole -**

wird der mit dem letzten **.hole**-Befehl geladene Datensatz wieder aufgegeben. Das kann sinnvoll sein, wenn der Datensatz gesperrt war und für andere Benutzer wieder freigegeben werden soll.

Sperroptionen bei **.hole**, **.lade** und **.belegen**

Bei den Kommandos **.belegen**, **.belegen** und **.belegen** kann das Sperren von Datensätzen mittels Sperroptionen und Sperrmodifikatoren gezielter beeinflusst werden.

Sperroptionen

Deutsch	Englisch	Beschreibung
keinesperre	nolock	Es wird keine Sperre gesetzt. Dies ist beim Lesen von Objekten im Einkauf, Verkauf und aus der Datenbank Betriebsauftrag sinnvoll, da diese ohne Angabe einer Sperroption immer mit Lesesperre geladen werden. In allen anderen Datenbankobjekten wird diese Option jedoch standardmäßig verwendet.

Deutsch	Englisch	Beschreibung
lesesperre	readlock	Dient zum Schutz gegen Schreibzugriffe. Die Sperre wird im Einkauf, Verkauf und der Datenbank Betriebsauftrag als Standardwert eingesetzt, um Vorgänge, die aus mehreren Datensätzen bestehen (z.B. Auftrag mit Kopf und mehreren Positionen), konsistent aufzugreifen.
aendernsperre	modificationlock	Dient zum Schutz vor Änderungen des Datensatzes durch andere Prozesse. Wirkt wie die Sperroption sperren , allerdings mit anderem Abbruchverhalten.
schreibsperre	writelock	Dient zum Schutz gegen Schreiber und Leser mit Lesesperre. Die Sperre wird beim Zurückschreiben mit .bringe benötigt. Wenn bei .hole eine Ändersperre gehalten wird, beschafft .bringe sich die Schreibsperre selbst. Diese Option kann mit Datenbankangabe zum Lesen eines Datensatzes mit Schreibsperre und ohne Datenbankangabe zur Umwandlung einer Ändersperre in eine Schreibsperre benutzt werden. Im zweiten Fall wird nur die Sperre verändert, nicht der Pufferinhalt. Dieser muss bereits beim Setzen der Ändersperre (oder Schreibsperre) gelesen worden sein.
freigeben	release	Gibt eine Sperre frei, wobei der Pufferzustand sonst unverändert bleibt. Diese Sperre ist nur sinnvoll, wenn der Puffer bereits geladen ist. Deshalb ist diese Option nur ohne Datenbank- und sonstige Angaben zulässig.

```
.load 0 modificationlock product "1000"
do something
.load 0 release
```

Sperrmodifikatoren

Neben den Sperroptionen gibt es noch Sperrmodifikatoren, die das Dialog- und Warteverhalten steuern, falls andere Prozesse angeforderte Sperren halten und so das FOP daran hindern, die gewünschte Sperre zu belegen.

Bei **keinesperre** und **freigeben** sind Modifikatoren nicht sinnvoll.

Deutsch	Englisch	Beschreibung
keinabbruch	notermination	Im Dialog ist kein normaler Abbruch möglich. Ein Abbruch wird wie ein Kritischer Abbruch des Sperrdialogs behandelt.
abbruch	cancel	Im Dialog ist Abbruch möglich. Der Benutzer kann das Lesen mit Sperre abbrechen.
langwarten	waitlong	Es wird lange (ca. 1 Minute) versucht, die Sperre zu bekommen, dann automatisch aufgegeben.
kurzwarten	waitshort	Es wird kurz (ca. 8 Sekunden) versucht, die Sperre zu bekommen, dann automatisch aufgegeben.
nichtwarten	nowait	Es wird einmal versucht, die Sperre zu belegen, dann sofort aufgegeben.
warten	wait	Es wird gewartet, bis die Sperre belegt wurde. Warten ist das Standardverhalten, wenn weder langwarten , kurzwarten noch nichtwarten angegeben wurde.

Sperrmodifikatoren sind kombinierbar. Eine sinnvolle Kombination ist z.B. **langwarten abbruch**. Nur allein sinnvoll sind die Sperrmodifikatoren **keinabbruch** und **nichtwarten**.

Das Verhalten der FO bei Abbruch während des Wartens auf eine Sperre oder bei Aufgabe eines Sperrversuchs hängt von der Sperroption ab: Ist kein Sperroperator (implizite Lesesperre in Einkauf, Fertigung, Betriebsauftrag) oder ausschließlich **sperren** angegeben, so beendet "Abbruch" das FO-Programm. Ist mindestens eine andere Sperroption angegeben, so beendet "Abbruch" das FO-Programm nicht. Außerdem wird die boolesche FO-Variable '**G|gesperrt**' gesetzt und der Datensatz ist ohne Sperre geladen. Die Variable **G|mehr** wird unabhängig vom Sperrzustand genau dann auf **G|true** gesetzt, wenn ein Datensatz geladen wurde.

Falls mindestens ein Sperroperator außer **sperren** angegeben wurde, nimmt '**G|gesperrt**' den Wert **G|true** an, wenn die Sperroperation erfolgreich war (**keinesperre** und **freigeben** sind immer erfolgreich.). Es hat den Wert **G|false**, wenn die Sperroperation durch Benutzerabbruch oder Zeitablauf gescheitert ist. Die Variable **G|mehr** ist unabhängig vom Sperrmodus gesetzt!



Auch wenn die Sperroperation scheitert (**G|gesperrt=G|false**), ist das Objekt geladen. Es kann also mit dem entsprechenden Puffer angesprochen werden.

Die Puffervariable **G|sperrius** beantwortet die Frage, welche Sperre im Puffer liegt. Die Werte dieser Variablen entsprechen den Sperroptionen. Die Variable zeigt den Sperrmodus bei jedem belegten Puffer an.

Sperren und .bringe

Das Schreiben benötigt eine Schreibsperre. **.bringe** darf auch aufgerufen werden, wenn nur eine Ändersperre vorliegt. Dann wird automatisch die Sperre auf **schreibsperre** verschärft, allerdings ohne ein "Abbruch möglich". Um den Abbruch zu ermöglichen, können Sie vorschalten:

.hole schreibsperre abbruch

Sperren und .setze abbruch

Wenn kein Sperrmodifikator angegeben ist, ist das Dialogverhalten beim Warten auf eine Sperre wie folgt:

Deutsch	Englisch	Beschreibung
.setze abbruch +	.set cancel +	Standardfall. Es wird mit Abbruchmöglichkeit gewartet. Ist keine der neuen Sperroptionen angegeben, so endet das FO-Programm bei Abbruch. Andernfalls ist G gesperrt = G false gesetzt.
.setze abbruch -	.set cancel -	Verhalten wie .setze abbruch +
.setze abbruch --	.set cancel --	Gewartet wird ohne Abbruchmöglichkeit. Siehe auch Abbrechen von Sperrdialogen .

Sperren und .aktion

Wie lange eine Sperre gehalten wird, hängt davon ab, ob die Sperre innerhalb einer Transaktion (**.aktion +|-**) geholt wird oder nicht. Ohne Transaktion steht eine Sperre, bis der Puffer geleert oder neu geladen wird oder bis die Sperre explizit freigegeben wird. Innerhalb einer Transaktion wird die Sperre immer mindestens bis zum Transaktionsende gehalten, auch wenn sie freigegeben oder der Puffer neu geladen oder geleert wird.

Eine Sperre kann damit maximal bis zum Ende des aufrufenden FOP oder EFOPs gehalten werden; denn am Ende eines FOPs werden alle Puffer freigegeben und alle Benutzertransaktionen beendet.



- Auch in Transaktionen des Abas Kerns werden Sperren der FO bis zum Transaktionsende gehalten. Das betrifft die Maskenaustritt- und Workflow-FOPs.
- Auch alle Lesesperren werden in Transaktionen bis zum Transaktionsende gehalten. Das betrifft insbesondere den Ein- und Verkauf und die Fertigung, wo auch ein **.hole** ohne Sperrangabe immer mit Lesesperre erfolgt. Um Sperrkonflikte zu vermeiden, kann, wenn es nicht auf die Konsistenz ganzer Vorgänge ankommt, mit

keinesperre gearbeitet werden.

- Dass Sperren immer bis zum Transaktionsende gehalten werden, ist die Sicht von außen. Für den Prozess selbst besteht die Sperre genau so lange, wie der Puffer geladen oder die Sperre freigegeben ist.

Der Befehl **.hole sperren** muss dem Kommando **.bringe** vorangehen, selbst wenn er in derselben Transaktion bereits geschrieben wurde. Die Sicht des FO-Programms selbst ist unabhängig von der Tatsache, ob eine Transaktion angefangen hat.



Möglicherweise müssen Sie beim Lesen auf mehrere andere Leser warten, oder es kommen während des Wartens neue Leser hinzu. Wenn Sie eine Ändersperre in eine Schreibsperre umwandeln oder einen Datensatz mit Schreibsperre laden wollen müssen Sie eventuell auf mehrere Prozesse mit Lesesperre warten.

Beispiel:

In den folgenden Beispielen wird versucht, Sperren zu bekommen. Um das Verhalten zu demonstrieren, wird beschrieben, was bei Sperrgegnern passiert, die die Sperre halten. Es gibt drei mögliche Sperrgegner:

Lesegegner

```
.type text xtread
.select sales "100001"
.read "Please wait, readlock is active!" U|xtread
```

Ändergegner

```
.type text xtread
.select modificationlock sales "100001"
.read "Please wait, modification lock is active!" U|xtread
```

Schreibgegner

```
.type text xtread
.select writelock sales "100001"
.read "Please wait, write lock is active!" U|xtread
```

Im Folgenden werden Beispiele im Hinblick auf die obigen Gegner beschrieben. Beachten Sie, dass das Feld **lockMode** pufferbezogen ist, d.h. es darf kein Pufferbuchstabe vorangestellt werden.

Grundverhalten:

```
.select lock sales "100907"
```

```
sales after select: 'H|idno' - locked: 'G|locked' lockmode: 'lockMode'
.rewrite
sales after rewrite: 'H|idno' - locked: 'G|locked' lockmode: 'lockMode'
```

Bei Lesegegnern:

Wartebox mit "belegt" ohne Abbruchmöglichkeit erscheint bei **.bringe**.

Bei Editorgegner oder Schreibgegner:

Wartebox mit "belegt: Abbruch möglich" erscheint bei **.hole**. Bei Abbruch endet das FO-Programm.

Variante 1

Die Schreibsperre sofort holen.

```
.select writelock sales "100907"
sales after select: 'H|idno' - locked: 'G|locked' - lockMode: 'lockMode'
? lock not active ? G|locked = false
.end          ? G|locked = false
.rewrite
sales after rewrite: 'H|idno' - locked: 'G|locked' - lockMode: 'lockMode'
```

Hier besteht die Möglichkeit, dass der Text "lock not active" ausgegeben wird und das FO-Programm vor dem **.bringe** endet.

Bei allen Gegnern:

Wartebox mit "belegt: Abbruch möglich" erscheint bei **.hole**. Bei "Abbruch" erscheint noch die erste Ausgabezeile.

Variante 1.1

Kein Abbruch möglich.

Die **.hole**-Zeile ersetzen durch:

```
.select writelock nottermination sales "100907"
```

Bei allen Gegnern:

Wartebox mit "belegt" ohne "Abbruch möglich" erscheint bei **.hole**.

Dasselbe kann erreicht werden durch

```
.set cancel --
.select writelock sales "100907"
```

Variante 1.2

Abbruch möglich.

Die **.hole**-Zeile ersetzen durch:

```
.set cancel --
.select writelock cancel sales "100907"
```

Bewirkt wieder das Originalverhalten mit "Abbruch möglich" in einem FO-Programm, das ansonsten nicht abgebrochen werden kann. Das FO-Programm endet hier erst nach der Ausgabe "Sperrung nicht erhalten".

Variante 1.3

Zeitgesteuertes Warten: Nichtwarten, Kurzwarten, Langwarten

Ersetze die **.hole**-Zeile durch:

```
.select writelock nowait sales "100907"
```

bzw. statt **nichtwarten** auch **kurzwarten** oder **langwarten**: Bei jedem Gegner wird entweder ohne jedes Warten oder nach kurzem oder langem Warten die Ausgabe "Sperrung nicht erhalten" erscheinen, und das FOP enden.

Variante 2

Nachträglich Schreibsperre holen und vorzeitig freigeben:

```
.select modificationlock sales "100907"
sales after editor lock:      'H|idno' - locked: 'G|locked' - lockMode: 'lockMode'
? lock not active           ? G|locked = false
.end                          ? G|locked = false
.select writelock
sales after select writelock: 'H|idno' - locked: 'G|locked' - lockMode: 'lockMode'
? lock not active           ? G|locked = false
.end                          ? G|locked = false
.rewrite
sales after rewrite:         'H|idno' - locked: 'G|locked' - lockMode: 'lockMode'
.select release
sales after release:         'H|idno' - locked: 'G|locked' - lockMode: 'lockMode'
```

Bei Lesegegnern:

Erst beim zweiten **.hole** **schreibsperre** erscheint die Meldung:

"Belegt: Abbruch möglich" (Beim Original war das erst bei **.bringe** der Fall und ohne "Abbruch möglich"!)

Bei Ändern- oder Schreibgegnern:

Wie gehabt: "Belegt: Abbruch möglich" beim ersten **.hole**
 Erreicht das FOP die letzte Zeile, steht keine Sperre mehr.

Variante 3

Mit **.aktion +|-**

Der Einsatz von **.aktion +|-** ist nur notwendig, wenn mehr als ein Objekt in einer Transaktion geändert oder gesperrt werden soll. Eine Klammerung mit **.aktion +|-** sichert dann ab, dass die Änderungen entweder alle gleichzeitig oder aber gar nicht wirksam werden. Gleichzeitig hat sie zur Folge, dass alle Sperren bis zum abschließenden **.aktion -** gehalten werden.

```
.type text xtread xtcustomer
.select lock sales "100906"
.formula U|xtcustomer = H|customer
.rewrite
.select release
.read "further lock" U|xtread
.select lock sales "100907"
.formula U|xtcustomer = H|customer
.rewrite
.select release
```

Während das Programm bei **.lesen** wartet, kann ohne die Klammerung mit **.aktion +** oder **.aktion -** ein anderer Prozess den Auftrag "100906" ändern und dort einen anderen Kunden eintragen. Im Folgenden läuft das Beispiel in einer Transaktion ab:

```
.type text xtread xtcustomer
.action+
.select lock sales "100906"
.formula U|xtcustomer = H|customer
.rewrite
.select release
.read "further lock" U|xtread
.select lock sales "100907"
.formula U|xtcustomer = H|customer
.rewrite
.select release
.action-
```

Jetzt bleibt die Sperre auf dem Auftrag für andere Prozesse bis **.aktion -** stehen. Aus Sicht des Programms ist der Wert von 'sperrmodus' nach **.hole freigeben** aber immer noch "freigeben".

Das Beispiel ist insofern ungeeignet, als kritische Abschnitte so kurz wie möglich gehalten werden sollten, um Deadlocks und die Behinderung anderer Benutzer zu vermeiden. Benutzereingaben dauern sehr lange und sollten in Transaktionen unbedingt vermieden werden. Auch längere Ausgaben sollten wenn möglich außerhalb von Transaktionen erfolgen.

Holen von Datensätzen durch dialoglose Suche

Mit den Selektions- und Sachmerkmalleisten können Sie sehr schnell dialoglos und ohne Eingaben in geöffnete Selektionsmasken suchen. Ist eine Selektionsmaske bekannt, gilt folgende Syntax:

\$Selektionsmasken-ID, Selektionskriterien

Siehe hierzu: [Selektionssyntax](#)

```
.select customer "$KUNDEN,nummer=10000!10010;@sort=Nummer;@richtung=rückwärts"
```

Funktion	Selektion nach Kunden ab Identnummer 10000 bis 10010 mit Sortierschlüssel "Nummer", rückwärts sortiert.
Ergebnis	Liefert alle Kundendatensätze von Identnummer 10000 bis 10010, absteigend nach Identnummer sortiert.

Für den [Einkauf](#) und [Verkauf](#):

offen	für unerledigte Datensätze
druck	für druckbereite Datensätze Angebot Auftrag Lieferschein Rechnung Provision Zahlung Bestellung Vorschlag
Ablage	für abgelegte Datensätze

Für die [Fertigung](#):

"Num"	Betriebsauftrag-/Arbeitsschein
"+'Num'"	Rückmeldung
Vorschlag	Fertigungsvorschläge
Obligo	Kapazitätsreservierungen einer zuvor gehalten Maschinengruppe
Druck	Betriebsaufträge für Arbeitspapieredruck
Übernahme	erfasste, nicht gebuchte Rückmeldungen
Ablage	alle Rückmeldebelege



Der erste Suchbegriff kann direkt als Text angegeben werden und muss dann in Anführungszeichen stehen. Er kann aber auch Inhalt einer Variablen sein, deren Name in Apostrophe eingeschlossen ist.

```
.select parts "V2"
.type text xtselection
'G|success' - 'H|swd'
.formula U|xtselection = "V3"
.select parts 'U|xtselection'
'G|success' - 'H|swd'
```

Weitere Datensätze einer Datenbank holen

Es wird unterschieden zwischen dem Holen des ersten Datensatzes und dem Holen aller folgenden Datensätze. Alternativ können Sie zum Laden weiterer Datensätze **.hole** **.weiter** verwenden.

Datensätze	Syntax im Normalfall
1. Datensatz holen	.hole ['Sperroptionen'] 'Datenbank/Gruppenangabe' 'Selektionsangabe'
Alle folgenden Datensätze holen	.hole ['Sperroptionen'] 'Datenbank/Gruppenangabe'

Die Selektionsangabe bewirkt, dass der erste Datensatz, auf den der Suchbegriff passt, geladen und ein Bereich für künftige Aufrufe von **.hole** festgelegt wird. Die folgenden Datensätze des Bereiches werden mit dem Kommando **.hole** **'Datenbank/Gruppenangabe'** geladen. Ist kein Datensatz mehr vorhanden, wird die Standardvariable **G|mehr** auf **G|false** gesetzt.



Wenn Sie **.hole** ohne Selektionsangabe aufrufen und es ist keine Selektion aktiv, dann wird eine neue Selektion über alle Sätze der Datenbank gestartet. Wenn Ihr Programm durchgelaufen ist und nachdem das letzte **.hole** nichts mehr gefunden hat, starten Sie damit eine neue Suche und potentiell einen neuen Schleifendurchlauf. Wenn Sie die Form **.hole** **.weiter** verwenden, und keine Suche aufgesetzt ist, wird keine neue Suche gestartet.

Suchbegriff lesen

```
.select customer ""
'G|success' - 'H|swd'
```

Ist der erste Suchbegriff ein Text ohne Zeichen, so wird der Suchbegriff über die Tastatur eingelesen. Als Anforderungstext dient der Datenbankname. Das obige Kommando führt also im Dialogfenster zur Aufforderung "Kunde" und wartet auf die Eingabe.



Es wird empfohlen, statt dessen eine Selektion zu verwenden, da man dann den Titel und

die Selektionskriterien flexibel festlegen kann.

.HOLE KOPF / .SELECT HEADER - Holen des Tabellenkopfes

.HOLE KOPF	.SELECT HEADER
deutsch	englisch
.HOLE KOPF .HOLE [MASKE AMASKE] KOPF .HOLE [MASKE] KOPF 'Selektionsstring'	.SELECT HEADER .SELECT [SCREEN ASCREEN] HEADER .SELECT [SCREEN] HEADER 'selection string'
.hole kopf lädt den Kopfteil. Das Kommando wird benötigt, wenn Positionen im Ein-/Verkauf mit .hole zeile durchlaufen werden, da dann der Kopfteil nicht mehr geladen ist. Ist in einem EFOP das Schlüsselwort maske oder amaske angegeben, bezieht sich der Befehl auf den aktuellen Maskenpuffer oder den A-Puffer, andernfalls auf das Bezugsobjekt H. Das Kommando .hole [MASKE] KOPF 'Selektionsstring' prüft, ob das zuvor geladene Objekt den Kriterien des Selektionsstrings entspricht. Wenn das geladene Objekt dem Selektionsstring entspricht, dann ist G mehr = G true, andernfalls ist G mehr = G false.	

Beispiel: **.hole [MASKE] KOPF 'Selektionsstring' / .select [SCREEN] HEADER 'selection string'**

Im folgenden Beispiel wird geprüft, ob die geladene Verkaufspositionszeile zu dem Verkaufsobjekt mit der Identnummer 201118 gehört.

```

..!interpreter english noabbrev
..
.select sales "(8665,3,0)"
.continue END ? G|mehr = G|false
..<META H|= 'P3'>
.println 'H|id' 'H|artikel' 'H|kopf^nummer'
..
.select header "$,kopf^nummer==201118"
.println belongs to data record: 201118 ? G|mehr = G|true
.println does NOT belong to data record: 201118 ? G|mehr = G|false
..
!END
end
.end

```

Lesen Sie hierzu auch **.hole maske - Zeilenoperationen in EFOPs**.



Synonym zu **.hole kopf** ist die Verwendung von **.hole zeile 0**

.HOLE ZEILE / .SELECT LINE - Holen von Tabellenzeilen

.HOLE ZEILE	.SELECT LINE
deutsch	englisch
.HOLE ZEILE .HOLE [MASKE AMASKE] ZEILE ['Zeilennummer' 'Zeilen-ID' 'Selektionsstring']	.SELECT LINE .SELECT [SCREEN ASCREEN] LINE ['line number' 'line ID' 'selection string']
.hole zeile beschafft die erste oder nächste Zeile der Tabelle. .hole zeile 0 positioniert auf den Kopf, synonym zu .hole kopf. Ist das Schlüsselwort maske bzw. amaske angegeben, bezieht sich der Befehl auf den M-Puffer bzw. auf den A-Puffer, andernfalls auf das Bezugsobjekt H. .hole zeile 'Zeilennummer' 'Zeilen-ID' 'Selektionsstring' beschafft die angegebene Zeile einer Tabelle. Fehlt eine Angabe zur Zeile, wird die nächste Zeile nach der aktuellen geholt.	

Auch in Datensätzen mit Tabellen ist nach einem **.hole** zunächst nur der Kopfteil des Datensatzes zugreifbar. Mit dem Kommando **.hole zeile** können Sie anschließend nacheinander die Tabellenzeilen durchlaufen.

Ein **.hole zeile** wird nur durch das unmittelbar vorausgehende **.hole zeile xxx** oder **.hole kopf** beeinflusst.

.hole zeile positioniert auf die nächste Zeile bzw. auf die erste Zeile (wenn man zuvor im Kopfteil stand) einer Tabelle. Die Variable **G|mehr** liefert **G|true**, wenn das Kommando erfolgreich war. Beim Aufruf in der letzten Zeile wird **G|mehr** gleich **G|false** und es wird wieder in den Kopfteil positioniert.

Nach Positionieren auf eine Tabellenzeile bleibt der Kopfteil des Datensatzes in der Regel weiterhin zugänglich. Treten Namenskonflikte zwischen Tabelle und Kopfteil auf, so hat der Tabellenteil Vorrang. Werden Positionen im Ein- oder Verkauf mit **.hole zeile** durchlaufen, ohne dass sie sich in einem Hülsenpuffer befinden, so ist der Kopfteil nicht mehr geladen. Mit dem Kommando **.hole [maske|amaske] kopf** können Sie den Kopfteil wieder laden.

Das Kommando **.hole [maske|amaske] zeile 'Zeilennummer'** lädt die entsprechende Zeile im M-, A- oder H-Puffer.

```
.select parts "403"
.continue END          ? G|success = G|false
.select line 3
```

```
.continue END          ? G|success = G|false
'H|rowNo'
!END
```

```
.select screen line 'M|rowCount'
'M|rowNo' - 'M|elemDescr' - 'M|elemQty'
```

Dieser Befehl erspart die Schleife "Hole Zeilen, solange vorhanden" und ist zweckmäßig, wenn Zeilen z.B. hinten angefügt werden.



Das Kommando **.hole zeile 'Zeilennummer'** kann in EFOPs immer, in FOPs jedoch nicht im Ein- oder Verkauf verwendet werden.

Ein wichtiger Anwendungsfall für das Kommando ist der Befehl

```
.select parts "403"
'G|success' - 'H|idno'
.type text xtselection
.formula U|xtselection = "$,elemDescr=Einkaufsteil eins;@language=en"
.select line 'U|xtselection'
'G|success' - 'H|rowNo'
```

bei einer Zeilenselektion (siehe auch **FO-Variable 'selzeile'**).

.hole [maske|amaske] zeile 'Zeilen-ID'

Als Zeilenangabe kann auch eine **Zeilen-ID** angegeben werden. Das ist der Inhalt des Felds **zid** einer Zeile. Ist die Zeile ein Kopfteil eines anderen Datensatzes innerhalb einer Hülse, kann die ID dieses Datensatzes als Zeilen-ID angegeben werden.

```
.type text xtRowId
.formula U|xtRowId = "(177,2,0,4)"
.select parts "V3"
.select line 'U|xtRowId'
'H|rowNo' - 'H|productListElem^swd'
```

Lesen Sie hierzu auch [Zeilenoperationen in EFOPs](#).

.HOLE MASKE / .SELECT SCREEN - Tabellenkopf/-zeilen in EFOPs

.HOLE MASKE	.SELECT SCREEN
deutsch	englisch
<code>.HOLE MASKE</code> <code>.HOLE MASKE [KOPF ZEILE]</code>	<code>.SELECT SCREEN</code> <code>.SELECT SCREEN [HEADER LINE]</code>
lädt den Kopfteil oder Tabellenteil des Bezugsobjektes M	

Lesen Sie hierzu `.hole ZEILE` und `.hole KOPF`

.HOLE AMASKE / .SELECT ASCREEN - Kopf oder Zeile des A-Puffers holen

.HOLE AMASKE	.SELECT ASCREEN
deutsch	englisch
<code>.HOLE AMASKE [KOPF ZEILE]</code>	<code>.SELECT ASCREEN [HEADER LINE]</code>
lädt den Kopfteil bzw. die nächste Zeile des A-Puffers	

Lesen Sie hierzu `.hole ZEILE` und `.hole KOPF`.

.SETZE AMASKE / .SET ASCREEN - A-Puffer zum M-Puffer machen

.SETZE AMASKE	.SET ASCREEN
deutsch	englisch
<code>.SETZE AMASKE + -</code>	<code>.SET ASCREEN + -</code>
macht den A-Puffer zum M-Puffer	

Nach `.setze amaske +` ist der bisherige A-Puffer der neue M-Puffer. Die Ereignisvariablen des A-Puffers werden dabei in die Ereignisvariablen des Maskenpuffers umgeladen. So wird z.B. der Inhalt von `G|avtkommd` in `G|evtkommd` umgeladen. Alle Zugriffe auf Puffervariable mit `M|` sowie die Befehle `.hole maske zeile` und `.mache maske zeile` wirken jetzt auf den bisherigen A-Puffer. Ein Aufruf von `.setze amaske +` ohne A-Puffer verursacht eine Fehlermeldung.

Nach `.setze amaske -` gilt wieder der Zustand wie vor `.setze amaske +`. Ereignisvariablen des A-Puffers und des Maskenpuffers werden wieder auf die früheren Werte gesetzt.

.SORTIERTABELLE / .SORTTABLE - Tabelle sortieren

.SORTIERTABELLE	.SORTTABLE
deutsch	englisch
<code>.SORTIERTABELLE -MASKE</code> 'Ordnungsangabe'	<code>.SORTTABLE -MASK 'Order specification'</code>

.SORTIERTABELLE	.SORTTABLE
Sortiert die Tabelle im Maskenpuffer nach der angegebenen Ordnungsangabe. Es sind sowohl Datenbankpuffer, als auch Infosystempuffer möglich. Das Kommando kann aber nicht bei allen Puffern verwendet werden. Dort wo man nicht in der Maske mit Mausclick auf die Spaltenüberschrift sortieren kann, kann auch das Kommando .sortiertabelle nicht verwendet werden. Das Argument das als Ordnungsangabe anzugeben ist, entspricht exakt der Syntax, wie bei der Selektionsoption @ordnung.	

Mit der Ordnungsangabe kann angegeben werden, nach welchen Variablen sortiert werden soll. Es werden bis zu zehn durch Komma voneinander getrennte Variablen oder Verknüpfungen erwartet. Bei jeder Variablen oder Verknüpfung können Sie zusätzlich angeben, ob vorwärts oder rückwärts sortiert werden soll.

Die Richtungsangabe folgt auf die Namensangabe und darf abgekürzt werden. Fehlt die Richtungsangabe, wird vorwärts sortiert.

Beispiel: Tabelle Preise/Rabatte sortieren nach der Spalte Menge

Das Programm sortiert im zuvor geladenen Puffer nach dem Feld **mgrenze** aufsteigend.

```
..!interpreter english declaration noabbrev
.sorttable -mask mgrenze
? Sortieren nicht möglich ? G|success = G|false
```

Beispiel: Tabelle Preise/Rabatte sortieren nach der Spalte Preis und Naturalrabatt

Das Programm sortiert im zuvor geladenen Puffer nach dem Feld **mpreis** aufsteigend und nach dem Feld **mnrab** absteigend.

```
..!interpreter english declaration noabbrev
.sorttable -mask mpreis,mnrab.rueckwaerts
? Sortieren nicht möglich ? G|success = G|false
```

.HOLE MASKE ZEILE / .SELECT MASK LINE - Zeilenselektion im geladenen Puffer

.hole [amaskeamaskeamaske] zeile 'Selektionsstring'

Um eine Zeile in einem geladenen Puffer zu suchen, kann ein Selektionsstring verwendet werden. Es sind sowohl **\$**-Selektion, als auch **%**-Selektion möglich. Zudem können Optionen wie **@bereich** oder **@richtung** angegeben werden.

Keinen Sinn machen Optionen, die sich auf die Suche von Objekten beziehen; diese sind in einer geöffneten Objektauswahl mit einem Schreibschutz versehen:

@gruppe, @zeilen, @sort, @ablageart.

Beispiel: Zeilenselektion im geladenen Puffer

Das Programm sucht im zuvor geladenen Puffer nach der Zeile, für die die Bedingung `vname=tename11` gilt.

```
.select company "V-02-01"
'G|success' - 'H|id'
.select line "$,varName=tename11;@language=en"
'G|success' - 'H|rowNo'
'H|varName'
```

Ausgabe:

```
tename11
```

Beispiel: Weitersuchen nach dem ersten Treffer

Das Programm lädt ein Teil und sucht dort rückwärts im Tabellenteil nach Zeilen, die auf die angegebene Bedingung passen.

```
.select parts "V3"
.continue END          ? G|success = G|false
.select line "$,timeLimUnit=10;@range=yes;@direction=backward;@language=en"
?rowNo: 'H|rowNo'      ? G|success = G|true
.select line
?rowNo: 'H|rowNo'      ? G|success = G|true
!END
```

Wird während einer Zeilenselektion explizit auf eine bestimmte Zeile positioniert (`.hole zeile 'Zeilennummer'` oder `.hole zeile 'Zeilen-ID'`), dann bleibt die Zeilenselektion dennoch aktiv.



Das bedeutet, dass ein darauf folgendes `.hole zeile` nicht auf die Zeile nach der gerade angesprungenen Zeile positioniert, sondern auf die nächste Zeile der Zeilenselektion.

Eine Zeilenselektion beendet immer einen u.U. vorhandenen Bereich in einer darüberliegenden Selektion der Objekte.

.HOLE METADATEN / .HOLE METADATA - Zugehörige Schemadaten laden

.HOLE METADATEN 'Pufferangabe' .LADE X METADATEN 'Pufferangabe'	.SELECT METADATA 'Pufferangabe' .LOAD X METADATA 'Pufferangabe'
<code>.HOLE METADATEN "M "</code>	<code>.SELECT METADATA "M "</code>
<code>.LADE X METADATEN "M "</code>	<code>.LOAD X METADATA "M "</code>
<code>.LADE X METADATEN "H "</code>	<code>.LOAD X METADATA "H "</code>
<code>.LADE X METADATEN "Ø "</code>	<code>.LOAD X METADATA "Ø "</code>

Die Pufferangabe umfasst einen Pufferbuchstaben eines gefüllten Puffers, gefolgt von einem |-Zeichen. Das Ergebnis ist ein mit einem Datensatz einer Variablentabelle oder eines Infosystems geladener H- oder Lade-Puffer.

Sie können mit diesem Befehl das Schema des Datensatzes ermitteln, der als Parameter im angegebenen Puffer geladen ist. Sie können diese Methode in FOPs verwenden, die in vielen Kontexten aufgerufen werden, und die deshalb dynamisch ermitteln müssen, welches Objekt in einem Puffer liegt. Im Standard gibt es eine Verwendung dieses Konstruktes in der Implementierung des [I-Buttons](#).

Es gibt folgende Ergebnisse:

- Ein Variablentabellensatz, wenn ein Datensatz im angegebenen Puffer liegt. Beim H- oder Lade-Puffer ist dies das einzig mögliche Ergebnis.
- Ein Infosystemdatensatz, wenn ein Infosystem im angegebenen Puffer liegt, nur beim M- oder A-Puffer.
- Ein neuer Infosystemdatensatz, wenn eine Selektion im angegebenen Puffer liegt, nur im EFOP einer Selektion.

.LADE / .LOAD - Datensätze laden

.LADE	.LOAD
deutsch	englisch
<code>.LADE X 'Datenbank/Gruppenangabe'</code> <code>'Selektionsangabe'</code>	<code>.LOAD X 'database/group specification'</code> <code>'selection specification'</code>
<code>.LADE X GRUPPE 'Gruppenangabe'</code> <code>'Selektionsangabe'</code>	<code>.LOAD X GROUP 'group specification'</code> <code>'selection specification'</code>
<code>.LADE X 'Datenbank/Gruppenangabe'</code> <code>.LADE X DAZU 'Verweisfeld'</code>	<code>.LOAD X 'database/group specification'</code> <code>.LOAD X ADD 'reference field'</code>
<code>.LADE X WEITER</code>	<code>.LOAD X CONTINUE</code>
<code>.LADE X</code>	<code>.LOAD X</code>
<code>.LADE X ZEILE ['n']</code>	<code>.LOAD X LINE ['n']</code>

.LADE	.LOAD
<code>.lade X 'Datenbank/Gruppenangabe'</code> lädt den nächsten Datensatz eines zuvor geladenen Bereichs mit <code>.lade X 'Datenbank/Gruppenangabe' 'Selektionsangabe'</code>. <code>X</code> steht für einen <code>.lade</code>-Puffer, der durch eine Ziffer von 0 bis 9 repräsentiert wird. Daneben gibt es auch Puffernamenvariablen. Ohne weiteres Argument setzt keine neue Suche auf. <code>'Datenbank/Gruppenangabe'</code> steht für einen Datenbanknamen. Als Gruppenangabe können Datenbanken, Gruppenlisten oder Verweisarten verwendet werden. <code>.lade X gruppe 'Gruppenangabe' 'Selektionsangabe'</code> definiert einen Bereich mit Gruppeneinschränkung und lädt den ersten Datensatz des Bereichs. <code>.lade X dazu 'Verweisfeld'</code> definiert einen Bereich mit maximal einem Element, nämlich dem Datensatz, auf den <code>'Verweisfeld'</code> zeigt. <code>.lade X weiter</code> schaltet auf das nächste Objekt im Bereich. <code>.lade X</code> löscht Puffer <code>X</code> und Bereich. <code>.lade X zeile ['n']</code> lädt die nächste bzw. die <code>n</code>-te Zeile bei Objekten mit Tabelle bzw. im Ein- und Verkauf. Der Bereich und das geladene Objekt bleiben erhalten.	

Als **Datenbank** sind dieselben Namen wie beim **Datenbankkommando** in der Kommandosyntax oder beim Kommando `.hole` zulässig. Auch der Oberbegriff "Objekt" ist erlaubt.

Sperroptionen und Sperrmodifikatoren finden Sie im Kapitel **Sperroptionen bei .belegen, .belegen und .belegen**.

Bei einer Schleife mit `.hole 'Datenbank'` beginnt, nachdem `G|mehr = G|false` wurde, die Suche wieder von vorne, und zwar mit leerer Selektionsbedingung. Bei `.lade` ist das nicht so. `G|mehr` bleibt hier `G|false` und es werden keine neuen Objekte geladen.

Zustände des .lade-Puffers

Puffer und Bereich leer	Initialer Zustand beim Starten des FOPs. Keine Feldzugriffe möglich.
Puffer belegt und Bereich definiert	Vorige <code> </code> -Operation setzte <code>G mehr</code> auf <code>G true</code> .

Alle `.lade`-Operationen sind auf leeren und gefüllten `.lade`-Puffern möglich. Ob ein Puffer `X` gefüllt ist, kann abgefragt werden mit

`... ? F|defined(X|id)`

Wann wird G|mehr auf G|true gesetzt?

<code>.lade X 'Datenbank/Gruppenangabe'</code>	Lädt den nächsten Datensatz eines zuvor geladenen Bereichs.
--	---

<code>.lade X dazu 'Verweisfeld'</code>	Der Verweis ist nicht leer, das Objekt, auf das verwiesen wurde, wurde geladen.
<code>.lade X weiter</code>	Der Puffer war vorher gefüllt, und es konnte ein nächstes Objekt geladen werden. Nach <code>.lade X dazu 'Verweis'</code> wird <code>.lade X weiter G mehr</code> immer auf <code>G false</code> setzen.
<code>.lade X</code>	Liefert niemals <code>G mehr = G true</code> , der <code>.lade</code> -Puffer ist anschließend immer leer.
<code>.lade X zeile ...</code>	Ist eine Ausnahme: Es ändert den Ladezustand nicht, setzt nur auf die passende Zeile. <code>G mehr</code> ist <code>G true</code> , falls es die betreffende Zeile im Puffer gibt.
	 Die Regel ist: Wenn nach <code>.lade X G mehr</code> auf <code>G true</code> gesetzt ist, ist der Puffer immer gefüllt.

Wann wird `G|mehr` auf `G|false` gesetzt?

Der Bereich ist erschöpft oder undefiniert.



Die Regel ist: Wenn `G|mehr` nach `.lade X` auf `G|false` gesetzt ist, ist der Puffer leer.

Feldzugriffe sind nur mit Pufferangabe möglich, also z.B. `'1|nummer'`. Auch die pufferbezogenen Prozeduren funktionieren (z.B. `'x|selzeile'`).

Beispiel:

Einfaches Beispiel eines Bereichs

```
.load 0 customer "$,idno=4!5;@language=en"
'G|success'
.load 0 continue
'G|success'
.load 0 continue
'G|success'
```

Beispiel für das Löschen eines Bereichs

```
.load 0 customer "$,idno=4!5;@language=en"
?'0|idno'      ? G|success = G|true
.load 0
?'0|idno'      ? G|success = G|true
```


Beispiel mit **.dazu**

```
.load 0 part "V3"
?'0|idno' - '0|swd' ? G|success = G|true
.. load vendor
.load 0 add 0|vendor
?'0|idno' - '0|swd' ? G|success = G|true
```

selzeile dient zum Ansprechen der gefundenen Zeile bei einer Zeilenselektion:

```
-----
.lade 0 objekt 'M|select'
.weiter END ? _G|mehr
.. wobei M|select eine Zeilenselektion mit dem Selektionssyntaxbestandteil @zeilen=ja ist
(siehe Selektionssyntax)
..
!NEXTROW
.lade 0 zeile '0|selzeile'
.weiter END ? _G|mehr
.. Hier passiert irgendeine Verarbeitung mit der Zeile
..
.. Nach der Verarbeitung, der mit 0|selzeile geladenen Zeile, soll die nächste Zeile zur
Bearbeitung geladen werden
.lade 0 objekt
```

.BRINGE / .REWRITE - Datensatz zurückschreiben

.BRINGE	.REWRITE
deutsch	englisch
.BRINGE	.REWRITE
.BRINGE MASKE	.REWRITE SCREEN
.BRINGE SPERREN	.REWRITE LOCK
.BRINGE PLAN	.REWRITE PLAN

.BRINGE	.REWRITE
.mache schreibt den in H.mache geladenen Datensatz zurück. Der H-Puffer kann durch .mache oder durch .mache gefüllt worden sein. Dieses Kommando entspricht im Wesentlichen dem Abspeichern nach einer Maskenveränderung. .bringe maske schreibt den Datensatz im Puffer M beim Ereignis Maskenaustritt zurück. .bringe sperren testet, ob .bringe durchgeführt werden kann. Wenn nicht, bricht das Programm mit einer Fehlermeldung ab. .bringe plan schreibt Objekte nach Änderung zurück und veranlasst die Disposition, die Änderung in der Planung zu berücksichtigen.	

.bringe schreibt den im Puffer H geladenen Datensatz zurück. Erst damit werden Änderungen am Datensatz dauerhaft wirksam. Dieses Kommando entspricht im Wesentlichen der Betätigung der Taste <Fertig> nach einer Maskenänderung. Bei Ein- und Verkaufsobjekten ist Folgendes zu beachten: Auch wenn alle Positionszeilen in derselben Maske dargestellt werden, so handelt es sich doch um eigenständige Datensätze. Der Befehl **.bringe** schreibt also immer nur die zuletzt mit **.hole sperren zeile** beschaffte Position (oder nur den Kopf) zurück. Hingegen wird z.B. bei einem Teil der gesamte Datensatz einschließlich Tabelle zurückgeschrieben.

Das Rückschreiben versionierter Datensätze ist nicht möglich und führt zum Fehlerabbruch.



Das Kommando **<Ändern>** kann mit der Erlaubnis **<Ändern>** ausgeführt werden; die Erlaubnis **<Ändern>** ist nicht notwendig.



Beim Befehl **.bringe** werden die Felder "Letzter Bearbeiter" ((it)zeichen) und "Änderungsdatum" ((it)stand) im Änderungs-Info zum Datensatz nicht aktualisiert.

Näheres hierzu: **.hole maske - Zeilenoperationen in ereignisgesteuerten FOPs (EFOPs) und Datenbankkommandos**.

.bringe plan - Zurückschreiben und Disposition informieren

Mit **.bringe plan** schreibt man Objekte nach Änderung durch die FO in die Datenbank zurück und - das bedeutet die Option **plan** - informiert die Disposition, damit diese im nächsten Lauf überprüft, ob diese Änderung Einfluss auf die momentane Planung hat und ggf. die Konsequenzen dieser Änderung in das Planungsgerüst einpflegt.

Folgende Objekte lassen sich durch **.bringe plan** in die Datenbank zurückschreiben:

Artikel (V-02-01)	löst eine Bedarfsrechnung für den Artikel in der internen Lagergruppe aus
----------------------	---

Artikellagergruppeneigenschaft (V-39-03)	löst eine Bedarfsrechnung für den Artikel und die Lagergruppe aus der Artikellagergruppeneigenschaft aus
offene Verkaufsposition ("ofob") vom Typ ARPOS (Artikelposition) (V-03-02)	löst eine Terminrechnung für die Verkaufsposition aus
offene Einkaufsposition vom Typ ARPOS (Artikelposition) offener externer Beschaffungsvorschlag (V-04-02)	löst eine Terminrechnung für die Einkaufsposition bzw. den externen Beschaffungsvorschlag aus
offener Fertigungsvorschlag (V-04-05)	löst eine Terminrechnung für den Fertigungsvorschlag aus

Der Versuch, andere als diese Objekte per **.bringe plan** zurückzuschreiben, wird mit der Fehlermeldung "Was tun?" quittiert.

Bei Änderung folgender Felder in Artikeln und Artikellagergruppeneigenschaften wird die Disposition bei **.bringe** auch ohne **plan** informiert:



- Dispositionsart
- Mindestbestand
- Minimale und maximale Beschaffungsmenge
- Losgröße
- Losbildungsfrist

.MACHE / .MAKE - Anlegen und Löschen von Datensätzen bzw. Zeilen

.MACHE	.MAKE
deutsch	englisch
.MACHE	.MAKE
.MACHE 'Datenbank' ['Gruppe']	.MAKE 'file name' ['group']
.MACHE [MASKE] ZEILE [+]	.MAKE [SCREEN] LINE [+]
.MACHE [MASKE] ZEILE -	.MAKE [SCREEN] LINE -
.MACHE [MASKE] ZEILE --	.MAKE [SCREEN] LINE --
.MACHE MASKE ZEILE +[0]	.MAKE SCREEN LINE +[0]
.MACHE MASKE ZEILE -0	.MAKE SCREEN LINE -0

.MACHE	.MAKE
.mache ermöglicht das Anlegen bestimmter Datensätze (Zusatztabelle, EDI, Inventur, Speditionsauftrag, Charge, Dokument) und das Anlegen und Löschen von Zeilen in beliebigen Datenbanken. .mache 'Datenbank' ['Gruppe'] legt einen neuen Datensatz im Puffer H an; möglich in folgenden Datenbanken: Zusatztabelle, EDI, Inventur, Speditionsauftrag, Charge, Dokument, Behälter. Ohne Angabe einer Gruppe wird von der FO ein Datensatz in Gruppe 0 erzeugt. Für 'Gruppe' ist gruppe0, gruppe1, ..., gruppe19 anzugeben. .mache [maske] zeile [+] legt eine neue Zeile im M-Puffer (maske) an. Die Zeile wird nach der aktuellen Zeile eingefügt und auf die erzeugte Zeile wird positioniert. .mache [maske] zeile - entfernt die aktuelle Tabellenzeile, .mache [maske] zeile -- entfernt alle Tabellenzeilen. .mache maske zeile +[0] erhöht die Geschwindigkeit beim Einfügen, .mache maske zeile -0 erhöht die Geschwindigkeit beim Löschen vieler Zeilen einer Maske. (0 für "Optimierung"). Zwischen + bzw. - und 0 darf kein Leerzeichen stehen. Wenn Sie per EFOP Zeilen einfügen oder löschen, bzw. Farben ändern, beachten Sie die Hilfe zum Kommando .farbe.	

Anlegen von Datensätzen

.mache 'Datenbank' ['Gruppe']

Das Kommando **.bringe** verändert die Variable **G|mehr** nicht, da der Befehl - sofern syntaktisch korrekt - immer erfolgreich ist. Erst mit **.bringe** wird der neu angelegte Datensatz tatsächlich geschrieben. Das Schreiben setzt voraus, dass die Pflichtfelder der jeweiligen Anwendung gesetzt wurden. Beim Schreiben wird eine neue, fortlaufende Identnummer vergeben, falls noch keine zugewiesen wurde.

Für **'Datenbank'** ist wie bei **.mache** und **.mache** der Name einer Abas Datenbank einzusetzen. Allerdings sind bei **.mache** nur folgende Datenbanken möglich: Zusatztabelle, EDI, Inventur, Speditionsauftrag, Charge und Dokument.

Ein zuvor im H-Puffer befindlicher Datensatz wird entladen und ein neuer Datensatz der gewünschten Datenbank und Gruppe angelegt. Wie üblich können Felder mittels **.formel** gesetzt werden, sofern sie beschreibbar sind. Der Datensatz steht (im Gegensatz zum Befehl **.hole**) unter voller Kontrolle der Anwendung, also genau so, als würde er in einer Maske bearbeitet werden. Es gilt daher die dortige Anwendungslogik z.B. in Bezug auf Schreibschutz und Pflichtfelder, d.h. es werden die **Anwendungsmethoden** aufgerufen. Das FO-Schreibrecht für eine Variable in der **Vartab** ist also für das Ändern nicht erforderlich.

Das folgende Beispiel legt einen Datensatz der Zusatztabelle 12, Gruppe 0 an.
Die Identnummer wird erst durch **.bringe** erzeugt, wenn sie nicht vorher noch gesetzt wird.

```
.make File12
```



.make akzeptiert KEINE Variablen als Argument für den Datenbanknamen, also z.B. nicht die Systemvariable '**hdatname.i**'.

Welche Gruppen angelegt werden dürfen und welche Pflichtfelder gesetzt werden müssen, ist anwendungsabhängig. Dazu die folgende Tabelle:

Anwendung	Mögliche Gruppen
Zusatztable	alle im Betriebsdatensatz definierten Gruppen (vgl. unter Betriebsdaten Feld zdgrp in <Firma> <Betr-Auftrag>)
Speditionsauftrag	Gruppe 1
EDI	Gruppe 0
Inventur	Gruppe 1
Charge	Gruppe 0
Dokument	Gruppe 1 und 2
Behälter	Gruppe 12

Anwendungsbeispiele

Beispiel: Speditionsauftrag

.make **spedition** **gruppe1**

erzeugt einen Speditionsauftrag. Dieser kann nun in Verkaufslieferscheinen verwendet werden.

```
.make transportcompany group1
.formula H|goodsRecipient = "4"
.rewrite
```

Beispiel: Preisauskunft

Mit **.make** **preise** **gruppe2** kann auf die **Preisauskunft** zugegriffen werden.

```
.make pricing group2
.formula H|custVendor = "1"
.formula H|product = "203"
.formula H|unitQty = 2
price:      'H|price'
entire value: 'H|itemValEntCurr'
```

Beispiel: Behälter

Mit **.make behaelter gruppe12** werden Artikelbestände auf einem Lagerplatz mit den verwendeten Behältern, sowie zusätzlich die unverpackten Lagerbestände ausgegeben. Je nach Ausprägung (z.B. Verwendung oder Charge) wird die summierte Menge des Bestandes angezeigt und die Übersichtlichkeit erhöht. Das Kommando wird, auch ohne, dass Behälter im Lager verwendet werden, in folgenden Infosystemen des Standardlieferumfangs verwendet:

Bestandsinfo (BESTAND)

Umlagerung mehrerer Artikelbestände (SQRELOCATION)

Lagermengenanalyse (SQANALYSIS)

```
..!interpreter english translate noabbrev
.input "DATEI.F"
..Vorbelegung des Kopfes der Behälterlagermengen
.make container group12
.formula H|artikel="E1"
.formula H|platz="F1"
..Verdichtungsebene wählen
..Standard: Verdichtung auf Lagerplatz
.copy H|platzverdichten = G|true
..copy H|lagerverdichten = G|true
..copy H|lgverdichten = G|true
..copy H|artverdichten = G|true
.formula H|ladetab = ""
..
Verdichteter Bestand für
Artikel: 'H|artikel'
Lagerplatz: 'H|platz'
L-Gruppe Lager L-Platz Artikel Menge Faktor Einheit Behälter Charge Verwendung
.select header
.select line
.continue END ? _G|mehr
!NXTLINE: nächste Zeile
'H|tlgruppe' 'H|tlager' 'H|tplatz' 'H|tartikel' 'H|mge' 'H|gebf' 'H|gebeinh' 'H|tbehaelter'
'H|ztcharge' 'H|verw'
..
.select line
.continue NXTLINE ? G|mehr
!END
```

Anlegen und Löschen von Zeilen

.make [maske] zeile [+|-|--] [+0|-0] legt eine Zeile an bzw. löscht eine oder alle Zeilen. Im **Dialogzustand** ist das Anfügen und Löschen von Zeilen erlaubt, wenn die Anwendung es erlaubt, d.h. wenn der Benutzer es darf. Im **Festplattenzustand** ist es nur in neu angelegten Objekten und in Zusatztabelle erlaubt.

.make [maske] zeile +

hängt eine Zeile an die aktuelle Tabellenzeile an; gleichzeitig wird auf die erzeugte Zeile positioniert. Ein Anfügen nach dem Kopfteil ist gleichbedeutend mit einem Einfügen vor der ersten Tabellenzeile bzw. mit

dem Erzeugen der ersten Tabellenzeile. Zeilen, die nur leere Felder enthalten, werden bei **.bringe** herausgefiltert.

.mache zeile kann nur auf dem Maskendatensatz (Zusatz **maske**) oder einem gesperrten Datensatz (**.hole sperren**) oder einem neu erzeugten Datensatz (**.mache**) ausgeführt werden.

Optimierung

.mache maske zeile +0|-0 (O für "Optimierung") beschleunigt das Einfügen bzw. Löschen von Zeilen. Die Option wirkt sich nur beim M-Puffer aus. Ohne Optimierung werden alle Befehle der Art

```
.formel M|Feldname = Wert
.mache [maske] zeile [+]
.mache [maske] zeile -
```

in der Maske sofort sichtbar. Mit der Optimierung wird die Maske erst am Ende des EFOPs aktualisiert.



Sie sollten die Optimierung nicht vornehmen, wenn das FOP noch Eingaben erfordert und Sie das Ergebnis der bisherigen Befehle in der Maske sehen wollen.



Die Optimierung lohnt sich nur, wenn eine gewisse Anzahl von Zeilen verändert wird. Selbst wenn nur einige wenige Tabellenzeilen eingefügt oder entfernt werden, die Tabelle aber bereits sehr viele Zeilen hat, müssen alle Zeilen (in der GUI nach dem Ende des EFOPs) neu aufgebaut werden. Das unmittelbare Einfügen oder Löschen von wenigen (<100) Tabellenzeilen geht in der GUI schneller als der völlige Neuaufbau von z.B. 1000 Zeilen.

Wenn dagegen (z.B. per Button **Tabellenzeilen laden**) eine Tabelle völlig neu aufgebaut werden muss, lohnt sich **+0|-0** immer; man sollte es dort auch verwenden, weil das Dialogverhalten schöner aussieht.

In EFOPs gilt: Folgt nach der FOP-Ereignisbehandlung noch eine Abas Ereignisbehandlung, so muss sichergestellt werden, dass die Ereigniszeile nicht verschoben oder gelöscht wird. Die nachfolgenden Ereignisbehandlungen würden sonst auf eine falsche Zeile angewendet werden. In diesem Fall ist daher das Löschen oder Einfügen von Zeilen bis zur aktuellen verboten. Wird es dennoch ausgeführt, bricht das EFOP mit einer entsprechenden Fehlermeldung ab. Davon betroffen sind z.B. die **Ereignisse feldfuell**, **feldpruef**, **buttonvor**, **zeileloevor**, **zeileeinvor**. Das Löschen oder Einfügen von Zeilen muss hier auf das jeweils abschließende Ereignis verlagert werden, also **feldaus**, **buttonnach** usw.

Verfügbarkeitsprüfung per FOP auswerten

```
.mache mzuordnung ['Gruppe']
```

Mit diesem Befehl können Sie prüfen, wie viel von Ihrem Artikel zu welchem Termin verfügbar, also ohne anhängende Reservierung, ist. Mit dem Befehl können keine Reservierungen erzeugt werden.

Beispiel: Verfügbarkeitsprüfung per FOP

```

..*****Maske Verfügbarkeit per FOP bedienen*****
.mache mzuordnung gruppe4
.. Herkunft ist matbedarfobj
.. vorgang^id ist die angefragte Position, auf die geprüft werden soll - hier aus einer Maske
.formel H|matbedarfobj = M|vorgang^id
.. Wunschtermin ist heute
.formel H|verfuegterm = "."
..
..*****Drücke Button Verfügbarkeitsprüfung*****
.formel H|verfuegpruef = ""
..*****Ergebniswerte in Kopf*****
.hole kopf
.. verfuegmge : Verfügbare Menge
.. verfueglmge : Verfügbare Menge auf Lager
.. verfuegbsmge : Verfügbare Menge durch Beschaffungen
'H|verfuegmge' 'H|verfueglmge' 'H|verfuegbsmge'
..*****Schleife über die Zeilen*****
.hole zeile
.weiter ENDE ? _G|mehr
!NEXTVERF
..*****Ausgabe der Termine*****
'H|mge ' 'H|einheit' 'H|tterm'
..
.hole zeile
.weiter NEXTVERF ? G|mehr
..*****Ende Programm*****
!ENDE

```

.LOESCHEN / .DELETE - Datensatz löschen

.LOESCHEN	.DELETE
deutsch	englisch
.LOESCHEN	.DELETE
löscht einen Stammdatensatz	

Einem **.loesch**-Kommando muss ein **.hole sperren**-Kommando vorausgegangen sein, d.h. der zu löschende Datensatz muss zuvor mit Sperre gelesen worden sein. Gelöscht werden nur Stammdaten. Für die FO bleibt der mit **.loesch** entfernte Datensatz weiterhin geladen, so dass auf seine Felder zugegriffen werden kann. Lediglich seine Datensatznummer wird auf 0 gesetzt.

Das Löschen versionierter Datensätze ist mit diesem Kommando nicht möglich und führt zum Fehlerabbruch.

Siehe auch Kapitel [Diverse Kommandos](#).



Nach **.loesch** werden alle **.merke**-Puffer gelöscht, da sich ein gelöscht Objekt

noch in einem der gemerkten Puffer (**.merke**, **.merke lifo**, **.merke fifo**) befinden kann.

.AKTION / .ACTION - Transaktion

.AKTION	.ACTION
deutsch	englisch
.AKTION + - .AKTION ABORT	.ACTION + - .ACTION ABORT
.aktion + - markiert einen Programmabschnitt mit zusammengehörigen Schreibzugriffen auf die Datenbank als Transaktion. .aktion ABORT bricht eine laufende Transaktion im FOP ab und macht ihre Auswirkungen auf die Datenbank rückgängig. Sonstige Wirkungen, z.B. auf das Dateisystem, werden nicht zurückgenommen.  Wenn in dem Transaktionsabschnitt Datenbankänderungen per Lader, AJO im ClientContext, EDP oder JEDP durchgeführt werden, so werden diese nicht Bestandteil der Transaktion, da sie in einem anderen Prozess stattfinden.  .aktion ABORT hat zur Folge, dass das Programm abbricht.	

.aktion +|- führt die Schreibzugriffe innerhalb der Transaktion alle gemeinsam oder gar nicht durch. Dies bedeutet im Einzelnen:

- Alle Änderungen werden erst beim Durchlaufen von **.aktion** - wirksam.
- Alle Sperren werden bis zum Durchlaufen von **.aktion** - gehalten.
- Um alle bisherigen Änderungen unwirksam werden zu lassen (absichtliches Rollback), reicht es, **.aktion ABORT** aufzurufen. Allerdings bricht dadurch der gesamte Prozess ab.

Durch diese Regeln ist die Datenkonsistenz gewährleistet.



Für die Anzahl der Schreibzugriffe pro Transaktion gibt es einen **Maximalwert**. Wird dieser überschritten, bricht das Programm ab. Dies hat zur Folge, dass die Transaktion zurückgerollt wird.

Die Klammerung mit **.aktion** ist immer dann zu verwenden, wenn die Anwendung das gemeinsame unteilbare Schreiben mehrerer Datensätze erzwingt. Als Beispiel dafür sei eine Buchung genannt, die mehrere Verkehrszahrendatensätze konsistent ändern muss, damit die Kontenbilanz erhalten bleibt. Ein Gegenbeispiel ist eine Massenänderung, z.B. von Stammdaten. Hier ist die Änderung eines Datensatzes unabhängig von der Änderung eines anderen. Das gilt auch dann, wenn Sie z.B. andere Aktionen erst nach der erfolgreichen Beendigung der Massenänderung durchführen können.

Hinweise:

Benutzereingaben	Zusammengehörige Schreibzugriffe sollen möglichst schnell abgewickelt werden und möglichst keine Benutzereingaben enthalten. Sind Anfragen an den Benutzer notwendig, lassen sie sich fast immer problemlos vor den Beginn der Zusammenfassung verlagern.
<Abbruch>	Wenn in einem FOP zusammengehörige Schreibzugriffe vorkommen, so sollte dieser Abschnitt mit <code>.setze abbruch -</code> <code>.setze abbruch --</code> durchlaufen werden.
Implizites Transaktionsende, Fehlerfall	Von einem impliziten Transaktionsende spricht man, wenn das abschließende <code>.aktion + -</code> nicht durchlaufen wird, z.B. weil es vergessen wurde oder weil das FOP wegen eines Fehlers abbrach. In diesem Fall findet nur dann ein Rollback statt, wenn das FOP unter unbedingter Abbruchsperre lief (<code>.setze abbruch --</code>).

Die Verwendung von `.aktion` ist grundsätzlich nur dann notwendig, wenn `.aktion` mehrmals durchlaufen wird. Beim Maskenaustritt ist beim Ändern stets noch die Transaktion der Maske aktiv, die Verwendung von `.aktion` ist daher unnötig, aber zulässig.

Die Verwendung von `.aktion` wirkt sich auch auf das Sperrverhalten aus: Grundsätzlich werden alle Lese- und Schreibsperrungen bis zum Ende der Transaktion gehalten. Im Ein- und Verkauf und in der Fertigung wird grundsätzlich mit Lesesperre gelesen. Das Halten der Sperren wirkt nur nach "außen", also gegen andere Prozesse. Es ersetzt nicht das Kommando `.hole sperren`.

Nähere Informationen zu Sperroptionen: [Sperroptionen bei .hole, .lade und .belegen](#).

Eine Schachtelung ist erlaubt, die Befehle `.aktion +` und `.aktion -` müssen allerdings immer paarweise verwendet werden. Offene Vorgänge, bei denen der Partner `.aktion -` fehlt, werden bei `.ende` erkannt und führen zu einer Fehlermeldung. Das Gleiche gilt, wenn das FOP mit Fehler beendet wird. Die Transaktion wird aber nur bei `.setze abbruch --` zurückgesetzt. Ansonsten wird die fehlende schließende Klammer `.aktion -` ergänzt.

`.setze unterbrechung` darf nur außerhalb von mit `.aktion` geklammerten Abschnitten vorkommen.

.aktion ABORT

Eine laufende Transaktion kann im FOP mit `.aktion ABORT` abgebrochen werden. Damit werden zugleich alle ihre Auswirkungen auf die Datenbank zurückgenommen. Sonstige Wirkungen, z.B. auf das Datenbanksystem, werden nicht rückgängig gemacht und unterliegen somit der Verantwortung des FO-Programmierers. Vor der Ausführung des Kommandos werden der laufende Serverprozess heruntergefahren und alle betroffenen Fenster geschlossen. Zuvor erscheint eine Meldung, die den Namen und die Zeilennummer des FOPs enthält, das den Abbruch veranlasst.

Wann laufen Transaktionen?

Eine Transaktion wird entweder explizit mit **.aktion +** vom FOP aus oder, bei einem Maskenaustritts- und beim Workflow-Ereignis in einem EFOP, von Abas gestartet.

Was geschieht bei geschachtelten Transaktionen?

Eine Klammerung mit

```
.aktion +
..
.aktion -
```

darf auch geschachtelt verwendet werden. Damit wirkt sie jedoch nicht anders als eine einzige große Transaktion. Ein Abbruch mit **.aktion ABORT** setzt somit immer die gesamte, also die äußerste offene Transaktion zurück.

Wie ist das Verhalten in Java-FOPs?

Derzeit ist das freiwillige Zurückrollen einer Transaktion nur um den Preis des Abbruchs des Serverprozesses möglich. Java-FOPs laufen in einem eigenen Prozess, der mit dem Serverprozess kommuniziert. Dieser Java-Prozess bricht nicht ab, sondern erhält eine `CommunicationException`. Diese ist eine `RuntimeException`. Falls das JFOP sie abfängt, kann es noch allgemeine Aufräumarbeiten durchführen; Datenbankaktionen oder andere Methodenaufrufe der FO-Klasse sind aber nicht mehr möglich. Im Normalfall wird die `CommunicationException` nicht abgefangen, woraufhin sich das JFOP beendet.

Anwendungsspezifische Datenbankkommandos

Verkehrszahlen

Eine Reihe spezieller Schlüsselwörter ermöglicht die Formulierung besonderer Bedingungen. Bei **.hole vkz** z.B. werden die monatlichen Verkehrszahlen und Umsatzzähler von Kunden, Lieferanten, Mitarbeitern, Artikeln, Sachkonten und Anlagen geladen.

Beispiel:

Ablauf 1	Ablauf 2	Ablauf 3	Ablauf 4
.hole jahr 02 .hole kunde .. .hole vkz ist .dazu vkz plan	.hole jahr 02 .hole kunde .. .hole vkz ist 03 .dazu vkz plan 03	.. .hole kunde .. .hole vkz ist ..	.. .hole kunde .. .hole vkz ist 01 .dazu vkz ist 02
Ausgabe	Ausgabe	Ausgabe	Ausgabe
Ist 'H s1' Plan 'D s1'	Ist03 'H s1' Plan03 'D s1'	Ist 's1'	Ist01 'H s1' Ist02 'D s1'

Zur Notation von **H|** und **D|** siehe Kapitel [Pufferbuchstaben](#).

Erklärung zu den Abläufen

Ablauf 1	Vergleich der Ist-/Planverkehrszahlen für das Geschäftsjahr 02
Ablauf 2	Vergleich der Ist-/Planverkehrszahlen für das Geschäftsjahr 03. Die Angabe des Geschäftsjahres mit dem Kommando <code>.hole vkz ...</code> oder mit dem Kommando <code>.dazu vkz ...</code> hat eine höhere Priorität als ein zuvor ausgeführtes Kommando <code>.hole jahr</code> .
Ablauf 3	Ist-Verkehrszahlenausgabe für das aktuelle Geschäftsjahr. Falls das Geschäftsjahr nicht definiert ist, wird das Standardgeschäftsjahr herangezogen.
Ablauf 4	Vergleich der Ist-Verkehrszahlen 01 mit Ist-Verkehrszahlen 02. Bevor die Verkehrszahlen selbst geholt werden, muss der Herkunftsdatensatz (Kunde, Lieferant, Artikel, Anlage usw.) der Verkehrszahlen bestimmt werden. Bevor die Kommandos <code>.hole vkz</code> oder <code>.dazu vkz</code> aufgerufen wurden, kann ein Geschäftsjahr bestimmt werden. Vor allem beim Vergleich über mehrere Geschäftsjahre ist es vorteilhaft, das Geschäftsjahr an das <code>.hole vkz</code>-Kommando bzw. <code>.dazu vkz</code>-Kommando anheften zu können. Das Geschäftsjahr kann auch relativ zum laufenden Geschäftsjahr angegeben werden, z.B. mit dem Kommando <code>.hole vkz ist -1</code>. Es werden die Ist-Verkehrszahlen des alten Geschäftsjahres in den Hauptpuffer geladen.

Beispiel: Setzen des Geschäftsjahres

```

.. -- set fiscal year 2012
.select year 12
.select customer
.. -- select actual monthly debits and credits - fiscal year 2012
.select vkz actual
.. -- select actual monthly debits and credits - fiscal year 2013
.add vkz actual 13
.. -- select planned monthly debits and credits
.select vkz target
.. -- select monthly debits and credits - previous fiscal year
.select vkz actual -1

```

Auch folgende Strukturen werden unterstützt. Ausgabe der Verkehrszahlen für die Kunden 2008-2010 im Geschäftsjahr 2012:

```

.select year 12
.select customer "2008!2010"
!MARK1: get monthly debits and credits of customer
.select vkz actual
.add vkz target
.. display
.select customer
..
.continue MARK1          ? G|success = G|true

```

Auch wenn keine Verkehrszahlen zu einem gültigen Geschäftsjahr existieren, wird ein vollständiger Datensatz mit dem Herkunftsdatensatz der Verkehrszahlen (Kunde, Lieferant, Teil usw.), dem Geschäftsjahr und der Art der Verkehrszahlen zur Verfügung gestellt. Die entsprechenden Verkehrszahlen sind null. Die FOP-Variable **G|mehr** wird auf **G|true** gesetzt.

Fehler, FOP wird abgebrochen

Wenn das Geschäftsjahr, zu dem die Verkaufszahlen geholt werden sollten, nach dem letzten in der Geschäftsjahrestabelle des Termindatensatzes aufgeführten liegt, so wird das FOP nach **.hole vkz ...** abgebrochen. Wenn der im FOP gesetzte Zeitraum sich über mehrere Geschäftsjahre erstreckt, so wird das FOP nach **.hole vkz ...** abgebrochen. Dies kann z.B. geschehen, nachdem der Zeitraum mit dem Kommando **.hole zeitraum** (z.B. **.hole zeitraum 1.1.00 - 31.12.03**) bestimmt wurde.

Laden von Planzahlen in die Datenbank per FOP

Planzahlen von Kunden, Lieferanten, Mitarbeitern, Teile, Konten und Anlagen können per FOP in die Datenbank geladen werden. Der Ablauf kann wie folgt aussehen.

Laden des Herkunftsdatensatzes der Verkehrszahlen	.hole kunde "nummer"
Laden der Verkehrszahlen mit einer Sperre	.hole vkz plan sperren
Monatszähler im Kundenstamm setzen	.formel s3 = s3 +1 oder .formel u12 = 12
Werte in die Datenbank schreiben	.bringe

Das Zurückschreiben inkonsistenter Datensätze wird verweigert. Dies gilt beispielsweise, wenn ein Geschäftsjahr auf zehn Monate verkürzt wurde und die Zähler für die Geschäftsmonate 11 oder 12 ungleich null sind.

.HOLE VKZ / .SELECT VKZ - Verkehrszahlen holen

.HOLE VKZ	.SELECT VKZ
deutsch	englisch
.HOLE VKZ [IST PLAN] ['Geschäftsjahr'] ['Währung']	.SELECT VKZ [ACTUAL TARGET] ['fiscal year'] ['currency']
lädt die monatlichen Verkehrszahlen und Umsatzzähler von Kunden, Lieferanten, Mitarbeitern, Artikeln, Sachkonten und Anlagen	

Bei der Eingabe können Konstanten, Variablen oder Mischformen verwendet werden.

Beispiel: Konstanten, Variablen, Mischform

Konstanten
.hole vkz ist 96-2

Variablen

```
.hole vkz 'xtyp' 'xtgjahr'
```

Mischform

```
.hole vkz ist 'xtgjahr'
```

```
.select parts ""
.add vkz actual "12" 'G|domesticCurrPrev'
```

(**ainland** ist die Inlandswährung vor der EURO-Umstellung.)
 Für beliebige Geschäftsjahre würde es so funktionieren:

```
.type text xtcurrency xtyear
.input VKZRAUM
.select parts ""
.formula U|xtyear = "12"
.formula U|xtcurrency = G|domesticCurrPrev
.formula U|xtcurrency = G|domesticCurrNew ? (G|euroTransData <= G|startDate &
'G|domesticCurrNew' <> "")
.add vkz actual 'U|xtyear' 'U|xtcurrency'
```

Ist bei den Kommandos **.hole** und **.dazu vkz** keine Währung angegeben, wird automatisch die Inlandswährung des ausgewählten Geschäftsjahres ergänzt. Statt der obigen Konstruktion kann also auch geschrieben werden:

```
.type text xtyear
.formula U|xtyear = "12"
.input VKZRAUM
.select parts ""
.add vkz actual 'U|xtyear'
```

Lesen Sie hierzu auch Kapitel [Vorhandensein der Verkehrszahlen - Setzen der FOP-Variablen G|mehr](#).

.HOLE LPLATZ / .SELECT LOCATION - Lagerplatz-Datensatz aus dem Pufferbereich holen

.HOLE LPLATZ	.SELECT LOCATION
deutsch	englisch
.HOLE LPLATZ	.SELECT LOCATION
holt einen gespeicherten Lagerplatz-Datensatz aus dem Pufferbereich	

.HOLE LMENGE / .SELECT QUANTITY - Lagermengen-Datensatz aus dem Pufferbereich holen

.HOLE LMENGE	.SELECT QUANTITY
deutsch	englisch
.HOLE LMENGE	.SELECT QUANTITY
holt einen gespeicherten Lagermengen-Datensatz aus dem Pufferbereich	

Das Kommando liefert Lagermengendaten. Da diese weder eine Identnummer noch ein Suchwort haben, wird mit dem bloßen Kommando kein Datensatz geholt. Um das Problem zu umgehen, muss ein Selektionsausdruck verwendet werden, der mit \$ oder % anfängt.

Das Beispiel setzt voraus, dass die Selektionsleiste LPLATZBEST für LMENGE existiert. Ist sie nicht vorhanden, legen Sie sie an mit:

```
edpimport.sh -p pass -I LPLATZBEST.imp
```

In diesem Beispiel wird die Lagermenge des Artikel **403** im Lagerplatz **F1** geholt.

```
.select quantity "$LPLATZBEST, artikel=403; platz=F1"
-- Stock (F1): 'H|totalQtySU'
```

In diesem Beispiel wird die Lagermenge des Artikels 403 von einem beliebigen Lagerplatz geholt.

```
.select quantity "$LPLATZBEST, artikel=403"
-- Stock: 'H|totalQtySU'
```

Wenn **G|mehr G|true** liefert, dann kann danach mit dem folgenden Kommando die nächste Lagermenge des Artikels 403 von irgendeinem Lagerplatz geholt werden.

```
.select quantity "$LPLATZBEST, artikel=403"
-- Stock: 'H|totalQtySU' - 'H|location' - 'H|location^wareh'
.. next stock
.select quantity
-- Stock: 'H|totalQtySU' - 'H|location' - 'H|location^wareh'
```

Die Variable **platz** liefert dann den Lagerplatz der geholten Lagermenge. **lager** liefert das Lager, in dem sich der Platz befindet.

Holen des Zahlungsobligos

Das Zahlungsobligo eines Kunden oder Lieferanten kann mit der FO durchlaufen werden. Zuerst muss der gewünschte Kunden- oder Lieferantendatensatz geholt werden.

```
.type text xtssel          ? F|defined(U|xtssel) = G|false
.load 1 customer "40001"
.continue END              ? G|success = G|false
..
.formula U|xtssel = "$,grpNo=2"
.formula U|xtssel = U|xtssel + ";@sort=OP_offen_nach_Konto_Termin"
.formula U|xtssel = U|xtssel + ";entAcct=" + 1|id
.formula U|xtssel = U|xtssel + ";@language=en"
..
.load 2 outstandingItems 'U|xtssel'
!NEXTOP
.continue END              ? G|success = G|false
..'2|rebetr' '1|name'
'2|invoiceAmt' - '1|descrOperLang'
..
.load 2 outstandingItems
.continue NEXTOP
```

durchläuft dann das Zahlungsobligo des Kunden bzw. Lieferanten.

Das Zahlungsobligo wird für Ein- und Verkauf gemeinsam geführt.

.HOLE VERKAUFS / .SELECT SALES - Für Datensätze des Verkaufs

.HOLE VERKAUFS	.SELECT SALES
deutsch	englisch
.HOLE VERKAUFS	.SELECT SALES
.HOLE VERKAUFS ABLAGE	.SELECT SALES FILINGSYSTEM
.HOLE VERKAUFS ALPHABETISCH	.SELECT SALES ALPHABETICAL
.HOLE VERKAUFS "'Bezeichnung'"	.SELECT SALES "'description'"
.HOLE VERKAUFS DRUCKBAREN	.SELECT SALES PRINTABLE
ANGEBOTE AUFTRAG LSCHEINE RECHNUNGEN Z AHLUNGEN	QUOTATIONS SORDERS DELNOTES INVOICES P AYMENTS
.HOLE VERKAUFS OFFENE	.SELECT SALES OUTSTANDING
ANGEBOTE AUFTRAG LSCHEINE RECHNUNGEN Z AHLUNGEN PROVISION	QUOTATIONS SORDERS DELNOTES INVOICES P AYMENTS COMMISSION
.HOLE VERKAUFS UEBERNAHME	.SELECT SALES TAKEOVER

.HOLE VERKAUFS

.SELECT SALES

.hole verkaufs holt die folgenden Vorgänge in der Reihenfolge der Nummern:

- Angebote
- noch nicht vollständig berechnete Aufträge
- Bestellungen
- noch nicht gebuchten Lieferscheine
- noch nicht gebuchten Rechnungen
- gebuchte Lieferscheine, die getrennt zu fakturieren sind

.hole verkaufs ablage holt Vorgänge aus der Verkaufsablage in der Reihenfolge, wie sie im Ablage-Suchbaum stehen.

.hole verkaufs alphabetisch holt Angebote, Aufträge, Lieferscheine und Rechnungen in der Reihenfolge der Suchwörter.

.hole verkaufs "'Bezeichnung'" holt Vorgänge mit dem Suchbegriff **'Bezeichnung'**.

.hole verkaufs druckbaren

angebote|auftrag|auftraege|lscheine|rechnungen|zahlungen holt Vorgänge aus der Druckwarteschlange.

.hole verkaufs offene

angebote|auftrag|auftraege|lscheine|rechnungen|zahlungen|provision holt folgende Vorgänge:

- Angebotsobligo
- Auftragsobligo
- Lieferobligo, positionsweise Fälligkeitsreihenfolge
- Rechnungsobligo für Lieferscheine ohne Auftrag
- Zahlungsobligo
 - offene Posten, wenn **art** = leer
 - Anzahlungen, wenn **art** = A
- Provisionsobligo

.hole verkaufs uebernahme holt alle noch nicht gebuchten Verkaufslieferscheine und Verkaufsrechnungen aufsteigend nach dem Erstellungsdatum.

.HOLE EINKAUFS / .SELECT PURCHASING - Für Datensätze des Einkaufs

.HOLE EINKAUFS	.SELECT PURCHASING
deutsch .HOLE EINKAUFS .HOLE EINKAUFS ABLAGE .HOLE EINKAUFS ALPHABETISCH .HOLE EINKAUFS "'Bezeichnung'" .HOLE EINKAUFS DRUCKBAREN BESTELLUNGEN LSCHEINE RECHNUNGEN ZAHLU NGEN .HOLE EINKAUFS OFFENE BESTELLUNGEN LSCHEINE RECHNUNGEN .HOLE EINKAUFS UEBERNAHME .HOLE EINKAUFS VORSCHLAG	englisch .SELECT PURCHASING .SELECT PURCHASING FILINGSYSTEM .SELECT PURCHASING ALPHABETICAL .SELECT PURCHASING "'description'" .SELECT PURCHASING PRINTABLE PORDERS DELNOTES INVOICES PAYMENTS .SELECT PURCHASING OUTSTANDING PORDERS DELNOTES INVOICES .SELECT PURCHASING TAKEOVER .SELECT PURCHASING SUGGESTION
.hole einkaufs holt die folgenden Vorgänge in der Reihenfolge der Nummern: • noch nicht vollständig berechnete Bestellungen • noch nicht gebuchte Lieferscheine • gebuchte und getrennt zu fakturierende Lieferscheine • noch nicht gebuchte Rechnungen .hole einkaufs ablage holt Vorgänge aus der Einkaufsablage. .hole einkaufs alphabetisch holt Bestellungen, Lieferscheine und Rechnungen in der Reihenfolge der Suchwörter. .hole einkaufs "'Bezeichnung'" holt Vorgänge mit dem Suchbegriff 'Bezeichnung'. .hole einkaufs druckbaren bestellungen lscheine rechnungen zahlungen holt Vorgänge aus der Druckwarteschlange. .hole einkaufs offene bestellungen lscheine rechnungen holt die folgenden Vorgänge: • gebuchte Lieferscheine ohne Bestellung und ohne gebuchte Rechnung • Bestellungen, auf die hin ein nicht getrennt zu fakturierender Lieferschein erfasst und gebucht wurde, für die aber noch keine Rechnung gebucht wurde • Lieferscheine, auf die hin eine getrennt zu fakturierende Bestellung erfasst und gebucht wurde, wofür aber noch keine Rechnung gebucht wurde .hole einkaufs uebernahme holt alle noch nicht gebuchten Einkaufslieferscheine und Einkaufsrechnungen aufsteigend nach dem Erstellungsdatum. .hole einkaufs vorschlag holt Bestellvorschläge nach Bestelltermin.	

.HOLE MZUORDNUNG / .SELECT MALLOCATION - FO-Zugriffe auf Materialzuordnungen

.HOLE MZUORDNUNG	.SELECT MALLOCATION
deutsch	englisch
.HOLE MZUORDNUNG	.SELECT MALLOCATION
holt Materialzuordnungen	

Reservierungen sind:

- Auftragspositionen
- Beistellteile im Einkauf
- [Entnahmeartikel](#) in der Fertigung

Wurde die Materialzuordnung geholt, kann der Zugriff auf die Reservierung erfolgen. Die Materialzuordnung wird über die Variable '**reserv**' geholt. Der Zugriff auf den reservierenden Vorgang, z.B. die Auftragsposition, erfolgt über die Variable '**vor**'.

Wurde die Reservierung geholt, kann auf die Materialzuordnung zugegriffen werden. Die Materialreservierung wird über die Variable '**eres**' geholt. Der Zugriff auf die Materialzuordnung erfolgt mit

```
.hole mzuordnung "$,reserv=" + materialreservierung_ID + ...
```

Der schnellste Zugriff erfolgt über den Schlüssel **MZ_Res_Bs_Bew_Art_Lag**.

Beschaffungen sind:

- Bestellvorschläge
- Umlagerungsvorschläge
- Bestellpositionen
- Fertigungsvorschläge

Wurde die Materialzuordnung geholt, kann der Zugriff auf die Beschaffung erfolgen. Dies geschieht über die Variable '**bs**'. Wurde die Beschaffung geholt, erfolgt der Zugriff auf die Materialzuordnung mit

```
.hole mzuordnung "$,bs=" + beschaffung_ID + ....
```

Der schnellste Zugriff erfolgt über den Schlüssel **MZuordnung_Beschaffung**.

Es gibt keine Materialzuordnungen zu Betriebsaufträgen, nur zu Fertigungsvorschlägen, zu denen Betriebsaufträge generiert wurden. Wurde der Betriebsauftrag geholt, kann über die Variable '**lres**' der Fertigungsvorschlag geholt werden. Der weitere Zugriff geschieht wie unter "Fertigungsvorschlag" beschrieben.

Zugangsbewegungen sind:

- Einkaufslieferschein

- Einkaufsrechnung mit Lagerbewegung
- Rückmeldung in der Fertigung

Wurde die Materialzuordnung geholt, kann über die Variable '**zbeweg**' der Zugriff auf die Zugangsbewegung erfolgen. Wurde die Zugangsbewegung geholt, erfolgt der Zugriff auf die Materialzuordnung mit

```
.hole mzuordnung "$,zbeweg=" + zugangsbewegung_ID + ...
```

Der schnellste Zugriff erfolgt über den Schlüssel **MZuordnung_Zugangsbew.**

Abgangsbewegungen sind:

- Verkaufslieferscheine
- Verkaufsrechnungen mit Lagerbewegung
- Rückmeldungen der **Entnahmeartikel**

Wurde die Materialzuordnung geholt, kann über die Variable '**abeweg**' der Zugriff auf die Abgangsbewegung erfolgen. Wurde die Abgangsbewegung geholt, erfolgt der Zugriff auf die Materialzuordnung mit

```
.hole mzuordnung "$,abeweg=" + abgangsbewegung_ID + ....
```

Der schnellste Zugriff erfolgt über den Schlüssel **MZuordnung_Abgangsbew.**

Sie können alle Materialzuordnungen eines Artikels mit folgendem Zugriff holen:

```
.hole mzuordnung "$,artikel=artikel_nummer;@sort=MZ_Art_Lag_Res_Bs_Bew"
```



vorgang_id bedeutet, dass die FO-Variable '**id**' immer gesetzt wird, wenn ein Vorgang geholt wurde.

.HOLE LOHNFERTIGUNG / .SELECT SUBCONTRACTING - FO-Zugriffe auf Lohnfertigung

.HOLE LOHNFERTIGUNG	.SELECT SUBCONTRACTING
deutsch	englisch
.HOLE LOHNFERTIGUNG	.SELECT SUBCONTRACTING
holt Lohnfertigungsvorschläge	

Neben den Fertigungs-, Bestell- und Umlagerungsvorschlägen gibt es auch Lohnfertigungsvorschläge. Der FO-Zugriff auf diese Vorschläge erfolgt mit:

.hole fertigungs vorschlag	Fertigungsvorschläge
.hole einkaufs vorschlag	Bestellvorschläge

<code>.hole umlagerungsvorschlaege</code>	Umlagerungsvorschläge
<code>.hole lohnfertigung</code>	Lohnfertigungsvorschläge

Die Fertigungsvorschläge bilden eine eigene Gruppe (**V-04-05**), wohingegen die anderen Vorschlagstypen alle durch Objekte aus der Gruppe **V-04-02** dargestellt werden. Die Unterscheidung erfolgt hier durch das Feld "Fremdbeschaffung", "Umlagern" oder "Lohnfertigung" (Variable **bsart**).

Alternativ zu den obengenannten **.hole**-Kommandos kann auch per Selektionsleiste auf die Beschaffungsvorschläge zugegriffen werden. Die jeweiligen Beschaffungsvorschläge werden dabei in der Reihenfolge ihrer Anfangs- bzw. Bestelltermine durchlaufen.

Bei der Freigabe von Fremdbeschaffungs-, Umlagerungs- oder Lohnfertigungsvorschlägen entstehen immer Bestellpositionen (auch Gruppe **V-04-02**). Der FO-Zugriff auf diese Positionen erfolgt wie auf jede andere Bestellposition. Auch in den Bestellpositionen weist das Feld "**bsart**" auf die Beschaffungsart hin.

.BUDGET / .BUDGET - Summierung von Kontenbeträgen

.BUDGET	.BUDGET
deutsch	englisch
<code>.BUDGET [-Optionen] 'Kontenbereich', 'Kontenbereich', ...</code>	<code>.BUDGET [-options] 'account range', 'account range', ...</code>
summiert die Beträge der aufgeführten Konten	
Mittels Optionen lässt sich die Gruppe der Konten einschränken.	

Die beiden Standardkonten "Sonstige Forderungen" und "Sonstige Verbindlichkeiten" werden vom **.budget**-Kommando wie Buchungskonten behandelt.

Das Kommando akzeptiert Nummern und Suchwörter von Konten und die Bereichssuche in der Form "von ! bis", beispielsweise **[10000!19999]**. Es bildet die Summen über alle bezeichneten Konten, wobei es sich auf den mit **.hole monat** voreingestellten Zeitraum bezieht, und hinterlegt die Ergebnisse in folgenden Variablen:

Variable	Inhalt
ssum	Soll-Bewegung im Zeitraum
hsum	Haben-Bewegung im Zeitraum
asoll	Anfangs-Soll
ahaben	Anfangs-Haben
asaldo	Anfangssaldo
nasaldo	Anfangssaldo vorzeichenverkehrt
esaldo	Endsaldo (asaldo + ssum - hsum)

Variable	Inhalt
nesaldo	Endsaldo vorzeichenverkehrt
jasaldo	Jahresanfangssaldo
esoll	End-Soll
ehaben	End-Haben

Näheres zu Variablen: **Vordefinierte Variablen** **asoll** und **ahaben** beinhalten die Summe der Verkehrszahlen bis zum Beginn des betrachteten Zeitraums und den Vortrag (auf der anzurechnenden Seite). Siehe auch Kapitel **Vordefinierte Variablen**.

Das Kommando **.budget** kann mit folgenden Optionen verwendet werden:

Option	Bedeutung
-S bzw. -V	Sammel- bzw. Verdichtungskonten inklusive. Fehlt diese Option, bleiben Verdichtungskonten unberücksichtigt. Nur die Konten "Forderungen" und "Verbindlichkeiten" sind optionsunabhängig und werden immer berücksichtigt.
-B	Bilanzkonten. Konten der Gewinn-Verlust-Rechnung bleiben unberücksichtigt.
-G	Gewinn-Verlust-Rechnung. Bilanzkonten bleiben unberücksichtigt.
-P	Mit dieser Option kann auf Planzahlen zugegriffen werden.
-K	Für die Ermittlung der Beträge aus den Verkehrszahlen der Kostenstellen/Kostenträger

Die Optionen **-B** und **-G** schließen sich gegenseitig aus. Alle Optionen müssen mit einem Minus eingeleitet werden.

.GEDRUCKT / .PRINTED - Druckwarteschlange

.GEDRUCKT	.PRINTED
deutsch	englisch
.GEDRUCKT	.PRINTED
.GEDRUCKT -ARCHIV	.PRINTED -ARCHIVE
.GEDRUCKT -RESET	.PRINTED -RESET

.GEDRUCKT	.PRINTED
.gedruckt holt einen Vorgang nach seinem Druck aus der Druckwarteschlange. .gedruckt -archiv archiviert generierte Dateien in der Datenbank "Dokument". Die Archivierung erfolgt beim Druck des Dokuments. Das archivierte Dokument kann selektiert, angezeigt oder nochmals gedruckt werden. .gedruckt -reset nimmt den im FO-Hauptpuffer geladenen Vorgang wieder in die Druckwarteschlange auf. Dazu wird im zugehörigen Datensatz die entsprechende Checkbox markiert. In der Regel wurde diese Markierung beim vorangehenden Ausdruck des Vorgangs automatisch entfernt. Die Kombination .gedruckt -archiv -reset ist nicht zulässig.	

Dieses Kommando ist nur bei der Ausgabe in eine Datei oder auf einen Drucker wirksam, nicht bei der Bildschirmausgabe.

Ein- und Verkaufsvorgänge (Angebote, Anfragen, Aufträge, Bestellungen, Lieferscheine, Rechnungen), Zahlungsträger, Arbeitsscheine und Betriebsaufträge können in eine Druckwarteschlange eingestellt werden, indem das Feld "Drucken" in diesen Vorgängen aktiviert wird. Zahlungsträger werden bei der Anlage automatisch in die Druckwarteschlange gestellt. Die Funktion bleibt so lange aktiv, wie sich ein Vorgang in der Druckwarteschlange befindet. Die Vorgänge in einer Druckwarteschlange können beispielsweise mit **.hole ... druckbaren** im FOP aufgegriffen und ausgedruckt werden. Mit dem Kommando **.gedruckt** holen Sie einen Vorgang nach seinem Druck aus der Druckwarteschlange, wobei es gleichgültig ist, wie der Vorgang beschafft wurde.

Zum Archivieren siehe auch [Dokumentenverwaltung](#).

.gedruckt bei Provisionsabrechnung

Verkaufsrechnungen gelangen nach der Buchung in eine Provisionsabrechnungswarteschlange, wenn ihnen ein Vertreter zugeordnet ist. **.gedruckt** holt Rechnungen, für die bereits eine Provisionsabrechnung erfolgt ist, aus der Provisionsabrechnungswarteschlange heraus, wenn die Rechnung mit dem Kommando **.hole ... druckbaren** beschafft wurde. Diese Warteschlange kann mit **.hole verkaufs offene provision** durchlaufen werden. Wurde die Rechnung auf andere Weise als mit **.hole ... druckbaren** aufgegriffen, so versucht **.gedruckt**, sie aus der Druckwarteschlange zu entfernen. Das Kommando ist nur bei Datei- oder Druckerausgabe wirksam.

Kommandos für Datensätze der Fertigung

.HOLE BETRIEBSAUFTRAG / .SELECT WORKORDER - Betriebsauftrag

.HOLE BETRIEBSAUFTRAG	.SELECT WORKORDER
deutsch	englisch
.HOLE BETRIEBSAUFTRAG	.SELECT WORKORDER
.HOLE BETRIEBSAUFTRAG "<Nummer>"	.SELECT WORKORDER "<number>"
.HOLE BETRIEBSAUFTRAG "+<Nummer>"	.SELECT WORKORDER "+<number>"
.HOLE BETRIEBSAUFTRAG VORSCHLAG	.SELECT WORKORDER SUGGESTION
.HOLE BETRIEBSAUFTRAG OBLIGO	.SELECT WORKORDER OUTSTANDING
.HOLE BETRIEBSAUFTRAG DRUCKBAREN	.SELECT WORKORDER PRINTABLE
.HOLE BETRIEBSAUFTRAG UEBERNAHME	.SELECT WORKORDER TAKEOVER
.HOLE BETRIEBSAUFTRAG ABLAGE	.SELECT WORKORDER FILINGSYSTEM

.hole betriebsauftrag Betriebsaufträge, die noch nicht vollständig abgearbeitet und deren Arbeitsscheine in Nummernreihenfolge sortiert sind

.hole betriebsauftrag "<Nummer>" Betriebsauftrag oder Arbeitsschein mit der angegebenen Identnummer

.hole betriebsauftrag "+<Nummer>" Rückmeldung zum Betriebsauftrag oder Arbeitsschein mit der angegebenen Identnummer

.hole betriebsauftrag vorschlag Fertigungsvorschläge

.hole betriebsauftrag obligo Kapazitätsreservierungen einer Maschinengruppe, d.h. alle Arbeitsgänge, die einer Maschinengruppe in einem bestimmten Zeitraum eingelastet sind; nach Anfangstermin sortiert.

.hole betriebsauftrag druckbaren Betriebsaufträge in der Druckwarteschlange, für die Arbeitspapiere zu drucken sind

.hole betriebsauftrag uebernahme Rückmeldungen, die erfasst, aber noch nicht gebucht sind

.hole betriebsauftrag ablage alle Rückmeldebelege

Das Kommando **.hole betriebsauftrag** ist gleichwertig mit **.hole fertigungs**.

Beispiel: Belegungsdaten einer Maschinengruppe für einen bestimmten Zeitraum

```

.. load department or workcenter
.select capacity "101"
.continue END          ? G|success = G|false
.. define period
.formula G|startDate="."
.formula G|dueDate="+30"
.select workorder outstanding
.continue END          ? G|success = G|false

```


Betriebsaufträge/Arbeitsscheine und Rückmeldungen werden in der gleichen Datenbank in zwei unterschiedlichen Gruppen gespeichert. Bei der Selektion mit <Nummer> wird der Standardsuchbaum Betriebsauftrag/Arbeitsschein verwendet. Dieser enthält aber nur die Objekte der Gruppe 1.



Da Rückmeldungen die gleiche Nummer erhalten wie die zugehörigen Betriebsaufträge/Arbeitsscheine, sind sie im Standardsuchbaum nicht enthalten. Bei der Selektion über + <Nummer> suchen Sie im Standardsuchbaum nach dem Schlüssel Rückmeldung in der Ablage. Da Betriebsaufträge und Arbeitsscheine nicht abgelegt werden, finden Sie dort auch nur die Objekte der Gruppe 2 (gebuchte Rückmeldungen). Nicht gebuchte Rückmeldungen können Sie daher nur selektieren, wenn Sie einen entsprechenden Suchbaum oder die Gruppe 2 mit angeben.

.HOLE FERTIGUNGS / .SELECT PRODUCTION - Betriebsauftrag

.HOLE FERTIGUNGS	.SELECT PRODUCTION
deutsch	englisch
.HOLE FERTIGUNGS	.SELECT PRODUCTION
Das Kommando .hole fertigungs ist gleichwertig mit .hole betriebsauftrag .	

.HOLE MBELEGUNG / .SELECT MACHLOADING - Maschinenbelegung

.HOLE MBELEGUNG	.SELECT MACHLOADING
deutsch	englisch
.HOLE MBELEGUNG	.SELECT MACHLOADING
.hole mbelegung lädt die Belegung der Maschinengruppe oder Abteilung, die im Hauptpuffer geladen ist, für den Zeitraum G adatum bis G edatum . Ausgewertet wird die geladene Tabelle mit der Variablen G mbe1 .	

.HOLE PLANKARTE / .SELECT PLANCHART - Beschaffung und Materialreservierung

.HOLE PLANKARTE	.SELECT PLANCHART
deutsch	englisch
.HOLE PLANKARTE	.SELECT PLANCHART

.HOLE PLANKARTE	.SELECT PLANCHART
Liefert pro Aufruf den nächsten geplanten Zugang (Beschaffung) oder Abgang (Materialreservierung) für die zuvor geholte Kombination aus Artikel und Lagergruppe, in der Reihenfolge ihrer Ist-Termine. Liegen Beschaffung und Reservierung auf dem gleichen Termin, wird zuerst die Beschaffung und dann die Reservierung berücksichtigt. Außerdem liefert das Kommando die Lagerbestände pro Verwendung/Projekt/Fertigteil Lohnfertigung. Diese können jeweils positiv oder negativ sein. Folgende Objekte können nach Aufruf des Kommandos im Hauptpuffer geladen sein: • Zugänge: positive Lagermengen (Tabellenzeile: Gebindemengen)(V-40-00) Beschaffungsvorschläge, Beschaffungspositionen (V-04-02, V-04-05) • Abgänge: negative Lagermengen (Tabellenzeile: Gebindemengen) (V-40-00) Materialreservierungen (V-04-04) Planmengen aus der rollierenden Absatzplanung (Tabellenzeile) (V-121-01) • Neutral: eigene Materialzuordnungen (V-28-00)	

```

..!interpreter english noabbrev
.select parts "V1"
.formula H|lgruppean = "Karlsruhe"
!NEXT:
.select planchart
.end ? _G|mehr
... Process receipt or issue ...
.continue NEXT

```

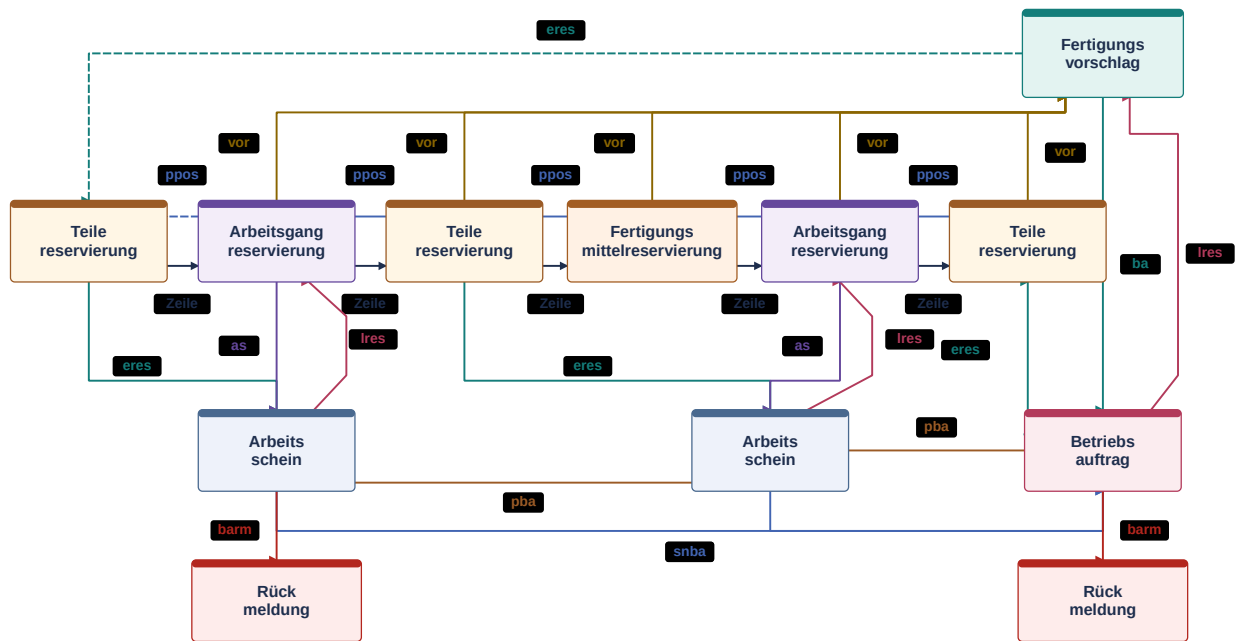
Objekte und Beziehungen in der Fertigung

Sie können über FO-Kommandos auf verschiedene Objekte der **Fertigung** zugreifen und sich von einem Objekt zu einem anderen Objekt bewegen. Es bestehen enge Verbindungen zur **Disposition**.

Nach dem Schaubild zur **Disposition** können Sie per FO von einer Auftragsposition zum Fertigungsvorschlag gelangen. Wenn Sie nun z.B. feststellen wollen, welche Rückmeldungen (RM) vorliegen, können Sie nach der Beschreibung in der folgenden Grafik zur Fertigung weitere Informationen erhalten. Auch diese Arbeitsweise wird fast ausschließlich über Verweisvariablen ermöglicht.



Nehmen Sie die Variablen **pba** und **snba** in die Maske auf. Sie können dadurch über die Verweisfelder zwischen den Objekten hin- und herspringen, ohne dass Sie schon die korrekte FO-Syntax beherrschen müssen. Über die Befehle **.hole zeile** können Sie im Tabellenteil der Teilereservierung eine Zeile weiter lesen. Mit **.hole dazu ppos** springen Sie wieder in eine Zeile zurück.



Durchgezogene Pfeile zeigen direkte Verbindungen; gestrichelte Strecken zeigen eine Fortsetzung der Folge.

Kommandos für Datensätze der BDE

.HOLE BDE / .SELECT PDC - BDE-Datensätze

.HOLE BDE	.SELECT PDC
deutsch	englisch
.HOLE BDE	.SELECT PDC
.HOLE BDE ALPHABETISCH	.SELECT PDC ALPHABETICAL
.HOLE BDE 'Bezeichnung'	.SELECT PDC 'description'
.HOLE BDE ASCHEIN 'Nummer'	.SELECT PDC JTICKET 'number'
.HOLE BDE OFFENE	.SELECT PDC OUTSTANDINGITEMS
.HOLE BDE OFFENE SPEZIFISCH	.SELECT PDC OUTSTANDINGITEMS SPECIFIC
.HOLE BDE UEBERNAHME	.SELECT PDC TAKEOVER
.HOLE BDE DRUCKBAREN	.SELECT PDC PRINTABLE

.HOLE BDE	.SELECT PDC
.hole bde holt alle Personal- und Auftragszeitpaare in Nummernreihenfolge. .hole bde alphabetisch holt alle Zeitpaare in der Reihenfolge der Suchwörter. .hole bde 'Bezeichnung' holt alle Zeitpaare mit dem Suchwort 'Bezeichnung'. .hole bde aschein 'Nummer' holt alle Zeitpaare des Betriebsauftrags- oder Arbeitsschein-Nummernbereiches 'Nummer' in zeitlicher Sortierung. .hole bde offene holt alle offenen Zeitstempelungen, sortiert nach Mitarbeiter, Anfangsdatum und -uhrzeit oder nach Mitarbeiter, Betriebsauftrags-, Arbeitsscheinnummer, Anfangsdatum und -uhrzeit. .hole bde offene spezifisch holt alle offenen Zeitstempelungen in zeitlicher Reihenfolge, wenn zuvor über die Variablen G adatum und G edatum Anfangstermin und Endtermin festgelegt wurden. Dies ist allerdings nur in Verbindung mit dem Kommando .hole mitarbeiter oder .hole kapazitaet möglich. .hole bde uebernahme holt alle erfassten, aber noch nicht übertragenen Zeitpaare. .hole bde druckbaren holt alle als fehlerhaft gekennzeichneten Zeitpaare.	

Kommandos für die Disposition

In der Disposition werden **.merke verwendung** und **.hole verwendung** verwendet. Damit erhalten Sie folgende Informationen:

- Bezugsquellen (Lager oder Beschaffung) geladener Materialreservierungen, negativer Lagermengen und Wertereihen
- Mengen geladener Materialreservierungen, negativer Lagermengen und Wertereihen
- Zuordnung geladener Beschaffungen (Bestellvorschlag, Bestellposition oder Fertigungsvorschlag) zu Reservierungen, negativer Lagermengen und Wertereihen.

Voraussetzung für die Anwendung dieser Kommandos ist, dass die Fertigungsliste aufgelöst ist; dazu muss die Disposition nach dem Erfassen des Auftrags durchgeführt sein oder die Fertigungsliste wurde durch Absteigen erzeugt.

.HOLE VERWENDUNG / .SELECT APPLICATION - Verwendungsnachweis holen

.HOLE VERWENDUNG	.SELECT APPLICATION
deutsch	englisch
.HOLE VERWENDUNG	.SELECT APPLICATION

.HOLE VERWENDUNG	.SELECT APPLICATION
.hole verwendung zeigt den Verwendungsnachweis des zuvor geholten Artikels an. Damit werden nacheinander alle Beschaffungen bzw. Reservierungen geladen, die durch .merke verwendung gesammelt wurden. Bei jedem Holen werden die beiden Variablen 'G bvkenn' (Text) und 'G bvmge' (real) vorbelegt.	

.MERKE VERWENDUNG / .RESERVE APPLICATION - Beschaffungen zu Materialreservierungen

.MERKE VERWENDUNG	.RESERVE APPLICATION
deutsch	englisch
.MERKE VERWENDUNG [+]	.RESERVE APPLICATION [+]
.merke verwendung: Ist eine Reservierung geladen, werden durch dieses Kommando die Bezugsquellen zusammengestellt, die sie bedienen. Ist eine Beschaffung geladen, werden durch dieses Kommando alle Reservierungen zusammengestellt, die von ihr bedient werden. .merke verwendung + liefert zusätzlich zu einer noch nicht eingeplanten variantenbezogenen Reservierung die zugehörige variantenbezogene Beschaffung und umgekehrt.	

Mehrstufige Fertigungslisten



Ein Aufruf von **.merke verwendung** setzt die Umgebung für **.hole verwendung** komplett neu. Die durch einen früheren Aufruf von **.merke verwendung** gesetzte Umgebung geht dabei verloren.

Bei der Auswertung mehrstufiger Fertigungslisten sollte deshalb nach **.merke verwendung** erst die komplette Stufe mit **.hole verwendung** durchlaufen und die gefundenen Objekte gemerkt werden bevor **.merke verwendung** auf der nächsten Stufe gemacht wird.

Auftragsbezogene Beschaffung und Reservierung

Auftragsbezogene Beschaffung und Reservierung

Für eine auftragsbezogene Beschaffung bzw. Reservierung sollten folgende Kommandos verwendet werden:



```
.merke verwendung [+]
.hole verwendung
.weiter ... ? _bvken"/"/br/"
```

Bei Artikeln mit auftragsbezogener Disposition, deren Mindestbestand größer als 0.0 ist, kann eine Beschaffung mehrere Reservierungen aus verschiedenen Kundenaufträgen bedienen.

Die Kommandos **.merke verwendung** und **.hole verwendung** spiegeln die Zuordnung zwischen Reservierungen und Beschaffungen so wider, wie sie am Ende des letzten Dispositionslaufs bzw. nach der letzten manuellen Zuordnung (Dispositionsart "restmengenbezogen"; <Betr-Auftrag> <Ändern>, [Materialreservierungen]) war. Zwischenzeitlich erfolgte Mengenänderungen, Terminänderungen, Stornierungen oder Lagerbestandsänderungen werden dabei nicht berücksichtigt.

Das Kommando **.hole verwendung** gibt zu den Beschaffungen und Reservierungen auch die Lagerbestände aus. Im Hauptpuffer können sich auch Tabellenzeilen einer Lagergruppenmenge befinden. Dieses Kommando kann folgende Objekte in den Hauptpuffer laden:



Zugänge: positive Lagermengen (Tabellenzeile: Gebindemengen)	(V-40-00)
Beschaffungsvorschläge, Beschaffungspositionen	(V-04-02, V04-05)
Abgänge: negative Lagermengen (Tabellenzeile: Gebindemengen)	(V-40-00)
Materialreservierungen	(V-04-04)
Wertereihen	(V-121-02)
Neutral: eigene Materialzuordnungen ("Eigene Zuordnung")	(V-28-00)

Beispiele

Es seien folgende Materialzugänge und -abgänge eingeplant:

Bestand	10.00	Mindestbestand	20
Losgröße	50	Losbildungsfrist	0
Dispositionsart (DA)	bedarfsbezogen	Beschaffungsart	B
Datum	Zugang	Abgang	DA Auftr./Best.

09.11.10	50.00		B B 123
09.11.10		15.00	B A 1000

Beispiel: Beschaffungsnachweis für die Reservierung zum Auftrag 1000

```
.select sales "1000"
.select line
.select add H|firstRes
.. reservation loaded
.reserve application
!NEXT:
.select application
'G|procureType' 'G|allocQtySUUse'
?'H|deadlineDay' ? G|success
.continue NEXT ? G|procureType <> ""
```

Der Beschaffungsnachweis für die Reservierung zum Auftrag 1000 liefert:

Quelle	Menge	Termin
L	10.00	
B	5.00	
09.11.10		

Beispiel: Verwendungsnachweis für die Beschaffungsposition der Bestellung 123

```
.select purchasing "123"
.select line
.. providing loaded
.reserve application
!NEXT:
.select application
'G|procureType' 'G|allocQtySUUse'
?'H|deadlineDay' ? G|success
.continue NEXT ? G|procureType <>> ""
```

Der Verwendungsnachweis für die Beschaffungsposition der Bestellung 123 liefert:

Verwendung	Menge	Termin
R	5.00	
09.11.10		

Verwendungsnachweis für eine Beschaffung

Wird ein Verwendungsnachweis für eine **Beschaffung** erstellt, bedeuten:

bvkenn	Wer wird bedient? * R Reservierung
bvmge	Mit welchem Anteil der Beschaffungsmenge in Lagereinheiten wird die geladene Reservierung bedient?

Wenn bei **.merke verwendung** eine Beschaffung im **.hole**-Puffer ist, werden mit jedem nachfolgenden **.hole verwendung** die Ziele geholt, in die die Mengen aus dieser Beschaffung einfließen. Die jeweils fließende Menge (in Lagereinheiten) wird nach jedem **.hole verwendung** in der Variablen **'G|bvmge'** hinterlegt. Wohin diese Menge fließt, wird in der Variablen **'G|bvkenn'** hinterlegt. Ist nach **.hole verwendung** **bvkenn** leer, konnten keine weiteren Beschaffungen bzw. Reservierungen geladen werden. Der Nachweis ist beendet. Nicht nachgewiesene Mengen fließen aufs Lager (Verwendungsnachweis). Folgende Ziele kommen in Frage:

bvkenn	Ziel	.hole-Puffer	Variable G mehr
R	Materialreservierung	Materialreservierung	G true
S	Eigene Zuordnung (siehe editierbare Plankarte, Zeilenlupe)	<Leer>	G true
L	Lager	<Leer> Mit erneutem .hole verwendung können ggf. weitere Bezugsquellen geladen werden.	G true
?	noch nicht zugeordnet	<Leer>	G false
<Leer>	Ende der Zuordnungsliste	<Leer>	G false

Als Abbruchkriterium für die Zuordnungsliste muss auf **bvkenn = ""** und **G|mehr = G|false** geprüft werden.

Beschaffungsnachweis für eine Materialreservierung

Wird ein Verwendungsnachweis für eine **Rückmeldung** erstellt, bedeuten:

bvkenn	Woraus wird beschafft? * L aus Lager * B aus Beschaffung
bvmge	Welcher Bedarfsmengenanteil in Lagereinheiten wird durch die geladene Bezugsquelle abgedeckt?

Wenn bei **.merke verwendung** eine Materialreservierung im **.hole-Puffer** ist, werden mit jedem nachfolgenden **.hole verwendung** die Quellen geholt, aus denen die Mengen in diese Materialreservierung einfließen.

Die Menge (in Lagereinheiten), die jeweils fließt, ist nach jedem **.hole verwendung** in der Variablen '**G|bvmge**' hinterlegt. Woher diese Menge fließt, ist in der Variablen '**G|bvkenn**' hinterlegt. Ist nach **.hole verwendung bvkenn** leer, konnten keine weiteren Beschaffungen bzw. Reservierungen geladen werden. Der Nachweis ist beendet. Nicht nachgewiesene Mengen wurden noch nicht von der Bestandsrechnung abgedeckt (Bezugsquellennachweis). Folgende Quellen kommen in Frage:

bvkenn	Quelle	.hole-Puffer	Variable G mehr
B	Beschaffung	Beschaffung	G true
S	Eigene Zuordnung (siehe editierbare Plankarte, Zeilenlupe)	Materialzuordnung	G true
L	Lager	Lagermenge	G true

bvkenn	Quelle	.hole-Puffer	Variable G mehr
W	Wertereihe	Wertereihe	G true
<Leer>	noch nicht zugeordnet	<Leer>	G false
<Leer>	Ende der Zuordnungsliste	<Leer>	G false

Als Abbruchkriterium für die Zuordnungsliste muss auf **bvkenn** = "" und **G|mehr** = **G|false** geprüft werden.

.merke verwendung und .hole verwendung in EDP

.merke verwendung / **.hole verwendung** ist auch per **EDP** verfügbar. Das entspricht einer Selektion über Materialzuordnungen. Es werden 3 Fälle unterschieden:

1. Wenn man auf der Beschaffung **<bs_id>** steht, ist **.merke verwendung** / **.hole verwendung** identisch mit der Selektion

Mzuordnung \$,bs=<bs_id>;einplan=y;aktiv=y;@sort=MZuordnung_Beschaffung

Diese Selektion durchläuft nacheinander die gleichen Objekte (Materialzuordnungen) wie **.hole verwendung**. Die Zuordnungsmenge (**BVMGE**) errechnet sich aus **zuomge * faktor**. Die jeweils zugeordnete Reservierung steht unter **reserv**, der Zuordnungstyp (**BVKENN**) ist

S (Eigene Zuordnung)	wenn !F empty(bis) && !F empty(zeich)
R (Reservierung)	in allen anderen Fällen

2. Man hat den Artikel **<art_id>** und die Lagergruppe **<lgr_id>** und sucht mit **.merke verwendung** / **.hole verwendung** alle Reservierungen, die aus dem Lagerbestand bedient werden. Das funktioniert mit der Selektion

Mzuordnung

\$,artikel=<art_id>;lgruppe=<lgr_id>;einplan=y;@sort=MZ_Art_Lag_Res_Bs_Bew

Die Zuordnungsmenge, zugeordnete Reservierungen und der Zuordnungstyp bestimmen sich hier genauso wie in Fall 1.

3. Wenn auf der Materialreservierung **<res_id>** steht, ist **.merke verwendung** / **.hole verwendung** identisch mit der Selektion

Mzuordnung \$,reserv=<res_id>;einplan=y;aktiv=y;@sort=MZ_Res_Bs_Bew_Art_Lag

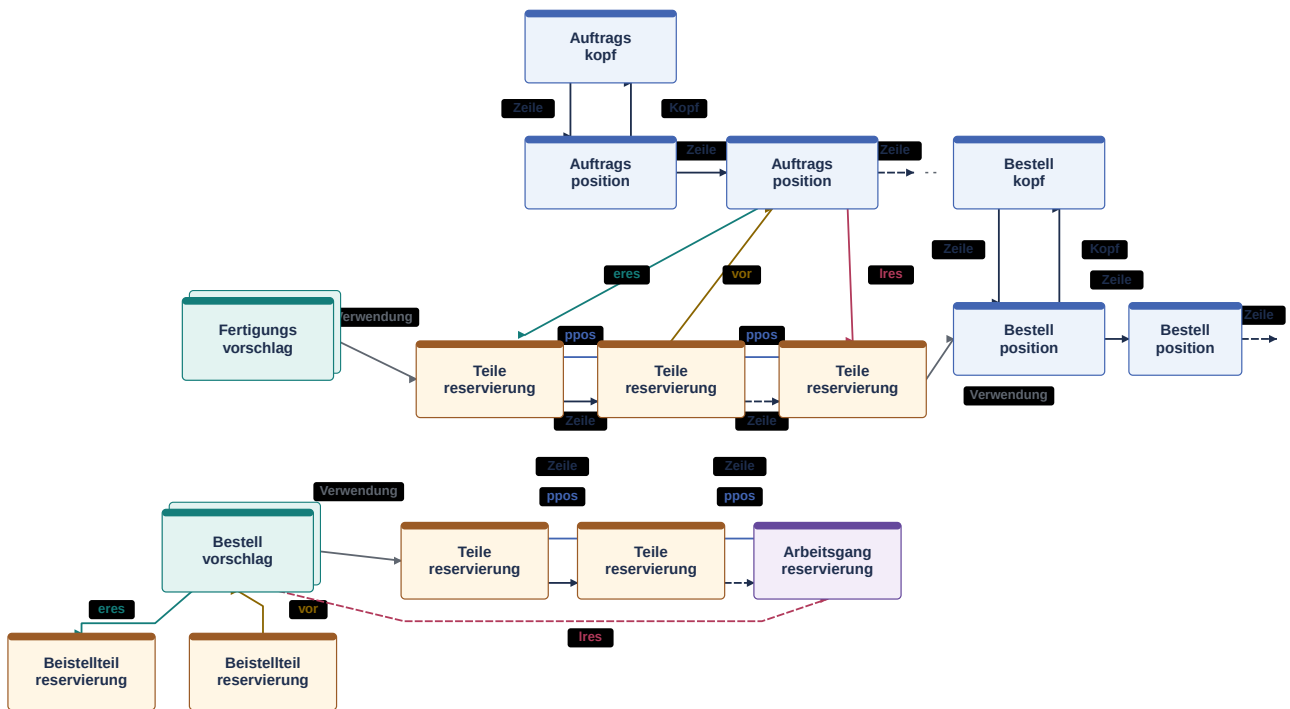
Die Zuordnungsmenge errechnet sich wie oben, die jeweils zugeordnete Beschaffung steht unter **bs**. Ist diese leer, wird die Reservierung aus dem Lager bedient. Mögliche Zuordnungstypen sind folgende:

S (Eigene Zuordnung)	wenn !F empty(bis) && !F empty(zeich)
----------------------	---------------------------------------

L (Lagerbestand)	wenn $F \mid \text{empty}(bs)$ und das Objekt ist eine Lagermenge, $bs^{dnr} = 40$
B (Beschaffung)	wenn $\neg F \mid \text{empty}(bs)$ und das Objekt ist eine Beschaffung, $bs^{dnr} = 4$

Objekte und Beziehungen in der Disposition

Sie können über FO-Kommandos auf verschiedene Objekte der Material- und Terminplanung zugreifen und sich von einem Objekt zu einem anderen Objekt bewegen. Es bestehen enge Verbindungen zur **Fertigung**.



Durchgezogene Pfeile zeigen direkte Verbindungen; gestrichelte Strecken zeigen eine Fortsetzung der Folge.

Auftragskopf (AU-Kopf>
und Auftragspositionen (AU-Pos.)

Wenn Sie über die FO einen Auftrag selektiert haben, befinden Sie sich zuerst im Auftragskopf. Über **.hole zeile** springen Sie in den Tabellenteil des Auftrags mit seinen Positionen. Durch erneuten Aufruf von **.hole zeile** springen Sie zur nächsten Position des Auftrages. Eine Auftragsposition enthält die Variablen **lres** (letzte Reservierung) und **eres** (erste Reservierung). Diese Variablen sind nicht in der Standardauftragsmaske enthalten, sondern können in individualisierte Masken aufgenommen werden. Da es sich um Verweisfelder auf die erste oder letzte Reservierung handelt, können Sie über diese Felder in die Teilereservierung absteigen.



Nehmen Sie zunächst die Variablen wie **eres** und **lres** in die Maske auf. Sie können dadurch über die Verweisfelder zwischen den Objekten hin- und springen, ohne dass Sie schon die korrekte FO-Syntax beherrschen müssen.

Teilereservierung (TE-RES)

Über die Variable **eres** und **lres** können Sie in die Teilereservierung springen. Führen Sie dazu **.hole dazu eres** oder **.hole dazu lres** aus. Danach können Sie **H|id** abfragen und haben nun die ID der Teilereservierung im Zugriff. Die Begriffe Teilereservierung und Auftragsfertigungsliste werden im Abas Umfeld fälschlicherweise oft als Synonyme verwendet.

In die Auftragsfertigungsliste kommen Sie aus der Auftragsposition durch Absteigen über den Button **Fertigungsliste**. Wenn Sie über **.hole dazu eres** aus der Auftragsposition in die Teilereservierung absteigen, haben Sie hier in der Regel eine andere ID im Zugriff. Es handelt sich hierbei um eine Teilereservierung, die nur eine Zeile im Tabellenteil enthält. In dieser Zeile ist der Artikel aus der Auftragsposition enthalten. Diese Teilereservierung stellt den eigentlichen Bedarf des Artikels dar, der sich aus der Auftragsposition ableitet.

Die eigentliche Auftragsfertigungsliste ist auch eine Teilereservierung, die sich aber eine Stufe unter der momentan selektierten Teilereservierung befindet. In der Teilereservierung gibt es die Verweisvariable **vor**. Über diesen Verweis können Sie in den reservierenden Vorgang springen. In der Beispielgrafik gelangen Sie darüber zurück in die Auftragsposition.

Beschaffungen
(u.a. FV) für die
Teilereservierung

Über die Teilereservierung wird der Bedarf der Auftragsposition dargestellt. Die Bedarfsdeckung über **eres**, **lres** oder **vor** ist nicht möglich, da der Bedarf durch verschiedene Beschaffer gedeckt werden kann.

Durch die Teilereservierung besteht ein Bedarf von 100 Stück. Im Lager liegen 40 Stück, die Restmenge muss nach einem Fertigungsvorschlag produziert werden.

In diesem Fall gibt es 2 Beschaffer: Lager und Fertigungsvorschlag (FV). Um die Verknüpfung zwischen Teilereservierung und Beschaffern darzustellen, ist eine Tabelle sinnvoll. Sie können diese Tabelle herstellen über die Kommandofolge

- **.merke**
- **.merke verwendung**
- **.hole verwendung**; mit diesem Kommando springt man eine Beschaffung weiter.

Am Ende der Beschaffungstabelle ist der H-Puffer leer. Das Ausstiegskriterium ist die Variable **bvkenn**. Wenn **bvkenn** leer ist, ist die Beschaffungstabelle durchlaufen. Wenn Sie das Kommando **.merke** auslösen, haben Sie durch das erste **.merke** im H-Puffer wieder die ursprüngliche Teilereservierung im Zugriff.

Beim Durchlaufen der Beschaffungstabelle können Sie wieder die **H|id** und die Variable **bvkenn** abfragen. Die wichtigsten **bvkenn**-Zustände sind **L für Lager** und **B für Beschaffung**. Über die Variable **H|id** können Sie die ID des Beschaffungsvorschlages abfragen. Über die Variable **H|typ** können Sie ermitteln, um welchen Beschaffungsvorschlag es sich handelt.

Wenn Sie z.B. einen Fertigungsvorschlag (FV) im Zugriff haben, können Sie über die Variable **eres** wieder in die Teilereservierung springen. Dann haben Sie die Auftragsfertigungsliste im Zugriff, wie Sie sie über das Absteigen in die Fertigungsliste aus dem Auftrag kennen.

Kommandos für Verbrauchsfolgeverfahren

Nach der Einführung der **Verbrauchsfolgeverfahren** LIFO (Last in first out) und FIFO (First in, first out) können Zugangsmengen voneinander unterschieden werden. Es kann z.B. vorkommen, dass eine Abgangsmenge durch mehrere Einzelmengen bedient wurde.

Mit dem Kommandopaar **.merke bindung** / **.hole bindung** kann man auflisten:

- zu einem Abgang alle Zugänge, die ihn bedient haben
- zu einem Zugang alle Abgänge, in die er eingeflossen ist

.HOLE BINDUNG / **.SELECT LINK** - Lagerzugänge und -abgänge

.HOLE BINDUNG	.SELECT LINK
deutsch	englisch
.HOLE BINDUNG	.SELECT LINK
.hole bindung liefert • zu einem Abgang alle Zugänge, die ihn bedient haben • zu einem Zugang alle Abgänge, in die er eingeflossen ist	

Das Verhalten des Paares **.merke bindung** / **.hole bindung** ist ähnlich dem Verhalten von **.merke verwendung** / **.hole verwendung**.

Nach **.hole bindung** werden die Variablen vom System wie folgt belegt:

bvmge	wird belegt mit der Menge in Lagereinheiten, die geflossen ist						
bvkenn	wird belegt mit der Bezugsquelle: <table border="1"> <tr> <td>B</td><td>für Beschaffung (alle Zugänge, die die Beschaffung bedient haben)</td></tr> <tr> <td>L</td><td>für Lager (alle Zugänge, die den Lagerabgang bedient haben) oder wohin eine Zugangsmenge geflossen ist (alle Abgänge, in die die Zugangsmenge eingeflossen ist)</td></tr> <tr> <td>R</td><td>für Reservierung</td></tr> </table>	B	für Beschaffung (alle Zugänge, die die Beschaffung bedient haben)	L	für Lager (alle Zugänge, die den Lagerabgang bedient haben) oder wohin eine Zugangsmenge geflossen ist (alle Abgänge, in die die Zugangsmenge eingeflossen ist)	R	für Reservierung
B	für Beschaffung (alle Zugänge, die die Beschaffung bedient haben)						
L	für Lager (alle Zugänge, die den Lagerabgang bedient haben) oder wohin eine Zugangsmenge geflossen ist (alle Abgänge, in die die Zugangsmenge eingeflossen ist)						
R	für Reservierung						

ursache	wird belegt mit dem Kennzeichen des Objekts, das gebucht wurde:	
	L	für Lieferschein
	R	für Rechnung
	F	für Rückmeldung in der Fertigung mit Betriebsauftrag
	O	für Rückmeldung in der Fertigung ohne Fertigungsvorschlag (von Hand erfasste Rückmeldung)
	S	für manuelle Lagerbuchung
zuganglj	Zugangsjournaleintrag	
abganglj	Abgangsjournaleintrag	

In den **.hole**-Puffer wird zusätzlich das Bindungsobjekt geladen. Als Abbruchkriterium für die Bindungsliste muss auf **bvkenn = ""** geprüft werden, wie bei **.merke verwendung** / **.hole verwendung**.

Beispiel:

In einem Artikel wurde das Verbrauchsfolgeverfahren FIFO eingestellt. Zu einer Bestellung mit der Menge 100 gibt es 2 gebuchte Lieferscheine:

- Einkaufslieferschein zur Teilmenge 70
- Einkaufslieferschein zur Teilmenge 30

Der Bestand von 100 wurde mit Verkaufslieferscheinen folgender Mengen abgebucht:

- Verkaufslieferschein mit der Menge 55
- Verkaufslieferschein mit der Menge 35
- Verkaufslieferschein mit der Menge 10

Alle Lieferscheine wurden zu einem Auftrag generiert.

1. Ablauf: Welche Vorgänge wurden durch den ersten Einkaufslieferschein mit der Liefermenge von 70 bedient?

.hole einkaufs 123456

In den **.hole**-Puffer wird der Lieferschein mit der Identnummer 123456 geladen.

.hole zeile

Die erste Zeile des Lieferscheins wird in den **.hole**-Puffer geladen.

.merke bindung

Zu der Lieferscheinposition werden alle bedienten Objekte gesammelt.

.hole bindung

liefert die erste Bindung (Zugangsmenge, geflossen in Abgang)

bvmge liefert 55

bvkenn liefert R (Reservierung)

ursache liefert L (Lieferschein)

in den **.hole**-Puffer wurde die Position des ersten Verkaufslieferscheins geladen.

.hole bindung

liefert die zweite Bindung

bvmge liefert 15

bvkenn liefert R (Reservierung)

ursache liefert L (Lieferschein)

in den **.hole**-Puffer wurde die Position des zweiten Verkaufslieferscheins geladen.

.hole bindung

keine weiteren Bindungen vorhanden.

bvmge liefert 0

bvkenn ist leer ("")

ursache ist leer ("")

kein Objekt im **.hole**-Puffer

2. Ablauf: Welche Vorgänge wurden durch die Bestellung mit der Bestellmenge von 100 bedient?

.hole einkaufs 789456

.hole zeile

.merke bindung

Zu der Bestellposition werden alle bedienten Objekte gesammelt.

.hole bindung

liefert die erste Bindung (Zugangsmenge, geflossen in Abgang)

bvmge liefert 55

bvkenn liefert R (Reservierung)

ursache liefert L (Lieferschein)

In **.hole**-Puffer wird die Reservierung zum Auftrag geladen. Die Menge 55 ist in den ersten Lieferschein zum Auftrag geflossen.

.hole bindung

liefert die zweite Bindung

bvmge liefert 35

bvkenn liefert R (Reservierung)

ursache liefert L (Lieferschein)

In den **.hole**-Puffer wird die Reservierung zum Auftrag geladen. Die Menge 35 ist in den zweiten Lieferschein zum Auftrag geflossen.

.hole bindung

liefert die dritte Bindung

bvmge liefert 10

bvkenn liefert R (Reservierung)

ursache liefert L (Lieferschein)

In den **.hole**-Puffer wird die Reservierung zum Auftrag geladen. Die Menge 10 ist in den dritten Lieferschein zum Auftrag geflossen.

.hole bindung

keine weiteren Bindungen vorhanden

bvmge liefert 0

bvkenn ist leer ("")

ursache ist leer ("")

kein Objekt im **.hole**-Puffer

3. Ablauf: Welche Vorgänge bedienen den Auftrag (die Reservierung)?

.hole verkaufs 456123

.hole zeile

.hole dazu res

Die Reservierung zur Auftragsposition wird in den **.hole**-Puffer geladen.

.merke bindung

Zu der Reservierung werden alle Beschaffungen gesammelt.

.hole bindung

liefert die erste Bindung

bvmge liefert 55

bvkenn liefert B (Beschaffung)

ursache liefert L (Lieferschein)

In den **.hole**-Puffer wird die Bestellung des ersten Lieferscheines geladen.

.hole bindung

liefert die zweite Bindung

bvmge liefert 35

bvkenn liefert B (Beschaffung)

ursache liefert L (Lieferschein)

In den **.hole**-Puffer wird die Bestellung des ersten Lieferscheines geladen.

.hole bindung

liefert die dritte Bindung

bvmge liefert 10

bvkenn liefert B (Beschaffung)

ursache liefert L (Lieferschein)

In den **.hole**-Puffer wird die Bestellung des zweiten Lieferscheines geladen.

.hole bindung

keine weiteren Bindungen vorhanden

bvmge liefert 0

bvkenn ist leer ("")

ursache ist leer ("")

kein Objekt im **.hole**-Puffer



Die Informationen zu den Bindungen werden entweder auf der Ebene "Beschaffung - Reservierung" oder der Ebene "Zugangsvorgang - Abgangsvorgang" geliefert. Startet man das **.merke bindung**-Kommando zu einer Beschaffung, werden alle bedienten Reservierungen aufgelistet. Startet man das **.merke bindung**-Kommando zu einem Abgangsvorgang, z.B. Verkaufslieferschein, werden alle Zugangsvorgänge aufgelistet.

Kommandos für die Materialwirtschaft

.HOLE TEILE / .SELECT PARTS - Artikeldatensatz an Lagergruppe anpassen

.HOLE TEILE	.SELECT PARTS
deutsch	englisch
.HOLE TEILE SPEZIFISCH 'Lagergruppe'	.SELECT PARTS SPECIFIC 'warehouse group'
.hole teile passt den Artikeldatensatz auf die Lagergruppe an	

Das Kommando **.hole teile spezifisch 'Lagergruppe'** passt den Artikel, der momentan im **.hole**-Puffer liegt, auf die Lagergruppe an, die in der Variablen **'lgruppe'** übergeben wurde. Dabei werden folgende Lagergruppeneigenschaften für den Artikel und die angegebene Lagergruppe in die zugehörigen Artikelfelder hineinkopiert:

- Dispositionsart
- Beschaffungsart
- Entnahmeart
- Zu- und Abgangslagerplatz
- Mindestbestand
- Losgröße
- Losbildungsfrist
- Umlagern aus Lagergruppe
- Lieferant 1
- Preisgruppe 1
- Einkaufspreis 1
- Währung für den Einkaufspreis 1
- Beschaffungsfrist 1
- Bestellnummer 1 beim Lieferanten 1

Wird der Artikel durch Umlagern aus einer anderen Lagergruppe beschafft, so wird auch seine Fertigungsliste modifiziert. Sie enthält dann nur eine Zeile, in der der Artikel selbst steht (als Beistellteil für den Umlagerungsvorgang).

Nach **.hole teile spezifisch 'Lagergruppe'** steht in dem Artikelfeld **lgruppe** die Lagergruppe, für die der Artikeldatensatz angepasst wurde. Da der Artikeldatensatz durch diesen Befehl modifiziert wird, darf er nicht in den Datenbestand zurückgeschrieben werden. Das wird dadurch sichergestellt, dass dieser Befehl nur dann erlaubt wird, wenn der Artikel nicht mit Sperre gelesen wurde.

Nachbewertung bei FO-Planpreisuordnung

Falls die Planpreisuordnung zu einem Artikel per FO wie im Beispiel unten erfolgt, wird bei aktiver Materialkostenverbuchung eine Nachbewertung aller unbewerteten Lagerbuchungen dieses Artikels durchgeführt, die zum Planpreis bewertet werden sollen.

Beispiel: Planpreisuordnung zu einem Artikel

```
.select lock parts "401"  
.continue END          ? G|success = G|false  
.formula H|plannedpriceSU1 = 4  
.rewrite
```

Kommandos für die Finanzbuchhaltung

.HOLE MONAT / .SELECT MONTH - Zeitraum für Finanzbuchhaltung wählen

.HOLE MONAT	.SELECT MONTH
deutsch	englisch
.HOLE MONAT	.SELECT MONTH
.HOLE MONAT 'Anfangsmonat' 'Endmonat'	.SELECT MONTH 'start month' 'end month'
.HOLE MONAT 1 2 3 NEU	.SELECT MONTH 1 2 3 NEW
.HOLE MONAT 1-12 13 14 15 ALT	.SELECT MONTH 1-12 13 14 15 OLD
.hole monat 'Anfangsmonat' 'Endmonat' legt einen Zeitraum für Finanzbuchhaltungslisten fest .hole monat 1 2 3 neu holt den ersten, zweiten oder dritten Monat des neuen Geschäftsjahres. .hole monat 1-12 13 14 15 alt holt die Monate 1 bis 12 des abgeschlossenen Geschäftsjahres oder die Nachbuchungsmonate.	

Mit dem Kommando **.hole monat** wird ein Zeitraum für Listen der Finanzbuchhaltung gewählt.

.hole monat 'Anfangsmonat' 'Endmonat'

Die Monatsangaben verstehen sich als Geschäftsmonate. Es können auch die Monate 13 bis 15 angesprochen werden. Für die Monate 1 bis 12 ist auf die Unterscheidung zwischen Geschäfts- und Kalendermonat zu achten, sofern das Geschäftsjahr nicht mit dem Januar beginnt. (Der Geschäftsjahresbeginn ist dem Feld **Letzter Jahresabschluss zum** des Firmenstammdatensatzes **Abschlussstermine** zu entnehmen.)

Während der Übergangszeit nach dem Beginn des neuen, aber vor Abschluss des alten Geschäftsjahres können schon vorläufige Auswertungen über die neuen Monate gedruckt werden, wenn der Zeitraum mit dem Zusatz **neu** ausgewählt wird.

```
.select month 1 new
```

Umgekehrt können die Monate 1 bis 12 des abgeschlossenen Jahres (gesammelt, aber nicht einzeln) mit dem Zusatz **alt** ausgewählt werden.

```
.select month 1-12 old
```

Wird einer der Nachbuchungsmonate 13 bis 15 ausgewählt, so ist der Zusatz **alt** implizit.

```
.select month 1-15
```

.HOLE JAHR / .SELECT YEAR - Besetzt die Datumsvariablen

.HOLE JAHR	.SELECT YEAR
deutsch	englisch
.HOLE JAHR .HOLE JAHR 'Geschäftsjahr' 'Anfangsgeschäftsmonat' 'Endgeschäftsmonat'	.SELECT YEAR .SELECT YEAR 'fiscal year' 'start month' 'end month'
besetzt die Variablen G ajahr , G ejahr , G amonat , G emonat , G adatum und G edatum .	

Letztes Jahresabschlussdatum ist der 31.12.2012

```
.select year 12 3 13
```

Die Variablen werden wie folgt belegt:

G|ajahr 12
G|ejahr 12
G|amonat 3
G|emonat 13
G|adatum 1.3.12
G|edatum 31.12.12

Nur wenn **G|ajahr**, **G|ejahr**, **G|amonat** und **G|emonat** bekannt sind, können die Verkehrszahlen (**ssum**, **hsum**, **asaldo**, **esaldo** usw.) und die Kontoauszüge ausgegeben werden. Die dynamischen Plandaten **pjasaldo**, **pasaldo**, **pssum**, **phsum** und **pesaldo** werden besetzt, wenn ein Zeitraum bekannt und ein Konto geholt ist. Wenn der Zeitraum bekannt ist, werden bei den folgenden Zusatzbedingungen die nachstehenden Variablen vorbelegt:

Ein Personenkonto wurde geholt.	umsatz
Zusätzliche Plandaten sind aktiviert.	pumsatz
Ein Teil der Gruppe 1 wurde geholt.	mzugang wzugang mabgang wabgang
Zusätzliche Plandaten sind aktiviert.	pmzugang pwzugang pmabgang pwabgang

Folgende Variablen können einbezogen werden:

gnjahr	n-tes Geschäftsjahr vom Geschäftsjahr. Wird beim Initialisieren vom FOP zusammen mit G gjahr und G abgjahr vorbelegt.
anjahr enjahr	n-tes Geschäftsjahr von G ajahr bzw. G ejahr. Beide werden zusammen mit G ajahr und G ejahr vorbelegt nach folgenden Kommandos: * .hole zeitraum * .hole monat, .hole monat alt, .hole monat neu * .hole jahr

Damit kann das Geschäftsjahr als Differenz zum laufenden Geschäftsjahr angegeben werden.

Geschäftsjahr gleich 12, laufendes Geschäftsjahr ist 12

```
.select year -1 1 3
```

oder

```
.type int xiyear      ? F|defined(U|xiyear) = G|false
.formula U|xiyear = -1
.select year 'U|xiyear' 1 3
```

oder

```
.type text xtyear      ? F|defined(U|xtyear) = G|false
.formula U|xtyear = "-1"
.select year 'U|xtyear' 1 3
```

Geschäftsjahr gleich 14, laufendes Geschäftsjahr ist 13

```
.select year +1 1 3
```

oder

```
.type int xiyear      ? F|defined(U|xiyear) = G|false
.formula U|xiyear = 1
.select year +'U|xiyear' 1 3
```

oder

```
.type text xtyear      ? F|defined(U|xtyear) = G|false
```

```
.formula U|xtyear = "+1"
.select year 'U|xtyear' 1 3
```

Für positive Differenzen und für 0 muss die Variable 'xx' als Textvariable definiert werden, sonst wird als Jahresangabe z.B. erkannt



.hole jahr 0 entspricht dem Jahr 2000
 .hole jahr +0 entspricht dem laufenden Jahr
 .hole jahr -0 entspricht dem laufenden Jahr

Außer der zweistelligen Darstellung der Jahreszahlen von Feldern der Arten "DATUM", "WOCHE", "JAHR" und "GJAHR" gibt es in FOPs die Möglichkeit, Felder dieser Arten mit vierstelliger Jahreszahl auszugeben. Dazu ist an den Variablennamen :4 anzuhängen. Die Möglichkeit besteht allerdings nur, wenn diese Variablen in "TEXT" umgewandelt werden, d.h., wenn sie in Apostrophe eingeschlossen sind.

Beispiel: Vierstellige Jahreszahl

```
.type date xdDate
.formula U|xdDate = "2.3.2010"
'U|xdDate' 'U|xdDate:4'
..
.type week xwWeek
.formula U|xwWeek = U|xdDate
'U|xwWeek' 'U|xwWeek:4'
..
'G|FY' 'G|FY:4'
```

Rumpfgeschäftsjahr

```
.hole jahr 'Geschäftsjahr'.|/ 1|2 'Anfangsgeschäftsmonat'
'Endgeschäftsmonat'
```

Nach dem Geschäftsjahr, gefolgt von einem Punkt oder Slash (/), können Sie angeben, ob es sich beim Rumpfgeschäftsjahr um das erste oder zweite Geschäftsjahr handelt.

Beispiel: Rumpfgeschäftsjahr angeben

Letzter Jahresabschluss im Termindatensatz zum 31.03.10
 Alter Jahresabschluss zum 31.12.09

Kommando	Anfang	Ende
.hole jahr 10.1 1 2	G ajahr G anjahr G amonat G adatum 10 1 1 01.01.10	G ejahr G enjahr G emonat G edatum 10 1 2 28.02.10
.hole jahr 10/2 1 2	G ajahr G anjahr G amonat G adatum 10 2 1 01.04.10	G ejahr G enjahr G emonat G edatum 10 2 2 31.05.10

Vergleich von Geschäftsjahren

Der Vergleich von Geschäftsjahren sollte mit dem FOP **GJAHR.CMP** stattfinden; zu Einzelheiten vgl. den Kopf des Programms. Als Beispiel zum Aufruf vgl. das FOP **auszug**. Der zu vergleichende Zeitraum sollte mit dem Kommando **.hole jahr** gesetzt werden. Zur Abfrage sollte das FOP **VkzRAUM** verwendet werden; das Kommando **.hole jahr** sollte über das FOP **VkzHOLJAHR** aufgerufen werden. Input- und Output-Parameter sind in diesen FOPs beschrieben.

Zugriff auf Datenbankobjekte

Datenbankobjekt	Kommando
Einheit	.hole einheit
Bezugsgröße	.hole bgr
Kostenart/Rechenregel	.hole koart
Kostenarten-BAB-Feld (BAB-Formular)	.hole bab
Umlage	.hole umlage

Bezugsgröße

Das Datenbankobjekt "Bezugsgröße" besitzt Ist- und Planverkehrszahlen. Beide Verkehrszahlenarten dürfen im Normal- und im Wartungsmodus angegeben werden. Den Verkehrszahlen der Bezugsgröße ist keine Währung zugeordnet. Deshalb darf beim FO-Zugriff auch keine Währung angegeben werden:

```
.hole bgr "100"
.hole sperren vkz ibez 00
```

oder

```
.hole sperren vkz pbez 00
```

oder (ohne Schreibzugriff)

```
.hole vkz pbez 00
```


Kostenart

Das Datenbankobjekt "Kostenart" besitzt ebenfalls Ist- und Planverkehrszahlen. Es fällt in dieselbe Rubrik wie Kosten, Kostenstellen und Kostenträger und bekommt deshalb auch dieselben Verkehrszahlenarten. Es können nur Planverkehrszahlen angegeben werden.

```
.select cotype "620"
.continue END          ? G|success = G|false
..
.select vkz actual 13 eur
.continue END          ? G|success = G|false
..
.select lock vkz target 13 eur
.continue END          ? G|success = G|false
..
```

Kostenarten-BAB-Feld

Ein Datenbankobjekt wie "Kostenarten-BAB-Feld" entsteht, wenn ein Konto, das auf eine Kostenart verdichtet wird, zusammen mit einer Kostenstelle oder einem Kostenträger bebucht wird. Das Objekt beinhaltet auch die Identnummern seiner Erzeuger, nämlich die Identnummer der Kostenart und der Kostenstelle oder des Kostenträgers. Für das Objekt gilt dieselbe Logik wie für bereits existierende BAB-Felder.

Für bereits existierende BAB-Felder gibt es Zugriffe wie

```
.hole Kostenfeld "...5400...101"
```

Ein Punkt steht für eine Leerstelle. Die Variable **feldnr** liefert **_5400__101**.

```
.hole Kostenfeld spalte|zeile
```

Für die Kostenarten-BAB-Felder gibt es dieselben Zugriffe, z.B.

```
.hole Koartfeld "...01...102"
```

Ein Punkt steht für eine Leerstelle. Die Variable **feldnr** liefert **01102**

```
.hole feld spalte|zeile
```

Über folgende Felder kann ohne Zusätze selektiert werden. Sie besitzen die Art "NL5" bzw. "NL60".



- **kokst** Kostenfeldnummern - Kostenstellenanteil
- **kotko** Kostenfeldnummern - Kontenanteil
- **kgkst** Kostenstellen/Kostenträger - Nummernanteil
- **kgtko** Kostenart - Nummernanteil

Beispiel: Selektion

```
.select accountdata "%,@sort=(CostFieldLine);EDSField=54000;@language=en"
```

Zuvor muss Folgendes eingetragen werden:

```
.select accountdata "%,@sort=(CostFieldLine);EDSField=`54000;@language=en"
```

Statt des Zugriffs auf die Kostenfelder bzw. Kostenarten-BAB-Felder per **.hole kosten feld** oder **.hole koart feld** ist der Zugriff über **rohlingslose Selektion** zu empfehlen.

Beispiel: Selektion

```
.select accountdata "%,@group=16;EDSField=54000;@language=en"
```

oder

```
.select cotype "%,EDSField=;@gruppe=4;@language=en"
```

Ein Kostenarten-BAB-Feld ist Träger aller Verkehrszahlenarten der neuen BAB-Anwendung:

Ist-Verkehrszahlen	entstehen durch Buchungserzeugung in der Finanzbuchhaltung
Proportionale Planverkehrszahlen	werden im BAB-Formular zeilenweise für die Kostenstelle des Formulars und die Kostenart aus der BAB-Formularzeile geplant
Fixe Planverkehrszahlen	werden im BAB-Formular zeilenweise für die Kostenstelle des Formulars und die Kostenart aus der BAB-Formularzeile geplant
Soll-Verkehrszahlen	werden zeilenweise nach folgender Formel errechnet: $\text{Soll-Kosten} = \text{fixe Plankosten} + \text{prop. Planverrechnungssatz} * \text{Ist-Beschäftigung}$ $\text{prop. Planverrechnungssatz} = \text{prop. Plankosten} / \text{Planbeschäftigung}$

Absolute Soll-Ist-Abweichung-Verkehrszahlen	werden zeilenweise nach folgender Formel errechnet: $\text{Soll-Kosten absolut (Betrag-Abweichung)} = \text{Ist-Kosten} - \text{Soll-Kosten}$
Relative Soll-Ist-Abweichung-Verkehrszahlen	werden zeilenweise nach folgender Formel errechnet: Soll-Kosten relativ (%-Abweichung): Soll-Kosten entsprechen 100 %; Ist-Kosten entsprechen x % $\% \text{-Abweichung} = \text{Ist-Kosten in \%} - \text{Soll-Kosten in \%} = x - 100$

Diese Verkehrszahlen werden unter dem Objekt "Kostenarten-BAB-Feld" (nicht im BAB-Formular) gespeichert und dynamisch in ein BAB-Formular hineingeladen.

Ist-Verkehrszahlen	.hole vkz iko 00 eur
Proportionale Planverkehrszahlen	.hole vkz pprop 00 eur
Fixe Planverkehrszahlen	.hole vkz pfix 00 eur .hole sperren vkz pfix 00 eur
Soll-Verkehrszahlen	.hole vkz soll 00 eur
Absolute Soll-Ist-Abweichung-Verkehrszahlen	.hole vkz siaa 00 eur
Relative Soll-Ist-Abweichung-Verkehrszahlen	.hole vkz siar 00 eur

Umlage

Zugriff auf die Köpfe der Kostenstellenumlagen:

```
.select assessment "$,@sort=Number;grpNo=1;@language=en"
```

Zugriff auf die Tabellenzeilen einer Kostenstellenumlage:

```
.select assessment "%,@sort=Number;@rows=(1);@language=en"
```

Die Zeilen zu einer bestimmten Kostenstellenumlage holen:

```
.type text xtssel
.load 1 assessment "$,@sort=(Number);@group=1"
.continue END          ? G|success = G|false
..
.formula U|xtssel "%,@sort=(StructureOfAssessment);rowNoAssessObj=" + 1|idno
.formula U|xtssel = U|xtssel + ";FY=13;receive="
.formula U|xtssel = U|xtssel + ";@language=en"
.load 2 assessment 'U|xtssel'
```

```
.continue END          ? G|success = G|false
```

Zugriff auf die Verkehrszahlen einer Kostenstellenumlage des Geschäftsjahres 04, Datenart "Ist":

```
.type text xtssel
.load 1 assessment "$,@sort=(Number);@group=1"
.formula U|xtssel = "$,@sort=(TransactionFigures);assessRef=" + 1|idno
.formula U|xtssel = U|xtssel + ";FY=13;dataType=Istuml;costCenter=;@language=en"
.load 2 assessment 'U|xtssel'
```

Bei den Kostenstellenumlagen-Verkehrszahlen gibt es neben der Datenart "IstUml" noch die Datenarten "PIUmlprop" und "PIUmlfix".

Zugriff auf Kostenträgermonatsumlagen:

```
.select assessment "$,@group=6;@language=en"
```

Folgender Zugriff liefert die Zeilen einer bestimmten Kostenträgermonatsumlage:

```
.type text xtssel
.load 1 assessment "$,@gruppe=6"
.formula xtssel = "$,@sort=(MonthlyAssessment);issuingCostCenter=" + 1|issuingCostCenter
.formula xtssel = xtssel + ";FY=" + 1|FY
.formula xtssel = xtssel + ";receive=;@language=en"
.load 2 assessment 'xtssel'
```

Zugriff auf Kostenträgerjahresumlagen:

```
.select assessment "$,@group=7;@language=en"
```

Folgender Zugriff liefert die Zeilen einer bestimmten Kostenträgerjahresumlage:

```
.type text xtssel
.load 1 assessment "$,@group=7"
.formula U|xtssel = "%,@sort=(AnnualAssessment);issuingCostCenter=" + 1|issuingCostCenter
.formula U|xtssel = U|xtssel + ";FY=" + 1|FY
.formula U|xtssel = U|xtssel + ";receive=;@language=en"
.load 2 assessment 'U|xtssel'
```

.HOLE BUCHUNG / .SELECT ENTRY - Buchungszeile holen

.HOLE BUCHUNG	.SELECT ENTRY
deutsch	englisch
.HOLE BUCHUNG .HOLE BUCHUNG " 'Nummer' "	.SELECT ENTRY .SELECT ENTRY " 'number' "
.hole buchung " 'Nummer' " holt eine Buchung	

Beispiel: Kostenverteiler einer Buchungszeile

Auf den Kostenverteiler einer Buchungszeile kann wie folgt zugegriffen werden.

```
.select entry "260"
-- 'H|idno'
.select line
'G|DBNoRecNo'
.select add H|costCenter
'G|DBNoRecNo'
.select line
'H|costCenter' 'H|percent'
```

```
.select entry "260"
-- 'H|idno'
.select line
'G|DBNoRecNo'
.load 1 accountdata 'H|costCenter^id'
.load 1 line
-- '1|costCenter' - '1|percent'
```

Buchungskreise

Um Buchungen buchungskreispezifisch zu holen, wird der entsprechende Schlüssel und zusätzlich die booleschen Variablen für den Buchungskreis angegeben.

Beispiel: Buchungskreispezifische Buchungen

Um Zugangsbuchungen des Buchungskreises "1" zu selektieren, ist folgende Selektion anzugeben:

```
.select entry "%,@sort=(StatementOfFixedAssets);code=ZU;allocSetOfBooks1=yes;@language=en"
```

.HOLE KONTO / .SELECT ACCOUNT - Konto holen

.HOLE KONTO	.SELECT ACCOUNT
deutsch	englisch
.HOLE KONT0	.SELECT ACCOUNT
.hole konto holt das angegebene Konto.	

.SETZE NUMMERNZÄHLER / .SET NUMBERCOUNTER - Nummernzähler setzen

.SETZE NUMMERNZÄHLER	.SET NUMBERCOUNTER
deutsch	englisch
.SETZE NUMMERNZÄHLER 'nummer' varname	.SET NUMBERCOUNTER 'idno' varname
.setze nummernzähler 'identnummer' varname erhöht den Nummernzähler identnummer um 1.	

Der neue Wert des Nummernzählers steht nach dem Aufruf in der benutzerdefinierten Variablen **varname**. Diese muss vom Typ INT oder TEXT sein. Sie müssen einen Pufferbezeichner verwenden. Wenn es sich um eine Textvariable handelt, dann wird die Formatvorlage berücksichtigt.

```

..!interpreter english noabbrev
..
.type int xicounter ? F|defined(U|xicounter) = G|false
.type text xtcounteridno ? F|defined(U|xtcounteridno) = G|false
.formula U|xtcounteridno = "100001"
..
.set numbercounter 'U|xtcounteridno' U|xicounter
.continue ERROR ? G|mehr = G|false
{foc-PRINTLN} new counter value: 'U|xicounter'
.continue END
..
!ERROR
error
..
!END
end of program
.end

```

Statusabfragen und Fehlerbehandlung

- FOP-Abbruch bei Syntaxfehlern.
- FOP-Abbruch bei fehlendem Pufferbuchstaben der Ergebnisvariable **zaehler**.
- FOP-Abbruch bei falscher Art der Ergebnisvariable **zaehler**.
- 'G|mehr' ist false, wenn der Nummernzähler nicht geladen werden konnte oder nicht änderbar ist, ansonsten true. Nur individuelle Nummernzähler können geladen werden.

- 'G|status' ist 0 beim erfolgreichen Erhöhen, 1 bei Überlauf des Zählers (d.h. der Nummernzähler hatte den Maximalwert erreicht und wurde wieder auf den Anfangswert zurückgesetzt).

Holen des zuvor geholten Datensatzes

Mit den folgenden Kommandos können Sie auf zuvor geholte Datensätze zurückgreifen:

Mit jedem **.hole**- Kommando wird ein geholter Datensatz durch einen anderen ersetzt. Um zwischenzeitlich nicht mehr geladene Datensätze schnell wieder greifbar zu machen, können Beschaffungen von Datensätzen gespeichert werden.

.hole speichert, welcher Datensatz zuletzt durch **.hole** beschafft wurde. Mit **.hole** (ohne weitere Angaben) wird der Datensatz erneut geladen. Der Inhalt kann inzwischen durch andere Benutzer verändert worden sein. Er wird in denselben Zustand versetzt wie zum Zeitpunkt des Speicherns. Dies gilt für Folgendes:

- der durch **.hole 'Datenbank' 'Suchbegriffe'** gesetzte Bereich
- bei Datensätzen mit Tabelle die aktuelle Zeile (**.hole zeile**)
- die gesetzten Bezugskonten

Mit **.merke verwendung +** kann der Beschaffungsstatus (siehe [Infosystem BSTATUS](#)) künftig nicht nur den schon eingeplanten Anteil einer Auftragsstückliste nachweisen, sondern die gesamte Auftragsstückliste, soweit sie streng auftragsbezogen ist (Dispositionsart "variantenbezogen").

Siehe auch [Kommandos für Verbrauchsfolgeverfahren](#) und [Kommandos für die Disposition](#).

```
.select parts "1000"  
'H|idno'  
.reserve  
.select parts "10001"  
'H|idno'  
.select  
'H|idno'
```



Der Ausgangsdatsatz (Beschaffung oder Reservierung) muss im H-Puffer oder im D-Puffer stehen. Das heißt, **.merke verwendung** geht nur auf Hauptpuffer und Dazu-Puffer, nicht jedoch bei Maskenobjekten.

Zum Verhalten der **.loeschen**-Puffer beim Kommando **.loeschen** siehe auch Kapitel [.loeschen - Datensatz löschen](#).

.MERKE / .RESERVE - Aktuellen Ladezustand von .hole- und .dazu-Puffer holen

.MERKE	.RESERVE
deutsch	englisch
.MERKE .MERKE FIFO .MERKE LIFO .MERKE BINDUNG .MERKE VERWENDUNG [+] .MERKE -	.RESERVE .RESERVE FIFO .RESERVE LIFO .RESERVE LINK .RESERVE APPLICATION [+] .RESERVE -
.hole speichert die Nummer, den Sperrzustand und die Tabellenzeile des aktuellen Datensatzes, sowie den Suchbereich. Diese Informationen können mit .hole wieder geladen werden. .merke fifo .merke lifo speichert die Nummer des aktuellen Datensatzes in einen Puffer (FIFO = first in, first out, LIFO = last in, first out). Der Datensatz wird immer am Ende des Puffers (der Warteschlange) gespeichert. Der Datensatz kann mehrfach in der Warteschlange gespeichert werden. Aktuelle Obergrenze für die Warteschlange ca. 100000 Einträge. Es kann nur entweder eine fifo- oder eine lifo-Warteschlange geben. Wenn Sie zwischen beiden wechseln, dann wird die Warteschlange initialisiert. Die Datensätze können mit .hole fifo, entsprechend mit .hole lifo wieder geladen werden. .merke bindung holt alle Zugänge zu einem Abgang, die ihn bedient haben oder alle Abgänge zu einem Zugang, in die er eingeflossen ist. In der Datenbankvariable zuganglj wird der Zugangsjournaleintrag und in der Datenbankvariablen abganglj wird der Abgangsjournaleintrag gespeichert. .merke verwendung [+] liefert Beschaffungen zu Materialreservierungen. .merke - führt dazu, dass alle mit .merke, .merke fifo, bzw. .merke lifo gemerkten Objekte aus dem Puffer gelöscht werden. D.h. ein nachfolgendes .hole, .hole fifo, bzw. .hole lifo liefert keinen Datensatz. Der Zustand der Variable G mehr ist G>false.	

.HOLE FIFO / .SELECT FIFO - Ersten Datensatz holen

.HOLE FIFO	.SELECT FIFO
deutsch	englisch
.HOLE FIFO	.SELECT FIFO
.hole fifo entfernt den ersten Datensatz aus der Warteschlange, die zuvor mit .merke fifo angelegt wurde und lädt ihn ungesperrt in den H-Puffer (FIFO = first in, first out).	

.HOLE LIFO / .SELECT LIFO - Letzten Datensatz holen

.HOLE LIFO	.SELECT LIFO
deutsch	englisch
.HOLE LIFO	.SELECT LIFO
.hole lifo entfernt den letzten Datensatz aus der Warteschlange, die zuvor mit .merke lifo angelegt wurde und lädt ihn ungesperrt in den H-Puffer (LIFO = last in, first out).	

FO-Zugriffe auf Sachmerkmalleisten- und Selektionsleistenrohlinge

Sachmerkmalleisten und Selektionsleisten können auch bei der Erstellung von FOPs genutzt werden.

Der Zugriff auf die Selektionsleisten erfolgt über Prozeduren:

'sml'	Zugriff auf Sachmerkmalleiste
'select'	Zugriff auf Selektionsleiste
'lgzeile'	Zugriff auf Objektauswahlzeile

Zusätzlich steht das Kommando **.setze rohling** zur Verfügung. Das standardmäßig verwendete Trennzeichen kann mit dem Kommando **.setze seltrenner** verändert werden.

.SETZE ROHLING / .SET TEMPLATE - Rohling laden

.SETZE ROHLING	.SET TEMPLATE
deutsch	englisch
.SETZE ROHLING SML .SETZE ROHLING SEL ['Identnummer Suchwort' [xdatname xidentnr [xgruppe]]] .SETZE ROHLING SEL "?" xdat xident	.SET TEMPLATE CHB .SET TEMPLATE SEL ['identity number search word' [xdatname xidentnr [xgruppe]]] .SET TEMPLATE SEL "?" xdat xident

.SETZE ROHLING

.SET TEMPLATE

`.setze rohling sm1` lädt den Sachmerkmalleistenrohling des zuvor geholten Objekts (nur für Artikel).

`.setze rohling sel` lädt den ersten Selektionsleistenrohling des zuvor geholten Objekts.

`.setze rohling sel 'Identnummer|Suchwort'` lädt den Selektionsleistenrohling mit der angegebenen Identnummer oder dem angegebenen Suchwort zum zuvor geholten Objekt.

`.setze rohling sel 'Identnummer|Suchwort' xdatname xidentnr [xgruppe]` lädt den Selektionsleistenrohling mit der angegebenen Identnummer oder Suchwort zum zuvor geholten Objekt und liefert zusätzlich die angegebenen Parameter:

<code>xdatname</code>	Name der Datenbank, auf die sich der Rohling bezieht, z.B. "Kunde"
<code>xidentnr</code>	Identnummer des Rohlings
<code>xgruppe</code>	Gruppennummer der Datenbank, auf die sich der Rohling bezieht

`.setze rohling sel "?" xdat xident` liefert Folgendes:

<code>xdat</code>	Namen der Datenbank in der FO-Sprache (nicht in der Bediensprache)
<code>xident</code>	Identnummer der Datenbank

```
.type text xtfiler
.type text xtident
.type text xtident2
.. set template
.set template "?" U|xtfiler U|xtident
.. U|xtfiler contains database name in FO-language
'U|xtfiler' - 'U|xtident'
.. change idno
.formula U|xtident2 = "%" + U|xtident
.select 'U|xtfiler' 'U|xtident2'
.. select a record from database 'U|xtfiler' by template 'U|xtident'
..
.continue END          ? G|success = G|false
'H|idno' - 'H|swd'
..
!END
.end
```

Das Kommando `.setze rohling` liest den entsprechenden Rohling aus dem Firmenstamm und belegt folgende globale FO-Textvariablen:

'G rohueb'	Überschriftszeile enthält die Merkmalsüberschriften durch Striche getrennt, kann bis zu acht Zeilen lang sein
'G rohein'	Einheitenzeile, ebenfalls durch Striche getrennt
'G rohstro'	Obere Strichzeile
'G rohstrm'	Mittlere Strichzeile
'G rohstru'	Untere Strichzeile

Angenommen, es existiere ein Selektionsrohling, bestehend aus den zwei Merkmalen "Identnummer" und "Suchwort", so würden die genannten FO-Variablen folgende Ausgabe erzeugen:

```
'G|rohstro'
'G|rohueb'
'G|rohstrm'
'select'
'select'
'G|rohstru'
```

Identnummer	Suchwort
10001	T1
1000	TEST

Das Layout des Listenkopfes ist identisch mit demjenigen der Maske, mit der über den entsprechenden Rohling selektiert werden kann. Die Ausgabe der Sachmerkmaleiste geschieht weiterhin mit der FO-Prozedur: 'sml'. Ein Aufruf von 'sml' oder 'select' ist auch dann erlaubt, wenn ihm **.setze rohling** nicht vorausgeht. In diesem Fall werden die Listenkopfvariablen jedoch nicht belegt. 'select' würde den ersten passenden Selektionsleistenrohling aus dem Firmenstamm verwenden.

.SETZE SELTRENNER / .SET SELSEPARATOR - Trennzeichen setzen

.SETZE SELTRENNER	.SET SELSEPARATOR
deutsch	englisch
.SETZE SELTRENNER .SETZE SELTRENNER trenner	.SET SELSEPARATOR .SET SELSEPARATOR separator
.setze seltrenner trenner definiert das benutzerdefinierte Trennzeichen. Der Standardzustand kann mit .setze seltrenner wiederhergestellt werden.	

Wird das Standardzeichen umdefiniert, dann erfolgt die Ausgabe der **FOP-Prozeduren** mit offenem linken und rechten Rand, d.h. auf das Trennzeichen zu Beginn und am Ende der Zeile wird verzichtet.

```
..!interpreter english noabbrev
.. ow1/SETZE.SELTRENNER
```

```
.select parts "10001"
-> 'H|select'
>
>
.set selseparator #
-> 'H|select'
>
>
.set selseparator
-> 'H|select'
.end
```

Als Trenner sind folgende Zeichen zulässig:



! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` { | } ~

Abhängig von der Bediensprache können Trennzeichen Bestandteil der Codierung sein.

Zugriff auf einzelne Merkmale einer Sachmerkmalleiste

Um der FO den Zugriff auf die einzelnen Merkmale einer Sachmerkmalleiste zu ermöglichen, kann das **.setze**-Kommando verwendet werden:

Editieren der Sachmerkmalleiste

Nach **.setze sml** wird stets **G|mehr** gesetzt.

Wird beim Editieren einer Sachmerkmalleiste eines der **EFOPs** Maskenprüfungs-FOP oder Maskeneintritts-FOP benutzt, steht im **A-Puffer** das Vaterobjekt zur Verfügung. So liefert z.B. in der Maske **Sachmerkmalleiste** die Variable **'A|nummer'** immer den Inhalt der Identnummer des zugehörigen Teils.

FO-Variablenname

Die Namen der Merkmale werden im Sachmerkmalleistenrohling über das Feld **'smvname'** (Variablenname) definiert. Zur Vergabe von Variablennamen siehe Abschnitt **Eigene Variable**. Die Namen werden darauf geprüft, ob sie mit einem bereits (im Vaterdatensatz) vergebenen Namen kollidieren. Namen dürfen nur für Zusatzfelder in der Leiste vergeben werden. Verweise auf bereits existierende Variablen sind jederzeit unter ihrem ursprünglichen Namen zu erreichen.

Der Zugriff auf die SML-Variable erfolgt über den Pufferbuchstaben **S|**.

Es sind sowohl Lese- als auch Schreibzugriff erlaubt. Existiert in dem geholten Objekt die Sachmerkmalleiste noch nicht, so wird sie mit dem Kommando **.setze** neu angelegt. Die darin definierten Variablen sind leer.

Existiert ein Teil Nr. 1 mit einer Sachmerkmalleiste, bestehend aus Identnummer (Zusatzfeld **qtext**, text) und einem Datumsfeld (Zusatzfeld **qdatum**), so kann die FO folgendermaßen darauf zugreifen:

```
.select parts "1"
.continue END          ? G|success = G|false
.set chb
.continue END          ? G|success = G|false
.formula S|qtext = "alt"
.formula S|qdatum = "10.01.2010"
.. attention: relating to buffer 'characteristicsBar'
!END
end
.end
```

Wurde der Datensatz mit Sperre geholt, kann er mit **.bringe** wie gewohnt zurückgeschrieben werden.

.SETZE SML / .SET CHB - Merkmale der Sachmerkmalreihe

.SETZE SML	.SET CHB
deutsch	englisch
.SETZE SML ['Identnummer Suchwort'] .SETZE SML -	.SET CHB ['identity number search word'] .SET CHB -
.setze sml packt die Merkmale der Sachmerkmalreihe aus.	
.setze sml - löscht die Sachmerkmalreihe. Dabei wird auch der Verweis (Feld sach) entfernt.	

Identnummer oder Suchwort des Sachmerkmalreihenrohlings können direkt angegeben werden:

.setze sml "4711" Wenn ein Teil im **.hole**-Puffer ist, versorgt das Kommando das Teil mit der Sachmerkmalreihe "4711", und zwar auch, wenn bereits eine Sachmerkmalreihe vorhanden ist. Die Merkmale der Reihe sind danach stets leer, auch wenn der Rohling nicht gewechselt wurde. Die im Rohling eingetragenen Edit-Initialwerte werden genau so in die Reihe übernommen wie im Dialogfall über die Maske "Teile". Um die Reihe erstmals zu belegen, bedarf es somit keiner Zuweisung an eine **q**-Variable (siehe Abschnitt [Eigene Variable](#)).

Besonderheit bei Sachmerkmalreihen

Wurde ein Teil geholt, für das keine Sachmerkmalreihe vorhanden ist, öffnet das Kommando **.setze sml** eine Box mit den vorhandenen Sachmerkmalreihen. Hier können Sie das gewünschte Objekt auswählen.

Chargenbindung an die FO

Die folgende Programmsequenz verdeutlicht, wie Sie Chargen in der FO erzeugen:

```
create new record:
.make lots group0
```

```
set fields:
.formula H|swd = "Charge1"
.formula H|descr = "Testcharge"
.formula H|extlotNo = "xy 123 dfg"
.formula H|product = "203"
.formula H|vendor = "1"
.....
save record:
.rewrite
```

Mit dem Befehl **.hole** können Chargen auch aufgegriffen und editiert werden.



Aus Gründen der Datenkonsistenz können der Artikel und der Lieferant bzw. die Eigencharge nur bei der Neuanlage einer Charge per FO gewechselt werden.

Die folgende Programmsequenz verdeutlicht, wie Sie Chargen per FO aufgreifen und ausgeben können:

```
headline
swd          ext-no      product      vendor
!NEXT
.select lots
.continue END ? G|success = G|false
'H|swd      '#H|idno      '#H|product      '#vendor^idno  '
.continue NEXT
!END
.output terminal
end
.end
```

Auch über eine Selektion können nur bestimmte Chargen geholt werden.

für eine rohlingslose Selektion aller Chargen, die mit der gleichen externen Chargennummer beginnen:

```
.. define variables
.type text xttext xtextNo
.. read extern lot no
.read "insert lot number:" U|xtextNo
..
.. define selection criteria
.formula U|xttext = "%,extLotNo=" + xtextNo + " ;@objectselection=dyn;@language=en"
..
.. select lots
.select lots 'U|xttext'
!NEXT
.continue END ? G|success = G|false
'H|extLotNo      ' # 'H|product      ' # 'H|vendor      '
..select next lot
```

```
.select lot
.continue NEXT
!END
.output terminal ...
end
.end
```

Sperren setzen

Weitere Themen finden Sie unter:

- **.hole sperren** - Zugriffssperre beim Holen eines Datensatzes

.BELEGEN / .OCCUPY - Programme synchronisieren

Standardmäßig werden Datensätze, die bearbeitet werden, vom System als gesperrt. Dadurch kann kein weiterer Benutzer den Datensatz ändern. Das System synchronisiert das Ändern der Datensätze. Wenn Sie den Zugriff auf Objekte wie z.B. eine Datei synchronisieren wollen oder um einen Datensatz zu sperren ohne ihn zu lesen, dann können Sie das Kommando **.belegen** verwenden.

.BELEGEN	.OCCUPY
deutsch	englisch
.BELEGEN .BELEGEN 'Stringsemaphor' ['Sperroptionen'] ['Sperrmodifikator'] .BELEGEN 'Datensatz-ID' ['Sperroptionen'] ['Sperrmodifikator']	.OCCUPY .OCCUPY 'string semaphor' ['lock options'] ['lock modifier'] .OCCUPY 'data record ID' ['lock options'] ['lock modifier']

.BELEGEN	.OCCUPY
<code>.belegen</code> ohne weitere Parameter hebt die Sperre auf. <code>.belegen 'Stringsemaphor' ['Sperroptionen'] ['Sperrmodifikator']</code>: Setzt die Sperre auf den Text <code>'Stringsemaphor'</code>. <code>.belegen 'Datensatz-ID' ['Sperroptionen'] ['Sperrmodifikator']</code>: Wenn der Sperrtext das Format einer Datensatz-ID "<code>(sn,dn,0)</code>" hat, wird nicht der Text selbst sondern der Datensatz gesperrt. Der Datensatz wird dabei nicht gelesen, es wird kein Puffer belegt. Es wird nicht geprüft, ob es den Datensatz wirklich gibt. Die <code>.belegen</code> Befehle haben folgende Eigenschaften: • Der Befehl <code>.belegen</code> kann in einem FO-Programm nur eine einzige Sperre setzen. Ein erneuter Aufruf von <code>.belegen</code> hebt die vorherige Sperre auf. • Bei FOP-Ende wird die Sperre automatisch freigegeben. • Der Sperrbefehl kann um eine definierte <code>Sperroption</code> und/oder einen definierten <code>Sperrmodifikator</code> ergänzt werden. Wenn keine Sperroption bzw. kein Sperrmodifikator angegeben wird, dann wird implizit die Sperroption <code>schreibsperre</code> und der Sperrmodifikator <code>abbruch</code> gesetzt. • Wenn zuvor, mithilfe des Kommandos <code>.setze abbruch --</code> der Abbruch des FO-Programms verhindert wird, dann gilt lediglich die Sperroption <code>schreibsperre</code>. • Wenn der Stringsemaphor oder die Datensatz-ID gesperrt werden kann, wird die Variable <code>G gesperrt</code> auf <code>G true</code> gesetzt, ansonsten auf <code>G false</code>.	



Halten Sie die Sperre so kurz wie möglich aufrecht. Führen Sie vorbereitende Aktionen vor dem Belegen der Sperre aus und geben Sie diese unmittelbar nach dem kritischen Abschnitt wieder frei. Achten Sie darauf, dass in der Sperrphase keine Dialoge mit dem Benutzer durchgeführt werden. Das könnte zu sehr langen Sperrdauern führen.

Beispiel:

In diesem Beispiel soll das Ausführen des FO-Programms synchronisiert werden. Durch das FO-Programm `owbhilfe/CONFIG.NEW.OCCUPY` wird der Inhalt einer Datei `owbhilfe/CONFIG.NEW.TXT` geändert. Es ist sinnvoll, dass immer nur ein Prozess (ein FO-Programmaufruf) die Datei ändert. Nachfolgende Prozesse (FO-Programmaufrufe) müssen warten, bis der jeweils vorhergehende Prozess (FO-Programmaufruf) die Sperre freigibt.

Programmaufruf

Beim Aufruf des FO-Programms werden 3 Argumente übergeben die in die Datei geschrieben werden sollen, z.B. `owbhilfe/CONFIG.NEW.OCCUPY customer 1 1111`.

Das Schreiben der Datei darf nur von einem Prozess aus möglich sein. Hierzu wird im Anweisungsteil des FO-Programms das Kommando `.occupy 'Stringsemaphor'` verwendet. z.B. `.occupy`

"**change config**". Dieses Kommando setzt für den Stringsemaphor "change config" eine Sperre mit der Sperroption **schreibsperre** und dem Sperrmodifikator **abbruch** für den aktuellen Prozess (FO-Programmaufruf). Wird das FO-Programm erneut ausgeführt (anderer Prozess), dann läuft es auf diese Sperre und eine Meldung wird ausgegeben.

```

..!interpreter english noabbrev englvar
.. owbfhilfe/CONFIG.NEW.OCCUPY
..
.type text xtfileWrite ? F|defined(U|xtfileWrite) = G|false
..
.formula U|xtfileWrite = "owbfhilfe/CONFIG.NEW.TXT"
..
.continue NOARGS ? G|argc = 1
count argumente: 'G|argc'
new arguments:   'G|arg1' | 'G|arg2' | 'G|arg3'
..
.. open config file
.output new 'U|xtfileWrite'
..
.. lock FO file
.occupy "change config"
..
'G|arg1' # 'G|arg2' # 'G|arg3' #
..
.box Lock
halt - in order to check locking
..
..close config file
.output
.. unlock FO file
.occupy
.continue END
..
!ERROR
cannot write to 'U|xtfileWrite'
.continue END
..
!NOARGS
No arguments
..
!END
end
.end

```

Installationsweite Sperre für alle Mandanten

.belegen 'Stringsemaphor' Installation

Mit diesem Kommando wird eine installationweite Sperre gesetzt. Sie gilt für alle Mandanten einer Installation. Die Wirkung entspricht dem Shellaufruf: **batchsperr.sh -i -t 'Stringsemaphor' -S**.

Rechnungswesenspezifische Sperren extern benutzbar

Folgende Stringsemaphoren können von außen gesetzt werden, z.B. per FO mit **.belegen** oder per **batchsperr.sh**:

Stringsemaphor	sperrt
FINANCIAL_ACCOUNTING	Buchungssynchronisation
FIXED_ASSET_ACCOUNTING	Abschreibungsvorschlag und Anlagenvorgang
MONTHLY_CLOSING	Monatsabschluss
IAAA	Innerbetriebliche Leistungsverrechnung

Damit ist es möglich, in Auswertungen diese Stringsemaphoren abzufragen z.B. mit **.belegen** "FINANCIAL_ACCOUNTING". Damit ist es ferner möglich, die Ausführung von Listen zu unterbinden, wenn das Finanzbuchungsjournal verbucht wird.

Weitere Sperroptionen und Sperrmodifikatoren finden Sie im Kapitel [Sperroptionen bei .belegen](#), [.belegen](#) und [.belegen](#).

.SETZE ALLEIN / .SET SINGLEUSER - Allein-Sperre setzen/lösen



Verwenden Sie das FO-Kommando **.SETZE ALLEIN** / **.SET SINGLEUSER** nicht mehr. Dieses FO-Kommando ist veraltet und wird mit einem der nächsten Upgrades abgeschafft.

.SETZE ALLEIN	.SET SINGLEUSER
Deutsch	Englisch
.SETZE ALLEIN + .SETZE ALLEIN - .SETZE ALLEIN +- .SETZE ALLEIN ++	.SET SINGLEUSER + .SET SINGLEUSER - .SET SINGLEUSER +- .SET SINGLEUSER ++
.setze allein + setzt eine Sperre, die verhindert, dass sich weitere Benutzer im gesperrten Mandanten anmelden können. .setze allein - löscht diese Sperre. .setze allein +- stellt fest, ob die jeweilige Sperre vom Benutzer selbst gehalten wird, falls ja, wird G mehr gesetzt. .setze allein ++ setzt eine Allein-Sperre, die das aktuelle Textkommando überdauert.	

Die [Allein-Sperre](#) wird veranlasst, wenn Aktivitäten im Abas Mandanten den Zugriff nur eines einzigen Benutzers gestatten. Änderungen (z.B. am Firmenstamm) gelingen nur, wenn außer dem Benutzer, der die

Änderung durchführt, kein weiterer Benutzer in den Mandanten eingeloggt ist. Andernfalls erscheint die Meldung "Dazu müssen Sie allein im Mandanten sein!"

.SETZE UNTERBRECHUNG / .SET INTERRUPT - Unterbrechung

.SETZE UNTERBRECHUNG	.SET INTERRUPT
deutsch	englisch
.SETZE UNTERBRECHUNG +	.SET INTERRUPT +
.SETZE UNTERBRECHUNG -	.SET INTERRUPT -
.SETZE UNTERBRECHUNG +-	.SET INTERRUPT +-
.SETZE UNTERBRECHUNG ++	.SET INTERRUPT ++
.setze unterbrechung + setzt eine Unterbrechung .setze unterbrechung - löscht eine Unterbrechung .setze unterbrechung +- stellt fest, ob die jeweilige Unterbrechung vom Benutzer selbst gehalten wird, falls ja, wird G mehr gesetzt. .setze unterbrechung ++ setzt eine Unterbrechung, die das aktuelle Textkommando überdauert	

.setze unterbrechung stoppt Mandanten auch, wenn Benutzer angemeldet sind. Offene Dateien werden geschrieben und nach dem Ende der Unterbrechung erneut geöffnet. Unterbrechungen dürfen nur außerhalb von mit .AKTION geklammerten Abschnitten vorkommen.

Eine Unterbrechung wird veranlasst, um Datenbanken und Datensätze vorübergehend dem Zugriff eines Benutzers zu entziehen. Der Benutzer muss den Mandanten nicht verlassen, die Datenbanken werden (z.B. nach Durchlauf der Sicherung) für ihn wieder geöffnet (batchsperr.sh).

.SETZE SICHERN / .SET SAVE - Sichern-Sperre

.SETZE SICHERN	.SET SAVE
deutsch	englisch
.SETZE SICHERN +	.SET SAVE +
.SETZE SICHERN -	.SET SAVE -
.SETZE SICHERN +-	.SET SAVE +-
.setze sichern + holt die Sichern-Sperre. .setze sichern - gibt sie wieder frei. .setze sichern +- stellt fest, ob die jeweilige Sichern-Sperre vom Benutzer selbst gehalten wird, falls ja, wird G mehr gesetzt.	

Kommandos für landesspezifische Einstellungen

.SETZE SPRACHEINSTELLUNGEN / .SET LANGUAGESETTINGS - Sprachspezifische Ausgaben

.SETZE SPRACHEINSTELLUNGEN	.SET LANGUAGESETTINGS
deutsch	englisch
<code>.SETZE SPRACHEINSTELLUNGEN "T" "P" "0"</code>	<code>.SET LANGUAGESETTINGS "T" "P" "0"</code>
<code>.setze spracheinstellungen "T"</code> setzt den Standardwert. Die Einstellungen wirken wie <code>.setze tsdtrenn</code>. Bei <code>.setze spracheinstellungen "P"</code> wirken auf den Variablenzugriff mit Apostroph in der Ausgabe die Felder der Ausgabesprache. <code>.setze spracheinstellungen "0"</code> gibt sprachabhängige Feldwerte in einer sprachunabhängigen Form aus.	

Betroffen von sprachabhängiger Ausgabe sind alle Felder der folgenden Arten:

- Datum/Zeit
- Aufzählungen
- Zahlen mit Dezimal- und Tausendertrennzeichen
- Boolean
- Verweise

Die Felder der Ausgabesprache sind die für die Lokalisierung relevanten Druckpuffervariablen:

- Sprache `P|lang`
- Tausendertrennzeichen `P|thsdSep`
- Dezimaltrennzeichen `P|decPt`
- Datumsformat `P|dateForm`
- Jahr vor Woche? `P|placeYearBeforeCW`
- Trennzeichen Datum-Uhrzeit `P|dateTimeSep`
- Trennzeichen in der Uhrzeit `P|timeSep`
- Trennzeichen Woche-Jahr `P|weekYearSep`



`.setze spracheinstellungen "0"` wirkt wie `'feld(L="0")'` auch auf Aufzählungen, Verweise und Boolean.

Die Angaben unmittelbar beim Variablenzugriff in der Form `'variable(L=spr)'` haben Vorrang vor der

Wirkung aller Formen des **.setze**-Kommandos.

Die Variablen im P-Puffer lassen sich jederzeit auch direkt setzen. Sie werden aber bei jedem Setzen der Sprache (**P|lang**) vorbelegt.

Für **P|thsdSep** und **P|decPt** sind folgende Angaben möglich:

- das gewünschte Zeichen
- "" für leer (sinnvoll nur bei **P|thsdSep**)

.SETZE DEZPUNKT / .SET DECPOINT - Dezimaltrennzeichen setzen

.SETZE DEZPUNKT	.SET DECPOINT
deutsch	englisch
.SETZE DEZPUNKT .SETZE DEZPUNKT 'Zeichen' .SETZE DEZPUNKT 'G dezpunkt'	.SET DECPOINT .SET DECPOINT 'character' .SET DECPOINT 'G dezpunkt'
.setze dezpunkt setzt das Dezimaltrennzeichen (auch Dezimalpunkt) bei der Ausgabe von Real-Zahlen auf den Standard, je nach der Art des Feldes. .setze dezpunkt 'Zeichen' setzt das Dezimaltrennzeichen auf das angegebene Zeichen. .setze dezpunkt 'G dezpunkt' setzt das Dezimaltrennzeichen auf das Zeichen, das im Konfigurationsdatensatz festgelegt ist.	

```
.type real xnumber
.formula U|xnumber = 4.56
.set decpoint "*"
'U|xnumber' display 4*56
.set decpoint ","
'U|xnumber' display 4,56
```

Das Dezimaltrennzeichen wird in der Variable **G|realsDecPt** abgelegt. Das Trennzeichen beeinflusst die Ausgabe aller Real-Variablen unabhängig von deren Art so lange, bis es aufgehoben wird.

.setze dezpunkt wirkt nur, wenn mit **.setze spracheinstellungen "T"** gearbeitet wird. **.setze spracheinstellungen "P"** und **.setze spracheinstellungen "0"** überlagern die Einstellungen von **.setze dezpunkt**, haben aber keine Auswirkung auf den Wert des Dezimaltrennzeichens oder **G|realsDecPt**.

.setze dezpunkt (ohne Argument) setzt den Standard wie folgt:

- Felder der Arten R und M werden mit dem Punkt als Dezimaltrennzeichen ausgegeben.

- Felder der Art RDT werden mit dem Dezimaltrennzeichen für die Bediensprache aus dem [Konfigurationsdatensatz](#) ausgegeben.

`.setze dezpunkt 0` hat dieselbe Wirkung wie ohne Argument. Es wird der Variablen `G|realsDecPt` der Wert 0 zugewiesen, damit bei der Verwendung von `G|realsDecPt` unterschieden werden kann, ob das Standard-Dezimaltrennzeichen gilt oder das Dezimaltrennzeichen leer ist.

Der aktuelle Wert für das Dezimaltrennzeichen kann mit der Variablen `G|realsDecPt` ermittelt und später wieder bei einem `.setze dezpunkt` als Argument verwendet werden.

```
.type t1 xt1saveDecPoint
.formula U|xt1saveDecPoint= 'G|realsDecPt'
.set decpoint "."
decPoint: .
.set decpoint 'U|xt1saveDecPoint'
decpoint: as before
```

Jetzt gilt wieder das alte Verhalten, gleichgültig, was vorher definiert war.



Wenn das aktuelle Dezimaltrennzeichen leer ist, sind `G|realsDecPt` und `U|xt1saveDecPoint` ebenfalls leer. `.setze dezpunkt 'U|xt1saveDecPoint'` wirkt dann wie ohne Argument; anstatt eines leeren Dezimaltrennzeichens wird das Standard-Dezimaltrennzeichen verwendet.

.SETZE TSDTRENN / .SET THOUSSEP - Tausendertrennzeichen setzen

.SETZE TSDTRENN	.SET THOUSSEP
deutsch	englisch
<code>.SETZE TSDTRENN</code> <code>.SETZE TSDTRENN 'Zeichen'</code> <code>.SETZE TSDTRENN 'G tsdtrenn'</code>	<code>.SET THOUSSEP</code> <code>.SET THOUSSEP 'character'</code> <code>.SET THOUSSEP 'G tsdtrenn'</code>
<code>.setze tsdtrenn</code> setzt das Tausendertrennzeichen bei der Ausgabe von Real-Zahlen auf den Standard, je nach der Art des Feldes. <code>.setze tsdtrenn 'Zeichen'</code> setzt das Tausendertrennzeichen auf das angegebene Zeichen. <code>.setze tsdtrenn 'G tsdtrenn'</code> setzt das Tausendertrennzeichen auf das Zeichen, das im Konfigurationsdatensatz festgelegt ist.	

```
.type real xnumber
.formula U|xnumber = 1432,77
```

```
.set thoussep "."
'U|xnumber' = 1.435,77
.set thoussep " "
'U|xnumber' = 1 435,77
```

Das Tausendertrennzeichen wird in der Variable **G|currThsdSepReal** abgelegt. Das Trennzeichen beeinflusst alle nachfolgenden Real-Variablen unabhängig von deren Art so lange, bis es aufgehoben wird.

.setze tsdtrenn wirkt nur, wenn mit **.setze spracheinstellungen "T"** gearbeitet wird. **.setze spracheinstellungen "P"** und **.setze spracheinstellungen "0"** überlagern die Einstellungen von **.setze tsdtrenn**, haben aber keine Auswirkung auf den Wert des Tausendertrennzeichens oder **G|currThsdSepReal**.

.setze tsdtrenn (ohne Argument) setzt den Standard wie folgt:

- Felder der Arten R und M werden ohne Tausendertrennzeichen ausgegeben.
- Felder der Art RDT werden mit dem Tausendertrennzeichen für die Bediensprache aus dem [Konfigurationsdatensatz](#) ausgegeben.
- Felder der Art RD haben den Wert aus dem Konfigurationsdatensatz für die Bediensprache, werden aber ohne Tausendertrennzeichen ausgegeben.

.setze tsdtrenn "" setzt das Tausendertrennzeichen und damit **G|currThsdSepReal** auf "leer". Diese Einstellung wirkt gleichermaßen für R, M und RDT. Ein leeres Tausendertrennzeichen ist z.B. nützlich bei Ausgaben auf Datei zur Datenübergabe.

.setze tsdtrenn 0 hat dieselbe Wirkung wie ohne Argument. Es wird der Variablen **G|currThsdSepReal** der Wert 0 zugewiesen, damit bei der Verwendung von **G|currThsdSepReal** unterschieden werden kann, ob das Standard-Tausendertrennzeichen oder das leere Tausendertrennzeichen gilt.

Der aktuelle Wert für das Tausendertrennzeichen kann mit der Variablen **G|currThsdSepReal** ermittelt und später wieder bei einem **.setze tsdtrenn** als Argument verwendet werden.

```
.type real xnumber
.type t1 xtmpCurrThsdSepReal
.formula U|xnumber = 1234,55
.formula U|xtmpCurrThsdSepReal = 'G|currThsdSepReal'
'U|xnumber' : 1234.55
.set thoussep "|"
'U|xnumber' : 1|234.55
.set thoussep 'U|xtmpCurrThsdSepReal'
```

Jetzt gilt wieder das alte Verhalten, gleichgültig, was vorher definiert war.



Wenn das aktuelle Tausendertrennzeichen leer ist, sind **G|currThsdSepReal** und

`U|xtmpCurrThsdSepReal` ebenfalls leer. `.setze tsdtrenn`
`'U|xtmpCurrThsdSepReal'` wirkt dann wie ohne Argument; anstatt eines leeren Tausendertrennzeichens wird das Standard-Tausendertrennzeichen verwendet.

System- und Abas Kommandos

.SYSTEM / .SYSTEM - Aufruf von Betriebssystemkommandos

.SYSTEM	.SYSTEM
deutsch	englisch
<code>.SYSTEM 'Betriebssystemkommando'</code> <code>[PASSWORD]</code>	<code>.SYSTEM 'operating system command'</code> <code>[PASSWORD]</code>
<code>.SYSTEM 'Betriebssystemkommando'</code> <code>[PASSWORD] ...</code>	<code>.SYSTEM 'operating system command'</code> <code>[PASSWORD] ...</code>
<code>.SYSTEM 'Betriebssystemkommando'</code> <code>[PASSWORD] EDIT</code>	<code>.SYSTEM 'operating system command'</code> <code>[PASSWORD] EDIT</code>
<code>.SYSTEM 'Betriebssystemkommando'</code> <code>[PASSWORD] HINTER</code>	<code>.SYSTEM 'operating system command'</code> <code>[PASSWORD] BACKGROUND</code>
<code>password</code> gibt unsichtbar ein Einmalpassword weiter. Ohne weitere Option erfolgt eine Ausgabe in das FO-Fenster. Bei <code>...</code> muss die Ausgabe im FO-Fenster quittiert werden. <code>edit</code> bewirkt eine Ausgabe in das FO-Fenster. <code>hinter</code> bewirkt keine Ausgabe.	

`'Betriebssystemkommando'` steht entweder für einen Text in Anführungszeichen (z.B. `"ls -lt [A-Z]*"`) oder für eine FO-Variable in Apostrophen, die das Kommando enthält, z.B. `'U|xsyscommand'`. Inhaltlich wird in beiden Fällen ein Betriebssystemkommando erwartet, wobei auch dessen Optionen und Parameter angegeben werden dürfen. Das angegebene Betriebssystemkommando wird auf dem Anwendungsserver mit Hilfe einer Shell gestartet. Es wird auf das Ende des Kommandos gewartet. Soll nicht gewartet werden, können Sie das mit Hilfe der Shell durch das "&" Zeichen am Kommandoende erreichen. Ist der Exit-Status des Betriebssystemkommandos 0, wird `G|mehr` auf `G|true` gesetzt, sonst auf `G|false`. Die FO-Variable `G|status` (Typ INT) liefert den Exit-Code.

Bei Verwendung der **GUI** wird die Ausgabe des Kommandos erst nach Beendigung des Kommandos angezeigt und das Kommando kann keine Eingabe einlesen.

password	Es wird unsichtbar ein Einmalpasswort weitergegeben. Die Option muss vor allen anderen angegeben werden. Das Einmalpasswort kann für den angemeldeten Benutzer oder für einen anderen Benutzer erzeugt werden. Aufruf des FO-Programms BATCHFOP über batchlg.sh mit der aktuellen Benutzeridentität: <pre>.type text xtsys .formula U xtsys = "batchlg.sh BATCHFOP" .system 'U xtsys' password background</pre> Aufruf eines EDP-Shellskripts mit der aktuellen Benutzeridentität: <pre>.type text xtsys .formula U xtsys = "edpimport.sh -b ... -I daten.csv -P" .system 'U xtsys' password</pre>
keine Option	Das Kommando bewirkt eine Ausgabe in das FO-Fenster.
...	Das Kommando bewirkt eine Ausgabe in das FO-Fenster. Die Ausgabe muss quittiert werden. <pre>.system "ls X*" ...</pre> listet alle mit "X" beginnenden Dateien auf, gefolgt von der Meldung "weiter mit <Return>"
edit	Das Kommando bewirkt eine Ausgabe in das FO-Fenster.
hinter	GUI: Es erfolgt keine Ausgabe. Wenn Sie diese Option nicht nur für einen Aufruf, sondern global setzen möchten, verwenden Sie .setze hintergrund +.

```
.system "sort a b -o c" background
```

sortiert die Dateien **a** und **b** in die Datei **c**. Das Kommando hat keine Ausgabe. Es wird auf die Ausgabe des Kommandos gewartet.

```
.system "batchlg.ctrl.sh -a &" background
```

startet den Batchspooler im Hintergrund. Es wird nicht auf das Ende des Systemkommandos gewartet, sondern sofort das nächste FO-Kommando ausgeführt.

Dieses Kommando kopiert **DATEI.CSV** aus dem Verzeichnis **win** ins Verzeichnis **rmtmp**.

```
.formula U|command = "cp win/DATEI.CSV rmtmp/DATEI.CSV"
.system 'U|command' background
```

.SETZE HINTERGRUND / .SET BACKGROUND - Bildschirminhalt nicht aktualisieren

.SETZE HINTERGRUND	.SET BACKGROUND
deutsch	englisch
.SETZE HINTERGRUND + .SETZE HINTERGRUND -	.SET BACKGROUND + .SET BACKGROUND -
.setze hintergrund + bewirkt, dass nach einem .system-Kommando der Bildschirm nicht geleert und wieder neu aufgebaut wird. Die Ausgaben des Systemkommandos bleiben ungelöscht. .setze hintergrund - bewirkt, dass nach einem .system-Kommando der Bildschirm in jedem Fall gelöscht und wieder neu aufgebaut wird.	

.setze hintergrund + beeinflusst die Wirkung aller folgenden **.system**-Aufrufe ohne Option so, als ob **.system hinter** aufgerufen worden wäre. **.system hinter** wirkt jedoch nur für einen Aufruf.

Die beiden **.setze hintergrund**-Kommandos sind mit Umsicht zu verwenden, da sie in Verbindung mit **.eingabe**-Kommandos leicht zu unerwünschten Effekten führen. Außerdem ist die Wirkung des Kommandos **.system** (ohne Option) nicht mehr erreichbar. Die Verwendung von **.setze hintergrund +** ist sinnvoll, wenn sicher ist, dass das Systemkommando keine Ausgabe bewirkt. In diesem Fall wäre **.setze hintergrund -** nachteilig, weil der Bildschirm flackern könnte.

.KOMMANDO / .COMMAND - Aufruf von Abas Kommandos

.KOMMANDO	.COMMAND
Deutsch	Englisch
<pre>.KOMMANDO -PARALLEL [-ID <Variablenname>] 'Abas Kommando' .KOMMANDO -WARTEN [-ID <M Variablenname>] 'Abas Kommando' .KOMMANDO -FENSTER 'maskid' [-NEU 'Abas '] [-ID <Variablenname>] 'Maskenaktionen' .KOMMANDO -BATCH 'Abas Kommando'</pre>	<pre>.COMMAND -PARALLEL [-ID <M variable- name>] 'Abas command' .COMMAND -WAIT [-ID <variable-name>] 'Abas command' .COMMAND -WINDOW 'maskid' [-NEW 'Abas command'] [-ID <variable-name>] 'screen actions' .COMMAND -BATCH 'Abas command'</pre>
<code>.kommando 'Abas Kommando'</code> führt ein Abas Kommando aus, als wäre es von der Kommandoübersicht aus aufgerufen worden. Mit <code>-ID <Variablenname></code> können Sie den Namen einer FO-Variablen angeben, in die die ID der geänderten Maske geschrieben wird. <code>.kommando -parallel 'Abas Kommando'</code> startet ein weiteres Kommando (Maske, FOP), das parallel zu dem FOP ausgeführt wird, das <code>.kommando</code> aufruft. Es wird nicht gewartet, bis das Kommando beendet ist. <code>.kommando -warten [-id 'M ymaskid'] 'Abas Kommando'</code> startet ein weiteres Kommando (Maske, FOP). <code>.kommando -warten</code> kehrt erst zurück, wenn die Maske wieder geschlossen wurde. Die Variable <code>'M ymaskid'</code> enthält die Identifizierung der Maske; es handelt sich um eine Skipvariable des Datensatzes, damit sie bei einem erneuten Aufruf des EFOPs wieder verwendet kann. <code>.kommando -fenster 'maskid' [-neu 'Abas Kommando'] 'Maskenaktionen'</code> verändert eine bereits geöffnete Maske. Mit der Option <code>-neu</code> wird eine neue Maske geöffnet, falls keine Maske mit der angegebenen ID <code>maskid</code> vorhanden ist. <code>.kommando -batch 'Abas Kommando'</code> führt ein Abas Kommando im Hintergrund aus, indem das Kommando von <code>edpimport.sh</code> ausgeführt wird.	

- [Maske unsichtbar öffnen](#)

Näheres zur Verwendung der Kommandosyntax finden Sie im Kapitel [Kommandosyntax für Abas Kommandos](#). Das Kommando kann parallel oder blockierend ausgeführt werden (s.u.).

Wenn es sich um ein Kommando handelt, das eine Maske öffnet, können auch automatisch [Maskenaktionen](#) ausgelöst werden, z.B. Felder vorbelegen, Buttons klicken, Zeilen einfügen oder löschen. Dies wird genauso überprüft wie eine Eingabe von Hand und es laufen die gleichen EFOPs. Das Maskeneintritts-FOP tritt noch vor den Maskenaktionen ein.



Um im Maskeneintritts-FOP zu erfahren, ob man von einem `.kommando` aus aufgerufen wurde, gibt man dort die Option `-id <Variablenname>` an. In diesem Fall ist in der

aufgerufenen Maske die Variable **G|fensterid** mit dem Wert der im Parameter **-id** angegebenen Variable belegt.

Kommando parallel zum FOP ausführen

```
.kommando -parallel 'Abas Kommando'  
.kommando -parallel -id <Variablenname> 'Abas Kommando'
```

Das Kommando wird parallel zu dem FOP ausgeführt, das **.kommando** aufruft; das heißt, der Befehl kommt sofort zurück. Es wird nicht darauf gewartet, dass beim Ausführen des Kommandos eine Maske geöffnet oder dass die Ausführung des Kommandos beendet wird (siehe aber [Kommando ausführen und auf dessen Ende warten](#) und [Verändern einer bereits geöffneten Datenmaske](#)). Die FO-Variable **G|mehr** wird nicht gesetzt.

Optional können Sie mit **-id <Variablenname>** den Namen einer FO-Variablen angeben. Wenn diese Variable vor dem Aufruf von **.kommando** leer ist, wird vom Programm ein zufälliger, aber eindeutiger Wert bestimmt. Dieser Wert identifiziert das ausgeführte Kommando und alle Datenmasken, die bei der Ausführung des Kommandos geöffnet werden. Wenn die Variable vor dem Aufruf von **.kommando** bereits einen Wert enthält, wird dieser Wert zur Identifizierung des Kommandos verwendet.

Der Wert dieser Variablen kann außerdem bei **.kommando -fenster** verwendet werden, um [einen Eintrag in eine bereits geöffnete Maske](#) vorzunehmen.

In einer Maske, die bei der Ausführung des Kommandos geöffnet wird, steht der Wert in der FO-Variablen **'G|fensterid'** zur Verfügung.



.kommando -parallel steht nur in der GUI und in der EDP-Schnittstelle zur Verfügung. In der EDP-Schnittstelle führt dieses Kommando zur Ausgabe einer [Statusmeldung](#) mit der Meldungsart CMDPARALLEL, in der das auszuführende Kommando als Hinweistext enthalten ist. Ein EDP-Clientprogramm hat dadurch die Möglichkeit, das Abas Kommando in einer sinnvollen Weise auszuführen.

Wenn Sie eine Maske öffnen wollen, in der später mit **.kommando -fenster** Einträge geändert werden sollen, dann sollten Sie **.kommando -fenster 'maskid' -neu 'Abas Kommando' -id <Variablenname> 'Maskenaktionen'** verwenden.



Vorteile:

- Wenn die Maske geschlossen wird, führt dieser Aufruf das Kommando erneut aus, das die Maske geöffnet hat.
- Wenn die Maske mit **.kommando -parallel** geöffnet wurde, dauert es mehrere Sekunden, bis die Maske angezeigt wird. Erst nach Ablauf dieser Zeit kann die Maske mit **.kommando -fenster** geändert werden.

Es soll eine Auftragsmaske geöffnet werden und der Kunde sowie eine Auftragsposition vorbelegt werden. Dies ist mit folgendem FOP erreichbar:

```
.type text xtcmd
.formula U|xtcmd = "<(Sales)> <(New)> ,(SalesOrder) ?
[englvar]=1|customer=1|[insert]=1|1:product=V2"
.command -parallel 'U|xtcmd'
```

Es soll eine Auftragsmaske geöffnet werden. Später sollen Felder in der Maske geändert werden.

```
.type text xtcmd
.type text xtmaskid
.formula U|xtcmd = "<(Sales)> <(New)> ,(SalesOrder) ?
[englvar]=1|customer=1|[insert]=1|1:product=V2"
.command -parallel -id U|xtmaskid 'U|xtcmd'
windowid: 'G|scrWindowId'
screenid: 'U|xtmaskid'
```

Die Variable '**M|xtmaskid**' enthält die (hier automatisch vergebene) Identifizierung der Maske "Auftrag". Es handelt sich um eine Skipvariable des Datensatzes, damit sie bei einem erneuten Aufruf des EFOPs wieder verwendet kann.

```
.type text xtmaskid xtcmd
.formula U|xtmaskid = "SalesOrder" + ":" + G|clientHostName
.formula U|xtcmd="<(Sales)> <(New)> ,SalesOrder ?
[englvar]=1|customer=1|[insert]=1|1:product=V2"
.command -parallel -id U|xtmaskid 'U|xtcmd'
windowid: 'G|scrWindowId'
screenid: 'U|xtmaskid'
```

Hier wird die Identifizierung der Maske vorgegeben. Damit kann ein weiteres FOP, das von diesem PC aus gestartet wird, in der bereits geöffneten Maske Felder ändern.

Kommando ausführen und auf dessen Ende warten

```
.kommando -warten 'Abas Kommando'
.kommando -warten -id 'M|ymaskid' 'Abas Kommando'
```

.kommando -warten führt ein Kommando aus. Der Befehl kehrt erst zurück, wenn die Ausführung des Kommandos beendet ist. Wenn das Kommando eine Datenmaske öffnet, ist dies erst dann der Fall, wenn die Datenmaske geschlossen wurde. Damit ist es zum Beispiel möglich, von einem Maskenprüfungs-FOP aus eine weitere Maske zu öffnen, in der weitere Eingaben gemacht werden müssen, bevor der erste Datensatz gespeichert wird.

Die FO-Variable **G|mehr** wird nicht gesetzt. Es ist nicht möglich, mit Hilfe von **G|mehr** festzustellen, ob bei der Ausführung des Kommandos ein Fehler aufgetreten ist oder die Maske mit "Speichern" oder mit "Abbrechen" verlassen wurde.

Optional können Sie mit **-id <Variablenname>** den Namen einer FO-Variablen angeben. Wenn diese einen Wert enthält, dann kann eine von dem Kommando geöffnete Maske von einem anderen FOP aus mit **.kommando -fenster** verändert werden.



.kommando -warten steht in der GUI und in der EDP-Schnittstelle zur Verfügung. Das FO-Kommando erzeugt eine Nachricht für den EDP-Client. In dieser Nachricht wird dem EDP-Client mitgeteilt, dass ein Abas Kommando auszuführen ist und der Server auf eine Antwort des Clients wartet (**Blocking Action**). Näheres zur Verwendung finden Sie unter **Verwendung in EDP**.

Es soll eine Auftragsmaske geöffnet werden und der Kunde sowie eine Auftragsposition vorbelegt werden. Erst nach dem Speichern der Auftragsmaske wird das FOP fortgesetzt. Dies ist mit folgendem FOP erreichbar:

```
.command -wait "<(Sales)> <(New)> ,(SalesOrder) ?
[englvar]=1|customer=1|[insert]=1|1:product=V2"
if screen is terminated, program will continue here
```

Es soll eine Maske geöffnet und im aufrufenden FOP geprüft werden, ob die Maske mit "Abbrechen" verlassen wurde.

Starten Sie das Kommando, das die Maske öffnet, folgendermaßen:

```
.command -wait "<(Sales)> <(New)> ,(SalesOrder) ?
[englvar]=1|customer=1|[insert]=1|1:product=V2"
```

Definieren Sie ein Maskenaustritts-FOP in der Maske, die mit **.kommando -warten** geöffnet wird. Wenn es durchlaufen wird, wurde die Maske mit "Speichern" verlassen.

```
record stored
```

Das Maskenaustritts-FOP wird nicht aufgerufen, wenn die Maske mit "Abbrechen" verlassen wird.

Es soll das Infosystem Scheckdruck gestartet werden und im aufrufenden FOP soll geprüft werden, welche Schecks gedruckt wurden.

Starten Sie das Infosystem folgendermaßen:

```
.type text xtdat
.formula U|xtdat="X"+pid+".DATA.TMP"
.command -wait -id U|xtdat "Infosystem SCHECKDRK"
```

In einem EFOP dieses Infosystems, z.B. dem Maskenaustritts-FOP, hat die FO-Variable '**G|fensterid**' denselben Wert wie die FO-Variable '**U|xtdat**'. Im Maskenaustritts-FOP kann dieser Wert zum Erzeugen einer Temporärdatei verwendet werden, in die eingetragen wird, welche Schecks gedruckt wurden. Die Datei kann gelesen werden, sobald das Infosystem geschlossen wurde.

Verwendung mit EDP

.kommando -warten kann grundsätzlich in **EDP-Aktionen** verwendet werden. Die Verwendung ist unter anderem für JEDP-Clientprogramme und im Abas Web Client möglich.



Einschränkung

Nicht möglich ist die Verwendung des Kommandos in **edpimport.sh**-Skripten.

Das FO-Kommando **.kommando -warten** erzeugt eine Nachricht für den EDP-Client. In dieser Nachricht wird dem EDP-Client mitgeteilt, dass ein Abas Kommando auszuführen ist. Danach wartet der Server so lange auf eine Antwort des Clients, wie durch das Timeout in der EDP-Serveroption **BLOCKINGACTIONTIMEOUT** eingestellt ist. Kam zu diesem Zeitpunkt noch keine Nachricht vom Client oder hat dieser zwischenzeitlich mit Abbruch reagiert, gilt die Kommandoausführung als gescheitert und das FOP beendet sich mit Fehler.

Verändern einer bereits geöffneten Datenmaske

```
.kommando -fenster 'maskid' 'Maskenaktionen'
```

```
.kommando -fenster 'maskid' -neu 'Abas Kommando' -id <Variablenname>
'Maskenaktionen'
```

```
.kommando -fenster 'maskid' -neu 'Abas Kommando' 'Maskenaktionen'
```

.kommando -fenster 'Maskenaktionen' verändert eine bereits geöffnete Maske. Der Aufruf kehrt erst zurück, wenn das Besetzen der Felder, das Drücken von Buttons und die anschließende Überprüfung abgeschlossen sind. Es können mehrere Aufrufe direkt nacheinander stehen, wenn mehr Vorbelegungen gemacht werden sollen, als in eine einzige FO-Anweisung passen.

Bei **.kommando -fenster 'maskid' -neu 'Abas Kommando' -id <Variablenname> 'Maskenaktionen'** bezeichnet **maskid** die Maske, die durch die **Maskenaktionen** verändert werden soll. Gibt es keine Maske mit dieser ID, wird mit dem **Abas Kommando** eine neue Maske geöffnet. In der Variable **Variablenname** wird die ID der neu geöffneten Maske gespeichert.

Wenn bei der Ausführung eines Kommandos mehrere Masken geöffnet werden, dann kann immer nur die oberste Maske geändert werden. Wenn es mehrere Masken mit dieser Identifizierung gibt, dann wird irgendeine der Masken geändert.

Ein neues Fenster oder eine geänderte Maske wird immer in den Vordergrund geholt. Dies stellt sicher, dass

Änderungen in bereits geöffneten Masken vom Benutzer auch bemerkt werden.

'Maskenaktionen' steht für die zulässigen [Maskenaktionen der Kommandosyntax](#) außer [Maske unsichtbar öffnen](#).

Die FO-Variable `G|mehr` wird auf `G|false` gesetzt, wenn der Aufruf scheitert. Dies kann folgende Ursachen haben:

- Der Benutzer hat die Maske geschlossen.
- Der Benutzer hat [Fenster] [Fenster wiederverwenden] deaktiviert.
- Von der Maske aus wurde ein Freitext oder eine Objektauswahl geöffnet. Die Maske ist dann für Eingaben gesperrt, bis der Freitext oder die Objektauswahl wieder geschlossen werden.
- Der Benutzer hat einen Eintrag in die Maske vorgenommen, dessen Überprüfung aber noch nicht abgeschlossen ist. In diesem Fall steht in der Fensterüberschrift "Bitte warten".
- Die Maske wurde gerade erst mit `.kommando -parallel` geöffnet, wird aber noch nicht angezeigt.



[Fenster wiederverwenden](#) ist aktiv, wenn Sie ein Fenster aus einem EFOP/FOP, bzw. mit den Buttons Info oder Aktion, über Aufrufparameter aufrufen.

Wenn der Aufruf von `.kommando -fenster` scheitert, kann das FOP eine Fehlermeldung ausgeben oder erneut mit `.kommando -parallel` die Maske öffnen. Eleganter ist es, dem Kommando, das die Maske öffnet, gleich beim Aufruf von `.kommando -fenster` die Option `-neu` mitzugeben. Falls keine Maske mit der angegebenen ID vorhanden ist, wird eine neue Maske geöffnet, um dort die Einträge vorzunehmen. Wenn dieses Kommando Leerzeichen enthält (das ist z.B. beim Aufruf von Infosystemen der Fall, siehe das folgende Beispiel), muss die entsprechende FO-Variable in Apostrophe eingeschlossen werden.



`.kommando -fenster` steht nur auf der GUI zur Verfügung.

Eine Infosystemmaske wurde von einem EFOP mit `.kommando -parallel -id` geöffnet. Öffnen Sie z.B. das Infosystem **BESTAND** und belegen den Artikel mit **V2** vor.

```
.type text xtmaskid xtcmd
.formula U|xtcmd="<(Infosystem)> BESTAND <(Call)> ? [englvar]=1 | artikel=V2 | start="
.command -parallel -id U|xtmaskid 'U|xtcmd'
```

Jetzt soll in dieser Infosystemmaske eine andere Auswahl angezeigt werden. Die alte Identifizierung der Infosystemmaske wurde in einem Feld `U|xtmaskid` gespeichert.

```
change content
.formula U|xtcmd="[englvar]=1|artikel=E2|start="
.command -window 'U|xtmaskid' 'U|xtcmd'
Status: 'G|success'
```


Falls das Eintragen der Werte in das Infosystem scheitert, z.B. weil die Maske beendet wurde, wird **G|mehr** auf **G|false** gesetzt.

Eine Infosystemmaske wurde von einem EFOP mit **.kommando -parallel -id** geöffnet. Öffnen Sie z.B. das Infosystem **BESTAND** und belegen den Artikel mit **V2** vor.

```
.type text xtmaskid xtcmd xtcmdneu
.formula U|xtcmd="<(Infosystem)> BESTAND <(Call)> ? [englvar]=1|artikel=V2 | start="
.command -parallel -id U|xtmaskid 'U|xtcmd'
xtmaskid: 'U|xtmaskid'
```

Jetzt soll in dieser Infosystemmaske eine andere Auswahl angezeigt werden. Die alte Identifizierung der Infosystemmaske wurde in einem Feld **U|xtmaskid** gespeichert. Die Infosystemmaske soll bei Bedarf hier geöffnet werden.

```
.formula U|xtcmd = "[englvar]=1|artikel=E2|start="
.formula U|xtcmdneu = "Infosystem BESTAND"
.command -window 'U|xtmaskid' -new "'U|xtcmdneu'" -id U|xtmaskid 'U|xtcmd'
```

Falls das Eintragen der Werte laut **'U|xtcmd'** in die Maske mit der Identifizierung **'U|xtmaskid'** scheitert (z.B. weil das Feld beim ersten Aufruf leer ist), wird das Infosystem erneut geöffnet und danach die Werte laut **'U|xtcmd'** eingetragen. Die neue Identifizierung der Maske muss in einem Feld gespeichert werden, damit bei einem erneuten Aufruf des EFOP die Werte in dieselbe Infosystemmaske eingetragen werden können.

Der Eintrag in **'U|xtcmd'** enthält Leerzeichen. Daher muss die Variable in Anführungszeichen eingeschlossen werden.

Die Identifizierung der Maske kann auch vorgegeben werden. Das ist besser als die automatische Bestimmung, wenn das EFOP nicht von einem Infosystem, sondern zum Beispiel von einer Stammdatenmaske aus aufgerufen wird. In diesem Fall kann die Identifizierung nicht in einem Skipfeld gespeichert werden, weil dieses nach dem Laden eines anderen Datensatzes neu initialisiert wird.

In der Artikelmaske kann von einem EFOP ein Infosystem aufgerufen werden, das die Plankarte des Artikels anzeigt.

```
.type text xtcmd xtcmdneu xtmaskid
idno: 'M|idno'
.formula U|xtcmd = "[englvar]=1|kart=" + 'M|idno' + "|start="
.formula U|xtcmdneu = "Infosystem" + " " + "PLANKARTE"
.formula U|xtmaskid = G|processId + "::-" + "BSTATUS"
.command -window 'U|xtmaskid' -new "'U|xtcmdneu'" 'U|xtcmd'
```

Als Identifizierung wird die Prozessnummer und das Suchwort des Infosystems verwendet.

Ausführen eines Abas Kommandos im Hintergrund

`.kommando -batch 'Abas Kommando'` führt ein Abas Kommando im Hintergrund aus. Dabei wird das Kommando in einem Subprozess mittels `edpimport.sh` ausgeführt. Der Befehl kehrt erst zurück, wenn die Ausführung des Kommandos beendet ist.

Der FO-Entwickler muss dafür weder selbst `edpimport.sh` über das FO-Kommando `.system` aufrufen noch eine Importdatei erstellen.

Fehler, die während der Ausführung auftreten, werden an das Frontend gemeldet; die FO-Variable `G|mehr` wird entsprechend gesetzt.

Per FOP drucken

Das nachfolgende EFOP, das an das Ereignis `Feldaustritt` des Feldes `Speditionsauftrag` eines `Lieferscheins` gekoppelt wird, führt folgende Aktionen aus:

- Startet das Infosystem `Speditionsauftrag drucken`
- Trägt die `id` des Speditionsauftrags aus dem Lieferschein in das Feld `vkspednr` ein
- Klickt den `Start`-Button
- Öffnet die Maske `drucken`, füllt die Felder `Layout` und `drucker` aus und starten den Ausdruck

```
.type text xtprintcmd
.type text xtcmd
.. initialisation print dialog with character protection
.assign U|xtprintcmd = "[englvar]=1|toBePrinted=FRACHTBRIEF|printer=bildschirm|[invisible]=1"
* "+#X"
.. Infosystem + print
.assign U|xtcmd = "(Infosystem) SPEDAUF ? [englvar]=1|vkspednr=" + M|shipOrder^id + "|start="
+ "|print=" + U|xtprintcmd + "|[invisible]=1"
.command -parallel 'U|xtcmd'
```

Der Kommandoparameter `invisible` verhindert, dass nach Abarbeiten der Befehle die Maske offen stehen bleibt. Näheres hierzu finden Sie im Kapitel [Maske unsichtbar öffnen](#).

Lader starten

Um ein Laderskript in ASCII zu verwenden, haben Sie 2 Möglichkeiten:

- In FOPs durch Kommando `.ende "Lesen LADERSKRIPT"`
- In `batchlg`-Aufrufen durch `.system ``"batchlg.sh LESELADER LADERSKRIPT" password hinter`

Dynamische Kommandos

FO-Befehle können einer Textvariablen zugewiesen und mit ihrer Hilfe ausgeführt werden.

Folgende Befehle können in einem EFOP an einem Feldereignis ausgeführt werden, um den Namen und den Wert des Ereignisfeldes mit dem FO-Befehl `.println` auszugeben, ohne dass das konkrete Feld bekannt ist:

```
.type text xtdyn
.formula U|xtdyn = "println -- Variable <'G|evtvar'> has value " + G|dblquote + G|apostroph +
G|evtvar + G|apostroph + G|dblquote
'U|xtdyn'
.'U|xtdyn'
```

Eine mögliche Ausgabe könnte demzufolge sein:

```
println -- Variable <'G|evtvar'> has value "'nort'"
-- Variable <nort> has value "New York"
```

Dynamische Kommandos sind veraltet!

Dynamische Kommandos sollten nicht mehr eingesetzt werden. Sie erschweren die Wartung des Programms und die automatische Übersetzung von FOPs in eine andere Befehlssprache kann scheitern.

In vielen Fällen können stattdessen **FO-Funktionen** eingesetzt werden. So kann der Anwendungsfall in obigem Beispiel durch einen Einzeiler vereinfacht werden:



```
.println -- Variable <'G|evtvar'> has value "'F|eval(G|evtvar)'"
```

Eine mögliche Ausgabe könnte demzufolge sein:

```
-- Variable <nort> has value "New York"
```

Kommandos für die PC-Client-Kommunikation

Über die Windows-Schnittstelle **DDE** (Dynamic Data Exchange) ist es möglich, Daten programmgesteuert zwischen Windows-Anwendungen auszutauschen. Die boolesche Variable `'G|dde'` gibt an, ob dies möglich.

.PC.COPY / .PC.COPY - Datei kopieren

.PC.COPY	.PC.COPY
deutsch	englisch
.PC.COPY [-PC] [-QUIET] [-CONV] [-BIN] 'Quelldatei' 'Zielfeld'	.PC.COPY [-PC] [-QUIET] [-CONV] [-BIN] 'source file' 'target file'
kopiert eine Datei vom Server auf den PC-Client oder umgekehrt. Wenn das Kopieren erfolgreich war, wird G mehr auf G true gesetzt, andernfalls auf G false. -PC gibt an, dass die Zielfeld auf dem PC-Client liegt. Fehlt -PC, liegt die Zielfeld auf dem Server. -CONV konvertiert von dem Abas Zeichensatz in den Zeichensatz UTF-16 (siehe Unicode-Unterstützung). Ist -CONV nicht angegeben, findet keine Konvertierung statt. -QUIET verhindert das Melden von Fehlern. (Statusvariable G mehr wird gesetzt.) Ist -QUIET nicht angegeben, werden Fehler wie fehlende Dateien oder fehlende Verzeichnisse gemeldet. -BIN steht für Binärdatei; es finden keinerlei Konvertierungen statt. Ist -BIN nicht angegeben, wird eine Zeilenende-Konvertierung ausgeführt (Server: New Line, PC-Client: Carriage Return + New Line). Die Optionen -BIN und -CONV dürfen nicht gleichzeitig angegeben werden.	

Enthält ein Pfad Leerzeichen, muss dieser in Anführungszeichen stehen.

Beispiel: Pfad mit Leerzeichen

```
.type text xttext
.formula U|xttext= G|dblquote + "C:\FO\Save" + "\" + "CUSTOMER.TXT" + G|dblquote
.pc.copy -pc -conv rmtmp/CUSTOMER.TXT 'U|xttext'
```

Eine Datei darf auch in ein Verzeichnis unterhalb des GUI-Temporärverzeichnisses '**G|cltempdir**' kopiert werden. In diesem Fall werden fehlende Unterverzeichnisse bei Bedarf angelegt. Bei anderen Zielverzeichnissen ist dies aus Sicherheitsgründen nicht möglich.

Beispiel: Kopieren in ein Verzeichnis unterhalb des GUI-Temporärverzeichnisses

```
.pc.copy -pc -conv /usr/s3/erp/BRIEF.TXT C:\BRIEFE\ERP\BRIEF.TXT
```

Der Name der Client-Datei im FO-Kommando **.pc.copy** kann auch Client-Umgebungsvariablen enthalten.

Beispiel: Umgebungsvariable

```
.pc.copy -PC -BIN part1003.jpg %TMP%\part1003.jpg
```

Nach dem Kopieren einer Datei zum PC-Client kann die Datei mit **.pc.exec** oder **.pc.exec** geöffnet werden.

Lesen Sie hierzu auch Kapitel [Datenaustausch mit externen Programmen](#).

.PC.OPEN / .PC.OPEN - Datei öffnen

.PC.OPEN	.PC.OPEN
Deutsch	Englisch
.PC.OPEN [-DATEI] 'Dateiname' - Keine REST API-Unterstützung .PC.OPEN -AKTION 'Aktion' [-PARAMETER 'Parameter'] [-DATEI] 'Dateiname'	.PC.OPEN [-FILE] 'file name' - No REST API support .PC.OPEN -ACTION 'action' [-PARAMETER 'parameter'] [-FILE] 'file name'
 Das Kommando .pc.open 'Dateiname' wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. .pc.open 'Dateiname' kann eine Datei auf dem PC-Client mit dem geeigneten Programm öffnen. Die Option -datei muss angegeben werden, wenn die Datei auf dem Server unterhalb des win-Verzeichnisses im Mandanten liegt. In diesem Fall muss als Dateiname die relative Pfadangabe ab dem Mandantenverzeichnis verwendet werden. Mit .pc.open -aktion 'Aktion' kann angegeben werden, was das PC-Programm, das gestartet wird, mit der Datei tun soll. Fehlt -aktion 'Aktion', wird die Datei wie bisher geöffnet. Mit .pc.open -aktion 'Aktion' -parameter 'Parameter' kann man einen speziellen Parameter für die angegebene Aktion eintragen. Welche Parameter erlaubt und sinnvoll sind, hängt von der Aktion ab und von dem Programm, das die Datei öffnet.	

Die Dateierweiterung bestimmt, welches Programm verwendet wird, z.B. **.doc** für Microsoft Word. Als Dateiname kann auch eine URL angegeben werden, z.B. <http://www.abas.de>.

Wenn die Datei, die mit **.pc.open** geöffnet werden soll, nicht existiert, wird maximal 10 Sekunden lang immer wieder die Existenz der Datei geprüft. Erst danach wird eine Fehlermeldung ausgegeben. Es wird nur dann mehrfach die Existenz der Datei geprüft, wenn ein absoluter Pfad angegeben wird.

Um die relative Pfadangabe zu umgehen, kann die Datei auch zuerst mit **.pc.open** auf den PC-Client kopiert und danach mit **.pc.open** geöffnet werden.

Beispiel: Auf relative Pfadangabe verzichten

Statt die Datei mit relativer Pfadangabe zu öffnen,

```
.pc.open -file win/excel/bab/roh_bab.xls
```

können Sie wie folgt vorgehen:

```
.pc.copy -bin -pc win/excel/bab/roh_bab.xls C:\temp\roh_bab.xls  
.pc.open C:\temp\roh_bab.xls
```

Sind im Dateinamen Leerzeichen enthalten, muss dieser in Anführungszeichen durch **G|dblquote** eingeschlossen werden, z.B.

.formel xtZiel = G|dblquote + G|cltempdir + G|dblquote

G|cltempdir ist das Temporärverzeichnis auf dem eigenen PC, z.B. **c:\Dokumente und Einstellungen\wk.SYSTEM3\Lokale Einstellungen\Temp**. Weil **G|cltempdir** Leerzeichen enthalten kann, dürfen die Anführungszeichen (**G|dblquote**) nicht fehlen.

Aktionen beim Öffnen der Datei

Welche Aktion bei **.pc.open -aktion 'Aktion'** von einem bestimmten Programm verwendet werden kann, ist nicht so leicht festzustellen. Man muss dazu die Windows-Registrierung einsehen und zu einer Dateierweiterung die Einträge der entsprechenden Anwendung suchen. **.pc.open** ohne **-aktion** funktioniert immer, **.pc.open -aktion print** (Drucken der Datei) funktioniert oft (Druck auf den Standarddrucker). Adobe Reader kann mit **.pc.open -aktion printto** auf einen bestimmten Drucker drucken, der mit der Option **-parameter** angegeben werden muss. Microsoft Word und Excel kennen außerdem **.pc.open -aktion edit**; damit wird die Datei zum Bearbeiten geöffnet, auch wenn auf dem PC die Programme "Word Viewer" und "Excel Viewer" installiert sind und bei **.pc.open** diese Programme gestartet werden.

Beispiel: Drucken auf Standarddrucker

Mit diesem Kommando wird die Datei **abc.pdf** auf dem Standarddrucker ausgegeben.

```
.pc.open -action print abc.pdf
```

Beispiel: Zusätzliche Parameter

Mit diesem Kommando wird die Datei **abc.pdf** auf dem Drucker HP Laserjet ausgegeben.

```
.pc.open -action printto -parameter \\drserver\HPlaserj abc.pdf
```

Der Dateiname im FO-Kommando `.pc.open` kann auch Client-Umgebungsvariablen enthalten.

Beispiel: Umgebungsvariable

```
.pc.open %TMP%\part1003.jpg
```

Wenn ein festgelegtes Programm zum Öffnen der Datei gestartet werden soll, kann auch `.pc.exec` verwendet werden.

.PC.EXEC / .PC.EXEC - Programm ausführen



Das Kommando `.PC.EXEC` / `.PC.EXEC` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

.PC.EXEC	.PC.EXEC
Deutsch	Englisch
<pre>.PC.EXEC 'Programmname' ['Argumente'] .PC.EXEC -WARTEN 'Programmname' ['Argumente']</pre>	<pre>.PC.EXEC 'program name' ['arguments'] .PC.EXEC -WAIT 'program name' ['arguments']</pre>
<code>.pc.exec 'Programmname' ['Argumente']</code> führt auf dem PC-Client das angegebene Programm aus. Das Kommando startet das PC-Programm lediglich, es wartet nicht auf seine Beendigung. Wenn das Programm gestartet werden konnte, wird G mehr auf G true gesetzt, andernfalls auf G false. Es wird keine Fehlermeldung ausgegeben. Der Exit-Code des gestarteten Programms kann nicht abgefragt werden. Die Option -warten bewirkt, dass der Befehl erst nach dem Ende des gestarteten Programms zurückkehrt. Diese Option ist nur auf der GUI verfügbar.	

Beispiel: Programm ausführen

```
.pc.exec format a:
```

Beispiel: Nächsten Befehl warten lassen

```
.pc.exec -wait "mspaint Picture1.png"  
.system "cp win/Picture1.png /u/Bilder" background
```

Das Kommando startet Microsoft Paint. Erst nach dem Schließen von Microsoft Paint wird das Bild **Picture1.png** in den Ordner **/u/Bilder** kopiert.

Viele Windows-Programme (z.B. Microsoft Word) werden nur einmal auf jedem PC gestartet. Wenn das Programm ein weiteres Mal gestartet werden soll, um z.B. eine zweite Datei zu öffnen, wird nur der Dateiname an die zuerst gestartete Programminstanz geschickt. Die zuletzt gestartete Programminstanz beendet sich; die zweite Datei wird von der zuerst gestarteten Programminstanz geöffnet. Bei solchen Programmen kann **.pc.exec -warten** nicht benutzt werden, denn der Aufruf kehrt schon zurück, wenn der Dateiname an die zuerst gestartete Programminstanz gesendet wurde und nicht erst nach der Bearbeitung der Datei.

Zum Start eines Programmes auf dem PC-Client kann auch **.pc.open** verwendet werden.

Wenn ein festgelegtes Programm zum Öffnen der Datei gestartet werden soll, kann auch **.pc.exec** verwendet werden.

Der Programmname und die Argumente im FO-Kommando **.pc.exec** können auch Client-Umgebungsvariablen enthalten.

Beispiel: Umgebungsvariable

```
.pc.exec %ComSpec% /c del %TMP%\part1003.jpg
```

.pc.exec kann grundsätzlich mit EDP verwendet werden. Sowohl mit dem Argument **-warten** als auch ohne. Die Verwendung ist unter anderem für JEDP-Clientprogramme möglich. **Nicht möglich** ist die Verwendung des Kommandos ist in **edpimport.sh**-Skripten.

.pc.exec ohne Option erzeugt eine Nachricht für den EDP-Client. Diese teilt dem EDP-Client mit, dass ein Abas Kommando auszuführen ist. Das FO-Programm läuft anschließend sofort weiter.

.pc.exec -warten erzeugt ebenfalls eine Nachricht für den EDP-Client. Diese teilt dem EDP-Client mit, dass ein Abas Kommando auszuführen ist. In diesem Fall wartet der Server anschließend so lange auf eine Antwort des Clients, wie im Timeout der EDP-Serveroption **BLOCKINGACTIONTIMEOUT** eingestellt ist. Kam bis zur eingestellten Zeit noch keine Nachricht vom Client oder hat dieser zwischenzeitlich mit Abbruch reagiert, dann gilt die Kommandoausführung als gescheitert und **G|mehr** hat den Wert **G|false**.

.PC.CLIP / .PC.CLIP - Inhalt der Zwischenablage holen



Verwenden Sie das FO-Kommando **.PC.CLIP / .PC.CLIP** nicht mehr. Dieses FO-Kommando ist veraltet und wird mit einem der nächsten Upgrades abgeschafft.

<code>.PC.CLIP</code>	<code>.PC.CLIP</code>
Deutsch	Englisch
<code>.PC.CLIP 'Dateiname'</code>	<code>.PC.CLIP 'file name'</code>
Das FO-Kommando <code>.pc.clip</code> holt den Inhalt der Windows-Zwischenablage und legt ihn in der angegebenen Datei ab.	

Der Inhalt der Zwischenablage kann so von einem FOP weiterverarbeitet werden.

Voraussetzung: In der ersten Zeile der Datei muss exakt folgender Text stehen, einschließlich der Punkte und Leerzeichen: `.. EKS-TEXT ..`

Kopplung von Abas an Microsoft Word und Excel



Die Kommandos `.ausgabe W` und `.ausgabe E` sowie das Kommando `.DDE.EXEC` und die vordefinierte Variable `G|ddekanal` werden ausschließlich im Abas Windows Client unterstützt. Sie sind über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

Mit den Kommandos

```
.ausgabe W
.ausgabe W : Vorlage
.ausgabe E
.ausgabe E : Parameter
```

steuern Sie die Ausgabe auf ein Microsoft Word/Excel-Dokument oder eine Word/Excel-Vorlage. Siehe hierzu Kapitel [Ausgabe in Microsoft Word und Excel](#).

Soll während des Programmlaufs ein neues Word-Dokument geöffnet werden, so geschieht dies mit dem Kommando

```
.drucken win.open
```

Danach folgt das Kommando `.drucken`. Dieses gibt den [Initialisierungs-String](#) aus `termcap.txt` auf das neue Dokument aus.

```
.dde.exec 'G|ddekanal' [FileSaveAs .Name="test.doc"]
.dde.exec 'G|ddekanal' [FileClose]
```



Damit bei `.ausgabe W` die richtigen Kommandos an Word gesendet werden, gibt es mehrere Einträge in `termcap.txt`:

<code>winword6</code>	Word 6.0/95 und ANSI Zeichensatz
-----------------------	----------------------------------

winword6A	Word 6.0/95 und ISO 8859-1 Zeichensatz
winword8	ab Word 97 und ANSI Zeichensatz
winword8A	ab Word 97 und ISO 8859-1 Zeichensatz

Optimierung der Ausgabegeschwindigkeit

Die zeilenweise Ausgabe ist bei hohen Zeilenzahlen sehr langsam. Allzu hohe Zeilenzahlen werden von Excel sogar mit einer Fehlermeldung zurückgewiesen, je nach Version z.B. Zeilenzahlen über 16376 oder über 12000.

Es ist besser, zuerst alle Zeilen auf eine Datei auszugeben und dann diese Datei mit Excel zu öffnen. Das dauert bei 10000 Zeilen wenige Sekunden, während mit **.ausgabe E** mehrere Minuten benötigt werden. Außerdem gibt es so keine Beschränkung bei der Anzahl der Zeilen.

Dazu geht man folgendermaßen vor:

1. Datei mit den Daten so anlegen, dass die Werte für die einzelnen Zellen durch Semikolons getrennt sind (csv-Datei)
2. Datei in ein Verzeichnis kopieren, auf das Excel zugreifen kann
3. Datei mit Excel öffnen **.ausgabe E : 'filename'.csv**

DDE-Kommandos in FOPs



Die DDE-Kommandos werden ausschließlich im Abas Windows Client unterstützt. Diese Kommandos sind über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

Zur Entwicklung eigener Anwendungen stehen für die Windows-DDE-Kommandos folgende FO-Kommandos zur Verfügung:

- **.dde.init** - Kanal öffnen
- **.dde.exit** - Kanal schließen
- **.dde.exec** - Befehl senden
- **.dde.send** - Daten senden
- **.dde.request** - Daten anfordern
- **.dde.info** - Liste der Applikationen
- **.dde.wait** - Warten auf Schließen des Dokuments

Über DDE (Dynamic Data Exchange) ist es möglich, Daten im Client-Server-Verhältnis zwischen Windows-Anwendungen auszutauschen. Die boolesche Variable '**G|dde**' gibt an, ob dies möglich. Die **FO-Variable G|ddekanal** enthält die aktuelle DDE-Kanalnummer. Die Ausgabe wird mit dem **.ausgabe-Kommando** gesteuert.

Texte, die als Parameter zu den FO-Kommandos angegeben werden, werden vom Abas Zeichensatz in der Windows-Zeichensatz konvertiert. Ein Text darf maximal 255 Zeichen lang sein.

Näheres hierzu: [Kommandos für PC-Client-Kommunikation](#).

.DDE.INIT / .DDE.INIT - Kanal öffnen



Das Kommando **.DDE.INIT** / **.DDE.INIT** wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

.DDE.INIT	.DDE.INIT
Deutsch	Englisch
.DDE.INIT G ddekanal 'Applikation' 'Thema'	.DDE.INIT G ddekanal 'application' 'topic'
.dde.init öffnet einen Kanal zu einer Applikation mit einem Thema. In der FO-Variablen G ddekanal wird eine Kanalnummer zurückgeliefert, die bei den weiteren Kommandos stets anzugeben ist.	

Ein Kanal zu Microsoft Word oder Microsoft Excel kann auch implizit durch **.ausgabe** geöffnet und in den folgenden Kommandos verwendet werden. Die Kanalnummer erfährt man über die Variable G|ddekanal.

Im Fehlerfall beendet sich das FOP.

.dde.init xk winword dokument13

öffnet einen Kanal zu dem in Microsoft Word geöffneten Dokument **Dokument13** und legt die Kanalnummer in 'xk' ab. Der erste geöffnete Kanal hat die Kanalnummer 0.

.DDE.EXIT / .DDE.EXIT - Kanal schließen



Das Kommando **.DDE.EXIT** / **.DDE.EXIT** wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

.DDE.EXIT	.DDE.EXIT
Deutsch	Englisch
.DDE.EXIT G ddekanal	.DDE.EXIT G ddekanal
.dde.exit schließt den Kanal mit der Nummer G ddekanal.	

`.dde.exit 'xk'` schließt den Kanal Nummer `'xk'`. Am FOP-Ende werden alle offenen DDE-Kanäle automatisch geschlossen.

.DDE.EXEC / .DDE.EXEC - Befehl senden



Das Kommando `.DDE.EXEC` / `.DDE.EXEC` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

<code>.DDE.EXEC</code>	<code>.DDE.EXEC</code>
Deutsch	Englisch
<code>.DDE.EXEC G ddekanal 'Befehl'</code>	<code>.DDE.EXEC G ddekanal 'command'</code>
<code>.dde.exec</code> sendet einen Befehl an eine Anwendung mit der Nummer <code>G ddekanal</code> . <code>G mehr</code> wird gesetzt.	

Im Fehlerfall beendet sich das FOP.

Weitere Informationen siehe [Kopplung von Abas an Microsoft Word und Excel](#).

Falls der Kanal zu Microsoft Word geöffnet wurde:

`.dde.exec 'xk' [InsertPara]`

fügt einen Absatz in das Dokument ein, zu dem zuvor mit `.dde.init` ein Kanal mit der Nummer `'xk'` geöffnet wurde.

.DDE.SEND / .DDE.SEND - Daten senden



Das Kommando `.DDE.SEND` / `.DDE.SEND` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

<code>.DDE.SEND</code>	<code>.DDE.SEND</code>
Deutsch	Englisch
<code>.DDE.SEND G ddekanal 'Objekt' 'Daten'</code>	<code>.DDE.SEND G ddekanal 'item' 'value'</code>
<code>.dde.send</code> sendet Daten zu einem Objekt in einer Anwendung mit der Nummer <code>G ddekanal</code> . <code>G mehr</code> wird gesetzt.	

Im Fehlerfall beendet sich das FOP.

Falls der Kanal zu Microsoft Excel geöffnet wurde:

`.dde.send 'xk' Z2S3 Beispiel`

sendet den Text "Beispiel" in die Zelle C2 (Zeile 2 Spalte 3) einer zuvor mit `.dde.init` geöffneten Excel-Tabelle.

.DDE.REQUEST / .DDE.REQUEST - Daten anfordern



Das Kommando `.DDE.REQUEST` / `.DDE.REQUEST` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

<code>.DDE.REQUEST</code>	<code>.DDE.REQUEST</code>
Deutsch	Englisch
<code>.DDE.REQUEST G ddekanal 'Objekt' request</code>	<code>.DDE.REQUEST G ddekanal 'item' request</code>
<code>.dde.request</code> fordert Daten von einer Anwendung mit der Nummer <code>G ddekanal</code> an und legt sie in der FO-Variablen <code>request</code> ab. Das Objekt wird ebenfalls angegeben. <code>G mehr</code> wird gesetzt.	

Im Fehlerfall beendet sich das FOP.

Falls der Kanal zu Microsoft Excel geöffnet wurde:

`.dde.request 'xk' Z1S3 xt`

holt den Inhalt der Zelle C1 (Zeile 1 Spalte 3) aus einer Excel-Tabelle und legt ihn in `'xt'` ab.

.DDE.INFO / .DDE.INFO - Liste der Applikationen



Das Kommando `.DDE.INFO` / `.DDE.INFO` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

<code>.DDE.INFO</code>	<code>.DDE.INFO</code>
Deutsch	Englisch
<code>.DDE.INFO 'Applikation' 'Thema' info</code>	<code>.DDE.INFO 'application' 'topic' info</code>

.DDE.INFO	.DDE.INFO
liefert eine Liste der aktuellen DDE-Server-Applikationen. Mit 'Applikation' bzw. 'Thema' lässt sich die Auswahl einschränken. Das Ergebnis steht in der FO-Variablen <code>info</code> . <code>G mehr</code> wird gesetzt.	

```
.dde.info "winword" "" xt
```

liefert z.B. "winword, C:\Users\meier\Desktop\Bestellung.docx\twinword, System", wenn in Microsoft Word die Datei `C:\Users\meier\Desktop\Bestellung.docx` geöffnet ist.

```
.dde.info "ABAS-EKS_DEMO" "SYSTEM" xt
```

`G|mehr` liefert dann "ja", wenn eine GUI im Mandanten DEMO läuft. Zum DDE-Servernamen der GUI siehe [Konfiguration des DDE-Servers](#).

.DDE.WAIT / .DDE.WAIT - Warten auf Schließen des Dokuments



Das Kommando `.DDE.WAIT` / `.DDE.WAIT` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

.DDE.WAIT	.DDE.WAIT
Deutsch	Englisch
.DDE.WAIT	.DDE.WAIT
<code>.dde.wait</code> wartet darauf, dass das zuvor mit <code>.ausgabe W</code> oder <code>.ausgabe E</code> erstellte Dokument geschlossen wird.	

Unterprogramm DDE.MENUE

1. Die boolesche Variable '`xybmit`' setzen

```
.formel xybmit = G|true ? menue = mit Excel-Ausgabe
.formel xybmit = G|false ? menue = ohne Excel
```

2. Unterprogramm DDE.INI starten, wenn `xybmit = G|true`

DDE.MENUE kann von einem Upgrade überschrieben werden.

Unterprogramm DDE.INI

Kundenspezifisch, für jedes FOP mit Excel-Anbindung werden folgende FOP-Variablen definiert:

```
.formel xy1dde1 = "1" ? xytlgp = "FOP_Name" # Tabellentyp
.formel xy2dde2 = "2" ? xytlgp = "FOP_Name" # Diagrammtyp (Säulen, Linien
etc.)
.formel xy3dde3 = "4" ? xytlgp = "FOP_Name" # Diagrammformat (xy3dde ist
von xy2dde abhängig)
```

Zusammenhang FO-Kommando - DDE-Kommando - Word/Excel-Befehl

FO-Kommando	DDE-Kommando	Word/Excel-Befehl
.dde.init	DDEInitiate	KANAL.ÖFFNEN
.dde.exit	DDETerminate	KANAL.SCHLIESSEN
.dde.exec	DDEExecute	AUSFÜHREN
.dde.send	DDEPoke	SENDEN
.dde.request	DDERequest	ABFRAGEN
.dde.info	-	-
-	DDETerminateAll	-

Beachten Sie auch die Hinweise in der Onlinehilfe der jeweiligen Windows-Programme.

Kommandos für sprachunabhängige Programmierung

.UEBERSETZEN / .TRANSLATE - Text in gewünschter Sprache holen

.UEBERSETZEN	.TRANSLATE
deutsch	englisch
.UEBERSETZEN -SPRACHE 'Kürzel' -NUMMER 'Nummer' -WOERTERBUCH 'Kürzel Nummer' -VARIABLE "Name" [-SCHLIESSEN]	.TRANSLATE -LANGUAGE 'code' -NUMMER 'number' -DICTIONARY 'code number' -VARIABLE "name" [-CLOSE]
.UEBERSETZEN -SPRACHE 'Kürzel' -TEXT 'Text' [-WOERTERBUCH 'Kürzel Nummer'] -VARIABLE "Name" [-NRVARIABLE "Name"] [-WBUCHVARIABLE "Name"] [-GROSSKLEIN] [-QUELLSPRACHE 'Kürzel'] [-SCHLIESSEN]	.TRANSLATE -LANGUAGE 'code' -TEXT 'text' [-DICTIONARY 'code number'] -VARIABLE "name" [-NUMVARIABLE "name"] [-DICTVARIABLE "name"] [-CAPLOWER] [-SOURCELANGUAGE 'code'] [-CLOSE]
.UEBERSETZEN -SCHLIESSEN	.TRANSLATE -CLOSE

.UEBERSETZEN		.TRANSLATE
Bei -sprache bezeichnet ' Kürzel ' das Sprachkürzel. Es wird 1-buchstabig (D , E , ...) oder als ISO-Kürzel (de , en , ...) angegeben. Sprachkürzel ":" liefert die sprachunabhängige Notation.		
Bei -nummer gibt ' Nummer ' die Nummer des Textes im Wörterbuch an, der gelesen werden soll.		
Bei -woerterbuch bezeichnet ' Kürzel Nummer ' das Wörterbuch:		
Wörterbuch	Kürzel	Nummer
Programmmeldungen	cc	0
Hilfemenüs	sh	1
FOPs	lg	2
.text -Kommandos in FOPs	tx	3
Masken	ma	4
Kommando-Menüs	me	5
Individuelle Schlüsselnamen	ku	6
Bedeutungen in Variablentabellen	vt	7
Property-Dateien	pr	8
Bei -variable bezeichnet ' Name ' eine FO-Variable der Art "text", in der die Übersetzung des gesuchten Textes abgelegt wird. Wird der Suchtext nicht gefunden, wird dieser in der Variablen abgelegt.		
-schliessen schließt das geöffnete Wörterbuch.		
Bei -text bezeichnet ' Text ' den deutschen Text, der gesucht und übersetzt werden soll.		
Bei -nrvariable bezeichnet ' Name ' eine FO-Variable der Art "int", in der die Nummer des gefundenen Textes abgelegt wird.		
Bei -wbuchvariable bezeichnet ' Name ' eine FO-Variable der Art "text", in der das Kürzel des Wörterbuchs, in dem der Text gefunden wurde, abgelegt wird.		
-grossklein bewirkt, dass die Groß-/Kleinschreibung von ' Text ' und die Schreibweise der deutschen Umlaute keine Rolle spielt (Normtext).		
Bei -quellsprache bezeichnet ' Kürzel ' das Sprachkürzel der Quellsprache. Es wird 1-buchstabig (D , E , ...) oder als ISO-Kürzel (de , en , ...) angegeben. Fehlt die Option wird "de" angenommen.		

Das Kommando **.uebersetzen** bietet Zweierlei:

- Holt einen Text mit einer bekannten Nummer in einer gewünschten Sprache aus einem bestimmten Wörterbuch
- Sucht einen deutschen Text in einem Wörterbuch in der gewünschten Sprache
Fehlt die Option **-woerterbuch**, wird in allen Wörterbüchern nach dem Text gesucht. Es kann eine

Liste von Wörterbüchern angegeben werden.

Sucht den Text "Überlauf des LIFO/FIFO-Puffers" im Wörterbuch **msg.cc.dic** und liefert das Ergebnis in Englisch.

```
..!interpreter english noabbrev
.type text xttext1 xttext2 xtorg
.formula U|xtorg = "Überlauf des LIFO/FIFO-Puffers"
.translate -language e -dictionary cc -text 'U|xtorg' -variable U|xttext1
.continue END ? G|mehr = G|false
'U|xttext1'
..
!END
Ende
.end
```

oder:

```
..!interpreter english noabbrev
.type text xttext1 xttext2 xtorg
.formula U|xtorg = "Überlauf des LIFO/FIFO-Puffers"
.translate -language e -dictionary cc -text "Überlauf des LIFO/FIFO-Puffers" -variable
U|xttext2
.continue END ? G|mehr = G|false
'U|xttext2'
..
!END
Ende
.end
```

Beide Beispiele liefern folgendes Ergebnis.

```
LIFO/FIFO buffer overflow
```

Die Wörterbücher **msg.ma.dic**, **msg.tx.dic** und **msg.cc.dic** werden nacheinander nach dem Text "Kunde" durchsucht und die erste Übersetzung aus den drei Wörterbüchern wird in Englisch zurückgeliefert.

```
..!interpreter english noabbrev
.type text xttext
.translate -language E -dictionary ma,tx,cc -text "Kunde" -variable U|xttext
.continue END ? G|mehr = G|false
'U|xttext'
..
!END
```

.end

Customer



Wenn der Suchtext gefunden wird, dann wird die Übersetzung in die Variable geschrieben. Der Wert von G|mehr wird auf ja gesetzt.
Wenn der Suchtext nicht gefunden wird, dann wird der Suchtext in die Variable geschrieben. Der Wert von G|mehr wird auf nein gesetzt.

.ATEXT / .ATEXT - Text einer Programmmeldung übersetzen

.ATEXT	.ATEXT
deutsch	englisch
.ATEXT [-SPRACHE 'Sprache'] 'Variablenname' 'Meldungsnummer' ['Text']	.ATEXT [-LANGUAGE 'language'] 'variable name' 'report number' ['text']
.atext holt aus dem Wörterbuch msg.cc.dic den Text mit der Meldungsnummer in der gewünschten Sprache in die angegebene FO-Variable.	

Die Sprache kann entweder als einbuchstabiges Sprachkürzel "D", "E", "F", ... oder als ISO-Kürzel "de", "en", "fr", ... angegeben werden. Standardwert für die Sprache ist Deutsch. Der letzte Parameter 'Text' ist reiner Kommentar und kann auch entfallen.



Statt des Kommandos .uebersetzen kann das leistungsfähigere .uebersetzen verwendet werden.

Damit das Übersetzungsprogramm der FO-Kommandos alle Kommandos erkennen und übersetzen kann, sind bestimmte Regeln zu beachten. Wird ein Kommandowort in eine Textvariable gestellt, wird es nicht als Kommando erkannt und bleibt auf Deutsch. Abas meldet bei Ausführung eines solchen FOPs einen Fehler bezüglich eines unbekannten Kommandos.

```
.type text xttext
.atext -language 'G|FOPCmdLang' U|xttext 3397 Formel
command: 'U|xttext'
```

Das obige .atext-Kommando holt den Text der Nr. 3397 in der Sprache 'G|lsprache' aus msg.cc.dic. Nach der Ausführung des Kommandos .atext steht in 'xcmd' entweder FORMEL oder FORMULA, je nach FO-Kommandosprache.

```
.type text xttext
.atext U|xttext 1
Text: 'U|xttext'
```

'U|xttext' enthält obigen Text mit der Nr. 1 aus **msg.cc.dic** in Deutsch.

.TEXT / .TEXT - Text einer FOP-Ausgabe übersetzen

.TEXT	.TEXT
deutsch	englisch
.TEXT 'Variable' 'Sprache' 'Text'	.TEXT 'variable' 'language' 'text'
'Variable' gibt den Namen einer FO-Variablen an, die den auszugebenden Text aufnehmen soll. Für 'Sprache' kann 'G kspr' (Korrespondenzsprache) oder ein Sprachkürzel eingesetzt werden, entweder 1-buchstabig (D, E, ...) oder als ISO-Kürzel (de, en, ...). 'Text' ist die Schlüsselinformation für den auszugebende Text in Deutsch.	

.text steuert eine sprachgebundene Textausgabe, indem es aus dem Wörterbuch **msg.tx.dic** den Text in der gewünschten Sprache in die angegebene FO-Variable holt. Ein mit .text aus **msg.tx.dic** geholter Text kann maximal 500 Zeichen lang sein. **Wörterbücher** sind vorbereitet und können jederzeit ergänzt werden.

Unter dem zuletzt gelesenen Text wird als Unterstreichung eine Linie dargestellt. Folgende Variablen werden beim Kommando .text belegt:

'G|strich1' mit "-"

'G|strich2' mit "=".

Holt den zur Korrespondenzsprache passenden Bezeichner für Angebot.

```
..!interpreter english noabbrev
.type text xttext
.text U|xttext E Angebot
'U|xttext'
'G|strich2'
```

display:

```
Quotation
=====
```



Wenn der Suchtext gefunden wird, dann wird die Übersetzung ausgegeben, ansonsten wird der Suchtext ausgegeben.

Vgl. hierzu auch das Kapitel [FO-Variablen bei Mehrsprachigkeit](#).

.SETZE TRANS / .SET TRANS - Von Übersetzung ausklammern

.SETZE TRANS	.SET TRANS
deutsch	englisch
.SETZE TRANS + .SETZE TRANS -	.SET TRANS + .SET TRANS -
.setze trans + beendet die Wirkung von .setze trans - und stellt die Standardeinstellung wieder her. .setze trans - lässt das Übersetzungsprogramm trans (ASCII) bzw. das Infosystem TRANS (GUI) alle folgenden Zeilen ignorieren. Sie werden also weder übersetzt noch in das Wörterbuch msg.lg.dic aufgenommen.	

In mehrsprachigen Mandanten wird kein FOP übersetzt. In den Sprachverzeichnissen liegen keine Arbeitsbereiche oder FOPs. Der FO-Kern übersetzt die Texte zur Laufzeit. Dadurch wird eine Übersetzung nur dann vorgenommen, wenn das FOP kein Kommando .setze trans enthält, oder in diesem FOP zuletzt .setze trans + ausgeführt wurde.

Wenn ein FOP mit .eingabe eingezogen wird, gilt dort standardmäßig .setze trans +. Wird wieder zum aufrufenden FOP zurückgekehrt, gilt dort wieder dessen alter Wert.

Da es in FOPs viele Sprungmarken und Sprünge gibt, wird es vorkommen, dass .setze trans übersprungen werden und dann Texte nicht übersetzt werden. Im anderen Fall könnten Texte, die nicht übersetzt werden dürfen, aber dennoch im Wörterbuch stehen, fälschlicherweise übersetzt werden.

Damit nicht zu viele .setze trans-Kommandos eingebaut werden müssen, werden die Texte der folgenden FO-Kommandos immer vom Kern übersetzt und können nicht mit .setze trans übersteuert werden:

- .fehler
- .hinweis
- .box
- .lesen
- .chart

Bei allen Kommandos wie <Text> <Neu> wird ein u.U. angegebenes Sprachverzeichnis ignoriert, d.h. <Text> A/DRUCKER <Ändern> wird zu <Text> DRUCKER <Ändern>.

Mit diesem Programmverhalten kann das übersetzte FOP als Generat im Sprachverzeichnis weder fehlen noch veraltet sein. Wenn ein FOP geändert oder in einen Mandanten kopiert wurde, ist das Programm sofort aktuell und lauffähig.

.SETZE MENUTRANS / .SET MENUTRANS - Markieren als Menüzeilen

.SETZE MENUTRANS	.SET MENUTRANS
deutsch	englisch
.SETZE MENUTRANS + .SETZE MENUTRANS -	.SET MENUTRANS + .SET MENUTRANS -
.setze menutrans + markiert für das Übersetzungsprogramm trans die folgenden Zeilen als Menüzeilen. .setze menutrans - beendet das Markieren der Zeilen.	

Im Übersetzungsprogramm **trans** (ASCII) bzw. im Infosystem **TRANS** (GUI) werden alle Zeilen, die zu einem **.menu**-Kommando gehören und zwei aufeinander folgende Größer-Zeichen (> >) enthalten, gesondert behandelt. Wenn aber die Menüalternativen eines **.menu**-Kommandos im FOP verteilt sind, können sie von **trans** bzw. TRANS nicht als zusammengehörig erkannt werden. Das ist dann der Fall, wenn nach einem **.menu** mit **.weiter** in einzelne FOP-Abschnitte gesprungen wird und dort eine **.menu**-Zeile der Form "Menüzeile 1" >> **.formel x = 1** geschrieben wurde. Diese Zeile liegt außerhalb des **.menu**-Kommandos und muss deshalb mit **.setze menutrans** als Menüzeile gekennzeichnet werden.

Temporäre Datei für dynamisches Menü erzeugen:

```
.type text xtdot
.type int xiselection
.formula xtdot = "."
.output new tmp
'U|xtdot' menu
.set menutrans +
one >> .formel U|xiselection = 1
two >> .formel U|xiselection = 2
end >> .end
'U|xtdot' 'U|xtdot'
.set menutrans -
.output TERMINAL
.input tmp
Selection = 'U|xiselection'
```

Sonstige FO-Kommandos

.HILFE / .HELP - Hilfefunktion



Verwenden Sie das FO-Kommando **.HILFE** / **.HELP** nicht mehr. Dieses FO-Kommando ist veraltet und wird mit einem der nächsten Upgrades abgeschafft.

.HILFE	.HELP
Deutsch	Englisch
.HILFE .HILFE 'Kapitel'	.HELP .HELP 'chapter'
.hilfe ruft ein Hilfefunktion auf.	

Das Kommando **.hilfe** zeigt Hilfefunktion an. Auch firmeninterne, individuelle Kapitel können angegeben werden. Über **EFOPs** kann so z.B. die individuelle Hilfe auf einen Button gelegt werden.

Da auch das aktuelle Maskenfeld bekannt ist, kann auf diese Weise die feldorientierte Hilfe aufgerufen werden.

.SETZE / .SET - Zeichen mit Sonderbedeutung festlegen

.SETZE	.SET
deutsch	englisch
.SETZE .SETZE 'Schlüsselwort' [= 'Zeichen']	.SET .SET 'key word' [= 'character']
.setze schaltet die Voreinstellung wieder ein.	
.setze 'Schlüsselwort' = 'Zeichen' definiert für das Schlüsselwort ein neues Zeichen.	

Einige Zeichen stehen nicht für sich selbst, sondern haben in FOPs eine Sonderbedeutung. Diese Zeichen lassen sich mit dem Kommando **.setze** umdefinieren. Das Gleichheitszeichen darf fehlen. Es wird unterschieden zwischen

- syntaktischer Bedeutung, z.B. Einleiten von Kommandos oder Bedingungen
- Bedeutung für die Formatierung, z.B. geschützte Leerzeichen, Zeilentrennzeichen



Für **'Zeichen'** dürfen nur ASCII-Zeichen, die keine Buchstaben oder Ziffern sind, eingesetzt werden. Ebenso können keine Variablen eingesetzt werden.



Verwenden Sie das Feature des Umdefinierens so selten wie möglich, da Sie die Syntax ihres Programm dann nicht mehr einfach erkennen. Zudem könnten Sie bei späteren kleineren Änderungen des Programms leicht übersehen, dass nicht die Standardeinstellungen aktiv sind. Schließlich können auch Werkzeuge, die zum Beispiel

nach einem bestimmten Variablennamen in Apostrophen suchen, nicht mehr das erwartete liefern.

Verwenden Sie statt dessen z.B. die Standardvariablen '**G|apostroph**' bzw.

'**G|kommandochar**', wenn Sie verhindern wollen, dass der FO-Interpreter die Zeichen Apostroph bzw. Punkt syntaktisch interpretiert.

Die möglichen Schlüsselworte werden in der folgenden Tabelle aufgeführt:

Schlüsselwort	Standard-Einstellung	FO-Variable	Funktion
kommando	.	kommandochar	syntaktisch, in Spalte 1: zur Angabe von Kommandos
kein Schlüsselwort	:	kommando2char	syntaktisch, in Spalte 1: zur Angabe von ausgabeabhängigen Kommandos
marke	!	markechar	syntaktisch, in Spalte 1: zum Setzen von Marken
frage	?	fragechar	syntaktisch: zur Angabe von Bedingungen
ersetzung	'	ersetzungchar	syntaktisch: zur Ersetzung von Variablen
trenner	;	trennerchar	formatierend: Zeilenende in Variableninhalten
leer	—	leerchar	formatierend: zur Kennzeichnung von geschützten Leerzeichen

Wie in der Tabelle zu sehen ist, ist für alle Sonderzeichen eine FO-Variable definiert, die den aktuellen Wert des Sonderzeichens enthält. Mit ihr können Sie den aktuellen Wert für ein Sonderzeichen merken und später wieder herstellen.

Für **Zeichen** können in **.setze Schlüsselwort Zeichen** keine Variablen eingesetzt werden. Verwenden Sie stattdessen ein dynamisches Kommando wie in folgendem Beispiel:

```
.type text command
.type text textSeparatorSemicolon
.type text textSeparatorPercent
.. default separator is ;
.formula U|textSeparatorSemicolon = "1;2;333"
.formula U|textSeparatorPercent = "4%55%666"
.println "print 1;2;333 separator ;"
.println ' U|textSeparatorSemicolon'
..
.println set separator -> %
.set separator %
```

```

..
.println "print 1;2;333 separator %"
.println ' U|textSeparatorSemicolon'
..Display:
..1;2;333
..
.println "print 4%55%666 separator %"
.println ' U|textSeparatorPercent'
..
.println set separator via command -> ;
.formula U|command = "set " + "separator " + ";"
.'U|command'
..
.println "print 1;2;333 separator ;"
.println ' U|textSeparatorSemicolon'

```

Beim Programmstart sind die in der Tabelle genannten Standardwerte aktiv. Eine Rückkehr zum Standardwert ist mit der parameterlosen Form **.setze** möglich.



Beachten Sie, dass mit **.setze** auch verschiedene andere Standardeinstellungen, z.B. Tausendertrennzeichen oder bestimmte Spracheinstellungen, auf ihren Standardwert zurückgesetzt werden.

Zum Schutz vor unbeabsichtigten Effekten kann ein Unterprogramm nicht die Einstellungen der *syntaktischen* Sonderzeichen (siehe [Tabelle](#)) seines Aufrufers ändern. Andernfalls müssten bei Änderung eines Unterprogramms alle Aufrufer dieses Unterprogramms gesucht und ggf. geändert werden, was nicht immer möglich ist. Umgekehrt ist es hingegen möglich, für ein Unterprogramm dessen Sonderzeichen zu modifizieren.

Es ist grundsätzlich auch möglich, einer syntaktischen oder formatierenden Funktion gar kein Zeichen zuzuweisen (sofern die jeweilige Option das zulässt). Die Syntax dafür lautet:

.setze 'Schlüsselwort'

Wenn Sie eine reine Textdatei ausgeben und keinerlei syntaktische Sonderzeichen interpretiert haben wollen, erreichen Sie das mit



```

.type text xtfilename
.formula U|xtfilename = "ow1/TEST"
.input -text 'U|xtfilename'

```

Es folgen Erläuterungen und Besonderheiten für die einzelnen Sonderzeichen.

.SETZE KOMMANDO / .SET COMMAND - Kommandozeichen setzen

.SETZE KOMMANDO	.SET COMMAND
deutsch	englisch
<code>.SETZE KOMMANDO 'Zeichen'</code>	<code>.SET COMMAND 'character'</code>
Setzt das Zeichen, mit dem in Spalte 1 Kommandos eingeleitet werden. Der Standardwert ist der Punkt.	

Kein Schlüsselwort - Angabe von ausgabeabhängigen Kommandos

Für das Zeichen zur Angabe von ausgabeabhängigen Kommandos, Standardwert Doppelpunkt, gibt es kein Schlüsselwort, denn es kann mit `.setze` nicht explizit modifiziert werden. Kommandos, die mit ihm gekennzeichnet sind, werden genau dann *nicht* ausgeführt, wenn die Ausgabe auf Datei geht; statt dessen werden sie unverändert auf die Datei geschrieben.

Dieses Feature sollte nur in Ausnahmefällen eingesetzt werden; der Doppelpunkt wird bereits im Zusammenhang mit HTML-Ausgabe als Kennzeichnung von HTML-Code benötigt.



Beim Setzen des Kommandozeichens wird dieses Zeichen auf <Strg> + A gesetzt.

.SETZE KOPIEREN / .SET COPY - Felder über Aufrufparameter initialisieren

.SETZE KOPIEREN	.SET COPY
deutsch	englisch
<code>.SETZE KOPIEREN Aufrufparameter ZielPuffer QuellePuffer</code>	<code>.SET COPY callparameter TargeBuffer SourceBuffer</code>
Initialisiert Felder über Aufrufparameter.	



Dieses Kommando funktioniert nur in FO-Programmen. Sie können dieses Kommando nicht in FO2-Programmen verwenden!

Sie können in einem FO-Programm mit einem [Aufrufparameter](#) mehrere Feldinhalte von einem Quell-Puffer in einen Ziel-Puffer kopieren. Der Aufrufparameter muss vorhanden sein. Der Aufrufparameter definiert, welche Felder aus dem Quell-Puffer in den Ziel-Puffer kopiert werden sollen.

Beispiel: Link zu Aufrufparameter in Hilfe einbauen

- Füllen der Felder des benutzerdefinierten Infosystems ISKUNDETEST (M-Puffer) aus einem Resultset (H-Puffer). Dazu wird ein Aufrufparameter (Identnummer 100253) verwendet.
- Das Infosystem wird durch den M-Puffer repräsentiert.
- Die Feldinhalte der Selektion werden durch den H-Puffer repräsentiert.

```
.type text idno ? F|defined(U|idno) = G|false
..
.copy U|idno = M|ynummerkunde
```

```
.println Nummer Kunde: 'U|idno'
.select customer 'U|idno'
.continue END ? G|mehr = G|false
..
.set copy 100253 M| H|
..
!END
.end
```

Im benutzerdefinierten Infosystem wird das Feld Kundennummer ausgefüllt. Der Button Start wird geklickt. Der zugehörige Kundendatensatz wird selektiert. Das Resultset liegt im H-Puffer. Die Felder der Infosystemmaske werden mithilfe des Aufrufparameters ausgefüllt.

.SETZE MARKE / .SET LABEL - Zeichen für Marke setzen

.SETZE MARKE	.SET LABEL
deutsch	englisch
<code>.SETZE MARKE = 'Zeichen'</code>	<code>.SET LABEL = 'character'</code>
Setzt das Zeichen für Marken. Standardwert ist das Ausrufezeichen.	

.SETZE FRAGE / .SET QUESTION - Zeichen für Bedingung setzen

.SETZE FRAGE	.SET QUESTION
deutsch	englisch
<code>.SETZE FRAGE 'Zeichen'</code>	<code>.SET QUESTION 'character'</code>
Setzt das einleitende Zeichen für Bedingungen. Standardwert ist das Fragezeichen.	

.SETZE ERSETZUNG / .SET REPLACEMENT - Variablenersetzungszeichen setzen

.SETZE ERSETZUNG	.SET REPLACEMENT
deutsch	englisch
<code>.SETZE ERSETZUNG 'Zeichen'</code>	<code>.SET REPLACEMENT 'character'</code>
Setzt das Variablenersetzungszeichen. Standardwert ist der Apostroph.	

`.setze ersetzung 'Zeichen'` setzt das Zeichen, mit dem man auf Variableninhalte zugreifen kann (Doppelpunkt ist dabei unzulässig).



Das Umsetzen des Ersetzungszeichens wirkt nicht bei den Kommandos `.formel`, `.kopieren` und `.zuweisen`. Dort ist prinzipiell der Apostroph das Zeichen für die Variablenersetzung (und in diesem Fall auch für die Konvertierung nach `text`).

.SETZE TRENNER / .SET SEPARATOR - Zeilenendezeichen für Variableninhalte setzen

.SETZE TRENNER	.SET SEPARATOR
deutsch	englisch
<code>.SETZE TRENNER 'Zeichen'</code>	<code>.SET SEPARATOR 'character'</code>
Setzt das Zeilenendezeichen für Variableninhalte. Standardwert ist das Semikolon.	

Das Trennzeichen für Zeilen (Zeilenendezeichen) für Variableninhalte hat bei deren links- oder rechtsbündiger Ausgabe Bedeutung:

```
.type text xttext
.formula U|xttext = "1;2;333"
'    U|xttext'
Display:
1
2
333
.set separator %
'    U|xttext'
Display:
1;2;333
.set separator ;
'    U|xttext'
Display:
1
2
333
```

.TRENNER / .SEPARATOR - Zeilenendezeichen für Variableninhalte setzen

.TRENNER	.SEPARATOR
deutsch	englisch
<code>.TRENNER 'Zeichen'</code>	<code>.SEPARATOR 'character'</code>
Setzt das Zeilenendezeichen für Variableninhalte. Standardwert ist das Semikolon.	

Die Kommandos `.setze trenner` und `.trenner` verhalten sich gleich.

.SETZE LEER / .SET EMPTY - Geschütztes Leerzeichen setzen

.SETZE LEER	.SET EMPTY
deutsch	englisch
.SETZE LEER 'Zeichen'	.SET EMPTY 'character'
Dient der Kennzeichnung von Leerzeichen als geschützte Leerzeichen. Standardwert ist der Unterstrich.	

Das geschützte Leerzeichen wird als normales Leerzeichen dargestellt, wirkt aber beim Zeilenumbruch nicht als Trenner. Der Text "25 km" z.B. wird nicht durch einen Zeilenumbruch zerrissen in "25" am Ende der alten Zeile und "km" am Beginn der neuen Zeile. Das geschützte Leerzeichen ('G|**leerchar**') wird im Blocksatz und Flattersatz berücksichtigt, also bei der Ausgabe durch ein Leerzeichen ersetzt. Im Tabellensatz hingegen wird es unverändert ausgegeben (z.B. "25_km").

.SETZE FOEXTERN / .SET FOEXTERN - Verhalten wie in Maske aktivieren

.SETZE FOEXTERN	.SET FOEXTERN
deutsch	englisch
.SETZE FOEXTERN + -	.SET FOEXTERN + -
Standardwert ist +.	
.setze foextern - hat zwei Wirkungen: - Felder werden so wie in der Maske ausgegeben - Schreib- und Lesbarkeit für FO ist wie für den Benutzer in der Maske	
.setze foextern + setzt wieder den Standardwert.	

.SETZE ABBRUCH / .SET CANCEL - Abbrechen des FOPs verhindern

.SETZE ABBRUCH	.SET CANCEL
deutsch	englisch
.SETZE ABBRUCH -	.SET CANCEL -
.SETZE ABBRUCH --	.SET CANCEL --
.SETZE ABBRUCH + [n]	.SET CANCEL + [n]
.setze abbruch - verhindert, dass das Programm abgebrochen werden kann. Wenn das Programm auf einen durch einen anderen Benutzer belegten Datensatz stößt, ist ein Abbruch trotzdem möglich. Dabei handelt es sich um einen normalen Abbruch des Sperrdialogs . Das FOP läuft weiter.	
.setze abbruch -- bewirkt, dass auch in Sperrsituationen nicht ohne weiteres abgebrochen werden kann. Ein Abbruch wird wie ein Kritischer Abbruch des Sperrdialogs behandelt.	
.setze abbruch + ermöglicht, dass das Programm abgebrochen werden kann. Mit der nichtnegativen Zahl n kann man den Exit-Status für EFOPs festlegen, der bei Abbruch durch den Benutzer zurückgegeben wird.	

Normalerweise darf ein FO-Programm durch den Benutzer abgebrochen werden. Das können Sie mit dem Kommando **.setze abbruch** verhindern. Das Kommando ist in Programmen sinnvoll, in denen mehrere zusammengehörige Tätigkeiten durchgeführt werden, so dass sie beim Abbruch unaufgeräumt bleiben oder in einem inkonsistenten Zustand hinterlassen würden.

.SETZE BEREICHSENDE / .SET RANGETO - Bearbeitung des Datensatzbereichs beenden

.SETZE BEREICHSENDE	.SET RANGETO
deutsch	englisch
.SETZE BEREICHSENDE +	.SET RANGETO +
erlaubt es, nach dem Speichern die Bearbeitung des Datensatzbereichs zu beenden	

Das FO-Kommando muss in einem EFOP mit dem Ereignis Maskenaustritt (**maskaus**), Workflow (**workflow**) oder Maskenende (**maskende**) ausgeführt werden.

Eine Maske wird geöffnet und dort mit "neu" ein Datensatz neu erfasst. Dann bleibt normalerweise nach dem Speichern die Maske im Modus "neu" und es kann ein weiterer Datensatz angelegt werden. Wenn jedoch im **maskaus**-FOP das Kommando

```
.set rangeto
```

ausgeführt wird, wird nach dem Speichern des ersten Datensatzes die Neuanlage beendet und die Maske geht in den Modus "leer" über.

.SETZE AUFZAEHLTYP / .SET ENUMERATIONTYPE - Aufzähltyp als aktuellen setzen

.SETZE AUFZAEHLTYP	.SET ENUMERATIONTYPE
deutsch	englisch
.SETZE AUFZAEHLTYP 'n'	.SET ENUMERATIONTYPE 'n'
versucht, den Aufzähltyp A'n' als aktuellen zu setzen, um anschließend seine Werte ermitteln zu können	

Die definierten Aufzähltypen können mit Variablen in **V-13-00** abgefragt werden. Dafür muss zunächst ein Aufzähltyp als aktueller gesetzt werden. Dies geschieht mit **.setze aufzaehltyp 'n'**.

Die Aufzähltypen **A'n'** sind nicht notwendigerweise lückenlos vorhanden. Wenn z.B. A73 und A75 existieren, folgt daraus nicht, dass auch A74 existiert. Zur Definition eines Maximalwert für **n** in einer Schleife ist die Variable **G|enummax** geeignet.

Die Statusvariable **G|mehr** gibt an, ob der Aufzähltyp existiert und der Befehl somit erfolgreich war. Anschließend können die Werte des Aufzähltyps über folgende Variablen abgefragt werden:

<code>enumdescr</code>	Titel
<code>enumnumpos</code>	Anzahl der Positionen
<code>enumpostext.i</code>	<code>Array(1..enumnumpos)</code> : Name der i-ten Position
<code>enumposcode.i</code>	<code>Array(1..enumnumpos)</code> : eindeutiger Code (int) der i-ten Position; dieser ist 0, wenn es sich um den Leereintrag handelt

Beispiel: Ausgaben aller Positionen eines Aufzähltyps

```
.type int xiindex
..
.set enumerationtype 7
A7 exists: 'G|success'
.end ? G|success = G|false
Titel A7: 'G|enumTypeTitle'
A7: 'G|noEnumTypePos' Positionen:
.formula xiindex = 1
!WHILE
Position['U|xiindex']='G|enumPosName.xiindex'      Int-Code['U|xiindex'
]='G|enumPosCode.xiindex'
.formula U|xiindex = U|xiindex + 1
.continue WHILE ? U|xiindex <= G|noEnumTypePos
```

.SETZE ABLAGEKENNZEICHEN / .SET FILINGSYSTEMCODE - Ablagekennzeichen setzen

.SETZE ABLAGEKENNZEICHEN	.SET FILINGSYSTEMCODE
deutsch	englisch
<code>.SETZE ABLAGEKENNZEICHEN +</code> <code>.SETZE ABLAGEKENNZEICHEN -</code>	<code>.SET FILINGSYSTEMCODE +</code> <code>.SET FILINGSYSTEMCODE -</code>
<code>.setze ablagekennzeichen +</code> gibt bei Ausgabe von Verweisen auf abgelegte Objekte ein Plus vor dem Suchwort oder der Identnummer aus. <code>.setze ablagekennzeichen -</code> unterdrückt bei Ausgabe von Verweisen auf abgelegte Objekte das Plus vor Suchwort oder Identnummer.	

Dieses Kommando übersteuert in beiden Fällen den voreingestellten Wert für das jeweilige FOP, der im Konfigurationsdatensatz im Feld `Ablagekennzeichen extern` angegeben ist.

.SETZE DRUCKEN / .SET PRINT - Zweiten Druckpuffer aktivieren

.SETZE DRUCKEN	.SET PRINT
deutsch	englisch
.SETZE DRUCKEN + -	.SET PRINT + -
Für dieses Kommando gibt es einen Stapel von zwei P-Puffern.	
Nach .setze drucken + ist der zweite P-Puffer aktiviert; er enthält eine exakte Kopie des ersten.	
Nach dem folgenden .setze drucken - wird der ursprüngliche P-Puffer wieder sichtbar; er bleibt die ganze Zeit unverändert.	

.SETZE WAEHRUNG / .SET CURRENCY - Währung vorbelegen

.SETZE WAEHRUNG	.SET CURRENCY
deutsch	englisch
.SETZE WAEHRUNG	.SET CURRENCY
.SETZE WAEHRUNG 'Währung'	.SET CURRENCY 'currency'
.setze waehrung 'Währung' belegt die globale FO-Variable G waehrung mit der angegebenen Währung.	

Mit Hilfe des FO-Kommandos **.setze waehrung 'Währung'** wird die globale FO-Variable **G|waehrung** mit der angegebenen Währung vorbelegt.

Die FO-Variable **G|waehrung** bestimmt die für die Auswertung relevante Buchungswährung. Diese Variable ist standardmäßig (Defaultwert) mit der **Inlandswährung** vorbelegt. Nach Bedarf kann in den Auswertungen auf eine andere Buchungswährung geändert werden. Verwendung findet die abweichende Vorbelegung der Buchungswährung im Rechnungswesenbereich in Zusammenhang mit der Euro-Umstellung bzw. bei der Verwendung der zweiten Kontowährung für die Ermittlung der Daten aus den Verkehrszahlen und Umsatzzähler der Personen- und Sachkonten, Kostenstellen, Kostenträger.

Auch Planverkehrszahlen können in diversen Währungen erfasst werden. In der Ermittlung dieser Werte muss die betroffene Buchungswährung **G|waehrung** mit **.setze waehrung 'Währung'** bestimmt werden.

Beispiel:

Die **Inlandswährung** ist die Währung EUR. Für das Bankkonto 18100 wurde die zweite Kontowährung CHF definiert. Dadurch werden für das Konto 18100 die Verkehrszahlen in EUR und zusätzlich in CHF geführt.

Durch die Vorbelegung der FO-Variable **G|waehrung** wird auf der FO die Entscheidung getroffen, welche Verkehrszahlen in welcher Buchungswährung für die Auswertung relevant sind:

```
.select year 12 1 12
.set currency CHF
.select account "18100"
```

```
'H|openBal' 'H|debTotal' 'H|credTotal' 'H|closingBal'
```

In der Euro-Eröffnungsphase (das letzte Geschäftsjahr der alten Inlandswährung) entstehen Finanzbuchungen in der alten Inlandswährung DEM aber auch die Euro-Eröffnungsbuchungen in der neuen Inlandswährung EUR. Um für die Auswertung relevante Buchungslisten zu bekommen, muss die Buchungswährung **G|waehrung** vorbelegt werden:

```
.select year 12 1 12
.set currency DEM
.select customer "1"
.select statement
```

Siehe hierzu das FO-Kommando **.setze waehrung** oder die FO-Variablen **G|waehrung** bzw. **G|auswaehr**

.SETZE AUSWAEHR / .SET OUTCURR - Ausgabewährung vorbelegen

.SETZE AUSWAEHR	.SET OUTCURR
deutsch	englisch
.SETZE AUSWAEHR .SETZE AUSWAEHR 'Währung'	.SET OUTCURR .SET OUTCURR 'currency'
.setze auswaehr 'Währung' belegt die globale FO-Variable G auswaehr mit der angegebenen Währung.	

Die Variable **G|auswaehr** ist standardmäßig (Defaultwert) mit der Inlandswährung (hinterlegt in der Maske Konfiguration zeigen) vorbelegt. Nach Bedarf kann in den Auswertungen auf eine andere Ausgabewährung geändert werden.

Die Variable **G|auswaehr** entscheidet, in welcher Währung die Verkehrszahlen (betroffen sind die Variablen **asaldo**, **esaldo**, **ssum**, **hsum**, **asoll**, **ahaben**, **kbesaldo**, ...) ausgegeben werden sollen. Die Werte werden zuerst in der Buchungswährung ermittelt und dann für die Ausgabe mit dem aktuellen Währungskurs in die Ausgabewährung **G|auswaehr** umgerechnet. Betroffen sind auch die Planverkehrszahlen.

Siehe hierzu das FO-Kommando **.setze waehrung** oder die FO-Variablen **G|waehrung** bzw. **G|auswaehr**

.SETZE BUCHUNGSKREIS / .SET SETOFBOOKS - Buchungskreis setzen

.SETZE BUCHUNGSKREIS	.SET SETOFBOOKS
deutsch	englisch
.SETZE BUCHUNGSKREIS .SETZE BUCHUNGSKREIS 'Suchwort' 'Identnummer' 'Index'	.SET SETOFBOOKS .SET SETOFBOOKS 'search word' 'identity number' 'index'
ermöglicht die Zugriffe auf den Buchungsumfang einer Rechnungslegung. Index kann den Wert 0, 1, 2, 3, 4 oder 5 annehmen.	

Für die Zugriffe auf den Buchungsumfang einer Rechnungslegung muss die Rechnungslegung per FO festgelegt werden können. Hier bietet sich die Lösung mit **.setze buchungskreis** an.

Grundsätzlich kann die Buchung per Identnummer oder Suchwort aufgegriffen werden. Es gibt jedoch keine Beziehung zwischen der Identnummer und der Buchungsnummer des Finanzbuchungsjournals. Das FO-Kommando **.hole vkz** und **G|**-Variablen werten den gesetzten Buchungskreis aus.

Für die spezifische Selektion einer Buchung anhand der Buchungsnummer eines Buchungskreises muss man über eine rohlingslose Selektion vorgehen.

- Das Kommando hat nur Auswirkungen, wenn die Funktion **Parallele Rechnungslegung** aktiviert ist. Dann liefert das Kommando immer das Nullelement des Datentyps von **bukreis**.
- Beim Betreten der FO ist kein Buchungskreis aktiv, damit sich das normale Selektionsverhalten über die Schlüssel ergibt, die dieses Attribut nicht enthalten.
- Bei aktiver "Paralleler Rechnungslegung" wird über Index 0 bzw. ohne Parameter der Buchungskreis abgeschaltet.
- Der Befehl wirkt sich auf folgende FO-Kommandos aus:
 - **Selektion von Buchungen**
 - **.hole auszug**
 - **.hole auszug vorgang**
 - **.hole buchung budatum**
 - **.hole kosten feld**
 - **Selektion von Verkehrszahlen**
 - **.hole vkz**
 - **Berechnung von Beträgen**
 - Berechnung der Textvariablen für Beträge, z.B. **esaldo**, **asaldo**, etc.
 - **.budget**
- Ist ein Buchungskreis nicht aktiv, ist er nicht setzbar und **.setze buchungskreis 'Index'** liefert einen Fehler, wenn der Index keinem Buchungskreis zugeordnet ist.

Textvariable **G|bukreis**

Der aktuelle gewählte Buchungskreis wird in der Textvariablen **G|bukreis** hinterlegt. **G|bukreis**

ist die Referenz auf den Buchungskreis. Der Zugriff auf die Buchungskreisnummer erfolgt per **G|bukreis^bknr**.

```
Actual accounting area: 'G|setOfBooks' - 'G|setOfBooks^setOfBooksNo'
.set setofbooks "IFRS"
New accounting area: 'G|setOfBooks' - 'G|setOfBooks^setOfBooksNo'
.select year 12 1 1
.select account "10000"
.select statement
.set setofbooks "HGB"
```

Rohlingslose Selektionen

Um Buchungen buchungskreispezifisch zu holen, werden der entsprechende Schlüssel und zusätzlich die booleschen Variablen für den Buchungskreis angegeben.

Um Zugangsbuchungen des Buchungskreises "2" zu selektieren, ist folgende Selektion anzugeben:

```
.select entry "$,@sort=Anlageauszug;bukreis2=ja;bukenn=ZU"
```

Siehe hierzu auch [Kommandos für die Finanzbuchhaltung](#)

.SORTIERE / .SORT - Textdateien sortieren

.SORTIERE	.SORT
deutsch	englisch
.SORTIERE 'Dateiname' ['Spalte'] ['Trennzeichen'] [ABWAERTS] [NUMERISCH]	.SORT 'filename' ['column'] ['separator'] [DOWN] [NUMERICAL]
Die datei wird zeilenweise nach einer Spalte sortiert. Jede Zeile kann mehrere Texte spaltenweise nebeneinander enthalten. Fehlt die Angabe der spalte, wird nach der ersten Spalte sortiert. Als Trennzeichen können keine Buchstaben oder Ziffern verwendet werden. Ist kein Trennzeichen angegeben, wird das Leerzeichen als Spaltentrenner gewertet. Fehlt die Angabe abwaerts, wird aufwärts sortiert. Fehlt die Angabe numerisch, wird alphabetisch sortiert. (Z.B. steht 13 alphabetisch vor 5, numerisch nach 5.)	

```
.sort ADRESSFILE 3 , numerical
```

Die Datei **ADRESSFILE** wird zeilenweise nach Spalte drei aufwärts numerisch sortiert. Trennzeichen ist das Komma. Die dritte Spalte befindet sich also nach dem zweiten Komma.



.sortiere ruft das im Abas Lieferumfang befindliche Programm **sort** auf, das Dateien im Abas Zeichensatz erwartet und diese nach dem **Unicode Collation Algorithm** sortiert.

Sortieren von Terminen

Datumsangaben sind sortierfähig, wenn sie in der Kurzform des sortierfähigen Datums nach ISO 8601 vorliegen, z.B. **20100408** für den 8.4.2010.

Da die Sortierung streng lexikalisch arbeitet, ist die gebräuchliche Schreibung des Datums nicht sortierfähig. Zum Beispiel kommt "08.04.10" lexikalisch *nach* "01.05.10", da 08 größer ist als 01.

Logging und Debugging

Zur Unterstützung von Logging-Mechanismen in FO-Programmen gibt es die Variable **P|loglevel** (**Vartab V-88-10**). Dadurch, dass sich diese Variable im P-Puffer befindet, kann sie vom Benutzer jederzeit im Dialog geändert werden, um für eine bestimmte Dialogaktion die Protokollierungsstufe zu setzen.

Die Protokollierungsstufe ist eine Zahl von 0 bis 9.

In gleicher Weise ist es möglich, das Debugging von FO-Programmen über die Variable **P | fopdebug** im P-Puffer einzuschalten. Dies ist insbesondere beim Testen des Druckprozesses von Nutzen.

Beide Variablen können sowohl über die FO also auch über den Druckdialog gesetzt werden. Im Druckdialog sind die Felder über das Register Erweitert und den Button Details erreichbar.

Nach Setzen von **P | fopdebug** auf **G | true** durch Zuweisung in einem FOP wird der Debug-Modus nicht sofort wirksam, sondern erst beim nächsten Start eines FO-Programms.

.SETZE DEBUG / .SET DEBUG - Debuggen von FOPs

.SETZE DEBUG	.SET DEBUG
deutsch	englisch
.SETZE DEBUG +	.SET DEBUG +
.SETZE DEBUG -	.SET DEBUG -
.SETZE DEBUG ++	.SET DEBUG ++

.SETZE DEBUG	.SET DEBUG
<code>.setze debug +</code> schaltet den Debug-Modus ein. <code>.setze debug -</code> schaltet den Debug-Modus aus. <code>.setze debug ++</code> setzt einen permanenten Debug-Modus, in dem jedes (E)FOP mit Debugger gestartet wird.	

Der Prompt sieht im Debug-Modus wie folgt aus: `debug >`.

Im Debug-Modus wird jedes FO-Programm zeilenweise abgearbeitet: Nach jeder Ausführung einer FOP-Zeile bleibt das FOP stehen und zeigt die Zeile an, die als nächste ausgeführt wird. Es können nun z.B. beliebige Variable ausgegeben oder andere Aktionen durchgeführt werden. Kommentarzeilen werden überlesen und nur auf den Bildschirm ausgegeben.

Sie können jedes FO-Programm im Debug-Modus starten. Dazu dient das FOP `debug`. Es fragt nach dem Namen des FOPs, das im Debug-Modus ausgeführt werden soll. Anschließend wird der Befehl `.setze debug +` ausgeführt.

Der Debug-Modus kann auch in einer Maske im **Kontextmenü** ein- und ausgeschaltet werden über Menüpunkt [FOP], [FOPs debuggen].

Der permanente Debug-Modus (`.setze debug ++`) gilt auch über das FOP-Ende hinaus - solange, bis er mit `.setze debug -` zurückgenommen wird.

Verwenden des FOPs DEBUG

Das zu debuggende Programm kann an das FOP `debug` alternativ als Parameter übergeben werden. In diesem Fall wird es nicht mehr von `debug` eingelesen. Dadurch ist es möglich, den vollständigen Debug-Befehl in die Kommandoübersicht aufzunehmen, statt das zu debuggende Programm jedes Mal einzugeben.

Testausgaben, die nur im Fall des Debuggens mit dem FOP `debug` ausgegeben werden sollen, kann man mit der Bedingung `stackframe1 = "DEBUG"` versehen. Solche Testausgaben haben den Vorteil, dass sie nach der endgültigen Fertigstellung des FOPs nicht entfernt werden müssen. Sie stehen dann bei späteren Wartungsarbeiten immer noch zur Verfügung.

Beispiel: Testausgaben

```
.select
.println actual records: 'H|idno' 'H|swd' ? stackframe1 = "DEBUG"
```

Der nächste Befehl im FOP `debug` ist das Einziehen des FOPs, das untersucht werden soll. Dieser Befehl wird angezeigt, aber noch nicht ausgeführt. Dies geschieht erst nach der Eingabe von n <Return>. Das Programm bleibt bei der ersten Kommandozeile stehen.

LG-Debugger Version 1.0 Minihilfe

Es sind folgende Kommandos erlaubt:

Kommando	Bedeutung
<code>b [[PRG:]znr]</code>	Breakpoint setzen
<code>bp</code>	Breakpoints zeigen
<code>c</code>	FOP läuft weiter bis zum nächsten Breakpoint
<code>d zahl</code>	Breakpoint löschen
<code>h</code> oder <code>?</code>	Diese Hilfe aufrufen
<code>l</code>	Nächste Zeile, die ausgeführt wird, ausgeben
<code>n</code>	Nächste Zeile ausführen, bei <code>.eingabe</code> nicht in Unterprogramm springen
<code>s</code>	Nächste Zeile ausführen, bei <code>.eingabe</code> in Unterprogramm springen
<code>p var</code>	Inhalt der Variablen ' <code>var</code> ' zeigen
<code>z zahl</code>	Debuglevel setzen

Kommando	Erläuterung
<code>n</code>	Sie können das FOP zeilenweise durchgehen.
<code>b</code>	In der Programmzeile, in der Sie gerade stehen, wird ein Breakpoint gesetzt.
<code>b zeilenr</code>	In der Zeile mit der angegebenen Nummer des aktuellen Programms wird ein Breakpoint gesetzt.
<code>b PROGRAMMNAME: zeilenr</code>	Im Programm <code>PROGRAMMNAME</code> (Pfad angeben) wird in der Zeile mit der angegebenen Nummer ein Breakpoint gesetzt.  Breakpoints auf Kommentarzeilen haben keine Wirkung, da sie überlesen werden.
<code>c</code>	Es wird ein Continue ausgeführt, d.h. das FOP läuft weiter, bis es auf den nächsten Breakpoint trifft. Wenn kein Breakpoint auftritt, läuft es bis zum Ende.
<code>d</code>	Breakpoints lassen sich wieder löschen. Die Nummer des Breakpoints erhalten Sie, wenn Sie <code>bp</code> ausführen.
<code>p</code>	Sie können sich die Werte jeder Variablen ansehen, auch diejenigen, die sich in Puffern (<code>M</code> <code>D</code> <code>H</code> <code>E</code>) befinden.

Kommando	Erläuterung
z	Sie können den Debug-Level ändern.
q	Der Debug-Modus wird während einer Debug-Session abgeschaltet. Das FOP läuft danach normal weiter.
bt	Liefert die Reihenfolge der eingezogenen Programme als .eingabe -Trace oder Backtrace

Die Taste <Abbruch> bricht das Programm, das im Debug-Modus läuft, regulär ab.

Der Debug-Level lässt sich ändern. Es werden folgende Level unterschieden:

Debug-Level	Wirkung
1	Nur Ausgaben. Sie erhalten momentan eine Ausgabe, wenn eine Variable automatisch erzeugt wird, die vorher nicht deklariert wurde.
2	FO-Programm zeilenweise ausführen; Debug-Level 2 ist Standard
3	Nur bei Punktbefehlen anhalten

.SETZE TRACE / .SET TRACE - Testprotokoll für Selektionen

.SETZE TRACE	.SET TRACE
deutsch	englisch
.SETZE TRACE	.SET TRACE
.SETZE TRACE +	.SET TRACE +
.SETZE TRACE -	.SET TRACE ++
.SETZE TRACE ++	.SET TRACE -
.setze trace + schaltet die Protokollierung für Selektionen ein.	
.setze trace ++ schaltet die Protokollierung für Selektionen ein und gibt zusätzlich Warnungen aus.	
.setze trace - schaltet die Protokollierung für Selektionen aus.	

Die Einstellung gilt für jede Selektion im FOP und im Dialog. Das Testprotokoll für Selektionen kann auch in einer Maske im **Kontextmenü** ein- und ausgeschaltet werden über Menüpunkt [FOP], [Testprotokoll für Selektionen].

Wird mit **.setze trace +** selektiert, erfolgt bei jeder Selektion ein Eintrag in der Protokolldatei.

Bei **.setze trace ++** werden zusätzlich Warnungen ausgegeben, z.B.

NOTE:\$:Gruppe nicht angegeben###\$,@sort=Nummer;@gr=1.2.6.7#nummer=

Dieser Hinweis erscheint ebenfalls in der Protokolldatei:

```
NOTE:$:Nicht optimale Suche - Feld leer: num###$,@sort=Nummer;@gr=1.2.6.7#nummer
```

Wird ein Schlüssel ohne Einschränkung benutzt, ist die Suche im Allgemeinen langsam.

Wird ein bedingter Schlüssel benutzt und ist das Bedingungsfeld in der Selektionsleiste widersprüchlich gesetzt, erhalten Sie die Meldung "Widersprüchliche Suche". Widersprüchlich ist eine Suche, wenn

Schlüsselart == "Feld leer" und Selektionskriterium != leer oder
 Schlüsselart == "Feld leer" und Selektionskriterium != leer.

Optimaler Schlüssel

Wurde mit **@sort** bzw. manuell in der Objektauswahl ein Schlüssel gewählt und findet das System einen besseren Schlüssel, also einen, der bei gleicher Treffermenge weniger Datensätze lesen muss, meldet das System:

Kein optimaler Schlüssel: X, Optimaler Schlüssel: Y

X	Der vom Benutzer gewählte Schlüssel
Y	Der vom System vorgeschlagene Schlüssel. Dies wäre auch der Schlüssel gewesen, wenn vom Benutzer kein Schlüssel vorgegeben worden wäre.

```
.set strace +
.select customer "$,such=Z!;@sort=Nummer"
```

Ausgabe: Kein optimaler Schlüssel: Nummer, Optimaler Schlüssel: Suchwort

Bei **.setze trace ++** werden diese Ausgaben in die Datei **sellog.*** geschrieben.

Nur bei **.setze strace +** erscheint bei ineffizienten Dialogselektionen eine Hinweismeldung auf dem Bildschirm.

Aufbau des Protokolls sellog

Das Protokoll liegt im Mandantenverzeichnis und heißt **sellog.<AufruferNr>**. Näheres zu **AufruferNr** siehe **FO-Variable 'G|aufrufern'**. Pro Selektion wird in **sellog** eine Zeile mit folgendem Format erzeugt. Das Trennzeichen zwischen den Spalten ist stets die Raute (#).

Spalte	Inhalt
1	Datum in 8-stelligem, sortierfähigem Format
2	Prozessnummer des Programmes, das den Eintrag erzeugt hat
3	Abas Kommando bzw. EDP-Kommando

Spalte	Inhalt
4	Falls die Selektion aus einem FOP gestartet wurde, erscheint hier der FO-Trace wie aus SYS.FEHL bekannt.
5	resultierende Selektionskriterien, also der Inhalt der FO-Variablen 'G selrohl' und 'G selkrit'
6	Nummer der Datei, in der selektiert wurde
Die folgenden sechs Spalten enthalten entscheidende Zähler:	
7	gelesene Sätze
8	passende Sätze (Spalte 7 minus Spalte 9)
9	unpassende Sätze (Spalte 7 minus Spalte 8)
10	unpassend aufgrund falscher Datensatzgruppe (aus Spalte 9)
11	unpassend aufgrund der angegebenen Selektionskriterien (aus Spalte 9)
12	unpassend aufgrund eines Trefferprüfungs-EFOPs (aus Spalte 9)
In der folgenden Spalte steht die zu Beginn ermittelte geschätzte Zahl von Treffern.	
13	Schätzwert (FO-Variable 'G seltreffercirca')
Die nächste Spalte enthält initiale Selektionskriterien.	
14	Initiale Selektionskriterien, mit denen die Selektion gestartet wurde
Die folgenden zwei Spalten enthalten entscheidende Zähler:	
15	unpassende aufgrund falscher Ablageart (aus Spalte 9)
16	unpassend aufgrund einer internen Kernbedingung (aus Spalte 9)
Die letzte Spalte enthält die Zeit, die die Selektion gedauert hat:	
17	Zeitdauer

Besonderheiten

Zu einer Selektion befinden sich unter Umständen zwei Zeilen im Protokoll **sellog**.

Zeile	Inhalt
1	Protokollierung der Zugriffe im Firmenstamm zum Aufsuchen des Selektionsrohlings (dabei ist in der Spalte 6 der Wert = 12).
2	Protokollierung der eigentlichen Selektion

Ein FOP **XY** starte eine Objektauswahl mit

`.hole KUNDE "%,nummer=1!;such="` In der Objektauswahl tippt der Benutzer die zusätzliche Bedingung `"Suchwort"="T!T"` ein.

Im `sellog`-Protokoll entsteht dadurch z.B. folgende Zeile:

`20101010#4711#Text zeigen XY`

`1→XY:2#%,nummer=1!;such=T!T;@gruppe=1.2.6.7;@ablageart=(Active);@sort=(Number)0#93#3#90#0#90#0#nummer=1!;such=#0#0#0D00h00m02s` Man kann nun in Spalte 13 sehen, dass die initialen Selektionskriterien `"nummer=1!;such="` waren und (nach der Änderung durch den Benutzer) daraus folgende Selektionskriterien resultierten:

`"%,nummer=1!;such=T!T;@sort=Nummer;@gr=1.2.6.7"`

Das FOP samt Zeilennummer ist in Spalte 4 angegeben: `"XY:2"`

In der letzten Spalte kann man sehen, dass die Selektion 2 Sekunden gedauert hat.

.SETZE VTRACE / .SET VTRACE - Protokollierung von Variablenänderungen

.SETZE VTRACE	.SET VTRACE
deutsch	englisch
<code>.SETZE VTRACE ++</code>	<code>.SET VTRACE ++</code>
<code>.SETZE VTRACE + -</code>	<code>.SET VTRACE + -</code>
<code>.SETZE VTRACE + - 'Variablenname 1'</code>	<code>.SET VTRACE + - 'variable name 1'</code>
<code>['Variablenname 2' ...]</code>	<code>['variable name 2' ...]</code>
<code>.SETZE VTRACE + - 'Puffername' </code>	<code>.SET VTRACE + - 'buffer name' </code>
<code>.setze vtrace ++</code> schaltet die Variablenüberwachung für <i>alle</i> Variablen ein. <code>.setze vtrace + -</code> schaltet die Variablenüberwachung ein bzw. aus. Vor oder nach diesem Aufruf können Sie Variablen oder Puffer auswählen, die protokolliert werden sollen. Auch nach einem Ausschalten bleiben die zuvor ausgewählten Puffer und Variablen markiert und werden nach erneutem Einschalten wieder protokolliert. Die Protokollierung ist beim Start der FO ausgeschaltet. <code>.setze vtrace + - 'Variablenname 1' ['Variablenname 2' ...]</code> schaltet die Variablenüberwachung für bestimmte Variablen ein bzw. aus. <code>.setze vtrace + - 'Puffername' </code> schaltet die Überwachung für alle Variablen im angegebenen Puffer ein bzw. aus.	

Das Kommando aktiviert und deaktiviert Trace-Ausgaben für Variablen. Protokolliert werden alle Schreibzugriffe auf die entsprechenden Variablen, also: `.formel`, `.kopieren`, `.zuweisen`, aber auch `.lesen`, `.datei -lesen`, sogar `.laenge 33` (ändert die FOP-Variablen `laenge`) oder `.hole kunde` (ändert die FOP-Variablen `kspr` und `ksi`).

Bei `.setze vtrace +|- 'Variablenname 1' ['Variablenname 2' ...]` beachten Sie die

Notation. Die Protokollierung erfolgt nur nach einem `.setze vtrace +`. Wenn Sie Variablen in Datenbankpuffern protokollieren, gilt die Protokollierung bei Tabellenfeldern für alle Zeilen. Wird der Datenbankpuffer neu gefüllt, geht die Protokollierungseinstellung verloren. Beim Maskenpuffer bleibt sie aber solange erhalten wie der Maskenpuffer existiert. Nur ein erneutes `.setze vtrace +` ist notwendig, um die Protokollierung wieder einzuschalten.

Für `.setze vtrace +|- 'Puffername' |`, z.B. `.setze vtrace + H|`, können auch Datenbankpuffer, insbesondere der Puffer `M|` angegeben werden, um ein Protokoll für alle Zugriffe auf den Maskenpuffer aus einem EFOP zu erhalten. Wird der Datenbankpuffer neu gefüllt, geht die Protokollierungseinstellung verloren. Beim Maskenpuffer bleibt sie aber solange erhalten wie der Maskenpuffer existiert.

Beispiel: FOP owprenglish/SETZE.VTRACE.1

```
.type real xnumber1
.type int xnumber2
.select customer "1"
.set vtrace +
.set vtrace + U|xnumber1 U|xnumber2 H|addr
.assign U|xnumber2 = 1
.set vtrace -
.formula U|xnumber1 = 47.11
.set vtrace +
.formula U|xnumber1 = U|xnumber1 + 1
.assign H|addr = "51"
```

Ausgabe:

```
LET owprenglish/SETZE.VTRACE.1:8: U|xnumber2 = "1"
LET owprenglish/SETZE.VTRACE.1:12: U|xnumber1 = "48.11"
LET owprenglish/SETZE.VTRACE.1:13: H|addr = "51"
```

Format der Protokollausgabe:

`LET FOPNAME:Zeile VNAME = Wert`

LET	Schlüsselwort
FOPNAME	Name des laufenden FO-Programms bzw. "Direktmodus"
zeile	Zeilennummer
VNAME	Name der geänderten Variablen
Wert	neuer Wert (Inhalt) der Variablen

.SETZE TRACE / .SET TRACE - Protokollierung von FO-Programmen

.SETZE TRACE	.SET TRACE
deutsch	englisch
.SETZE TRACE .SETZE TRACE + - .SETZE TRACE +F	.SET TRACE .SET TRACE + - .SET TRACE +F
.setze trace Protokollierung für die Entwicklung und Fehlersuche in FO-Programmen .setze trace + schaltet die Zeilenprotokollierung ab der nächsten Anweisung ein bzw. aus. Es kann nur eine Anweisungsfolge überwacht werden. .setze trace +F gibt den Trace der ausgeführten Befehle auf Datei aus. Der Dateiname wird gemeldet.	

Format der Protokollausgabe:

TRACE+ FOPNAME:Zeile Anweisung

trace	Schlüsselwort
+	Zeichen dafür, dass die Bedingung für das Kommando <i>true</i> war oder keine Bedingung gestellt wurde
-	Zeichen dafür, dass die Bedingung für das Kommando <i>false</i> war
FOPNAME	Name des laufenden FO-Programms bzw. "Direktmodus"
zeile	Zeilennummer
Anweisung	FO-Kommando

Beispiel: FOP owprenglish/SETZE.TRACE.1

```
.set trace +
.select customer
.set trace -
.select customer
```

Ausgabe:

```
TRACE+ owprenglish/SETZE.TRACE.1:4: .select customer
TRACE+ owprenglish/SETZE.TRACE.1:5: .set trace -
```

Mit der Variable `P|tracelevel` können Sie das Verhalten des `.setze trace`-Kommandos ersetzen und noch feiner steuern.

Debug-Modus im Infosystem FO-Programm ausführen

Im [Infosystem FO-Programm ausführen](#) (Suchwort TEXTZEIGEN, Arbeitsbereich is, Identnummer 10169) können Sie das Programm wahlweise im Debug-Modus und mit geänderter Protokollierungsstufe starten. Sie haben die Möglichkeit, das Programm mit dem FO-Kommando `.eingabe` oder mit dem Kommando `.eingabe FO-Programmname .eingabe` zu starten. Standardmäßig wird mit dem FO-Kommando `.eingabe` gestartet.

Debuggen von EFOPs auf der GUI

Auf der grafischen Oberfläche (GUI) kann das Debuggen aller EFOPs, die für die bearbeitete Maske konfiguriert sind, im Kontextmenü über das Kommando `[FOP] [FOPs debuggen]` eingeschaltet werden. Die wiederholte Auswahl dieses Menüpunktes schaltet den Debug-Modus wieder aus. Der Modus gilt nur für das Fenster, in dem er geändert wurde.

Zugriff auf Freitexte

Auf Freitexte wird über Variablen, z.B. `ftext` zugegriffen. Diese enthalten aber nicht den textuellen Inhalt, sondern nur einen internen Namen des Textes.

```
.hole kunde "1"
'H|ftext'
```

Der Programmausschnitt gibt den Namen aus, dieser ist aber für den Benutzer bedeutungslos.



Eine Verwendung des Namens des Freitextes ist nur innerhalb der beschriebenen Freitextkommandos erlaubt. Es ist z.B. nicht zulässig, sich den Namen in einer Textvariablen zu merken und später wieder zu verwenden. Der Name kann sich innerhalb einer Sitzung auch ändern.

Bei den im Folgenden beschriebenen Kommandos steht `'H|ftext'` beispielhaft für eine Variable des Typs Freitext und das Kommando `.hole kunde` für ein Kommando, das einen Datensatz lädt.

Lesen eines Freitextes und Ausgabe in eine Liste

Kommando	<code>.hole kunde</code> <code>.eingabe 'H ftext'</code>
Wirkung	Der am gelesenen Datensatz hängende Text wird als FO-Programm interpretiert; daraus wird die Ausgabe des FO-Programms erzeugt.

```
.hole Kunde "1"
.eingabe 'H|ftext'
```

Lesen von einzelnen Freitextzeilen

Die Zeilen des Freitextes können wie die Zeilen einer FO-Datei mit `.hole von` oder mit `.datei` ermittelt werden. Die Angabe einer Freitextvariablen ist in diesen Kommandos an der Stelle des Dateinamens zulässig.

Kommando	<code>.hole von 'H ftext' x1 x2 x3</code>
Wirkung	Die nächste Freitextzeile wird gelesen und ihre Einträge werden Inhalte der Variablen x1, x2, x3.

Anlegen oder löschen von Freitexten

Mit dem Kommando `.formel` kann ein neuer Freitext erzeugt oder ein vorhandener überschrieben werden. Wie bei allen Dateiangaben in FOPs gelten die [Regeln für FO-Dateinamen](#).

Kommando	<code>.formel Freitext = 'FO-Datei'</code> Freitext ist ein Freitextfeld eines gesperrten Datensatzes, 'FO-Datei' ist der Name einer FO-Datei
Allgemeine Wirkung	Der Inhalt der Datei 'FO-Datei' wird Freitext. War vorher ein Freitext vorhanden, wird er überschrieben. Im Freitext wird der Inhalt der Datei abgelegt, deren Name hinter dem Gleichheitszeichen steht. Diese Datei muss bereits vorhanden sein. Für die Dateinamen gelten die bekannten Konventionen: Groß- und Kleinschreibung werden nicht unterschieden, die Datei wird in den zulässigen Verzeichnissen gesucht. Ist der Name der Leertext ("") oder die zugewiesene Datei ist leer, wird der Freitext gelöscht.
Wirkung beim Speichern	Die Änderung wird erst dann nach außen wirksam, wenn der Datensatz gespeichert wird: * bei Datensätzen im Puffer H erst beim Kommando <code>.bringe</code> * bei Datensätzen im Puffer M erst, wenn Sie speichern. Beim Verwerfen wird diese Freitextänderung zurückgenommen.
Zuweisen einer Leeren Datei oder eines Leerstrings	Wenn Sie einen Freitext löschen oder eine leere Datei an einen Freitext zuweisen, ist das Freitextfeld leer.

Schreiben von Freitexten

Zum Schreiben von Freitexten steht die Funktion `.datei -eingabe 'dateiname'` zur Verfügung.

Es wird eine neue Temporärdatei erstellt, der Name steht in `G|tmpfilename`.

Der Inhalt ist eine Kopie des Inhaltes von `<dateiname>`.

Wenn `<dateiname>` den Namen einer existierenden Datei oder eines Freitextes enthält, ist `mehr = true`.

Wenn die Datei nicht existiert, dann ist `mehr = false`. Wenn der Dateiname leer ist, dann ist die Temporärdatei leer und ebenfalls `mehr = true`.

Der Ersatzcode für `.datei -eingabe 'dateiname'` ist, mit den Hilfsvariablen `U|tmpfilename` und `U|xhlf`, die sonst nicht im Programm vorkommen, der Folgende:

(`U|dateiname` sei der konkrete Wert von `<dateiname>`).

```
.art int xhilf
.art text tmpfilename
.datei -tempname U|tmpfilename
.aus auf 'U|tmpfilename'
.weiter END ? F|empty(U|dateiname)
.datei -oeffnen U|xhilf 'U|dateiname'    --- hier wird "mehr" gesetzt
.weiter END ? _G|mehr
.datei --schliessen U|xhilf
.eingabe -text 'U|dateiname'            --- hier wird die Datei gelesen aber nicht "mehr" gesetzt
!END
.aus bild
```

Nun enthält **U|xtmpname** einen gültigen Dateinamen und eine passend gefüllte Datei. **mehr** ist gesetzt und beantwortet die Frage, ob die Datei in <dateiname> existiert.

Dagegen das neue Konstrukt: **.datei -eingabe 'U|dateiname'**

Nun enthält **G|tmpfilename** den passenden Dateinamen der gefüllten Temporärdatei und **mehr** beantwortet die Frage, ob **U|dateiname** existiert hat und lesbar war.

Kopieren von Freitexten

Wenn in der FO ein Freitextfeld ausgegeben wird, erhalten Sie einen Dateinamen der folgenden Form:

```
(4711,52,0)
```

Sie können Freitexte auch direkt kopieren:

```
.formel H|ftext = 'H|ftext2'
```

Es ist auch möglich, zu kopieren ohne Apostroph und mit **.kopieren**:

```
.kopieren H|ftext = H|text2
```

Sie können **'H|ftext'** wie folgt benutzen:

```
.eingabe 'H|ftext'
.datei -oeffnen 'H|ftext'
.datei -lesen 'H|ftext'
```

Beispiel: Kopie einer Datei auf den Client

`M|clientdateiname` sei ein `DV0`-Feld, in dem schon ein passender Dateiname steht:

```
.weiter LEERFT ? _F|empty(M|ftext)
.hole von 'M|ftext'
.pc.copy -PC 'M|ftext' 'M|clientdateiname'
```

Diese Codestelle nutzt aus, dass die Datei `'M|ftext'` so lange existiert, wie `.hole von ...` geöffnet ist. Bei einer Programmänderung wie:

```
.weiter LEERFT ? _F|empty(M|ftext)
.datei -eingabe 'M|ftext'
.pc.copy -PC 'G|tmpfilename' 'M|clientdateiname'
```

können Sie

```
.weiter LEERFT ? _F|empty(M|ftext)
```

weglassen. Das führt zu einer leeren Temporärdatei, die sich auch auf den Client kopieren lässt.

Exportieren von Freitexten

Zum Exportieren von Freitexten in eine Datei steht beim Kommando `.datei` eine Option sowie die Variable: `G|tmpfilename` zur Verfügung:

```
.datei -eingabe "dateiname"
G|tmpfilename
```

Das Kommando hat folgendes Verhalten:

Es liest den als Parameter bei `-eingabe` angegebenen Dateinamen der Art `DVM2` und kopiert den Inhalt in eine Temporärdatei. Als Dateiname sind alle FO-Dateinamen wie beim `.datei -oeffnen` oder `.hole von` zulässig, also insbesondere auch Freitexte.

Ist der Dateiname ein Leerstring, entsteht eine leere Datei. Die Funktion erfindet einen neuen eindeutigen Namen für diese Temporärdatei. Die Datei hat die gleiche Lebensdauer wie eine mit `.datei -fop -tempname` erstellte Datei, existiert so lange wie das FOP läuft, und wird am Ende gelöscht.

Wenn Sie die Funktion mehrfach aufrufen, entsteht jedesmal eine neue Datei mit einem neuen eindeutigen Namen. Dabei wird `G|tmpfilename` überschrieben. Wenn der Name noch gebraucht wird, müssen Sie ihn sich in einer anderen Variable merken. Das Kommando untersucht den angegebenen Dateinamen und unterscheidet die folgenden Fälle:

- Der Dateiname ist leer oder (0,0,0). Es wird eine leere Temporärdatei erstellt und ihr Name in `G|tmpfilename` geschrieben. `G|mehr` wird true
- Der Dateiname benennt einen Freitext; der Inhalt des Freitextes wird in eine Temporärdatei und ihr Name in `G|tmpfilename` geschrieben. `G|mehr` wird true.
- Der Dateiname ist ein gültiger DVM2-Dateiname; der normierte Dateiname wird in `G|tmpfilename` geschrieben. `G|mehr` bedeutet, dass die Datei existiert und lesbar ist. Sonst: `G|mehr` wird false und `G|tmpfilename` wird leer.

Es gilt nach einem `.datei -eingabe "dateiname"`:

- Wenn immer `G|mehr` true ist, kann die Datei in `G|tmpfilename` anschließend gelesen werden.
- Bei `G|mehr` false und gefülltem `G|tmpfilename` gilt: `"dateiname"` ist ein gültiger DVM2-Dateiname.
- Bei `G|mehr` false und leerem `G|tmpfilename` gilt: `"dateiname"` ist **kein** gültiger DVM2-Dateiname.

Folgende Optionen wirken analog zu `.datei -tempname` auf den Variablennamen in `G|tmpfilename`:

- `-fop`: Die Datei lebt so lange, wie das FOP aktiv ist.
- `-maske`: Die Datei lebt im EFOP bis zum Maskenende des FOPs.
- `JFOP API: getReader(), getTempFileName()` in `ReadableBuffer`

Für den Export eines Freitextes gibt es neben der Funktion `getReader` in `ReadableBuffer` die Funktion:

```
string getTempFileName(string field)
```

Sie kopiert den im Feld `field` stehenden Freitext, oder die Datei eines im Feld angegebenen Dateinamens in eine Temporärdatei und liefert den Namen der Temporärdatei zurück. Das funktioniert auch bei leerem Freitextfeld oder sonstigen Dateinamen. Die Datei mit diesem Namen muss aber vorhanden sein. Die Funktion

```
Reader getReader(string field)
```

verwendet intern `getTempFileName()` und ist daher etwas weniger restriktiv.

Drucken von Freitexten individuell steuern

Sollen die Freitexte aus dem Artikel nicht auf den Vorgangspapieren gedruckt werden, kann dies über die boolesche Variable `xdruckftext` im individualisierbaren Standard-FOP `ev/FOPKOPF` gesteuert werden. Die Standardeinstellung ist, dass die Freitexte auf Vorgangspapieren gedruckt werden. Wenn Sie diesen Druck nicht wünschen, muss die Variable `xdruckftext` auf TRUE gesetzt werden.

EFOP - Programmierung ereignisgesteuerter FOPs

Ein FOP kann nicht nur explizit durch ein Kommando gestartet werden ("Kommando-FOP", Kommandosyntax dafür ist <Text> 'FOPNAME' <Zeigen>), sondern es kann auch durch ein Ereignis automatisch ausgelöst werden. Ein ereignisgesteuertes FOP wird als EFOP bezeichnet. In den folgenden Abschnitten werden die Konfiguration und Aufrufumgebung von EFOPs, die zur Verfügung stehenden Ereignisse und besondere Kommandos für EFOPs dargestellt.



EFOPs können nur bei Selektionen in GUI, ASCII und EDP ausgeführt werden. EFOPs können nicht im FOP oder beim Druck von Selektionsergebnissen ausgeführt werden.

Testen von EFOPs

Um die Entwicklung und das Testen von EFOPs zu erleichtern, gelangt man aus allen Masken unmittelbar in den **FO-Direktmodus**: Rechtsklick in einem Feld > [FOP] > [Direktmodus]. Voraussetzung ist die entsprechende Berechtigung. Der Zustand im Direktmodus verhält sich so, als wäre im aktuellen Feld ein Funktionstastenergebnis eingetreten.

Administration von EFOPs

Um ereignisgesteuerte FOPs (EFOPs) nutzen zu können, muss die Funktion "ereignisgesteuerte FOPs" in der Anwendungskonfiguration des Mandanten aktiviert sein. Ferner benötigt ein EFOP-Programmierer die Berechtigung zur EFOP-Programmierung und zu bestimmten Wartungsarbeiten.

Diese erhält er durch den Eintrag **FOP-Anbindung** im Erlaubnis-Datensatz seiner Passwortdefinitionen.

In einer GUI-Maske kann für diese Maske über das Kontextmenü im Untermenü "FOP" die **Ereignissteuerung konfiguriert** werden.

Ereignisse

Bei bestimmten Ereignissen besteht die Möglichkeit, automatisch benutzerdefinierte Programme aufzurufen. Diese werden als EFOPs (ereignisgesteuerte FOPs) bezeichnet. Sie können als Java-FOPs oder herkömmliche FOPs implementiert werden.

Die Aktivitäten eines ereignisgesteuerten FOPs sind für den Benutzer in der Regel nicht von denen des Abas Systems zu unterscheiden.

Sie können EFOPs Ihren Anforderungen gemäß anlegen und **mit einem beliebigen Ereignis verknüpfen**. Bei Datenbanken können pro Ereignis beliebig viele EFOPs angegeben werden, wobei auch eine Mischung von Java-EFOPs und herkömmlichen zulässig ist. Diese EFOPs haben Zugriff auf den gerade editierten Datensatz und können z.B. alle Felder lesen und alle nicht schreibgeschützten Feldinhalte ändern. Zudem besteht bei bestimmten Ereignissen die Möglichkeit, durch einen Exit-Status Informationen an das Programm zurückzugeben und so den weiteren Programmablauf zu beeinflussen, z.B. für individuelle Anpassungen, Eingriffe in die Dialogführung und die Einrichtung individueller Datenstrukturen.

Zusammenwirken mehrerer EFOPs für ein Ereignis

Durch die Möglichkeit, mehrere EFOPs mit einem Ereignis zu verknüpfen, kann unabhängige Funktionalität in unabhängigen Programmen implementiert werden. Insbesondere kann man an ein und das selbe Ereignis sowohl EFOPs aus dem Lieferumfang von Abas als auch eigene anhängen.

Die einzelnen EFOPs laufen dabei vollständig isoliert voneinander (d.h., ein EFOP sieht keine Variablen, gefüllten Puffer oder anderen Zustände seiner Vorgänger). Sie werden in der Reihenfolge ausgeführt, wie sie in der [Konfigurationsdatei](#) stehen.

Bei prüfenden Ereignissen (z.B. Maskenprüfung, Feldprüfung) werden die konfigurierten EFOPs nur bis zum ersten Exit-Status ungleich 0 ausgeführt. Sonst werden alle konfigurierten EFOPs ausgeführt, auch dann, wenn eines davon z.B. abbricht.

Das Konfigurieren mehrerer EFOPs hat keinen messbaren Performance-Nachteil gegenüber der Alternative, nur ein EFOP zu konfigurieren, das mehrere Unterprogramme aufruft.

Für das Zusammenwirken der EFOPs gilt Folgendes:

Exit-Status	Der Exit-Status des letzten ausgeführten EFOPs wird zurückgegeben.
Cursorpositionierung	Der letzte Aufruf von <code>.cursor</code> innerhalb der ausgeführten EFOPs wird wirksam.
Bildschirmausgabe	Die Bildschirmausgabe erfolgt sofort. Die Ausgabe für jedes EFOPs erfolgt in einem eigenen Ausgabefenster. Dieses trägt den Namen des EFOPs im Fenster-Titel. Dadurch ist die Ausgabe dem EFOP zugeordnet.
FO-Fenster soll nach diesem EFOP geschlossen werden	Jedes der EFOPs hat sein eigenes Ausgabefenster, daher wird der Befehl <code>.setze fensterschliessen</code> + sofort nach dem jeweiligen EFOP wirksam und beeinflusst auch nicht Vorgänger oder Nachfolger.
Fehlermeldung	Alle <code>.fehler</code> werden wirksam, und zwar jeweils am Ende der Ausführung des jeweiligen EFOPs.
Hinweise	Alle <code>.hinweis</code> werden wirksam, und zwar jeweils am Ende der Ausführung des jeweiligen EFOPs (außer bei Option <code>-sofort</code> , wo sie sofort wirksam werden).
Charts, Farben, Feldschutz	<code>.chart</code> , <code>.farbe</code> und <code>.schutz</code> wird sofort wirksam.

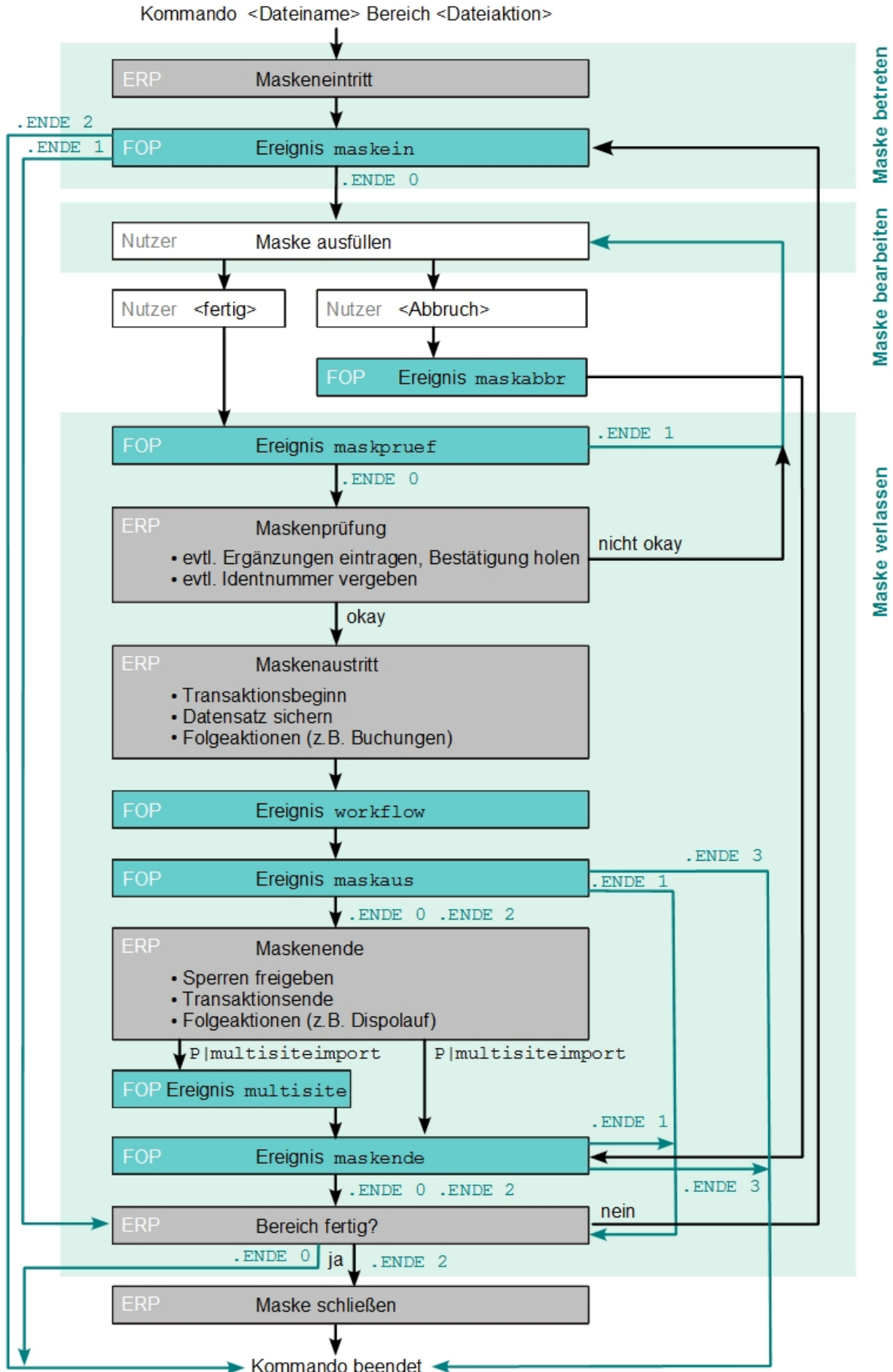
Neuer Feldwert beim Ereignis "Feld ausgefüllt" (Variable 'evtfeldneu')	Alle EFOPs sehen den neuen Feldwert. Dieser Wert wird gespeichert. Wenn Sie in ein Eingabefeld in der Maske einen Wert eingeben oder ändern und anschließend das Feld verlassen, dann werden nacheinander folgende Events ausgelöst: 1. FieldFill 2. FieldValidation 3. FieldExit Wenn das Ereignis FieldFill ausgelöst wird, dann wird der vom Benutzer eingegebene Wert zum Inhalt der globalen Variable G evtfeldneu.  Unterprogramme, die direkt im Anschluss aufgerufen werden, haben über die globale Variable G evtfeldneu Zugriff auf den vom Benutzer eingegebenen Wert. Sobald das Event FieldValidation ausgelöst wird, ist der Inhalt der globalen Variable G evtfeldneu leer. Im Maskenfeld wird der vom Benutzer eingegebene und ggf. von Kern modifizierte Wert ausgegeben.
Zuweisbarkeit beim Ereignis "Feldprüfung"	Im feldpruef-Ereignis sind Zuweisungen an andere Felder erst nach Zuweisung an das geprüfte Feld erlaubt. Wenn eins der angehängten FOPs diese Zuweisung ausführt, dürfen auch alle Nachfolger an andere Felder zuweisen.
Gedrückter Ende-Button	Der letzte Aufruf von .ende 'exitstatus' 'buttonname' innerhalb der ausgeführten EFOPs wird wirksam.

Die Ereignisse, die ein Programm starten können, lassen sich in die folgende Gruppen einteilen:

Sie können sich sämtliche Ereignisse von Kommandos **anzeigen** lassen.

Maskenereignisse

Beim Maskenereignis sieht der Ablauf beim Editieren eines Datensatzbereichs wie folgt aus:



Sie können bei den Ereignissen Maskenaustritt und Maskenende "für den Benutzer" durch die Angabe von **.ende 1** die Maske schließen. Wenn Sie das Kommando **.ende 2** verwenden, beachten Sie folgendes:

- **.ende 2** gleich im 1. geladenen Objekt - es wird eine Fehlermeldung ausgegeben und die Maske öffnet sich nicht.
- **.ende 2** ab dem 2. geladenen Objekt - es wird eine Fehlermeldung ausgegeben und die Maske wird im Modus leer geöffnet.

Das Kommando **.ende 3** bricht das Durchlaufen des Bereichs ab.

Ereignisse, die sich auf eine Maske oder auf die Änderung eines Datensatzes beziehen

Maskeneintritt maskein	Bei Maskeneintritt sind alle Vorbelegungen von Feldern durch den <i>Kern</i> bereits erfolgt. Die (optionalen) Vorbelegungen von Feldern durch <i>Kommandoparameter</i> sind noch nicht erfolgt. Die Maske für den Datensatz ist aber noch nicht sichtbar (Bei der Bereichsauswahl "von ! bis" ist u.U. noch der Vorgänger sichtbar). Das EFOP kann festlegen, ob der Dialog zulässig ist, und den Datensatz verändern oder ergänzen.
Maskenprüfung maskpruef	Der Benutzer will den Datensatz speichern. Der Datensatz ist noch nicht abschließend geprüft und ergänzt. Bei der Neuanlage eines Datensatzes ist die Identnummer noch nicht vergeben und ergänzende Positionen sind noch nicht generiert. Das EFOP kann den Datensatz verändern und festlegen, ob der Maskenaustritt zulässig ist.
Workflow workflow	Beim Workflow-Ereignis ist der Datensatz erfolgreich geprüft, bereits gesichert (auf Festplatte geschrieben!), aber noch gesperrt. Tritt beim Abbruch nicht ein.
Maskenaustritt maskaus	Hier gilt das Gleiche wie bei Workflow. Zusätzlich kann das EFOP den Datensatz verändern und mit .bringe maske nochmals schreiben. Bei diesem Ereignis gelten nur die Änderungsrechte des Festplattenzustandes .
Maskenabbruch maskabbr	Bei Maskenabbruch hat der Benutzer <Abbruch> betätigt und die Rückfrage bereits bestätigt. Der Datensatz ist noch geladen und gesperrt, wird aber nicht editiert. Das EFOP kann ggf. Aktionen rückgängig machen, die vorbereitend während des Dialogs getätigt wurden.
Maskenende maskende	Das Ereignis tritt sowohl beim Speichern als auch beim Abbruch ein. Eine Unterscheidung ist über den Wert von ' G evttaste ' möglich. Der Datensatz ist nicht mehr gesperrt.



EFOPs an **workflow**- und **maskaus**-Ereignissen dürfen **keine Dialoge** enthalten! Sie laufen immer im Transaktionskontext, in dem alle geschriebenen Datensätze, Nummernzähler oder Sondersperren noch exklusiv gesperrt sind. Durch einen Dialog würden diese Sperren lange gehalten. Währenddessen könnte kein anderer Benutzer oder Batchprozess diese Objekte mit Sperre lesen.

Grundsätzlich sollten Dialoge in allen EFOPs vermieden werden, da bei vom Kern aufgerufenen inneren Editoren auch alle anderen EFOPs innerhalb von Transaktionen aufgerufen werden können. So werden beispielsweise beim Schreiben eines Basisartikels in dessen Schreibtransaktion alle zugehörigen Artikel geschrieben, wobei alle für Artikel konfigurierte EFOPs ausgeführt werden.

In den EFOPs, die beim Speichern eines Datensatzes in einer Maske aufgerufen werden (**maskpruef**, **maskaus**, **maskende**, **maskabbr**) ist das aktuelle Eingabefeld bekannt.

Über das aktuelle Eingabefeld stehen, wie beim Auslösen von Symbolleistenbuttons oder Menüpunkten, folgende Informationen zur Verfügung:



- **evtfokuszeile** Fokus-Maskenzeile beim Speichern
- **evtfokusvar** Fokus-Feld laut Variablentabelle beim Speichern
- **evtfokusvarori** Fokus-Feld (ursprünglicher Name) beim Speichern
- **evtfokusvarengl** Fokus-Feld (englischer Name) beim Speichern

Beim Maskeneintritt kann das EFOP das Editieren des Datensatzes verweigern. Auf der GUI wird beim Maskeneintritt aus ergonomischen Gründen immer eine Zeile eingefügt, sofern noch keine vorhanden ist und das Zeile-einfügen-FOP (**zeileeinvar** oder **zeileeinnach**) dies zulässt. Falls es dies nicht erlaubt, werden Fehlermeldungen des Zeile-einfügen-FOPs nicht ausgegeben. Andernfalls würde z.B. jede Neuanlage eines Objektes mit einer Fehlermeldung beginnen. Das **zeileeinvar**-FOP kann also den Befehl **.fehler ...** unabhängig davon starten, ob es wegen eines Maskeneintritts oder zum Einfügen einer Zeile aufgerufen wurde.

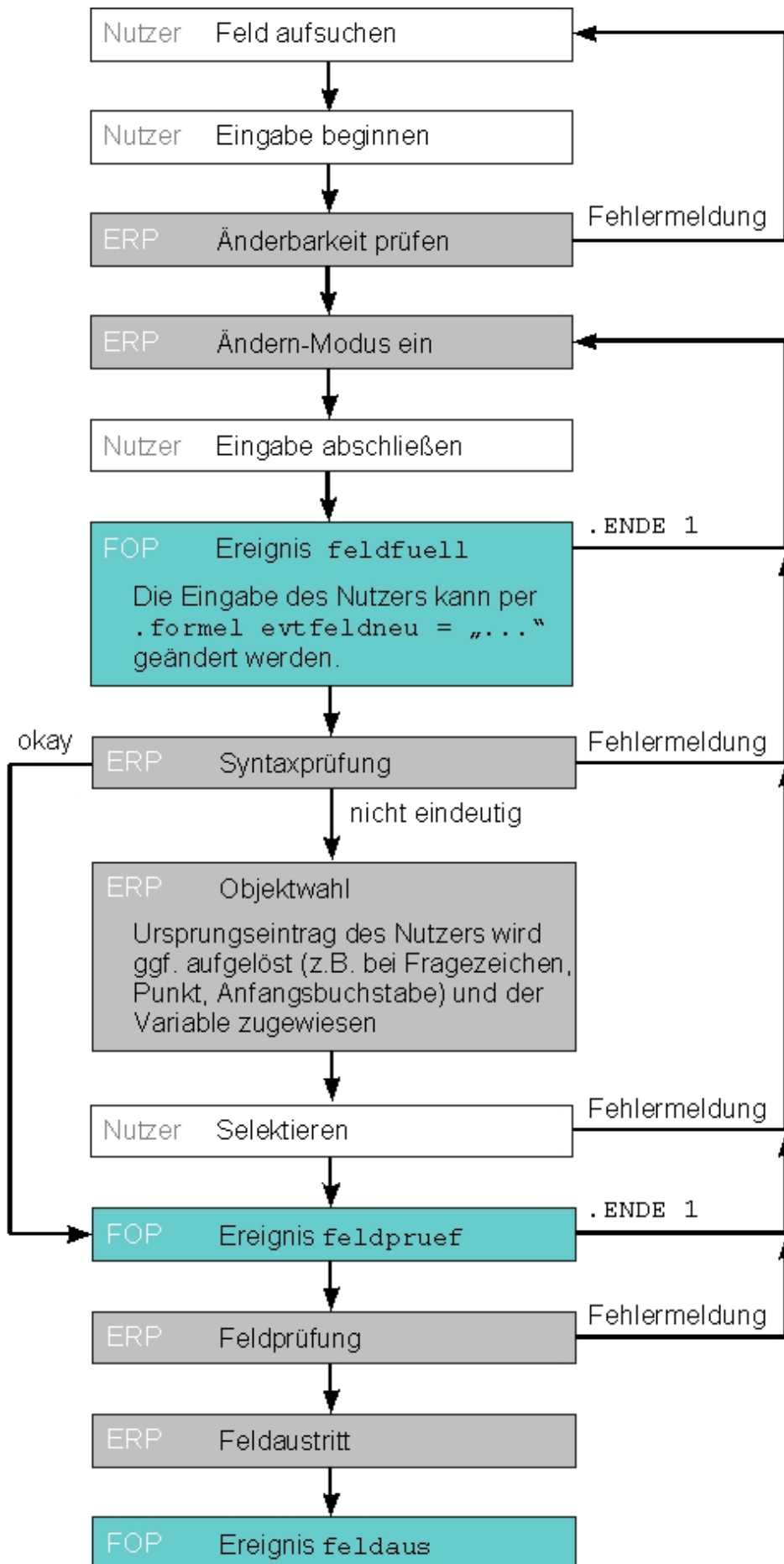
Bei der Maskenprüfung kann das EFOP die Maske als falsch oder unvollständig klassifizieren und damit den Benutzer am Verlassen der Maske hindern.

Während der Ausführung lang dauernder EFOPs wird auf der grafischen Benutzeroberfläche (GUI) nach 3 Sekunden ein Meldungsfenster mit einem Button "Abbrechen" angezeigt. Nach einem Klick auf diesen Button wird das EFOP abgebrochen (sofern der Abbruch nicht mit **.setze abbruch -** verboten wurde).

Das Ereignis **Multisite-Event** wird beim Speichern eines Datensatzes nach dem Maskenaustritt-FOP und vor dem Maskenende-FOP ausgelöst.

Feldereignisse

Beim Feldereignis sieht der Ablauf beim Editieren eines Feldes wie folgt aus:



Ereignisse, die sich auf ein Maskenfeld beziehen

Feld ausgefüllt feldfuell	Unmittelbar nach dem Beenden eines Feldeintrages (das meist mit <Return> erfolgt) kann ein EFOP gestartet werden, das den Feldinhalt ggf. noch verändern kann. Beispielsweise kann die Objektauswahl bei Verweisfeldern gesteuert werden oder der Eintrag auf andere Art beeinflusst werden. Zum Zeitpunkt des Ereigniseintritts hat noch keinerlei Feldprüfung stattgefunden.  Im EFOP steht die Benutzereingabe noch nicht in der entsprechenden Feldvariablen, sondern in der Textvariablen G evtfeldneu. Beim Klick auf das Kalendersymbol eines Datumsfeldes steht in G evtfeldneu ein "?". Soll das Feld geändert werden, so muss die Zuweisung auch an diese Variable erfolgen. Die Zuweisung an die Feldvariable selbst ist wirkungslos und führt zu einer Fehlermeldung. Bei diesem Ereignis kann das EFOP die Eingabe zurückweisen und den Benutzer am Verlassen des Feldes hindern.
--	---

Feldprüfung

feldpruef

Dieses Ereignis wird ausgelöst, wenn die Feldeingabe auf richtige Syntax geprüft ist und ggf. eine Selektion stattgefunden hat (z.B. nach der Eingabe eines Fragezeichens in ein Verweisfeld). Die inhaltlichen Prüfungen durch das Programm stehen jedoch noch bevor. Das EFOP kann die Eingabe akzeptieren oder ablehnen. Anderes sollte in diesem EFOP normalerweise nicht geschehen. Die Möglichkeit zur Änderung des Feldwertes besteht beim Feldfüll- und beim Feldaustrittsereignis.

Zuweisungen an andere Variablen des Datensatzes sind in dieser Situation verboten, da das Ereignisfeld einen vom Programm noch nicht geprüften Wert enthält. Der Datensatz könnte also aus der Sicht des Programms in einem -illegalen Zustand sein.



Zuweisungen an das Ereignisfeld selbst sind nicht verboten. Wird die Zuweisung akzeptiert (und werden somit alle Folgeaktionen ausgelöst), ist dies endgültig. Selbst wenn das EFOP danach einen Fehler-Exit-Status erzeugt (z.B. weil es der Benutzer abbricht), kann der ursprüngliche Feldwert nicht mehr restauriert werden. Der Bearbeitungsmodus wird in jedem Fall verlassen. Vorzuziehen sind deshalb Feldänderungen beim Feldfüll-Ereignis. Nach einer erfolgreichen Zuweisung an das Ereignisfeld ist der Datensatz wieder im geprüften Zustand und Zuweisungen an andere Felder sind möglich. In aller Regel sollte aber im Feldprüfereignis besser keine Zuweisung erfolgen. Für Folgeaktionen ist der Feldaustritt geeigneter.

Bei diesem Ereignis kann das EFOP die Eingabe zurückweisen und den Benutzer am Verlassen des Feldes hindern.

Feldaustritt fel daus	Beim Feldaustritt wurde die Benutzereingabe angenommen und vom Programm auf seine Gültigkeit geprüft. Es können Folgeaktionen ausgeführt werden, beispielsweise zusätzliche Maskenfelder mit Werten vorbelegen, Meldungen ausgeben, insbesondere im Hinblick auf die später folgende Gesamtmaskenprüfung.
-------------------------------------	---

Buttonereignisse

Jedes Buttonereignis kann mit mehreren EFOPs verknüpft werden. Je nach Art des Buttons gibt es verschiedene Möglichkeiten.

Ablauf beim Klick auf einen BU3-Button

Schritt	Ebene	Aktion	Bemerkung
1	Benutzer	Button klicken (BU3)	
2	ERP	Zulässigkeit prüfen	nicht erlaubt → Aktion wird abgebrochen
3	FOP	Ereignis buttonvor	.ENDE 1 → Aktion wird abgebrochen
		.ENDE 0	
4	ERP	Buttonfunktion auslösen	
5	FOP	Ereignis buttonnach	

Ablauf beim Klick auf einen TN-Button

Schritt	Ebene	Aktion	Bemerkung
1	Benutzer	Button klicken (TN)	
2	FOP	Ereignis buttonvor	.ENDE 1 → Aktion wird abgebrochen
		.ENDE 0	
3	ERP	Freitext editieren	
4	FOP	Ereignis buttonnach	

Ablauf beim Klick auf einen BU8-Button

Schritt	Ebene	Aktion	Bemerkung
1	Benutzer	Button klicken (BU8)	
2	ERP	Zulässigkeit prüfen	nicht erlaubt → Aktion wird abgebrochen
3	FOP	Ereignis buttonvor	.ENDE 1 → Aktion wird abgebrochen

Schritt	Ebene	Aktion	Bemerkung
		. ENDE 0	

Schritt	Ebene	Aktion			Bemerkung
		FOP	Subma ske: Ereigni s maske in		
		ERP	Subma ske ausfüll en mit allen übliche n EFOPs		
		FOP	Subma ske: Ereigni s maskp ruef		
		FOP	Subma ske: Ereigni s workf low		
		FOP	Subma ske: Ereigni s maska us	.Ende 1 → Subma ske wird geschl ossen	

Schritt	Ebene	Aktion	Bemerkung
4	FOP	Ereignis buttonnach	

Ablauf beim Klick auf einen BU10-Button

Schritt	Ebene	Aktion	Bemerkung
1	Benutzer	Button klicken (BU10)	
2	ERP	Zulässigkeit prüfen	nicht erlaubt → Aktion wird abgebrochen
3	FOP	Ereignis buttonvor	.ENDE 1 → Aktion wird abgebrochen
		.ENDE 0	
4	FOP	Maske: Ereignis maskaus	.ENDE 1 → Maske wird geschlossen
5	ERP	Maskenende	
6	FOP	Ereignis maskende	.ENDE 1 → Maske wird geschlossen

Schritt	Ebene	Aktion			Bemerkung
		FOP	Subma ske: Ereigni s maske in	Aufruf ermask e nicht gesper rt	
		ERP	Subma ske ausfüll en mit allen übliche n EFOPs		
		FOP	Subma ske: Ereigni s maskp ruef		
		FOP	Subma ske: Ereigni s workf low		
		FOP	Subma ske: Ereigni s maska us	.Ende 1 → Subma ske wird geschl ossen	

Schritt	Ebene	Aktion	Bemerkung
7	FOP	Maske: Ereignis maskein	
8	FOP	Ereignis buttonnach	

Ablauf beim Klick auf einen BU12-Button

Schritt	Ebene	Aktion	Bemerkung
1	Benutzer	Button klicken (BU12)	
2	ERP	Zulässigkeit prüfen	nicht erlaubt → Aktion wird abgebrochen
3	FOP	Ereignis buttonvor	.ENDE 1 → Aktion wird abgebrochen
		.ENDE 0	
4	FOP	Maske: Ereignis maskaus	.ENDE 1 → Maske wird geschlossen
5	ERP	Maskenende	
6	FOP	Ereignis maskende	.ENDE 1 → Maske wird geschlossen
7	FOP	Maske: Ereignis maskein	

Schritt	Ebene	Aktion			Bemerkung
		FOP	Subma ske: Ereigni s maske in	Aufruf ermask e gesper rt	
		ERP	Subma ske ausfüll en mit allen übliche n EFOPs		
		FOP	Subma ske: Ereigni s maskp ruef		
		FOP	Subma ske: Ereigni s workf low		
		FOP	Subma ske: Ereigni s maska us	.Ende 1 → Subma ske wird geschl ossen	

Schritt	Ebene	Aktion	Bemerkung
7	FOP	Ereignis buttonnach	

Einschränkungen



- Das **buttonnach**-Ereignis bei BU10- und B12-Buttons wird erst nach dem Beenden der Submaske ausgelöst.
- Für DVZ-Buttons kann derzeit kein EFOP konfiguriert werden.
- Bei URL1-Buttons tritt derzeit kein Ereignis ein und kann demzufolge auch kein EFOP konfiguriert werden.
- Beim Öffnen von Teilmasken oder Zeilenlupen und beim Seitenwechsel in einer Maske tritt kein Ereignis ein.
- Wenn Maskenergebnisse durch Klicken eines BU10-, oder BU12-Buttons ausgelöst werden, dann enthält die Variable **G|evtvar** den Namen des Buttons.
- Nach dem Klicken eines Subeditor-Buttons steht die Datensatz-ID im **buttonnach**-Ereignis in der FO-Variablen **G|evtsubeditorid** zur Verfügung.

Ereignisse, die sich auf einen Button beziehen

Button geklickt, vor der Abas Behandlung buttonvor	Greift das EFOP <i>vor</i> der Abas Behandlung des Ereignisses, gilt: • Der Button wird betätigt und das Ereignis tritt unmittelbar danach ein, ohne dass es von Abas zuvor behandelt wird. • Mit dem Exit-Status 1 kann die Abas Behandlung verhindert werden. • Der Name des Ereignisses im EFOP ('G evtart') ist buttonvor • Bei einem zeilenbezogenen Button ist die aktuelle Zeile geladen. • Das Ereignisfeld hat den Namen des entsprechenden Buttons oder Menüpunktes.
Button geklickt, nach der Abas Behandlung buttonnach	Greift das EFOP <i>nach</i> der Abas Behandlung des Ereignisses, gilt: * Der Button wurde geklickt, das Ereignis wurde weder vom EFOP noch von Abas verboten. Die Abas Behandlung ist abgeschlossen. * - Der Exit-Status (.ende 'Exit-Status') des EFOPs ist ohne Bedeutung. * - Der Name des Ereignisses im EFOP ('G evtart') ist buttonnach * Bei einem zeilenbezogenen Button ist die aktuelle Zeile geladen.

Editieren von Freitexten

Für das Editieren von Freitexten können individuelle Ereignisbehandlungen definiert werden. Damit können Sie z.B.

- das Anschauen oder Editieren des Freitextes unterbinden
- den Freitext vorbelegen
- den editierten Freitext überprüfen
- das Löschen des Freitextes verhindern

Zu diesem Zweck treten beim Editieren von Freitexten wie bei allen anderen Buttons die Ereignisse **buttonvor** und **buttonnach** ein, die Sie behandeln können.

Für das Ereignis vor dem Editieren oder Löschen gilt:

- Es tritt beim Editieren oder beim Löschen ein:
 - Direkt nach dem Drücken des Freitextbuttons. Die Behandlung im Kern hat noch nicht stattgefunden.
 - Beim Löschen von Freitexten direkt nach dem Auslösen des Löschvorgangs (GUI: Kontextmenü). Die Frage, ob wirklich gelöscht werden soll, wurde noch nicht gestellt.
- Mit dem Exit-Status 1 kann das Öffnen bzw. das Löschen des Freitextes verboten werden. Nicht dokumentierte Exit-Status sind wie immer nicht zulässig. Wenn der Freitext angezeigt, aber nicht geändert werden darf, ist dazu kein Freitext-EFOP notwendig. Dies lässt sich mit **.schutz** erreichen.
- Die Ereignisart ('**G|evtstart**') ist **buttonvor**.
- Ob bei dem Ereignis der Freitext im Zeigen- oder Bearbeitungsmodus angezeigt oder ob er gelöscht werden soll, ist anhand von '**G|evttaste**' zu unterscheiden, mögliche Werte sind:
 - evttaste = "ändern"
 - evttaste = "zeigen"
 - evttaste = "löschen"

Es erscheint bei

.ende 1

eine Standardfehlermeldung, die mit

.fehler 'beliebiger Text'



individuell gestaltet werden kann. Die angegebene Fehlermeldung muss immer in Bediensprache erscheinen. Bei Mandanten mit mehreren Bediensprachen sollte diese Fehlermeldung am besten über das Wörterbuch ermittelt werden. Die Fehlermeldung erscheint erst nach dem Beenden des Freitexteditors. Es sollte nicht statt **.fehler** mit **.box** oder **.hinweis** gearbeitet werden.

Es darf nur mit dokumentierten Methoden auf den Freitext zugegriffen werden (z.B. mit **.ausgabe**, **.formel**, **.eingabe**). Der Freitext ist nicht unbedingt unter einem

bestimmten Namen im Dateisystem zu finden, da er eventuell mit Abbruch verlassen wurde und Änderungen erst mit dem Speichern der Maske wirksam werden.

Für das Ereignis *nach* dem Editieren oder Löschen gilt:

- Es tritt beim Editieren und beim Löschen ein:
 - Direkt nachdem der Benutzer den Texteditor verlassen hat,
 - nachdem der Freitext gelöscht wurde.
- Der Exit-Status hat keine Bedeutung.
- Die Ereignisart ('G|evtart') ist **buttonvor**.
- Das Editieren, Zeigen und Löschen von Freitexten ist anhand der gesetzten Taste zu unterscheiden:
 - evttaste = "ändern": beim Editieren
 - evttaste = "zeigen": beim Anzeigen
 - evttaste = "löschen": beim Löschen



Es darf nur mit dokumentierten Methoden auf den Freitext zugegriffen werden (z.B. mit **.ausgabe**, **.formel**, **.eingabe**). Der Freitext ist nicht unbedingt unter einem bestimmten Namen im Dateisystem zu finden, da er eventuell mit Abbruch verlassen wurde und Änderungen erst mit dem Speichern der Maske wirksam werden.

Feldbezogene Funktionen über Symbolleistenbuttons

In Masken hat man die Möglichkeit, "universelle" Funktionen auszulösen, die sich auf das aktuelle Eingabefeld beziehen, z.B. "suche diesen Feldinhalt mit Google". Die Funktion kann durch einen Symbolleistenbutton, einen Menüpunkt unter [Ausführen] bzw. [Tabelle] oder die entsprechende Tastenkombination ausgelöst werden und bezieht sich jeweils auf das aktuelle Eingabefeld. Beispiele für solche "universellen" Funktionen sind:

- Suche weiterer Datensätze mit dem gleichen Feldinhalt
- Anzeige aller EFOPs zu diesem Feld
- Anzeige einer firmenspezifischen Hilfe zum Eingabefeld
- Anzeige der Änderungshistorie dieses Felds
- Übergabe des Feldinhalts an ein anderes Programm oder an die Zwischenablage
- Rechtschreibprüfung zum Eingabefeld.

Die Funktion muss jeweils per EFOP implementiert und mit einem Symbolleistenbutton verknüpft werden. Symbolleistenbuttons belassen den Cursor auf dem aktuellen Feld, während sonstige Buttons beim Klick selbst den Cursor erhalten. Über das Fokusfeld stehen analog zum Ereignisfeld folgende Informationen zur Verfügung:

evtfokuszeile	Fokus-Maskenzeile bei Ereignis
evtfokusvar	Fokus-Feld laut Variablentabelle bei Ereignis
evtfokusvarori	ursprünglicher Name des Fokus-Felds bei Ereignis

<code>evtfokusvarengl</code>	englischer Name des Fokus-Felds bei Ereignis
------------------------------	--

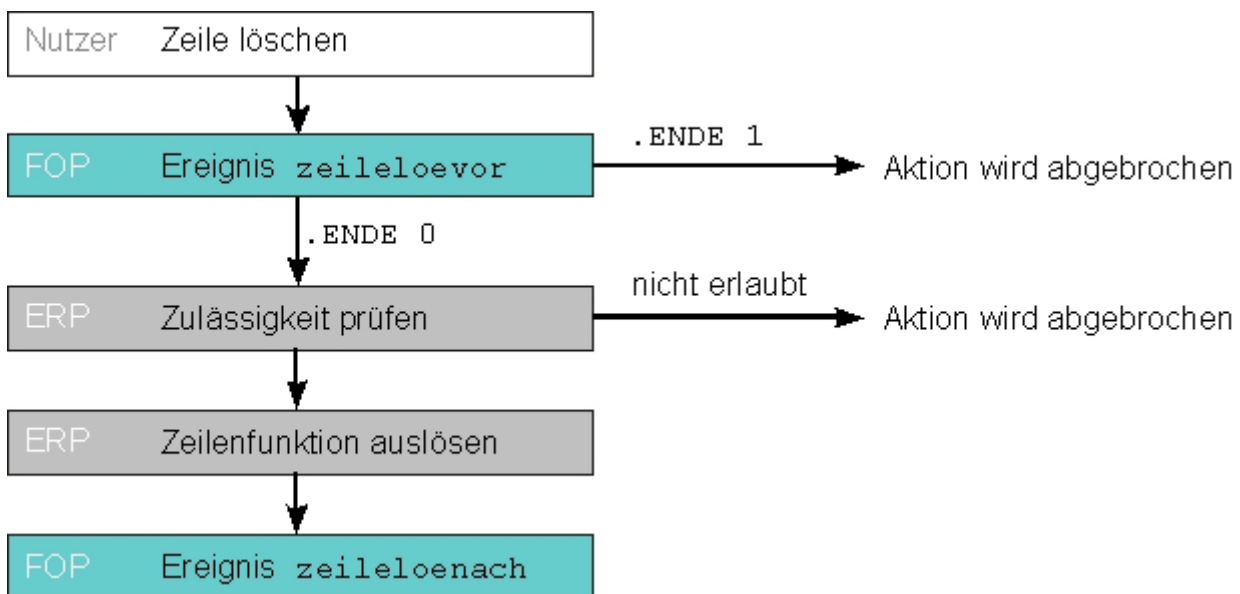
Der Name des Symbolleistenbuttons selbst steht in `evtvar`. Bei sonstigen Buttons gilt `G|evtart = evtfokusvar`.

Beim Auslösen von **Buttons über EDP** kann das Bezugsfeld explizit angegeben werden, auf das sich die Funktion des Buttons bezieht.

Zeilenereignisse - EFOPs in Tabellen

Löschen einer Zeile

Ablauf beim Löschen einer Zeile:



Für das EFOP "Zeile löschen" *vor* der Abas Behandlung gilt:

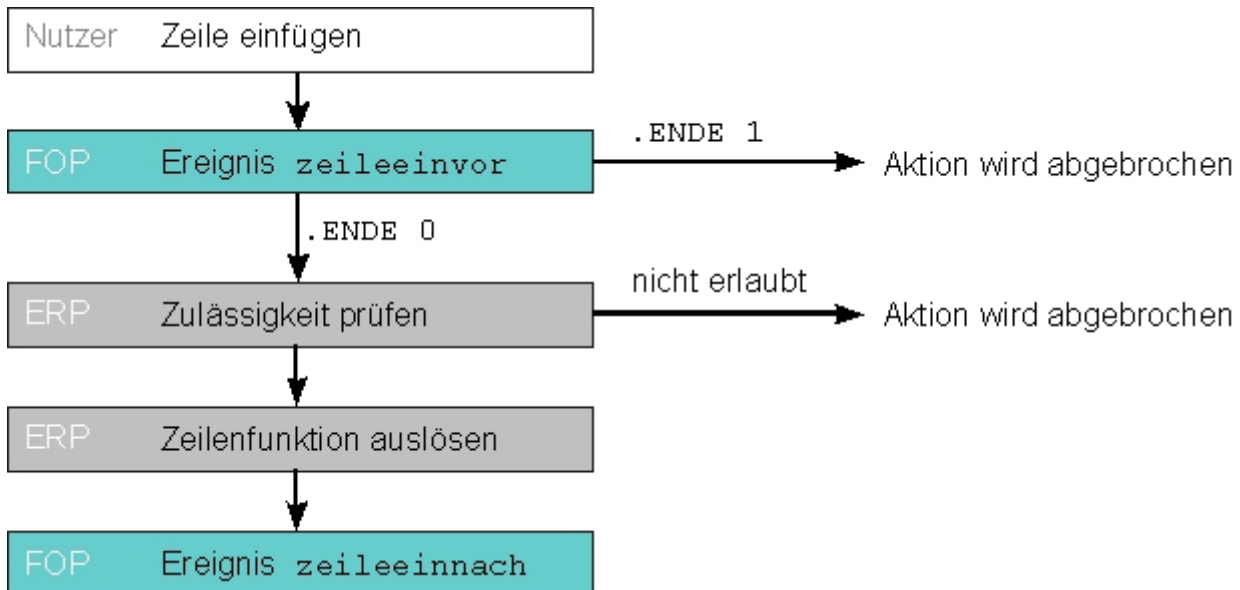
- Es soll eine Zeile gelöscht werden. Die Abas Behandlung hat noch nicht stattgefunden.
- Mit dem Exit-Status 1 kann das Löschen verhindert werden.
- Der Name des Ereignisses im EFOP ('`G|evtart`') ist `zeileloevor`
- Die aktuelle Zeile ist geladen und noch vorhanden.

Für das EFOP "Zeile löschen" *nach* der Abas Behandlung gilt:

- Es wurde eine Zeile gelöscht. Die Aktion ist weder vom EFOP noch von Abas verboten worden.
- Der Exit-Status hat keine Bedeutung.
- Der Name des Ereignisses im EFOP ('`G|evtart`') ist `zeileloenach`.
- Die Zeile ist gelöscht. Im Normalfall ist die nachfolgende Zeile geladen. Ist jedoch keine oder nur noch eine einzige und zudem leere Zeile vorhanden, so ist der Kopf geladen (`evtzeile = 0`)

Einfügen einer Zeile

Ablauf beim Einfügen einer Zeile:



Für das EFOP "Zeile einfügen" *vor* der Abas Behandlung gilt:

- Es soll eine Zeile eingefügt werden. Die Abas Behandlung hat noch nicht stattgefunden.
- Mit Exit-Status 1 kann das Einfügen verhindert werden.
- Der Name des Ereignisses im EFOP ('G|evtart') ist **zeileeeinvor**.
- Die aktuelle Zeile ist geladen.

Für das EFOP "Zeile einfügen" *nach* der Abas Behandlung gilt:

- Es wurde eine Zeile eingefügt. Die Aktion ist weder vom EFOP noch von Abas verboten worden.
- Der Exit-Status hat keine Bedeutung.
- Der Name des Ereignisses im EFOP ('G|evtart') ist **zeileeeinnach**.
- Die eingefügte Zeile (normalerweise eine Leerzeile) ist geladen.

Wechseln der aktuellen Zeile

Dieses Ereignis gibt es nur in einer **Baumannsicht**. Beim Wechseln der aktuellen Zeile gilt folgender Ablauf:

Schritt	Ebene	Aktion
1	Benutzer	Zum anderen Eintrag in die Baumannsicht wechseln
2	FOP	Ereignis zeilenwechsel

Für das EFOP **Zeile wechseln** gilt:

- In einer Baumannsicht wurde die Zeile gewechselt.

- Der Exit-Status hat keine Bedeutung.
- Der Name des Ereignisses im EFOP ('G|evtart') ist **zeilewechsel**.
- Die ausgewählte Zeile ist geladen.

Bewegen von Zeilen

Im Zusammenhang mit dem Bewegen von Tabellenzeilen treten Ereignisse ein, mit denen Sie die Bewegung kontrollieren, unterbinden oder nachbehandeln können. Es ist beispielsweise möglich, nach dem Bewegen der Tabellenzeilen ihre Nummerierung oder Farbgestaltung neu zu ermitteln. In diesem Kontext treten z.B. bei folgenden Benutzeraktionen Ereignisse ein:

- Markieren einer Zeile
- Verschieben von zuvor markierten Zeilen.
- Drag-and-drop einer Zeile (nur GUI).
- Sortieren nach einer Tabellenspalte nach einem Klick auf die Spaltenüberschrift (nur auf der GUI).

Die dabei eintretenden Ereignisse sind:

- Die Zeile wird markiert.
- Die Zeilenbewegung steht unmittelbar bevor.
- Die Zeilenbewegung hat gerade stattgefunden.

Die EDP-Schnittstelle löst alle drei Ereignisse bei Ausführung der Option Move Rows aus.

Die einzelnen Ereignisse werden typischerweise folgendermaßen genutzt: Mit dem Markierereignis kann verhindert werden, dass eine Reihe von Zeilen vergeblich markiert wird, die in der betreffenden Maske nicht vertauscht werden können. Beim Ereignis "Zeile verschoben, nach der Abas Behandlung" können Nachbehandlungen durchgeführt werden. Das Sortieren nach einer Tabellenspalte setzt keine vorherige Markierung voraus und kann demzufolge nur über das Vorher-Ereignis verhindert werden.

Wenn ein EFOP das Markieren oder Verschieben für eine der Zeilen verbietet, wird die gesamte Operation nicht durchgeführt. Beim Verschieben von Zeilen werden ihre Farben in aller Regel mitgenommen. Nur bei sehr speziellen Anwendungen, die beim Verschieben zusätzlich Zeilen einfügen oder löschen (z.B. Materialzuordnung), trifft dies ggf. nicht zu.

Markieren einer Zeile

- Zeile wurde markiert
- Ereignisart ('G|evtart') ist **zeilemark**.

Eintreten des Ereignisses

Wenn eine Zeile markiert werden soll. Dies ist bereits dann der Fall, wenn die Zeile zum Drag-and-drop angefasst wird, und demzufolge auch dann, wenn sie nur in eine andere Anwendung gezogen werden wird.

Das Ereignis tritt nicht ein, wenn eine Markierung aufgehoben wird. Das Ereignis tritt mehrfach ein, wenn mehrere Zeilen gleichzeitig markiert werden. Das Ereignis tritt nicht ein, wenn nach einer Tabellenspalte sortiert wird (Klick auf die Spaltenüberschrift)

Bedeutung des Exit-Status

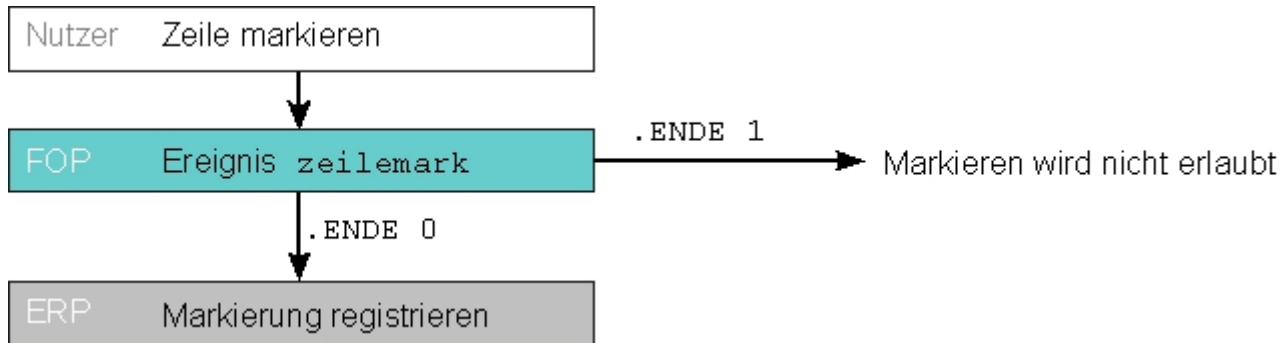
Exit-Status 0: Das Markieren wird erlaubt.

Exit-Status 1: Das Markieren wird nicht erlaubt.

Gesetzte Variablen

Es ist die Zeile geladen, die markiert werden soll, 'G|evtzeile' enthält ihre Nummer.

Ablauf beim Markieren einer Zeile:



Verschieben einer Zeile

Ablauf:

Schritt	Ebene	Aktion	Fehlerbehandlung
1	Benutzer	Zeile verschieben	
2	ERP	generelle Zulässigkeit prüfen	nicht erlaubt → Abbruch
3	FOP	Ereignis zeilebewpruef	. ENDE 1 → Abbruch
4	ERP	Dürfen markierte Zeilen verschoben werden?	nicht erlaubt → Abbruch
5	FOP	Ereignis zeilebewvor	. ENDE 1 → Abbruch
6	ERP	Dürfen Zeilen an gewünschte Position verschoben werden?	nicht erlaubt → Abbruch
7	ERP	Zeilenfunktion auslösen	
8	FOP	Ereignis zeilebewnach	

Für das EFOP **Zeile verschieben vor** der Abas Behandlung gilt:

- Die Ereignisse **zeilebewpruef** und **zeilebewvor** treten ein, bevor Zeilen ihre Position ändern. Dies geschieht z.B. beim Sortieren nach einer Tabellenspalte, Drag-and-drop, Markierte Zeilen verschieben. Die Behandlung im Kern hat noch nicht stattgefunden.
- Bedeutung des Exit-Status:

0	OK
1	Das Bewegen der Zeilen kann verboten werden.

Die einfache Angabe **.fehler** verursacht die Standardfehlermeldung: Zeilenvertauschung nicht zulässig.

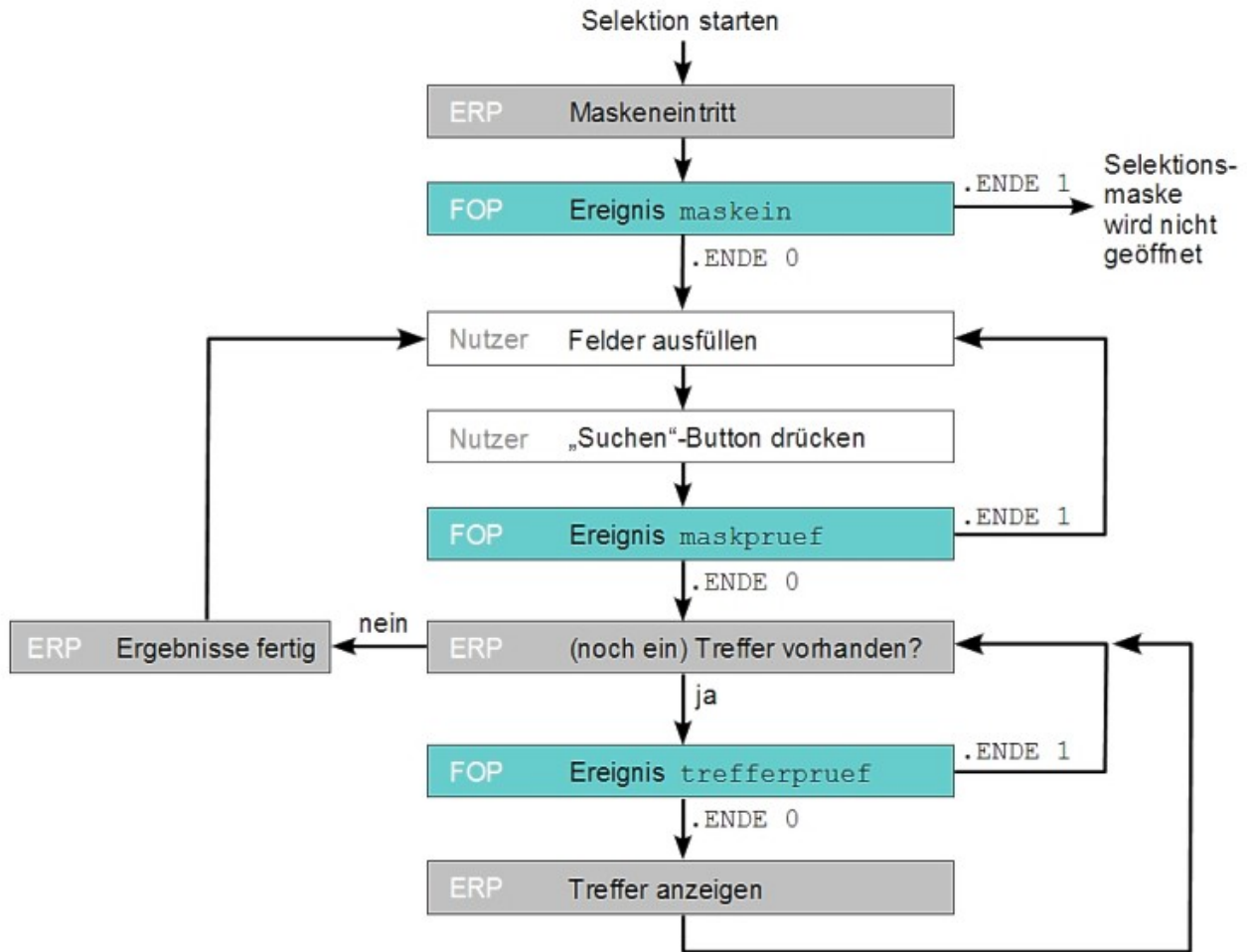
- Beim Ereignis **zeilebewpruef** wird für jede markierte Zeile geprüft, ob sie verschoben werden darf. Hier ist die Ereignisart **'G|evtart' zeilebewpruef**. **'G|evtzeile'** beschreibt die Nummer der zu prüfenden Zeile. Wenn das EFOP einen Fehler liefert, dürfen die Zeilen nicht verschoben werden.
- Das nächste Ereignis, an dem ein EFOP eingefügt werden kann, ist **zeilebewvor**. Hier wird geprüft, ob Zeilen an die gewünschte Position verschoben werden dürfen. Die Ereignisart (**'G|evtart'**) ist **zeilebewvor**. **'G|evtzeile'** enthält ihre Nummer der Zeile, vor die eingefügt werden soll.
- Das Ereignis weist folgende Besonderheiten auf:
 - Es sollte nur für die Prüfung der Zulässigkeit verwendet werden. Andere Aktionen können beim zugehörigen Nach-Ereignis ausgeführt werden.
 - Wie bei allen Ereignissen mit einem Nachfolgeereignis kann bei diesem Ereignis der Cursor nicht positioniert werden.
 - Bei diesem Ereignis dürfen Zeilenoperationen (Einfügen, Löschen) nur beim Sortieren von Zeilen nach einer Tabellenspalte ausgeführt werden, d.h. wenn die Variable **'G|evttaste'** den Wert Kalkulieren hat.

Für das EFOP **Zeile verschieben nach** der Abas Behandlung gilt:

- Das Ereignis tritt ein, nachdem Zeilen ihre Position geändert haben. Dies geschieht z.B. beim Sortieren nach einer Tabellenspalte, Drag-and-drop, Markierte Zeilen verschieben. Die Behandlung im Kern hat stattgefunden.
- Der Exit-Status hat keine Bedeutung. Das Programm sollte mit **.ende** oder **.ende 0** verlassen werden.
- Die Ereignisart (**'G|evtart'**) ist **zeilebewnach**.
- Es ist die Zeile geladen, auf deren Position eingefügt wurde. **'G|evtzeile'** enthält ihre Nummer. Beim Sortieren von Zeilen nach einer Tabellenspalte ist die aktuelle Zeile immer 1. Die Variable **'G|evttaste'** hat dann den Wert "kalkulieren", während sie beim Zeilenvertauschen den Wert "*" hat. Auf diese Weise lässt sich das Sortierereignis erkennen.
- Mit diesem Ereignis können Farben neu gesetzt werden.

Selektionsergebnis

Beim Selektionsergebnis sieht der Ablauf wie folgt aus:



Ein Selektionseignis entsteht wie folgt: Bei jedem "Treffer" in einer Selektion, d.h. für jeden Datensatz, der die Selektionskriterien erfüllt, tritt ein **trefferpruef**-Ereignis ein. Das Konfigurieren des **trefferpruef**-FOPs ist im Kapitel [Selektionssyntax](#) beschrieben.

Der gefundene Datensatz befindet sich im [Festplattenzustand](#) im Puffer M, bei einer Zeilenselektion ist die Variable [selzeile](#) gesetzt. Seine Felder dürfen geändert werden; der Benutzer sieht dann den geänderten Wert. Für benutzerdefinierte [Skipfelder](#) ist das sehr hilfreich. Bei [Festplattenfeldern](#) ist dies zumeist nicht sinnvoll, ist aber für eventuelle Sonderfälle dennoch zulässig.

Ein Exit-Status 0 bedeutet, dass der Datensatz akzeptiert wird und bei einer interaktiven Selektion auch angezeigt wird. Ein Exit-Status 1 bedeutet, dass der Datensatz ausgelassen wird.

In den Selektions-EFOPs kann auf den [A-Puffer](#) zugegriffen werden. Wird in einer Maske eine Objektauswahl über ein Verweisfeld geöffnet, so steht im A-Puffer das Vaterobjekt zur Verfügung.

Einschränkung



Da EFOPs nicht aus einem FOP heraus aufgerufen werden können, sind bei einer Selektion mit dem **.hole**- bzw. **.lade**-Kommando keine EFOPs möglich. Die Selektion läuft dort so, als ob keine EFOPs im Rohling definiert worden wären. Bei einer Selektion über die EDP-Schnittstelle erfolgt hingegen der Aufruf der Selektions-EFOPs.

Diese Einschränkung macht die Verwendung von [@trefferpruef](#) in FO-Programmen und

beim Druck des Ergebnisses der Objektauswahl unmöglich.



Bei einer großen Treffermenge kann ein solches EFOP die Performance deutlich verschlechtern.

Spezielle, generierte Ereignisse

Außer den bereits genannten Ereignissen werden an weiteren Stellen aus dem Kern EFOPs gestartet. Sie dienen zum Auslösen spezieller Funktionen. Für diese EFOPs sind drei Ereignisarten reserviert:

druck	Wird beim Drucken ausgelöst. Dieses Ereignis ist z.B. in einem Datengenerier-FOP gesetzt.
kernbutton	Wird an diversen Stellen, z.B. im Drucken beim Generieren von Layouts ausgelöst.
kernmaskaus	Wird z.B. beim Generieren von Masken und Layouts im Maskenaustritt eines Infosystems ausgelöst.

Die zugehörigen EFOPs gehören zum Lieferumfang und sind nicht anpassbar, es sei denn dies ist für eine bestimmte Funktionalität explizit definiert. Lediglich das Ereignis **druck** ist in Datengenerier-FOPs sichtbar.

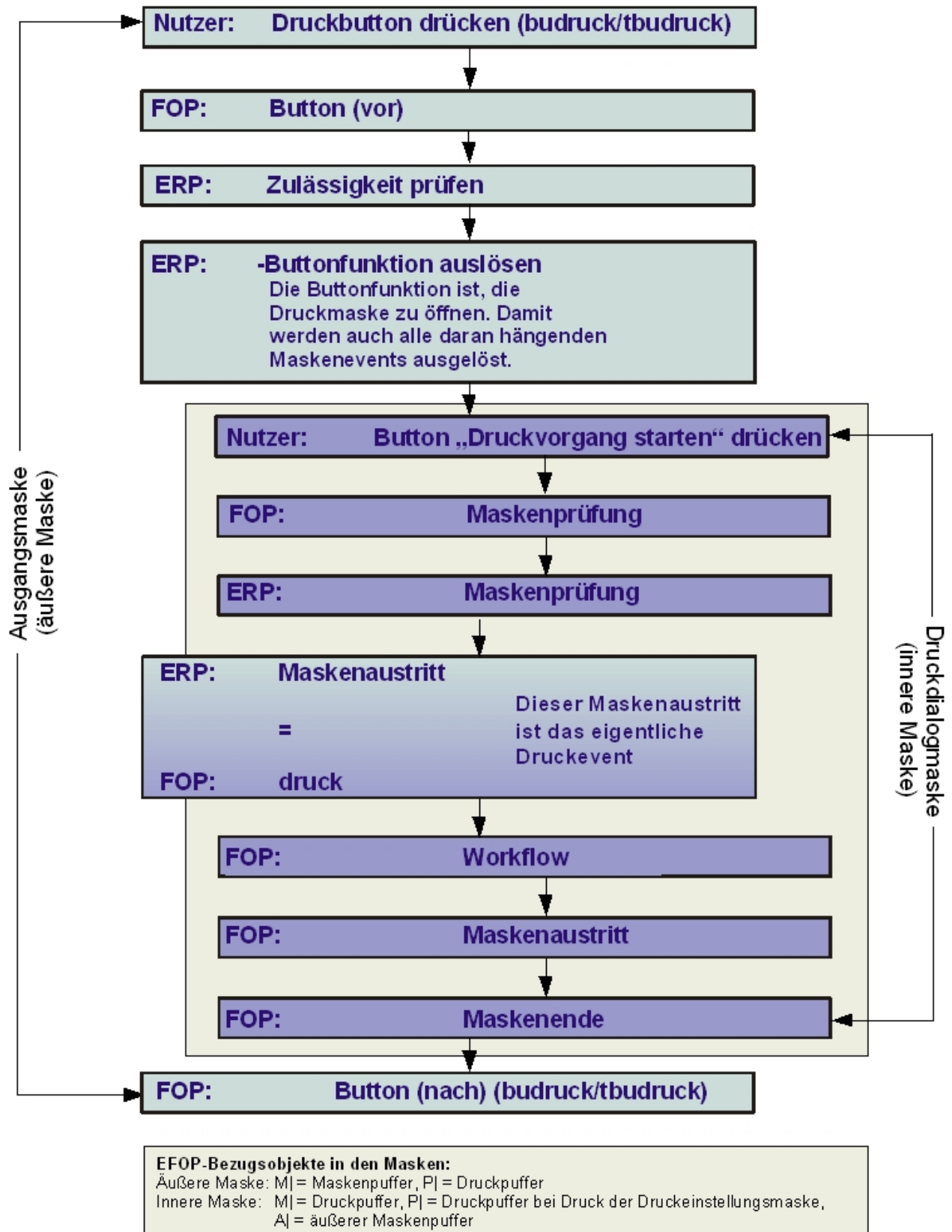
Button buinfosys Der Kern generiert hier ein Ereignis **kernbutton** mit der festgeschriebenen Ereignisbehandlung is/FOP.BUINFOSYS. Diese können Sie aber unterbinden, indem Sie beim Ereignis **buttonvor** für diesen Button ein EFOP konfigurieren, das einen Exit-Status 1 hat.

Alternativ können Sie zusätzlich zur Behandlung im Kern ein eigenes **buttonnach**-FOP konfigurieren. Dieses wird *nach* der Behandlung im Kern aufgerufen.

Druckereignis

Das Druck-FOP wird beim Drucken ausgelöst.

Ablauf beim Druckevent



Namenskonventionen für EFOPs

Programmnamen für EFOPs müssen den beschriebenen Konventionen folgen. Bestandteil der Programmnamen sind u.a. Kürzel für Ereignisse.

Ereigniskürzel

Kürzel	Ereignis
SE	Maskeneintritt
SV	Maskenprüfung
SX	Maskenaustritt
SC	Maskenabbruch
SEE	Maskenende
FF	Feld ausgefüllt
FV	Feldprüfung
FX	Feldaustritt
BB	Button before
BA	Button after
RDB	Zeile löschen vor
RDA	Zeile löschen danach
RIB	Zeile einfügen vor
RIA	Zeile einfügen danach
RH	Zeile markieren
RMV	Zeile bewegen Prüfung
RMB	Zeile bewegen vor
RMA	Zeile bewegen danach

Programmnamen für EFOPs in Datenbanken

Programmnamen für EFOPs zur Verwendung in Datenbanken müssen folgende Struktur besitzen:

SXXXX.Datenbank.Gruppe[.Feldname].Ereigniskürzel.Dateikennung

Es bedeuten:

Bestandteil	Bedeutung
S	Konstante für Screen (Maske)
XXXX	4-stellige Maskennummer mit führenden Nullen
Datenbank	Datenbanknummer 3-stellig
Gruppe	Gruppennummer 2-stellig
Feldname	Name des Feldes, mit dem das EFOP verbunden ist (optional)

Bestandteil	Bedeutung
Ereigniskürzel	Abkürzung des Ereignisses
Dateikennung	.F0 oder .F02

Beispiel: Feldaustritt

Ein EFOP, das mit dem Ereignis **Feldaustritt** des Feldes **ellex** in der **Maske 2** gekoppelt werden soll:
S0002.002.01.ELEX.FX.F0

Beispiel: Maskeneintritt

Ein EFOP, das mit dem Ereignis **Maskeneintritt** in der **Maske 2** gekoppelt werden soll:
S0002.002.01.SE.F0

Bei gruppenübergreifenden EFOPs wird die Gruppennummer durch XX ersetzt. Bei datenbankübergreifenden EFOPs wird die Datenbanknummer durch XXX ersetzt.

Programmnamen für EFOPs in Infosystemen

Programmnamen für EFOPs zur Verwendung in Infosystemen müssen folgende Struktur besitzen:

IS.Infosystem.Gruppe[.Feldname].Ereigniskürzel.Dateikennung

Es bedeuten:

Bestandteil	Bedeutung
IS	Kennung für Infosystem
Infosystem	Suchwort des Infosystems
Feldname	Name des Feldes, mit dem das EFOP verbunden ist (optional)
Ereigniskürzel	Abkürzung des Ereignisses
Dateikennung	.F0 oder .F02

Beispiel: Buttonnach

Ein EFOP, das mit dem Ereignis **Buttonnach** des Startbuttons des Infosystems mit dem Suchwort **Test** gekoppelt werden soll:
IS.TEST.BSTART.BA.F02



Unterprogramme müssen im Programmkopf mit **include** angegeben werden, damit sie beim Exportieren des Infosystems berücksichtigt werden.

Programmnamen, die von diesem Schema abweichen, müssen aussagekräftig sein.

Konfigurieren von EFOPs

Die Vorgehensweise bei Konfigurieren eines EFOPs für ein bestimmtes Ereignis hängt vom Typ der jeweiligen Anwendung ab.

Konfigurieren von EFOPs für Selektionsleisten

EFOPs werden durch [Selektionsoptionen](#) oder in Feldern des zugrunde liegenden Rohlings festgelegt. Siehe auch [Selektionsereignis](#).

Für Selektionsleisten wird bisher keine Notwendigkeit gesehen, mehrere EFOPs pro Ereignis zu ermöglichen. Hier ist weiterhin nur eines möglich. Abhilfe: Verteiler-EFOP schreiben (Verteiler-EFOPs müssen in Java geschrieben werden, sobald eines der EFOPs in Java geschrieben ist). Allerdings beeinflussen sich die im Verteiler aufgerufenen EFOPs gegenseitig.

Konfigurieren von EFOPs für Infosysteme

EFOPs werden in Feldern der Maske zur [Einrichtung des Infosystems](#) festgelegt.

Für Infosysteme wird bisher keine Notwendigkeit gesehen, mehrere EFOPs pro Ereignis zu ermöglichen. Hier ist weiterhin nur eines möglich. Abhilfe: Verteiler-EFOP schreiben (Verteiler-EFOPs müssen in Java geschrieben werden, sobald eines der EFOPs in Java geschrieben ist). Allerdings beeinflussen sich die im Verteiler aufgerufenen EFOPs gegenseitig.

Konfigurieren von EFOPs für Datenbankmasken

Wenn Sie nur die aktuell konfigurierten EFOPs ermitteln oder anschauen wollen, können Sie dies am schnellsten mit dem [Infosystem FO-Konfigurationsprüfung](#) erledigen. Falls Sie die EFOPs auch ändern wollen, ist der bequemste Weg über das Kontextmenü der betreffenden Maske (Rechtsklick), Unterpunkt *FOP > alle FOPs zeigen/bearbeiten*.

Falls Sie EFOPs konfigurieren wollen, stehen zwei Möglichkeiten zur Verfügung:

[Menügesteuertes Konfigurieren](#)

[Einträge in der Konfigurationsdatei](#)

Menügesteuertes Konfigurieren

Einschalten von EFOPs

Beim menügesteuerten Konfigurieren wird das FOP an Ort und Stelle angehängt. Gehen Sie wie folgt vor:

GUI	ASCII
1. Rufen Sie die Maske im gewünschten Modus auf. 2. Klicken Sie mit der rechten Maustaste in das Feld, an das Sie ein FOP anhängen wollen. 3. Wählen Sie den Menüpunkt <i>FOP</i>, den Unterpunkt <i>FOP anschalten</i> und das Ereignis, mit dem Sie das FOP verbinden wollen. 4. Geben Sie den Namen des Programms, das Sie mit dem Ereignis verknüpfen wollen, in das sich öffnende Fenster ein. Eine Leereingabe ist hier unzulässig. Wenn Sie - anstatt ein Programm auszuführen - den Direktmodus auf Entwicklungsmandanten benötigen, können Sie den Programmnamen FOP.EIN eintragen. In einer Produktivumgebung ist es aus Sicherheitsgründen nicht ratsam automatisch in den Direktmodus umzuschalten.  5. Bestätigen Sie die folgende Meldung "FOP anschalten?". 6. Wenn Sie ein weiteres FOP zu diesem Ereignis anschalten wollen, wiederholen Sie die Schritte ab Punkt 2.	1. Rufen Sie die Maske mit dem gewünschten Kommando auf. 2. Achten Sie darauf, dass der Cursor auf dem gewünschten Feld steht, wenn Sie ein FOP an dieses Feld anhängen wollen. 3. Betätigen Sie <Do> <Hilfe>. 4. Wählen Sie im Menü "Flexible Oberfläche" die Option <i>FOP anschalten</i>. 5. Wählen Sie im folgenden Menü das Ereignis, mit dem Sie das FOP verbinden wollen. 6. Geben Sie den Namen des Programms, das Sie mit dem Ereignis verknüpfen wollen, in das sich öffnende Fenster ein. Eine Leereingabe ist hier unzulässig. 7. Bestätigen Sie die folgende Meldung "FOP anschalten?" mit <Return>. Parallel zur Meldung werden Maske, Kommando, (Feld), Ereignis und der Name des FOP angezeigt. Um die Anbindung zu widerrufen, Geben Sie <N> <Return> ein. 8. Wenn Sie ein weiteres FOP zu diesem Ereignis anschalten wollen, wiederholen Sie die Schritte ab Punkt 2.
Funktionstastenergebnisse stehen in der GUI nicht zur Verfügung.	Funktionstastenergebnisse: Wenn Sie ein FOP durch Betätigung einer Funktionstaste starten wollen, drücken Sie diese. Es erscheint ein weiteres Menü, in dem Sie festlegen, ob das FOP nur bei Betätigung der Taste auf dem Cursorfeld, unabhängig von einem Maskenfeld, nur im Kopfteil der Maske oder nur in einer Tabelle gestartet werden soll.
Buttonereignisse: Rechte Maustaste auf den Button, Menüpunkt <i>FOP</i>, Menüpunkt <i>FOP anschalten</i>, Menüpunkt <i>Button</i>.	Buttonereignisse: Steht der Cursor auf einem Buttonfeld, lassen sich über die Konfigurationsmenüs EFOPs für Buttonereignisse dieses Feldes (<Do> <Hilfe>) an- und abschalten.

GUI	ASCII
maskenglobale Buttons: Ereignisse für maskenglobale Buttons lassen sich über jedes beliebige Feld konfigurieren mit Rechtsklick, Menüpunkt <i>FOP</i>, Menüpunkt <i>FOP anschalten</i>, Menüpunkt <i>globale Buttons</i>.	maskenglobale Buttons: Ereignisse für maskenglobale Buttons (das sind solche, die mit <Do> <Do> ausgelöst werden) lassen sich in Nicht-Button-Feldern konfigurieren.
Liste der vorhandenen FOPs: Die Liste aller bereits vorhandenen FOPs erreichen Sie über jedes beliebige Feld mit Rechtsklick, Menüpunkt <i>FOP</i>, Menüpunkt <i>alle FOPs zeigen/bearbeiten</i>. Wenn Sie eine Zeile auswählen und mit OK bestätigen, wird das entsprechende FOP im Editor geöffnet.	Liste der vorhandenen FOPs: Über das Menü "Flexible Oberfläche" können Sie sich alle bereits vorhandenen FOPs auflisten zu lassen. Wenn Sie eine Zeile auswählen, wird das entsprechende FOP im Editor geöffnet.
Neue EFOPs: Neu konfigurierte EFOPs werden für andere Arbeitsplätze erst nach einer der folgenden Aktionen wirksam: • neues Einloggen • in danach geöffneten Fenstern • in bereits geöffneten Fenstern nach Rechtsklick, Menüpunkt <i>FOP</i>, Menüpunkt <i>FOP-Konfiguration neu laden</i> • in Masken nach erfolgreicher Ausführung eines der Menüpunkte <i>FOP anschalten</i>, <i>FOP ausschalten</i>, <i>alle FOPs zeigen/bearbeiten</i>	Neue EFOPs: Im Menü "Flexible Oberfläche" finden Sie die erweiterten Maskeninformation. Neu konfigurierte EFOPs werden für andere Arbeitsplätze erst nach einer der folgenden Aktionen wirksam: • neues Einloggen • erfolgreiche Ausführung der Kommandos fop - und anschließend fop + • in Masken nach <Do> <Hilfe> erfolgreiche Ausführung eines der Menüpunkte <i>FOP anschalten</i>, <i>FOP ausschalten</i>, <i>alle FOPs zeigen/bearbeiten</i>

Wenn das FOP das erste zu dieser Maske ist, wird es in der Datei **fop.txt** hinten angefügt. Andernfalls wird es direkt vor den EFOPs dieser Maske eingetragen. Dies hat den Vorteil, dass die EFOPs einer Maske nicht über die Datei **fop.txt** verstreut werden, sondern immer zusammen stehen. Da ein neues EFOP einer Maske direkt vor allen bestehenden EFOPs einer Maske eingetragen wird, wird es im Falle mehrerer passender EFOPs auch als erstes ausgeführt. Wenn (im Ausnahmefall) die Reihenfolge der EFOPs bei einem Ereignis eine Rolle spielt, muss das eingetragene EFOP anschließend [per Hand an die gewünschte Stelle](#) geschoben werden. Beachten Sie dabei, dass das Stop-Zeichen (**[S]** in Spalte 6 der Datei **fop.txt**) die Suche nach weiteren EFOPs zu diesem Ereignis stoppt.

Ausschalten von EFOPs

GUI	ASCII
1. Rufen Sie die Maske im gewünschten Modus auf. 2. Klicken Sie mit der rechten Maustaste in das Feld, von dem ein FOP abgekoppelt werden soll. 3. Wählen Sie den Menüpunkt <i>FOP</i>, den Unterpunkt <i>FOP ausschalten</i> und das Ereignis. 4. Wählen Sie im Untermenü das Ereignis, von dem das FOP abgekoppelt werden soll.	1. Rufen Sie die Maske mit dem gewünschten Kommando auf. 2. Achten Sie darauf, dass der Cursor auf dem gewünschten Feld steht, wenn Sie ein FOP an dieses Feld anhängen wollen. 3. Betätigen Sie <Do> <Hilfe>. 4. Wählen Sie im Menü "Flexible Oberfläche" die Option <i>FOP ausschalten</i>. 5. Wählen Sie im folgenden Menü das Ereignis, von dem das FOP abgekoppelt werden soll.

Mit diesem Befehl wird nur ein EFOP ausgeschaltet, und zwar das erste passende. In einem Fenster wird zuerst die Liste aller passenden FOPs ausgegeben und danach die Bestätigung zum Ausschalten des ersten FOPs dieser Liste verlangt.

Wenn mehrere oder alle EFOPs für ein Ereignis ausgeschaltet werden sollen, muss dieser Vorgang wiederholt werden. Alternativ kann auch [die Konfigurationsdatei fop.txt](#) editiert werden. Da ein neues EFOP einer Maske direkt vor allen bestehenden EFOPs dieser Maske eingetragen wird, wird es demzufolge als erstes wieder gelöscht. Somit kann ein versehentlich konfiguriertes EFOP unmittelbar danach leicht wieder gelöscht werden.

Einträge in der Konfigurationsdatei

Die im Mandantenverzeichnis liegende Datei [fop.txt](#) legt in Form einer Ereignistabelle die Zuordnung von FO-Programmen zu Ereignissen fest. Wenn Sie das Verfahren "Menügesteuertes Konfigurieren" verwenden, werden automatisch Einträge in dieser Datei erzeugt und Sie brauchen die im Folgenden beschriebene Syntax nicht zu kennen. Die Ereignistabelle ist etwa folgendermaßen strukturiert:

Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5	Spalte 6		Spalte 7
Maske	Kommando/Modus	Ereignis	Funktionstaste	Feld	Feldart (Kopf/Tabelle)		FOP-Name
0	Neu	maskei n	-	-	-		FOP.KUNDE
5	Ändern	maskau s	-	-	-		FOP.BUCHUN G
1	*	maskau s	-		-		FOP.LIEFER
26	*	maskau s	-		-		FOP.LIEFER
2	*	feldau s	-	bem	K	[C]	FOP.PRUEF
2	*	feldau s	-	bem	K		FOP.PRUEF2

Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5	Spalte 6	Spalte 7
36	Ändern	fkttas te	kalkul	*	K	FOP.INFO

Eine Zeile kann 7 oder 8 Einträge haben. Die Spalte mit [C] oder [S] kann fehlen.

Der Stern steht für "beliebig". So wird z.B. bei Austritt aus den Masken 1 und 26 der Tabelle **FOP.LIEFER** unabhängig davon aufgerufen, mit welchem Kommando der Maskeneintritt erfolgte. Das FOP wird wie mit dem Kommando `<Text> 'FOP' <Zeigen>` gestartet. Alle laut Passwortdefinitionen verfügbaren Verzeichnisse werden nach dem FOP durchsucht.

Verwenden Sie nicht Einträge, die nur oder fast nur aus Sternen bestehen, wie z.B. **32 ***
*** * * * MYFOPHANDLER**



Das EFOP **MYFOPHANDLER** wird bei jedem Ereignis aufgerufen, obwohl es tatsächlich nur für wenige Ereignisse eine Behandlung hat. Das kann sich negativ auf die Performance auswirken.

Bei Ereignissen, die sich auf die gesamte Maske beziehen, werden die Spalten 4 bis 6 der Ereignistabelle nicht ausgewertet. Der Stern in diesen Feldern ist jedoch als Platzhalter notwendig. Als Alternative zum Stern wird auch das Minus toleriert.

Für Feldereignisse müssen sowohl Spalte 5, Spalte 6 als auch Spalte 7 belegt werden.

Jedem Ereignis können mehrere FOPs zugeordnet werden. Haben Sie zu einem Ereignis mehrere Zeilen angelegt, können Sie mit der Angabe in Spalte 6 festlegen, ob weitere Zeilen für dieses Ereignis ausgewertet werden sollen oder nicht. Die EFOPs werden in der Reihenfolge ausgeführt, in der sie in der Datei **fop.txt** eingetragen sind. Fehlt diese Angabe in Spalte 6, wird nur die erste Zeile ausgewertet. Mögliche Werte sind:

[S]	Stop: nicht nach weiteren EFOPs für dieses Ereignis suchen (Default)
[C]	Continue: nach weiteren EFOPs für dieses Ereignis suchen

Die Angabe wird daran erkannt, dass sie in eckigen Klammern steht (die darauf folgende Angabe "FOP-Datei" kann keine eckigen Klammern enthalten, da dies kein zulässiger FOP-Name wäre).

Beispiel: Eine Zeile in der Datei fop.txt, bei der nach weiteren EFOPs gesucht wird

```
# mailto-Button für Kunde Lieferant Sachbearbeiter Mitarbeiter Adressen
25 * buttonnach * mailto K [C] FOP.SENDTO
```

Nach einer manuellen Änderung der Ereignistabelle sollte in jedem Fall eine dieser Aktionen durchgeführt werden, da sie auch eine Syntaxüberprüfung auslösen. Diese Syntaxüberprüfung findet auch bei jedem Programmstart statt (GUI: Öffnen einer Maske). Fehler werden mit der Zeilen- und der Eintragsnummer gemeldet. Wir empfehlen aus Gründen der Systematik, den FOP-Namen das Suffix ".FOP" anzuhängen.

Konfigurieren von EFOPs für maskenlose Kommandos

Maskenlose Kommandos wie z.B. die Übertragen-Kommandos im Einkauf und Verkauf werden in der Konfigurationsdatei analog zu den Kommandos mit Maske eingetragen, wobei als Maskennummer die zu den bearbeitenden Objekten "gehörende" Maske herangezogen wird; also diejenige Maske, die beim Editieren dieser Objekte verwendet wird. Wenn Sie sich über die richtigen Einträge in der Konfigurationsdatei unsicher sind, können Sie den Trick verwenden, das Kommando aufzurufen und sich einen [Trace aller Ereignisse](#) anzeigen zu lassen.

Dann werden alle auftretenden EFOPs wie auch Objekt, Kommando usw. gemeldet, so dass Sie zugleich erfahren, welche Einträge in der Datei **fop.txt** (manuell) vorzunehmen sind. Eventuell wird aber gar nichts gemeldet, denn Ereignisse gibt es nur für maskenlose Kommandos, für die ein entsprechender Anschluss im Abas Kern geschaffen wurde.

Bei welchen maskenlosen Kommandos können EFOPs eingehängt werden?

- Übertragen-Kommandos im Einkauf und Verkauf
- Löschen-Kommandos für Stammdaten
- Ein- und Verkauf: Lieferscheine und Rechnungen gemäß Vorschlagsliste erzeugen
- <Betr-Auftrag> <Übertragen> (Restmengenbuchungen bezogen auf einen Betriebsauftrag oder Arbeitsschein durchführen, überträgt Rückmeldung in Betriebsauftrag oder Arbeitsschein)
- Rückmeldebelege buchen

Die Liste unterliegt Erweiterungen.

Kommandoänderung während des Editierens

In Anwendungen, die ein Schreiben und Wiederladen ermöglichen sowie in der [editierbaren Fertigungsliste](#) kann sich das Ausgangskommando ändern.

<Verkauf> <Neu> [Auftrag]	<Neu>
Erste Position anlegen	<Neu>
Auftragsfertigungsliste editieren	<Neu>
Zurück zum Ausgangsauftrag	<Ändern>
Zweite Position anlegen	<Ändern>
Auftragsfertigungsliste editieren	<Ändern>

Dieser Umstand muss bei der Konfigurierung von EFOPs beachtet werden. Wenn beispielsweise in der editierbaren Auftragsfertigungsliste ein EFOP nur für das Kommando <Neu> konfiguriert wird, wird es nach dem Absteigen und der Rückkehr zum Ausgangsauftrag nicht mehr ausgeführt.

Infosystem FO-Konfigurationsprüfung

Das Infosystem 10087 FOPTXTP, Arbeitsbereich sy für System, hat folgende Funktion:

- Prüfung der **fop.txt** auf Mehrfachbelegungen der Ereignisse.

Ein Ereignis ist mehrfach belegt, wenn die Inhalte der Felder Maske, Kommando, Ereignis, Feld, Feldart identisch sind. Das kann auch bei der Verwendung eines Sterns der Fall sein.



Der Eintrag * ist ein Platzhalter für einen beliebigen Inhalt.

Die Steuerzeichen [C] oder [S] werden bei der Prüfung nicht berücksichtigt.

Bedienung und Ausgabe

Mit Klick auf Start wird die komplette **fop.txt** dargestellt. Blöcke sind farblich kenntlich gemacht. Wenn **Mehrfachbelegungen** vorkommen, wird deren Anzahl im gleichnamigen Feld ausgegeben und die entsprechenden Zeilen farblich markiert. Wenn Sie die Mehrfachbelegung für eine bestimmte Maske prüfen möchten, tragen Sie die Maskennummer im Feld **Maske** ein.

In der Tabelle werden die entsprechenden Informationen ausgegeben.

Maskenbezogene Kommandos

.CURSOR / .CURSOR - Cursorsteuerung

.CURSOR	.CURSOR
deutsch	englisch
.CURSOR 'Feldname' ['Zeilennummer']	.CURSOR 'field name' ['line number']
Stellt den Cursor auf das angegebene Feld	

Das Kommando **.cursor** steuert den Cursor auf ein beliebiges Feld, wobei **'Zeilennummer'** eine Zahl und **'Feldname'** die Variable des Feldes ist, auf das der Cursor gestellt werden soll. Existiert die Variable nicht, erscheint die Fehlermeldung "Variable nicht gefunden". Der Name der Variablen kann z.B. über die **Maskeninfo** ermittelt werden.

Bei Zeilennummer 0 wird der Cursor in den Kopfteil positioniert. Fehlt die Zeilenangabe, wird er in die aktuelle Zeile, ggf. auch in den Kopfteil, positioniert. Wird eine zu große Zeilennummer angegeben, erscheint eine entsprechende Fehlermeldung und der Cursor wird auf der GUI nicht positioniert.

Wenn der Benutzer ein Feld ändert und das Feld durch einen Klick auf ein anderes Feld verlässt, dann wird auf das angeklickte Feld positioniert. Beim Feldaustritts-FOP wird dann das Kommando **.cursor** ignoriert. Wenn nach Änderung eines Feldes das Feld mit der Taste <Tabulator> verlassen wird, dann wird beim Feldaustritts-FOP das Kommando **.cursor** ausgeführt. **.cursor** wird generell nur beim letzten **Ereignis** einer Ereignisgruppe und nicht bei allen Ereignissen respektiert.

Das FO-Kommando **.cursor** wird von EDP vollständig unterstützt. Die Zeilennummer aus dem **.cursor**-Befehl wird als Editorstatus ROW sichtbar. Zusätzlich wird auch das Feld aus dem **.cursor**-Befehl als Editorstatus FIELD sichtbar. Unter folgenden Umständen wird das Feld zusätzlich im Kopf gesucht:

- Der Befehl wird bei geladener Tabellenzeile ausgeführt.
- Es war keine Zeile angegeben.

- Das Feld ist in der Tabelle nicht vorhanden.

Siehe auch [Steuerung des Dialogs durch FOPs](#).

Beispiel: Cursor auf ein bestimmtes Feld positionieren

Dieses Beispiel positioniert den Cursor in das Feld "Bemerkungen", das mit der Variablen **bem** verknüpft ist.

```
.cursor comments
```

In diesem Beispiel wird der Cursor in die dritte Zeile der Tabelle in das Feld **mge** gesetzt.

```
.cursor unitQty 3
```

Das Positionieren ist nur bei Ereignissen sinnvoll, die in einer Maske ausgelöst werden. Darüber hinaus ist die Positionierung nur beim letzten **Ereignis** einer Ereignisgruppe möglich, da sich andernfalls die Ereigniszeile oder das Ereignisfeld innerhalb einer Ereignisgruppe ändern kann. Das **buttonvor**-FOP ist z.B. nicht das letzte Ereignis seiner Gruppe (es wird **buttonnach** folgen), weshalb Sie dort nicht positionieren können. Wenn das **buttonvor**-FOP aber mit **.ende 1** (Fehlerstatus) endet, steht fest, dass kein weiteres EFOP aufgerufen wird, so dass die Positionierung durchgeführt wird.

Vom Positionieren sollte aus ergonomischen Gründen sparsam Gebrauch gemacht werden.



Besonderheit bei der grafischen Oberfläche

Bei der grafischen Oberfläche wird das mit **.cursor** angegebene Feld bei Feldereignissen nur bei TAB ausgewertet, nicht aber bei Mausklick. Bei Mausklick hat der Wunsch des Benutzers Vorrang.



Beim Ändern von Checkboxes und Comboboxen ist **.cursor** in keinem Fall wirksam.

.FEHLER / .ERROR - Fehlermeldung

.FEHLER	.ERROR
deutsch	englisch
.FEHLER	.ERROR
.FEHLER beliebiger Text	.ERROR any text
.FEHLER [-FELD feldname] [-ZEILE zeilennummer] [-NUMMER fehlernummer] [-MELDUNG meldung]	.ERROR [-FIELD field name] [-LINE line number] [-NUMBER error number] [-MESSAGE message]

.FEHLER	.ERROR
Dieses Kommando gibt Fehlermeldungen aus. Im FOP erfolgt die Ausgabe sofort. Im EFOP kann das .fehler-Kommando mehrfach ausgeführt werden. Jeder Aufruf von .fehler überschreibt die aktuellen Informationen. Die Ausgabe erfolgt am Programmende. .fehler beliebiger Text führt das FOP weiter aus. Nach der Rückkehr in die Maske wird in der Statuszeile der angegebene Text ausgegeben. .fehler -NUMMER gibt die der Nummer zugeordneten Meldung aus msg.cc.dic aus. Diese Ausgabe erfolgt immer in der Bediensprache. Der Vorteil von Fehlernummern ist, dass Übersetzungen aus msg.cc.dic verwendet werden. .fehler -NUMMER und .fehler beliebiger Text können kombiniert werden. .fehler initialisiert die mit .fehler erzeugten Informationen.  Die Optionen FELD und ZEILE können nur bei EFOPs verwendet werden.	

Beispiel: Fehlertext zu Fehlernummer ausgeben

```
.. displays error message 130 from msg.cc.dic
.error -NUMBER 130
```

```
.type int xiquantity
.type int ximaxQuantity
.assign U|ximaxQuantity = 10
.read "Insert quantity" U|xiquantity
.continue END          ? U|xiquantity <= U|ximaxQuantity
.error quantity: 'U|xiquantity' oversized, maximum quantity: 'U|ximaxQuantity'
!END
.end
```

Beispiel: EFOP: Maske Artikel - Feld Zeichnung/Norm

Der Cursor wird in das Feld **elemQty** positioniert und die Meldung **invalid elemQty** ausgegeben.

```
..!interpreter english noabbrev englvar
.. run program
.. error occurs
```



```
.error -field elemQty -line 3 -message invalid elemQty  
.end 1
```

Unabhängig davon, ob Sie **.ende** 0 oder **.ende** 1 verwenden, wird die mit **.fehler** definierte Positionierung des Cursors immer ausgeführt und die zugehörige Meldung ausgegeben.

Beispiel: EFOP: .error-Kommando mehrfach verwenden

Im EFOP werden beim Ereignis Maskenprüfung mehrere **.fehler**-Kommandos ausgeführt.

```
..!interpreter english noabbrev englvar  
.. run program  
.error -field zipCode -message zipCode invalid  
.. continue program  
.error -field descrOperLang -message descrOperLang invalid  
.. continue program  
.error -field phoneNo -message phoneNo invalid  
.end 1
```

Das zuletzt ausgeführte **.fehler**-Kommando definiert die Positionierung des Cursors und die Fehlerausgabe.

Beispiel: FOP mit .error-Kommando

```
..!interpreter english noabbrev englvar  
.. run program  
.error -number 131  
.. alternate  
.. .error -message invalid information  
continue program  
.end 1
```

Wird in einem FOP das **.fehler**-Kommando verwendet, dann wird die Fehlermeldung sofort ausgegeben.

Im Lader wird **.fehler** als Fehler gewertet, so dass das Laderprogramm abbricht. Vgl. daher auch das alternative Kommando **.hinweis**.

Die Nutzung des Kommandos **.fehler** ist einer Fehlerausgabe des EFOPs mittels des Kommandos **.box** meist überlegen, da keine FOP-Fenster über die Maske gelegt werden und die Meldung auch nach dem FOP-Ende stehen bleibt.

In EFOPs, die eine Benutzeraktion durch ihren Exit-Status verbieten können (z.B. im Feldprüfungs-FOP), sollte auch eine erklärende Fehlermeldung eingebaut werden. Wenn dies vergessen wird, bringt Abas die

Standardfehlermeldung "Aktion nicht erlaubt oder abgebrochen".

Das Kommando `.fehler` ohne Argumente, schaltet alle Fehlermeldungen, auch die Standardfehlermeldungen, ab.

Anführungszeichen in Textausgaben



Die Kommandos (`.hinweis`, `.fehler`, `.menue`, `.box` und `.println`) geben Text wie angegeben aus. Dies ist analog zum [Direktmodus](#). Dadurch können Sie Anführungszeichen ausgeben, z.B.:

`.fehler` Im Unterprogramm trat der Fehler "keine Netzverbindung" auf.

`.hinweis` Rechnungsabschlusspositionen wurden ergänzt.

.HINWEIS / .NOTE - Meldungen

.HINWEIS	.NOTE
deutsch	englisch
<code>.HINWEIS</code>	<code>.NOTE</code>
<code>.HINWEIS [-SOFORT] [-AKTUALISIEREN] [-DIAGNOSE] ('n')</code>	<code>.NOTE [-IMMEDIATELY] [-UPDATE] [-ERRORREPORT] ('n')</code>
<code>.HINWEIS [-SOFORT] [-AKTUALISIEREN] [-DIAGNOSE] 'beliebiger Text'</code>	<code>.NOTE [-IMMEDIATELY] [-UPDATE] [-ERRORREPORT] 'any text'</code>
<code>.hinweis</code> gibt einzeilige Meldungen von EFOPs aus. <code>.hinweis -sofort ('n')</code> oder <code>'beliebiger Text'</code> bewirkt die sofortige Ausgabe des Hinweises. <code>.hinweis -sofort</code> löscht den Inhalt der Statusleiste. <code>.hinweis -aktualisieren</code> zeigt auf der GUI alle Änderungen an Maskenfeldern sofort an; normalerweise werden diese erst am Ende des EFOPs angezeigt. <code>.hinweis -diagnose</code> erzeugt eine FOP-Administrationsmeldung.	

Auf der GUI erscheint eine Meldung in der Statusleiste.

Einzeilige Meldungen von EFOPs können nicht nur mit dem Kommando `.fehler` ausgegeben werden, sondern auch mit dem Kommando `.hinweis`. Dieses wird im [Datenbanklader](#) nicht als Fehler gewertet. Syntax und Verwendung sind analog dem Kommando `.fehler`.

Im Gegensatz zu `.fehler` kann für die GUI die sofortige Ausgabe des Hinweises erzwungen werden mit `.hinweis -sofort ('n')` oder `'beliebiger Text'`.



Der Text im Feld Status, der ausgegeben werden soll, darf nicht mit "(" beginnen. Fügen Sie gegebenenfalls vor die Klammer ein Leerzeichen ein.

```
.type text xttest ? F|defined(U|xttest) = G|false  
.formula U|xttest = " (43)"  
.note -immediately 'U|xttest'
```

```
.note -immediately information 1  
..  
.note -immediately information 2  
..  
.note -immediately
```

Damit wird während der Laufzeit des EFOPs mehrfach eine Statusmeldung ausgegeben. Am Ende des EFOPs wird die Statusleiste gelöscht.

Anführungszeichen in Textausgaben



Direkt Text ausgebende Kommandos (`.hinweis`, `.fehler`, `.menue`, `.box` und `.println`) stellen den Text genau so dar wie angegeben, analog dem [Direktmodus](#). So ist es auf einfache Weise möglich, Anführungszeichen auszugeben, z.B.:

```
.fehler Im Unterprogramm trat der Fehler "keine Netzverbindung"  
auf.
```

```
.hinweis Rechnungsabschlusspositionen wurden ergänzt.
```

.FARBE / .COLOUR - Setzen von Farben in Masken

.FARBE	.COLOUR
Deutsch	Englisch
.FARBE 'Optionen' [-FELD] 'Feldname' ['Zeilennummer'] .FARBE 'Optionen' [-ZEILE] 'Zeilennummer' .FARBE 'Optionen' -SPALTE 'Feldname' .FARBE 'Optionen' -TABELLE .FARBE 'Optionen' 'Statusbereich-Option' - Keine REST API-Unterstützung	.COLOUR 'options' [-FIELD] 'field name' ['line number'] .COLOUR 'options' [-LINE] 'line number' .COLOUR 'options' -COLUMN 'field name' .COLOUR 'options' -TABLE .COLOUR 'options' 'status area option' - No REST API support
Optionen -VORDERGRUND 'Farbe' -HINTERGRUND 'Farbe' -RESET	Options -FOREGROUND 'colour' -BACKGROUND 'colour' -RESET
Ermöglicht die Einstellung der Schriftfarbe (-vordergrund) und Hintergrundfarbe (-hintergrund) vom EFOP aus, sowie das Zurücksetzen der Farbe (-reset). Die Farbe muss als RGB-Wert angegeben werden, z.B. 255 0 0. [-feld] 'Feldname' ['Zeilennummer'] setzt die Farbe des Feldes in der angegebenen Zeile. Wenn die Zeilenangabe fehlt, wird 0 angenommen (Kopfteil). -zeile 'Zeilennummer' setzt die Farbe einer einzelnen Zeile. Zeile 0 ist der Kopfteil der Maske. -spalte 'Feldname' setzt die Farbe einer einzelnen Spalte in der Tabelle. -tabelle, nur in Verbindung mit der Option -reset verwendbar, setzt die Farbe für die ganze Tabelle zurück.	
 Das Kommando .farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] 'Statusbereich-Option' wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar. .farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] 'Statusbereich-Option' setzt die Farbe der GUI-Statusleiste.	

Überlagerungshierarchie bei Farben in der Tabelle

Im Tabellenteil einer Maske überlagern sich Farben nach festen Regeln:



- Die Zellfarbe hat Vorrang vor der Zeilenfarbe.
- Die Zeilenfarbe hat Vorrang vor der Spaltenfarbe.

- Trifft mehr als eine Farbe auf eine Zelle zu, gilt immer die jeweils spezifischere Einstellung.

Wenn Sie sowohl die Schriftfarbe als auch die Hintergrundfarbe einstellen wollen, dann müssen Sie das Kommando **.farbe** mit beiden Argumenten verwenden.

Verwenden Sie das Kommando mit nur einem Argument, wird für das nicht verwendete Argument die Standardeinstellung verwendet.

Wollen Sie die Farbe auf die Standardeinstellung zurücksetzen, müssen Sie explizit **-reset** angeben.



Eine Kombination von Farbangaben und **-reset** ist nicht möglich.

Beispiel: RGB-Farbwerte

Farbe	R	G	B
weiß	255	255	255
rot	255	0	0
grün	0	255	0
blau	0	0	255
gelb	255	255	0
schwarz	0	0	0

Mit **-reset** werden die Farben auf die Standardfarben zurückgesetzt. Wenn nach dem Speichern oder Verwerfen ein neuer Datensatz angezeigt wird, geschieht dies automatisch. **-reset** hat die gleiche Wirkung wie **-1 -1 -1**.

Das **Infosystem is/FARBEN** bietet die Möglichkeit, eine Palette benutzerdefinierter Farben für die Verwendung in FO-Programmen zu definieren.

Im Einzelnen können Sie folgendes tun:

Farbe eines Maskenfeldes setzen

Sie können von einem EFOP aus für ein Maskenfeld zwei Farben setzen:

- mit der Option **-vordergrund** die Schriftfarbe
- mit der Option **-hintergrund** die Feldfarbe
- mit der Option **-reset** Zurücksetzen der Feldfarbe und Schriftfarbe

Beispiel: Hintergrund- und Vordergrund-Farbe definiert setzen

Roten Hintergrund und weißen Vordergrund für das Feld **ofmge** in Zeile **U|xirowNo** der Rechnungsmaske des Verkaufs setzen.

```
.type int xirowNo
.formula U|xirowNo = 1
.colour -background 255 0 0 -foreground 255 255 255 -field openQtyTradeUnit 'U|xirowNo'
```

Die Funktion kann auf Wunsch abgeschaltet werden. Dazu gibt es unter den GUI-Optionen im Menüpunkt "Einstellungen" auf der Seite "Farben" im Abschnitt "Allgemein" das Feld "Anwendungsseitige Feldfarben verwenden". In diesem Abschnitt liegt auch das Feld "Negative Zahlenwerte farblich hervorheben".



Vorrang vor der Hervorhebung negativer Zahlenwerte

Die Farben, die von der Anwendung einschließlich ihrer EFOPs gesetzt werden, haben Vorrang vor der farblichen Hervorhebung von negativen Zahlenwerten. Ist die Textfarbe eines Feldes per EFOP gesetzt, wird ein negativer Zahlenwert in der GUI nicht mehr automatisch rot eingefärbt.



Sie können nur die Farbe von Eingabe- und Ankreuzfeldern, nicht von Buttons, und Auswahlfeldern festlegen.

Bei schreibgeschützten Feldern wird anders als beim Setzen der [Farbe für alle Felder einer Zeile](#) exakt die angegebene Farbe verwendet. Wenn Sie die Farbe für mehrere Felder setzen und für schreibgeschützte Felder eine etwas dunklere Farbe verwenden wollen, damit man diese Felder von beschreibbaren Feldern unterscheiden kann, dann müssen Sie die etwas dunklere Farbe beim Aufruf von [.farbe](#) angeben

Beispiel: Hintergrundfarbe abhängig vom Schreibschutz eines Feldes

Setzt die Hintergrundfarbe für zwei Felder. Wenn ein Feld schreibgeschützt ist, dann wird eine etwas dunklere Farbe verwendet.

```
.colour -background "243 246 188" -field unitQty 'M|currTabRow' ? F|modifiable(M|unitQty)
.colour -background "211 214 156" -field unitQty 'M|currTabRow' ? _F|modifiable(M|unitQty)
.colour -background "243 246 188" -field tradeUnit 'M|currTabRow' ? F|modifiable(
M|tradeUnit)
.colour -background "211 214 156" -field tradeUnit 'M|currTabRow' ? _F|modifiable(
M|tradeUnit)
```



Oft soll eine bestimmte Farbe einen bestimmten Zustand des Datensatzes anzeigen, z.B. ein roter Feldhintergrund einen Wert, der unter oder über einem Soll-Wert liegt. Dann reicht ein einziges Maskeneintritts-FOP, das die Farbe beim Laden eines Datensatzes in die Maske setzt, nicht aus. Auch wenn das Feld durch eine Benutzereingabe indirekt geändert wird, muss die Farbe eventuell neu gesetzt werden (Feldaustritts-FOP).

Farbe für alle Felder einer Zeile oder des Kopfteils setzen

`.farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] -zeile 'Zeilennummer'` oder

`.farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] 'Zeilennummer'` setzt die Farbe für alle Felder einer Tabellenzeile oder des Kopfteils.

`.farbe -reset 'Zeilennummer'` setzt die Farbe der angegebenen Tabellenzeile oder des Kopfteils auf die Defaultfarbe zurück. Beim Reset für den gesamten Kopfteil werden alle Kopffelder zurückgesetzt, unabhängig davon, ob sie einzeln oder gemeinsam eingefärbt wurden.



Zeile 0 ist der Kopfteil der Maske

Verwenden Sie die Option `-zeile` mit der Angabe `'Zeilennummer' = 0`, um die Farbe für alle Kopffelder einer Maske in einem Schritt zu setzen oder zurückzusetzen.

Beispiel: Hintergrund für Zeile ändern

Setzt die Hintergrundfarbe für die Zeile `U|xirowNo`

```
.type int xirowNo
.formula U|xirowNo = 1
.colour -background 227 224 255 -line 'U|xirowNo'
```

Bei schreibgeschützten Feldern wird automatisch eine etwas dunklere Farbe verwendet, damit diese Felder von beschreibbaren Feldern unterschieden werden können. Setzen Sie z.B. die Farbe Weiß als Hintergrundfarbe für eine Zeile, werden alle schreibgeschützten Felder in dieser Zeile mit einem hellgrauen Hintergrund angezeigt.

Wenn Sie von einer Maske aus eine weitere Maske öffnen, dann können Sie nur für die zuletzt geöffnete Maske die Farbe für alle Felder einer Zeile setzen.

Öffnen Sie von einer Bestellung aus die Maske *Drucken*, können Sie nur die Farbe für eine Zeile in der Maske *Drucken* setzen, nicht aber für eine Zeile in der Bestellung.

Farbe für alle Felder einer Spalte setzen

`.farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] -spalte 'Feldname'`

`.farbe -reset -spalte 'Feldname'`

setzt die Farbe für alle Felder einer Tabellenspalte bzw. setzt sie auf die Defaultfarben zurück.

Beispiel: Hintergrund für Spalte 'elemQty' in der Artikelmaske ändern

Setzt die Hintergrundfarbe für die Spalte **elemQty**

```
.colour -background 227 224 255 -column elemQty
```

Bei schreibgeschützten Feldern wird automatisch eine etwas dunklere Farbe verwendet, damit man diese Felder von beschreibbaren Feldern unterscheiden kann. Wenn man z.B. Weiß als Hintergrundfarbe setzt, werden alle schreibgeschützten Felder in dieser Zeile mit einem hellgrauen Hintergrund angezeigt.

Falls außerdem für einzelne Felder dieser Zeile eine andere Farbe gesetzt wurde, bleibt diese Farbe erhalten.

Farbe der GUI-Statusleiste setzen



Das Kommando **.farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] 'Statusbereich-Option'** wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

Von einem EFOP aus können für die GUI die Farbe für Schrift und Hintergrund der Statusleiste und ihrer Teilbereiche definiert werden, z.B. um datensatzabhängige Besonderheiten wie "Premiumkunde", "Kunde zahlt nicht" usw. sichtbar zu machen.

```
<code>.farbe [ </code>-vordergrund 'Farbe'] <span class="<code>[.trans"> </code>-hintergrund 'Farbe']  
'Statusbereich-Option' </span>
```

Als Statusbereich-Option wird eine der folgenden Optionen angegeben.

Statusbereich-Option	
-statusleiste	Standardfarben <i>aller</i> Teilbereiche der Statusleiste setzen
-statustext	Farben des Hinweistextes ändern
-statusbenutzer	Individuelle Farben des Benutzerzeichens ändern
-statusmandant	Individuelle Farben des Mandantennamens ändern
-statusaufrufer	Individuelle Farben der Aufrufernummer ändern
-statuszeilen	Individuelle Farben der Zeilenangabe ändern

siehe [Farbtabelle](#)

Blaue Statusleiste mit weißer Schrift. Der Hinweistext, abweichend davon, rot hinterlegt:

```
.colour -foreground 255 255 255 -background 0 0 255 -statusbar  
.colour -background 255 0 0 -statustext
```


Hinweistexte in weißer Schrift auf rotem Hintergrund, der Rest der Statusleiste in der gewohnten Farbe:

```
.colour -foreground 255 255 255 -background 255 0 0 -statustext
```

Die fünf Teile der Statusleiste in den Hintergrundfarben rot, weiß, gelb, grün und blau. Die Schriftfarben passend dazu weiß, blau, rot, blau und wieder weiß:

```
.colour -foreground 255 255 255 -background 255 0 0 -statustext
.colour -foreground 0 0 255 -background 255 255 255 -statusrows
.colour -foreground 255 0 0 -background 255 255 0 -statususer
.colour -foreground 0 0 255 -background 0 255 0 -statusclient
.colour -foreground 255 255 255 -background 0 0 255 -statususer
```

.SCHUTZ / .PROTECTION - Schreib-/Sichtschutz

.SCHUTZ	.PROTECTION
deutsch	englisch
.SCHUTZ ['Optionen'] [-FELD] 'Feldname' ['Zeilennummer'] .SCHUTZ ['Optionen'] -ZEILE 'Zeilennummer' .SCHUTZ ['Optionen'] -SPALTE 'Feldname' .SCHUTZ ['Optionen'] -TABELLE .SCHUTZ ['Optionen'] -ALLES	.PROTECTION ['options'] [-FIELD] 'field name' ['line number'] .PROTECTION ['options'] -LINE 'line number' .PROTECTION ['options'] -COLUMN 'field name' .PROTECTION ['options'] -TABLE .PROTECTION ['options'] -ALL
Optionen	options
-GESCHUETZT -UNSICHTBAR -AUSNAHME -RESET	-PROTECTED -HIDDEN -EXCEPTION -RESET

.SCHUTZ	.PROTECTION										
[-feld] 'Feldname' ['Zeilennummer'] setzt einen Schreibschutz auf das Feld in der angegebenen Zeile. Wenn die Zeilenangabe fehlt, wird 0 angenommen (Kopfteil). -zeile 'Zeilennummer' setzt einen Schreibschutz auf eine einzelne Zeile. Zeile 0 ist der Kopfteil der Maske. -spalte 'Feldname' setzt einen Schreibschutz für eine Spalte in der Tabelle. Diese Schutzoption bleibt unberührt vom Löschen und Einfügen von Zeilen. -tabelle setzt einen Schreibschutz für die ganze Tabelle, aber nicht für den Kopfteil der Maske. Diese Schutzoption bleibt unberührt vom Löschen und Einfügen von Zeilen. -alles setzt einen Schreibschutz für alle Maskenfelder und Buttons											
<table border="1"> <thead> <tr> <th colspan="2" data-bbox="127 763 794 817">Optionen</th></tr> </thead> <tbody> <tr> <td data-bbox="127 817 794 936">-geschuetzt</td><td data-bbox="794 817 1471 936">setzt den Schreibschutz. Sie ist der Standardwert, wenn keine Art des Schutzes angegeben wurde, d.h., man kann sie auch weglassen.</td></tr> <tr> <td data-bbox="127 936 794 996">-unsichtbar</td><td data-bbox="794 936 1471 996">setzt einen Sichtschutz</td></tr> <tr> <td data-bbox="127 996 794 1167">-ausnahme</td><td data-bbox="794 996 1471 1167">macht zusammen mit -geschuetzt ein geschütztes Feld ausnahmsweise schreibbar. Zusammen mit -unsichtbar wird ein unsichtbares Feld ausnahmsweise sichtbar.</td></tr> <tr> <td data-bbox="127 1167 794 1384">-reset</td><td data-bbox="794 1167 1471 1384">bewirkt in Kombination mit -geschuetzt oder -unsichtbar ein Aufheben des Schutzes. Der Aufruf muss dieselben Parameter wie der Schutz haben. Ist keine zweite Option angegeben, wird -geschuetzt verwendet.</td></tr> </tbody> </table>		Optionen		-geschuetzt	setzt den Schreibschutz. Sie ist der Standardwert, wenn keine Art des Schutzes angegeben wurde, d.h., man kann sie auch weglassen.	-unsichtbar	setzt einen Sichtschutz	-ausnahme	macht zusammen mit -geschuetzt ein geschütztes Feld ausnahmsweise schreibbar. Zusammen mit -unsichtbar wird ein unsichtbares Feld ausnahmsweise sichtbar.	-reset	bewirkt in Kombination mit -geschuetzt oder -unsichtbar ein Aufheben des Schutzes. Der Aufruf muss dieselben Parameter wie der Schutz haben. Ist keine zweite Option angegeben, wird -geschuetzt verwendet.
Optionen											
-geschuetzt	setzt den Schreibschutz. Sie ist der Standardwert, wenn keine Art des Schutzes angegeben wurde, d.h., man kann sie auch weglassen.										
-unsichtbar	setzt einen Sichtschutz										
-ausnahme	macht zusammen mit -geschuetzt ein geschütztes Feld ausnahmsweise schreibbar. Zusammen mit -unsichtbar wird ein unsichtbares Feld ausnahmsweise sichtbar.										
-reset	bewirkt in Kombination mit -geschuetzt oder -unsichtbar ein Aufheben des Schutzes. Der Aufruf muss dieselben Parameter wie der Schutz haben. Ist keine zweite Option angegeben, wird -geschuetzt verwendet.										

Das Kommando setzt einen Schreib- oder Sichtschutz für eins der folgenden Elemente:

- Maskenfeld
- Button
- Tabellenzeile
- Tabellenspalte
- ganze Tabelle
- Kopfteil der Maske
- gesamte Maske

Der Schutz gilt für Benutzer, nicht für FOPs, es sei denn, es wurde **.setze foextern -** gesetzt.

Sie können jeweils nur einen Schutz pro Aufruf von **.schutz** setzen oder aufheben, also nur eine der Alternativen **-feld**, **-zeile**, **-spalte**, **-tabelle**, oder **-alles** verwenden.

Der **Schreibschutz** mit **-geschuetzt** verhindert, dass der Benutzer einen Eintrag in das angegebene Feld vornehmen kann. Das Feld kann aber immer noch über ein EFOP geändert werden. Ein Button kann zwar noch gedrückt werden und es wird auch das **buttonvor**-FOP aufgerufen, die Folgeaktionen des Abas Kerns bzw. des **buttonnach**-FOPs werden aber nicht ausgeführt. Wenn ein Feld einen Alias hat, wird der Schreibschutz auch für das Alias-Feld gesetzt. Das gilt auch für die **Ausnahmen**.

Der **Sichtschutz** mit **-unsichtbar** verhindert, dass der Benutzer den Inhalt des Feldes sehen kann: Anstelle des Feldinhaltes werden Sterne angezeigt. Der Inhalt des Feldes kann aber immer noch über ein EFOP abgefragt werden.

Das Kommando **.schutz** kann einen Schreibschutz in der Maske oder einen Schutz in der **Vartab** oder im Abas Kern nicht aufheben. Der Schreibschutz kann aus dem Kern kommen - dann sind keine Ausnahmen möglich.

Ausnahmen

Ein Feld ist geschützt, wenn das Feld, die Zeile, die Spalte, die Tabelle oder der ganze Datensatz geschützt ist. Es ist nicht geschützt, wenn eine Ausnahme auf dem Feld, der Zeile, der Spalte oder der Tabelle definiert wurde. **Ausnahmen** von einem Schreibschutz oder Sichtschutz kann man mit den Optionen **-ausnahme -geschuetzt** und **-ausnahme -unsichtbar** definieren. Sie wirken stärker als der zuvor mit **.schutz** gesetzte Schutz. Diese Ausnahmen sind sinnvoll angewendet, wenn man z.B. die ganze Maske oder ganze Zeilen geschützt hat, aber einzelne Felder von diesem Schutz ausnehmen will.

Die Wirkung der Kombination der Schutz- und Ausnahmeregeln ist grundsätzlich unabhängig von der Reihenfolge, in der die Regeln definiert wurden. Einschränkung: Wird die Angabe **-feld** weggelassen, muss das Feld am Ende der Zeile stehen.

Die Optionen **-geschuetzt** und **-unsichtbar**, ggf. kombiniert mit **-ausnahme**, heben sich zwar in der Wirkung gegenseitig auf, können und müssen aber unabhängig voneinander ein- und ausgeschaltet werden.



Abweichung bei Checkboxes

Checkboxen mit Sichtschutz sind, unabhängig von ihrem Inhalt, nicht markiert.

Die erste Anweisung setzt den Schreibschutz für das Feld **elem** in der ersten Zeile der Tabelle. Die zweite Anweisung verhindert die Anzeige des Feldinhalts von **vpr**. Hier wurde zur besseren Lesbarkeit die Option **-feld** verwendet, obwohl **.schutz -unsichtbar vpr** ebenso korrekt wäre.

```
.protection productListElem 1  
.protection -hidden -field salesPrice
```

Die erste Anweisung setzt den Schreibschutz für alle Felder und Buttons im Kopfteil der Maske, die zweite Anweisung verhindert die Anzeige der Inhalte der Felder in der dritten Tabellenzeile.

```
.protection -line 0  
.protection -hidden -line 3
```

Diese Anweisungen setzen den Schreibschutz für alle Felder und Buttons in den Datensätzen, in denen das benutzerdefinierte Feld **yreadonly** den Wert **G|true** hat. Die Angabe **-geschuetzt** kann weggelassen werden. Die zweite Zeile hebt den Schutz wieder auf. Die Angabe **-geschuetzt** könnte auch hier weggelassen werden.

```
.protection -all ? yreadonly = G|true  
.protection -reset -all -protected ? yreadonly = G|false
```

Setzt den Schreibschutz auf die ganze Maske, falls **yreadonly** den Wert **G|true** hat. Das Feld **yreadonly**, der Tabellenbutton **ytabbutton** und die Zeile 3 bleiben immer ungeschützt.

```
.protection -protected -exception -field yreadonly  
.protection -protected -all ? M|yreadonly = G|true  
.protection -protected -exception -column ytabbutton  
.protection -line 3 -protected -exception
```



Wenn Sie beobachten wollen, welcher Schutz gerade gesetzt ist, können Sie das mit Hilfe der passenden **Metafunktionen** **F|protected()** und **F|hidden()** für die einzelnen Schutzflaggen tun. Wollen sie dagegen feststellen, welcher Schutz effektiv auf ein Feld wirkt, sollten sie die Funktionen **F|modifiable()** und **F|visible()** verwenden.

.CHART / .CHART - Diagramme in Masken

.CHART	.CHART
deutsch	englisch
<pre> .CHART -INIT [-FILE 'Name'] .CHART -PARAM [Diagramm-Optionen] [Achsen-Optionen] .CHART -CONSTANTY 'Position' [Optionen] .CHART -STRIPEX 'Anfangsindex' 'Endindex' .CHART -STRIPEY 'Anfangswert' 'Endwert' .CHART -STARTVALUES ['Bezeichnung'] [Optionen] .CHART -VALUE 'x-Wert' 'y-Wert' 'Durchmesser' [Optionen] .CHART -VALUE 'Anfangswert' 'Endwert' [Optionen] .CHART -VALUE 'Wert' [Optionen] .CHART -SHOW [-FILE 'Dateiname'] 'Feldname' 0 1 </pre>	<pre> .CHART -INIT [-FILE 'name'] .CHART -PARAM [options chart] [options axes] .CHART -CONSTANTY 'position' [options] .CHART -STRIPEX 'initial index' 'final index' .CHART -STRIPEY 'initial value' 'final value' .CHART -STARTVALUES ['description'] [options] .CHART -VALUE 'x-value' 'y-value' 'diameter' [options] .CHART -VALUE 'initial value' 'final value' [options] .CHART -VALUE 'value' [options] .CHART -SHOW [-FILE 'file name'] 'field name' 0 1 </pre>
.chart -init initialisiert ein Diagramm. Mit .chart -param werden die Merkmale der Diagrammanzeige gesetzt. Bei mehreren Anzeigeeoptionen müssen entsprechend viele .chart -param-Anweisungen verwendet werden. .chart -constanty erzeugt eine horizontale Linie. .chart -stripex und .chart -stripey erzeugen vertikale und horizontale Streifen variabler Breite. .chart -startvalues eröffnet eine Datenserie. .chart -value setzt die Werte der Datenserien verschiedener Diagrammtypen (Blasen-, Linien-, usw.). .chart -show zeigt ein Diagramm an.	

.chart zeigt auf der GUI Diagramme innerhalb von Masken an. Es müssen mehrere **.chart**-Anweisungen hintereinander ausgeführt werden. Das Kommando ist nur per **EFOP**, z.B. in **buttonnach**, verfügbar.

Konventionen:

- Die x-Achse wird horizontal dargestellt, die y-Achse wird vertikal dargestellt.
- Der Wertebereich der x-Achse wird bei allen Diagrammen außer den Blasendiagrammen automatisch durch den kleinsten und größten x-Wert bestimmt.

- Texte, die unter den Optionen angegeben werden, müssen in Anführungszeichen eingeschlossen sein.

Lesen Sie auch folgende Kapitel:

Zeit- und Datumsdarstellung in Diagrammen
 Anzeigen von Diagrammen in einer Zeilenlupe
 Farben in Diagrammen verwenden

.chart -init - Diagramm initialisieren

Diese Anweisung muss immer ausgeführt werden, bevor ein Diagramm angezeigt werden kann (Ausnahme:

`.chart -show [-file 'Dateiname']`).

`.chart -init [-file 'Name']`

Normalerweise verwendet `.chart` eine Temporärdatei, um die Optionen bis zur Anzeige des Diagramms zu speichern. Die Temporärdatei wird nach der Anzeige gelöscht. Mit `-file` können Sie den Namen einer Datei angeben, die anstelle der Temporärdatei benutzt wird. Diese Datei wird nach dem Anzeigen nicht gelöscht.

.chart -param - Merkmale der Diagrammdarstellung

`.chart -param [Diagramm-Optionen] [Achsen-Optionen]`

Hiermit werden die Merkmale der Diagrammanzeige gesetzt. Bei mehreren Anzeigeeoptionen müssen entsprechend mehr `.chart -param`-Anweisungen verwendet werden:

```
.chart -PARAM -CHARTTYPE bar
.chart -PARAM -TITLECHART "Percentual efficiency"
.chart -PARAM -TITLEY "%"
.chart -PARAM -STACKTYPE stacked
.chart -PARAM -MARKERX "Mon.; Tue.; Wed.; Thu.; Fri.; Sat.; Sun."
.chart -PARAM -MINMAXY 0 200
```

.chart -param Diagramm-Optionen

`.chart -param [-charttype 'Typ'] [-titlechart 'Überschrift'] [-legend NONE] [-stacktype 'Stapeltyp'] [-options]`

Der Diagrammtyp (`.chart -param -charttype 'Typ'`) ist vor allen anderen Optionen anzugeben. Mit einem Diagrammtyp können beliebig viele Datenserien dargestellt werden, mit Blasendiagrammen allerdings nur eine einzige:

Blasendiagramm	Die Datenserie besteht aus x/y-Durchmesserwerten (jeweils eine Blase an Position x/y) mit einem bestimmten Durchmesser).
----------------	--

andere Diagramme	Jede Datenserie besteht aus y-Werten, die zu festen x-Werten (siehe -markerx) gehören.
------------------	--

Für **.chart -param** gelten folgende Optionen:

-charttype 'Typ'	Bestimmt den Diagrammtyp und gilt dann für alle Datenserien. Jedoch kann auch für einzelne Datenserien ein eigener Diagrammtyp gesetzt werden (siehe .chart -startvalues -charttype 'Typ').												
	<table> <tr> <td>PIE</td><td>Tortendiagramm</td></tr> <tr> <td>BAR</td><td>Säulendiagramm</td></tr> <tr> <td>LINES</td><td>Liniendiagramm</td></tr> <tr> <td>CURVE</td><td>Kurvendiagramm</td></tr> <tr> <td>CURVEAREA</td><td>ausgefülltes Kurvendiagramm</td></tr> <tr> <td>BUBBLE</td><td>Blasendiagramm</td></tr> </table>	PIE	Tortendiagramm	BAR	Säulendiagramm	LINES	Liniendiagramm	CURVE	Kurvendiagramm	CURVEAREA	ausgefülltes Kurvendiagramm	BUBBLE	Blasendiagramm
PIE	Tortendiagramm												
BAR	Säulendiagramm												
LINES	Liniendiagramm												
CURVE	Kurvendiagramm												
CURVEAREA	ausgefülltes Kurvendiagramm												
BUBBLE	Blasendiagramm												
-titlechart 'Überschrift'	Überschrift des Diagramms. Diese darf mehrzeilig sein. Ein Semikolon bezeichnet einen Zeilenumbruch.												
-legend NONE	Keine Legende anzeigen												
-stacktype 'Stapeltyp'	Als Stapeltyp sind möglich: <table> <tr> <td>STACKED</td><td>Zeichnet die Säulen über- statt nebeneinander</td></tr> </table>	STACKED	Zeichnet die Säulen über- statt nebeneinander										
STACKED	Zeichnet die Säulen über- statt nebeneinander												

.chart -param Achsen-Optionen

Für beide Achsen lassen sich folgende Gestaltungen vornehmen.

Für die x-Achse

```
.chart -param [-titlex 'Überschrift'] [-markerx 'Text 1'; 'Text 2'; 'Text 3'; ...] [-formatx 'Format'] [-minmaxx 'Anfangswert' 'Endwert'] [-anglex 'Winkel'] [-minorgridx]
```

Analog für die y-Achse

```
.chart -param [-titley 'Überschrift'] [-markery 'Text 1'; 'Text 2'; 'Text 3'; ...] [-formaty 'Format'] [-minmaxy 'Anfangswert' 'Endwert'] [-decimalsy 'Anzahl'] [-angley 'Winkel'] [-minorgridy]
```

-titlex 'Überschrift'	Überschrift der x-Achse. Diese darf mehrzeilig sein. Ein Semikolon bezeichnet einen Zeilenumbruch.
------------------------------	--

<pre>-markerx 'Text 1'; 'Text 2'; 'Text 3'; ...</pre>	Beschriftung der x-Achse. Ohne diese Angabe werden zur Datenreihe passende Werte auf die x-Achse geschrieben (eventuell entsprechend FORMATX). Die einzelnen Texte müssen durch ein Semikolon voneinander getrennt sein. Nicht sinnvoll wenn -formatx verwendet wird. Für jeden Text kann auch ein eigener .chart-Befehl angegeben werden. Dies ist vor allem dann sinnvoll, wenn die Texte insgesamt länger als eine FO-Zeile sind.																						
<pre>-formatx 'Format'</pre>	Format, in dem die Beschriftung der x-Achse angezeigt wird. Nicht sinnvoll wenn -markerx verwendet wird. <table border="1" data-bbox="804 678 1460 947"> <tr> <td>THH:mm THH:mm:ss</td><td>Uhrzeit</td></tr> <tr> <td>Ddd.MM.yyyy Ddd.MM.yy</td><td>Datum</td></tr> <tr> <td>Xdd.MM.yy HH:mm</td><td>Datum und Uhrzeit</td></tr> </table> Es können auch die in der Windows-Systemsteuerung unter "Regions- und Sprachoptionen" einzustellenden Formate angezeigt werden: <table border="1" data-bbox="804 1146 1460 1895"> <tr> <td>NUMBER</td><td>Zahl (z.B. 123.456.789,00)</td></tr> <tr> <td>CURRENCY</td><td>Währung (z.B. 123.456.789,00 Euro)</td></tr> <tr> <td>DATE</td><td>Kurzes Datum (z.B. 14.11.2010)</td></tr> <tr> <td>LONGDATE</td><td>Langes Datum (z.B. Donnerstag, 12. November 2010)</td></tr> <tr> <td>TIME</td><td>Uhrzeit (z.B. 12:52:58)</td></tr> <tr> <td>DATETIME</td><td>Kurzes Datum und Uhrzeit (z.B. 14.11.2010 12:52:58)</td></tr> <tr> <td>SCIENTIFIC</td><td>Wissenschaftliche Zahl (z.B. 1.23e+002)</td></tr> <tr> <td>PERCENTAGE</td><td>Prozentangabe mit zwei Nachkommastellen (z.B. 12.92 %)</td></tr> </table> Siehe auch Zeit- und Datumsdarstellung in Diagrammen.	THH:mm THH:mm:ss	Uhrzeit	Ddd.MM.yyyy Ddd.MM.yy	Datum	Xdd.MM.yy HH:mm	Datum und Uhrzeit	NUMBER	Zahl (z.B. 123.456.789,00)	CURRENCY	Währung (z.B. 123.456.789,00 Euro)	DATE	Kurzes Datum (z.B. 14.11.2010)	LONGDATE	Langes Datum (z.B. Donnerstag, 12. November 2010)	TIME	Uhrzeit (z.B. 12:52:58)	DATETIME	Kurzes Datum und Uhrzeit (z.B. 14.11.2010 12:52:58)	SCIENTIFIC	Wissenschaftliche Zahl (z.B. 1.23e+002)	PERCENTAGE	Prozentangabe mit zwei Nachkommastellen (z.B. 12.92 %)
THH:mm THH:mm:ss	Uhrzeit																						
Ddd.MM.yyyy Ddd.MM.yy	Datum																						
Xdd.MM.yy HH:mm	Datum und Uhrzeit																						
NUMBER	Zahl (z.B. 123.456.789,00)																						
CURRENCY	Währung (z.B. 123.456.789,00 Euro)																						
DATE	Kurzes Datum (z.B. 14.11.2010)																						
LONGDATE	Langes Datum (z.B. Donnerstag, 12. November 2010)																						
TIME	Uhrzeit (z.B. 12:52:58)																						
DATETIME	Kurzes Datum und Uhrzeit (z.B. 14.11.2010 12:52:58)																						
SCIENTIFIC	Wissenschaftliche Zahl (z.B. 1.23e+002)																						
PERCENTAGE	Prozentangabe mit zwei Nachkommastellen (z.B. 12.92 %)																						

<code>-minmaxx 'Anfangswert' 'Endwert'</code>	Wertebereich der x-Achse. Ohne diese Angabe wird der Wertebereich der x-Achse so gewählt, dass der kleinste und der größte x-Wert den Wertebereich bilden.
<code>-minorgridx</code>	Es werden zusätzliche Gitterlinien eingeblendet.

Die Beschreibungen gelten analog für die y-Achse.

.chart -constanty - Horizontale Linien

Mit dem Parameter `-constanty` horizontale Linien in das Diagramm eingezeichnet.

`.chart -constanty 'Position' -title 'Bezeichnung'`

<code>-title 'Bezeichnung'</code>	fügt zur Linie eine Beschriftung hinzu
-----------------------------------	--

.chart -stripex und .chart -stripey - Vertikale und horizontale Streifen

Mit dem Parameter `-stripex` werden vertikale Streifen, mit dem Parameter `-stripey` horizontale Streifen in das Diagramm eingezeichnet. Bei einem Tortendiagramm wird dieser Befehl ignoriert.

`.chart -stripex 'Anfangsindex' 'Endindex'`

`.chart -stripex 'Anfangswert' 'Endwert'`

Bei `-stripex` muss jeweils der 0-basierte Index innerhalb der Datenserie angegeben werden (und nicht die Beschriftung der x-Achse). Bei `-stripey` muss jeweils der y-Wert angegeben werden.

Die Breite des Streifens ist "Endindex - Anfangsindex" bzw. "Endwert - Anfangswert".

.chart -startvalues - Datenserie eröffnen

Das Kommando eröffnet eine Datenserie.

`.chart -startvalues ['Bezeichnung'] [-color 'Farbnummer'] [-rgbcolor 'Rot-Wert' 'Grün-Wert' 'Blau-Wert'] [-charttype 'Typ']`

<code>'Bezeichnung'</code>	Die Bezeichnung der Datenserie wird in die Legende geschrieben.
<code>-color 'Farbnummer'</code>	für alle Werte einer Datenserie. Die Farbnummer bezieht sich auf eine fest vorgegebene Farbpalette .
<code>-rgbcolor 'Rot-Wert' 'Grün-Wert' 'Blau-Wert'</code>	für alle Werte einer Datenserie. RGB-Farbwerte müssen ≥ 0 und ≤ 255 sein. Alle drei Werte auf Maximum (<code><code>255 255 255</code></code>) ergibt Weiß, alle drei Werte auf Minimum (<code><code>0 0 0</code></code>) ergibt Schwarz.
	<code>-charttype 'Typ'</code>

.chart -value - Datenserien verschiedener Diagrammtypen

Mit diesem Befehl wird ein Wert einer Datenserie gesetzt. Mehrere Werte werden durch entsprechend viele **.chart -value**-Anweisungen gesetzt.

Statt der Werte kann auch die symbolische Wertbezeichnung **null** eingesetzt werden.

```
.chart -VALUE null
```

Dieses Kommando bewirkt, dass der entsprechende Datenpunkt nicht dargestellt wird. Es ist insbesondere bei gestapelten Säulendiagrammen (**STACKED**) nützlich, wenn die oberste Teilsäule die Höhe 0 haben soll.



Bei Verwendung des Wertes 0 statt **NULL** wird die entsprechende Teilsäule trotzdem mit 1 Pixel Höhe dargestellt. In der 3D-Darstellung hat die Säule dann eine unerwünschte Farbe auf der Oberseite.

```
.chart -INIT
.chart -PARAM -CHARTTYPE bar
.chart -PARAM -TITLECHART "Percental efficiency"
.chart -PARAM -TITLEY "%"
.chart -PARAM -STACKTYPE STACKED
.chart -PARAM -MARKERX "Mon.; Tue.; Wed.; Thu.; Fri.; Sat.; Sun."
.chart -PARAM -MINMAXY 0 200
.chart -STARTVALUES "normal load" -RGBCOLOR 0 255 0
.chart -VALUE 70
.chart -VALUE 80
.chart -VALUE 100
.chart -VALUE 100
.chart -VALUE 60
.chart -VALUE 20
.chart -VALUE 20
.chart -STARTVALUES "overload" -RGBCOLOR 255 0 0
.chart -VALUE null
.chart -VALUE null
.chart -VALUE 10
.chart -VALUE 70
.chart -VALUE null
.chart -VALUE null
.chart -VALUE null
.chart -SHOW ychart 0
```

.chart -value für Blasendiagramm

.chart -value 'x-Wert' 'y-Wert' 'Durchmesser' [Optionen] Jeder Wert in einem Blasendiagramm entspricht einer Blase.

.chart -value für alle anderen Diagramme

`.chart -value 'Wert' [Optionen]` Als Wert wird der y-Wert angegeben. Für den x-Wert wird für jeden Aufruf von `.chart -value` jeweils der nächste der Werte verwendet, die mit `.chart -param -markerx` gesetzt wurden (oder 1, 2, 3 usw., wenn keine x-Werte gesetzt wurden).

.chart -value - Optionen für Diagrammtypen

<code>-color 'Farbnummer'</code>	Die Farbnummer bezieht sich auf eine fest vorgegebene Farbpalette . Sie deutet an, ob der Eröffnungswert höher ist als der Schlusswert.
<code>-rgbcolor 'Rot-Wert' 'Grün-Wert' 'Blau-Wert'</code>	RGB-Farbwerte müssen ≥ 0 und ≤ 255 sein. Alle drei Werte auf Maximum (<code><code>255 255 255</code></code>) ergibt Weiß, alle drei Werte auf Minimum (<code><code>0 0 0</code></code>) ergibt Schwarz.
<code>-tip 'Tooltip'</code>	Wenn Sie einen Tooltip anlegen wollen, müssen Sie ihn für alle Werte anlegen.

```
.chart -VALUE 'U|xtvalue' -DATA 'U|xtcmd' -TIP 'U|xttip' -COLOR 7
```

.chart -show - Diagramm anzeigen

`.chart -show [-file 'Dateiname'] 'Feldname' 0|1` zeigt ein Diagramm an.

<code>-file 'Dateiname'</code>	Die Datei muss mit Hilfe von <code>.chart -init -file 'Dateiname'</code> angelegt worden sein. Das Diagramm, das dabei angezeigt wurde, wird jetzt erneut angezeigt.
<code>Feldname</code>	Name des Feldes in der Maske
<code>0 1</code>	<code>0</code> wenn sich das Feld im Kopfteil der Maske befindet <code>1</code> wenn sich das Feld in der Tabelle der Maske befindet

```
.chart -SHOW ychart 0
```

stellt im Diagrammfeld `'ychart'` des Kopfteils ein Diagramm mit den Daten dar, die durch die vorherige `.chart`-Befehle angegeben wurden.

```
.chart -SHOW -FILE 'U|xtfile' M|ychart 0
```

stellt im Diagrammfeld '**ychart**' des Kopfteils ein Diagramm mit den Daten aus der Datei '**U|xfile**' dar.



Option -FILE

Durch ein Abas Upgrade kann sich das Format der Datei ändern. Es ist daher nicht garantiert, dass die erstellten Dateien weiterhin angezeigt werden. Die Option **-file** sollte deshalb nur bei Tests verwendet werden und nicht, um Diagramme zu speichern.

Fehlerhafte Anzeige - Berechtigungen



Die Anzeige der Charts in der Abas GUI kann bei manchen Benutzern fehlerhaft sein, weil die Berechtigungen nicht richtig konfiguriert sind. Voraussetzung ist:

- Das Kommando **<Text> <Zeigen>** ist erlaubt.
- Das Kommando **<WEB-Container> <Zeigen>** ist erlaubt.
- Das Infosystem **DOWNLOAD** funktioniert ohne Fehler.

Zeit- und Datumsdarstellung in Diagrammen

Bei der Zeit- und Datumsdarstellung werden die Werte normalerweise auch als Markierungen verwendet. Dies ist sinnvoll, wenn zwischen den Werten immer derselbe Abstand besteht (z.B. bei Jahreszahlen: 2008, 2009, 2010 usw.) oder wenn die Werte eine symbolische Bedeutung haben (z.B. Monate: Jan, Feb, Mar), nicht jedoch, wenn die Zeit oder das Datum auf der Achse dargestellt wird und die Werte unregelmäßige Abstände haben.

Mit Hilfe von **-formatx 'Format'** bzw. **-formaty 'Format'** muss dem Programm mitgeteilt werden, dass auf der Achse eines Diagramms Zeit- oder Datumswerte dargestellt werden. Die Werte selbst müssen als Double übergeben werden. Der Wertebereich wird mit **-minmaxy 'Anfangswert' 'Endwert'** definiert. **-markerx** und **-markery** dürfen dann nicht verwendet werden.

Der Vorkommaanteil eines Zeit-Datum-Wertes ist die Anzahl der Tage seit dem 30.12.1899. Der Nachkommaanteil ist die Uhrzeit, d.h. 0.5 entspricht 12 Uhr. Auch der Wertebereich und der Abstand zwischen den Markierungen auf der Achse wird so dargestellt.

Setzen von Datumswerten in FOPs Die Anzahl der Tage seit dem 30.12.1899 erhält man mit Hilfe einer Variablen der Art "GD".

Im folgenden Beispiel sind **term** und **sterm** Datenbankvariablen der Art "DATUM", z.B. eines Auftrags.

```
.type GD xdms
.type int xistart xiend
.formula U|x dms = "30.12.1899"
.formula U|xistart = H|startDateWeek - U|x dms
.formula U|x iend = H|deadline - U|x dms
..
..
.chart -PARAM -FORMATY Ddd.MM.yy
```

```
.chart -VALUE 'U|xistart' 'U|xiend'
```

Ein Tag: Werte werden im Format 00:00 bis 24:00 dargestellt.

```
-formaty THH:mm -minmaxy 0 1
```

Zwei Tage: Werte werden im Format 00:00 bis 24:00 dargestellt.

```
-formaty THH:mm -minmaxy 0 2
```

Anzeigen von Diagrammen in einer Zeilenlupe

Ist in der Zeilenlupe ein Maskenfeld der Art CHART vorhanden, die Zeilenlupe aber geschlossen, so ist es möglich, zuerst das Diagramm in der Zeilenlupe mit dem Kommando **.chart** zu erzeugen und danach die Zeilenlupe mit dem Kommando **.cursor 'Variablenname'** zu öffnen.

Nähere Informationen zum Kommando **.cursor** finden Sie in den Kapiteln **.cursor - Cursorsteuerung** und **Steuerung des Abas Dialogs durch FOPs**.

Farben in Diagrammen verwenden

Farben sollten immer für alle Werte oder nicht angegeben werden. Wenn keine Farben angegeben werden, werden die Säulen und Linien einer Datenserie in derselben Farbe dargestellt.

Mit **-color 'Farbnummer'** können Sie auf die vorgegebene Farbpalette zugreifen. Sie ist in der Symbolleiste des Diagramms über den Button Farbe zu finden. Außerdem können Sie alle vordefinierten Farben im Infosystem 10201 **Farben** sehen. In der Zeilenlupe des Infosystems finden Sie auch die Farbnummer für die Verwendung in Charts.

Diagramm-Beispiele

Das **Infosystem Chart-Demo** (Suchwort CHARTS) zeigt die folgenden Beispieldiagramme an. In der Maske des Infosystems kann jedes Diagramm durch ein FOP oder ein Java-FOP dargestellt werden, das den FO-Befehl **.chart** verwendet. Es wird jeweils ein Auswahlménü geöffnet, in dem der Diagrammtyp gewählt werden kann. In dem dargestellten Diagramm werden alle möglichen Optionen von **.chart** benutzt.

Tortendiagramm

```
.chart -INIT
.chart -PARAM -CHARTTYPE pie
.chart -PARAM -TITLECHART "Turnover per country"
.chart -PARAM -MARKERX "Germany; France; Spain; Italy; Poland; Hungary"
.chart -STARTVALUES 1999
.chart -VALUE 100000
.chart -VALUE 90000
.chart -VALUE 40000
.chart -VALUE 20000
.chart -VALUE 10000
.chart -VALUE 12000
.chart -STARTVALUES 2000
.chart -VALUE 110000
```

```
.chart -VALUE 75000
.chart -VALUE 50000
.chart -VALUE 45000
.chart -VALUE 60000
.chart -VALUE 8000
.chart -SHOW 'M|chfeld' 'M|chtab'
```

Säulendiagramm

```
.chart -INIT
.chart -PARAM -CHARTTYPE bar
.chart -PARAM -TITLECHART "Turnover share per country; only EU"
.chart -PARAM -TITLEY "million EUR;per year"
.chart -PARAM -MARKERX "Germany; France; Spain; Italy; Poland; Hungary"
.chart -PARAM -FORMATY number
.chart -CONSTANTY 500 -RGBCOLOR 0 255 0 "Average"
.chart -CONSTANTY 100 -RGBCOLOR 255 0 0 "Minimum"
.chart -STARTVALUES 1999 -COLOR 3
.chart -VALUE 1000
.chart -VALUE 900
.chart -VALUE 400
.chart -VALUE 100
.chart -VALUE 50
.chart -VALUE 20
.chart -STARTVALUES 2000 -COLOR 4
.chart -VALUE 1100
.chart -VALUE 750
.chart -VALUE 500
.chart -VALUE 450
.chart -VALUE 600
.chart -VALUE 80
.chart -STARTVALUES "Debit" -CHARTTYPE lines -RGBCOLOR 0 0 255
.chart -VALUE 900
.chart -VALUE 800
.chart -VALUE 350
.chart -VALUE 90
.chart -VALUE 45
.chart -VALUE 19
.chart -SHOW 'M|chfeld' 'M|chtab'
```

Diagramm mit gestapelten Säulen

Im Gegensatz zum [Säulendiagramm](#) werden hier die Säulen übereinander dargestellt.

```
.chart -INIT
.chart -PARAM -CHARTTYPE bar
.chart -PARAM -TITLECHART "Turnover share per country"
.chart -PARAM -TITLEY "% of turnover"
.chart -PARAM -STACKTYPE stacked100
.chart -PARAM -MARKERX "1995; 1996; 1997; 1998; 1999"
.chart -PARAM -MINMAXY 0 100
```

```
.chart -STARTVALUES "France"
.chart -VALUE 1000
.chart -VALUE 900
.chart -VALUE 800
.chart -VALUE 900
.chart -VALUE 950
.chart -STARTVALUES "Germany"
.chart -VALUE 1100
.chart -VALUE 750
.chart -VALUE 500
.chart -VALUE 600
.chart -VALUE 450
.chart -STARTVALUES "Italy"
.chart -VALUE 1100
.chart -VALUE 750
.chart -VALUE 500
.chart -VALUE 800
.chart -VALUE 900
.chart -SHOW 'M|chfeld' 'M|chtab'
```

Liniendiagramm

```
.type text xtchart xttip xtdata
.type text xtcolor
.type real xrwert
.formula U|xtchdat = "CHART.LINES.DAT"
.chart -INIT -FILE 'U|xtchdat'
.chart -PARAM -CHARTTYPE lines
.chart -PARAM -TITLECHART "Profit/month"
.chart -PARAM -MARKERX "Jan"
.chart -PARAM -MARKERX "Feb"
.chart -PARAM -MARKERX "Mar"
.chart -PARAM -MARKERX "Apr"
.chart -PARAM -MARKERX "May"
.chart -PARAM -MARKERX "Jun"
.chart -PARAM -MARKERX "Jul"
.chart -PARAM -MARKERX "Aug"
.chart -PARAM -MARKERX "Sep"
.chart -PARAM -MARKERX "Oct"
.chart -PARAM -MARKERX "Nov"
.chart -PARAM -MARKERX "Dec"
.chart -PARAM -MINMAXY -4000 20000
.chart -PARAM -FORMATY number
.formula U|xtcolor = " 0 127 255"
.formula U|xrwert = 2000.0
.chart -STARTVALUES 1999 -RGBCOLOR 'U|xtcolor'
.formula U|xttip = "Account"
.formula U|xttip = U|xttip + " " + "44000"
.formula U|xtdata = "<" + "Account" + ">" + "44000" + "<" + "View" + ">"
.chart -VALUE 1000 -TIP 'U|xttip' -DATA 'U|xtdata'
.chart -VALUE 'U|xrwert'
.chart -VALUE 3000
```

```
.chart -VALUE 4000
.chart -VALUE 6000
.chart -VALUE 4000
.chart -VALUE 15000
.chart -VALUE 14000
.chart -VALUE -1000
.chart -VALUE 11000
.chart -VALUE 12000
.chart -VALUE 13000
.fo xtcOLOR=" 0 0 192"
.chart -STARTVALUES 2000 -RGBCOLOR 'U|xtcolor'
.chart -VALUE 2000
.chart -VALUE 3000
.chart -VALUE 4000
.chart -VALUE 5000
.chart -VALUE 11000
.chart -VALUE 5000
.chart -VALUE 16000
.chart -VALUE 15000
.chart -VALUE -2000
.chart -VALUE 12000
.chart -VALUE 12500
.chart -VALUE 13000
.chart -SHOW -FILE 'U|xtchdat' 'M|chfeld' 'M|chtab'
```

Kurvendiagramm

```
.chart -INIT
.chart -PARAM -CHARTTYPE curve
.chart -PARAM -TITLECHART "Depreciation run"
.chart -PARAM -TITLEY "TEUR"
.chart -PARAM -LEGEND none
.chart -PARAM -MARKERX "2000;2001;2002;2003;2004;2005;2006;2007;2008;2009"
.chart -STRIPEX 0 4
.chart -STRIPEY 3000 10000
.chart -STARTVALUES "Machine 1" -LINEWIDTH 3
.chart -VALUE 10000
.chart -VALUE 9000
.chart -VALUE 8000
.chart -VALUE 7000
.chart -VALUE 6000
.chart -VALUE 5000
.chart -VALUE 4000
.chart -VALUE 3000
.chart -VALUE 2000
.chart -VALUE 1000
.chart -STARTVALUES "Machine 2"
.chart -VALUE 11000
.chart -VALUE 9900
.chart -VALUE 8910
.chart -VALUE 8019
.chart -VALUE 7217
```



```
.chart -VALUE 6495
.chart -VALUE 5845
.chart -VALUE 5261
.chart -VALUE 4735
.chart -VALUE 4261
.CHART -SHOW ychart 0
```

Blasendiagramm

```
.chart -INIT
.chart -PARAM -CHARTTYPE bubble
.chart -PARAM -TITLECHART "Product portfolio (turnover)"
.chart -PARAM -TITLEX "Market attractiveness"
.chart -PARAM -TITLEY "Market growth"
.chart -PARAM -MINMAXX 0 10
.chart -PARAM -MINMAXY -5 5
.chart -PARAM -OPTIONS
.chart -CONSTANTX 5 -RGBCOLOR 0 0 0
.chart -CONSTANTY 0 -RGBCOLOR 0 0 0
.chart -STARTVALUES
.chart -VALUE 6 -1 2.500 -TIP "Product A" -COLOR 1
.chart -VALUE 8 -2 1.200 -TIP "Product B" -COLOR 1
.chart -VALUE 3 -3 2.100 -TIP "Product C" -COLOR 1
.chart -VALUE 2 1 1 -TIP "Product D; Turnover: 0" -RGBCOLOR "255 0 0"
.chart -VALUE 5 1 1.200 -TIP "Product E" -COLOR 1
.chart -VALUE 6 2 1.750 -TIP "Product F" -COLOR 1
.chart -SHOW 'M|chfeld' 'M|chtab'
```

Infosystem Chart-Demo

Das Infosystem 10168 CHARTS, Arbeitsbereich demo, hat folgende Funktion:

- Demonstration für die Möglichkeiten des FO-Befehls **.chart**.

FOP

Beim Klick auf diesen Button wird ein Auswahlménü geöffnet, in dem der Diagrammtyp gewählt werden kann. Danach startet das FOP, das dieses Diagramm anzeigt.

Java-FOP

Beim Klick auf diesen Button wird ein Auswahlménü geöffnet, in dem der Diagrammtyp gewählt werden kann. Danach startet das Java-FOP, das dieses Diagramm anzeigt.

Ausgabe

In diesen Feldern werden die Diagramme angezeigt, die mit dem zugehörigen Button **FOP** oder **Java-FOP** ausgewählt werden.

Ereignisse bei maskenlosen Kommandos

Maskenlose Kommandos, die einen Datensatz oder einen Bereich von Datensätzen bearbeiten, weisen eine hohe Analogie zu Kommandos mit Maske auf. Sie folgen einem sehr ähnlichen Standardablauf. Deswegen

werden für diese Kommandos keine neuartigen Ereignisse definiert, sondern die bekannten maskenbezogenen Ereignisse verallgemeinert. Da keine Editierphase vorhanden ist, werden nur datensatzglobale Ereignisse generiert, nicht aber Ereignisse wie "Feldaustritt" oder "Zeile löschen". Auf die wenigen Unterschiede zu Maskenkommandos wird im Folgenden explizit hingewiesen.

Der Standardablauf ist folgender:

1. Kommandoeingabe
2. bei entsprechender Anwendung einleitende Fragen zur Dialoggestaltung
3. Laden des ersten bzw. nächsten Datensatzes
4. Maskeneintritts-FOP
5. Maskenprüfungs-FOP
6. Prüfung durch Abas
7. bei entsprechender Anwendung eine Bestätigungsanfrage
8. Schreiben des Datensatzes auf Festplatte
9. a) bei Ablehnung in 4, 5, 6 oder 7: Maskenabbruch-FOP
b) bei Erfolg: Workflow-FOP, Maskenaustritts-FOP
10. Entsperren des Datensatz
11. Maskenende-FOP
12. Fortsetzen mit 3

Die Bedeutung des Exit-Status unter 4. ist analog zu Maskenkommandos. Die Bedeutung des Exit-Status unter 5. ähnelt der bei Maskenkommandos, allerdings führt der Exit-Status unter 1. nicht zurück in die Editierphase, da eine solche nicht existiert, sondern zu Punkt 9a. Aus demselben Grund führt eine Ablehnung in 6. oder 7. ebenfalls zu Punkt 9a.

Im EFOP stehen die bekannten 'evt'-Variablen zur Verfügung. Der aktuelle Datensatz ist geladen (Maskenpuffer, M|). Dies gilt auch beim Löschen-Kommando.



Die Änderungsrechte für Felder richten sich in den EFOPs von maskenlosen Kommandos immer nach der Variablentabelle.

Protokoll der stattfindenden Ereignisse

Im Zweifelsfall können Sie sich alle Ereignisse anzeigen lassen. Dafür gibt es verschiedene Möglichkeiten.

Unter GUI können Sie im Kontextmenü einer Maske unter dem Eintrag "FOP" verschiedene Trace-Modi einschalten. Diese Einstellung gilt aber nur in der aktuellen Maske.

Per Eintrag in der Konfigurationsdatei `fop.txt`:

```
* * * * * TRACE.FOP
```

TRACE.FOP gehört zum Standardlieferumfang. Es dient dazu, Ereignisse in einem kleinen Fenster zu

protokollieren. Wenn der Exit-Status eine Bedeutung hat, kann er im Dialog gesetzt werden. Statt **TRACE . FOP** können Sie auch **FOP . EVENT** und **FOP . EV** benutzen: **FOP . EVENT** gibt zusätzlich alle Umgebungsvariablen des EFOPs aus und bietet die Möglichkeit, in den Direktmodus zu wechseln. **FOP . EV** liefert eine knappe, dialogfreie Ausgabe und benötigt die wenigsten Tasteneingaben zum Weitermachen.

Anzeige und Editieren der konfigurierten EFOPs

In Masken kann man mit Kontextmenü [FOP] [alle FOPs zeigen/bearbeiten] ein FOP auswählen und im Editor öffnen.

Datensatzzustand und Zugriffsrechte in EFOPs

Wenn ein EFOP für einen Datensatz gestartet wird, liegt der Datensatz bei allen Ereignissen außer Maskenaustritt im **Dialogzustand** vor.

Eine spezielle Situation liegt beim Ereignis "Maskenaustritt" vor: Der Datensatz liegt im Festplattenzustand vor. Er ist bereits geschrieben. Hier wird die vom Dialog gehaltene Sperre unterbrechungsfrei an die FO weitergereicht. Dadurch kann der Dialogdatensatz nach Veränderung nochmals mit **.bringe maske** geschrieben werden. Bei anderen Ereignissen ist **.bringe maske** zwar erlaubt, aber wirkungslos.

Im Unterschied zu **maskaus** ist bei **maskende** der bearbeitete Datensatz nicht mehr gesperrt. Er kann somit auch nicht mehr verändert und mit **.bringe maske** erneut geschrieben werden. Er ist allerdings weiterhin in dem Zustand verfügbar, in dem er auf der Festplatte geschrieben wurde. Das **maskende**-Ereignis tritt im Gegensatz zu **maskaus** auch bei Abbruch ein.

Ob ein FOP mit **maskaus** oder **maskende** verknüpft wird, stellt einen wichtigen Unterschied dar: Wird in dem EFOP auf eine Eingabe des Benutzers gewartet, so ist dringend zu empfehlen, das Ereignis **maskende** zu verwenden. Da es keine Garantie gibt, dass der Benutzer den Dialog tatsächlich sofort beendet, kann der Datensatz auf unbestimmte Zeit gesperrt bleiben. Dies kann zu Deadlocks (gegenseitiges Warten auf Sperren), zumindest aber zu Behinderungen anderer Benutzer führen. Eine typische Aktion für ein **maskende**-FOP ist das Anzeigen einer von Abas automatisch vergebenen Identnummer in einer Quittierbox.

Soll der Datensatz hingegen verändert oder ein zweiter Datensatz geschrieben werden, so muss unbedingt das Ereignis **maskaus** verwendet werden. Andernfalls könnten andere Benutzer, da die Sperre aufgehoben ist, bereits mit dem Datensatz weiterarbeiten. **maskaus**-FOPs, die beide Arten von Aufgaben erfüllen, sollten auf die beiden Ereignisse aufgeteilt werden, insbesondere dann, wenn es bekannte Probleme mit Sperren oder Deadlocks gibt.

Zeilenoperationen in EFOPs

Bei feldbezogenen Ereignissen ist die gerade editierte Zeile automatisch die aktuelle. Sie muss nicht mit einem Befehl wie **.hole zeile** geladen werden. Um eine andere Zeile zur aktuellen zu machen, starten Sie das Kommando

.hole maske zeile oder **.hole amaske zeile**

Zur Positionierung auf den Kopfteil wählen Sie

`.hole maske kopf` oder `.hole amaske kopf`

Wie in anderen Fällen verweist das Schlüsselwort **maske** auf den Datensatz in der Maske bzw. auf den M-Puffer und **amaske** auf den Datensatz im **A-Puffer**. Die folgenden Aussagen sind nur für **maske** ausgeführt, gelten analog auch für das Schlüsselwort **amaske**.

Dem Kommando `.hole maske zeile` kann auch eine Zeilennummer oder eine Zeilen-ID mitgegeben werden. Um eine Zeile zu suchen, kann ein Selektionsstring verwendet werden.

`.hole zeile 'Zeilennummer'|'Zeilen-ID'|"Selektionsstring"`

Lesen Sie hierzu Kapitel `.hole zeile` - [Holen von Tabellenzeilen](#).

In EFOPs können, sofern die jeweilige Anwendung es zulässt, Zeilen eingefügt und gelöscht werden:

`.make maske zeile +`

fügt eine Zeile nach der aktuellen ein.

`.make maske zeile -`

löscht eine Zeile.

Abfrage der aktuellen Zeilennummer

Ist ein Datensatz geladen, können Sie mit Hilfe der Variablen **aktzeile** die aktuelle Zeilennummer abfragen.

```
.type int xiactrow
rowcount: 'M|countTabRows'
.formula U|xiactrow = 'M|currTabRow'
.select screen line 'M|countTabRows'
.. insert new row at the end
.make screen line +
.. back to widow line
.select screen line 'U|xiactrow'
```



Die Variable **aktzeile** kann sowohl in Kommando-FOPs als auch in EFOPs verwendet werden, allerdings in Kommando-FOPs nicht für die Köpfe der Ein- und Verkaufssätze (Gruppe 1), da die Positionen (Gruppe 2) eigenständige Datensätze sind.

Zeilenvariablen in EFOPs

In einem EFOP kann die aktuelle Zeile von der Ereigniszeile verschieden sein, also

`M|aktzeile <> M|evtzeile`

Das ist dann der Fall, wenn die Zeile, in der das Ereignis stattfand, inzwischen gelöscht wurde oder erst angelegt werden soll.

Ein Datensatz habe fünf Zeilen. Sie löschen die letzte. Dann sind die Zeilenvariablen folgendermaßen belegt:

Ereignis	evtzeile	aktzeile	tzeile
zeileloevor	5	5	5
zeileloenach	5	4	4

Ein Datensatz habe fünf Zeilen. Sie befinden sich in der letzten und versuchen, eine weitere Zeile anzufügen.

Ereignis	evtzeile	aktzeile	tzeile
zeileloevor	6	5	5
zeileloenach	6	6	6

Ist in einem EFOP die Ereigniszeile nicht mehr oder noch nicht vorhanden, so wird stattdessen die letzte noch vorhandene Zeile geladen. Ist keine Zeile vorhanden, so wird der Kopfteil geladen. In diesem Fall (**aktzeile** = 0) kann auch nicht auf Zeilenvariable zugegriffen werden.

Steuerung des Dialogs durch EFOPs

Steuerung des Aussehens von Masken

Durch Setzen der Variablen **P|maskkontextfop** können Sie das Aussehen von Masken bei jedem Event an die aktuelle Situation **anpassen**.

Cursorsteuerung

.CURSOR 'Feldname' 'Zeilennummer' dient dazu, ein bestimmtes Maskenfeld anzuspringen.

Siehe auch Kapitel **.cursor** - **Cursorsteuerung**.

Betätigung eines Buttons per EFOP

Der Klick auf einen Button wird ausgelöst durch die Anweisung

.formel buttonvariable = "" Sie führt dazu, dass die Ereignisbehandlung des Abas Kerns für **buttonvariable** aufgerufen wird. Dies ist auch der Fall, wenn Sie auf den Button klicken. Jedoch bestehen einige sehr wichtige Unterschiede:

- Es werden, genau wie in EDP, ausschließlich die dialoglosen Abas Aktionen ausgeführt. Sie erhalten also

z.B. keine Rückfragen, und es werden keine Submasken geöffnet.

- Es werden keine EFOPs ausgeführt, da bereits ein FOP läuft.
- Es können nur Buttons der Art BU3 (Anwendungsbuttons im Kopfteil bzw. in der Tabelle) angeklickt werden.

Maske "Kunde", Feld **browurl** (URL im Browser anzeigen):

```
.formula M|callBrowserURL = ""
```

Ergebnis: Es geschieht nichts, da Anzeigen im Browser eine Dialogaktion ist.

Fertigungsvorschläge, Feld **ladetab** (Tabelle füllen)

```
.formula M|loadTab = ""
```

Ergebnis: Die Tabelle wird geladen, falls Daten vorhanden.

Das Drücken eines Buttons ist per FOP nur dann erlaubt, wenn es auch einem Benutzer erlaubt wäre. Andernfalls erscheint die entsprechende Fehlermeldung und das FOP scheitert. Ist ein Datensatz nur im **Festplattenzustand** geladen (wie z.B. beim Kommando **.hole**), so ist die Betätigung von Buttons nicht erlaubt, denn die Kernfunktionen sind in diesem Fall ohnehin nicht aktiv.

Exit-Status

Mit dem Kommando **.ende** kann ein Status in Form einer ganzen Zahl oder einer Variablen zurückgegeben werden.

- Status 0 bedeutet "OK".
- Status ungleich 0 (z.B. **. ende 1**) bedeutet Fehler.
- Status 10 wird standardmäßig zurückgegeben, wenn ein FOP vom Benutzer abgebrochen wird.

Fehlen der Exit-Status oder das **.ende**-Kommando, wird der Status 0 zurückgegeben.

Der Fehlerstatus hat folgende Auswirkungen auf den nachfolgenden Dialog.

Ereignis	Status	Bedeutung	Erläuterung
Maskeneintritt	1	Zugriff verweigert.	Dieser Datensatz darf nicht editiert werden.
Maskeneintritt	2	Kommando abgebrochen.	Dieser und (sofern angefordert) die folgenden Datensätze dürfen nicht editiert werden.

Ereignis	Status	Bedeutung	Erläuterung
Maskenprüfung	1	Falsch oder unvollständig.	Die Maske bleibt im Dialog. Das Speichern wird verweigert.
Maskenaustritt/Maskeneinde	1	Maske schließen.	Die Maske wird sofort geschlossen.
Maskenaustritt/Maskeneinde	2	Maske am Bereichsende schließen.	Nach der Bearbeitung aller angeforderten Datensätze wird die Maske geschlossen. Es wird also keine leere Maske angezeigt.
Feld ausgefüllt	1	Falscher Wert.	Das Feld bleibt im Bearbeitungsmodus.
Feldprüfung	1	Falscher Wert.	Das Feld bleibt im Bearbeitungsmodus.

Buttonereignis

Ereignis	Status	Bedeutung
Ereignis tritt vor der Abas Behandlung ein	1	Mit diesem Exit-Status kann die Abas Behandlung verhindert werden.

Zeilenbezogene Ereignisse in Tabellen

Ereignis	Status	Bedeutung
Löschen vor Abas Behandlung	1	Mit diesem Exit-Status kann das Löschen verhindert werden.
Einfügen vor Abas Behandlung	1	Mit diesem Exit-Status kann das Einfügen verhindert werden.

Der Fehlerstatus wird nicht durch Abas, sondern durch das FOP gemeldet. (Zur Erstellung einer Fehlermeldung vgl. die Beschreibung der Kommandos `.fehler` und `.hinweis` im Kapitel [Maskenbezogene Kommandos](#).) Wenn jedoch durch den Fehlerstatus eine Benutzeraktion unterbunden wird und im FOP keine Fehlermeldung und kein Hinweis angegeben wurde, erhalten Sie eine neutrale Fehlermeldung mit dem Wortlaut: "Aktion nicht erlaubt oder abgebrochen".

Die Meldung macht das Programmverhalten verständlicher, vor allem im Fall des Programmabbruchs (Fehlerabbruch, Benutzerabbruch), wo der EFOP-Programmierer gar keine Möglichkeit hat, eine Fehlermeldung abzusetzen.

Exit-Status bei abgebrochenem FOP

Wird ein FOP abgebrochen, so wird standardmäßig ein Exit-Status von 10 zurückgegeben. Ein vom Standard abweichender Wert lässt sich definieren durch

`.setze abbruch + n`

wobei n eine natürliche Zahl ist. "Abbruch nicht als Fehler werten" schreibt sich also

.setze abbruch + 0

Speichern und Fenster schließen per EFOP

Beim Ereignis **buttonnach** können Sie aus dem EFOP heraus Aktionen des Menüs Datei verwenden. Insbesondere können Sie den Datensatz speichern und/oder das Fenster schließen.

Verwenden Sie hierzu das FO-Kommando **.end 0 <File-Action>**. Für **<File-Action>** sind folgende sprachunabhängige Werte möglich:

File-Action	Entspricht GUI-Menüpunkt
FileSaveNext	
FileSaveClose	
FileCancelNext	
FileAbort	
FileCancelClose	

Wenn Sie Änderungen vorgenommen haben, dann werden Sie bei **Cancel**- und **Abort**-Aktionen gefragt, ob Sie speichern möchten. In diesem Fall können Sie aus dem EFOP heraus das Verwerfen von Änderungen oder das Beenden des zu editierenden Bereichs nicht erzwingen.



Die Aktion des Menüs Datei wird nur dann ausgeführt, wenn der Button vom Benutzer gedrückt wurde. Wenn der Button mit einem **Kommandoparameter** gedrückt wurde, dann wird die Aktion nicht ausgeführt.

Dialogzustand und Festplattenzustand eines Datensatzes

Alle Änderungen an Datenbankobjekten erfolgen über Zugriffsmethoden der jeweiligen Anwendung, z.B. der Bestellung, des Angebotes, des Artikel. Man spricht hierbei auch von Anwendungsmethoden. Sie stellen die Einhaltung der jeweiligen Anwendungslogik sicher. Anwendungsmethoden erfüllen im Einzelnen folgende Aufgaben:

- Zulässigkeit der Operation (z.B. Feldänderung, Zeile löschen, Zeile einfügen, Datensatz speichern) prüfen
- Operation ausführen
- Folgeaktionen ausführen (z.B. weitere Felder ändern, die vom geänderten Feld abhängig sind)
- ggf. Darstellung der Änderungen in der Maske

Im Dialogzustand sind diese Anwendungsmethoden aktiv. Der Dialogzustand liegt z.B. dann vor, wenn ein Datensatz in einer Maske editiert wird oder ein neuer Datensatz per **.mache** angelegt wird. Änderungen am Datensatz im Dialogzustand sind durch den Benutzer oder durch die FO möglich.

Im Festplattenzustand sind die Anwendungsmethoden nicht aktiv. Dafür sind aber auch nur Felder änderbar, deren Anwendungsmethoden in jedem Fall eine Änderung erlauben und keine Folgeaktionen auslösen. Dies wird aus der **Vartab**, Feld **FO-Schutz** ermittelt. Der Festplattenzustand liegt z.B. bei

Trefferprüf-Ereignis vor oder wenn ein Datensatz durch ein maskenloses Kommandos bearbeitet wird oder wenn er per **.hole** oder **.lade** geladen wurde. Änderungen am Datensatz im Festplattenzustand sind nur durch die FO möglich.



Ein Administrator mit den höchsten Zugriffsrechten kann zwar "FO-änderbar" auf "ja" setzen, dies darf aber nur nach Rücksprache mit der Forterro Deutschland Abas GmbH erfolgen. Die Konsistenz der Anwendung kann andernfalls verloren gehen und für eventuelle Datenschäden und deren Reparatur trägt der Ändernde die Verantwortung.

	Dialogzustand	Festplattenzustand
Sichtbarkeit von Feldern für FO	alle Felder sind sichtbar	wird durch die Vartab , Feld "FO-bekannt" bestimmt
Änderbarkeit von Feldern durch FO	wird durch die Anwendungsmethoden bestimmt	wird durch die Vartab , Feld "FO-änderbar" bestimmt
Zulässigkeit des Einfügens und Löschens von Zeilen durch FO	wird durch die Anwendungsmethoden bestimmt	nur bei Zusatztabelle und bei .mache zulässig
Folgeaktionen	werden durch Anwendungsmethoden ausgeführt	werden nicht ausgeführt, dafür sind aber auch nur Operationen ohne Folgeaktion zulässig

Mehrsprachige Programmierung

Die **FO-Programmiersprache**, also die Sprache, in der FO-Schlüsselwörter anzugeben sind, kann Deutsch oder Englisch sein. Sie wird über die **Konfiguration** eingestellt. Sie können diesen Konfigurationseintrag außer Kraft setzen, indem Sie in der ersten Zeile des betreffenden FOPs nach **..!interpreter** die Sprache angeben. Der **Datenbanklader** startet immer in Deutsch.

Sprachabhängige Listen und Ausdrücke

Wenn FO-Programme für die Installation vorbereitet werden, ist die gewünschte Sprache für ihre Ausgabe noch nicht bekannt. Daher hält Abas die Standardlisten bzw. -ausdrücke in den angebotenen Sprachen vor. Dabei wird auf verschiedene Verzeichnisse zurückgegriffen. Die Listen für die englische Ausgabe liegen z.B. im Verzeichnis **E**. Daran werden die Verzeichnisse für **ek**, **vk** usw. angehängt, also **E/vk**, **E/ek** usw. Texte des Grundbereichs können ebenfalls sprachabhängig sein, z.B. **E/DRUCKER**. Die Regel lautet also: Bei sprachabhängigen Listen wird das Sprachkürzel als Verzeichnisname vorangestellt.

Wenn Sie FO-Programme erfassen, die sprachabhängig sind, müssen Sie beim Ablegen den gesamten Namen angeben. Das Programm nimmt keine impliziten Ergänzungen vor. Wenn ein FOP gesucht wird (mit **<Text> ... <Zeigen>** oder **.eingabe ...**), so geschieht das zuerst sprachabhängig, dann sprachunabhängig. Haben Sie Englisch eingestellt, wird "name" also in der Reihenfolge **E/name E/*/name name */name** gesucht. Dabei steht der Stern für alle Verzeichnisse, die mit einem Passwort verknüpft sind. Innerhalb der Sprachverzeichnisse wird in der Reihenfolge gesucht, in der die zugänglichen Verzeichnisse in den **Passwortdefinitionen** angegeben sind. Zum Anlegen von sprachabhängigen **ow**-Verzeichnissen siehe unter **Arbeitsbereiche und Texte bearbeiten**.

Wenn Sie den Textnamen ohne Angabe eines Sprachverzeichnisses angegeben (z.B.: **st/LKU**, **ow1/PROG**), so wird dieser Text ausschließlich im geöffneten Bediensprachenverzeichnis gesucht. Zusätzlich aber kann der Text durch Voranstellen der Bediensprache in jedem beliebigen Bediensprachenverzeichnis und durch Voranstellen von **D/** auch im Installationssprachenverzeichnis geändert werden.



Bediensprache sei Englisch:

```
<text> X <edit> ändert E/X  
<text> E/X <edit> ändert E/X  
<text> D/X <edit> ändert X
```

Dateinamen, die in **fop.txt** auf ein **.eingabe**-Kommando folgen, werden im EFOP zuerst im Bediensprachenverzeichnis und danach im Installationssprachenverzeichnis gesucht. EFOPs, die sprachabhängig sind, müssen unbedingt auch in die verschiedenen Bediensprachen übersetzt werden, um dort laufen zu können.

FOPs und Mehrsprachigkeit

Sprachabhängige FOPs stehen in eigenen Sprachenbereichen, die jeweils durch ein **Sprachkürzel** benannt sind. Der Sprachenbereich wird dem Verzeichnis vorangestellt. FOPs können auch ohne eine englische

Fassung im E-Verzeichnis ihre Texte in Englisch ausgeben.

<Text> XXX <Ändern> bzw. <Text> XXX <Neu> startet automatisch die Übersetzung des Textes XXX in die installierten **Bediensprachen**.

Daneben stehen die Funktionen des Übersetzungswerkzeugs **trans** zur Verfügung. Damit können neue Texte aus FOPs an das Wörterbuch angehängt werden.

Wenn es sich um einen mehrsprachigen Mandanten handelt, dann wird zusätzlich Deutsch als weitere Bediensprache installiert und dort wo z.B. ein E-Verzeichnis (E für Englisch) vorhanden war, ein D-Verzeichnis (D für Deutsch) hinzugefügt.

Wenn Sie die Quellsprache einer Quelldatei in FOPs angeben wollen, benutzen Sie in der **..!interpreter**-Zeile den Eintrag der Form **src=iso-Kürzel**, also z.B. **src=de**.



Datendateien bzw. Temporärdateien werden oftmals in dasselbe Verzeichnis gespeichert wie FOPs. Da sie durch ihren Namen nicht von diesen zu unterscheiden sind, werden sie von **trans** als solche behandelt. Auf diese Weise wird **msg.lg.dic** aufgebläht, insbesondere dann, wenn Sie einen der trans-Menüpunkte [FO-Programme eines bestimmten Arbeitsbereichs in Wörterbuch aufnehmen] oder [FO-Programme aller **ow**-Arbeitsbereiche in Wörterbuch aufnehmen] wählen. Deswegen müssen alle Dateien, die kein FOP sind, in eigene Verzeichnisse gespeichert werden, deren Name nicht mit **ow** beginnt. Außerdem sollte der Menüpunkt [FO-Programme aller **ow**-Arbeitsbereiche in Wörterbuch aufnehmen] in **trans** mit Bedacht gewählt werden.

Wörterbuch für die Bedeutung von Standardvariablen

Über [**Vartab**-Bedeutungen] im Menü des Programms **trans** lässt sich das Wörterbuch **msg.vt.dic** verwalten. Da keine eigenen Texte hinzugefügt werden können, ähnelt das darunterliegende Menü dem Menü [Programm-Meldungen].



Texte werden im mehrsprachigen Abas so in das Wörterbuch **msg.lg.dic** aufgenommen, wie sie in der FO-Quelle vorliegen. Variablen, die am Ende einer Zeile stehen und nicht übersetzt werden, werden nicht abgeschnitten. Der Vorteil dieses Verfahrens ist, dass Tabellenstrukturen besser zu erkennen sind und die Übersetzung so auszurichten ist, dass ein rechter Flatterrand vermieden werden kann.

Eigene FO-Texte können nicht angepasst werden. Es empfiehlt sich daher, die eigenen FOP-Texte mit **trans** an das FO-Wörterbuch anzuhängen, um die neuen unübersetzten Texte erkennen und bearbeiten zu können.

Übersetzungslogik bei FOPs

Bei **.kommando** können das Kommando und die Klammern separat aufgeführt werden, damit das Kommando korrekt in die Bediensprachen übersetzt werden kann.

```
.type text xttext  
.formula U|xttext = "<" + "(" + "Customer" + ")" + ">"  
display: 'U|xttext'
```

Besser ist jedoch ein Vorgehen dieser Art:

Statt separater Anweisung kann also besser folgendermaßen geschrieben werden:

```
.type text xttext  
.formula U|xttext = "<(Customer)>"
```

Diese Übersetzungslogik gilt aber nicht für Ausgaben. Zum Beispiel wird <Kunde> nicht übersetzt, weil davon ausgegangen wird, dass ein Laderskript generiert werden soll, dessen Übersetzung unerwünscht ist.

Individuelle FOPs führen bei folgender Konstellation zu Fehlern: Existiert eine Anweisung der Art

```
.formel U|xttext = "<Kunde>"
```

und wird 'U|xttext' anschließend in eine Datei (Laderskript) geschrieben, so wird "<Kunde>" übersetzt und das Laderskript ist bei falscher Bediensprache nicht mehr lauffähig. Probleme entstehen jedoch nur dann, wenn die sprachunabhängige FO-Programmierung vergessen wurde. In diesem Fall muss mit **.setze trans -/+** nachgebessert werden.

Übersetzung der FO-Befehle eines FOPs

Wenn eine Übersetzungsanforderung gestellt wird, kann ein FOP auf die Befehlssprache Englisch umgestellt werden. Eine Übersetzungsanforderung kann auf folgende Arten gestellt werden:

- durch Aufruf des Übersetzungsprogramms **trans** (für ein einzelnes oder alle FOPs)
- durch Konfigurieren der FO-Sprache Englisch (für alle FOPs)
- durch das Update, wenn die FO-Sprache Englisch konfiguriert ist (für alle vorgefundenen und alle neu installierten FOPs).

Bei einer Übersetzungsanforderung wird ein FOP auf die Befehlssprache Englisch umgestellt, wenn in seiner **interpreter**-Zeile **german** und **translate** angegeben ist (Reihenfolge beliebig). Es werden auch alle FOPs übersetzt, die keine **interpreter**-Zeile enthalten. In diese Dateien wird die Zeile **..!interpreter english** eingetragen.

Ein FOP wird *nicht übersetzt*, falls es die **interpreter**-Zeile **german ohne translate** enthält. Auf diese Weise kann ein FOP dauerhaft mit deutschen Kommandos in einer englischen Umgebung verbleiben.

Es werden nur Befehlszeilen umgestellt, und in diesen nur Befehle und Optionen. Das Übersetzen von Variablenamen ist nicht möglich.

FO-Variablen bei Mehrsprachigkeit

isprache	Kürzel der Installationssprache (derzeit immer "D")
ispri	ISO-Code der Installations- oder Landessprache
bsprache	Kürzel der Bediensprache
bspri	ISO-Code der Bediensprache
lsprache	Kürzel der FO-Sprache der aktuellen Programmdatei (im JFOP leer)
lspri	ISO-Code der FO-Sprache der aktuellen Programmdatei (im JFOP leer)
esprache	Kürzel der Excel-Sprache
espri	ISO-Code der Excel-Sprache

Folgende Variablen werden bei **.hole** und **.dazu** bei einem Kunden- oder Lieferantendatensatz automatisch belegt:

kspr	Kürzel der Korrespondenzsprache
kspri	ISO-Code der Korrespondenzsprache
ksi	Korrespondenzsprachenindex: Laufende Nummer der Korrespondenzsprache, wie in der Sprachentabelle in [Konfiguration zeigen] dargestellt. Die Korrespondenzsprachen sind wie die Bediensprachen durchnummeriert.  Die FO ersetzt 'tebez.ksi' durch 'tebez1' bzw. 'tebez2' usw.



Wenn beim Editoraufruf aus einem FOP heraus die **Bediensprache** ('**bspri**') des Benutzers berücksichtigt werden soll, muss der Inhalt von '**bspri**' abgefragt werden, damit der Editor in der eingestellten Bediensprache arbeitet.

Entwicklungswerkzeuge

Weitere Entwicklungswerkzeuge finden Sie in den [Abas Tools](#). Diese liegen hier auf [Deutsch](#) und [Englisch](#) zum Download bereit.

Programm zur Auflistung aller FO-Kommandos

Eine vollständige Liste aller FO-Kommandos sowie Listen aller Parameter und aller Optionen lassen Sie auf der Shell ausgeben mit dem Programm

```
getfocmd [-s de|en] [-a|-k <FO-Kommando>]
```

-s de	FO-Kommandosprache Deutsch
-s en	FO-Kommandosprache Englisch
-a	alle Kommandos, ohne Optionen
-k <FO-Kommando>	nur das angegebene Kommando mit seinen Optionen

FO-Batchbetrieb

Für den FO-[Batch](#)-Betrieb stehen 4 Programme zur Verfügung:

runfop.sh	Programm zum unmittelbaren Start von FO-Programmen von der Shell aus
batchlg.sh	Programm zum unmittelbaren Start von FO-Programmen von der Shell aus (alt)
batchlg.spooler.sh	Programm zum Start von FO-Programmen und Ladem über eine Auftragswarteschlange (Spooler)
batchsperr.sh	Programm zur Synchronisation von Shellprogrammen und FO-Programmen

Ausführen von FO-Programmen - runfop.sh

[runfop.sh](#) ist ein Shellskript, mit dem ein FOP gestartet werden kann. Der Aufruf erfolgt mit

[runfop.sh](#) [-a args] [-i input]... [-l lang] [-p password] [-d] FOP-Name Das Shellskript beendet sich, wenn das angegebene FO-Programm abgearbeitet ist oder ein Fehlerabbruch erfolgte.

Das FO-Programm [FOP-Name](#) wird im Batch ausgeführt. Dabei gelten folgende Regeln:

- Sollte das aufgerufene FOP interaktive Eingaben erfordern (FO-Kommandos [.lesen](#), [.box](#), [.menue](#)), müssen diese als Dialogantworten mit der Option [-i](#) mitgegeben werden. Diese Option kann mehrfach angegeben werden, einmal je Dialogantwort. Es genügt, die Eingabestrings in der richtigen Reihenfolge jeweils mit der Option [-i](#) anzugeben. Durch [-i](#) wird der Dialogmodus aktiviert; das FOP läuft dann nicht im reinen Batchmodus.

- Bei interaktivem Aufruf kann das Abas Passwort durch den Benutzer unsichtbar eingegeben werden. Batch-Aufrufe müssen entweder mit einem Passwort erfolgen, das mit der Option **-p** anzugeben ist, oder ein automatischer Login muss möglich sein. Dafür gibt es folgende Möglichkeiten:
 - Der Aufruf erfolgt mit einem **Systemlogin**, das als Login in einem aktiven Passwortsatz eingetragen ist.
 - Der Aufruf erfolgt in einem Kontext, der ein Einmalpasswort weitergibt, z.B. mit dem FO-Kommando **.system ... -PASSWORT**.
- Argumente für das aufgerufene FOP können mit der Option **-a** übergeben werden. Dabei gilt, dass mehrere Argumente für das FOP in einer einzelnen Zeichenfolge und mit Leerzeichen getrennt zu übergeben sind, z.B. **-a "arg1 arg2 arg3"**.
- **runfop.sh** muss in einem Mandantenverzeichnis gestartet werden.

Dialogmodus vermeiden



Alternativ zur Dialogabfrage mit **-i** können die Werte auch als benannte Parameter mit **-a** übergeben werden (siehe [Parameter weiterreichen](#)). So müssen sie nicht im Dialog eingelesen werden, und der Dialogmodus wird nicht aktiviert.

Ausgabesteuerung

Die Ausgabe des FOPs erfolgt über den Standardausgabestrom **stdout** (UNIX-Strom 1) und kann somit auch in eine Datei geleitet und dort gespeichert werden. Eine FO-Ausgabe auf den Bildschirm läuft durch. Es erfolgt keine Unterbrechung der Ausgabe mit der Meldung **"- weiter mit <Return>"** wie in der GUI.

Fehlermeldungen des FOPs (**. fehler**) erfolgen über den Fehlerausgabestrom **stderr** (UNIX-Strom 2) und können ebenfalls in eine Datei ausgegeben werden. Diese Datei kann dieselbe sein, in die auch **stdout** geleitet wird, oder eine separate.

- FO- und Fehlerausgabe in separaten Dateien:

```
runfop.sh -i anton -i karl AUSWERT > AUSWERT.AUS 2> AUSWERT.ERR
```

+ Das Programm **runfop.sh** ruft das FO-Programm **AUSWERT** mit den Parametern **anton** und **karl** auf, das die Parameter mit **.lesen** abfragt. Die Ausgabe erfolgt in die Datei **AUSWERT.AUS**, die Fehlerausgabe in die Datei **AUSWERT.ERR**.

- FO- und Fehlerausgabe in dieselbe Datei:

```
runfop.sh -i anton -i karl AUSWERT > AUSWERT.AUS 2>&1
```

Das Programm wird wie oben ausgeführt. Ergebnis und Fehler werden jedoch zusammen in die Datei **AUSWERT.AUS** ausgegeben.

Parameter weiterreichen

Mit der Option **-a** können zusätzliche Argumente an das FOP weitergereicht werden, so dass sie dort nicht im Dialog eingelesen werden müssen.

```
runfop.sh -a "=" xmin=1 xmax=10" FOP-Name
```

Damit sind im FOP die beiden Variablen '**xmin**' und '**xmax**' gesetzt.

Bediensprache setzen

Mit der Option **-l** kann die Bediensprache angegeben werden, unter der das FOP ausgeführt wird, z.B. **-l en**.

Fehler beim Batchbetrieb mit runfop.sh

Im Folgenden finden Sie eine Auflistung möglicher Exit-Status, die beim Einsatz von **runfop.sh** auftreten können:

Exit-Status	Bedeutung
1	Programm mit Fehler beendet, z.B. wegen fehlerhaftem Aufruf oder einem Problem beim Programmstart.
9	Systemfehler. Vermutlich wurde eine Diagnosemeldung ausgegeben.
sonstige ungleich 0	Das FO-Programm konnte nicht korrekt ausgeführt werden oder beendete sich aufgrund eines "normalen" Fehlers selbst mit einem Fehlerstatus mittels .ende 1 .  Unterschied zu batchlg.sh Bei einem Fehler im FOP kehrt das Shellskript batchlg.sh mit exit status 0 zurück.

Ausführen von FO-Programmen (alt) - batchlg.sh

batchlg.sh ist ein Shellskript, mit dem ein FOP gestartet werden kann. Der Aufruf erfolgt mit

batchlg.sh FOP-Name [-PASSARGS] [Parameter ...] Das Shellskript beendet sich, wenn das angegebene FO-Programm abgearbeitet ist.

Das FO-Programm **FOP-Name** kann mit den durch die Parameter vorgegebenen Bedingungen im Batch laufen, sofern folgende Voraussetzungen erfüllt sind:

- Das aufgerufene FOP darf keine interaktiven Eingaben erfordern.

- Der Aufruf muss entweder mit einem **Systemlogin** erfolgen, das als Login in einem aktiven Passwortsatz eingetragen ist, oder die Umgebungsvariable **EKSPASSWORT** muss mit einem gültigen Abas Passwort belegt sein.
- **batchlg.sh** muss aus dem Mandantenverzeichnis oder mit einer geeigneten Belegung der Umgebungsvariablen '**MANDANTDIR**' gestartet werden.

Die angegebenen Parameter stehen dem Kommando **.lesen** in je einer Zeile zur Verfügung, es sei denn **-PASSARGS** wurde angegeben.

Die Ausgabe der ASCII-Oberfläche erfolgt über den Standardausgabestrom **stdout** (UNIX-Strom 1) und kann somit auch in eine Datei geleitet und dort gespeichert werden. Eine Ausgabe auf den Bildschirm läuft durch, eine Unterbrechung der Ausgabe mit der Meldung "- weiter mit <Return>" erfolgt nicht.

Fehlermeldungen erfolgen über den Fehlerausgabestrom **stderr** (UNIX-Strom 2) und können ebenfalls in eine Datei ausgegeben werden. Diese Datei kann dieselbe sein, in die auch **stdout** geleitet wird, oder eine separate.

```
batchlg.sh AUSWERT anton karl > AUSWERT.AUS 2> AUSWERT.ERR
```

Das Programm **batchlg.sh** ruft das FO-Programm **AUSWERT** mit den Parametern **anton** und **karl** auf, das die Parameter mit **.lesen** abfragt. Die Ausgabe erfolgt in die Datei **AUSWERT.AUS**, die Fehlerausgabe in die Datei **AUSWERT.ERR**.

```
batchlg.sh AUSWERT anton karl > AUSWERT.AUS 2>&1
```

Wie oben, Ergebnis und Fehler werden jedoch zusammen in die Datei **AUSWERT.AUS** ausgegeben.



Eine so erzeugte Ausgabedatei wird in der Regel auch Bildschirmsteuersequenzen beinhalten. Soll sie frei davon sein, muss die Ausgabe über ein **.ausgabe**-Kommando im FOP selbst erfolgen.

Parameter weiterreichen

Die Option **-PASSARGS** bei **batchlg.sh** veranlasst, dass zusätzliche Argumente an das FOP weitergereicht werden, so dass sie dort nicht im Dialog eingelesen werden müssen.

```
batchlg.sh -PASSARGS FOP-Name = xmin=1 xmax=10
```

Damit sind im FOP die beiden Variablen '**xmin**' und '**xmax**' gesetzt.

Laderstart mit batchlg.sh

Mit **batchlg.sh** können Sie auch **Datenbanklader** starten. Dazu dient das Standard-FO-Programm **LESELADER**, an das der Name des zu startenden Laders zu übergeben ist. Folgendes Laderskript soll ausgeführt werden:

```
<Kunde> 1
<Ändern>
'tele 0721/123456-0
<Fertig>
```

Sie führen den Lader mit folgendem Aufruf auf der Shell aus:

```
batchlg.sh LESELADER BATCHLADER > BATCH.AUS 2> BATCH.ERR
```

Auch hier empfiehlt es sich, die Ausgabe auf Datei umzuleiten.

Fehler beim Batchbetrieb mit batchlg.sh

Im Folgenden finden Sie eine Auflistung möglicher Exit-Status, die beim Einsatz von **batchlg.sh** auftreten können:

Exit-Status	Bedeutung
1	Programm mit Fehler beendet, z.B. EOF in der Eingabe, obwohl noch Eingabe erwartet wurde.
3,4	Fehler beim Starten der Datenbank Programm wurde nicht gestartet. Mögliche Ursache bei gelegentlichem Auftreten: Betrieb wegen Wartung, Wiederanlauf, Nachtsicherung unterbrochen. Bei dauerhaftem Auftreten: Umgebungsvariable ungeeignet oder Zugriffsrechte unzureichend.
8	Signal eingeschlagen (Systemfehler oder SIGHUP, SIGPIPE) Ursache meist Wiederanlauf, evtl. Abmelden des interaktiven Benutzers, der das Programm gestartet hat (nohup fehlt?)
9	Rekursiver Systemfehler. Vermutlich wurde eine Diagnosemeldung ausgegeben.

sonstige ungleich 0	Programm durch Fehler beendet. Eventuell wurde eine Diagnosemeldung ausgegeben.  Wenn es lediglich zu "normalen" Fehlermeldungen bei der Abarbeitung eines Laderskriptes kommt, wird das Programm normal beendet und kein Exit-Status ungleich 0 gesetzt, siehe Fehlerbehandlung im Lader.
---------------------	---

batchlg.spooler.sh

Das Shellskript batchlg.spooler.sh bietet Funktionen, um FO-Programme oder Laderskripte kontrolliert über den Batchspooler zu starten. Damit können Aufrufe von batchlg.sh indirekt abgesetzt werden, d.h. der Aufruf von **batchlg.sh** erfolgt nicht unmittelbar, sondern zu einem vom Batchspooler festgelegten Zeitpunkt. Dieser kann je nach Rechnerbelastung quasi sofort oder auch erst deutlich später sein.

batchlg.spooler.sh gibt dem **Batchspooler** den Auftrag, ein FOP oder ein Laderskript über batchlg.sh zu starten. Der Spooler arbeitet diese Aufträge dann stets einen nach dem anderen ab. Durch diesen Umweg wird erreicht, dass Aufträge immer abgegeben werden können, auch dann, wenn das System nicht läuft, sondern z.B. Wartungseingriffe durchgeführt werden oder die Nachtsicherung aktiv ist. Aufträge werden sukzessive bearbeitet. Dadurch entsteht auch dann nur eine begrenzte Last auf dem Rechner, wenn sehr viele Aufträge gleichzeitig aufgegeben werden. Die Bearbeitung erfolgt in einer kontrollierten Umgebung. Fehler und Probleme werden protokolliert. (Weitere Informationen zum Batchspooler finden Sie im Kapitel [Batchspooler](#).)

Optionen

-f 'FOP-Name'	Das FO-Programm ' FOP-Name ' wird ausgeführt
-l 'Ladername'	Das Laderprogramm ' Ladername ' wird ausgeführt. Beachten Sie auch den unten stehenden Hinweis zum Aufruf des FOPs LADER. ENDE , der im Laderskript enthalten sein muss.
-s 'Skriptname'	Das Shellskript ' Skriptname ' wird ausgeführt. Es muss für den Batchbenutzer Ausführungsrechte besitzen und im Suchpfad des batchlg-Benutzers liegen, oder beim Aufruf muss der vollständige Pfad zum Skript angegeben werden. Um Shellskripte zu starten, ist kein FO-Programm oder Lader notwendig.
-x <Spoolername>	Es wird ein anderer Spooler als der Standardspooler " spooler " verwendet. Verschiedene Spooler werden über einen frei wählbaren Namen unterschieden und können parallel aktiv sein.
-b	Bewirkt einen automatischen Start des Spoolers, sofern er nicht bereits läuft.

-c	Vor dem Start des Jobs wird eine Kopie der angegebenen Datei im Spool-Verzeichnis erstellt, und das Original kann sofort nach dem Abschicken des Auftrags wieder gelöscht werden. Ohne diese Option darf die angegebene Datei erst nach dem Ende des Jobs gelöscht werden.
-p <Passwort>	Abas Passwort, unter dem der Spoolerauftrag laufen soll (nicht bei -s).
-n <Sequenznummer>	Frei wählbare Sequenznummer, um die Aufträge in einer anderen Reihenfolge als in der ihrer Erstellung abarbeiten zu lassen. Die Sequenznummer darf Ziffern und Buchstaben und auch Sonderzeichen enthalten. Die Reihenfolgebildung erfolgt lexikalisch über das Dateisystem. Da demnach zum Beispiel 9 nach 100 käme, sollten Sie bei einer numerischen Sequenzbezeichnung stets gleich viele Ziffern für alle Sequenznummern und ggf. auch führende Nullen verwenden (009 und 100 statt 9 und 100). Ferner sollten Sie keine Zeichen in der Sequenznummer benutzen, die in Dateinamen problematisch sind.

Starten des Laders BATCHLADER über den Batchspooler. Laderskriptinhalt:

```
<Kunde> 1
<Ändern>
'tele 0721/123456-0
<Fertig>
<Text> LADER.ENDE
<Zeigen>
```



Am Ende eines über den Batchspooler gestarteten Laders muss das FOP **LADER . ENDE** aufgerufen werden, da der Batchspooler nur dann das fehlerfreie Ende des Laders erkennen kann.

Erstellen des Spooler-Auftrags:

batchlg.spooler.sh -l BATCHLADER Das Ergebnis des Laderauftrags kann in der Datei **batchlg.ctrl.log** im Unterverzeichnis **batchlg.spooler** des Mandanten eingesehen werden. Dort müsste nach erfolgreicher Ausführung des Laders etwa Folgendes stehen:

```
START:10.11.2009-12:35.06# batchlg.spooler/cf...LINKSux: Lader # BATCHLADER ENDE
:10.11.2009-12:35.06# batchlg.spooler/cf...LINKSux: Lader # BATCHLADER
```

Aufträge für den batchlg-Spooler aus der FO mit 'AUZEICHEN'

Den Aufträgen, die Sie für den **batchlg**-Spooler aus der FO heraus aufgeben, werden über die Umgebungsvariable '**AUZEICHEN**' das Bearbeiterzeichen des Auftraggebers mitgegeben. Damit

sehen Sie, sofern der Spooler mit einem festen Passwort arbeitet, zusätzlich zum Bearbeiterzeichen des Ausführenden auch das Bearbeiterzeichen des Auftraggebers an allen Stellen, an denen das Bearbeiterzeichen 'G|bzeichen' verwendet wird. Der Batchläufer hat als Zeichen "Eigenes Zeichen/Auftraggeber-Zeichen".

Der Sachbearbeiter "sa" startet per FO einen Spooler-Auftrag. Der Spooler arbeitet mit dem Passwort "batchlg" und dem Zeichen "batch". Der Auftrag erscheint dann z.B. im Kommandologbuch unter dem Zeichen "batch/sa".

Für direkt gestartete Batchläufer verwenden Sie besser ein [Einmalpasswort](#).

Fehler beim Batchbetrieb mit batchlg.spooler.sh

Im Folgenden finden Sie eine Auflistung möglicher Exit-Codes von Fehlern, die beim Einsatz von `batchlg.spooler.sh` auftreten können. Sie werden in der Datei `batchlg.ctrl.log` hinter "result:" protokolliert. Die Fehler kommen in der Regel vom aufgerufenen Programm. Siehe auch [Fehler beim Batchbetrieb mit batchlg.sh](#).

Fehlernummer	Bedeutung
11	Es fehlt die Datei <code>LADER.ENDE.MARKE</code> , das heißt, der Lader hat das normale Programmende nicht erreicht.
12	Es wurden Fehler in der Fehlerdatei protokolliert.

batchsperr.sh

`batchsperr.sh` erlaubt es, eine Sperre zu setzen und zu prüfen, die auch mit dem FO-Kommando `.belegen` geholt werden kann. Dadurch können Shellprogramme mit FO-Programmen synchronisiert werden. Es gibt folgende Optionen:

```
usage: batchsperr.sh [-s|p|e|r|w] [ -i ] [ -z dn sn rep | -t lock | -l dn sn ] [ -a ] [ -W anz ] [
-d sec ] [ -x script ]
{{ * Option -s: Set lock and check, can call several times}}
{{ * Option -p: Check if there is a lock Result: ==0 Lock is (has been) set}}
{{ * Option -e: End lock process}}
{{ * Option -w: as s but wait for the lock}}
{{ * Option -W: Wait until <anz> processes wait for lock}}
{{ * Option -r: Read get lock}}
{{ * Option -o; Select edit lock}}
{{ * Option -z sperre rep limit: set counting locks, x means no limit}}
{{ * Option -t lock: String lock}}
{{ * Option -l dn sn: Normal lock}}
{{ * Option -x script: (always the last option!) execute script under lock}}
{{ * Option -S set lock, result 2 if lock could not be occupied}}
{{ * Option -a: Output arity of lock}}
{{ * Option -m: Special lock for clients}}
{{ * Option -i: Lock for installation}}
{{ * Option -d: Enter the waiting period in sec for -w -x option}}
```

-s	Setzen einer Sperre (kleines "s") (zusammen mit -t oder -x). Der Aufruf kann mehrfach erfolgen. Wenn die Sperre bereits besteht, ist die Wirkung dieselbe wie bei der Option -p. Das Ergebnis des Aufrufs ist: <table border="1" data-bbox="804 394 1458 577"> <tr> <td>0</td><td>Die Sperre ist gesetzt.</td></tr> <tr> <td>2</td><td>Die Sperre besteht nicht.</td></tr> <tr> <td>sonst</td><td>Ein Fehler ist aufgetreten.</td></tr> </table> batchsperr.sh mit -s ohne weitere Option belegt Stringsemaphor "BATCH".	0	Die Sperre ist gesetzt.	2	Die Sperre besteht nicht.	sonst	Ein Fehler ist aufgetreten.
0	Die Sperre ist gesetzt.						
2	Die Sperre besteht nicht.						
sonst	Ein Fehler ist aufgetreten.						
-S	Setzen einer Sperre (großes "S") (zusammen mit -t oder -x). Wirkt wie -s (kleines "s"), liefert aber den Exit-Status 2, wenn die Sperre nicht durch diesen Aufruf gesetzt wurde. Der Aufruf kann mehrfach erfolgen. Er liefert aber nur beim ersten Mal den Wert 0. <table border="1" data-bbox="804 954 1458 1169"> <tr> <td>0</td><td>Die Sperre ist gesetzt.</td></tr> <tr> <td>2</td><td>Die Sperre konnte nicht gesetzt werden.</td></tr> <tr> <td>sonst</td><td>Ein Fehler ist aufgetreten.</td></tr> </table>	0	Die Sperre ist gesetzt.	2	Die Sperre konnte nicht gesetzt werden.	sonst	Ein Fehler ist aufgetreten.
0	Die Sperre ist gesetzt.						
2	Die Sperre konnte nicht gesetzt werden.						
sonst	Ein Fehler ist aufgetreten.						
-p	Prüfung, ob die Sperre zum Zeitpunkt der Prüfung gehalten wird (zusammen mit -t oder -x). Das Ergebnis des Aufrufs ist: <table border="1" data-bbox="804 1330 1458 1509"> <tr> <td>0</td><td>Die Sperre ist gesetzt.</td></tr> <tr> <td>2</td><td>Die Sperre besteht nicht.</td></tr> <tr> <td>sonst</td><td>Ein Fehler ist aufgetreten.</td></tr> </table>	0	Die Sperre ist gesetzt.	2	Die Sperre besteht nicht.	sonst	Ein Fehler ist aufgetreten.
0	Die Sperre ist gesetzt.						
2	Die Sperre besteht nicht.						
sonst	Ein Fehler ist aufgetreten.						
-e	Aufheben der Sperre (zusammen mit -t oder -x). Das Ergebnis des Aufrufs ist: <table border="1" data-bbox="804 1637 1458 1899"> <tr> <td>0</td><td>Falls eine Sperre bestand, wurde sie beendet. Falls keine Sperre bestand, wird dies bestätigt.</td></tr> <tr> <td>sonst</td><td>Ein Fehler ist aufgetreten.</td></tr> </table>	0	Falls eine Sperre bestand, wurde sie beendet. Falls keine Sperre bestand, wird dies bestätigt.	sonst	Ein Fehler ist aufgetreten.		
0	Falls eine Sperre bestand, wurde sie beendet. Falls keine Sperre bestand, wird dies bestätigt.						
sonst	Ein Fehler ist aufgetreten.						

-x 'Skript mit Argumenten'

Ausführen eines Skripts unter dem Schutz einer Sperre aus. Die Sperre ist belegt, solange das Skript läuft, d.h. die Ausführung des Kommandos

.belegen in einem FOP wird währenddessen blockiert. Ist die Sperre bereits belegt, wird auf das Ende der Sperre gewartet und dann das Skript ausgeführt.

Die Option **-t** legt fest, welche Sperre verwendet werden soll. Ohne nähere Angabe wird die Standardsperre verwendet. **'Skript mit Argumenten'** steht für die komplette Aufrufzeile eines Skripts, das unter dem Schutz der Sperre gestartet werden soll.

Verwenden Sie auf der Shell nur Großbuchstaben. Das Argument bei **.belegen** wird entsprechend umgewandelt. Bei



batchsperr.sh wird nicht umgewandelt. Geben Sie ferner die komplette Aufrufzeile für das Skript als ein einziges Argument an. Sie erreichen dies durch das Umschließen mit Apostrophen oder mit Anführungszeichen.

Der folgende Aufruf von **batchsperr.sh** hat dieselbe Wirkung wie das sich anschließende FOP:

```
batchsperr.sh -t "SPERRE" -x 'Skript
mit Argumenten'
```

```
.type text xtscript
.formula U|xtscript = "script with
arguments"
.occupy "writelock"
.system 'U|xtscript' background
.occupy
```

-t 'Sperrwort'

Setzt (-s), beendet (-e) oder prüft (-p) die angegebene Sperre.

Diese Sperre kann auch mit **.belegen** gesetzt und geprüft werden. Auf diese Weise ist es möglich, beliebige Aktionen auf der Shell mit FO-Programmen zu synchronisieren und eine Sperre in EFOPs zu setzen, die das EFOP überdauert. Beim Setzen der Sperre mit **.belegen 'Sperrwort1'** bleibt das Semaphor "Sperrwort1" gesperrt bis zum Aufheben der Sperre durch **.belegen** oder setzen eines anderen Semaphores mit **.belegen 'Sperrwort2'** oder durch Ende des EFOPs. Dagegen bleibt die mit **batchsperr.sh -S -t 'Sperrwort'** erzeugte Sperre auch nach Programm- und Prozessende bis zum expliziten Aufheben mit **batchsperr.sh -e -t 'Sperrwort'** erhalten.



Verwenden Sie auf der Shell nur Großbuchstaben. Das Argument beim Kommando **.belegen** wird entsprechend umgewandelt. Bei **batchsperr.sh** wird nicht umgewandelt.

Eine Maske soll zu einem Zeitpunkt nur von höchstens einem Benutzer geöffnet werden können.

Maskeneintritts-FOP:

Sperre dauerhaft extern belegen mit dem Skript **batchsperr.sh**. (Nur

batchsperr.sh -S liefert ein Ergebnis, wenn die Sperre vorher schon belegt war!)

```
.type text xtsys
.formula U|xtsys = "batchsperr.sh -S -t MASKE4711"
.system 'U|xtsys' background
?lock formula MASKE4711 active ?
G|success = false
.end 1 ? G|success = G|false
.. batchsperr.sh lock is active
```

Maskenende-FOP:


```
-i -t 'Stringsemaphor' -S
```

Mit diesem Kommando wird eine [installationweite Sperre](#) gesetzt. Sie gilt für alle Mandanten einer Installation.



batchsperr.sh kann mehrfach aufgerufen werden. Die Aufrufe laufen asynchron: Das Ergebnis einer Prüfung mit der Option **-p** kann 0 sein, weil ein weiteres **batchsperr.sh** mit der Option **-e** die soeben gesetzte Sperre bereits wieder aufgehoben hat, bevor die Prüfung greifen konnte. Die Reihenfolge mehrerer **batchsperr.sh** kann nicht beobachtet werden.

Variablen in FOPs finden

Das Shellskript **vargrep.sh** sucht einen oder mehrere Variablennamen in allen oder nur in den kundeneigenen FO-Texten. Der Aufruf lautet:

```
vargrep.sh [-i] 'Variablenamendatei'
```

Mit **vargrep.sh** werden die Variablennamen, die in der Variablenamendatei aufgeführt sind, in allen FOPs gesucht, bei Ergänzung durch die Option **-i** nur in den individuellen. Die gesuchten Namen müssen in der Datei zeilenweise und mit vollständigem Namen hinterlegt sein, da **vargrep.sh** nach allen zulässigen Abkürzungen dieser Namen sucht.

Die Arbeitsbereiche **fax/**, **log/**, **ofil/**, **tb/**, **tmp/**, **y2k*/** und **[A-Z]/** (Sprachunterverzeichnisse) werden *nicht* durchsucht. Textnamen mit der Endung ".TMP" und ".DOT" werden ebenfalls ausgelassen.

Standard-FOPs

Im Folgenden werden nützliche und häufig verwendete Unterprogramme beschrieben, die zum Standardlieferungsumfang gehören. Diese Programme können **individualisiert** werden.

KOPF - Gestaltung des Reportkopfes

Das FOP zur Gestaltung des Reportkopfes gibt folgende Daten aus: Mandant, Bearbeiter, Datum, Uhrzeit, Seitennummer, Listennamen und Überschrift. Das FOP existiert in vier Varianten:

Ausführung	Ausgabe auf	Schmalschrift
KOPF.BROWSER	Browser	nein
KOPF.NOBROWSER	alle anderen	nein
KOPFB.BROWSER	Browser	ja
KOPFB.NOBROWSER	alle anderen	ja

ARB - aufgerufener Arbeitsbereich

Setzt die Variable '**xarb**' auf den Namen des aufgerufenen Arbeitsbereichs.

```
nach Aufruf von ek/BE ist xarb = "ek/"
```

Setzt die Variable '**tname**' auf den Namen des aufgerufenen FOPs ohne Arbeitsbereichsangabe.

```
nach Aufruf von ek/BE ist tname=BE
```

Den eigentlichen Aufrufer ermitteln Bei einem individuellen FOP, das Standard-FOPs aufruft, die ihrerseits ARB aufrufen, erhalten Sie nicht das gewünschte Ergebnis. Denn ARB liefert ein individuelles Verzeichnis wie **ow1**, während die Standardprogramme ein Lieferverzeichnis voraussetzen. Dieses Problem besteht z.B. in Container-EFOPs, die nur mehrere EFOPs zusammenfassen, oder in individuellen Wrapper-Programmen, die Standardprogramme aufrufen und dabei noch bestimmte Ausgangsbedingungen setzen und/oder Zusatzmaßnahmen durchführen.

Durch Setzen der Variablen '**baseframe**' können Sie definieren, wer in ARB als eigentlicher Aufrufer gewertet werden soll. Der Standardwert für '**baseframe**' ist 1, d.h., das per Kommando aufgerufene FOP ist der eigentliche Aufrufer.

Im FOP **ow/X** sollen zwei Standard-FOPs aufgerufen werden, die Sie nicht anpassen wollen und in deren Unterprogrammen ARB benutzt wird. ARB soll dann jedoch nicht **ow/** liefern, sondern die tatsächlichen Arbeitsbereiche der Standard-FOPs. Implementierung von **ow/X**:

```
.formula U|xbaseframe = G|framecount + 1
.input ek/BE
```

```
.. After ARB call: xarb = "ek/"  
.INPUT vk/ABC  
.. After ARB call: xarb = "vk/"
```

Siehe auch unter [Stack-Informationen für FOPs](#).

TABLEEREN - Tabelle leeren

Leert die Tabelle des Maskenpuffers. Das FOP befindet sich im Verzeichnis **is**.

DRUCKER - Ausgabemedium wählen

Ermöglicht dem Benutzer die Auswahl des Ausgabemediums (Bildschirm, ein bestimmter Drucker usw.).

DATEI.F - Ausgabeformat für Datei

Schaltet das Ausgabeformat auf die typischen Werte für die Ausgabe auf Datei, also kein Seitenumbruch und keine Ränder. DATEI.F wird bei jedem Upgrade aktualisiert.

BILD.F - Ausgabeformat für Bildschirm

Schaltet das Ausgabeformat auf die typischen Werte für die Ausgabe auf den Bildschirm.

BLANKO.F - Ausgabeformat normal

Setzt Ränder und Seitenlängen für eine "normale" Ausgabe.

BLANKOB.F - Ausgabeformat breit

Setzt Ränder und Seitenlängen für eine breite Ausgabe.

USTVA.N.BETRAG - Steuer berechnen

Berechnet die Bemessungsgrundlage und den Steuerbetrag in der Ausgabewährung. Diese wird festgelegt durch '**xakurs**'.

FIRMA.KURS - Zugriff auf Währungen

Initialisiert verschiedene Felder zum einfachen Zugriff auf Währungen und Kurse. Dieses FO-Programm ist nur einmal aufzurufen. Im weiteren Programmablauf dient das Programm **FIRMA.KURS.GET** dem Zugriff auf die initialisierten Felder. Die folgenden Felder werden von **FIRMA.KURS** initialisiert:

- Kurs vor eurorelevanter Zeit ('xfkkurs')
- Kurs in eurorelevanter Zeit ('xfnkkurs')
- Einheit vor eurorelevanter Zeit ('xfeinh')

- Einheit in eurorelevanter Zeit ('xfneinh')
- Währungskürzel ('xfland')
- Kursdatum vor eurorelevanter Zeit ('xfkursdat')
- Kursdatum in eurorelevanter Zeit ('xfnkursdat')

Es werden historische Währungskurse unterstützt. Falls eine Kursselektion zu einem anderen als dem aktuellen Datum erfolgen soll, so ist dieses Bezugsdatum als Parameter anzugeben. Zu diesem Zweck wird die globale FO-Variable '**xfbezdat**' von **FIRMA.KURS** ausgewertet.

FIRMA.KURS.GET - Betrag umrechnen

Rechnet den Betrag '**xxwbetr**' in der Währung '**xxwvon**' in die Währung '**xxwnach**' zum Zeitpunkt '**xxwdat**' um.



'**xxwdat**' gilt nur für den Fall der Euro-Umstellung. Auf historische Währungskurse kann durch das Setzen dieses Parameters nicht zugegriffen werden. Hierfür ist eine Datumsübergabe bei der Initialisierung durch **FIRMA.KURS** erforderlich.

ZEITRAUM - Zeitraum lesen

Liest einen Zeitraum ein. Der Standardzeitraum reicht vom 1.1. bis heute.

URL - Anzeige einer Internetseite konfigurieren

Die Anzeige einer Internetseite kann mit dem FOP **URL** als Abas Menüpunkt in individuellen Menüs konfiguriert werden. Das FOP erwartet als Parameter eine URL.

GUI -> Menüpunkt -> Eigenschaften

Name	Suchmaschine der abas Software
Kommando	<Text> URL http://www.abas.de/kunden/isearch/index.htm <Zeigen>
Name	Aktuelles
Kommando	<Text> URL -DATEI lokale Datei <Zeigen>

ENUMMENU - Standardaufzählungen einschränken

Soll der Benutzer nur bestimmte Elemente einer Standardaufzählung auswählen können, kann das FOP **is/ENUMMENU** nützlich sein. Es generiert ein FO-Menü mit bestimmten Einträgen einer Aufzählungsart. Die Nummern der aufzunehmenden Einträge müssen in einem Array hinterlegt werden. Der ausgewählte Menüeintrag wird einem Maskenfeld zugewiesen. Der Name dieses Maskenfeldes, das eine Standardaufzählungsart besitzen muss, muss dem FOP auch als Parameter übergeben werden. Die zu nutzende Aufzählung für das Menü wird aus der Art des Felds bestimmt.

Die Parameter finden Sie im Kommentar in is/ENUMMENU.

Das FOP ist lediglich in EFOPs verwendbar.

EXEC.COMMAND4REFERENCE - Datensatz zu einem Verweisfeld öffnen

Wenn man aus einem Dialog heraus auf Basis eines Verweises - etwa per Buttonklick - einen neuen Dialog starten möchte, kann das mit dem EFOP **EXEC.COMMAND4REFERENCE** erledigt werden. Das Programm startet ein Kommando zu einem in der Variablen "U|xxmyobject" gegebenem Datenobjekt.

Voreingestellt ist das Datenbankkommando 12 (Zeigen). Es kann durch die Angabe eines Kommandos in einer selbst zu definierenden Variablen "U|xxmycmmmd" selbst bestimmt werden.

öffnet beim Maskeneintritt in die Maske Fertigungsmittel zusätzlich die Maske Abteilung

```
.type P8 xxmyobject
.copy U|xxmyobject = M|abtlg
.continue ERROR ? M|abtlg = ""
.input "is/EXEC.COMMAND4REFERENCE"
.continue END
!ERROR
no department available
!END
```

Quick Reference

.ABSATZ / .PARA - Absatzschutz

.ABSATZ	.PARA
deutsch .ABSATZ .ABSATZ 'Zeilenanzahl 1' .ABSATZ 'Zeilenanzahl 1' 'Zeilenanzahl 2'	englisch .PARA .PARA 'number lines 1' .PARA 'number lines 1' 'number lines 2'
.absatz setzt für den unmittelbar folgenden Absatz den Absatzschutz auf 'Zeilenanzahl 2' und für alle weiteren Absätze wieder auf 'Zeilenanzahl 1'. .absatz 'Zeilenanzahl 1' setzt für alle Absätze bis auf weiteres den Absatzschutz auf 'Zeilenanzahl 1', d.h. mindestens die angegebene Anzahl Zeilen eines Absatzes bleibt zusammen auf einer Seite. .absatz 'Zeilenanzahl 1' 'Zeilenanzahl 2' setzt für alle Absätze bis auf weiteres den Absatzschutz auf 'Zeilenanzahl 1' und merkt die 'Zeilenanzahl 2' vor.	

Weitere Details

.AKTION / .ACTION - Transaktion

.AKTION	.ACTION
deutsch .AKTION + - .AKTION ABORT	englisch .ACTION + - .ACTION ABORT
.aktion + - markiert einen Programmabschnitt mit zusammengehörigen Schreibzugriffen auf die Datenbank als Transaktion. .aktion ABORT bricht eine laufende Transaktion im FOP ab und macht ihre Auswirkungen auf die Datenbank rückgängig. Sonstige Wirkungen, z.B. auf das Dateisystem, werden nicht zurückgenommen.  Wenn in dem Transaktionsabschnitt Datenbankänderungen per Lader, AJO im ClientContext, EDP oder JEDP durchgeführt werden, so werden diese nicht Bestandteil der Transaktion, da sie in einem anderen Prozess stattfinden.  .aktion ABORT hat zur Folge, dass das Programm abbricht.	

Weitere Details

.ART / .TYPE - Variablenart angeben

.ART	.TYPE
deutsch <code>.ART 'Art' 'Variablenname'</code> <code>'Variablenname' ...</code>	englisch <code>.TYPE 'specify type' 'variable name'</code> <code>'variable name' ...</code>
deklariert die aufgeführten Variablen mit der angegebenen Art. Zugriffe auf diese Variablen erfolgen mit Pufferangabe U .	

[Weitere Details](#)

.ATEXT / .ATEXT - Text einer Programmmeldung übersetzen

.ATEXT	.ATEXT
deutsch <code>.ATEXT [-SPRACHE 'Sprache']</code> <code>'Variablenname' 'Meldungsnummer'</code> <code>['Text']</code>	englisch <code>.ATEXT [-LANGUAGE 'language']</code> <code>'variable name' 'report number'</code> <code>['text']</code>
<code>.atext</code> holt aus dem Wörterbuch <code>msg.cc.dic</code> den Text mit der Meldungsnummer in der gewünschten Sprache in die angegebene FO-Variable.	

[Weitere Details](#)

.AUSGABE / .OUTPUT - Ausgabemedium festlegen

.AUSGABE	.OUTPUT
----------	---------

deutsch	englisch
.AUSGABE	.OUTPUT
.AUSGABE BILDSCHIRM	.OUTPUT TERMINAL
.AUSGABE PAUSE	.OUTPUT PAUSE
.AUSGABE S1 [KOPIEN='Anzahl']	.OUTPUT S1 [COPIES='number']
.AUSGABE FAX 'Faxnummer'	.OUTPUT FAX 'facsimile number'
.AUSGABE F1 'Faxnummer'	.OUTPUT F1 'facsimile number'
.AUSGABE MAIL [lokale Domain]	.OUTPUT MAIL [local domain] parameter1
Parameter1 ... Parameter8	... parameter8
.AUSGABE AN 'Dateiname'	.OUTPUT ADDEDTO 'file name'
.AUSGABE AUF 'Dateiname'	.OUTPUT NEW 'file name'
.AUSGABE AUF AN 'Dateiname'	.OUTPUT NEW ADDEDTO 'file name'
-ENCODIERUNG 'Encodierung'	-ENCODING 'encoding'
.AUSGABE W [: Vorlage] - Keine REST API-Unterstützung	.OUTPUT W [: template] - No REST API support
.AUSGABE E [: Parameter] - Keine REST API-Unterstützung	.OUTPUT E [: parameter] - No REST API support
.AUSGABE D [: CLIP CLIP+] - Keine REST API-Unterstützung	.OUTPUT D [: CLIP CLIP+] - No REST API support
.AUSGABE A [KOPIEN='Anzahl'] - veraltet, wird abgeschafft	.OUTPUT A [COPIES='number'] - deprecated, will be deleted

.ausgabe und .ausgabe bildschirm steuert die Ausgabe auf den Bildschirm. Der vorhergehende Bildschirminhalt wird gelöscht. Bei .ausgabe bildschirm erfolgt die Ausgabe nach jeder Zeile. .ausgabe ... bzw. .ausgabe pause bzw. .ausgabe bildschirm ... bzw. .ausgabe bildschirm pause bewirkt einen Stopp. Die FO wartet mit der weiteren Erzeugung von Ausgabertext, damit der angezeigte Text gelesen werden kann. Auf dem Bildschirm wird die Stoppstelle im Ausgabertext durch drei Punkte angezeigt. Erst nach <Return> wird der Bildschirminhalt gelöscht und das FOP arbeitet weiter.

.ausgabe S1 [kopien='Anzahl'], .ausgabe S2 [kopien='Anzahl'] usw. steuert die Ausgabe auf den Systemdrucker 1, Systemdrucker 2 usw. Als Anzahl kann angegeben werden, wie viele Exemplare gedruckt werden sollen.

.ausgabe fax 'Faxnummer'

.ausgabe F1 'Faxnummer', .ausgabe F2 'Faxnummer' usw. steuert den Faxkanal 1, Faxkanal 2 usw. an. Dahinter folgt die Faxnummer des Empfängers. Die Verwendung von Variablen ist erlaubt, z.B.

.ausgabe F1 'faxnr'. .ausgabe mail [lokale Domain] Parameter1 ... Parameter8 steuert die Ausgabe auf E-Mail.

.ausgabe an 'Dateiname' fügt den Ausgabertext an das Ende der angegebenen Datei an. Der bisherige Text in der Datei bleibt erhalten, er wird um den angefügten Text verlängert. .ausgabe auf 'Dateiname' steuert die Ausgabe auf eine Datei des angegebenen Namens. Der bisherige Text in der Datei geht verloren! .ausgabe auf | an 'Dateiname' -encodierung 'Encodierung' legt die Datei in der angegebenen Encodierung an. Zur Verfügung stehen: ABAS-S3, UTF-8, ISO-8859-1 und ISO-8859-5. Nur die Encodierungen ABAS-S3 und UTF-8 unterstützen alle Unicode-Zeichen, bei den anderen gehen die Zeichen verloren, die nicht im angegebenen Zeichensatz enthalten sind.



Die Kommandos **.ausgabe W**, **.ausgabe E** und **.ausgabe D** werden ausschließlich im Abas Windows Client unterstützt. Diese Kommandos sind über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

.ausgabe W und **.ausgabe W : Vorlage** steuern die Ausgabe auf Microsoft Word. **.ausgabe E** und **.ausgabe E : Parameter** steuern die Ausgabe auf Microsoft Excel.

.ausgabe D : CLIP steuert die Ausgabe bis zum nächsten **.ausgabe**-Kommando zeilenweise in die Zwischenablage. Dort steht immer die zuletzt ausgegebene Zeile. **.ausgabe D : CLIP+** merkt alle folgenden Ausgaben und kopiert sie beim nächsten **.ausgabe**-Kommando in die Zwischenablage. **.ausgabe D** bewirkt dasselbe wie **.ausgabe D : CLIP**.



Verwenden Sie das FO-Kommando **.ausgabe A** nicht mehr. Dieses FO-Kommando ist veraltet und wird mit einem der nächsten Upgrades abgeschafft.

.ausgabe A [kopien='Anzahl'] steuert die Ausgabe auf den Arbeitsplatzdrucker. Als Anzahl kann angegeben werden, wie viele Exemplare gedruckt werden sollen.

[Weitere Details](#)

.BELEGEN / .OCCUPY - Programme synchronisieren

.BELEGEN	.OCCUPY
deutsch	englisch
.BELEGEN	.OCCUPY
.BELEGEN 'Stringsemaphor'	.OCCUPY 'string semaphor' ['lock options'] ['lock modifier']
['Sperroptionen'] ['Sperrmodifikator']	options'] ['lock modifier']
.BELEGEN 'Datensatz-ID'	.OCCUPY 'data record ID' ['lock options'] ['lock modifier']
['Sperroptionen'] ['Sperrmodifikator']	

`.belegen` ohne weitere Parameter hebt die Sperre auf.

`.belegen 'Stringsemaphor' ['Sperroptionen'] ['Sperrmodifikator']`: Setzt die Sperre auf den Text `'Stringsemaphor'`.

`.belegen 'Datensatz-ID' ['Sperroptionen'] ['Sperrmodifikator']`: Wenn der Sperrtext das Format einer Datensatz-ID `"(sn,dn,0)"` hat, wird nicht der Text selbst sondern der Datensatz gesperrt. Der Datensatz wird dabei nicht gelesen, es wird kein Puffer belegt. Es wird nicht geprüft, ob es den Datensatz wirklich gibt.

Die `.belegen` Befehle haben folgende Eigenschaften:

- Der Befehl `.belegen` kann in einem FO-Programm nur eine einzige Sperre setzen. Ein erneuter Aufruf von `.belegen` hebt die vorherige Sperre auf.
- Bei FOP-Ende wird die Sperre automatisch freigegeben.
- Der Sperrbefehl kann um eine definierte `Sperroption` und/oder einen definierten `Sperrmodifikator` ergänzt werden. Wenn keine Sperroption bzw. kein Sperrmodifikator angegeben wird, dann wird implizit die Sperroption `schreibsperre` und der Sperrmodifikator `abbruch` gesetzt.
- Wenn zuvor, mithilfe des Kommandos `.setze abbruch --` der Abbruch des FO-Programms verhindert wird, dann gilt lediglich die Sperroption `schreibsperre`.
- Wenn der Stringsemaphor oder die Datensatz-ID gesperrt werden kann, wird die Variable `G|gesperrt` auf `G|true` gesetzt, ansonsten auf `G|false`.

[Weitere Details](#)

.HOLE BETRIEBSAUFTRAG / .SELECT WORKORDER - Betriebsauftrag

<code>.HOLE BETRIEBSAUFTRAG</code>	<code>.SELECT WORKORDER</code>
deutsch	englisch
<code>.HOLE BETRIEBSAUFTRAG</code>	<code>.SELECT WORKORDER</code>
<code>.HOLE BETRIEBSAUFTRAG "<Nummer>"</code>	<code>.SELECT WORKORDER "<number>"</code>
<code>.HOLE BETRIEBSAUFTRAG "+<Nummer>"</code>	<code>.SELECT WORKORDER "+<number>"</code>
<code>.HOLE BETRIEBSAUFTRAG VORSCHLAG</code>	<code>.SELECT WORKORDER SUGGESTION</code>
<code>.HOLE BETRIEBSAUFTRAG OBLIGO</code>	<code>.SELECT WORKORDER OUSTANDING</code>
<code>.HOLE BETRIEBSAUFTRAG DRUCKBAREN</code>	<code>.SELECT WORKORDER PRINTABLE</code>
<code>.HOLE BETRIEBSAUFTRAG UEBERNAHME</code>	<code>.SELECT WORKORDER TAKEOVER</code>
<code>.HOLE BETRIEBSAUFTRAG ABLAGE</code>	<code>.SELECT WORKORDER FILINGSYSTEM</code>

.hole betriebsauftrag Betriebsaufträge, die noch nicht vollständig abgearbeitet und deren Arbeitsscheine in Nummernreihenfolge sortiert sind

.hole betriebsauftrag "<Nummer>" Betriebsauftrag oder Arbeitsschein mit der angegebenen Identnummer

.hole betriebsauftrag "+<Nummer>" Rückmeldung zum Betriebsauftrag oder Arbeitsschein mit der angegebenen Identnummer

.hole betriebsauftrag vorschlag Fertigungsvorschläge

.hole betriebsauftrag obligo Kapazitätsreservierungen einer Maschinengruppe, d.h. alle Arbeitsgänge, die einer Maschinengruppe in einem bestimmten Zeitraum eingelastet sind; nach Anfangstermin sortiert.

.hole betriebsauftrag druckbaren Betriebsaufträge in der Druckwarteschlange, für die Arbeitspapiere zu drucken sind

.hole betriebsauftrag uebernahme Rückmeldungen, die erfasst, aber noch nicht gebucht sind

.hole betriebsauftrag ablage alle Rückmeldebelege

[Weitere Details](#)

.BLOCKSATZ / .JUSTIFIED - Blocksatz

.BLOCKSATZ	.JUSTIFIED
deutsch .BLOCKSATZ	englisch .JUSTIFIED
Verteilt den Text durch Auffüllen mit Leerzeichen vom linken bis zum rechten Rand.	

[Weitere Details](#)

.BOX / .BOX - Ausgabe mehrzeiliger Meldungen

.BOX	.BOX
deutsch .BOX [-BUTTON 'Buttonset'] [-TITEL Text...]	englisch .BOX [-BUTTON 'buttonset'] [-TITLE text...]
gibt eine Dialogbox mit einer mehrzeiligen Meldung per FOP oder EFOP aus. -button 'Buttonset' legt fest, welche der Buttons "OK", "Ja", "Nein" und/oder "Abbruch" angezeigt werden. -titel Text schreibt den angegebenen Text in den oberen Rahmen der Dialogbox.	

[Weitere Details](#)

.BRINGE / .REWRITE - Datensatz zurückschreiben

.BRINGE	.REWRITE
----------------	-----------------

deutsch	englisch
<code>.BRINGE</code>	<code>.REWRITE</code>
<code>.BRINGE MASKE</code>	<code>.REWRITE SCREEN</code>
<code>.BRINGE SPERREN</code>	<code>.REWRITE LOCK</code>
<code>.BRINGE PLAN</code>	<code>.REWRITE PLAN</code>

`.bringe` schreibt den in **H** geladenen Datensatz zurück. Der H-Puffer kann durch `.hole` oder durch `.mache` gefüllt worden sein. Dieses Kommando entspricht im Wesentlichen dem Abspeichern nach einer Maskenveränderung.

`.bringe maske` schreibt den Datensatz im Puffer **M** beim Ereignis Maskenaustritt zurück.

`.bringe sperren` testet, ob `.bringe` durchgeführt werden kann. Wenn nicht, bricht das Programm mit einer Fehlermeldung ab.

`.bringe plan` schreibt Objekte nach Änderung zurück und veranlasst die Disposition, die Änderung in der Planung zu berücksichtigen.

Weitere Details

`.BROWSER` / `.BROWSER - URL` im Browser anzeigen

<code>.BROWSER</code>	<code>.BROWSER</code>
deutsch	englisch
<code>.BROWSER [-NEU] [-KOMMANDOFENSTER]</code>	<code>.BROWSER [-NEW] 'URL'</code>
<code>'URL'</code>	<code>.BROWSER [-COMMANDWINDOW] 'URL' - No</code>
<code>.BROWSER [-KOMMANDOFENSTER] 'URL' -</code>	<code>REST API support</code>
Keine REST API-Unterstützung	<code>.BROWSER [-NEW] -FILE 'filename' - No</code>
<code>.BROWSER [-NEU] -DATEI 'filename' -</code>	<code>REST API support</code>
Keine REST API-Unterstützung	

`.browser 'URL'` zeigt die angegebene URL in einem Browserfenster an.



Die Kommandos `.browser [-FILE]` und `.browser [-COMMANDWINDOW]` werden ausschließlich im Abas Windows Client unterstützt. Diese Kommandos sind über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

`.browser -kommandofenster` zeigt die URL nicht in einem Browserfenster an, sondern in der Kommandoübersicht.

`.browser -datei 'filename'` zeigt eine Datei an. `'filename'` ist der Name einer Datei im Unterverzeichnis **win** des Mandantenverzeichnisses. Sie müssen den Pfad relativ zum Mandantenverzeichnis und beginnend mit **win/** angeben.

Mit der Option **-neu** wird immer ein neues Browserfenster geöffnet.

Weitere Details

.BUDGET / .BUDGET - Summierung von Kontenbeträgen

.BUDGET	.BUDGET
deutsch <code>.BUDGET [-Optionen] 'Kontenbereich', 'Kontenbereich', ...</code>	englisch <code>.BUDGET [-options] 'account range', 'account range', ...</code>
summiert die Beträge der aufgeführten Konten Mittels Optionen lässt sich die Gruppe der Konten einschränken.	

[Weitere Details](#)

.CHART / .CHART - Diagramme in Masken

.CHART	.CHART
deutsch <code>.CHART -INIT [-FILE 'Name'] .CHART -PARAM [Diagramm-Optionen] [Achsen-Optionen] .CHART -CONSTANTY 'Position' [Optionen] .CHART -STRIPEX 'Anfangsindex' 'Endindex' .CHART -STRIPEY 'Anfangswert' 'Endwert' .CHART -STARTVALUES ['Bezeichnung'] [Optionen] .CHART -VALUE 'x-Wert' 'y-Wert' 'Durchmesser' [Optionen] .CHART -VALUE 'Anfangswert' 'Endwert' [Optionen] .CHART -VALUE 'Wert' [Optionen] .CHART -SHOW [-FILE 'Dateiname'] 'Feldname' 0 1</code>	englisch <code>.CHART -INIT [-FILE 'name'] .CHART -PARAM [options chart] [options axes] .CHART -CONSTANTY 'position' [options] .CHART -STRIPEX 'initial index' 'final index' .CHART -STRIPEY 'initial value' 'final value' .CHART -STARTVALUES ['description'] [options] .CHART -VALUE 'x-value' 'y-value' 'diameter' [options] .CHART -VALUE 'initial value' 'final value' [options] .CHART -VALUE 'value' [options] .CHART -SHOW [-FILE 'file name'] 'field name' 0 1</code>

`.chart -init` initialisiert ein Diagramm.
 Mit `.chart -param` werden die Merkmale der Diagrammanzeige gesetzt. Bei mehreren Anzeigeeoptionen müssen entsprechend viele `.chart -param`-Anweisungen verwendet werden.
`.chart -constanty` erzeugt eine horizontale Linie.
`.chart -stripex` und `.chart -stripey` erzeugen vertikale und horizontale Streifen variabler Breite.
`.chart -startvalues` eröffnet eine Datenserie.
`.chart -value` setzt die Werte der Datenserien verschiedener Diagrammtypen (Blasen-, Linien-, usw.).
`.chart -show` zeigt ein Diagramm an.

.CURSOR / .CURSOR - Cursorsteuerung

.CURSOR	.CURSOR
deutsch .CURSOR 'Feldname' ['Zeilennummer']	englisch .CURSOR 'field name' ['line number']
Stellt den Cursor auf das angegebene Feld	

.DATEI / .FILE - Lesen aus Dateien

.DATEI	.FILE
deutsch .DATEI .DATEI -OEFFNEN <Dateivariablen> 'Dateiname' .DATEI -TEST -OEFFNEN <Dateivariablen> 'Dateiname' .DATEI -LESEN <Dateivariablen> 'Variablenliste' .DATEI -SCHLIESSEN <Dateivariablen> .DATEI -TEST 'Dateiname' .DATEI -TEMPNAME Variable .DATEI [-FOP] .DATEI [-MASKE] .DATEI [-FREIGEBEN 'Variable'] .DATEI [-UEBERNEHMEN "Text"] .DATEI [-SUFFIX 'Text']	englisch .FILE .FILE -OPEN 'descriptor name' 'file name' .FILE -TEST -OPEN 'descriptor name' 'file name' .FILE -READ 'descriptor name' 'list of variables' .FILE -CLOSE 'descriptor name' .FILE -TEST 'file name' .FILE -TEMPNAME variable .FILE [-FOP] .FILE [-SCREEN] .FILE [-RELEASE 'variable'] .FILE [-ACCEPT "text"] .FILE [-SUFFIX 'text']

.datei -oeffnen <Dateivariablen> 'Dateiname' setzt die Dateivariablen, mit der im Folgenden die Datei referenziert wird.

.datei -lesen <Dateivariablen> 'Variablenliste' liest in der zuvor geöffneten Datei mit dem angegebenen Deskriptor. [Erlaubte Trennzeichen](#)

.datei -schliessen <Dateivariablen> schließt die Datei mit dem angegebenen Deskriptor.

.datei -test 'Dateiname' prüft, ob die angegebene Datei mit **.datei -oeffnen ...** geöffnet werden könnte und setzt **G|mehr** entsprechend.

.datei -tempname Variable ist eine Grundfunktion. Sie definiert einen für die Dauer eines FO-Programmlaufs eindeutigen Temporärdateinamen und sorgt dafür, dass die Datei bei Programmende automatisch gelöscht wird. Die Variable ist vom Typ **Text**.

Bei **-fop** und im Standardfall gilt der Dateiname nur für einen FO-Programmlauf. Danach wird die Datei gelöscht.

Bei **-maske** gilt der Dateiname für mehrere EFOPs, vom Maskeneintritts-FOP bis zum Ende- oder Abbruch-FOP.

Bei **-freigeben** wird die Datei samt Verzeichnis so umbenannt, dass ihr Name an asynchron gestartete Programme übergeben werden kann. Die Datei wird dann nicht mehr gelöscht.

Bei **-uebernehmen** kann eine zuvor durch **-freigeben** umbenannte Datei in den Temporärbereich übernommen werden.

Mit **-suffix** führt mehrfaches Aufrufen im selben FOP mit derselben Variable bzw. demselben Text zum selben Dateinamen.

Weitere Details

.DAZU / .ADD - Weiteren Datensatz holen

.DAZU	.ADD
deutsch	englisch
.DAZU	.ADD
.DAZU 'Verweisfeld'	.ADD 'reference field'
.DAZU VKZ 'Daten'	.ADD VKZ 'data'
.dazu gibt den D-Puffer frei. .dazu 'Verweisfeld' lädt ein Verweisobjekt in den D-Puffer. .dazu vkz 'Parameter' lädt die monatlichen Verkehrszahlen und Umsatzzähler in den D-Puffer. Funktion und Parameter sind wie bei .hole vkz.	

Weitere Details

.DDE.EXEC / .DDE.EXEC - Befehl senden

.DDE.EXEC	.DDE.EXEC
-----------	-----------

deutsch .DDE.EXEC G ddekanal 'Befehl'	englisch .DDE.EXEC G ddekanal 'command'
.dde.exec sendet einen Befehl an eine Anwendung mit der Nummer G ddekanal. G mehr wird gesetzt.	

[Weitere Details](#)

.DDE.EXIT / .DDE.EXIT - Kanal schließen

.DDE.EXIT	.DDE.EXIT
deutsch .DDE.EXIT G ddekanal	englisch .DDE.EXIT G ddekanal
.dde.exit schließt den Kanal mit der Nummer G ddekanal.	

[Weitere Details](#)

.DDE.INFO / .DDE.INFO - Liste der Applikationen

.DDE.INFO	.DDE.INFO
deutsch .DDE.INFO 'Applikation' 'Thema' info	englisch .DDE.INFO 'application' 'topic' info
.dde.info liefert eine Liste der aktuellen DDE-Server-Applikationen. Mit 'Applikation' bzw. 'Thema' lässt sich die Auswahl einschränken. Das Ergebnis steht in der FO-Variablen info. G mehr wird gesetzt.	

[Weitere Details](#)

.DDE.INIT / .DDE.INIT - Kanal öffnen

.DDE.INIT	.DDE.INIT
deutsch .DDE.INIT G ddekanal 'Applikation' 'Thema'	englisch .DDE.INIT G ddekanal 'application' 'topic'
.dde.init öffnet einen Kanal zu einer Applikation mit einem Thema. In der FO-Variablen G ddekanal wird eine Kanalnummer zurückgeliefert, die bei den weiteren Kommandos stets anzugeben ist.	

[Weitere Details](#)

.DDE.REQUEST / .DDE.REQUEST - Daten anfordern

.DDE.REQUEST	.DDE.REQUEST
deutsch .DDE.REQUEST G ddekanal 'Objekt' request	englisch .DDE.REQUEST G ddekanal 'item' request
.dde.request fordert Daten von einer Anwendung mit der Nummer G ddekanal an und legt sie in der FO-Variablen request ab. Das Objekt wird ebenfalls angegeben. G mehr wird gesetzt.	

[Weitere Details](#)

.DDE.SEND / .DDE.SEND - Daten senden

.DDE.SEND	.DDE.SEND
deutsch .DDE.SEND G ddekanal 'Objekt' 'Daten'	englisch .DDE.SEND G ddekanal 'item' 'value'
.dde.send sendet Daten zu einem Objekt in einer Anwendung mit der Nummer G ddekanal. G mehr wird gesetzt.	

[Weitere Details](#)

.DDE.WAIT / .DDE.WAIT - Warten auf Schließen des Dokuments

.DDE.WAIT	.DDE.WAIT
deutsch .DDE.WAIT	englisch .DDE.WAIT
.dde.wait wartet darauf, dass das zuvor mit .ausgabe W oder .ausgabe E erstellte Dokument geschlossen wird.	

[Weitere Details](#)

.DRUCKEN / .PRINT - Druckausgabe

.DRUCKEN	.PRINT
-----------------	---------------

deutsch <code>.DRUCKEN 'Anzahl Exemplare'</code> <code>.DRUCKEN 'Schrifteigenschaft 1'</code> <code>'Schrifteigenschaft 2' ...</code> <code>'Schrifteigenschaft n'</code>	englisch <code>.PRINT 'number of copies'</code> <code>.PRINT 'font attribute 1' 'font</code> <code>attribute 2' ... 'font attribute n'</code>
<code>.drucken 'Anzahl Exemplare'</code> löst den Ausdruck aus. Bei <code>.ausgabe</code> auf Bildschirm oder Datei bleiben die <code>.drucken</code>-Kommandos unberücksichtigt. <code>.drucken 'Schrifteigenschaft 1' 'Schrifteigenschaft 2' ... 'Schrifteigenschaft n'</code> steuert die Schriftgestaltung, die der Drucker ermöglicht.	

[Weitere Details](#)

.EDITIERE / .EDITING - Systemaufrufe

.EDITIERE	.EDITING
deutsch <code>.EDITIERE [-READONLY -WRITE] [-RIGHT</code> <code>'n'] [-HTML [-CONTENTSWIDTH 'w']]</code> <code>'Datei'</code>	englisch <code>.EDITING [-READONLY -WRITE] [-RIGHT</code> <code>'n'] [-HTML [-CONTENTSWIDTH 'w']]</code> <code>'file'</code>
Das Kommando ruft den Texteditor für die angegebene Datei auf. Mit <code>-READONLY</code> wird die Datei im Nur-Lesen-Modus angezeigt, mit <code>-WRITE</code> kann die Datei geschrieben werden. <code>-RIGHT 'n'</code> setzt in der GUI den rechten Rand in Spalte 'n'. <code>-HTML</code> öffnet in der GUI den HTML-Freitexteditor. <code>-CONTENTSWIDTH 'w'</code> setzt in der GUI die Größe des HTML-Freitexteditors so, dass die für den Inhalt verfügbare Breite 'w' Millimeter beträgt. Der Standardwert ist 180; dies entspricht der Breite einer DIN A 4 Seite im Hochformat abzüglich links und recht jeweils 3 Zentimeter Rand.	

[Weitere Details](#)

.EINGABE / .INPUT - Unterprogramm ausführen

.EINGABE	.INPUT
deutsch <code>.EINGABE</code> <code>.EINGABE 'Dateiname'</code> <code>.EINGABE -TEXT 'Dateiname'</code> <code>.EINGABE <Klassenname>.java</code> <code>[Argumente]</code>	englisch <code>.INPUT</code> <code>.INPUT 'file name'</code> <code>.INPUT -TEXT 'file name'</code> <code>.INPUT <class name>.java [arguments]</code>

`.eingabe` startet den [Direktmodus](#).

`.eingabe 'Dateiname'` interpretiert den Dateinamen bzw. den Inhalt der Variablen als FO-Programm und führt es aus.

`.eingabe -text 'Dateiname'` interpretiert die Datei als reinen Text und gibt ihn auf das aktuelle Ausgabegerät aus.

`.eingabe <Klassenname>.java [Argumente]` interpretiert `<Klassenname>.java` als JFOP und führt es aus.

[Weitere Details](#)

.ENDE / .END - FO-Ende

<code>.ENDE</code>	<code>.END</code>
deutsch	englisch
<code>.ENDE</code>	<code>.END</code>
<code>.ENDE ['Exit-Status'] [.]</code>	<code>.END ['exit status'] [.]</code>
<code>.ENDE 'Exit-Status' 'Buttonname'</code>	<code>.END 'exitstatus' 'button name'</code>
<code>.ENDE 'Abas Kommando' - Keine REST API-Unterstützung</code>	<code>.END 'Abas command' - No REST API support</code>
<code>.ENDE 0 'Abas Kommando'</code>	<code>.END 0 'Abas command'</code>
<code>.ende</code> beendet das aktuell laufende FO-Programm, aber nicht das aufrufende Java-Programm. Mit <code>'Exit-Status'</code> kann man den Erfolg des Programms definieren. Als Werte sind natürliche Zahlen zulässig, wobei 0 oder das Fehlen einer Angabe für den Erfolg und jeder andere Wert für einen Misserfolg steht. Ausnahme: Bei den Ereignissen <code>maskaus</code> und <code>maskende</code> werden Werte ungleich 0 zur Dialogsteuerung verwendet. Siehe auch Grafik Maskenereignisse. Ausnahme: <code>.ende 0 'Abas Kommando'</code> legt bei individuellen BU8-, BU10- und BU12-Buttons das Kommando fest, mit dem die Submaske geöffnet wird. Siehe auch Buttons. Diese drei Buttonarten öffnen beim Klick durch den Benutzer eine Submaske. Für Standardbuttons ist die Submaske vorgegeben, für individuelle Buttons legt der Benutzer im <code>buttonvor</code>-Event fest, welche Submaske geöffnet werden soll, und zwar mit dem <code>.ende</code> Kommando. Mit einem abschließenden Punkt kann das FO-Ausgabefenster ohne Quittierung geschlossen werden. Mit <code>Buttonname</code> kann im <code>buttonnach</code>-FOP und im <code>felldaus</code>-FOP das Klicken eines Ende-Buttons (Art "BU10", "BU12" oder "BU18") simuliert werden. Siehe auch: Speichern und Fenster schließen per EFOP.	
 Das Kommando <code>.ende 'Abas Kommando'</code> wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.	
<code>.ende 'Abas Kommando'</code> hat im EFOP keine Wirkung. Der Exitstatus ist 0. Im FOP hat der Exitstatus keine Wirkung. Nach dem FOP wird das angegebene 'Abas Kommando' ausgeführt.	

[Weitere Details](#)

.FARBE / .COLOUR - Setzen von Farben in Masken

.FARBE	.COLOUR
deutsch	englisch
.FARBE 'Optionen' [-FELD] 'Feldname' ['Zeilennummer']	.COLOUR 'options' [-FIELD] 'field name' ['line number']
.FARBE 'Optionen' [-ZEILE] 'Zeilennummer'	.COLOUR 'options' [-LINE] 'line number'
.FARBE 'Optionen' -SPALTE 'Feldname'	.COLOUR 'options' -COLUMN 'field name'
.FARBE 'Optionen' -TABELLE	.COLOUR 'options' -TABLE
.FARBE 'Optionen' 'Statusbereich-Option' - Keine REST API-Unterstützung	.COLOUR 'options' 'status area option' - No REST API support
Optionen	Options
-VORDERGRUND 'Farbe'	-FOREGROUND 'colour'
-HINTERGRUND 'Farbe'	-BACKGROUND 'colour'
-RESET	-RESET

Ermöglicht die Einstellung der Schriftfarbe (**-vordergrund**) und Hintergrundfarbe (**-hintergrund**) vom EFOP aus, sowie das Zurücksetzen der Farbe (**-reset**). Die Farbe muss als RGB-Wert angegeben werden, z.B. 255 0 0.

[-feld] 'Feldname' ['Zeilennummer'] setzt die Farbe des Feldes in der angegebenen Zeile. Wenn die Zeilenangabe fehlt, wird 0 angenommen (Kopfteil).

-zeile 'Zeilennummer' setzt die Farbe einer einzelnen Zeile. Zeile 0 ist der Kopfteil der Maske.

-spalte 'Feldname' setzt die Farbe einer einzelnen Spalte in der Tabelle.

-tabelle, nur in Verbindung mit der Option **-reset** verwendbar, setzt die Farbe für die ganze Tabelle zurück.



Das Kommando **.farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] 'Statusbereich-Option'** wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

.farbe [-vordergrund 'Farbe'] [-hintergrund 'Farbe'] 'Statusbereich-Option' setzt die Farbe der GUI-Statusleiste.

[Weitere Details](#)

.FEHLER / .ERROR - Fehlermeldung

.FEHLER	.ERROR
---------	--------

deutsch .FEHLER .FEHLER beliebiger Text .FEHLER [-FELD feldname] [-ZEILE zeilennummer] [-NUMMER fehlernummer] [-MELDUNG meldung]	englisch .ERROR .ERROR any text .ERROR [-FIELD field name] [-LINE line number] [-NUMBER error number] [-MESSAGE message]
Dieses Kommando gibt Fehlermeldungen aus. Im FOP erfolgt die Ausgabe sofort. Im EFOP kann das .fehler-Kommando mehrfach ausgeführt werden. Jeder Aufruf von .fehler überschreibt die aktuellen Informationen. Die Ausgabe erfolgt am Programmende. .fehler beliebiger Text führt das FOP weiter aus. Nach der Rückkehr in die Maske wird in der Statuszeile der angegebene Text ausgegeben. .fehler -NUMMER gibt die der Nummer zugeordneten Meldung aus msg.cc.dic aus. Diese Ausgabe erfolgt immer in der Bediensprache. Der Vorteil von Fehlernummern ist, dass Übersetzungen aus msg.cc.dic verwendet werden. .fehler -NUMMER und .fehler beliebiger Text können kombiniert werden. .fehler initialisiert die mit .fehler erzeugten Informationen.	
 Die Optionen FELD und ZEILE können nur bei EFOPs verwendet werden.	

Weitere Details

.FLATTERSATZ / .UNJUSTIFIED - Flattersatz

.FLATTERSATZ	.UNJUSTIFIED
deutsch .FLATTERSATZ	englisch .UNJUSTIFIED
Füllt die Zeilen wortweise vom linken bis zum rechten Rand. Das Schriftbild ist links bündig und "flattert" rechts.	

Weitere Details

.FORMAT / .FORMAT - Ausgabeformat umschalten

.FORMAT	.FORMAT
deutsch .FORMAT HTML XML [-ENCODIERUNG 'Encodierung'] .FORMAT ABAS	englisch .FORMAT HTML XML [-ENCODING 'encoding'] .FORMAT ABAS
.format schaltet das Ausgabeformat auf HTML, XML oder den Standardwert (Abas Zeichensatz). Zulässige Werte für Encodierung sind UTF-8 und ISO-8859-1. Ohne Angabe gilt ISO-8859-1.	

.FORMEL / .FORMULA - Variablen Werte zuweisen

.FORMEL	.FORMULA
deutsch .FORMEL 'Variablenname' = 'Ausdruck'	englisch .FORMULA 'variable name' = 'expression'
Der 'Variablen' wird ein neuer Inhalt zugewiesen, und zwar das Rechenergebnis des 'Ausdrucks'.	

.GEDRUCKT / .PRINTED - Druckwarteschlange

.GEDRUCKT	.PRINTED
deutsch .GEDRUCKT .GEDRUCKT -ARCHIV .GEDRUCKT -RESET	englisch .PRINTED .PRINTED -ARCHIVE .PRINTED -RESET
.gedruckt holt einen Vorgang nach seinem Druck aus der Druckwarteschlange. .gedruckt -archiv archiviert generierte Dateien in der Datenbank "Dokument". Die Archivierung erfolgt beim Druck des Dokuments. Das archivierte Dokument kann selektiert, angezeigt oder nochmals gedruckt werden. .gedruckt -reset nimmt den im FO-Hauptpuffer geladenen Vorgang wieder in die Druckwarteschlange auf. Dazu wird im zugehörigen Datensatz die entsprechende Checkbox markiert. In der Regel wurde diese Markierung beim vorangehenden Ausdruck des Vorgangs automatisch entfernt. Die Kombination .gedruckt -archiv -reset ist nicht zulässig.	

Interpreter-Einstellungen

HTML- und XML-Ausgabe

Eingabe von Dateien, Temporärdateien

Weitere Details

Allgemeine Datenbankkommandos

Weitere Details

Anwendungsspezifische Datenbankkommandos

Weitere Details

Kommandos für Datensätze der Fertigung

Weitere Details

Kommandos für Datensätze der BDE

Weitere Details

Kommandos für die Disposition

Weitere Details

Kommandos für Verbrauchsfolgeverfahren

Weitere Details

Kommandos für die Materialwirtschaft

Weitere Details

Kommandos für die Finanzbuchhaltung

Weitere Details

Holen des zuvor geholten Datensatzes

Weitere Details

FO-Zugriffe auf Sachmerkmalleisten- und Selektionsleistenrohlinge

Weitere Details

Zugriff auf einzelne Merkmale einer Sachmerkmalleiste

[Weitere Details](#)

Sperren setzen

[Weitere Details](#)

Sonstige FO-Kommandos

[Weitere Details](#)

Logging und Debugging

[Weitere Details](#)

Maskenbezogene Kommandos

[Weitere Details](#)

.HILFE / .HELP - Hilfekapitel

.HILFE	.HELP
deutsch .HILFE .HILFE 'Kapitel'	englisch .HELP .HELP 'chapter'
.hilfe ruft ein Hilfekapitel auf.	

[Weitere Details](#)

.HINWEIS / .NOTE - Meldungen

.HINWEIS	.NOTE
deutsch .HINWEIS .HINWEIS [-SOFORT] [-AKTUALISIEREN] [-DIAGNOSE] ('n') .HINWEIS [-SOFORT] [-AKTUALISIEREN] [-DIAGNOSE] 'beliebiger Text'	englisch .NOTE .NOTE [-IMMEDIATELY] [-UPDATE] [-ERRORREPORT] ('n') .NOTE [-IMMEDIATELY] [-UPDATE] [-ERRORREPORT] 'any text'

.hinweis gibt einzeilige Meldungen von EFOPs aus.
 .hinweis -sofort ('n') oder 'beliebiger Text' bewirkt die sofortige Ausgabe des Hinweises.
 .hinweis -sofort löscht den Inhalt der Statusleiste.
 .hinweis -aktualisieren zeigt auf der GUI alle Änderungen an Maskenfeldern sofort an; normalerweise werden diese erst am Ende des EFOPs angezeigt.
 .hinweis -diagnose erzeugt eine [FOP-Administrationsmeldung](#).

[Weitere Details](#)

.HOLE / .SELECT - Datensätze holen

.HOLE	.SELECT
deutsch	englisch
.HOLE Datenbankname [Selektionskriterien]	.SELECT database name [selection criteria]
.HOLE Datenbankname ""	.SELECT database name ""
.HOLE -	.SELECT -
.hole Datenbankname holt den nächsten Datensatz aus diesem Bereich. .hole Datenbankname "Identnummer/Suchwort" holt den Datensatz mit der Identnummer bzw. dem Suchwort aus der angegebenen Datenbank. .hole Datenbankname 'Textvariable' holt den Datensatz bzw. die Datensätze aus der angegebenen Datenbank. Die Textvariable beinhaltet Identnummer, Suchwort, Id oder einen Selektionsstring. .hole Datenbankname "" fordert die Eingabe eines Suchbegriffs über Tastatur an. .hole - gibt den geholten Datensatz wieder auf. Wenn ein Datensatz mit dem Kommando .merke gemerkt wurde, dann können Sie diesen mit dem Kommando .hole laden. Der Sperrzustand bleibt dabei erhalten.	

[Weitere Details](#)

.HOLE AMASKE / .SELECT ASCREEN - Kopf oder Zeile des A-Puffers holen

.HOLE AMASKE	.SELECT ASCREEN
deutsch	englisch
.HOLE AMASKE [KOPF ZEILE]	.SELECT ASCREEN [HEADER LINE]
lädt den Kopfteil bzw. die nächste Zeile des A-Puffers	

[Weitere Details](#)

.HOLE BDE / .SELECT PDC - BDE-Datensätze

.HOLE BDE	.SELECT PDC
deutsch	englisch
.HOLE BDE	.SELECT PDC
.HOLE BDE ALPHABETISCH	.SELECT PDC ALPHABETICAL
.HOLE BDE 'Bezeichnung'	.SELECT PDC 'description'
.HOLE BDE ASCHEIN 'Nummer'	.SELECT PDC JTICKET 'number'
.HOLE BDE OFFENE	.SELECT PDC OUTSTANDINGITEMS
.HOLE BDE OFFENE SPEZIFISCH	.SELECT PDC OUTSTANDINGITEMS SPECIFIC
.HOLE BDE UEBERNAHME	.SELECT PDC TAKEOVER
.HOLE BDE DRUCKBAREN	.SELECT PDC PRINTABLE

.hole bde holt alle Personal- und Auftragszeitpaare in Nummernreihenfolge.
.hole bde alphabetisch holt alle Zeitpaare in der Reihenfolge der Suchwörter.
.hole bde 'Bezeichnung' holt alle Zeitpaare mit dem Suchwort '**Bezeichnung**'.
.hole bde aschein 'Nummer' holt alle Zeitpaare des Betriebsauftrags- oder Arbeitsschein-Nummernbereiches '**Nummer**' in zeitlicher Sortierung.
.hole bde offene holt alle offenen Zeitstempelungen, sortiert nach Mitarbeiter, Anfangsdatum und -uhrzeit oder nach Mitarbeiter, Betriebsauftrags-, Arbeitsscheinnummer, Anfangsdatum und -uhrzeit.
.hole bde offene spezifisch holt alle offenen Zeitstempelungen in zeitlicher Reihenfolge, wenn zuvor über die Variablen **G|adatum** und **G|edatum** Anfangstermin und Endtermin festgelegt wurden. Dies ist allerdings nur in Verbindung mit dem Kommando **.hole mitarbeiter** oder **.hole kapazitaet** möglich.
.hole bde uebernahme holt alle erfassten, aber noch nicht übertragenen Zeitpaare.
.hole bde druckbaren holt alle als fehlerhaft gekennzeichneten Zeitpaare.

Weitere Details

.HOLE BINDUNG / .SELECT LINK - Lagerzugänge und -abgänge

.HOLE BINDUNG	.SELECT LINK
deutsch	englisch
.HOLE BINDUNG	.SELECT LINK

.hole bindung liefert
 * zu einem Abgang alle Zugänge, die ihn bedient haben * zu einem Zugang alle Abgänge, in die er eingeflossen ist

Weitere Details

.HOLE BUCHUNG / .SELECT ENTRY - Buchungszeile holen

.HOLE BUCHUNG	.SELECT ENTRY
deutsch .HOLE BUCHUNG .HOLE BUCHUNG "'Nummer'"	englisch .SELECT ENTRY .SELECT ENTRY "'number'"
.hole buchung "'Nummer'" holt eine Buchung	

[Weitere Details](#)

.HOLE DATEI / .SELECT FILE - Datensätze aus Zusatztabelle

.HOLE DATEI	.SELECT FILE
deutsch .HOLE DATEI .HOLE DATEI2HOLE DATEI60 .HOLE DATEI VORLAGE	englisch .SELECT FILE .SELECT FILE2SELECT FILE60 .SELECT FILE TEMPLATE
.hole datei greift auf die Zusatztabellen 1 bis 60 zu. .hole datei vorlage holt Objekte aus der Zusatztabelle zum Wiedervorlagetermin.	

[Weitere Details](#)

.HOLE DAZU / .SELECT ADD - Weiteren Datensatz holen

.HOLE DAZU	.SELECT ADD
deutsch .HOLE DAZU .HOLE DAZU 'Datensatzverweis'	englisch .SELECT ADD .SELECT ADD 'record reference'
lädt, wie .dazu, ein weiteres Verweisobjekt. Im Unterschied zu .dazu wird das Verweisobjekt in den H-Puffer geladen. Um den Ausgangsdatsatz zu laden, verwenden Sie erneut .hole`dazu`	

[Weitere Details](#)

.HOLE EINKAUFS / .SELECT PURCHASING - Für Datensätze des Einkaufs

.HOLE EINKAUFS	.SELECT PURCHASING
deutsch .HOLE EINKAUFS .HOLE EINKAUFS ABLAGE .HOLE EINKAUFS ALPHABETISCH .HOLE EINKAUFS "'Bezeichnung'" .HOLE EINKAUFS DRUCKBAREN BESTELLUNGEN LSCHEINE RECHNUNGEN ZAHLUNGEN .HOLE EINKAUFS OFFENE BESTELLUNGEN LSCHEINE RECHNUNGEN .HOLE EINKAUFS UEBERNAHME .HOLE EINKAUFS VORSCHLAG	englisch .SELECT PURCHASING .SELECT PURCHASING FILINGSYSTEM .SELECT PURCHASING ALPHABETICAL .SELECT PURCHASING "'description'" .SELECT PURCHASING PRINTABLE PORDERS DELNOTES INVOICES PAYMENTS .SELECT PURCHASING OUTSTANDING PORDERS DELNOTES INVOICES .SELECT PURCHASING TAKEOVER .SELECT PURCHASING SUGGESTION
.hole einkaufs holt die folgenden Vorgänge in der Reihenfolge der Nummern: * noch nicht vollständig berechnete Bestellungen * noch nicht gebuchte Lieferscheine * gebuchte und getrennt zu fakturierende Lieferscheine * noch nicht gebuchte Rechnungen .hole einkaufs ablage holt Vorgänge aus der Einkaufsablage. .hole einkaufs alphabetisch holt Bestellungen, Lieferscheine und Rechnungen in der Reihenfolge der Suchwörter. .hole einkaufs "'Bezeichnung'" holt Vorgänge mit dem Suchbegriff 'Bezeichnung'. .hole einkaufs druckbaren bestellungen lscheine rechnungen zahlungen holt Vorgänge aus der Druckwarteschlange. .hole einkaufs offene bestellungen lscheine rechnungen holt die folgenden Vorgänge: * gebuchte Lieferscheine ohne Bestellung und ohne gebuchte Rechnung * Bestellungen, auf die hin ein nicht getrennt zu fakturierender Lieferschein erfasst und gebucht wurde, für die aber noch keine Rechnung gebucht wurde * Lieferscheine, auf die hin eine getrennt zu fakturierende Bestellung erfasst und gebucht wurde, wofür aber noch keine Rechnung gebucht wurde .hole einkaufs uebernahme holt alle noch nicht gebuchten Einkaufslieferscheine und Einkaufsrechnungen aufsteigend nach dem Erstellungsdatum. .hole einkaufs vorschlag holt Bestellvorschläge nach Bestelltermin.	

Weitere Details

.HOLE FERTIGUNGS / .SELECT PRODUCTION - Betriebsauftrag

.HOLE FERTIGUNGS	.SELECT PRODUCTION
deutsch .HOLE FERTIGUNGS	englisch .SELECT PRODUCTION

Das Kommando `.hole fertigungs` ist gleichwertig mit `.hole betriebsauftrag`.

[Weitere Details](#)

.HOLE FIFO / .SELECT FIFO - Ersten Datensatz holen

<code>.HOLE FIFO</code>	<code>.SELECT FIFO</code>
deutsch <code>.HOLE FIFO</code>	englisch <code>.SELECT FIFO</code>
<code>.hole fifo</code> entfernt den ersten Datensatz aus der Warteschlange, die zuvor mit <code>.merke fifo</code> angelegt wurde und lädt ihn ungesperrt in den H-Puffer (FIFO = first in, first out).	

[Weitere Details](#)

.HOLE GRUPPE / .SELECT GROUP - Datenbankgruppe holen

<code>.HOLE GRUPPE</code>	<code>.SELECT GROUP</code>
deutsch <code>.HOLE GRUPPE 'Gruppenangabe' 'Selektionsangabe'</code>	englisch <code>.SELECT GROUP 'group specification' 'selection specification'</code>
Holt Datensätze aus der <code>'Gruppenangabe'</code> . Als <code>'Gruppenangabe'</code> können Datenbanken, Gruppenlisten oder Verweisarten verwendet werden.	

[Weitere Details](#)

.HOLE JAHR / .SELECT YEAR - Besetzt die Datumsvariablen

<code>.HOLE JAHR</code>	<code>.SELECT YEAR</code>
deutsch <code>.HOLE JAHR</code> <code>.HOLE JAHR 'Geschäftsjahr' 'Anfangsgeschäftsmonat' 'Endgeschäftsmonat'</code>	englisch <code>.SELECT YEAR</code> <code>.SELECT YEAR 'fiscal year' 'start month' 'end month'</code>
besetzt die Variablen <code>G ajahr</code> , <code>G ejahr</code> , <code>G amonat</code> , <code>G emonat</code> , <code>G adatum</code> und <code>G edatum</code> .	

[Weitere Details](#)

.HOLE KONTO / .SELECT ACCOUNT - Konto holen

.HOLE KONTO	.SELECT ACCOUNT
deutsch .HOLE KONTO	englisch .SELECT ACCOUNT
.hole konto holt das angegebene Konto.	

[Weitere Details](#)

.HOLE KOPF / .SELECT HEADER - Holen des Tabellenkopfes

.HOLE KOPF	.SELECT HEADER
deutsch .HOLE KOPF .HOLE [MASKE AMASKE] KOPF .HOLE [MASKE] KOPF 'Selektionsstring'	englisch .SELECT HEADER .SELECT [SCREEN ASCREEN] HEADER .SELECT [SCREEN] HEADER 'selection string'
.hole kopf lädt den Kopfteil. Das Kommando wird benötigt, wenn Positionen im Ein-/Verkauf mit .hole zeile durchlaufen werden, da dann der Kopfteil nicht mehr geladen ist. Ist in einem EFOP das Schlüsselwort maske oder amaske angegeben, bezieht sich der Befehl auf den aktuellen Maskenpuffer oder den A-Puffer, andernfalls auf das Bezugsobjekt H. Das Kommando .hole [MASKE] KOPF 'Selektionsstring' prüft, ob das zuvor geladene Objekt den Kriterien des Selektionsstrings entspricht. Wenn das geladene Objekt dem Selektionsstring entspricht, dann ist G mehr = G true, andernfalls ist G mehr = G false.	

[Weitere Details](#)

.HOLE LIFO / .SELECT LIFO - Letzten Datensatz holen

.HOLE LIFO	.SELECT LIFO
deutsch .HOLE LIFO	englisch .SELECT LIFO
.hole lifo entfernt den letzten Datensatz aus der Warteschlange, die zuvor mit .merke lifo angelegt wurde und lädt ihn ungesperrt in den H-Puffer (LIFO = last in, first out).	

[Weitere Details](#)

.HOLE LMENGE / .SELECT SQUANTITY - Lagermengen-Datensatz aus dem Pufferbereich holen

.HOLE LMENGE	.SELECT SQUANTITY
deutsch .HOLE LMENGE	englisch .SELECT SQUANTITY
holt einen gespeicherten Lagermengen-Datensatz aus dem Pufferbereich	

[Weitere Details](#)

.HOLE LOHNFERTIGUNG / .SELECT SUBCONTRACTING - FO-Zugriffe auf Lohnfertigung

.HOLE LOHNFERTIGUNG	.SELECT SUBCONTRACTING
deutsch .HOLE LOHNFERTIGUNG	englisch .SELECT SUBCONTRACTING
holt Lohnfertigungsvorschläge	

[Weitere Details](#)

.HOLE LPLATZ / .SELECT LOCATION - Lagerplatz-Datensatz aus dem Pufferbereich holen

.HOLE LPLATZ	.SELECT LOCATION
deutsch .HOLE LPLATZ	englisch .SELECT LOCATION
holt einen gespeicherten Lagerplatz-Datensatz aus dem Pufferbereich	

[Weitere Details](#)

.HOLE MASKE / .SELECT SCREEN - Tabellenkopf/-zeilen in EFOPs

.HOLE MASKE	.SELECT SCREEN
deutsch .HOLE MASKE .HOLE MASKE [KOPF ZEILE]	englisch .SELECT SCREEN .SELECT SCREEN [HEADER LINE]
lädt den Kopfteil oder Tabellenteil des Bezugsobjektes M	

[Weitere Details](#)

.HOLE MASKE ZEILE / .SELECT MASK LINE - Zeilenselektion im geladenen Puffer

.HOLE [MASKE AMASKE] ZEILE 'Selektionsstring'	.SELECT [SCREEN ASCREEN] LINE 'selection string'
--	---

Sucht in einem geladenen Puffer nach einer Zeile anhand eines Selektionsstrings.
Es sind \$-Selektion und %-Selektion möglich. Optionen wie @bereich und @richtung werden unterstützt. Objektbezogene Optionen (@gruppe, @zeilen, @sort, @ablageart) sind in einer geöffneten Objektauswahl gesperrt.

[Weitere Details](#)

.HOLE MBELEGUNG / .SELECT MACHLOADING - Maschinenbelegung

.HOLE MBELEGUNG	.SELECT MACHLOADING
deutsch .HOLE MBELEGUNG	englisch .SELECT MACHLOADING

.hole mbelegung lädt die Belegung der Maschinengruppe oder Abteilung, die im Hauptpuffer geladen ist, für den Zeitraum G|adatum bis G|edatum. Ausgewertet wird die geladene Tabelle mit der Variablen G|mbel.

[Weitere Details](#)

.HOLE MONAT / .SELECT MONTH - Zeitraum für Finanzbuchhaltung wählen

.HOLE MONAT	.SELECT MONTH
deutsch .HOLE MONAT .HOLE MONAT 'Anfangsmonat' 'Endmonat' .HOLE MONAT 1 2 3 NEU .HOLE MONAT 1-12 13 14 15 ALT	englisch .SELECT MONTH .SELECT MONTH 'start month' 'end month' .SELECT MONTH 1 2 3 NEW .SELECT MONTH 1-12 13 14 15 OLD

.hole monat 'Anfangsmonat' 'Endmonat' legt einen Zeitraum für Finanzbuchhaltungslisten fest.
.hole monat 1|2|3 neu holt den ersten, zweiten oder dritten Monat des neuen Geschäftsjahres.
.hole monat 1-12|13|14|15 alt holt die Monate 1 bis 12 des abgeschlossenen Geschäftsjahres oder die Nachbuchungsmonate.

[Weitere Details](#)

.HOLE MZUORDNUNG / .SELECT MALLOCATION - FO-Zugriffe auf Materialzuordnungen

.HOLE MZUORDNUNG	.SELECT MALLOCATION
deutsch .HOLE MZUORDNUNG	englisch .SELECT MALLOCATION
holt Materialzuordnungen	

[Weitere Details](#)

.HOLE OBJEKT / .SELECT OBJECT - Allgemeines Datenbankobjekt laden

.HOLE OBJEKT	.SELECT OBJECT
deutsch .HOLE OBJEKT "sel"	englisch .SELECT OBJECT "sel"
holt den über die Objektauswahl oder Bereichsdefinition sel gefundenen Datensatz.	

[Weitere Details](#)

.HOLE PLANKARTE / .SELECT PLANCHART - Beschaffung und Materialreservierung

.HOLE PLANKARTE	.SELECT PLANCHART
deutsch .HOLE PLANKARTE	englisch .SELECT PLANCHART

Liefert pro Aufruf den nächsten geplanten Zugang (Beschaffung) oder Abgang (Materialreservierung) für die zuvor geholte Kombination aus Artikel und Lagergruppe, in der Reihenfolge ihrer Ist-Termine. Liegen Beschaffung und Reservierung auf dem gleichen Termin, wird zuerst die Beschaffung und dann die Reservierung berücksichtigt. Außerdem liefert das Kommando die Lagerbestände pro Verwendung/Projekt/Fertigteil Lohnfertigung. Diese können jeweils positiv oder negativ sein.

Folgende Objekte können nach Aufruf des Kommandos im Hauptpuffer geladen sein:

- Zugänge:
positive Lagermengen (Tabellenzeile: Gebindemengen)(V-40-00)
Beschaffungsvorschläge, Beschaffungspositionen (V-04-02, V-04-05)
- Abgänge:
negative Lagermengen (Tabellenzeile: Gebindemengen) (V-40-00)
Materialreservierungen (V-04-04)
Planmengen aus der rollierenden Absatzplanung (Tabellenzeile) (V-121-01)
- Neutral:
eigene Materialzuordnungen (V-28-00)

[Weitere Details](#)

.HOLE SPERREN / .SELECT LOCK - Zugriffssperre beim Holen eines Datensatzes

.HOLE SPERREN	.SELECT LOCK
----------------------	---------------------

deutsch

```
.HOLE ['Sperroptionen']
['Sperrmodifikatoren'] 'Datenbankname'
'Suchbegriff'
.HOLE 'Sperroptionen' ZEILE
```

Sperroptionen	Sperrmodifikatoren
SPERREN	KEINABBRUCH
KEINESPERRE	ABBRUCH
LESESPERRE	LANGWARTEN
AENDERNSPERRE	KURZWARTEN
SCHREIBSPERRE	NICHTWARTEN
FREIGEBEN	WARTEN

englisch

```
.SELECT 'lock options' ['lock
modifier'] 'file name' 'search term'
.SELECT 'lock options' LINE
```

lock options	lock modifier
LOCK	NOTERMINATION
NOLOCK	CANCEL
READLOCK	WAITLONG
MODIFICATIONLOCK	WAITSHORT
WRITELOCK	NOWAIT
RELEASE	WAIT

```
KEINESPERRE
LESESPERRE
AENDERNSPERRE
SCHREIBSPERRE
FREIGEBEN ABRUCH
LANGWARTEN
KURZWARTEN
NICHTWARTEN
WARTEN NOLOCK
READLOCK
MODIFICATIONLOCK
WRITELOCK
RELEASE CANCEL
WAITLONG
WAITSHORT
NOWAIT
WAIT
```

`.hole 'Sperroptionen' ['Sperrmodifikatoren'] 'Datenbankname' 'Suchbegriff'` liest den angegebenen Datensatz und sperrt ihn für andere Benutzer.

`.hole 'Sperroptionen' zeile` beschafft eine Einkaufs-/Verkaufsposition.

Weitere Details

.HOLE TEILE / .SELECT PARTS - Artikeldatensatz an Lagergruppe anpassen

.HOLE TEILE

.SELECT PARTS

deutsch .HOLE TEILE SPEZIFISCH 'Lagergruppe'	englisch .SELECT PARTS SPECIFIC 'warehouse group'
.hole teile passt den Artikeldatensatz auf die Lagergruppe an	

[Weitere Details](#)

.HOLE VERKAUFS / .SELECT SALES - Für Datensätze des Verkaufs

.HOLE VERKAUFS	.SELECT SALES
deutsch .HOLE VERKAUFS .HOLE VERKAUFS ABLAGE .HOLE VERKAUFS ALPHABETISCH .HOLE VERKAUFS "'Bezeichnung'" .HOLE VERKAUFS DRUCKBAREN ANGEBOTE AUFTRAG LSCHEINE RECHNUNGEN Z AHLUNGEN .HOLE VERKAUFS OFFENE ANGEBOTE AUFTRAG LSCHEINE RECHNUNGEN Z AHLUNGEN PROVISION .HOLE VERKAUFS UEBERNAHME	englisch .SELECT SALES .SELECT SALES FILINGSYSTEM .SELECT SALES ALPHABETICAL .SELECT SALES "'description'" .SELECT SALES PRINTABLE QUOTATIONS SORDERS DELNOTES INVOICES P AYMENTS .SELECT SALES OUTSTANDING QUOTATIONS SORDERS DELNOTES INVOICES P AYMENTS COMMISSION .SELECT SALES TAKEOVER

.hole verkaufs holt die folgenden Vorgänge in der Reihenfolge der Nummern:
 * Angebote * noch nicht vollständig berechnete Aufträge * Bestellungen * noch nicht gebuchten
 Lieferscheine * noch nicht gebuchten Rechnungen * gebuchte Lieferscheine, die getrennt zu fakturieren
 sind .hole verkaufs ablage holt Vorgänge aus der Verkaufsablage in der Reihenfolge, wie sie im
 Ablage-Suchbaum stehen.
 .hole verkaufs alphabetisch holt Angebote, Aufträge, Lieferscheine und Rechnungen in der
 Reihenfolge der Suchwörter.
 .hole verkaufs "'Bezeichnung'" holt Vorgänge mit dem Suchbegriff 'Bezeichnung'.
 .hole verkaufs druckbaren angebote | auftrag | auftraege | lscheine |
 rechnungen | zahlungen holt Vorgänge aus der Druckwarteschlange.
 .hole verkaufs offene angebote | auftrag | auftraege | lscheine | rechnungen |
 zahlungen | provision holt folgende Vorgänge:
 * Angebotsobligo * Auftragsobligo * Lieferobligo, positionsweise Fälligkeitsreihenfolge * Rechnungsobligo
 für Lieferscheine ohne Auftrag * Zahlungsobligo * offene Posten, wenn art = leer * Anzahlungen, wenn
 art = A * Provisionsobligo .hole verkaufs uebernahme holt alle noch nicht gebuchten
 Verkaufslieferscheine und Verkaufsrechnungen aufsteigend nach dem Erstellungsdatum.

[Weitere Details](#)

.HOLE VERWENDUNG / .SELECT APPLICATION - Verwendungsnachweis holen

.HOLE VERWENDUNG	.SELECT APPLICATION
deutsch .HOLE VERWENDUNG	englisch .SELECT APPLICATION
.hole verwendung zeigt den Verwendungsnachweis des zuvor geholten Artikels an. Damit werden nacheinander alle Beschaffungen bzw. Reservierungen geladen, die durch .merke verwendung gesammelt wurden. Bei jedem Holen werden die beiden Variablen 'G bv kenn' (Text) und 'G bvmge' (real) vorbelegt.	

[Weitere Details](#)

.HOLE VKZ / .SELECT VKZ - Verkehrszahlen holen

.HOLE VKZ	.SELECT VKZ
deutsch .HOLE VKZ [IST PLAN] ['Geschäftsjahr'] ['Währung']	englisch .SELECT VKZ [ACTUAL TARGET] ['fiscal year'] ['currency']
lädt die monatlichen Verkehrszahlen und Umsatzzähler von Kunden, Lieferanten, Mitarbeitern, Artikeln, Sachkonten und Anlagen	

[Weitere Details](#)

.HOLE VON / .SELECT FROM - Lesen aus einer Datei

.HOLE VON	.SELECT FROM
deutsch .HOLE VON .HOLE VON 'Dateiname' 'Freitext' 'Variablenliste' [^]	englisch .SELECT FROM .SELECT FROM 'file name' 'free text' 'list of variables' [^] BREAK
.hole von 'Dateiname' 'Variablenliste' [^] liest eine Zeile aus einer FO-Datei (einschließlich Freitext) in die Variablen der Variablenliste ein. Wie bei .zuweisen und anders als bei .formel findet eine Wertebereichsprüfung statt. Die Variablenliste besteht aus Variablenname Trennzeichen [Variablenname Trennzeichen ...]. Trennzeichen sind ASCII-Sonderzeichen, z.B. ; (Semikolon). Die Trennzeichen dürfen innerhalb des Befehls variieren. Erlaubte Trennzeichen Die Angabe des Hochzeichens (^) erzwingt das Lesen bis zum Zeilenende. .hole von (ohne weitere Argumente) schließt Dateien, die mit .hole von 'Dateiname' 'Variablenliste' geöffnet wurden.	

.HOLE WEITER / .SELECT CONTINUE - Weiterschalten

.HOLE ['Sperroptionen'] WEITER	.SELECT ['Sperroptionen'] CONTINUE
deutsch .HOLE WEITER	englisch .SELECT CONTINUE
schaltet im Bereich weiter, ohne dass der Datenbankname wiederholt werden muss	

.HOLE ZEILE / .SELECT LINE - Holen von Tabellenzeilen

.HOLE ZEILE	.SELECT LINE
deutsch .HOLE ZEILE .HOLE [MASKE AMASKE] ZEILE ['Zeilennummer' 'Zeilen-ID' 'Selektionsstring']	englisch .SELECT LINE .SELECT [SCREEN ASCREEN] LINE ['line number' 'line ID' 'selection string']
.hole zeile beschafft die erste oder nächste Zeile der Tabelle. .hole zeile 0 positioniert auf den Kopf, synonym zu .hole kopf. Ist das Schlüsselwort maske bzw. amaske angegeben, bezieht sich der Befehl auf den M-Puffer bzw. auf den A-Puffer, andernfalls auf das Bezugsobjekt H. .hole zeile 'Zeilennummer' 'Zeilen-ID' 'Selektionsstring' beschafft die angegebene Zeile einer Tabelle. Fehlt eine Angabe zur Zeile, wird die nächste Zeile nach der aktuellen geholt.	

..!INTERPRETER / ..!INTERPRETER - Arbeitsweise des FO-Interpreters

..!INTERPRETER	..!INTERPRETER
deutsch ..!interpreter ..!interpreter [declaration] [german english] [englvar] [translate] [noabbrev]	englisch ..!interpreter ..!interpreter [declaration] [german english] [englvar] [translate] [noabbrev]

declaration verlangt, dass jede vom Benutzer definierte Variable vor ihrer Verwendung mit **.art** deklariert werden muss.

german oder **english** legt die Befehlssprache des FOPs fest.

englvar legt die **Variablensprache** des FOPs auf Englisch fest.

Ist die Option **translate** gesetzt, wird bei einer **Übersetzungsanforderung** die Befehlssprache dieses Programms umgestellt.

noabbrev verlangt, dass Variablen nicht abgekürzt werden.

Weitere Details

.KOMMANDO / .COMMAND - Aufruf von Abas Kommandos

.KOMMANDO	.COMMAND
deutsch .KOMMANDO -PARALLEL [-ID <Variablenname>] 'Abas Kommando' .KOMMANDO -WARTEN [-ID <M Variablenname>] 'Abas Kommando' .KOMMANDO -FENSTER 'maskid' [-NEU 'Abas '] [-ID <Variablenname>] 'Maskenaktionen' .KOMMANDO -BATCH 'Abas Kommando' BREAK	englisch .COMMAND -PARALLEL [-ID <M variable- name>] 'Abas command' .COMMAND -WAIT [-ID <variable-name>] 'Abas command' .COMMAND -WINDOW 'maskid' [-NEW 'Abas command'] [-ID <variable-name>] 'screen actions' .COMMAND -BATCH 'Abas command' BREAK
.kommando 'Abas Kommando' führt ein Abas Kommando aus, als wäre es von der Kommandoübersicht aus aufgerufen worden. Mit -ID <Variablenname> können Sie den Namen einer FO-Variablen angeben, in die die ID der geänderten Maske geschrieben wird. .kommando -parallel 'Abas Kommando' startet ein weiteres Kommando (Maske, FOP), das parallel zu dem FOP ausgeführt wird, das .kommando aufruft. Es wird nicht gewartet, bis das Kommando beendet ist. .kommando -warten [-id 'M ymaskid'] 'Abas Kommando' startet ein weiteres Kommando (Maske, FOP). .kommando -warten kehrt erst zurück, wenn die Maske wieder geschlossen wurde. Die Variable 'M ymaskid' enthält die Identifizierung der Maske; es handelt sich um eine Skipvariable des Datensatzes, damit sie bei einem erneuten Aufruf des EFOPs wieder verwendet kann. .kommando.fenster 'maskid' [-neu 'Abas Kommando'] 'Maskenaktionen' verändert eine bereits geöffnete Maske. Mit der Option -neu wird eine neue Maske geöffnet, falls keine Maske mit der angegebenen ID maskid vorhanden ist. .kommando -batch 'Abas Kommando' führt ein Abas Kommando im Hintergrund aus, indem das Kommando von edpimport.sh ausgeführt wird.	

Weitere Details

.KOPIEREN / .COPY - Variablenwerte kopieren

.KOPIEREN	.COPY
-----------	-------

deutsch .KOPIEREN 'X Variablenname' = 'Ausdruck'	englisch .COPY 'X variable name' = 'expression'
Der 'Variablen' wird das Rechenergebnis des 'Ausdrucks' zugewiesen. X steht für ein Bezugsobjekt.	

[Weitere Details](#)

.LADE / .LOAD - Datensätze laden

.LADE	.LOAD
deutsch .LADE X 'Datenbank/Gruppenangabe' 'Selektionsangabe' .LADE X GRUPPE 'Gruppenangabe' 'Selektionsangabe' .LADE X 'Datenbank/Gruppenangabe' .LADE X DAZU 'Verweisfeld' .LADE X WEITER .LADE X .LADE X ZEILE ['n']	englisch .LOAD X 'database/group specification' 'selection specification' .LOAD X GROUP 'group specification' 'selection specification' .LOAD X 'database/group specification' .LOAD X ADD 'reference field' .LOAD X CONTINUE .LOAD X .LOAD X LINE ['n']
.lade X 'Datenbank/Gruppenangabe' lädt den nächsten Datensatz eines zuvor geladenen Bereichs mit .lade X 'Datenbank/Gruppenangabe' 'Selektionsangabe'. X steht für einen .lade-Puffer, der durch eine Ziffer von 0 bis 9 repräsentiert wird. Daneben gibt es auch Puffernamenvariablen. Ohne weiteres Argument setzt keine neue Suche auf. 'Datenbank/Gruppenangabe' steht für einen Datenbanknamen. Als Gruppenangabe können Datenbanken, Gruppenlisten oder Verweisarten verwendet werden. .lade X gruppe 'Gruppenangabe' 'Selektionsangabe' definiert einen Bereich mit Gruppeneinschränkung und lädt den ersten Datensatz des Bereichs. .lade X dazu 'Verweisfeld' definiert einen Bereich mit maximal einem Element, nämlich dem Datensatz, auf den 'Verweisfeld' zeigt. .lade X weiter schaltet auf das nächste Objekt im Bereich. .lade X löscht Puffer X und Bereich. .lade X zeile ['n'] lädt die nächste bzw. die n-te Zeile bei Objekten mit Tabelle bzw. im Ein- und Verkauf. Der Bereich und das geladene Objekt bleiben erhalten.	

[Weitere Details](#)

.LAENGE / .LENGTH - Seitenlänge

.LAENGE	.LENGTH
---------	---------

deutsch .LAENGE .LAENGE 'Anzahl Zeilen'	englisch .LENGTH .LENGTH 'number lines'
legt die Anzahl der Zeilen pro Seite fest	

[Weitere Details](#)

.LESEN / .READ - Tastatureingaben einlesen

.LESEN	.READ
deutsch .LESEN "Text" Variablenname	englisch .READ "text" variablename
Liest Tastatureingaben ein. Die Eingabe muss zur Art der Variablen passen. "Text" wird als Anforderungstext zur Eingabe benutzt.	

[Weitere Details](#)

.LINKS / .LEFT - Linker Rand

.LINKS	.LEFT
deutsch .LINKS 'n' [MM] .LINKS + - 'n' [MM] .LINKS 'n' + - 'm' [MM]	englisch .LEFT 'n' [MM] .LEFT + - 'n' [MM] .LEFT 'n' + - 'm' [MM]
.links 'n' setzt den linken Rand bis zur Spalte n, d.h., der folgende Text beginnt in Spalte n. .links '+n' vergrößert, .links '-n' verkleinert einen bestehenden linken Rand um n Spalten. .links 'n' '+m' vergrößert, .links 'n' '-m' verkleinert in jedem mehrzeiligen Absatz den Abstand der ersten Zeile zum linken Rand um m Spalten. .links 'n' [MM] setzt den linken Rand auf n Millimeter.	

[Weitere Details](#)

.LOESCHEN / .DELETE - Datensatz löschen

.LOESCHEN	.DELETE
deutsch .LOESCHEN	englisch .DELETE
löscht einen Stammdatensatz	

[Weitere Details](#)

.MACHE / .MAKE - Anlegen und Löschen von Datensätzen bzw. Zeilen

.MACHE	.MAKE
deutsch .MACHE .MACHE 'Datenbank' ['Gruppe'] .MACHE [MASKE] ZEILE [+] .MACHE [MASKE] ZEILE - .MACHE [MASKE] ZEILE -- .MACHE MASKE ZEILE +[0] .MACHE MASKE ZEILE -0	englisch .MAKE .MAKE 'file name' ['group'] .MAKE [SCREEN] LINE [+] .MAKE [SCREEN] LINE - .MAKE [SCREEN] LINE -- .MAKE SCREEN LINE +[0] .MAKE SCREEN LINE -0
.mache ermöglicht das Anlegen bestimmter Datensätze (Zusatztable, EDI, Inventur, Speditionsauftrag, Charge, Dokument) und das Anlegen und Löschen von Zeilen in beliebigen Datenbanken. .mache 'Datenbank' ['Gruppe'] legt einen neuen Datensatz im Puffer H an; möglich in folgenden Datenbanken: Zusatztable, EDI, Inventur, Speditionsauftrag, Charge, Dokument, Behälter. Ohne Angabe einer Gruppe wird von der FO ein Datensatz in Gruppe 0 erzeugt. Für 'Gruppe' ist gruppe0, gruppe1, ..., gruppe19 anzugeben. .mache [maske] zeile [+] legt eine neue Zeile im M-Puffer (maske) an. Die Zeile wird nach der aktuellen Zeile eingefügt und auf die erzeugte Zeile wird positioniert. .mache [maske] zeile - entfernt die aktuelle Tabellenzeile, .mache [maske] zeile -- entfernt alle Tabellenzeilen. .mache maske zeile +[0] erhöht die Geschwindigkeit beim Einfügen, .mache maske zeile -0 erhöht die Geschwindigkeit beim Löschen vieler Zeilen einer Maske. (0 für "Optimierung"). Zwischen + bzw. - und 0 darf kein Leerzeichen stehen. Wenn Sie per EFOP Zeilen einfügen oder löschen, bzw. Farben ändern, beachten Sie die Hilfe zum Kommando .farbe.	

[Weitere Details](#)

.MENUE / .MENU - Menüauswahlfenster

.MENUE	.MENU
deutsch .MENUE [(0,0,'Zeilennummer')]["Überschrift"] ['Spaltenformatzeile'] Menüzeile 1 [>> 'Punktbefehl' Text] Menüzeile 2 [>> 'Punktbefehl' Text] Menüzeile n [>> 'Punktbefehl' Text] ..	englisch .MENU [(0,0,'line number')] ["header"] ['tab line'] menu line 1 [>> 'dot command' text] menu line 2 [>> 'dot command' text] menu line n [>> 'dot command' text] ..

.menue erstellt in der Ausgabe ein Menü, auf dessen Zeilen mit den Tasten **<Pfeil auf>** und **<Pfeil ab>** oder dem Anfangsbuchstaben der gewünschten Zeile positioniert werden kann. Mit **<Return>** wird die Menüzeile ausgewählt.

Die **Überschrift** wird über die Fensterbreite zentriert.

Die getroffene Auswahl kann mit der Variablen **'G|menue'** abgefragt werden. Bei **<Abbruch>** ist der Wert der Variablen **'G|menue'** = 0, ansonsten gleich der Nummer der gewählten Menüzeile.

Die **Zeilennummer** legt die Position des **Menüauswahlbalkens** fest. Ist die **Zeilennummer** größer als die Anzahl der Menüzeilen, wird auf die erste Zeile positioniert.

Die **Menüzeilen** dürfen Texte und Variablen enthalten (z.B. **'G|frei'**), die wie üblich durch ihren Wert ersetzt werden.

Die **Spaltenformatzeile** ermöglicht, die **Auswahltexte in Spalten** darzustellen. Ein Auswahltext wird nur dann entsprechend dem Spaltenformat angezeigt, wenn die Spaltentexte durch ein Leerzeichen getrennt sind. Auswahltexte, die nicht zu diesem Spaltenformat passen, werden in der GUI unformatiert, d.h. durchgängig angezeigt, z.B. Trennlinien.

Durch das **Anbinden von Punktbefehlen und Text** kann das Menü interaktiv bedient werden.

Weitere Details

.MERKE / .RESERVE - Aktuellen Ladezustand von .hole- und .dazu-Puffer holen

.MERKE	.RESERVE
deutsch	englisch
.MERKE	.RESERVE
.MERKE FIFO	.RESERVE FIFO
.MERKE LIFO	.RESERVE LIFO
.MERKE BINDUNG	.RESERVE LINK
.MERKE VERWENDUNG [+]	.RESERVE APPLICATION [+]
.MERKE -	.RESERVE -

.merke speichert die Nummer, den Sperrzustand und die Tabellenzeile des aktuellen Datensatzes, sowie den Suchbereich. Diese Informationen können mit **.hole** wieder geladen werden.

.merke fifo **.merke lifo** speichert die Nummer des aktuellen Datensatzes in einen Puffer (FIFO = first in, first out, LIFO = last in, first out). Der Datensatz wird immer am Ende des Puffers (der Warteschlange) gespeichert. Der Datensatz kann mehrfach in der Warteschlange gespeichert werden. Aktuelle Obergrenze für die Warteschlange ca. 100000 Einträge. Es kann nur entweder eine **fifo**- oder eine **lifo**-Warteschlange geben. Wenn Sie zwischen beiden wechseln, dann wird die Warteschlange initialisiert.

Die Datensätze können mit **.hole fifo**, entsprechend mit **.hole lifo** wieder geladen werden.

.merke bindung holt alle Zugänge zu einem Abgang, die ihn bedient haben oder alle Abgänge zu einem Zugang, in die er eingeflossen ist.

In der Datenbankvariable **zuganglj** wird der Zugangsjournaleintrag und in der Datenbankvariablen **abganglj** wird der Abgangsjournaleintrag gespeichert.

.merke verwendung [+] liefert Beschaffungen zu Materialreservierungen.

.merke - führt dazu, dass alle mit **.merke**, **.merke fifo**, bzw. **.merke lifo** gemerkten Objekte aus dem Puffer gelöscht werden. D.h. ein nachfolgendes **.hole**, **.hole fifo**, bzw. **.hole lifo** liefert keinen Datensatz. Der Zustand der Variable **G|mehr** ist **G|false**.

Weitere Details

.MERKE VERWENDUNG / .RESERVE APPLICATION - Beschaffungen zu Materialreservierungen

.MERKE VERWENDUNG	.RESERVE APPLICATION
deutsch .MERKE VERWENDUNG [+]	englisch .RESERVE APPLICATION [+]
.merke verwendung: Ist eine Reservierung geladen, werden durch dieses Kommando die Bezugsquellen zusammengestellt, die sie bedienen. Ist eine Beschaffung geladen, werden durch dieses Kommando alle Reservierungen zusammengestellt, die von ihr bedient werden. .merke verwendung liefert zusätzlich zu einer noch nicht eingeplanten variantenbezogenen Reservierung die zugehörige variantenbezogene Beschaffung und umgekehrt.	

Weitere Details

.OBEN / .UP - Kopfzeilen

.OBEN	.UP
deutsch .OBEN	englisch .UP
definiert den Obenpuffer. Der folgende Text bis zum nächsten Kommando erscheint als Kopf auf jeder folgenden Seite.	

Weitere Details

.PC.COPY / .PC.COPY - Datei kopieren

.PC.COPY	.PC.COPY
deutsch	englisch
.PC.COPY [-PC] [-QUIET] [-CONV] [-BIN] 'Quelldatei' 'Zielfeldatei'	.PC.COPY [-PC] [-QUIET] [-CONV] [-BIN] 'source file' 'target file'
kopiert eine Datei vom Server auf den PC-Client oder umgekehrt. Wenn das Kopieren erfolgreich war, wird G mehr auf G true gesetzt, andernfalls auf G false. -PC gibt an, dass die Zielfeldatei auf dem PC-Client liegt. Fehlt -PC, liegt die Zielfeldatei auf dem Server. -CONV konvertiert von dem Abas Zeichensatz in den Zeichensatz UTF-16 (siehe Unicode-Unterstützung). Ist -CONV nicht angegeben, findet keine Konvertierung statt. -QUIET verhindert das Melden von Fehlern. (Statusvariable G mehr wird gesetzt.) Ist -QUIET nicht angegeben, werden Fehler wie fehlende Dateien oder fehlende Verzeichnisse gemeldet. -BIN steht für Binärdatei; es finden keinerlei Konvertierungen statt. Ist -BIN nicht angegeben, wird eine Zeilenende-Konvertierung ausgeführt (Server: New Line, PC-Client: Carriage Return + New Line). Die Optionen -BIN und -CONV dürfen nicht gleichzeitig angegeben werden.	

[Weitere Details](#)

.PC.EXEC / .PC.EXEC - Programm ausführen

.PC.EXEC	.PC.EXEC
deutsch	englisch
.PC.EXEC 'Programmname' ['Argumente'] .PC.EXEC -WARTEN 'Programmname' ['Argumente']	.PC.EXEC 'program name' ['arguments'] .PC.EXEC -WAIT 'program name' ['arguments']
.pc.exec 'Programmname' ['Argumente'] führt auf dem PC-Client das angegebene Programm aus. Das Kommando startet das PC-Programm lediglich, es wartet nicht auf seine Beendigung. Wenn das Programm gestartet werden konnte, wird G mehr auf G true gesetzt, andernfalls auf G false. Es wird keine Fehlermeldung ausgegeben. Der Exit-Code des gestarteten Programms kann nicht abgefragt werden. Die Option -warten bewirkt, dass der Befehl erst nach dem Ende des gestarteten Programms zurückkehrt. Diese Option ist nur auf der GUI verfügbar.	

[Weitere Details](#)

.PC.OPEN / .PC.OPEN - Datei öffnen

.PC.OPEN	.PC.OPEN
----------	----------

deutsch	englisch
<code>.PC.OPEN [-DATEI] 'Dateiname' - Keine REST API-Unterstützung</code>	<code>.PC.OPEN [-FILE] 'file name' - No REST API support</code>
<code>.PC.OPEN -AKTION 'Aktion' [-PARAMETER 'Parameter'] [-DATEI] 'Dateiname'</code>	<code>.PC.OPEN -ACTION 'action' [-PARAMETER 'parameter'] [-FILE] 'file name'</code>



Das Kommando `.pc.open 'Dateiname'` wird ausschließlich im Abas Windows Client unterstützt. Dieses Kommando ist über die REST API sowie in allen darauf basierenden Benutzeroberflächen, z. B. dem Abas Web Client, und Schnittstellen nicht verfügbar.

`.pc.open 'Dateiname'` kann eine Datei auf dem PC-Client mit dem geeigneten Programm öffnen. Die Option `.datei` muss angegeben werden, wenn die Datei auf dem Server unterhalb des win-Verzeichnisses im Mandanten liegt. In diesem Fall muss als Dateiname die relative Pfadangabe ab dem Mandantenverzeichnis verwendet werden.

Mit `.pc.open .aktion 'Aktion'` kann angegeben werden, was das PC-Programm, das gestartet wird, mit der Datei tun soll. Fehlt `.aktion 'Aktion'`, wird die Datei wie bisher geöffnet.

Mit `.pc.open .aktion 'Aktion' -parameter 'Parameter'` kann man einen speziellen Parameter für die angegebene Aktion eintragen. Welche Parameter erlaubt und sinnvoll sind, hängt von der Aktion ab und von dem Programm, das die Datei öffnet.

[Weitere Details](#)

.PRINTLN / .PRINTLN - Text ausgeben

<code>.PRINTLN</code>	<code>.PRINTLN</code>
deutsch	englisch
<code>.PRINTLN [-TEXT] [-XML] [-LFSUPPRESS] [-NOMOREOPTIONS] [-SCREEN] [-DEBUG] [-LOGLEVEL x] 'Text'</code>	<code>.PRINTLN [-TEXT] [-XML] [-LFSUPPRESS] [-NOMOREOPTIONS] [-SCREEN] [-DEBUG] [-LOGLEVEL x] 'text'</code>

`.println -text` gibt den reinen Text ohne Interpretation, z.B. von Variableninhalten, aus.

`.println -xml` maskiert Metazeichen nicht, wenn das Ausgabeformat XML oder HTML ist.

`.println -lfsuppress` gibt den angegebenen Text aus, ohne dass ein Zeilenendezeichen angefügt wird.

`.println -nomoreoptions` zeigt an, dass die folgende Angabe nicht als Option, sondern als Text interpretiert werden soll.

`.println -screen` sorgt für die Ausgabe von Texten in das Ausgabefenster statt in eine evtl. geöffnete Ausgabedatei.

`.println -debug` und `.println -loglevel x` wirken wie `-screen`, nur dass die Ausgabe mit

`.println 'Quelldatei': 'Quellezeile':` eingeleitet wird. Bei `-loglevel x` erfolgt die Ausgabe nur, wenn `P|loglevel > x` ist.

[Weitere Details](#)

.RECHTS / .RIGHT - Rechter Rand

.RECHTS	.RIGHT
deutsch	englisch
.RECHTS 'n' [MM]	.RIGHT 'n' [MM]
.RECHTS + - 'n' [MM]	.RIGHT + - 'n' [MM]
.rechts 'n' setzt den rechten Rand bis zur Spalte n, d.h., der Text endet spätestens in Spalte n - 1. Überhängender Text wird auf die Folgezeilen verteilt, wobei im Blocksatz und Flattersatz die Silbentrennung wirksam wird. .rechts +'n' vergrößert, .links -'n' verkleinert den rechten Rand um n Spalten. .rechts 'n' [MM] setzt den rechten Rand auf n Millimeter.	

[Weitere Details](#)

.SCHUTZ / .PROTECTION - Schreib-/Sichtschutz

.SCHUTZ	.PROTECTION
deutsch	englisch
.SCHUTZ ['Optionen'] [-FELD]	.PROTECTION ['options'] [-FIELD]
'Feldname' ['Zeilennummer']	'field name' ['line number']
.SCHUTZ ['Optionen'] -ZEILE	.PROTECTION ['options'] -LINE 'line number'
'Zeilennummer'	
.SCHUTZ ['Optionen'] -SPALTE	.PROTECTION ['options'] -COLUMN 'field name'
'Feldname'	
.SCHUTZ ['Optionen'] -TABELLE	.PROTECTION ['options'] -TABLE
.SCHUTZ ['Optionen'] -ALLES	.PROTECTION ['options'] -ALL
Optionen	options
-GESCHUETZT	-PROTECTED
-UNSICHTBAR	-HIDDEN
-AUSNAHME	-EXCEPTION
-RESET	-RESET

[-feld] 'Feldname' ['Zeilennummer'] setzt einen Schreibschutz auf das Feld in der angegebenen Zeile. Wenn die Zeilenangabe fehlt, wird 0 angenommen (Kopfteil).

-zeile 'Zeilennummer' setzt einen Schreibschutz auf eine einzelne Zeile. Zeile 0 ist der Kopfteil der Maske.

-spalte 'Feldname' setzt einen Schreibschutz für eine Spalte in der Tabelle. Diese Schutzoption bleibt unberührt vom Löschen und Einfügen von Zeilen.

-tabelle setzt einen Schreibschutz für die ganze Tabelle, aber nicht für den Kopfteil der Maske. Diese Schutzoption bleibt unberührt vom Löschen und Einfügen von Zeilen.

-alles setzt einen Schreibschutz für alle Maskenfelder und Buttons

Optionen	
-geschuetzt	setzt den Schreibschutz. Sie ist der Standardwert, wenn keine Art des Schutzes angegeben wurde, d.h., man kann sie auch weglassen.
-unsichtbar	setzt einen Sichtschutz
-ausnahme	macht zusammen mit -geschuetzt ein geschütztes Feld ausnahmsweise schreibbar. Zusammen mit -unsichtbar wird ein unsichtbares Feld ausnahmsweise sichtbar.
-reset	bewirkt in Kombination mit -geschuetzt oder -unsichtbar ein Aufheben des Schutzes. Der Aufruf muss dieselben Parameter wie der Schutz haben. Ist keine zweite Option angegeben, wird -geschuetzt verwendet.

Optionen	
-geschuetzt	setzt den Schreibschutz. Sie ist der Standardwert, wenn keine Art des Schutzes angegeben wurde, d.h., man kann sie auch weglassen.
-unsichtbar	setzt einen Sichtschutz
-ausnahme	macht zusammen mit -geschuetzt ein geschütztes Feld ausnahmsweise schreibbar. Zusammen mit -unsichtbar wird ein unsichtbares Feld ausnahmsweise sichtbar.
-reset	bewirkt in Kombination mit -geschuetzt oder -unsichtbar ein Aufheben des Schutzes. Der Aufruf muss dieselben Parameter wie der Schutz haben. Ist keine zweite Option angegeben, wird -geschuetzt verwendet.

Weitere Details

.SEITE / .PAGE - Seitenbereich und Seitenumbruch

.SEITE	.PAGE
deutsch .SEITE .SEITE 'Seitennummer' .SEITE 'erste Seite'-'letzte Seite'	englisch .PAGE .PAGE 'page number' .PAGE 'first page'-'last page'
.seite löst einen Seitenvorschub aus, falls die bisherige Seite schon angebrochen war. Auf dem Bildschirm wird der Seitenwechsel durch eine Raute (#) gekennzeichnet. .seite 'Seitennummer' löst einen Seitenvorschub aus und setzt einen Seitenzähler. Die Nummer der neuen Seite wird in der FO-Variablen G seite gespeichert. .seite 'erste Seite'-'letzte Seite' gibt die Seiten des angegebenen Bereichs aus. Mit der ersten Zeile der letzten Seite beendet sich der FO-Interpreter.	

[Weitere Details](#)

.SETZE NUMMERNZÄHLER / .SET NUMBERCOUNTER - Nummernzähler setzen

.SETZE NUMMERNZÄHLER	.SET NUMBERCOUNTER
deutsch .SETZE NUMMERNZÄHLER 'nummer' varname	englisch .SET NUMBERCOUNTER 'idno' varname
.setzenummernzähler 'identnummer' varname erhöht den Nummernzähler identnummer um 1.	

[Weitere Details](#)

.SETZE / .SET - Zeichen mit Sonderbedeutung festlegen

.SETZE	.SET
deutsch .SETZE .SETZE 'Schlüsselwort' [= 'Zeichen']	englisch .SET .SET 'key word' [= 'character']
.setze schaltet die Voreinstellung wieder ein. .setze 'Schlüsselwort' = 'Zeichen' definiert für das Schlüsselwort ein neues Zeichen.	

[Weitere Details](#)

.SETZE ABBRUCH / .SET CANCEL - Abbrechen des FOPs verhindern

.SETZE ABBRUCH	.SET CANCEL
deutsch .SETZE ABBRUCH - .SETZE ABBRUCH -- .SETZE ABBRUCH + [n]	englisch .SET CANCEL - .SET CANCEL -- .SET CANCEL + [n]
.setze abbruch - verhindert, dass das Programm abgebrochen werden kann. Wenn das Programm auf einen durch einen anderen Benutzer belegten Datensatz stößt, ist ein Abbruch trotzdem möglich. Dabei handelt es sich um einen normalen Abbruch des Sperrdialogs. Das FOP läuft weiter. .setze abbruch -- bewirkt, dass auch in Sperrsituationen nicht ohne weiteres abgebrochen werden kann. Ein Abbruch wird wie ein Kritischer Abbruch des Sperrdialogs behandelt. .setze abbruch ermöglicht, dass das Programm abgebrochen werden kann. Mit der nichtnegativen Zahl n kann man den Exit-Status für EFOPs festlegen, der bei Abbruch durch den Benutzer zurückgegeben wird.	

[Weitere Details](#)

.SETZE ABLAGEKENNZEICHEN / .SET FILINGSYSTEMCODE - Ablagekennzeichen setzen

.SETZE ABLAGEKENNZEICHEN	.SET FILINGSYSTEMCODE
deutsch .SETZE ABLAGEKENNZEICHEN + .SETZE ABLAGEKENNZEICHEN -	englisch .SET FILINGSYSTEMCODE + .SET FILINGSYSTEMCODE -
.setze ablagekennzeichen + gibt bei Ausgabe von Verweisen auf abgelegte Objekte ein Plus vor dem Suchwort oder der Identnummer aus. .setze ablagekennzeichen - unterdrückt bei Ausgabe von Verweisen auf abgelegte Objekte das Plus vor Suchwort oder Identnummer.	

[Weitere Details](#)

.SETZE ALLEIN / .SET SINGLEUSER - Allein-Sperre setzen/lösen

.SETZE ALLEIN	.SET SINGLEUSER
---------------	-----------------

deutsch	englisch
<code>.SETZE ALLEIN +</code>	<code>.SET SINGLEUSER +</code>
<code>.SETZE ALLEIN -</code>	<code>.SET SINGLEUSER -</code>
<code>.SETZE ALLEIN +-</code>	<code>.SET SINGLEUSER +-</code>
<code>.SETZE ALLEIN ++</code>	<code>.SET SINGLEUSER ++</code>

`.setze allein` setzt eine Sperre, die verhindert, dass sich weitere Benutzer im gesperrten Mandanten anmelden können.

`.setze allein -` löscht diese Sperre.

`.setze allein +-` stellt fest, ob die jeweilige Sperre vom Benutzer selbst gehalten wird, falls ja, wird G|mehr gesetzt.

`.setze allein ++` setzt eine Allein-Sperre, die das aktuelle Textkommando überdauert.

[Weitere Details](#)

.SETZE AMASKE / .SET ASCREEN - A-Puffer zum M-Puffer machen

<code>.SETZE AMASKE</code>	<code>.SET ASCREEN</code>
deutsch	englisch
<code>.SETZE AMASKE + -</code>	<code>.SET ASCREEN + -</code>
macht den A-Puffer zum M-Puffer	

[Weitere Details](#)

.SETZE AUFZAEHLTYP / .SET ENUMERATIONTYPE - Aufzähltyp als aktuellen setzen

<code>.SETZE AUFZAEHLTYP</code>	<code>.SET ENUMERATIONTYPE</code>
deutsch	englisch
<code>.SETZE AUFZAEHLTYP 'n'</code>	<code>.SET ENUMERATIONTYPE 'n'</code>
versucht, den Aufzähltyp A'n' als aktuellen zu setzen, um anschließend seine Werte ermitteln zu können	

[Weitere Details](#)

.SETZE AUSGABE / .SET OUTPUT - Substitutionen in der Ausgabezeile

<code>.SETZE AUSGABE</code>	<code>.SET OUTPUT</code>
deutsch	englisch
<code>.SETZE AUSGABE "Ersetzung"</code>	<code>.SET OUTPUT "replacement"</code>

erlaubt, Substitutionen für die Ausgabezeile global einzustellen

[Weitere Details](#)

.SETZE AUSWAEHR / .SET OUTCURR - Ausgabewährung vorbelegen

.SETZE AUSWAEHR	.SET OUTCURR
deutsch .SETZE AUSWAEHR .SETZE AUSWAEHR 'Währung'	englisch .SET OUTCURR .SET OUTCURR 'currency'
.setze auswaehr 'Währung' belegt die globale FO-Variable G auswaehr mit der angegebenen Währung.	

[Weitere Details](#)

.SETZE BEREICHSENDE / .SET RANGETO - Bearbeitung des Datensatzbereichs beenden

.SETZE BEREICHSENDE	.SET RANGETO
deutsch .SETZE BEREICHSENDE +	englisch .SET RANGETO +
erlaubt es, nach dem Speichern die Bearbeitung des Datensatzbereichs zu beenden	

[Weitere Details](#)

.SETZE BUCHUNGSKREIS / .SET SETOFBOOKS - Buchungskreis setzen

.SETZE BUCHUNGSKREIS	.SET SETOFBOOKS
deutsch .SETZE BUCHUNGSKREIS .SETZE BUCHUNGSKREIS 'Suchwort' 'Identnummer' 'Index'	englisch .SET SETOFBOOKS .SET SETOFBOOKS 'search word' 'identity number' 'index'
ermöglicht die Zugriffe auf den Buchungsumfang einer Rechnungslegung. Index kann den Wert 0, 1, 2, 3, 4 oder 5 annehmen.	

[Weitere Details](#)

.SETZE DEBUG / .SET DEBUG - Debuggen von FOPs

.SETZE DEBUG	.SET DEBUG
deutsch .SETZE DEBUG + .SETZE DEBUG - .SETZE DEBUG ++	englisch .SET DEBUG + .SET DEBUG - .SET DEBUG ++
.setze debug + schaltet den Debug-Modus ein. .setze debug - schaltet den Debug-Modus aus. .setze debug ++ setzt einen permanenten Debug-Modus, in dem jedes (E)FOP mit Debugger gestartet wird.	

[Weitere Details](#)

.SETZE DEZPUNKT / .SET DECPOINT - Dezimaltrennzeichen setzen

.SETZE DEZPUNKT	.SET DECPOINT
deutsch .SETZE DEZPUNKT .SETZE DEZPUNKT 'Zeichen' .SETZE DEZPUNKT 'G dezpunkt englvar decPt '	englisch .SET DECPOINT .SET DECPOINT 'character' .SET DECPOINT 'G dezpunkt englvar decPt '
.setze dezpunkt setzt das Dezimaltrennzeichen (auch Dezimalpunkt) bei der Ausgabe von Real-Zahlen auf den Standard, je nach der Art des Feldes. .setze dezpunkt 'Zeichen' setzt das Dezimaltrennzeichen auf das angegebene Zeichen. .setze dezpunkt G dezpunkt setzt das Dezimaltrennzeichen auf das Zeichen, das im Konfigurationsdatensatz festgelegt ist.	

[Weitere Details](#)

.SETZE DIAGNOSE / .SET ERRORREPORT - Fehlerprotokoll

.SETZE DIAGNOSE	.SET ERRORREPORT
deutsch .SETZE DIAGNOSE +	englisch .SET ERRORREPORT +

erstellt bei Abbruch der FO wegen eines Fehlers die [Administrationsmeldung 2399: FOP: Programm-Abbruch](#). Durch das Feld [Vorbelegung für FO-Kommando](#) `.setze diagnose` in [Konfiguration zeigen] können Sie die Vorbelegung für das Fehlerprotokoll für alle FO-Programme steuern. Ist die Option gesetzt, wirkt die Funktion des Feldes wie das Kommando `.setze diagnose +` zu Beginn jedes FO-Programms. Sie können das Fehlerprotokoll jederzeit mit `.setze diagnose -` deaktivieren.

[Weitere Details](#)

.SETZE DOPPELPUNKT / .SET COLON - HTML-Befehle markieren

<code>.SETZE DOPPELPUNKT</code>	<code>.SET COLON</code>
deutsch <code>.SETZE DOPPELPUNKT +</code>	englisch <code>.SET COLON +</code>
setzt den Doppelpunkt als Interpretationszeichen für HTML-Befehle	

[Weitere Details](#)

.SETZE DRUCKEN / .SET PRINT - Zweiten Druckpuffer aktivieren

<code>.SETZE DRUCKEN</code>	<code>.SET PRINT</code>
deutsch <code>.SETZE DRUCKEN + -</code>	englisch <code>.SET PRINT + -</code>
Für dieses Kommando gibt es einen Stapel von zwei P-Puffern. Nach <code>.setze drucken +</code> ist der zweite P-Puffer aktiviert; er enthält eine exakte Kopie des ersten. Nach dem folgenden <code>.setze drucken -</code> wird der ursprüngliche P-Puffer wieder sichtbar; er bleibt die ganze Zeit unverändert.	

[Weitere Details](#)

.SETZE ERSETZUNG / .SET REPLACEMENT - Variablenersetzungszeichen setzen

<code>.SETZE ERSETZUNG</code>	<code>.SET REPLACEMENT</code>
deutsch <code>.SETZE ERSETZUNG 'Zeichen'</code>	englisch <code>.SET REPLACEMENT 'character'</code>
Setzt das Variablenersetzungszeichen. Standardwert ist der Apostroph.	

[Weitere Details](#)

.SETZE FENSTERSCHLIESSEN / .SET CLOSEWINDOW - Ausgabefenster schließen

.SETZE FENSTERSCHLIESSEN	.SET CLOSEWINDOW
deutsch .SETZE FENSTERSCHLIESSEN + -	englisch .SET CLOSEWINDOW + -
.setze fensterschliessen legt fest, dass am Ende das FO-Ausgabefenster geschlossen werden soll. .setze fensterschliessen - schaltet diesen Programmzustand wieder aus.	

[Weitere Details](#)

.SETZE FOEXTERN / .SET FOEXTERN - Verhalten wie in Maske aktivieren

.SETZE FOEXTERN	.SET FOEXTERN
deutsch .SETZE FOEXTERN + -	englisch .SET FOEXTERN + -
Standardwert ist +. .setze foextern - hat zwei Wirkungen: - Felder werden so wie in der Maske ausgegeben - Schreib- und Lesbarkeit für FO ist wie für den Benutzer in der Maske .setze foextern + setzt wieder den Standardwert.	

[Weitere Details](#)

.SETZE FORMATIERT / .SET FORMATTED - Substitutionen für Variablen mit Formatierung

.SETZE FORMATIERT	.SET FORMATTED
deutsch .SETZE FORMATIERT "Ersetzung"	englisch .SET FORMATTED "replacement"
erlaubt, Substitutionen bei Variablenzugriffen auf formatierte Variablen für die Ausgabe global einzustellen	

[Weitere Details](#)

.SETZE FRAGE / .SET QUESTION - Zeichen für Bedingung setzen

.SETZE FRAGE	.SET QUESTION
deutsch .SETZE FRAGE 'Zeichen'	englisch .SET QUESTION 'character'
Setzt das einleitende Zeichen für Bedingungen. Standardwert ist das Fragezeichen.	

[Weitere Details](#)

.SETZE HINTERGRUND / .SET BACKGROUND - Bildschirminhalt nicht aktualisieren

.SETZE HINTERGRUND	.SET BACKGROUND
deutsch .SETZE HINTERGRUND + .SETZE HINTERGRUND -	englisch .SET BACKGROUND + .SET BACKGROUND -
.setze hintergrund + bewirkt, dass nach einem .system-Kommando der Bildschirm nicht geleert und wieder neu aufgebaut wird. Die Ausgaben des Systemkommandos bleiben ungelöscht. .setze hintergrund - bewirkt, dass nach einem .system-Kommando der Bildschirm in jedem Fall gelöscht und wieder neu aufgebaut wird.	

[Weitere Details](#)

.SETZE INLINE / .SET INLINE - Fließtext beibehalten

.SETZE INLINE	.SET INLINE
deutsch .SETZE INLINE + .SETZE INLINE -	englisch .SET INLINE + .SET INLINE -
.setze inline + verhindert, dass durch Kommandozeilen ein Absatzende (Zeilenumbruch) erzeugt wird. .setze inline - hebt die Wirkung wieder auf.	

[Weitere Details](#)

.SETZE KOMMANDO / .SET COMMAND - Kommandozeichen setzen

.SETZE KOMMANDO	.SET COMMAND
-----------------	--------------

deutsch .SETZE KOMMANDO 'Zeichen'	englisch .SET COMMAND 'character'
Setzt das Zeichen, mit dem in Spalte 1 Kommandos eingeleitet werden. Der Standardwert ist der Punkt.	

[Weitere Details](#)

.SETZE KOPIEREN / .SET COPY - Felder über Aufrufparameter initialisieren

.SETZE KOPIEREN	.SET COPY
deutsch .SETZE KOPIEREN Aufrufparameter ZielPuffer QuellePuffer	englisch .SET COPY callparameter TargeBuffer SourceBuffer
Initialisiert Felder über Aufrufparameter.	

[Weitere Details](#)

.SETZE LEER / .SET EMPTY - Geschütztes Leerzeichen setzen

.SETZE LEER	.SET EMPTY
deutsch .SETZE LEER 'Zeichen'	englisch .SET EMPTY 'character'
Dient der Kennzeichnung von Leerzeichen als geschützte Leerzeichen. Standardwert ist der Unterstrich.	

[Weitere Details](#)

.SETZE LF / .SET LF - Zeilenvorschub setzen

.SETZE LF	.SET LF
deutsch .SETZE LF .SETZE LF + .SETZE LF -	englisch .SET LF .SET LF + .SET LF -
gilt nur für die Ausgabe nach Microsoft Windows .setze lf + sendet [Einfügen Absatz] nach jeder Zeile und erzeugt damit einen Zeilenvorschub (line feed). Diese Einstellung ist der Standard. .setze lf - verhindert einen Zeilenwechsel. Dies ist wichtig für Formulare unter Microsoft Word.	

[Weitere Details](#)

.SETZE MARKE / .SET LABEL - Zeichen für Marke setzen

.SETZE MARKE	.SET LABEL
deutsch .SETZE MARKE = 'Zeichen'	englisch .SET LABEL = 'character'
Setzt das Zeichen für Marken. Standardwert ist das Ausrufezeichen.	

[Weitere Details](#)

.SETZE MENUTRANS / .SET MENUTRANS - Markieren als Menüzeilen

.SETZE MENUTRANS	.SET MENUTRANS
deutsch .SETZE MENUTRANS + .SETZE MENUTRANS -	englisch .SET MENUTRANS + .SET MENUTRANS -
.setze menutrans + markiert für das Übersetzungsprogramm trans die folgenden Zeilen als Menüzeilen. .setze menutrans - beendet das Markieren der Zeilen.	

[Weitere Details](#)

.SETZE NACHFRAGE / .SET DEMAND - Variable im Dialog einlesen

.SETZE NACHFRAGE	.SET DEMAND
deutsch .SETZE NACHFRAGE + .SETZE NACHFRAGE -	englisch .SET DEMAND + .SET DEMAND -
.setze nachfrage + liest eine im FOP verwendete Variable, die noch nicht belegt ist, im Dialog ein. .setze nachfrage - beendet das Einlesen. Wenn das FOP es auf eine noch nicht belegte Variable trifft, bricht es mit einer Fehlermeldung ab.	

[Weitere Details](#)

.SETZE ROHLING / .SET TEMPLATE - Rohling laden

.SETZE ROHLING	.SET TEMPLATE
-----------------------	----------------------

deutsch	englisch
<code>.SETZE ROHLING SML</code>	<code>.SET TEMPLATE CHB</code>
<code>.SETZE ROHLING SEL</code>	<code>.SET TEMPLATE SEL ['identity</code>
<code>['Identnummer Suchwort' [xdatname</code>	<code>number search word' [xdatname xidentnr</code>
<code>xidentnr [xgruppe]]</code>	<code>[xgruppe]]</code>
<code>.SETZE ROHLING SEL "?" xdat xident</code>	<code>.SET TEMPLATE SEL "?" xdat xident</code>

`.setze rohling sml` lädt den Sachmerkmalleistenrohling des zuvor geholten Objekts (nur für Artikel).
`.setze rohling sel` lädt den ersten Selektionsleistenrohling des zuvor geholten Objekts.
`.setze rohling sel 'Identnummer|Suchwort'` lädt den Selektionsleistenrohling mit der angegebenen Identnummer oder dem angegebenen Suchwort zum zuvor geholten Objekt.
`.setze rohling sel 'Identnummer|Suchwort' xdatname xidentnr [xgruppe]` lädt den Selektionsleistenrohling mit der angegebenen Identnummer oder Suchwort zum zuvor geholten Objekt und liefert zusätzlich die angegebenen Parameter:

<code>xdatname</code>	Name der Datenbank, auf die sich der Rohling bezieht, z.B. "Kunde"
<code>xidentnr</code>	Identnummer des Rohlings
<code>xgruppe</code>	Gruppennummer der Datenbank, auf die sich der Rohling bezieht

`.setze rohling sel "?" xdat xident` liefert Folgendes:

<code>xdat</code>	Namen der Datenbank in der FO-Sprache (nicht in der Bediensprache)
<code>xident</code>	Identnummer der Datenbank

[Weitere Details](#)

.SETZE SELTRENNER / .SET SELSEPARATOR - Trennzeichen setzen

<code>.SETZE SELTRENNER</code>	<code>.SET SELSEPARATOR</code>
deutsch	englisch
<code>.SETZE SELTRENNER</code>	<code>.SET SELSEPARATOR</code>
<code>.SETZE SELTRENNER trenner</code>	<code>.SET SELSEPARATOR separator</code>

`.setze seltrenner trenner` definiert das benutzerdefinierte Trennzeichen. Der Standardzustand kann mit `.setze seltrenner` wiederhergestellt werden.

[Weitere Details](#)

.SETZE SICHERN / .SET SAVE - Sichern-Sperre

.SETZE SICHERN	.SET SAVE
deutsch .SETZE SICHERN + .SETZE SICHERN - .SETZE SICHERN +- .setze sichern + holt die Sichern-Sperre . .setze sichern - gibt sie wieder frei. .setze sichern +- stellt fest, ob die jeweilige Sichern-Sperre vom Benutzer selbst gehalten wird, falls ja, wird G mehr gesetzt.	englisch .SET SAVE + .SET SAVE - .SET SAVE +- .set save + holds the Sichern-Sperre . .set save - gives it back. .set save +- sets whether the respective Sichern-Sperre is held by the user himself, if yes, G mehr is set.

[Weitere Details](#)

.SETZE SML / .SET CHB - Merkmale der Sachmerkmaliste

.SETZE SML	.SET CHB
deutsch .SETZE SML ['Identnummer Suchwort'] .SETZE SML - .setze sml packt die Merkmale der Sachmerkmaliste aus. .setze sml - löscht die Sachmerkmaliste. Dabei wird auch der Verweis (Feld sach) entfernt.	englisch .SET CHB ['identity number search word'] .SET CHB - .set chb packs the features of the Sachmerkmaliste. The reference (field sach) is removed.

[Weitere Details](#)

.SETZE SPRACHEINSTELLUNGEN / .SET LANGUAGESETTINGS - Sprachspezifische Ausgaben

.SETZE SPRACHEINSTELLUNGEN	.SET LANGUAGESETTINGS
deutsch .SETZE SPRACHEINSTELLUNGEN "T" "P" "0" .setze spracheinstellungen "T" setzt den Standardwert. Die Einstellungen wirken wie .setze tsdtrenn . Bei .setze spracheinstellungen "P" wirken auf den Variablenzugriff mit Apostroph in der Ausgabe die Felder der Ausgabesprache. .setze spracheinstellungen "0" gibt sprachabhängige Feldwerte in einer sprachunabhängigen Form aus.	englisch .SET LANGUAGESETTINGS "T" "P" "0" .set languagesettings "T" sets the standard value. The settings work like .set tsdtrenn . For .set languagesettings "P" the fields of the output language affect the variable access with apostrophe in the output. .set languagesettings "0" gives language-dependent field values in a language-independent form.

.SETZE STOP / .SET STOP - Bildschirm seitenweise füllen

.SETZE STOP	.SET STOP
deutsch .SETZE STOP + .SETZE STOP -	englisch .SET STOP + .SET STOP -
gilt nur auf der ASCII-Oberfläche .setze stop + wartet nach jeder Bildschirmseite auf Tastendruck, um die nächste Seite anzuzeigen. Wenn jedoch <Fertig> gedrückt wird, nicht mehr warten. .setze stop - scrollt den ausgegebenen Text weiter, ohne zu warten.	

.SETZE STRACE / .SET STRACE - Testprotokoll für Selektionen

.SETZE STRACE	.SET STRACE
deutsch .SETZE STRACE .SETZE STRACE + .SETZE STRACE - .SETZE STRACE ++	englisch .SET STRACE .SET STRACE + .SET STRACE ++ .SET STRACE -
.setze strace + schaltet die Protokollierung für Selektionen ein. .setze strace ++ schaltet die Protokollierung für Selektionen ein und gibt zusätzlich Warnungen aus. .setze strace - schaltet die Protokollierung für Selektionen aus.	

.SETZE SUBSTITUTION / .SET SUBSTITUTION - Substitutionen für Variablen

.SETZE SUBSTITUTION	.SET SUBSTITUTION
deutsch .SETZE SUBSTITUTION "Ersetzung"	englisch .SET SUBSTITUTION "replacement"
erlaubt, Substitutionen bei Variablenzugriffen auf Ausgabezeilen global einzustellen.	

.SETZE TRACE / .SET TRACE - Protokollierung von FO-Programmen

.SETZE TRACE	.SET TRACE
deutsch .SETZE TRACE .SETZE TRACE + - .SETZE TRACE +F	englisch .SET TRACE .SET TRACE + - .SET TRACE +F
.setze trace Protokollierung für die Entwicklung und Fehlersuche in FO-Programmen .setze trace + schaltet die Zeilenprotokollierung ab der nächsten Anweisung ein bzw. aus. Es kann nur eine Anweisungsfolge überwacht werden. .setze trace +F gibt den Trace der ausgeführten Befehle auf Datei aus. Der Dateiname wird gemeldet.	

[Weitere Details](#)

.SETZE TRANS / .SET TRANS - Von Übersetzung ausklammern

.SETZE TRANS	.SET TRANS
deutsch .SETZE TRANS + .SETZE TRANS -	englisch .SET TRANS + .SET TRANS -
.setze trans + beendet die Wirkung von .setze trans - und stellt die Standardeinstellung wieder her. .setze trans - lässt das Übersetzungsprogramm trans (ASCII) bzw. das Infosystem TRANS (GUI) alle folgenden Zeilen ignorieren. Sie werden also weder übersetzt noch in das Wörterbuch msg.lg.dic aufgenommen.	

[Weitere Details](#)

.SETZE TRENNER / .SET SEPARATOR - Zeilenendezeichen für Variableninhalte setzen

.SETZE TRENNER	.SET SEPARATOR
deutsch .SETZE TRENNER 'Zeichen'	englisch .SET SEPARATOR 'character'
Setzt das Zeilenendezeichen für Variableninhalte. Standardwert ist das Semikolon.	

[Weitere Details](#)

.SETZE TSDTRENN / .SET THOUSSEP - Tausendertrennzeichen setzen

.SETZE TSDTRENN	.SET THOUSSEP
deutsch .SETZE TSDTRENN .SETZE TSDTRENN 'Zeichen' .SETZE TSDTRENN 'G tsdtrenn englvar decPt '	englisch .SET THOUSSEP .SET THOUSSEP 'character' .SET THOUSSEP 'G tsdtrenn englvar decPt '
.setze tsdtrenn setzt das Tausendertrennzeichen bei der Ausgabe von Real-Zahlen auf den Standard, je nach der Art des Feldes. .setze tsdtrenn 'Zeichen' setzt das Tausendertrennzeichen auf das angegebene Zeichen. .setze tsdtrenn 'G tsdtrenn' setzt das Tausendertrennzeichen auf das Zeichen, das im Konfigurationsdatensatz festgelegt ist.	

[Weitere Details](#)

.SETZE UNTERBRECHUNG / .SET INTERRUPT - Unterbrechung

.SETZE UNTERBRECHUNG	.SET INTERRUPT
deutsch .SETZE UNTERBRECHUNG + .SETZE UNTERBRECHUNG - .SETZE UNTERBRECHUNG +- .SETZE UNTERBRECHUNG ++	englisch .SET INTERRUPT + .SET INTERRUPT - .SET INTERRUPT +- .SET INTERRUPT ++
.setze unterbrechung + setzt eine Unterbrechung .setze unterbrechung - löscht eine Unterbrechung .setze unterbrechung +- stellt fest, ob die jeweilige Unterbrechung vom Benutzer selbst gehalten wird, falls ja, wird G mehr gesetzt. .setze unterbrechung ++ setzt eine Unterbrechung, die das aktuelle Textkommando überdauert	

[Weitere Details](#)

.SETZE VARQUOTE / .SET VARQUOTE - HTML-Code in Variableninhalten

.SETZE VARQUOTE	.SET VARQUOTE
------------------------	----------------------

deutsch .SETZE VARQUOTE + .SETZE VARQUOTE -	englisch .SET VARQUOTE + .SET VARQUOTE -
.setze varquote + bewirkt, dass in Zeilen, die mit Doppelpunkt beginnen, HTML-relevante Zeichen in Variableninhalten gequotet, also nicht interpretiert werden. .setze varquote - bewirkt, dass in Doppelpunktzeilen HTML-Code in Variableninhalten interpretiert wird. (Standard)	

Weitere Details

.SETZE VERWEIS / .SET REFERENCE - Datenübernahmemodus setzen

.SETZE VERWEIS	.SET REFERENCE
deutsch .SETZE VERWEIS + .SETZE VERWEIS - .SETZE VERWEIS 'Format'	englisch .SET REFERENCE + .SET REFERENCE - .SET REFERENCE 'format'
.setze verweis + schaltet den Übernahmemodus ein, d.h. Felder verschiedener Arten werden in speziellen Darstellungen ausgegeben. .setze verweis - schaltet den Übernahmemodus wieder aus; alle Felder werden wie gewohnt ausgegeben. .setze verweis 'Format' bestimmt die Darstellung von Verweisen:	
Format	Darstellung
EXT	Verweise werden dargestellt, wie durch die Art des Feldes definiert. Das ist die Standardeinstellung.
REF	Verweise werden in der ID-Schreibweise, also wie bei der Art ID bzw. VID dargestellt.
NUM	Verweise werden mit Identnummernecho dargestellt, also wie bei der Art P oder VP.
SW	Verweise werden mit Suchwortechno dargestellt, also wie bei der Art PS oder VPS.

Weitere Details

.SETZE VTRACE / .SET VTRACE - Protokollierung von Variablenänderungen

.SETZE VTRACE	.SET VTRACE
---------------	-------------

deutsch	englisch
<code>.SETZE VTRACE ++</code>	<code>.SET VTRACE ++</code>
<code>.SETZE VTRACE + -</code>	<code>.SET VTRACE + -</code>
<code>.SETZE VTRACE + - 'Variablenname 1'</code>	<code>.SET VTRACE + - 'variable name 1'</code>
<code>['Variablenname 2' ...]</code>	<code>['variable name 2' ...]</code>
<code>.SETZE VTRACE + - 'Puffername' </code>	<code>.SET VTRACE + - 'buffer name' </code>

`.setze vtrace ++` schaltet die Variablenüberwachung für *alle* Variablen ein.

`.setze vtrace +|-` schaltet die Variablenüberwachung ein bzw. aus. Vor oder nach diesem Aufruf können Sie Variablen oder Puffer auswählen, die protokolliert werden sollen. Auch nach einem Ausschalten bleiben die zuvor ausgewählten Puffer und Variablen markiert und werden nach erneutem Einschalten wieder protokolliert. Die Protokollierung ist beim Start der FO ausgeschaltet.

`.setze vtrace +|- 'Variablenname 1' ['Variablenname 2' ...]` schaltet die Variablenüberwachung für bestimmte Variablen ein bzw. aus.

`.setze vtrace +|- 'Puffername' |` schaltet die Überwachung für alle Variablen im angegebenen Puffer ein bzw. aus.

[Weitere Details](#)

.SETZE WAEHRUNG / .SET CURRENCY - Währung vorbelegen

<code>.SETZE WAEHRUNG</code>	<code>.SET CURRENCY</code>
deutsch	englisch
<code>.SETZE WAEHRUNG</code>	<code>.SET CURRENCY</code>
<code>.SETZE WAEHRUNG 'Währung'</code>	<code>.SET CURRENCY 'currency'</code>

`.setze waehrung 'Währung'` belegt die globale FO-Variable **G|waehrung** mit der angegebenen Währung.

[Weitere Details](#)

.SORTIERE / .SORT - Textdateien sortieren

<code>.SORTIERE</code>	<code>.SORT</code>
deutsch	englisch
<code>.SORTIERE 'Dateiname' ['Spalte']</code>	<code>.SORT 'filename' ['column']</code>
<code>['Trennzeichen'] [ABWAERTS]</code>	<code>['separator'] [DOWN] [NUMERICAL]</code>
<code>[NUMERISCH]</code>	

Die **Datei** wird zeilenweise nach einer Spalte sortiert. Jede Zeile kann mehrere Texte spaltenweise nebeneinander enthalten.

Fehlt die Angabe der **Spalte**, wird nach der ersten Spalte sortiert.

Als **Trennzeichen** können keine Buchstaben oder Ziffern verwendet werden. Ist kein Trennzeichen angegeben, wird das Leerzeichen als Spaltentrenner gewertet.

Fehlt die Angabe **abwärts**, wird aufwärts sortiert.

Fehlt die Angabe **numerisch**, wird alphabetisch sortiert. (Z.B. steht 13 alphabetisch vor 5, numerisch nach 5.)

[Weitere Details](#)

.SORTIERTABELLE / .SORTTABLE - Tabelle sortieren

.SORTIERTABELLE	.SORTTABLE
deutsch .SORTIERTABELLE -MASKE 'Ordnungsangabe '	englisch .SORTTABLE -MASK 'Order specification'
Sortiert die Tabelle im Maskenpuffer nach der angegebenen Ordnungsangabe. Es sind sowohl Datenbankpuffer, als auch Infosystempuffer möglich. Das Kommando kann nicht bei allen Puffern verwendet werden. Die Ordnungsangabe entspricht der Syntax der Selektionsoption @ordnung .	

[Weitere Details](#)

.SYSTEM / .SYSTEM - Aufruf von Betriebssystemkommandos

.SYSTEM	.SYSTEM
deutsch .SYSTEM 'Betriebssystemkommando' [PASSWORD] .SYSTEM 'Betriebssystemkommando' [PASSWORD]SYSTEM 'Betriebssystemkommando' [PASSWORD] EDIT .SYSTEM 'Betriebssystemkommando' [PASSWORD] HINTER	englisch .SYSTEM 'operating system command' [PASSWORD] .SYSTEM 'operating system command' [PASSWORD]SYSTEM 'operating system command' [PASSWORD] EDIT .SYSTEM 'operating system command' [PASSWORD] BACKGROUND
password gibt unsichtbar ein Einmalpassword weiter. Ohne weitere Option erfolgt eine Ausgabe in das FO-Fenster. Bei ... muss die Ausgabe im FO-Fenster quittiert werden. edit bewirkt eine Ausgabe in das FO-Fenster. hinter bewirkt keine Ausgabe.	

.TABELLENSATZ / .TABLERECORD - HTML-Tabellen erzeugen

.TABELLENSATZ	.TABLERECORD
deutsch	englisch
.TABELLENSATZ	.TABLERECORD
.TABELLENSATZ TITEL	.TABLERECORD TITLE
.TABELLENSATZ UEBERSCHRIFT	.TABLERECORD HEADLINE
.TABELLENSATZ ZEILE	.TABLERECORD LINE
.TABELLENSATZ ENDE	.TABLERECORD END
.TABELLENSATZ UNFORMATIERT	.TABLERECORD UNFORMATTED
.TABELLENSATZ SPALTE	.TABLERECORD COLUMN

.tabellensatz beendet einen Flatter- oder Blocksatz.
 Mit .tabellensatz titel, .tabellensatz ueberschrift, .tabellensatz zeile, .tabellensatz ende, .tabellensatz unformatiert und .tabellensatz spalte wird die HTML-Ausgabe einer Tabelle gesteuert.

.TEXT / .TEXT - Text einer FOP-Ausgabe übersetzen

.TEXT	.TEXT
deutsch	englisch
.TEXT 'Variable' 'Sprache' 'Text'	.TEXT 'variable' 'language' 'text'

'Variable' gibt den Namen einer FO-Variablen an, die den auszugebenden Text aufnehmen soll.
 Für 'Sprache' kann 'G|kspr' (Korrespondenzsprache) oder ein Sprachkürzel eingesetzt werden, entweder 1-buchstabig (D, E, ...) oder als ISO-Kürzel (de, en, ...).
 'Text' ist die Schlüsselinformation für den auszugebende Text in Deutsch.

.TRENNER / .SEPARATOR - Zeilenendezeichen für Variableninhalte setzen

.TRENNER	.SEPARATOR
deutsch	englisch
.TRENNER 'Zeichen'	.SEPARATOR 'character'

Setzt das Zeilenendezeichen für Variableninhalte. Standardwert ist das Semikolon.

.UEBERSETZEN / .TRANSLATE - Text in gewünschter Sprache holen

.UEBERSETZEN	.TRANSLATE	deutsch .UEBERSETZEN -SPRACHE 'Kürzel' -NUMMER 'Nummer' -WOERTERBUCH 'Kürzel Nummer' -VARIABLE "Name" [-SCHLIESSEN] .UEBERSETZEN -SPRACHE 'Kürzel' -TEXT 'Text' [- WOERTERBUCH 'Kürzel Nummer'] -VARIABLE "Name" [- NRVARIABLE "Name"] [- WBUCHVARIABLE "Name"] [- GROSSKLEIN] [- QUELLSPRACHE 'Kürzel'] [- SCHLIESSEN] .UEBERSETZEN -SCHLIESSEN
--------------	------------	--

englisch

```
.TRANSLATE -LANGUAGE
'code' -NUMBER 'number'
-DICTIONARY 'code|number'
-VARIABLE "name" [-CLOSE]
.TRANSLATE -LANGUAGE
'code' -TEXT 'text' [-
DICTIONARY 'code|number']
-VARIABLE "name" [-
NUMVARIABLE "name"] [-
DICTVARIABLE "name"] [-
CAPLOWER] [-
SOURCELANGUAGE 'code'] [-
CLOSE] .TRANSLATE -CLOSE
```

Bei **-sprache** bezeichnet '**Kürzel**' das Sprachkürzel. Es wird 1-buchstabig (**D**, **E**, ...) oder als ISO-Kürzel (**de**, **en**, ...) angegeben. Sprachkürzel ":" liefert die sprachunabhängige Notation.

Bei **-nummer** gibt '**Nummer**' die Nummer des Textes im Wörterbuch an, der gelesen werden soll.

Bei **-woerterbuch** bezeichnet '**Kürzel|Nummer**' das Wörterbuch:

Wörterbuch	Kürzel	Nummer
Programmmeldungen	cc	0
Hilfemenüs	sh	1
FOPs	lg	2
.text-Kommandos in FOPs	tx	3
Masken	ma	4
Kommando-Menüs	me	5
Individuelle Schlüsselnamen	ku	6
Bedeutungen in Variablentabellen	vt	7
Property-Dateien	pr	8

Bei **-variable** bezeichnet '**Name**' eine FO-Variable der Art "text", in der die Übersetzung des gesuchten Textes abgelegt wird. Wird der Suchtext nicht gefunden, wird dieser in der Variablen abgelegt.

-schliessen schließt das geöffnete Wörterbuch. Bei **.text** bezeichnet '**Text**' den deutschen Text, der gesucht und übersetzt werden soll. Bei **-nrvariable** bezeichnet '**Name**' eine FO-Variable der Art "int", in der die Nummer des gefundenen Textes abgelegt wird. Bei **-wbuchvariable** bezeichnet '**Name**' eine FO-Variable der Art "text", in der das Kürzel des Wörterbuchs, in dem der Text gefunden wurde, abgelegt wird. **-grossklein** bewirkt, dass die Groß-/Kleinschreibung von '**Text**' und die Schreibweise der deutschen Umlaute keine Rolle spielt (Normtext). Bei **-quellsprache** bezeichnet '**Kürzel**' das Sprachkürzel der Quellsprache. Es wird 1-buchstabig (**D**, **E**, ...) oder als ISO-Kürzel (**de**, **en**, ...) angegeben. Fehlt die Option wird "de" angenommen.

Weitere Details

.UNTEN / .DOWN - Fußzeilen

.UNTEN	.DOWN
--------	-------

deutsch <code>.UNTEN</code>	englisch <code>.DOWN</code>
definiert den Unterpuffer. Der folgende Text bis zum nächsten Kommando erscheint als Fuß auf jeder folgenden Seite.	

[Weitere Details](#)

.VTAB / .VTAB - Vertikaltabulatoren

<code>.VTAB</code>	<code>.VTAB</code>
deutsch <code>.VTAB 'Zeilennummer 1' 'Zeilennummer 2' ...</code>	englisch <code>.VTAB 'line number 1' 'line number 2' ...</code>
Definiert Vertikaltabulatoren (Zeilen) für ein Ausgabegerät oder das Kommando <code>.zeile</code> . Die Zeilennummern werden Inhalte der Variablen <code>vt1</code> , <code>vt2</code> usw. Standardeinstellungen sind <code>vt1=1</code> , <code>vt2=70</code> , <code>vt3=10</code> und <code>vt4=20</code> .	

[Weitere Details](#)

.WEITER / .CONTINUE - Rücksprung zum Aufrufer

<code>.WEITER</code>	<code>.CONTINUE</code>
deutsch <code>.WEITER</code> <code>.WEITER "Markenname"</code>	englisch <code>.CONTINUE</code> <code>.CONTINUE "label name"</code>
<code>.weiter</code> bewirkt einen Rücksprung zum aufrufenden FOP bzw. auf oberster Ebene das Ende des FO-Programms. <code>.weiter "Markenname"</code> bewirkt einen Sprung zur Marke .	

[Weitere Details](#)

.ZEIGEN / .VIEW - Fensterbreite festlegen

<code>.ZEIGEN</code>	<code>.VIEW</code>
deutsch <code>.ZEIGEN SG.NORM SG.KLEIN</code>	englisch <code>.VIEW SG.NORM SG.KLEIN</code>
legt die Zeichenanzahl pro Zeile und damit die Breite des FO-Ausgabefensters fest <code>SG.NORM</code> normale Zeichenbreite, 80 Zeichen <code>SG.KLEIN</code> kleine Zeichenbreite, 132 Zeichen	

[Weitere Details](#)

.ZEILE / .LINE - Nächste Ausgabezeile

.ZEILE	.LINE
deutsch .ZEILE 'vt1' .ZEILE 'Zeilennummer'	englisch .LINE 'vt1' .LINE 'line number'
.zeile 'vt1' springt zu der durch .VTAB vereinbarten Zeilenposition vt1. .zeile 'Zeilennummer' schreibt den nachfolgenden Text in die jeweilige Zeile.	

[Weitere Details](#)

.ZUWEISEN / .ASSIGN - Zuweisen mit Wertebereichsprüfung

.ZUWEISEN	.ASSIGN
deutsch .ZUWEISEN 'X Variablenname' = 'Ausdruck'	englisch .ASSIGN 'X variable name' = 'expression'
Der 'Variablen' wird das Rechenergebnis des 'Ausdrucks' zugewiesen, wenn es im Wertebereich der Variablen liegt. X steht für ein Bezugsobjekt.	

[Weitere Details](#)